BİLGİSAYAR MÜHENDİSLİĞİ · 1. SINIF

```
// Program.cs
Console.WriteLine("Merhaba, C#!");

string ad = Console.ReadLine() ?? "";
Console.WriteLine($"Hoş geldin, {ad}");
```

C# ile Programlamaya Giriş

Konu anlatımı, sınav tuzakları ve çözümlü soru bankası

15	188	2	2
bölüm	soru	mini proje	deneme sınavı

HAZIRLAYAN

Ahmet Şerif Kart · BARÜ – Bartın Üniversitesi

C# 14 · .NET 10 · Visual Studio · 2026

İçindekiler

Bu Kitap Nasıl Kullanılır? . . . 7

1 Programlamaya ve C#'a Giriş . . . 9

1.1 Program ve programlama dili . . . 9
1.2 C# nedir? . . . 9
1.3 C# programı nasıl çalışır? . . . 10
1.4 Kurulum . . . 11
1.5 İlk program: Merhaba, Dünya! . . . 13
1.6 Üst düzey deyimler ve klasik biçim . . . 14
1.7 Write, WriteLine ve kaçış dizileri . . . 15
1.8 Yorum satırları . . . 17
1.9 Yazım kuralları . . . 17
1.10 Hata türleri . . . 18

2 Değişkenler, Veri Tipleri ve Operatörler . . . 26

2.1 Değişken nedir? . . . 26
2.2 İsimlendirme kuralları . . . 27
2.3 Temel veri tipleri . . . 28
2.4 Sabitler (const) ve var . . . 31
2.5 Aritmetik operatörler . . . 32
2.6 Tip dönüşümü (casting) . . . 34
2.7 Atama ve bileşik atama operatörleri . . . 36
2.8 Artırma ve azaltma: x++ ve ++x . . . 37
2.9 Math sınıfı . . . 38
2.10 Taşma ve ondalık hassasiyet . . . 39
2.11 Bit operatörleri . . . 41

3 Klavyeden Girdi ve Biçimli Çıktı . . . 48

3.1 Console.ReadLine ile satır okuma . . . 48
3.2 Sayı okuma . . . 50
3.3 Türkçe bilgisayarlarda ondalık virgül . . . 53
3.4 Biçimli çıktı . . . 55
3.5 Kaçış dizileri ve @"..." metinler . . . 59
3.6 Türkçe karakterler konsolda . . . 60
3.7 Örnek uygulama: Vücut kitle indeksi . . . 60

4 Koşullu İfadeler . . . 67

4.1 İlişkisel operatörler ve bool ifadeler . . . 67
4.2 if deyimi . . . 68
4.3 if-else . . . 70
4.4 else-if merdiveni . . . 71
4.5 İç içe if ve sarkan else . . . 72
4.6 Mantıksal operatörler . . . 73
4.7 Operatör önceliği . . . 75
4.8 Üçlü operatör (?:) . . . 76
4.9 switch deyimi . . . 76
4.10 switch ifadesi ve desenler . . . 78
4.11 string ve double karşılaştırma . . . 80
4.12 Örnek uygulama: VKİ yorumu . . . 81

5 Döngüler . . . 90

5.1 Neden döngü? . . . 90
5.2 while döngüsü . . . 90
5.3 do-while döngüsü . . . 91
5.4 for döngüsü . . . 92
5.5 Sonsuz döngü, break ve continue . . . 93
5.6 İç içe döngüler . . . 94
5.7 Yıldız desenleri . . . 95
5.8 Döngü izleme tablosu ve off-by-one . . . 96
5.9 Klasik döngü algoritmaları . . . 97
5.10 Kullanıcı girdisiyle döngüler . . . 100

6 Metotlar . . . 110

6.1 Neden metot? . . . 110
6.2 Metodun anatomisi . . . 111
6.3 Klasik biçimde metotlar . . . 112
6.4 void ve değer döndüren metotlar . . . 113
6.5 return ile erken çıkış . . . 115
6.6 İfade gövdeli metotlar . . . 116
6.7 Parametre ve argüman . . . 117
6.8 Metot çağrıldığında ne olur? . . . 117
6.9 Değer ile geçirme . . . 119
6.10 İsteğe bağlı ve adlandırılmış parametreler . . . 123
6.11 Aşırı yükleme (overloading) . . . 124
6.12 Kapsam (scope) ve yerel değişkenler . . . 127
6.13 params: değişken sayıda argüman . . . 128
6.14 Algoritmaları metotlara dönüştürmek . . . 129
6.15 Hazır metotlar: Math ve Random . . . 130

7 Diziler . . . 141
7.1 Neden dizi? . . . 141
7.2 Dizi tanımlama ve oluşturma . . . 141
7.3 Length ve indeksler . . . 143
7.4 Diziyi dolaşmak . . . 145
7.5 Temel dizi algoritmaları . . . 147
7.6 Girdi ile dizi doldurma . . . 148
7.7 Diziler başvuru tipidir . . . 149
7.8 Diziyi kopyalama . . . 150
7.9 Array sınıfı . . . 151
7.10 Diziler ve metotlar . . . 152
7.11 Ters çevirme ve sıralama . . . 154
7.12 Değişken sayıda parametre: params . . . 155
7.13 İki boyutlu diziler . . . 155
7.14 Düzensiz (jagged) diziler . . . 157

8 String ve Karakter İşlemleri . . . 166
8.1 string nedir? . . . 166
8.2 string değiştirilemez (immutable) . . . 168
8.3 Temel string üyeleri . . . 170
8.4 Büyük/küçük harf ve Türkçe I sorunu . . . 174
8.5 string karşılaştırma: ==, Equals ve StringComparison . . . 174
8.6 Sıralama karşılaştırması: CompareTo ve Compare . . . 175
8.7 Karakter karakter dolaşma ve char . . . 176
8.8 StringBuilder . . . 178
8.9 Dönüşümler: string, sayı ve char . . . 179
8.10 Klasik string algoritmaları . . . 180

P1 Öğrenci Not Sistemi . . . 191

S Deneme Vize Sınavı . . . 201

9 Sınıflar ve Nesneler . . . 218
9.1 Nesneye yönelik programlama fikri . . . 218
9.2 Sınıf ve nesne . . . 218
9.3 Alanlar ve metotlar . . . 219
9.4 Erişim belirleyiciler ve kapsülleme . . . 220
9.5 Özellikler (property) . . . 221
9.6 Kurucular (constructor) . . . 223
9.7 this anahtar sözcüğü . . . 225
9.8 ToString metodu . . . 226
9.9 Referans değişkenler . . . 227
9.10 null ve NullReferenceException . . . 229
9.11 static üyeler . . . 232
9.12 Nesne dizileri . . . 234
9.13 struct ile class farkı . . . 235

10 Kalıtım . . . 248

10.1 Kod tekrarı problemi . . . 248
10.2 Kalıtım sözdizimi . . . 248
10.3 Tek taban sınıf kuralı . . . 250
10.4 Neler kalıtılır? . . . 250
10.5 protected erişim . . . 251
10.6 Kurucular ve base(...) . . . 252
10.7 Kurucuların çalışma sırası . . . 253
10.8 Metot ezme: virtual ve override . . . 254
10.9 base.Metot() ile taban sınıfın metodunu çağırma . . . 255
10.10 Yukarı dönüşüm: taban tipli değişken . . . 256
10.11 Gizleme (hiding) ve new . . . 256
10.12 sealed: ezilemez, türetilemez . . . 258
10.13 Aşağı dönüşüm: is ve as . . . 259
10.14 object sınıfı: her sınıfın atası . . . 260
10.15 is-a ve has-a: kalıtım mı, kompozisyon mu? . . . 261
10.16 Sınıf hiyerarşisi örneği . . . 262

11 Polimorfizm, Soyut Sınıflar ve Arayüzler . . . 275

11.1 Polimorfizm nedir? . . . 275
11.2 İki tip: referans tipi ve nesne tipi . . . 276
11.3 Dinamik bağlama: hangi metot çalışır? . . . 276
11.4 Polimorfik diziler . . . 280
11.5 Soyut sınıflar (abstract class) . . . 281
11.6 Arayüzler (interface) . . . 283
11.7 Soyut sınıf mı, arayüz mü? . . . 286
11.8 IComparable<T>: kendi nesnelerini sıralamak . . . 287
11.9 object'ten gelen sanal metotlar . . . 288
11.10 Tam örnek: ödeme sistemi . . . 289

12 İstisna Yönetimi . . . 302

12.1 İstisna nedir? . . . 302
12.2 İstisna hiyerarşisi . . . 303
12.3 try ve catch . . . 304
12.4 Birden çok catch . . . 305
12.5 finally: her durumda çalışan blok . . . 306
12.6 throw: istisna fırlatmak . . . 307
12.7 throw; ile throw ex; farkı . . . 309
12.8 Kendi istisna sınıfını yazmak . . . 310
12.9 using: kaynakları otomatik bırakmak . . . 311
12.10 Kullanıcı girdisini doğrulamak . . . 312
12.11 TryParse mi, istisna mı? . . . 313

P2 Banka Hesabı Simülasyonu . . . 324

13 Koleksiyonlar, Generics ve LINQ . . . 333
13.1 Dizilerin sınırı: koleksiyonlar . . . 333
13.2 List: büyüyen dizi . . . 334
13.3 Dolaşırken listeden eleman silmek . . . 336
13.4 Sıralama: IComparable ve lambda . . . 337
13.5 Dictionary: anahtar–değer çiftleri . . . 338
13.6 HashSet, Queue ve Stack . . . 340
13.7 Generics . . . 342
13.8 LINQ'e giriş . . . 344
13.9 Temsilciler, Func, Action ve event'e giriş . . . 346

14 Dosya İşlemleri . . . 357
14.1 Neden dosya? . . . 357
14.2 Dosya yolları ve çalışma klasörü . . . 358
14.3 File sınıfıyla yazma ve okuma . . . 358
14.4 Olmayan dosya: FileNotFoundException . . . 360
14.5 StreamWriter ve StreamReader . . . 361
14.6 Kodlama: UTF-8 ve Türkçe karakterler . . . 362
14.7 CSV dosyası ve Ogrenci sınıfı . . . 363
14.8 Satır ve kelime sayma . . . 364
14.9 Klasörler . . . 365
14.10 async/await ile dosya işlemleri . . . 365
14.11 JSON ile kaydetme: System.Text.Json . . . 367

15 Özyineleme (Recursion) . . . 377
15.1 Özyineleme nedir? . . . 377
15.2 Temel durum ve özyinelemeli durum . . . 378
15.3 Çağrı yığını (call stack) . . . 379
15.4 Sayılarla özyineleme . . . 380
15.5 Fibonacci ve tekrar eden hesaplar . . . 381
15.6 String ve dizilerle özyineleme . . . 383
15.7 Hanoi kuleleri . . . 385
15.8 Özyinelemeli ikili arama . . . 386
15.9 Temel durum unutulursa: Stack overflow . . . 386
15.10 Özyineleme mi, döngü mü? . . . 387
15.11 Çağrıdan önce mi, sonra mı? . . . 387

S Deneme Final Sınavı . . . 396

A C# Cheat Sheet . . . 416

B Terimler Sözlüğü . . . 422

C Sık Karşılaşılan Hata Mesajları . . . 429

Bu Kitap Nasıl Kullanılır?

Bu kitap, bilgisayar mühendisliğinin ilk yılında C# dersini alan ve daha önce hiç kod yazmamış öğrenci için yazıldı. Bilgisayarın ne olduğunu, derleyicinin ne işe yaradığını kabaca biliyorsun; ama değişken, döngü, sınıf gibi kavramlarla henüz tanışmadın. Hedef iki tane: **C#'ı gerçekten öğrenmen** ve **vize/finalde soru kaçırmaman**.

Kitap **C# 14** ve **.NET 10 LTS** için yazıldı. Kodları Visual Studio 2026 Community'de (Windows) ya da VS Code'da C# Dev Kit eklentisiyle yazabilirsin. Komut satırını tercih edersen `dotnet new console` ile proje oluştur, `dotnet run` ile çalıştır.

Kitabın yapısı

Kitap, tipik bir iki dönemlik "Programlamaya Giriş" dersinin sırasını takip eder:

Kısım	Bölümler	Ne öğreneceksin
Temeller	1–8	Program yapısı, değişken, girdi-çıktı, koşul, döngü, metot, dizi, string
Proje 1 ve vize denemesi	—	İlk sekiz bölümü birleştiren Öğrenci Not Sistemi
Nesneye yönelik programlama	9–12	Sınıf, nesne, kalıtım, polimorfizm, arayüz, istisna
Proje 2	—	9–12 bölümü birleştiren Banka Hesabı uygulaması
Veri ve dosya	13–14	Koleksiyonlar, generics, LINQ, dosya okuma ve yazma
Algoritmalar	15	Özyineleme
Final denemesi	—	Tüm dönemi birleştiren sınav
Ekler	A–C	Cheat sheet, terimler sözlüğü, hata mesajları

Her bölüm aynı düzende ilerler: önce **"Bu bölümde öğreneceklerin"** listesi, sonra konu anlatımı ve örnekler, ardından **Bölüm Özeti**, en sonda **Sorular** ve yeni bir sayfada **Çözümler**. Vize ve final de aynı üç soru türünden oluşur; soru sayısı daha fazladır.

Kutular

Metnin içinde renkli kutular göreceksin. Hepsinin bir görevi var:

> **İpucu**
>
> Kodu kısaltan yollar, Visual Studio ve VS Code kısayolları, akılda tutma yöntemleri.

> **Sık Yapılan Hata**
>
> Öğrencilerin en sık düştüğü tuzaklar. Sınavda puan kaybettiren hataların çoğu bu kutulardan çıkar.

> **Sınavda Çıkar**
>
> Hocaların sormayı çok sevdiği konular. Vize/final öncesi sadece bu kutuları okuyarak bile hızlı tekrar yapabilirsin.

> **Yeni C# Notu**
>
> C# 12 ile 14 arasında gelen kolaylaştırıcı özellikler (ör. birincil kurucu, `[1, 2, 3]` koleksiyon ifadesi). Bilmek iyidir, ama sınavda klasik yazımı kullan.

Ayrıca bölümün başındaki **hedef** kutusu o bölümde öğreneceklerini, bölüm sonundaki **özet** kutusu en önemli noktaları, proje bölümlerindeki **geliştir** kutusu ise programı kendi isteğine göre genişletebileceğin yerleri gösterir.

Kod ve çıktılar

Kitaptaki **her program gerçekten derlendi ve çalıştırıldı**; çıktı kutularındaki yazılar elle yazılmadı, programın gerçek çıktısı olarak kitaba eklendi. Yani bir örneği kendi bilgisayarında çalıştırdığında aynı sonucu görmelisin.

```
Console.WriteLine("Kod kutusu böyle görünür.");
```

Çıktı

```
Kod kutusu böyle görünür.
```

Programın klavyeden bir şey okuduğu örneklerde, senin yazdığın kısım çıktı kutusunda **mor** renkte gösterilir; tıpkı terminalde gördüğün gibi:

```
Console.Write("Adın: ");
string ad = Console.ReadLine() ?? "";   // boş gelirse ad boş metin olur
Console.WriteLine($"Selam {ad}!");
```

Çıktı

```
Adın: Ayşe
Selam Ayşe!
```

> **Not: Ondalık sayılar ve virgül**
>
> Kitaptaki çıktılar **Türkçe ayarlı bir Windows'ta** çalıştırılmış gibi verildi; bu yüzden ondalık ayraç **virgüldür** (`3,5`). Kodda ise nokta yazarsın (`3.5`). Bu farkın nedenini Bölüm 2–3'te göreceksin.

Sorular nasıl çözülmeli?

Her bölümün sonunda üç tür soru var:

- **Çoktan seçmeli (5 soru):** Kavramları ölçer. Önce cevabı kendin bul, sonra şıklara bak.
- **Çıktı tahmini (3 soru):** "Bu kod ne yazdırır?" Sınavların vazgeçilmezi. Kodu **kâğıt üzerinde, satır satır** izle; değişkenlerin değerini bir tabloya yaz. Bilgisayarda çalıştırmadan önce cevabını mutlaka kâğıda yaz.
- **Kod yazma (2 soru):** Küçük programlar. Önce kendin yaz, Visual Studio'da ya da `dotnet run` ile çalıştır, sonra çözümle karşılaştır. Çözümdeki kod tek doğru cevap değildir; senin kodun da doğru çıktı veriyorsa doğrudur.

> **İpucu: Çalışma önerisi**
>
> Bölümü oku → örnekleri **elle yazarak** (kopyala-yapıştır yok!) çalıştır → soruları çöz → yanlışlarını çözümlerden kontrol et → bir hafta sonra sadece kutuları ve özeti tekrar oku. Programlama izleyerek değil, yazarak öğrenilir.

BÖLÜM 1

Programlamaya ve C#'a Giriş

- **Bu Bölümde Öğreneceklerin**
  - C# programının yazıldıktan sonra nasıl çalıştığını (`.cs` → C# derleyicisi → IL → CLR + JIT → makine kodu)
  - SDK, çalışma zamanı (runtime) ve CLR arasındaki farkı
  - Visual Studio 2026 Community veya VS Code ile .NET 10 kurulumunu
  - İlk programını yazıp hem Visual Studio'dan hem terminalden (`dotnet run`) çalıştırmayı
  - Üst düzey deyimler ile klasik `Main` biçimi arasındaki farkı
  - `Write` ile `WriteLine` farkını, kaçış dizilerini (`\n`, `\t`, `\"`) ve yorum satırlarını
  - Derleme hatası, çalışma zamanı hatası, mantık hatası ve uyarı arasındaki farkı

1.1 Program ve programlama dili

Program, bilgisayara ne yapacağını adım adım anlatan komutlar dizisidir. İşlemci (CPU) yalnızca kendi **makine dilini** (0 ve 1'lerden oluşan komutlar) anlar. İnsanların makine diliyle program yazması çok zor olduğu için **yüksek seviyeli programlama dilleri** (high-level language) geliştirilmiştir: C#, C, Python bunlardan birkaçıdır.

Yüksek seviyeli dilde yazdığımız metne **kaynak kod** (source code) denir. Kaynak kodun makinenin anlayacağı hale gelmesi için çevrilmesi gerekir. Bunun iki klasik yolu vardır:

- **Derleyici** (compiler): Bütün programı bir kerede çevirir, sonra çevrilmiş hali çalıştırılır. (C, C++)
- **Yorumlayıcı** (interpreter): Kodu satır satır okuyup o anda çalıştırır. (klasik Python)

C# ikisinin ortasında bir yol izler. Bunu iki başlık sonra göreceğiz.

1.2 C# nedir?

C# (okunuşu "si şarp"), Microsoft'un geliştirdiği genel amaçlı, nesne yönelimli bir programlama dilidir. Tasarımcısı, Microsoft'ta bir ekibe başkanlık eden **Anders Hejlsberg**'dir. Dil 2000 yılında duyuruldu, ilk sürümü 2002'de yayımlandı ve o günden beri her yıl yeni özellikler kazandı.

Bu kitap **C# 14** ve **.NET 10** ile çalışır. .NET 10, Kasım 2025'te çıkan bir **LTS** (Long-Term Support, uzun süreli destek) sürümüdür; yıllarca güncelleme alacağı için kitap boyunca bunu kullanıyoruz.

C# yalnızca Windows'a özgü bir dil değildir. Programları çalıştıran platform olan **.NET açık kaynaklıdır** (kaynak kodu herkese açıktır) ve Windows, macOS ve Linux'ta çalışır. C# ile şunları yapabilirsin:

Alan	Örnek
Masaüstü uygulamaları	Windows Forms, WPF, .NET MAUI
Web siteleri ve API'ler	ASP.NET Core
Oyunlar	**Unity** oyun motorunda kodlar C# ile yazılır
Bulut ve mobil	Azure servisleri, .NET MAUI ile telefon uygulamaları

C# dosyalarının uzantısı `.cs`'dir ("C Sharp"in kısaltması; `#` işareti dosya adlarında sorun çıkardığı için kullanılmaz).

> **Sınavda Çıkar**
>
> "C# ile .NET aynı şey midir?" sorusunun cevabı **hayır**dır. C# bir **dildir**; .NET ise bu dille yazılmış programları derleyen ve çalıştıran **platformdur** (araçlar + çalışma ortamı + hazır kütüphaneler). Aynı platformda F# ve Visual Basic gibi başka diller de çalışır.

1.3 C# programı nasıl çalışır?

C# programı iki aşamada çalışır:

1. **Derleme:** C# derleyicisi (adı *Roslyn*), `.cs` dosyasındaki kaynak kodu **IL** denen ara bir dile çevirir ve sonucu bir `.dll` dosyasına yazar. IL (Intermediate Language, ara dil) ne insan dilidir ne de işlemcinin dili; .NET'e özel bir ara koddur.
2. **Çalıştırma:** Program başlatılınca **CLR** (Common Language Runtime, ortak dil çalışma zamanı) devreye girer. CLR'ın içindeki **JIT** (Just-In-Time) derleyicisi, bir metot ilk kez çağrıldığında onun IL kodunu o bilgisayarın işlemcisinin anlayacağı **makine koduna** çevirir; işlemci de bu makine kodunu çalıştırır.

```
Merhaba.cs        kaynak kod (senin yazdığın)
    |  C# derleyicisi (derleme aşaması)
    v
Merhaba.dll       IL, yani ara dil
    |  CLR + JIT (program çalışırken)
    v
makine kodu       işlemcinin gerçekten çalıştırdığı komutlar
```

Adım	Kim yapar?	Girdi	Çıktı
1. Derleme	C# derleyicisi	`Merhaba.cs` (kaynak kod)	`Merhaba.dll` (IL)
2. Çalıştırma	CLR + JIT	`Merhaba.dll`	Programın çıktısı

Tam derlenen dillerle karşılaştırma

C ve C++ gibi dillerde derleyici, kaynak kodu **doğrudan makine koduna** çevirir. C# ise önce IL'e derler, makine koduna çevirmeyi çalışma anına bırakır:

	C, C++ (tam derlenen)	C#
Derleyicinin çıktısı	Makine kodu	IL (ara dil)
Makine koduna çevirme	Derleme sırasında, bir kez	Çalışırken, JIT ile
Çıktı nerede çalışır?	Derlendiği işlemci ve işletim sisteminde	.NET çalışma zamanı olan her bilgisayarda
Açılış hızı	Çok hızlı	Biraz yavaş (JIT çalışmak zorunda)

Bu yapının büyük avantajı şudur: Aynı `.dll` dosyası Windows'ta, macOS'ta, Linux'ta değiştirilmeden çalışır (Windows'a özgü özellikler kullanmadığın sürece). Yeter ki o bilgisayarda .NET çalışma zamanı olsun. Buna **platform bağımsızlığı** denir. C veya C++ ile yazılmış bir programı ise her platform için ayrı ayrı derlemen gerekir.

Windows'ta derleme sonunda `Merhaba.exe` dosyasını da görürsün. Bu dosya yalnızca küçük bir **başlatıcıdır**; asıl program `Merhaba.dll` içindeki IL'dir.

SDK, Runtime ve CLR

Bu üç kavram birbirine çok karıştırılır:

Kavram	Açılımı	Ne işe yarar?	İçinde ne var?
CLR	Common Language Runtime	IL'i çalıştırır (JIT, bellek yönetimi)	Sadece çalıştırma motoru
.NET Runtime	.NET çalışma zamanı	Hazır .NET programlarını çalıştırmaya yeter	CLR + hazır sınıf kütüphanesi (`Console` gibi)
.NET SDK	Software Development Kit	Program **geliştirmek** için gerekir	Runtime + C# derleyicisi + `dotnet` komutu

Kısaca: **SDK** ⊃ **Runtime** ⊃ **CLR**. Program yazacağın için sana **SDK** lazım.

> **Sınavda Çıkar**
>
> "SDK, Runtime ve CLR arasındaki fark nedir?" ve "C# neden platform bağımsızdır?" klasik sorulardır. Cevabın anahtar kelimeleri: **IL** ve **CLR**. C# kaynak kodu doğrudan makine koduna değil IL'e derlenir; IL'i her platformdaki CLR çalıştırır. IL'i makine koduna çeviren parça **JIT**'tir.

> **Sık Yapılan Hata**
>
> "C# yorumlanan bir dildir" ya da "C# derlenen bir dildir" demek tek başına eksik kalır. Doğrusu: C# önce **derlenir** (IL'e), sonra IL, **JIT** tarafından çalışırken makine koduna çevrilir. İki aşama da vardır.

1.4 Kurulum

Programı yazmak için iki şeye ihtiyacın var: **.NET 10 SDK** ve bir editör. Aşağıdaki iki yoldan birini seç. Windows kullanıyorsan Visual Studio en rahatıdır; macOS ya da Linux'ta VS Code kullanırsın. Kitaptaki bütün örnekler iki ortamda da aynen çalışır.

Yol 1: Visual Studio 2026 Community (Windows)

Visual Studio, Microsoft'un hepsi bir arada geliştirme ortamıdır (IDE, Integrated Development Environment): editör, derleyici ve hata ayıklayıcı (debugger) tek pencerede. **Community** sürümü öğrenciler ve bireysel kullanıcılar için ücretsizdir.

1. visualstudio.microsoft.com adresinden **Visual Studio 2026 Community** yükleyicisini indir.
2. Yükleyicide **".NET masaüstü geliştirme"** (*.NET desktop development*) iş yükünü işaretle ve kur. .NET 10 SDK da bununla birlikte gelir.
3. Visual Studio'yu aç → **Create a new project** → şablon listesinden **Console App** (C#) seç → **Next**.
4. Proje adına `Merhaba` yaz → **Next**.
5. **Framework** olarak **.NET 10.0**'ı seç. Altındaki **"Do not use top-level statements"** kutusunu **işaretleme**. → **Create**.
6. Açılan `Program.cs` dosyasına ilk programı yaz ve **Ctrl+F5** tuşuna bas. Çıktı bir konsol penceresinde görünür; pencere "Press any key to close this window" yazısıyla açık kalır.

Menü adları Türkçe kurulumda Türkçe görünür; adımların sırası aynıdır.

> **Sık Yapılan Hata: Yanlış şablonu seçmek**
>
> Şablon listesinde iki benzer isim görebilirsin: **Console App** ve **Console App (.NET Framework)**. İkincisi, yalnızca Windows'ta çalışan eski .NET Framework içindir. Kitabın örnekleri için **Console App** seç ve sürüm olarak .NET 10.0'ı işaretle.

"Do not use top-level statements" kutusu işaretliyse Visual Studio, `class Program` ve `Main` içeren klasik biçimli bir dosya üretir. Bu biçimi birazdan inceleyeceğiz. Kitap boyunca kutu **boş** kalacak; böylece dosya doğrudan kod satırıyla başlar.

Yol 2: VS Code, C# Dev Kit ve .NET 10 SDK (Windows, macOS, Linux)

1. dotnet.microsoft.com/download adresinden **.NET 10 SDK** yükleyicisini indir ve kur. (macOS'ta Homebrew kullanıyorsan: `brew install --cask dotnet-sdk`.)
2. Bir terminal aç (Windows'ta *Komut İstemi* veya *PowerShell*) ve kurulumu kontrol et:

```
dotnet --version
10.0.401
```

Sürüm numarasının sonu farklı olabilir; ama `10` ile başlamalı. "komut bulunamadı" (*not recognized*) hatası alırsan terminali kapatıp yeniden aç; yine olmazsa SDK'yı tekrar kur.

1. code.visualstudio.com adresinden VS Code'u kur.
2. Sol menüden **Extensions** (Ctrl+Shift+X) bölümüne gir, **"C# Dev Kit"** (Microsoft) eklentisini kur. Eklenti; kod tamamlama, hata gösterme ve hata ayıklama özelliklerini getirir.
3. Bilgisayarda bir klasör aç (ör. `CsDersleri`) ve VS Code'da **File → Open Folder** ile bu klasörü aç.

> **İpucu**
>
> VS Code'da **Ctrl+Shift+P** ile komut paletini açıp `.NET: New Project` yazarsan eklenti, proje oluşturmayı menülerle yapar. Aşağıdaki terminal komutları aynı işi tek satırda görür.

Terminalde dotnet komutları

Her iki yolda da arka planda `dotnet` komutu çalışır. VS Code'da **Terminal → New Terminal** ile alttaki terminali açıp şunları yaz:

```
dotnet new console -n Merhaba     ← Merhaba adlı klasörde yeni proje oluştur
cd Merhaba                        ← klasörün içine gir
dotnet run                        ← derle ve çalıştır
Hello, World!
```

`dotnet new console` komutu iki dosyalı küçük bir proje üretir:

```
Merhaba/
    Merhaba.csproj     ← proje dosyası: hangi .NET sürümü, hangi ayarlar
    Program.cs         ← kaynak kodun burada
```

`Merhaba.csproj` dosyasının içi şöyledir:

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>

</Project>
```

Bu dosyada iki ayar bizi ilgilendirir. `ImplicitUsings`, `Console` gibi çok kullanılan sınıfların bulunduğu **ad alanlarını** (namespace) senin yerine hazır tutar; bu yüzden kodun başına bir şey yazmayacağız. `Nullable`, boş değerlerle ilgili olası hataları derleyicinin uyarması içindir; bunu Bölüm 3'te göreceksin.

Komut	Ne yapar?
`dotnet new console -n Merhaba`	Yeni bir konsol projesi oluşturur
`dotnet build`	Derler: `bin/Debug/net10.0/Merhaba.dll` oluşur
`dotnet run`	Gerekirse derler, sonra programı çalıştırır

Sık Yapılan Hata: `dotnet run` yanlış klasörde

`dotnet run`, komutu yazdığın klasörde bir `.csproj` dosyası arar. Proje klasörünün (`Merhaba`) bir üstündeyken çalıştırırsan "çalıştırılacak proje bulunamadı" türünden bir hata alırsın. Önce `cd Merhaba` ile klasöre gir.

Yeni C# Notu: Tek dosyalık program: dotnet run Merhaba.cs

.NET 10 SDK ile birlikte, proje dosyası olmadan tek bir `.cs` dosyasını doğrudan çalıştırabilirsin:

```
dotnet run Merhaba.cs
```

Dosyadaki kod arka planda derlenip hemen çalıştırılır. Hızlı denemeler için çok kullanışlıdır. Bu özelliğe *file-based app* (dosya tabanlı uygulama) denir.

Çevrimiçi derleyici

Hiçbir şey kurmadan denemek istersen dotnetfiddle.net gibi çevrimiçi derleyicileri kullanabilirsin: tarayıcıda kodu yaz, **Run**'a bas, çıktı aşağıda görünür. Telefonda bile çalışır. Ama:

- Sitenin desteklediği C# ve .NET sürümü, sitenin kendi seçimlerine bağlıdır. Yeni özellikler çalışmayabilir.
- Bu sitelerdeki başlangıç kodu genellikle klasik biçimdedir ve en üstte `using System;` bulunur.
- Dosya işleme gibi konular ve büyük projeler için uygun değildir.

Kısaca: çevrimiçi derleyici kısa denemeler içindir; ödev ve projelerde yerel kurulumu kullan.

1.5 İlk program: Merhaba, Dünya!

Programlama geleneğidir: Her yeni dilde ilk program ekrana bir selam yazar. C#'ta bu program tek satırdır:

merhaba.cs

```
Console.WriteLine("Merhaba, Dünya!");
```

Çıktı

```
Merhaba, Dünya!
```

Programı çalıştırmak için Visual Studio'da **Ctrl+F5**, VS Code'da terminalde `dotnet run` yeter. Tek satırı parçalarına ayıralım:

Kod	Anlamı
`Console`	Konsol penceresini (siyah ekranı) temsil eden hazır **sınıf** (class). `System` ad alanındadır.
`.`	"İçindeki" demektir: `Console` sınıfının içindeki `WriteLine`.
`WriteLine`	Bir **metot** (method), yani yapılacak iş. Parantez içindekini ekrana yazar ve **alt satıra geçer**.
`( ... )`	Metoda verilen bilgi (**argüman**, argument).
`"Merhaba, Dünya!"`	Çift tırnak içindeki metin bir **string**'dir (karakter dizisi).
`;`	Her **deyim** (statement) noktalı virgülle biter.

`Console.WriteLine(...)` satırını gördüğünde "ekrana yaz" diye oku. Sınıf, metot ve argüman kavramlarının derinine sonraki bölümlerde ineceğiz. Şimdilik kalıbı bil.

Programın başka bir şey yazmadığına dikkat et: ne sınıf var ne `Main`. Bunun nedeni bir sonraki başlıkta.

> **Sık Yapılan Hata: C# büyük/küçük harf duyarlıdır**
>
> C# **case-sensitive** bir dildir: `Console` ile `console`, `WriteLine` ile `Writeline` farklı şeylerdir. Bu yazımların hiçbiri çalışmaz. Kural: .NET'in hazır adları (`Console`, `WriteLine`, `String`) her kelimenin ilk harfi büyük yazılır.

1.6 Üst düzey deyimler ve klasik biçim

C# programı her zaman bir `Main` metodundan başlar. Ama C# 9'dan beri bu iskeleti yazmak zorunda değilsin: dosyaya doğrudan deyimleri yazarsan derleyici onları senin için gizli bir `Main` metodunun içine koyar. Buna **üst düzey deyimler** (top-level statements) denir. Kodlar yukarıdan aşağıya, yazıldığı sırayla çalışır. Kitabın bütün programları bu biçimdedir.

Aynı programın **klasik** biçimi şöyledir:

klasik.cs

```
class Program
{
    static void Main(string[] args)
    {
        Console.WriteLine("Merhaba, Dünya!");
    }
}
```

Çıktı

```
Merhaba, Dünya!
```

İki program aynı IL'e derlenir ve aynı çıktıyı verir. Fark yalnızca iskeleti kimin yazdığıdır:

Kod	Anlamı
`class Program`	`Program` adında bir **sınıf** tanımlıyoruz. Klasik biçimde her kod bir sınıfın içinde olmak zorundadır.
`{ ... }`	Süslü parantezler bir **blok** oluşturur: sınıfın veya metodun gövdesi. Her açılan `{` kapanmalıdır.
`static void Main(string[] args)`	**Main metodu**: Programın başladığı yer. CLR çalıştırırken ilk olarak burayı arar.
`Console.WriteLine(...)`	Ekrana yazan deyim; üst düzey biçimdeki satırın aynısı.

`Main` satırındaki kelimelerin (`static`, `void`, `string[] args`) tam anlamını Bölüm 6'da, `class` kavramını Bölüm 9'da öğreneceksin. Şimdilik "program buradan başlar" bilgisi yeter.

Eski kitaplarda ve sınav kâğıtlarında bu iskeletin üstünde `using System;` satırını da görürsün. `Console` sınıfı `System` ad alanında olduğundan bu satır, onu kısa adıyla kullanmanı sağlar. Projemizde `ImplicitUsings` ayarı açık olduğu için `System` hazır gelir; satırı yazarsan da program yine derlenir.

> **Sınavda Çıkar**
>
> Üniversite sınavlarında ve eski kaynaklarda **klasik biçim** çok yaygındır; "bu programı yaz" denirse `class Program` ve `Main` ile yazman istenebilir. İkisini de tanı. Dikkat edilecekler: `Main` büyük `M` ile yazılır (`main` yazarsan derleyici giriş noktası bulamaz ve CS5001 hatası verir) ve bir projede üst düzey deyim yazılan dosya yalnız bir tanedir.

Bu kitapta ilerleyen bölümlerde metotları ve sınıfları ana kodun **altına** yazacağız. Üst düzey deyimlerin bulunduğu dosyada ana kod her zaman en üsttedir.

1.7 Write, WriteLine ve kaçış dizileri

Ekrana yazmanın iki temel yolu vardır:

- `Console.WriteLine(x)` → `x`'i yazar, sonra **alt satıra geçer**.
- `Console.Write(x)` → `x`'i yazar, **aynı satırda kalır**.

yazFarki.cs

```
Console.Write("Bir ");
Console.Write("İki ");
Console.WriteLine("Üç");
Console.WriteLine("Dört");
Console.WriteLine();
Console.WriteLine("Beş");
```

Çıktı

```
Bir İki Üç
Dört

Beş
```

Parametresiz `Console.WriteLine()` sadece alt satıra geçer, yani boş bir satır bırakır.

Bir string'in içine doğrudan yazılamayan ya da özel anlamı olan karakterler için **kaçış dizileri** (escape sequence) kullanılır. Hepsi ters bölü `\` ile başlar:

Kaçış dizisi	Anlamı
`\n`	Yeni satır (alt satıra geç)
`\t`	Sekme (tab), genelde birkaç boşluk kadar ileri atlar
`\"`	Çift tırnak karakteri
`\'`	Tek tırnak karakteri
`\\`	Ters bölü karakterinin kendisi

kacis.cs

```
Console.WriteLine("Ad\tSoyad\tNot");
Console.WriteLine("Ali\tYılmaz\t85");
Console.WriteLine("Hoca \"çok kolay\" dedi.");
Console.WriteLine("Dosya: C:\\cs\\Merhaba.cs");
Console.Write("Satır 1\nSatır 2\n");
```

Çıktı

```
Ad      Soyad   Not
Ali     Yılmaz  85
Hoca "çok kolay" dedi.
Dosya: C:\cs\Merhaba.cs
Satır 1
Satır 2
```

İpucu: Ters bölüden kurtulmak: @ ile yazılan metin

Dosya yollarındaki her ters bölüyü ikiye katlamak yorucudur. Metnin başına `@` koyarsan (**verbatim string**) içindeki `\` karakterleri olduğu gibi yazılır: `Console.WriteLine(@"C:\cs\Merhaba.cs");` Dikkat: `@` ile yazılan metinde `\n` artık yeni satır olmaz; harfi harfine `\n` yazılır.

Sık Yapılan Hata: Tek tırnak ile çift tırnak aynı şey değildir

C#'ta metin her zaman **çift tırnak** `" "` içine yazılır. Tek tırnak `' '` yalnızca **tek bir karakter** içindir (Bölüm 2). `Console.WriteLine('Selam');` yazarsan "karakter sabitinde çok fazla karakter var" anlamına gelen CS1012 hatası alırsın.

Sınavda Çıkar

Çıktı sorularında `Write` / `WriteLine` karışımı ve `\n` çok sık kullanılır. Kural basit: Ekrandaki "imleç" `WriteLine`'dan ve `\n`'den sonra alt satıra iner, `Write`'tan sonra olduğu yerde kalır. Soruyu çözerken imlecin nerede olduğunu takip et.

Metin birleştirme

`+` operatörü iki string'i yan yana ekler (*concatenation*). Sayılarla kullanıldığında ise toplama yapar. İkisi bir arada olduğunda işlem **soldan sağa** yapılır ve bir string ile karşılaşıldığı andan itibaren sonuç metne dönüşür:

birlestirme.cs

```
Console.WriteLine("C# " + "14");
Console.WriteLine(1 + 2 + " elma");
Console.WriteLine("elma " + 1 + 2);
Console.WriteLine("elma " + (1 + 2));
```

```
Çıktı
C# 14
3 elma
elma 12
elma 3
```

İkinci satırda önce `1 + 2 = 3` hesaplanır, sonra `"3 elma"` olur. Üçüncü satırda ise önce `"elma " + 1` → `"elma 1"`, sonra `"elma 1" + 2` → `"elma 12"` olur. Parantez kullanırsan toplama önce yapılır. Birleştirmenin daha rahat bir yolunu (`$"..."`) sonraki bölümlerde göreceksin.

> **Sık Yapılan Hata**
>
> `"Toplam: " + 5 + 3` ifadesi `Toplam: 8` değil **`Toplam: 53`** yazdırır. Sayıları toplamak istiyorsan parantez kullan: `"Toplam: " + (5 + 3)`. Bu, vize sınavlarının en sevilen tuzağıdır.

1.8 Yorum satırları

Yorumlar (comments), derleyicinin tamamen görmezden geldiği, kodu okuyan insana yönelik notlardır:

yorumlar.cs

```
// Tek satırlık yorum: satırın sonuna kadar geçerlidir
Console.WriteLine("Merhaba"); // kodun yanına da yazılabilir

/* Çok satırlı yorum:
   Bu iki işaret arasındaki her şey yok sayılır.
Console.WriteLine("Bu satır çalışmaz"); */

Console.WriteLine("Yorumlar çalışmaz, ben çalışırım.");
```

```
Çıktı
Merhaba
Yorumlar çalışmaz, ben çalışırım.
```

Yorum türü	Sözdizimi	Kullanım
Tek satır	`// ...`	Kısa açıklamalar
Çok satır	`/* ... */`	Uzun açıklamalar, kod parçasını geçici olarak devre dışı bırakma
XML belge yorumu	`/// <summary>...</summary>`	Sınıf ve metotların üstüne yazılan belge notu

`///` ile başlayan **belge yorumları** (documentation comments), Visual Studio'da bir metodun üzerine fareyle gelince açıklama olarak görünür. Bunları metotları ve sınıfları öğrendikten sonra kullanacaksın; şimdilik varlığını bil.

> **İpucu**
>
> Bir veya birkaç satırı seçip yorum satırına çevirmek için Visual Studio'da **Ctrl+K, Ctrl+C**, VS Code'da **Ctrl+/** (macOS'ta **Cmd+/**) tuşlarını kullan. Yorumdan çıkarmak için Visual Studio'da **Ctrl+K, Ctrl+U**; VS Code'da aynı tuşa tekrar basman yeter.

1.9 Yazım kuralları

Derleyici yazım konusunda çok titizdir. Şimdilik şu dört kuralı bil:

Kural	Açıklama
Büyük/küçük harf	`Console` ≠ `console`. Her harf birebir aynı olmalı.
Deyim sonu	Her deyim `;` ile biter.
Bloklar	Her açılan `{` bir `}` ile kapanır.
Girinti	Boşluk ve girinti derleyici için önemsizdir; ama okunabilirlik için bloğun içi 4 boşluk içeriden yazılır. Kitapta `{` hep alt satırdadır (C# geleneği).

Boşluklar, boş satırlar ve girinti derleyici için anlamsızdır. `Console . WriteLine ( "a" ) ;` bile derlenir, ama kimse böyle yazmaz.

> **Sık Yapılan Hata: Satır sonu mu, deyim sonu mu?**
>
> C#'ta satırın bitmesi deyimin bittiği anlamına gelmez; deyimi bitiren `;` işaretidir. Tersine, bir deyimi iki satıra bölebilirsin; derleyici `;` görene kadar okur. `;` unutmak, öğrencilerin en sık yaptığı hatadır.

1.10 Hata türleri

Program yazarken dört farklı durumla karşılaşırsın: üç tür hata ve uyarılar. Hangisi olduğunu anlamak, sorunu çözmenin yarısıdır.

1. Derleme hatası (compile-time error)

Kod C#'ın **yazım kurallarına** (sözdizimi, *syntax*) uymuyorsa derleyici derlemez, `.dll` dosyası oluşmaz, program hiç çalışmaz. Visual Studio ve VS Code bu hataların altını **kırmızı dalgalı çizgiyle** gösterir. En klasik örnek unutulan noktalı virgüldür:

eksikNoktali.cs

```
Console.WriteLine("Birinci satır");
Console.WriteLine("İkinci satır")
Console.WriteLine("Üçüncü satır");
```

Derleyici çıktısı – program derlenmez

```
eksikNoktali.cs(2,34): error CS1002: ; expected
```

Derleyici hatanın **dosya adını, satır ve sütun numarasını**, `error` kelimesini, `CS` ile başlayan **hata kodunu** ve kısa bir açıklamayı yazar. Burada `CS1002` kodu "`;` bekleniyordu" demektir. Visual Studio'da hatalar alttaki **Error List** (Hata Listesi) penceresinde de toplanır; satıra çift tıklarsan imleç hatanın yerine gider.

Bir başka klasik hata, büyük/küçük harf karışıklığıdır:

kucukHarf.cs

```
console.WriteLine("Merhaba");
```

Derleyici çıktısı – program derlenmez

```
kucukHarf.cs(1,1): error CS0103: The name 'console' does not exist in the current context
```

`CS0103` kodu "böyle bir ad yok" demektir: derleyici `console` diye bir şey bilmiyor, çünkü sınıfın adı `Console`'dur.

> **Not: Mesajlar Türkçe görünebilir**
>
> Türkçe Windows'ta ve Türkçe Visual Studio'da derleyici mesajları Türkçe gelebilir; bu kitaptaki çıktılar ise İngilizcedir. Önemli olan **`CS` kodudur**: `CS1002` her dilde `CS1002`'dir. Hatayı internette ararken mesajı değil kodu ara; Ek C'de en sık görülen hataların Türkçe açıklamalarını bulabilirsin.

Aynı anda birden çok hata varsa derleyici hepsini listeler. Ama ilk hata çoğu zaman diğerlerinin sebebidir; **her zaman listenin en üstündeki hatadan başla**.

2. Çalışma zamanı hatası (runtime error)

Kod kurallara uygundur ve derlenir; ama **çalışırken** yapılamayacak bir işlemle karşılaşılır ve program çöker. C#'ta bunlara **istisna** (exception) denir. Aşağıdaki programda `payda` adlı değişkende `0` var. Değişkenleri Bölüm 2'de öğreneceksin; şimdilik "adı olan bir kutu" diye oku:

sifiraBolme.cs

```
int pay = 10;
int payda = 0;
Console.WriteLine("Hesap başlıyor...");
Console.WriteLine(pay / payda);
Console.WriteLine("Bu satır hiç çalışmaz.");
```

Çıktı – program yakalanmamış istisna ile sonlanır

```
Hesap başlıyor...
Unhandled exception. System.DivideByZeroException: Attempted to divide by zero.
   at Program.<Main>$(String[] args) in sifiraBolme.cs:line 4
```

İlk satır yazdırıldı, ama tam sayıyı sıfıra bölmeye çalışınca `DivideByZeroException` oluştu ve program orada durdu. Çıktıdaki `Unhandled exception` ("yakalanmamış istisna") ifadesi, programın bu hatayla başa çıkamadığını söyler. Ardından istisnanın türü (`System.DivideByZeroException`) ve nedeni gelir. `at ...` ile başlayan satır hatanın **hangi satırda** olduğunu gösterir: yukarıdaki kodda `line 4`. Windows'ta bu satırda dosya adının önünde tam klasör yolu da görünür. İstisnaları nasıl yakalayacağımızı Bölüm 12'de öğreneceğiz.

> **Not: Neden doğrudan 10 / 0 yazmadık?**
>
> `Console.WriteLine(10 / 0);` yazarsan derleyici sıfıra bölme olacağını önceden görür ve programı derlemeyi reddeder (CS0020, "sabit sıfıra bölme"). Yani bu bir **derleme hatası**dır. Değişken kullanınca sıfır ancak program çalışırken ortaya çıkar; bu yüzden hata çalışma zamanında yakalanır.

3. Mantık hatası (logic error)

En sinsi hata budur: Program derlenir, çalışır, çökmez; ama **yanlış sonuç** üretir. Bilgisayar sana bir şey söylemez, hatayı sen bulmak zorundasın.

ortalama.cs

```
// İki notun ortalamasını hesaplamak istiyoruz: (70 + 90) / 2 = 80
Console.WriteLine(70 + 90 / 2);
Console.WriteLine((70 + 90) / 2);
```

Çıktı

```
115
80
```

İlk satır 80 yerine 115 yazdırdı, çünkü bölme toplamadan **önce** yapılır: `70 + 45 = 115`. Doğru sonuç için parantez gerekir.

Uyarılar (warnings)

Derleyici bazen kodu hatasız derler ama bir şeyin yanlış olabileceğini düşünür ve seni **uyarır**. Uyarı bir hata değildir: program yine de derlenir ve çalışır.

uyariOrnek.cs

```
int kullanilmayan = 5;
Console.WriteLine("Program çalıştı.");
```

Derleyici uyarısı (warning)

```
uyariOrnek.cs(1,5): warning CS0219: The variable 'kullanilmayan' is assigned but its value is never used
```

Çıktı

```
Program çalıştı.
```

`kullanilmayan` değişkenine bir değer atandı ama değer hiçbir yerde kullanılmadı. Derleyici bunun bir unutkanlık olabileceğini söyler: `warning CS0219`. Hata mesajlarının başında `error`, uyarıların başında `warning` yazar. Uyarıyı görmezden gelme; çoğu zaman gerçek bir hatanın habercisidir. Bu kitaptaki programlar, uyarıyı bilerek gösterdiğimiz yerler dışında, tek bir uyarı bile vermeden derlenir.

Özet tablo

Tür	Ne zaman ortaya çıkar?	Program çalışır mı?	Örnek
Derleme hatası (`error CSxxxx`)	Derleme aşamasında	Hayır, `.dll` oluşmaz	Eksik `;`, yanlış yazılmış `Console`
Çalışma zamanı hatası	Program çalışırken	Başlar ama yarıda çöker	Değişkenle sıfıra bölme
Mantık hatası	Sonuçlara bakınca	Evet, ama yanlış sonuç verir	Unutulan parantez
Uyarı (`warning CSxxxx`)	Derleme aşamasında	Evet, normal çalışır	Kullanılmayan değişken

Sınavda Çıkar

"Aşağıdaki hata hangi türdendir?" sorusu çok sık çıkar. İpucu: Eksik `;`, kapanmayan `}`, yanlış harf büyüklüğü, tanımsız ad → **derleme**. Sıfıra bölme, dizinin dışına çıkma → **çalışma zamanı**. Yanlış formül → **mantık**. Derleme hatasında program hiç başlamaz; çalışma zamanı hatasında bir kısmı çalışıp yarıda kesilir; mantık hatasında sonuna kadar çalışır.

Bölüm Özeti – Bilmen Gerekenler

- C# kaynak kodu (`.cs`), C# derleyicisi ile **IL**'e (`.dll`) derlenir; **CLR** içindeki **JIT**, IL'i çalışırken makine koduna çevirir. Platform bağımsızlığı buradan gelir.
- **SDK** = Runtime + derleyici ve araçlar; **Runtime** = CLR + hazır kütüphaneler. Geliştirici için .NET SDK gerekir. C# bir dildir, .NET bir platformdur.
- Kurulum: Visual Studio 2026 Community (Console App, .NET 10) veya VS Code + C# Dev Kit + .NET 10 SDK. Terminalde `dotnet new console -n Merhaba` ve `dotnet run`.
- Bu kitapta program doğrudan `Console.WriteLine(...)` gibi **üst düzey deyimlerle** başlar. Klasik biçim `class Program { static void Main(string[] args) { ... } }` sınavda sorulabilir.
- C# **büyük/küçük harf duyarlıdır.** Her deyim `;` ile biter; her `{` bir `}` ile kapanır.
- `WriteLine` yazıp alt satıra geçer, `Write` aynı satırda kalır. `\n` yeni satır, `\t` sekme, `\"` tırnak, `\\` ters bölü.
- `+` string ile kullanılınca birleştirir; işlem **soldan sağa** yapılır: `1 + 2 + "a"` → `"3a"`, `"a" + 1 + 2` → `"a12"`.
- Yorumlar: `//`, `/* */`, belge yorumu `///`.
- Hata türleri: **derleme** (`error CSxxxx`, program başlamaz), **çalışma zamanı** (istisna, program çöker), **mantık** (yanlış sonuç). **Uyarı** hata değildir ama dikkate alınmalıdır.

Sorular

A) Çoktan Seçmeli

1 C# kaynak kodu derleyici tarafından derlendiğinde asıl çıktı olarak hangisi üretilir?

(A) Doğrudan işlemcinin makine kodunu içeren, tek platformda çalışan bir dosya
(B) IL (ara dil) içeren bir `.dll` dosyası
(C) Aynı kodun `.cs` uzantılı yeni bir kopyası
(D) Yalnızca hata listesini içeren bir `.log` dosyası
(E) Projeleri gruplayan bir `.sln` dosyası

2 IL kodunu, program **çalışırken** makine koduna çeviren bileşen hangisidir?

(A) C# derleyicisi
(B) Visual Studio'nun hata listesi penceresi
(C) `dotnet new` komutu
(D) CLR içindeki JIT derleyicisi
(E) `.csproj` proje dosyası

3 Aşağıdaki satırlardan hangisi (tek başına bir programda) hatasız derlenir?

(A) `console.WriteLine("Selam");`
(B) `Console.Writeline("Selam");`
(C) `Console.WriteLine('Selam');`
(D) `Console.WriteLine("Selam")`
(E) `Console.WriteLine("Selam");`

4 Aşağıdakilerden hangisi bir **çalışma zamanı** (runtime) hatasıdır?

(A) Tam sayı bir değerin, içinde `0` olan bir değişkene bölünmesi

(B) `Console.WriteLine("Merhaba")` satırının sonundaki `;` işaretinin unutulması

(C) `Console` kelimesinin `console` diye yazılması

(D) Kapanmayan bir süslü parantez `{`

(E) Ortalama hesaplanırken parantezin unutulması

5 `Console.WriteLine(1 + 2 + "A" + 3 + 4);` komutu ekrana ne yazdırır?

(A) `12A34`

(B) `3A7`

(C) `3A34`

(D) `12A7`

(E) Derleme hatası verir

B) Çıktı Tahmini

6 Aşağıdaki program ekrana ne yazdırır?

```
Console.Write("C");
Console.WriteLine("#");
Console.Write("1");
Console.Write("4\n");
Console.WriteLine("N\tE\tT");
```

7 Aşağıdaki program ekrana ne yazdırır?

```
Console.WriteLine(10 + 5 + "10");
Console.WriteLine("10" + 5 + 5);
Console.WriteLine("10" + (5 + 5));
Console.WriteLine(10 + 5 * 2);
```

8 Aşağıdaki program ekrana ne yazdırır?

```
// Console.WriteLine("Bir");
Console.Write("İki "); /* Console.WriteLine("Üç"); */
Console.WriteLine("\"Dört\"");
Console.WriteLine("Beş\\Altı");
Console.Write("Yedi\n");
Console.WriteLine("Sekiz");
```

C) Kod Yazma

9 Ekrana aşağıdaki **kimlik kartını** yazdıran `Kimlik` adlı programı üst düzey deyimlerle yaz. Her satır için ayrı bir `Console.WriteLine` kullan.

```
************************
* Ad    : Zeynep Kaya   *
* Bölüm : Bilgisayar M. *
* Sınıf : 1             *
************************
```

10 **Tek bir** `Console.WriteLine` komutu ve kaçış dizileri kullanarak aşağıdaki çıktıyı üreten `TekSatir` programını yaz:

```
Ders    Kredi
"C#"    6
C:\ders\cs
```

(İlk iki satırda sütunlar arasında **tab** karakteri vardır.)

Çözümler

Cevap anahtarı

1 B	2 D	3 E	4 A	5 C

1 Cevap: B

C# derleyicisi `.cs` dosyasını doğrudan makine koduna değil, **IL** koduna çevirir ve sonucu `.dll` dosyasına yazar. (A) şıkkındaki yol, C ve C++ gibi tam derlenen dillerin yoludur. (C) yanlıştır; kaynak kod zaten `.cs` dosyasındadır. (D) derlemenin asıl ürünü değildir. (E) `.sln` bir çözüm dosyasıdır, projeleri gruplar; derleme çıktısı değildir.

2 Cevap: D

IL'i çalışma anında makine koduna çeviren parça, CLR'ın içindeki **JIT** derleyicisidir. (A) şıkkındaki C# derleyicisi, kaynak kodu IL'e çevirir ve bu iş program çalışmadan önce biter. (B) bir editör penceresidir. (C) yeni proje oluşturur. (E) proje ayarlarını tutan bir dosyadır.

3 Cevap: E

C# büyük/küçük harf duyarlıdır, deyimler `;` ile biter ve metin çift tırnak ister. (A) `console` yanlış yazılmıştır (CS0103). (B) `Writeline` yanlıştır; doğrusu `WriteLine`. (C) tek tırnak yalnızca tek karakter içindir, "Selam" için çift tırnak gerekir (CS1012). (D) sonunda `;` yoktur (CS1002). Yalnızca (E) bütün kurallara uyar.

4 Cevap: A

Sıfıra bölme, kod derlendikten sonra **program çalışırken** ortaya çıkar (`DivideByZeroException`). B, C ve D derleme hatasıdır (eksik `;`, yanlış harf büyüklüğü, kapanmayan blok). E mantık hatasıdır: program çalışır ama yanlış sonuç verir.

5 Cevap: C

İşlem soldan sağa yapılır. `1 + 2` iki sayıdır, toplanır: `3`. Sonra `3 + "A"` → `"3A"` olur; artık sonuç bir metindir. `"3A" + 3` → `"3A3"`, `"3A3" + 4` → `"3A34"`. (A) şıkkı sayıların baştan yan yana yazılacağını varsayar. (B) son iki sayının toplanacağını varsayar (`3 + 4`); ama metinle karşılaşıldıktan sonra `+` birleştirir. (D) iki hatayı birden yapar. (E) yanlıştır; sayı ile metni `+` ile birleştirmek geçerlidir.

6

`Write("C")` aynı satırda kalır, `WriteLine("#")` yazıp alt satıra geçer: ilk satır `C#`. `"1"` ve `"4\n"` yan yana yazılır, `\n` alt satıra geçirir: ikinci satır `14`. Son satırda harfler arasında tab boşluğu vardır.

```
Çıktı
C#
14
N       E       T
```

7

1. satır: `10 + 5 = 15`, sonra `15 + "10"` → `"1510"`.
2. satır: `"10" + 5` → `"105"`, sonra `"1055"`.

3. satır: Parantez önce: `5 + 5 = 10` → `"10" + 10` → `"1010"`.
4. satır: Çarpma önce: `5 * 2 = 10`, `10 + 10 = 20`.

Çıktı
```
1510
1055
1010
20
```

8

`//` ile başlayan satır ve `/* ... */` arasındaki kısım yorumdur, çalışmaz. `Write("İki ")` satırda kaldığı için `"Dört"` aynı satıra gelir. `\"` tırnak karakterini, `\\` tek bir ters bölüyü yazdırır. `Yedi` sonundaki `\n` yüzünden alt satıra geçilir, `Sekiz` de kendi satırında görünür.

Çıktı
```
İki "Dört"
Beş\Altı
Yedi
Sekiz
```

9

Her satırı ayrı bir string olarak yazdırmak yeterli. Kenarların hizalı olması için boşlukları dikkatle say: her satır tam 25 karakter olmalı.

kimlik.cs
```
Console.WriteLine("*************************");
Console.WriteLine("* Ad    : Zeynep Kaya   *");
Console.WriteLine("* Bölüm : Bilgisayar M. *");
Console.WriteLine("* Sınıf : 1             *");
Console.WriteLine("*************************");
```

Çıktı
```
*************************
* Ad    : Zeynep Kaya   *
* Bölüm : Bilgisayar M. *
* Sınıf : 1             *
*************************
```

10

Satır sonları için `\n`, tab için `\t`, tırnak için `\"`, ters bölü için `\\` kullanılır.

tekSatir.cs
```
Console.WriteLine("Ders\tKredi\n\"C#\"\t6\nC:\\ders\\cs");
```

Çıktı
```
Ders    Kredi
"C#"    6
C:\ders\cs
```

BÖLÜM 2

Değişkenler, Veri Tipleri ve Operatörler

Bu Bölümde Öğreneceklerin

- Değişken tanımlamayı, değer atamayı ve isimlendirme kurallarını
- C#'ın temel veri tiplerini (`int`, `double`, `decimal`, `char`, `bool`, `string`) ve hangisinin ne zaman kullanılacağını
- `const` ile sabit, `var` ile tip çıkarımı yapmayı
- Aritmetik operatörleri, **tam sayı bölmesini** ve `%` (mod) operatörünü
- Otomatik ve zorunlu tip dönüşümünü (casting)
- `x++` ile `++x` arasındaki farkı (sınavların bir numaralı tuzağı)
- `Math` sınıfını, banker yuvarlamasını, taşmayı (overflow) ve ondalık sayı hassasiyetini
- Bit operatörlerini (`&`, `|`, `^`, `~`, `<<`, `>>`) kısaca

2.1 Değişken nedir?

Değişken (variable), bellekte bir değeri saklamak için ayrılmış, **adı** ve **tipi** olan bir kutudur. Kutunun tipi, içine ne tür bir değer konabileceğini belirler: tam sayı, ondalıklı sayı, karakter, metin...

C#'ta bir değişkeni kullanmadan önce **tanımlaman** (declaration) gerekir. Tanımlarken ilk değeri hemen verebilir, sonradan da **atayabilirsin** (assignment):

```
tip  ad  =  değer;
int  yas =  19;
```

Programın ana kodunda tanımladığın değişkenlere **yerel değişken** (local variable) denir; bu bölümdeki bütün değişkenler yereldir. Ekrana yazdırırken `$` ile başlayan bir metin kullanacağız: süslü parantez içine yazdığın değişkenin ya da ifadenin değeri metnin içine yerleşir. Buna **metin içine değer yerleştirme** (string interpolation) denir; ayrıntısı Bölüm 3'te.

degiskenOrnek.cs

```
int yas = 19;              // tanımlama + ilk değer verme
double boy;                // sadece tanımlama
boy = 1.78;                // sonradan atama
string ad = "Can";

Console.WriteLine($"{ad} {yas} yaşında, boyu {boy}");

yas = 20;                  // değeri değişti, eski değer kayboldu
yas = yas + 1;             // sağ taraf önce hesaplanır: 20 + 1
Console.WriteLine($"Yeni yaş: {yas}");
```

```
Çıktı
Can 19 yaşında, boyu 1,78
Yeni yaş: 21
```

`yas = yas + 1;` satırı matematikte saçma görünür ama programlamada `=` **eşittir değil, atamadır**: "Sağdaki ifadeyi hesapla, sonucu soldaki değişkene koy." Önce `yas + 1` yani `21` hesaplanır, sonra `yas` değişkenine yazılır.

> **Not: Çıktıda virgül görmen normal**
>
> Yukarıdaki çıktıda boy `1,78` yazıldı; kodda ise `1.78` yazmıştık. Türkçe ayarlı bir bilgisayarda C# ondalık sayıları ekrana **virgülle** yazar (`3,5`). Koddaki sayı sabitleri ise her zaman **noktalıdır** (`3.5`); kodda `3,5` yazamazsın. Bu kitaptaki bütün çıktılar Türkçe ayarlı bir Windows bilgisayarda görünen çıktılardır. Bilgisayarının dil ve bölge ayarı İngilizce ise aynı program `1.78` yazar; bu bir hata değildir. Nedenini ve sayıları klavyeden okurken ne olduğunu Bölüm 3'te göreceksin.

> **Sık Yapılan Hata: İlk değer verilmemiş değişken**
>
> Değer vermeden bir yerel değişkeni **okuyamazsın**. C# bu durumda programı derlemez ve hata kodu `CS0165` olur:
>
> ```
> int sayi;
> Console.WriteLine(sayi);
> ```
>
> ```
> Derleyici çıktısı – program derlenmez
> ilkDegerYok.cs(2,19): error CS0165: Use of unassigned local variable 'sayi'
> ```

Aynı blokta aynı adla ikinci bir değişken **tanımlayamazsın** (`int x = 5; int x = 6;` yazarsan `CS0128` alırsın). Ama var olan değişkene istediğin kadar yeni değer **atayabilirsin** (`x = 6;`).

2.2 İsimlendirme kuralları

Değişken, metot ve sınıf adlarına **tanımlayıcı** (identifier) denir. Kurallar:

1. Harf, rakam ve alt çizgi `_` içerebilir.
2. **Rakamla başlayamaz:** `2sayi` geçersiz, `sayi2` geçerli.
3. Boşluk ve `-`, `+`, `.`, `$` gibi özel karakterler içeremez: `ogrenci no`, `ogrenci-no`, `$fiyat` geçersiz.
4. C#'ın **anahtar kelimeleri** (keyword) ad olarak kullanılamaz: `int`, `class`, `public`, `if`, `for`, `new`, `string`…
5. Büyük/küçük harf duyarlıdır: `sayi`, `Sayi` ve `SAYI` üç farklı isimdir.

Anahtar kelimenin başına `@` koyarsan (`@class`) derleyici onu ad olarak kabul eder. Bunu yalnızca bil; kendi kodunda kullanma.

Kurallara ek olarak herkesin uyduğu **isimlendirme gelenekleri** (naming conventions) vardır. Uymazsan program çalışır ama kodun "acemi" görünür:

Ne için?	Gelenek	Örnek
Yerel değişken ve parametre	**camelCase**: küçük harfle başla, sonraki kelimeler büyük	`ogrenciSayisi`, `notOrtalamasi`
Metot, sınıf, özellik	**PascalCase**: her kelime büyük harfle	`OrtalamaHesapla`, `Ogrenci`, `Ad`
Sabit (`const`)	**PascalCase**	`Pi`, `MaksOgrenci`

Metot, sınıf ve özellik kavramlarını ilerleyen bölümlerde öğreneceksin; şimdilik değişken adlarının `camelCase` olduğunu bil.

> **İpucu**
>
> C#, `öğrenciSayısı` gibi Türkçe harfli adlara izin verir; ama farklı bilgisayarlarda dosya kodlaması ve klavye sorunları çıkabildiği için **kullanmaman önerilir**. `ogrenciSayisi` yaz. Türkçe karakterleri yalnızca tırnak içindeki metinlerde (`"Öğrenci"`) kullan.

> **Sınavda Çıkar**
>
> "Aşağıdakilerden hangisi geçerli bir değişken adıdır?" sorusunda sırayla kontrol et: Rakamla başlıyor mu? Boşluk, `-` veya `$` var mı? Anahtar kelime mi (`class`, `int`, `new`...)? `_toplam`, `sayi2`, `ortalama` **geçerlidir**; `2sayi`, `ogrenci no`, `$fiyat`, `class` **geçersizdir**.

2.3 Temel veri tipleri

C#'ta her değerin bir **tipi** vardır. En çok kullanacağın tipler şunlardır:

Tip	Boyut	Değer aralığı / örnek	Ne için?
`byte`	1 bayt	0 … 255	Çok küçük, işaretsiz tam sayılar
`short`	2 bayt	−32 768 … 32 767	Küçük tam sayılar (nadiren)
`int`	4 bayt	yaklaşık ±2,1 milyar	**Varsayılan tam sayı tipi**
`long`	8 bayt	yaklaşık $\pm 9{,}2 \times 10^{18}$	Çok büyük tam sayılar: `8000000000L`
`float`	4 bayt	~7 basamak hassasiyet	Ondalıklı sayı: `3.14f`
`double`	8 bayt	~15–16 basamak hassasiyet	**Varsayılan ondalıklı tip**
`decimal`	16 bayt	28–29 basamak, ondalık sistemde tam	**Para:** `249.90m`
`char`	2 bayt	Tek karakter: `'A'`, `'7'`, `'ş'`	Karakter (Unicode)
`bool`	1 bayt	Sadece `true` veya `false`	Doğru/yanlış bilgisi
`string`	değişken	`"Merhaba"`	Metin

Pratikte en çok kullanacakların: tam sayı için **`int`**, ondalıklı sayı için **`double`**, para için **`decimal`**, karakter için **`char`**, doğru/yanlış için **`bool`**, metin için **`string`**.

veriTipleri.cs

```
byte kat = 12;
short yil = 2026;
int ogrenciSayisi = 1450;
long dunyaNufusu = 8_100_000_000L;   // _ okunabilirlik için
float sicaklik = 23.5f;
double pi = 3.141592653589793;
decimal fiyat = 249.90m;
char harf = 'C';
bool gectiMi = true;
string okul = "Teknik Üniversite";

Console.WriteLine($"{kat} {yil} {ogrenciSayisi}");
Console.WriteLine(dunyaNufusu);
Console.WriteLine($"{sicaklik} {pi} {fiyat}");
Console.WriteLine($"{harf} {gectiMi} {okul}");
```

Çıktı

```
12 2026 1450
8100000000
23,5 3,141592653589793 249,90
C True Teknik Üniversite
```

Sayıların içine yazılan alt çizgi `_` yalnızca okunabilirlik içindir; `8_100_000_000` ile `8100000000` aynı sayıdır. Ayrıca `bool` değerini kodda `true` yazarsın ama ekrana `True` basılır.

Her tamsayı tipinin sınırları, tipin üzerindeki `MaxValue` ve `MinValue` ile öğrenilir:

sinirDegerler.cs

```
Console.WriteLine(int.MaxValue);
Console.WriteLine(int.MinValue);
Console.WriteLine(long.MaxValue);
Console.WriteLine(short.MinValue);
Console.WriteLine(byte.MaxValue);
```

Çıktı

```
2147483647
-2147483648
9223372036854775807
-32768
255
```

İşaretli ve işaretsiz tipler

Tam sayı tiplerinin her birinin bir **işaretsiz** (unsigned) kardeşi vardır. İşaretsiz tip eksi sayı tutamaz, ama aynı bayt sayısıyla pozitif tarafta iki kat büyük sayılara çıkar. Adında `u` olan tip işaretsizdir (`ushort`, `uint`, `ulong`). Tek istisna `byte`'tır: Adında `u` yoktur ama işaretsizdir; işaretli karşılığının adı `sbyte`'tır.

İşaretli	Aralık	İşaretsiz	Aralık
`sbyte`	−128 … 127	`byte`	0 … 255
`short`	−32 768 … 32 767	`ushort`	0 … 65 535
`int`	yaklaşık ±2,1 milyar	`uint`	0 … 4 294 967 295
`long`	yaklaşık ±9,2 × 10^{18}	`ulong`	0 … 18 446 744 073 709 551 615

isaretsiz.cs

```
Console.WriteLine($"{sbyte.MinValue} {sbyte.MaxValue}");
Console.WriteLine($"{byte.MinValue} {byte.MaxValue}");
Console.WriteLine($"{ushort.MaxValue} {uint.MaxValue}");
Console.WriteLine(ulong.MaxValue);

uint a = 3;
uint b = 5;
Console.WriteLine(a - b);            // eksiye inmez, başa döner
```

Çıktı

```
-128 127
0 255
65535 4294967295
18446744073709551615
4294967294
```

`3 - 5` işlemi `uint` ile yapılınca −2 çıkmaz; sayı en küçük değerin altına inip en büyük uca döner. Bu, aşağıdaki **Taşma** başlığında göreceğin sessiz taşmanın aynısıdır. Sabit değerlerde `L` ekinin işaretsiz karşılıkları da vardır: `42u` bir `uint`, `42UL` bir `ulong` sabitidir. Günlük işlerde `int`, `long` ve `double` yeter; `byte` ham veri ve 0–255 arası değerler için kullanılır.

Sık Yapılan Hata: İşaretsiz tipe eksi değer koymak

`byte`, `ushort`, `uint` ve `ulong` eksi sayı tutamaz. Eksi bir sabiti bu tiplere atarsan program derlenmez (CS0031):

```
byte b = -1;
uint u = -5;
Console.WriteLine($"{b} {u}");
```

Derleyici çıktısı – program derlenmez

```
isaretsizHata.cs(1,10): error CS0031: Constant value '-1' cannot be converted to a 'byte'
isaretsizHata.cs(2,10): error CS0031: Constant value '-5' cannot be converted to a 'uint'
```

Not: `string` bir metin tipidir

`string`, bir metni (karakter dizisini) tutar ve **değiştirilemez** (immutable) bir tiptir: Bir `string` oluşturduktan sonra içindeki karakterleri değiştiremezsin, ancak yeni bir `string` üretirsin. Şimdilik onu diğer tipler gibi kullanacağız; ayrıntıları Bölüm 8'de göreceksin.

Sabit değerlerin (literal) tipi

Koda doğrudan yazdığın değerlere **sabit değer** (literal) denir ve onların da bir tipi vardır:

- Ondalık noktası olmayan sayılar (`42`) → **`int`** (`int`'e sığmayacak kadar büyükse kendiliğinden `long` olur)
- Ondalık noktası olan sayılar (`3.14`) → **`double`**
- Sonuna `L` eklenmiş tam sayılar (`42L`) → `long`
- Sonuna `f` eklenmiş ondalıklılar (`3.14f`) → `float`
- Sonuna `m` eklenmiş ondalıklılar (`3.14m`) → `decimal`
- Tek tırnak (`'A'`) → `char`, çift tırnak (`"A"`) → `string`

Bu yüzden `float oran = 0.75;` satırı derlenmez: `0.75` bir `double`'dır ve `float` değişkene `f` eki olmadan konamaz.

```
float oran = 0.75;
Console.WriteLine(oran);
```

Derleyici çıktısı – program derlenmez

```
floatHata.cs(1,14): error CS0664: Literal of type double cannot be implicitly converted to type
'float'; use an 'F' suffix to create a literal of this type
```

Doğrusu `0.75f` yazmaktır. Aynı şekilde para için `decimal fiyat = 19.99;` yazarsan hata alırsın; `19.99m` yazmalısın.

> **Sık Yapılan Hata: `char` ve `string` karıştırma**
>
> `char c = "A";` derlenmez; `char` tek tırnak ister: `char c = 'A';`. Tersine `string s = 'Merhaba';` da derlenmez. Kural: **tek karakter → tek tırnak**, **metin → çift tırnak**.

2.4 Sabitler (const) ve var

Değeri program boyunca **hiç değişmeyecek** bir değer için `const` kullanılır. Sabite tanımlandığı satırda değer vermek zorundasın ve değer derleme sırasında bilinmelidir (ör. klavyeden okunan bir sayı sabit olamaz). Sabite sonradan atama yapmaya çalışırsan derleme hatası alırsın:

sabitOrnek.cs

```
const double Pi = 3.14159;
const int HaftalikDers = 4;
double yaricap = 2.0;

Console.WriteLine($"Alan: {Pi * yaricap * yaricap}");
Console.WriteLine($"Dönemlik ders: {HaftalikDers * 14}");
```

Çıktı

```
Alan: 12,56636
Dönemlik ders: 56
```

```
const int Max = 100;
Max = 200;
```

Derleyici çıktısı – program derlenmez

```
sabitHata.cs(2,1): error CS0131: The left-hand side of an assignment must be a variable, property or
indexer
```

`var` anahtar kelimesiyle tipi yazmadan değişken tanımlayabilirsin; derleyici tipi sağdaki değerden **çıkarır** (type inference):

varOrnek.cs

```
var sayi = 10;        // int
var oran = 2.5;       // double
var mesaj = "Selam";  // string

Console.WriteLine($"{sayi * oran} {mesaj}");
Console.WriteLine($"{sayi.GetType()} {oran.GetType()} {mesaj.GetType()}");
```

Çıktı

```
25 Selam
System.Int32 System.Double System.String
```

İkinci satır, derleyicinin çıkardığı tipleri gösterir; `System.Int32` aslında `int`'in uzun adıdır. `var` tipsiz demek **değildir**: C# yine **statik tiplidir**, tip derleme sırasında belirlenir ve sonradan değişmez. `var sayi = 10;` dedikten sonra `sayi = "on";` yazamazsın. Ayrıca `var x;` gibi değersiz tanımlama da yapılamaz, çünkü çıkarım yapılacak bir değer yoktur. Yeni başlayan biri olarak tipleri **açıkça yazmanı** öneririz; tipleri düşünmek öğrenmenin parçasıdır.

2.5 Aritmetik operatörler

Operatör	İşlem	Örnek	Sonuç
`+`	Toplama	`7 + 2`	`9`
`-`	Çıkarma	`7 - 2`	`5`
`*`	Çarpma	`7 * 2`	`14`
`/`	Bölme	`7 / 2`	`3` (!)
`%`	Kalan (mod)	`7 % 2`	`1`

Tam sayı bölmesi

Bu bölümün **en önemli kuralı**: İki tam sayı birbirine bölünürse sonuç da **tam sayıdır**; ondalık kısım **atılır** (yuvarlanmaz, kesilir). Sonucun ondalıklı olması için işlenenlerden **en az birinin** ondalıklı olması gerekir:

bolme.cs

```
Console.WriteLine(7 / 2);
Console.WriteLine(7.0 / 2);
Console.WriteLine(7 / 2.0);
Console.WriteLine(9 / 10);
Console.WriteLine(-7 / 2);
Console.WriteLine(7 % 2);
Console.WriteLine(10 % 3);
Console.WriteLine(3 % 10);
Console.WriteLine(-7 % 3);
Console.WriteLine(7 % -3);
Console.WriteLine(7.5 % 2);
```

Çıktı
```
3
3,5
3,5
0
-3
1
1
3
-1
1
1,5
```

- `9 / 10` → `0,9` değil, **`0`**. Ondalık kısım kesildi.
- `-7 / 2` → `-3,5`'in ondalığı kesilir: **`-3`** (sıfıra doğru kesme; `-4` değil).
- `%` operatörü bölmeden kalanı verir. `3 % 10` → `3`, çünkü 3, 10'a 0 kere bölünür ve 3 kalır.
- Kalanın işareti **soldaki sayının** işaretiyle aynıdır: `-7 % 3` → `-1`, `7 % -3` → `1`.
- `%` ondalıklı sayılarda da çalışır: `7.5 % 2` → `1,5`.

Tam sayıyı sıfıra bölersen program çalışırken hata verir (Bölüm 12'de göreceksin).

> **Sınavda Çıkar: `%` operatörünün klasik kullanımları**
>
> - **Tek/çift kontrolü:** `sayi % 2 == 0` ise çift.
> - **Bölünebilme:** `sayi % 5 == 0` ise 5'e tam bölünür.
> - **Son basamak:** `sayi % 10` → `1234 % 10 = 4`.
> - **Son basamağı atma:** `sayi / 10` → `1234 / 10 = 123`.
>
> Son iki madde, "bir sayının basamaklarını topla / ters çevir" sorularının temelidir (Bölüm 5).

İşlem önceliği

Matematikteki gibi çarpma, bölme ve mod (`*`, `/`, `%`) toplama ve çıkarmadan (`+`, `-`) **önce** yapılır. Aynı öncelikteki işlemler **soldan sağa** yapılır. Parantez her şeyden önce gelir.

oncelik.cs
```cs
Console.WriteLine(2 + 3 * 4);
Console.WriteLine((2 + 3) * 4);
Console.WriteLine(10 - 4 - 3);
Console.WriteLine(20 / 4 * 2);
Console.WriteLine(2 + 10 % 4);
Console.WriteLine(17 % 5 * 2 + 1);
```

Çıktı
```
14
20
3
10
4
5
```

`20 / 4 * 2` ifadesinde `/` ve `*` aynı öncelikte olduğu için soldan sağa gidilir: `(20 / 4) * 2 = 10`. `20 / 8` değil!

Bu bölümde göreceğin operatörlerin yüksekten düşüğe öncelik sırası şöyledir (karşılaştırma ve mantıksal operatörlerin ayrıntısı Bölüm 4'te):

1. Parantez `( )`
2. Son ek `x++`, `x--`
3. Ön ek ve tekli işlemler: `++x`, `--x`, `-x`, `!x`, `~x` ve cast `(int)x`
4. `*`, `/`, `%`
5. `+`, `-`
6. `<<`, `>>`
7. `<`, `>`, `<=`, `>=`
8. `==`, `!=`
9. `&`, sonra `^`, sonra `|` (bit operatörleri)
10. `&&`, sonra `||`
11. Atama: `=`, `+=`, `-=`, `*=`, `/=`, `%=`

Şüphede kaldığında **parantez kullan**; hem okunur hem de güvenlidir.

2.6 Tip dönüşümü (casting)

Otomatik (genişleyen) dönüşüm

Küçük bir tipi **daha büyük** bir tipe koyarken veri kaybı olmayacağı için C# dönüşümü kendisi yapar (*implicit conversion*):

`byte → short → int → long → float → double`

`char` da `int`'e, `int` ve `long` da `decimal`'e otomatik dönüşür. Ama `double` ile `decimal` arasında dönüşüm her iki yönde de **zorunludur**; `decimal` ile `double`'ı doğrudan toplayamaz ya da çarpamazsın.

```
int a = 5;
double d = a;      // d = 5  (otomatik)
```

Bir işlemde farklı tipler varsa, küçük olan büyüğe dönüştürülür: `int + double` → `double`. Tam sayı bölmesi kuralının nedeni de budur: `7 / 2.0` işleminde `7`, `7.0`'a dönüştürülür.

Zorunlu (daralan) dönüşüm

Büyük bir tipi **daha küçük** bir tipe koymak veri kaybına yol açabileceği için C# bunu kendiliğinden yapmaz; senin açıkça istemen gerekir. Buna **cast** denir ve parantez içinde hedef tip yazılır. Tam sayıya dönüştürmenin ikinci yolu `Convert.ToInt32(...)`'dir:

donusum.cs

```
double d = 3.99;
int i = (int)d;                       // ondalık kısım KESİLİR
Console.WriteLine(i);
Console.WriteLine((int)-3.99);
Console.WriteLine(Convert.ToInt32(d)); // yuvarlar
Console.WriteLine(Convert.ToInt32(3.5));
Console.WriteLine(Convert.ToInt32(2.5));

int toplam = 7, adet = 2;
Console.WriteLine(toplam / adet);
Console.WriteLine((double)toplam / adet);
Console.WriteLine((double)(toplam / adet));
```

```
Çıktı
3
-3
4
4
2
3
3,5
3
```

Çıktıdaki satırları sırayla oku:

- `(int)3.99` → `3`: Cast **keser**, yuvarlamaz. `(int)-3.99` de `-3` olur.
- `Convert.ToInt32(3.99)` → `4`: `Convert` **yuvarlar**. Tam ortadaki değerde (`.5`) ise **çift sayıya** yuvarlar: `3,5` → `4`, `2,5` → `2`. Bu davranışın adı banker yuvarlamasıdır; `Math.Round` bölümünde yeniden göreceğiz.
- `(double)toplam / adet`: Cast bölmeden **önce** yapılır. `toplam` → `7,0` olur, `7,0 / 2 = 3,5`.
- `(double)(toplam / adet)`: Önce parantez içi: `7 / 2 = 3` (tam sayı bölmesi!), sonra `3` → `3,0`. Ondalık kısım çoktan kaybolmuştur; ekrana `3` yazılır.

> **Sık Yapılan Hata: Ortalama hesabında tam sayı bölmesi**
>
> Notları `int` olan bir öğrencinin ortalamasını `int toplam = 85 + 90 + 72; double ort = toplam / 3;` diye hesaplarsan sonuç `82,33` değil **`82`** olur. Bölme `int / int` olarak yapılıp ondalık atıldıktan sonra `double`'a çevrilir. Doğrusu: `toplam / 3.0` veya `(double)toplam / 3`.

Daralan dönüşüm cast olmadan yazılırsa derlenmez:

```
int x = 3.5;
Console.WriteLine(x);
```

```
Derleyici çıktısı – program derlenmez
daralmaHata.cs(1,9): error CS0266: Cannot implicitly convert type 'double' to 'int'. An explicit
conversion exists (are you missing a cast?)
```

Hata mesajının sonundaki "açık bir dönüşüm var" ipucu, `(int)` cast'inden söz eder. Derleyici hatayı yalnızca gösterir; veri kaybını göze alıp almayacağına sen karar verirsin.

char ve sayılar

`char` aslında bir sayıdır: Her karakterin bir **Unicode** (ve ilk 128 karakter için **ASCII**) numarası vardır. `'A'` = 65, `'a'` = 97, `'0'` = 48. Bu yüzden karakterlerle aritmetik yapılabilir:

charSayi.cs

```
char c = 'A';
Console.WriteLine(c + 1);              // char + int → int
Console.WriteLine((char)(c + 1));      // tekrar karaktere çevir
Console.WriteLine((int)'a');
c++;                                   // c artık 'B'
Console.WriteLine(c);
Console.WriteLine('7' - '0');          // rakam karakterini sayıya çevirme
```

Çıktı
```
66
B
97
B
7
```

> **İpucu: Bilmen gereken üç ASCII değeri**
>
> `'A'` = **65**, `'a'` = **97**, `'0'` = **48**. Gerisini bunlardan hesaplarsın: `'C'` = 67, `'z'` = 122. Büyük-küçük harf farkı her zaman **32**'dir.

2.7 Atama ve bileşik atama operatörleri

Bir değişkenin değerini kendisine bağlı olarak değiştirmek çok yaygın olduğu için kısa yazımlar vardır:

Kısa yazım	Uzun hali
`x += 5`	`x = x + 5`
`x -= 5`	`x = x - 5`
`x *= 5`	`x = x * 5`
`x /= 5`	`x = x / 5`
`x %= 5`	`x = x % 5`

bilesikAtama.cs
```
int x = 10;
x += 5;     // 15
x *= 2;     // 30
x -= 4;     // 26
x /= 4;     // 6  (tam sayı bölmesi)
x %= 4;     // 2
Console.WriteLine(x);

double d = 10;
d /= 4;
Console.WriteLine(d);
```

Çıktı
```
2
2,5
```

> **Sınavda Çıkar: Bileşik atamanın gizli cast'i**
>
> `byte b = 10; b = b + 5;` **derlenmez** (`CS0266`), çünkü `b + 5` işleminin sonucu `int`'tir ve `int` otomatik olarak `byte`'a dönüşmez. Ama `b += 5;` **derlenir**: Bileşik atama, sonucu sol taraftaki tipe kendisi cast eder. Yani `b += 5` aslında `b = (byte)(b + 5)` demektir.

2.8 Artırma ve azaltma: x++ ve ++x

Bir değişkeni 1 artırmak (`x = x + 1`) için `x++` veya `++x`, 1 azaltmak için `x--` veya `--x` yazılır. Tek başına bir satırda kullanıldıklarında **ikisi arasında fark yoktur**. Fark, bir **ifadenin içinde** kullanıldıklarında ortaya çıkar:

- **Son ek** (postfix) `x++`: Önce `x`'in **mevcut değerini kullan**, sonra artır.
- **Ön ek** (prefix) `++x`: Önce **artır**, sonra yeni değeri kullan.

artirma.cs

```
int a = 5;
int b = a++;     // b = 5 (eski değer), sonra a = 6
Console.WriteLine($"a={a} b={b}");

int c = 5;
int d = ++c;     // önce c = 6, sonra d = 6
Console.WriteLine($"c={c} d={d}");

int x = 3;
Console.WriteLine(x++);
Console.WriteLine(x);
Console.WriteLine(++x);

int y = 10;
int z = y-- - --y;   // 10 - 8
Console.WriteLine($"y={y} z={z}");

int w = 5;
w = w++;             // w artmaz!
Console.WriteLine(w);
```

Çıktı

```
a=6 b=5
c=6 d=6
3
4
5
y=8 z=2
5
```

`y-- - --y` ifadesini adım adım çözelim:

1. `y--`: Değer olarak **10** kullanılır, sonra `y` = 9 olur.
2. `--y`: Önce `y` = 8 olur, değer olarak **8** kullanılır.
3. `z = 10 - 8 = 2`, `y`'nin son değeri 8.

> **Sınavda Çıkar**
>
> `++`/`--` soruları her sınavda vardır. Çözerken bir **değişken tablosu** tut ve her `++`'dan sonra değeri güncelle. Altın kural: **"Artı işareti değişkenin neresindeyse, artırma o zaman olur."** Önde (`++x`) → önce artır. Arkada (`x++`) → kullandıktan sonra artır.

> **Sık Yapılan Hata**
>
> `w = w++;` satırı `w`'yu **artırmaz**! Önce eski değer (5) bir kenara alınır, `w` 6 olur, sonra eski değer (5) `w`'ya geri atanır. Sonuç: `w` hâlâ 5. Bu satırı gerçek kodda asla yazma; ama sınavda görürsen tuzağı tanı.

2.9 Math sınıfı

C#'ın hazır `Math` sınıfı sık kullanılan matematik işlemlerini sunar. Ayrıca bir şey eklemen gerekmez:

Metot	Ne yapar?	Örnek → Sonuç
`Math.Pow(a, b)`	a^b (üs alma), sonuç `double`	`Math.Pow(2, 10)` → `1024`
`Math.Sqrt(x)`	Karekök, sonuç `double`	`Math.Sqrt(81)` → `9`
`Math.Abs(x)`	Mutlak değer	`Math.Abs(-7)` → `7`
`Math.Max(a, b)`	Büyük olan	`Math.Max(3, 8)` → `8`
`Math.Min(a, b)`	Küçük olan	`Math.Min(3, 8)` → `3`
`Math.Round(x)`	En yakın tam sayıya yuvarla (**banker yuvarlaması**)	`Math.Round(3.7)` → `4`
`Math.Ceiling(x)`	Yukarı yuvarla	`Math.Ceiling(2.1)` → `3`
`Math.Floor(x)`	Aşağı yuvarla	`Math.Floor(2.9)` → `2`
`Math.PI`	π sabiti	`3,141592653589793`

`Math.Pow`, `Math.Sqrt`, `Math.Ceiling` ve `Math.Floor` ondalık değer döndürür; sonucun ondalığı sıfırsa ekrana tam sayı gibi yazılır (`1024`).

mathOrnek.cs

```
Console.WriteLine(Math.Pow(2, 10));
Console.WriteLine(Math.Sqrt(2));
Console.WriteLine(Math.Abs(-4.5));
Console.WriteLine(Math.Max(10, Math.Min(25, 17)));
Console.WriteLine(Math.Ceiling(-2.5) + " " + Math.Floor(-2.5));
Console.WriteLine(Math.PI);

// Hipotenüs: a = 3, b = 4
double c = Math.Sqrt(Math.Pow(3, 2) + Math.Pow(4, 2));
Console.WriteLine($"Hipotenüs: {c}");
```

Çıktı

```
1024
1,4142135623730951
4,5
17
-2 -3
3,141592653589793
Hipotenüs: 5
```

Yuvarlama: banker yuvarlaması

`Math.Round` tam ortadaki bir değeri (`.5`) gördüğünde okulda öğrendiğin gibi yukarı yuvarlamaz; **en yakın çift sayıya** yuvarlar. Buna **banker yuvarlaması** (banker's rounding) denir. Amaç, çok sayıda yuvarlamada hatanın bir yöne birikmemesidir. İstersen `MidpointRounding.AwayFromZero` seçeneğiyle okul usulü yuvarlamayı isteyebilirsin. `Math.Round` ikinci parametre olarak kaç basamak bırakılacağını da alır:

yuvarlama.cs

```
Console.WriteLine(Math.Round(0.5));
Console.WriteLine(Math.Round(1.5));
Console.WriteLine(Math.Round(2.5));
Console.WriteLine(Math.Round(3.5));
Console.WriteLine(Math.Round(-2.5));
Console.WriteLine(Math.Round(2.5, MidpointRounding.AwayFromZero));
Console.WriteLine(Math.Round(3.14159, 2));
Console.WriteLine(Math.Round(0.125m, 2));
Console.WriteLine(Math.Round(0.125m, 2, MidpointRounding.AwayFromZero));
Console.WriteLine(Convert.ToInt32(2.5));
Console.WriteLine((int)2.5);
```

Çıktı

```
0
2
2
4
-2
3
3,14
0,12
0,13
2
2
```

`0,5` → `0`, `1,5` → `2`, `2,5` → `2`, `3,5` → `4`: Hep çift sayıya gidildi. `-2,5` de `-2`'ye yuvarlandı. `AwayFromZero` ile `2,5` → `3` olur. Aynı kural `decimal` için de geçerlidir: `0,125` iki basamağa `0,12` olur, `AwayFromZero` ile `0,13`. Para hesabında hangi yuvarlamanın istendiğine dikkat et. Son iki satır da özettir: `Convert.ToInt32(2.5)` çifte yuvarlar, `(int)2.5` keser; ikisi bu örnekte aynı çıkar ama nedenleri farklıdır.

> ● **Sınavda Çıkar: `Math.Round(2.5)` kaçtır?**
>
> Cevap **2**'dir, `3` değil! `Math.Round(x)` ve `Convert.ToInt32(x)` tam ortada çift sayıya gider: `0,5 → 0`, `1,5 → 2`, `2,5 → 2`, `3,5 → 4`. Sonucun `3` çıkmasını istiyorsan `MidpointRounding.AwayFromZero` yazmak zorundasın. `Math.Round` `double` döndürür; `int` değişkene koymak için `(int)Math.Round(x)` yazılır.

2.10 Taşma ve ondalık hassasiyet

Her tipin bir sınırı vardır. `int`'in en büyük değerine 1 eklersen, sayı bir saatin 12'den 1'e dönmesi gibi **en küçük değere döner**. Buna **taşma** (overflow) denir ve C# varsayılan olarak bu durumda hata **vermez**; sonuç sessizce yanlış çıkar:

tasma.cs

```
int enBuyuk = int.MaxValue;
Console.WriteLine(enBuyuk);
Console.WriteLine(enBuyuk + 1);

int bin = 100_000;
int saniye = bin * bin;              // taşar!
long dogru = 100_000L * bin;
Console.WriteLine($"{saniye} / {dogru}");

byte kucuk = 250;
kucuk += 10;                         // 260, byte'a sığmaz
Console.WriteLine(kucuk);
```

Çıktı

```
2147483647
-2147483648
1410065408 / 10000000000
4
```

`bin * bin` işleminde iki `int` çarpılır; sonuç `int` sınırını aşar ve anlamsız bir sayı çıkar. Sonucu `long` bir değişkene koymak yetmez; çarpmanın kendisinin `long` olması için bir tarafa `L` ekleyerek işleme `long` olarak başlamak gerekir. `byte` örneğinde de `260`, 0–255 aralığına sığmadığı için başa dönüp `4` olur.

> **İpucu: Derleyici sabit taşmayı yakalar**
>
> Değerleri derleme sırasında bilinen bir ifade taşıyorsa (ör. `int.MaxValue + 1`) derleyici `CS0220` hatası verir. Bu yüzden yukarıdaki örneklerde değişken kullandık. Taşma çalışma sırasında, değişkenlerle ortaya çıkıyorsa C# sessiz kalır; asıl tehlike budur.

Sessiz taşma istemiyorsan işlemi `checked` ile yaparsın. Taşma olursa program `OverflowException` (taşma istisnası) fırlatıp durur:

kontrollu.cs

```
int enBuyuk = int.MaxValue;
Console.WriteLine("Hesap başlıyor");
int sonuc = checked(enBuyuk + 1);
Console.WriteLine(sonuc);
```

Çıktı – program yakalanmamış istisna ile sonlanır

```
Hesap başlıyor
Unhandled exception. System.OverflowException: Arithmetic operation resulted in an overflow.
   at Program.<Main>$(String[] args) in kontrollu.cs:line 3
```

Çıktıdaki ilk satır yazıldı, ardından `OverflowException` ile program kapandı; son `Console.WriteLine` hiç çalışmadı. İstisnaları ve onları yakalamayı Bölüm 12'de öğreneceksin. Bir bloğun tamamını denetlemek için `checked { ... }` yazılabilir.

Ondalıklı sayılar da bilgisayarda ikilik (binary) sistemde tutulduğu için bazı değerler **tam olarak** saklanamaz. `==` operatörü iki değerin eşit olup olmadığını sorar ve `true` ya da `false` verir (Bölüm 4):

hassasiyet.cs

```
double d1 = 0.1, d2 = 0.2;
Console.WriteLine(d1 + d2);
Console.WriteLine(d1 + d2 == 0.3);
Console.WriteLine(1.0 - 0.9);

decimal m1 = 0.1m, m2 = 0.2m;
Console.WriteLine(m1 + m2);
Console.WriteLine(m1 + m2 == 0.3m);

float f = 1.0f / 3;
double d = 1.0 / 3;
decimal m = 1.0m / 3;
Console.WriteLine($"{f} {d} {m}");
```

```
Çıktı
0,30000000000000004
False
0,09999999999999998
0,3
True
0,33333334 0,3333333333333333 0,333333333333333333333333333333
```

`double` ile `0,1 + 0,2` tam `0,3` etmez, bu yüzden karşılaştırma `False` çıkar. `decimal` değerleri **ondalık sistemde** tuttuğu için `0,1m + 0,2m` tam `0,3m`'dir. Son satır üç tipin hassasiyetini gösterir: `float` yaklaşık 7, `double` yaklaşık 15–16, `decimal` 28 basamak tutar.

> **Sık Yapılan Hata: Ondalıklı sayıları == ile karşılaştırma**
>
> `0.1 + 0.2 == 0.3` sonucu `false`'tur! İki `double`'ı karşılaştırırken aralarındaki farkın çok küçük olup olmadığına bak: `Math.Abs(a - b) < 1e-9`. Para gibi kesin hesap gereken yerlerde ise baştan **`decimal`** kullan.

Para hesabında `decimal` kullan ve değerlere `m` ekini yazmayı unutma. Sonucu kuruşa yuvarlamak için `Math.Round(x, 2)` yeterlidir:

para.cs

```
decimal fiyat = 19.99m;
int adet = 3;
decimal toplam = fiyat * adet;
decimal kdvli = Math.Round(toplam * 1.20m, 2);
Console.WriteLine($"Toplam: {toplam}");
Console.WriteLine($"KDV'li: {kdvli}");
```

```
Çıktı
Toplam: 59,97
KDV'li: 71,96
```

`decimal` ile `int` birlikte kullanılabilir (`int` otomatik `decimal` olur), ama `fiyat * 1.20` yazarsan `1.20` bir `double` olduğu için derleme hatası alırsın (`CS0019`). Her zaman `1.20m` yaz.

2.11 Bit operatörleri

Bilgisayar sayıları ikilik (binary) düzende, yani 0 ve 1'lerden oluşan bitlerle tutar. Bit operatörleri tam sayıların bitleri üzerinde **bit bit** işlem yapar. Sınavlarda sık sorulmaz ama bilmen gerekir:

- `a & b` (VE): İki bit de 1 ise 1.
- `a | b` (VEYA): En az biri 1 ise 1.
- `a ^ b` (özel VEYA, XOR): Bitler **farklıysa** 1.
- `~a` (DEĞİL): Bütün bitleri ters çevirir.
- `a << n` (sola kaydır): Bitleri `n` basamak sola iter; sayıyı 2^n ile çarpmak gibidir.
- `a >> n` (sağa kaydır): Bitleri `n` basamak sağa iter; sayıyı 2^n'e tam bölmek gibidir.

```
a      = 1100   (12)
b      = 1010   (10)
a & b  = 1000   (8)
a | b  = 1110   (14)
a ^ b  = 0110   (6)
```

bitOperatorleri.cs

```
int a = 0b1100;   // 12 (0b ile ikilik sayı yazılır)
int b = 0b1010;   // 10

Console.WriteLine($"a & b  = {a & b}");
Console.WriteLine($"a | b  = {a | b}");
Console.WriteLine($"a ^ b  = {a ^ b}");
Console.WriteLine($"~a     = {~a}");
Console.WriteLine($"a << 2 = {a << 2}");
Console.WriteLine($"a >> 1 = {a >> 1}");
Console.WriteLine($"2 ^ 3  = {2 ^ 3}");
Console.WriteLine($"13 & 1 = {13 & 1}");
```

Çıktı

```
a & b  = 8
a | b  = 14
a ^ b  = 6
~a     = -13
a << 2 = 48
a >> 1 = 6
2 ^ 3  = 1
13 & 1 = 1
```

`~a` her zaman `-a - 1` verir: `~12` → `-13`. Son satır, `x & 1` işleminin kullanımını gösterir: Bir sayının en sağdaki biti 1 ise sayı **tektir**, 0 ise **çifttir**. Yani `sayi & 1` ifadesi `sayi % 2` ile aynı sonucu verir (negatif olmayan sayılarda).

Sınavda Çıkar: `^` üs alma değildir

C#'ta üs alma operatörü yoktur. `2 ^ 3` ifadesi 8 değil, bitleri karşılaştıran XOR işlemi olduğu için `1` verir. Üs almak için `Math.Pow(2, 3)` yaz.

Sık Yapılan Hata: `&` ile `==` karıştırma

`&` operatörünün önceliği `==`'den **düşüktür**. `x & 1 == 0` yazarsan C# bunu `x & (1 == 0)` diye okur ve `int` ile `bool`'u `&`'lamaya çalıştığı için derlemez:

```
int x = 6;
Console.WriteLine(x & 1 == 0);
```

Derleyici çıktısı – program derlenmez

```
bitHata.cs(2,19): error CS0019: Operator '&' cannot be applied to operands of type 'int' and
'bool'
```

Doğrusu parantezle yazmaktır: `(x & 1) == 0`.

Bölüm Özeti – Bilmen Gerekenler

- Değişken = ad + tip + değer. Kullanmadan önce **tanımla**; yerel değişkeni değer vermeden okuyamazsın (`CS0165`).
- İsim rakamla başlayamaz, boşluk, `-`, `$` ve anahtar kelime içeremez (`@` öneki yalnızca bilgi). Değişken `camelCase`, sınıf ve metot `PascalCase`, `const` sabiti `PascalCase`.

- Temel tipler: `byte`, `short`, `int`, `long`, `float`, `double`, `decimal`, `char`, `bool`, `string`. Varsayılanlar: tam sayı → `int`, ondalık → `double`. Para → `decimal` (`m` eki). `float` için `f` eki. `int.MaxValue` ve `int.MinValue` sınırları verir.
- İşaretsiz tipler eksi sayı tutmaz: `byte` (0–255), `ushort`, `uint`, `ulong`; işaretli karşılığı `sbyte`'tır. `byte b = -1;` derlenmez (CS0031); `uint` ile `3 - 5` eksi çıkmaz, en büyük uca döner.
- `char` tek tırnak, `string` çift tırnak. `const` değişmez; `var` tipi çıkarır ama C# yine statik tiplidir.
- **`int / int` = `int`**: `7 / 2 = 3`, `9 / 10 = 0`. Ondalık sonuç için birini `double` yap.
- `%` kalan verir (işareti soldaki sayınınkiyle aynı): tek/çift, bölünebilme, son basamak (`n % 10`).
- Genişleyen dönüşüm otomatik, daralan dönüşüm cast ister: `(int)3.99 = 3` (kesme), `Convert.ToInt32(3.99) = 4` (yuvarlama). `int x = 3.5;` derlenmez (`CS0266`).
- `x++` önce kullanır sonra artırır; `++x` önce artırır sonra kullanır. `w = w++` hiçbir şey yapmaz.
- `b += 5` gizli cast içerir; `b = b + 5` (b `byte` ise) derlenmez.
- `Math.Round` tam ortada **çift sayıya** gider: `Math.Round(2.5) = 2`. Okul usulü için `MidpointRounding.AwayFromZero`.
- Tam sayı taşınca varsayılan olarak **hata vermez**, tersine döner; `checked` ile `OverflowException` alırsın. `double`'lar `==` ile karşılaştırılmaz; para için `decimal`.
- Bit operatörleri: `&`, `|`, `^`, `~`, `<<`, `>>`. `^` üs değildir; `x & 1` tek/çift verir.

Sorular

A) Çoktan Seçmeli

1 Aşağıdakilerden hangisi **geçerli** bir C# değişken adıdır?

A) `2ortalama` B) `_ortalama2` C) `ogrenci no` D) `class` E) `$fiyat`

2 `double sonuc = 15 / 4;` satırından sonra `Console.WriteLine(sonuc);` komutu ekrana ne yazar?

A) `3,75`
B) `4`
C) `3,0`
D) `3`
E) Derleme hatası verir

3 Aşağıdaki satırlardan hangisi **derlenmez**? (Her satır tek başına bir programdadır.)

A) `double d = 10;` B) `long l = 100;` C) `float f = 2.5;` D) `decimal m = 5;` E) `int i = 'A';`

4 `int a = 4; int b = a++ + ++a;` satırları çalıştıktan sonra `a` ve `b`'nin değerleri nedir?

A) `a = 6, b = 10` B) `a = 5, b = 9` C) `a = 6, b = 8` D) `a = 6, b = 9` E) `a = 5, b = 10`

5 `Console.WriteLine((int)7.9 + 17 % 5 * 2);` komutu ne yazdırır?

(A) `12` (B) `14` (C) `11,9` (D) `8` (E) `11`

B) Çıktı Tahmini

6 Aşağıdaki program ekrana ne yazdırır?

```
int a = 17, b = 5;
double c = a / b;
double d = (double)a / b;
Console.WriteLine($"{a / b} {a % b}");
Console.WriteLine($"{c} {d}");
Console.WriteLine(a / (double)b * 0);
Console.WriteLine(b / a + b % a);
```

7 Aşağıdaki program ekrana ne yazdırır?

```
int n = 3;
int p = n++ + n++;
int q = ++n * 2;
int r = n-- - --n;
Console.WriteLine($"{n} {p} {q} {r}");
n += n++ * 2;
Console.WriteLine(n);
```

8 Aşağıdaki program ekrana ne yazdırır?

```
char h = 'a';
h = (char)(h + 2);
Console.WriteLine(h);
Console.WriteLine(h + 1);
Console.WriteLine((char)(h - 32));
Console.WriteLine("" + h + 1);
Console.WriteLine(Math.Round(6.5) + Math.Abs(-2));
Console.WriteLine(Math.Round(7.5) + " " + Math.Round(8.5));
```

C) Kod Yazma

9 Santigrat derecesini Fahrenhayt'a çeviren `sicaklik` programını yaz. Formül: **F = C × 9 / 5 + 32**. Programda `double c = 37.5;` değişkeni olsun ve sonucu `37,5 C = 99,5 F` biçiminde yazdırsın. (İpucu: Formülde tam sayı bölmesine dikkat et!)

10 Saniye cinsinden verilen bir süreyi **saat, dakika ve saniyeye** ayıran `sure` programını yaz. `int toplamSaniye = 5025;` için çıktı `1 saat 23 dakika 45 saniye` olmalıdır. Sadece `/` ve `%` operatörlerini kullan.

Çözümler

Cevap anahtarı

1	2	3	4	5
B	D	C	A	E

1 Cevap: B

Değişken adı harf veya `_` ile başlayabilir; içinde rakam olabilir. `_ortalama2` bu kurallara uyar. `2ortalama` rakamla başlıyor, `ogrenci no` boşluk içeriyor, `class` anahtar kelime, `$fiyat` ise `$` içeriyor; C# adlarında `$` kullanılamaz.

2 Cevap: D

Sağ taraf önce hesaplanır: `15 / 4` iki tam sayının bölümüdür, sonuç `3` olur. Sonra `3`, `double`'a otomatik dönüşür. C# ondalığı sıfır olan `double`'ı `3,0` diye değil `3` diye yazar; bu yüzden C şıkkı yanlıştır. `3,75` elde etmek için `15 / 4.0` yazılmalıydı (A). Sonuç yuvarlanmaz, kesilir (B yanlış). `int` → `double` otomatik dönüşüm olduğu için kod derlenir (E yanlış).

3 Cevap: C

`2.5` bir `double` sabit değeridir; `double` → `float` daralan dönüşümdür ve otomatik yapılmaz (`CS0664`). `2.5f` yazılmalıydı. A, B ve D'de `int` sabiti daha geniş bir tipe (`double`, `long`, `decimal`) otomatik dönüşür. E de derlenir: `char` → `int` genişleyen dönüşümdür ve `i` değişkeni 65 olur.

4 Cevap: A

`a++`: Değer olarak **4** kullanılır, `a` = 5 olur. `++a`: `a` = 6 olur, değer olarak **6** kullanılır. `b = 4 + 6 = 10`, `a` = 6. D şıkkı, `++a` için `a`'nın zaten 5 olduğunu unutup `4 + 5 = 9` hesaplayanların cevabıdır. B ve E şıkları ise `a`'yı 5'te bırakır; oysa iki artırma işlemi de yapılmıştır.

5 Cevap: E

Öncelik sırası: Cast → `(int)7.9 = 7` (kesme, yuvarlama değil). Sonra `%` ve `*` soldan sağa: `17 % 5 = 2`, `2 * 2 = 4`. En son toplama: `7 + 4 = 11`. A şıkkı `7,9`'u `8`'e yuvarlayanların (`8 + 4`), C şıkkı cast'i hiç hesaba katmayanların (`7,9 + 4`), B şıkkı `17 % (5 * 2)` diye hesaplayanların (`7 + 7`), D şıkkı ise toplamayı öne alanların (`(7 + 17) % 5 * 2`) cevabıdır.

6

- `a / b` = `17 / 5` = 3, `a % b` = 2.
- `c = a / b` → önce tam sayı bölmesi (3), sonra `3,0`; ekrana `3` yazılır. `d` = `17,0 / 5 = 3,4`.
- `17 / 5,0 * 0` = `3,4 * 0` = `0` (sonuç `double`).
- `b / a` = `5 / 17` = 0, `b % a` = `5 % 17` = 5 → `0 + 5 = 5`.

Çıktı

```
3 2
3 3,4
0
5
```

7

Değişken tablosu ile izleyelim:

Satır	İşlem	n	p	q	r
`int n = 3;`		3			
`p = n++ + n++`	3 kullanılır, n = 4; 4 kullanılır, n = 5	5	7		
`q = ++n * 2`	n = 6 olur, 6 kullanılır	6	7	12	
`r = n-- - --n`	6 kullanılır, n = 5; n = 4 olur, 4 kullanılır	4	7	12	2

İlk `Console.WriteLine` bu yüzden `4 7 12 2` yazar. Son satır: `n += n++ * 2` ifadesi `n = n + (n++ * 2)` demektir ve soldaki `n`'nin değeri (**4**) en başta alınır. Sonra `n++ * 2` → `4 * 2 = 8` (n geçici olarak 5 olur), ve `n = 4 + 8 = 12` atanır.

Çıktı
```
4 7 12 2
12
```

8

- `h = (char)(h + 2)`: `'a'` (97) + 2 = 99 `int` olur; cast ile tekrar `char` yapılınca `'c'` elde edilir.
- `h + 1`: `char + int` → `int`: 100.
- `h - 32` = 67 = `'C'` (büyük harf).
- `"" + h + 1`: Önce `""` ile `string` olur → `"c"`, sonra `"c1"`.
- `Math.Round(6.5)`: Tam ortada çift sayıya gider → **6**, `Math.Abs(-2)` = 2, toplam 8.
- `Math.Round(7.5)` → 8, `Math.Round(8.5)` → **8** (çift sayı). Okul usulüyle `8 9` bekleyenler tuzağa düşer.

Çıktı
```
c
100
C
c1
8
8 8
```

9

`9 / 5` yazarsak tam sayı bölmesi `1` verir ve sonuç yanlış çıkar. `c * 9 / 5` biçiminde `c` ile başlamak (c zaten `double`) sorunu çözer; çünkü `c * 9` zaten `double`'dır.

sicaklik.cs
```
double c = 37.5;
double f = c * 9 / 5 + 32;
Console.WriteLine($"{c} C = {f} F");
```

Çıktı
```
37,5 C = 99,5 F
```

Çıktıda `37,5` ve `99,5` virgüllü görünür; kodda ise noktalı yazdık. Sayıları belirli basamakta yazdırmayı Bölüm 3'te öğreneceksin.

10

Bir saatte 3600 saniye var. Önce `/ 3600` ile saati bul, `% 3600` ile kalan saniyeyi al; kalanı `/ 60` ile dakikaya böl, `% 60` ile kalan saniyeyi bul.

sure.cs

```
int toplamSaniye = 5025;
int saat = toplamSaniye / 3600;
int kalan = toplamSaniye % 3600;
int dakika = kalan / 60;
int saniye = kalan % 60;
Console.WriteLine($"{saat} saat {dakika} dakika {saniye} saniye");
```

Çıktı

```
1 saat 23 dakika 45 saniye
```

BÖLÜM 3

Klavyeden Girdi ve Biçimli Çıktı

Bu Bölümde Öğreneceklerin

- `Console.ReadLine()` ile klavyeden satır okumayı ve `string?` tipinin ne demek olduğunu
- `??` ile boş girdiye karşı önlem almayı
- Okunan metni `Convert`, `int.Parse` ve `int.TryParse` ile sayıya çevirmeyi
- Bozuk girdide oluşan `FormatException` hatasını
- **Türkçe bilgisayarlarda `3.5` yazınca sayının 35 olması tuzağını**
- Çıktıyı `$"..."`, `{x:F2}`, `{x,8}` ve `string.Format` ile biçimlendirmeyi
- `@"..."` metinleri, `\n` gibi kaçış dizilerini ve Türkçe karakter sorununu

3.1 Console.ReadLine ile satır okuma

Şimdiye kadar yazdığımız programlarda bütün değerler koda gömülüydü. Kullanıcıdan veri almak için `Console.ReadLine()` metodunu kullanırız. Bu metot, kullanıcı bir şey yazıp **Enter**'a basana kadar bekler ve yazılan satırı **metin** olarak döndürür. Sayı bile yazılsa dönen şey metindir; sayıya çevirmeyi biraz sonra göreceğiz.

İpucu

Okumadan önce `Console.Write` ile kullanıcıya ne girmesi gerektiğini söyle (*prompt*). `WriteLine` yerine `Write` kullanırsan imleç aynı satırda kalır ve kullanıcı cevabını sorunun yanına yazar.

Derleyici uyarısı: string?

En basit okuma şöyle yazılır:

uyariOrnegi.cs

```
Console.Write("Adın: ");
string ad = Console.ReadLine();
Console.WriteLine($"Merhaba {ad}!");
```

```
Derleyici uyarısı (warning)
uyariOrnegi.cs(2,13): warning CS8600: Converting null literal or possible null value to non-nullable type.
```

Program çalışır ama derleyici uyarı verir. Sebep `ReadLine()` metodunun dönüş tipidir: `string` değil, **`string?`**. Soru işareti "bu değer **boş (null)** olabilir" demektir. `null`, "hiçbir değer yok" anlamına gelen özel bir durumdur.

`ReadLine()` ne zaman `null` döndürür? Girdi akışı kapandığında: kullanıcı Windows'ta **Ctrl+Z** ve Enter'a, macOS ve Linux'ta **Ctrl+D**'ye bastığında ya da girdi bir dosyadan geliyor ve dosya bittiğinde. Derleyici bu ihtimali görür ve `string` türündeki `ad` değişkenine bu değeri koymak istediğin için uyarır.

> **Sık Yapılan Hata: Uyarıyı görmezden gelme**
>
> Bu kitapta derleyici uyarıları hata sayılır; uyarısız program yazarız. Uyarıyı susturmak için `Console.ReadLine()!` yazmak da mümkündür ama bu derleyiciye "eminim, `null` gelmez" demektir. `null` gelirse program yine çöker. Biz uyarıyı gerçekten çözen kalıbı kullanacağız.

Boş gelirse boş metin: ?? işleci

`??` işleci iki değer arasında seçim yapar: **soldaki değer `null` değilse onu, `null` ise sağdakini verir.**

bosBirlesme.cs

```
string? ham = null;
string ad = ham ?? "Bilinmiyor";
Console.WriteLine(ad);

string? ham2 = "Deniz";
string ad2 = ham2 ?? "Bilinmiyor";
Console.WriteLine(ad2);
```

Çıktı

```
Bilinmiyor
Deniz
```

`ham` boş olduğu için sağdaki `"Bilinmiyor"` seçildi; `ham2` doluydu, kendi değeri korundu. `ReadLine()` ile birlikte sağ tarafa boş metin (`""`) koyarsak, kitap boyunca kullanacağımız kalıp ortaya çıkar:

merhabaAd.cs

```
Console.Write("Adın: ");
string ad = Console.ReadLine() ?? "";
Console.WriteLine($"Merhaba {ad}!");
```

Çıktı

```
Adın: Ayşe
Merhaba Ayşe!
```

"Boş gelirse boş metin al" demek bu satırın özeti. Artık `ad` değişkeni `string` olarak kalır ve uyarı yoktur. `$"..."` içindeki `{ad}`, değişkenin değeriyle yer değiştirir; ayrıntısını bu bölümün ikinci yarısında göreceksin.

> **Sınavda Çıkar**
>
> `??` yalnızca `null` için çalışır. Kullanıcı hiçbir şey yazmadan Enter'a basarsa `ReadLine()` `null` değil, **boş metin** (`""`) döndürür. Bu durumda `??` sağdaki değere geçmez.

Satırın tamamı okunur

`ReadLine()` Enter'a kadar yazılan her şeyi, **boşluklar dahil** tek parça olarak alır:

satirOkuma.cs

```
Console.Write("Adın soyadın: ");
string ad = Console.ReadLine() ?? "";
Console.Write("Şehrin: ");
string sehir = Console.ReadLine() ?? "";

Console.WriteLine($"Ad: [{ad}]");
Console.WriteLine($"Şehir: [{sehir}]");
Console.WriteLine($"Ad uzunluğu: {ad.Length}");
```

Çıktı

```
Adın soyadın: Ayşe Nur Kaya
Şehrin: Eskişehir
Ad: [Ayşe Nur Kaya]
Şehir: [Eskişehir]
Ad uzunluğu: 13
```

Köşeli parantezleri, okunan metnin tam olarak nerede başlayıp bittiğini görmek için ekledik. `ad.Length` metindeki karakter sayısını verir (Bölüm 8'de ayrıntısı var); boşluklar da sayıldı.

Bir satırdaki birden çok kelimeyi ayrı değişkenlere bölmek (`"12 5"` → `12` ve `5`) için metni parçalamak gerekir; bunu Bölüm 8'de öğreneceksin. Şimdilik her değeri **ayrı satırda** isteyeceğiz. Tek karakter okumak için de satırı okuyup ilk karakterini alırsın: `char c = metin[0];` (boş metinde bu hata verir; indeksleri Bölüm 8'de göreceksin).

3.2 Sayı okuma

Convert ve Parse

`ReadLine()` her zaman metin verir. Sayıyla işlem yapmak için metni sayıya çevirmek gerekir. En kısa yol `Convert` sınıfıdır:

toplayici.cs

```
Console.Write("Birinci sayı: ");
int a = Convert.ToInt32(Console.ReadLine());
Console.Write("İkinci sayı: ");
int b = Convert.ToInt32(Console.ReadLine());

Console.WriteLine($"Toplam = {a + b}");
```

Çıktı

```
Birinci sayı: 12
İkinci sayı: 30
Toplam = 42
```

`Convert.ToInt32` metin alıp `int` döndürür. Bu metot `null` kabul ettiği için `ReadLine()` ile doğrudan kullanılabilir ve uyarı vermez. Diğer tipler için de benzer metotlar vardır:

Metot	Döndürdüğü tip	Örnek metin
`Convert.ToInt32(metin)`	`int`	`"42"`
`Convert.ToInt64(metin)`	`long`	`"9000000000"`
`Convert.ToDouble(metin)`	`double`	`"3,5"`
`Convert.ToDecimal(metin)`	`decimal`	`"19,90"`
`Convert.ToBoolean(metin)`	`bool`	`"true"`

Aynı işi `int.Parse(...)` ve `double.Parse(...)` da yapar. Bunlar metin ister ve `null` kabul etmez; bu yüzden `ReadLine()` sonucuna `??` eklenir:

parseOrnek.cs

```
Console.Write("Yaş: ");
string yasMetni = Console.ReadLine() ?? "";
int yas = int.Parse(yasMetni);

Console.Write("Boy (m): ");
double boy = double.Parse(Console.ReadLine() ?? "");

Console.WriteLine($"{yas} yaşında, {boy} m");
```

Çıktı

```
Yaş: 19
Boy (m): 1,68
19 yaşında, 1,68 m
```

Girdi satırındaki `1,68`'e dikkat et: ondalık sayı **virgülle** girildi. Bunun sebebini kısa süre sonra göreceksin.

> **Sınavda Çıkar**
>
> `Convert.ToInt32` ile `int.Parse` benzer işi yapar ama `null` karşısında farklıdır: `Convert.ToInt32((string?)null)` sessizce `0` döndürür, `int.Parse(null)` ise `ArgumentNullException` fırlatır. Boş metin (`""`) ikisinde de `FormatException` verir.

Sayıdan sonra metin okumak sorun çıkarmaz; her `ReadLine()` çağrısı ayrı bir satır okur:

sayiSonraMetin.cs

```
Console.Write("Yaşın: ");
int yas = Convert.ToInt32(Console.ReadLine());
Console.Write("Adın soyadın: ");
string ad = Console.ReadLine() ?? "";

Console.WriteLine($"Ad: [{ad}], yaş: {yas}");
```

Çıktı

```
Yaşın: 19
Adın soyadın: Zeynep Ak
Ad: [Zeynep Ak], yaş: 19
```

> **Not: Satır sonu tuzağı yok**
>
> Bazı okuma yöntemleri sayıyı okuyup Enter'ı tamponda bırakır; ardından gelen satır okuması boş döner. `ReadLine()` her seferinde satırın tamamını tüketir, bu yüzden bu tuzak C# kitabında yok. Kural basit: **satırı oku, gerekiyorsa çevir.**

Sığmayan büyük sayı da hata verir: `int` sınırlarının dışındaki bir değer (`3000000000` gibi) `OverflowException` fırlatır.

Yanlış girdi: FormatException

Program sayı beklerken kullanıcı başka bir şey yazarsa `Convert.ToInt32` bir `FormatException` fırlatır ve program çöker:

yanlisGirdi.cs

```
Console.Write("Yaşın: ");
int yas = Convert.ToInt32(Console.ReadLine());
Console.WriteLine($"Gelecek yıl {yas + 1} olacaksın.");
```

Çıktı – program yakalanmamış istisna ile sonlanır

```
Yaşın: yirmi
Unhandled exception. System.FormatException: The input string 'yirmi' was not in a correct format.
   at Program.<Main>$(String[] args) in yanlisGirdi.cs:line 2
```

Çıktıdaki ilk satır hatanın türünü (`System.FormatException`) ve nedenini söyler. İngilizce olan mesajdan çok **sınıfın adını** tanımak önemlidir. `at ... in yanlisGirdi.cs:line 2` satırları hatanın programın hangi satırında oluştuğunu gösterir. Çalışma sırasında oluşan bu tür hatalara istisna (exception) denir; yakalamayı Bölüm 12'de öğreneceksin.

> **Sık Yapılan Hata: Ondalıklı metni tam sayıya çevirmek**
>
> `Convert.ToInt32("3,5")` ve `int.Parse("3,5")` **hata verir**; `"3,5"` bir tam sayı değildir. Kullanıcı ondalıklı sayı girebilecekse önce `Convert.ToDouble` ile oku, sonra tam sayıya çevir. Boş Enter da (`""`) aynı hatayı üretir.

Güvenli yol: TryParse

Çökmek yerine "çevrilebildi mi?" diye sorabiliriz. `int.TryParse` çeviriyi dener; **başarılı olursa `true`, olmazsa `false` döndürür.** Sonucu ise `out` ile belirtilen değişkene koyar:

tryParseTemel.cs

```
bool aTamam = int.TryParse("42", out int a);
bool bTamam = int.TryParse("4x2", out int b);

Console.WriteLine($"42  -> {aTamam}, a = {a}");
Console.WriteLine($"4x2 -> {bTamam}, b = {b}");
```

Çıktı

```
42  -> True, a = 42
4x2 -> False, b = 0
```

`out int a` yazımı `a` değişkenini orada tanımlar ve çeviri sonucu ona yazılır. Çeviri başarısızsa değişken `0` olur. `out`'un ayrıntısını Bölüm 6'da göreceksin; şimdilik kalıbı ezberle. Kullanıcı girdisiyle şöyle kullanılır:

tryParseGiris.cs

```
Console.Write("Yaşın: ");
string metin = Console.ReadLine() ?? "";

if (int.TryParse(metin, out int yas))
{
    Console.WriteLine($"Gelecek yıl {yas + 1} yaşında olacaksın.");
}
else
{
    Console.WriteLine("Geçerli bir sayı girmedin.");
}
```

Çıktı

```
Yaşın: yirmi
Geçerli bir sayı girmedin.
```

`if` ilk bloğu çeviri başarılıysa, `else` bloğunu değilse çalıştırır; koşulları Bölüm 4'te ayrıntılı işleyeceğiz. Girdi `20` olsaydı program `Gelecek yıl 21 yaşında olacaksın.` yazardı. Ondalık sayılar için `double.TryParse(metin, out double x)` kalıbı aynıdır.

> **Sınavda Çıkar**
>
> `TryParse` hiçbir zaman istisna fırlatmaz. Başarısız olursa `false` döner ve `out` değişkeni sıfır değerini (`0`) alır. `Parse` ve `Convert` ise bozuk metinde `FormatException` fırlatır.

3.3 Türkçe bilgisayarlarda ondalık virgül

`Convert.ToDouble`, `double.Parse`, `Console.WriteLine` ve `$"..."` sayıları çevirirken ve yazarken bilgisayarın **bölge ayarını**, yani kültürünü (culture) kullanır. Türkçe bir bilgisayarda kültür `tr-TR`'dir ve bu kültürde ondalık ayırıcı **virgül**, binlik ayırıcı **nokta**dır:

kulturBilgi.cs

```
using System.Globalization;

Console.WriteLine(CultureInfo.CurrentCulture.Name);

double x = 1234.5;
Console.WriteLine(x);
Console.WriteLine($"{x:N2}");
Console.WriteLine(3.14);
```

Çıktı

```
tr-TR
1234,5
1.234,50
3,14
```

`using System.Globalization;` satırı dosyanın **en başına** yazılır; `CultureInfo` bu alan adındadır. Koda yazdığın sabitler her zaman noktalıdır (`3.14`), çünkü kod kültürden bağımsızdır. Ama **ekrana yazılan** sayı kültüre göre biçimlenir: `3.14` sabiti `3,14` olarak görünür.

	Türkçe (`tr-TR`)	İngilizce (`en-US`)
Ondalık ayırıcı	`,`	`.`
Binlik ayırıcı	`.`	`,`

3.5 yazınca 35 çıkması

Sorun okuma tarafında. Klavyeden **nokta** ile `3.5` yazarsan Türkçe kültür bunu "binlik ayırıcılı bir sayı" sanar:

kulturTuzagi.cs

```
Console.Write("Sayı: ");
double x = Convert.ToDouble(Console.ReadLine());
Console.WriteLine(x);
Console.WriteLine(x * 2);
```

Çıktı

```
Sayı: 3.5
35
70
```

Hata mesajı yok; program sessizce **35** okudu. `tr-TR`'de nokta binlik ayırıcı olduğu için `3.5` metni "3 bin 5" gibi yorumlandı ve nokta yok sayıldı. Daha kötüsü: `1.234` yazan kullanıcı `1,234` değil, **`1234`** okunur. Doğru girdi virgüllüdür:

kulturDogru.cs

```
Console.Write("Sayı: ");
double x = Convert.ToDouble(Console.ReadLine());
Console.WriteLine(x);
Console.WriteLine(x * 2);
```

Çıktı

```
Sayı: 3,5
3,5
7
```

Bu yüzden bu kitaptaki programlarda ondalık girdiler **virgülle** verilir ve çıktıdaki ondalık sayılar da virgüllüdür.

> **Sık Yapılan Hata: Nokta ile girilen ondalık sayı**
>
> Türkçe kültürde `Convert.ToDouble("3.5")` hata vermez, **35** döndürür. Kullanıcı `3.5` yazınca programın sessizce yanlış hesaplaması en tehlikeli durumdur. Test ederken ondalık girdiyi virgülle ver; programı herkes için güvenli yapmak istiyorsan bir sonraki alt başlıktaki yöntemi kullan.

Sabit kültür: InvariantCulture

Programın her bilgisayarda aynı davranmasını istiyorsan kültürü kendin seçersin. `CultureInfo.InvariantCulture`, bölgeden bağımsız bir kültürdür: ondalık ayırıcı **nokta**, binlik ayırıcı **virgüldür**.

degismezKultur.cs

```
using System.Globalization;

Console.Write("Sayı (nokta ile): ");
string metin = Console.ReadLine() ?? "";
double x = double.Parse(metin, CultureInfo.InvariantCulture);

Console.WriteLine(x);
Console.WriteLine(x.ToString(CultureInfo.InvariantCulture));
Console.WriteLine(x.ToString("F2", CultureInfo.InvariantCulture));
string bicimli = string.Format(CultureInfo.InvariantCulture, "{0:N2}", 1234567.891);
Console.WriteLine(bicimli);
```

Çıktı

```
Sayı (nokta ile): 3.5
3,5
3.5
3.50
1,234,567.89
```

`double.Parse` metni nokta kuralıyla okudu (`3.5` gerçekten 3,5 oldu). Bir değeri ekrana yazarken de `ToString(CultureInfo.InvariantCulture)` ya da `string.Format(CultureInfo.InvariantCulture, ...)` ile nokta kuralını zorlayabilirsin. İlk `WriteLine` ise kültür belirtmediği için yine virgül yazdı. `Convert.ToDouble(metin, CultureInfo.InvariantCulture)` biçimi de vardır.

> **İpucu**
>
> Kitaptaki çıktılar Türkçe bir Windows'ta göreceğin gibidir: ondalık virgül, binlik nokta. Senin bilgisayarının dili İngilizceyse çıktıların noktalı görünür; program yanlış değildir, sadece kültürün farklıdır. Sınavda genellikle Türkçe kültür varsayılır.

3.4 Biçimli çıktı

Dize enterpolasyonu

Çıktıyı değerlerle birleştirmenin en okunaklı yolu `$"..."` ile başlayan **enterpolasyonlu metindir** (string interpolation). Metnin içinde süslü parantezlere yazdığın ifade hesaplanır ve sonucu yerine konur:

interpolasyon.cs

```
string ad = "Deniz";
int yas = 20;
double ort = 3.456789;

Console.WriteLine($"{ad} {yas} yaşında.");
Console.WriteLine($"Ortalama: {ort:F2}");
Console.WriteLine($"{ad}'in ortalaması {ort:F1}, yaşı {yas}");
Console.WriteLine($"Gelecek yıl {yas + 1} yaşında olacak.");
Console.WriteLine($"Süslü parantez: {{ad}} = {ad}");
Console.WriteLine("Ad: {ad}");
Console.WriteLine($"Ad: {ad}");
```

```
Çıktı
Deniz 20 yaşında.
Ortalama: 3,46
Deniz'in ortalaması 3,5, yaşı 20
Gelecek yıl 21 yaşında olacak.
Süslü parantez: {ad} = Deniz
Ad: {ad}
Ad: Deniz
```

- `{ad}` ve `{yas + 1}` → süslü parantezin içine yalnızca değişken değil, her ifade yazılabilir.
- `{ort:F2}` → iki nokta üst üsteden sonra **biçim belirteci** gelir.
- Süslü parantez karakterinin kendisini yazdırmak için onu **iki kez** yaz (koddaki "Süslü parantez" satırı).
- Sondan bir önceki satırda `$` yok; bu yüzden metin olduğu gibi yazıldı.

> **Sık Yapılan Hata: $ işaretini unutmak**
>
> `"Ad: {ad}"` yazarsan hata almazsın, derleyici de uyarmaz; ekranda kelimesi kelimesine `Ad: {ad}` görünür. Süslü parantezli bir metin yazdırıyorsan başındaki `$`'ı kontrol et.

Biçim belirteçleri

İki nokta üst üsteden sonra yazılan harf, değerin **nasıl** yazılacağını belirler:

bicimler.cs

```
double pi = 3.14159;
int buyuk = 1234567;

Console.WriteLine($"F2   -> {pi:F2}");
Console.WriteLine($"F0   -> {pi:F0}");
Console.WriteLine($"0.00 -> {pi:0.00}");
Console.WriteLine($"N0   -> {buyuk:N0}");
Console.WriteLine($"N2   -> {buyuk:N2}");
Console.WriteLine($"D8   -> {42:D8}");
Console.WriteLine($"P0   -> {0.256:P0}");
```

```
Çıktı
F2   -> 3,14
F0   -> 3
0.00 -> 3,14
N0   -> 1.234.567
N2   -> 1.234.567,00
D8   -> 00000042
P0   -> %26
```

Belirteç	Anlamı
`F2`	Sabit noktalı sayı, 2 ondalık basamak (**yuvarlar**)
`F0`	Ondalık basamak yok
`0.00`	Özel biçim: en az bir tam, tam iki ondalık basamak
`N0`	**Binlik ayırıcılı** sayı, ondalık yok
`N2`	Binlik ayırıcılı, 2 ondalık
`D8`	Tam sayıyı en az 8 basamak yapar, başa sıfır koyar
`P0`	Yüzde: değeri 100 ile çarpıp yüzde işareti ekler

`N0` ile `1234567` sayısı Türkçe kültürde `1.234.567` yazıldı (binlik ayırıcı nokta). `F2` ise virgülden sonra iki basamak bırakır ve virgül kullanır.

> **İpucu**
>
> Para birimi için `C` belirteci de vardır (`{x:C}`) ama simge ve yazı tipi sorunları çıkarabildiği için bu kitapta kullanmıyoruz. Bunun yerine `{x:N2} TL` yaz.

Biçimlendirme değeri **yuvarlar**, tür dönüşümü ise **keser**. Aynı sayıya üç farklı yaklaşım:

yuvarlama.cs

```
double x = 2.678;
Console.WriteLine($"{x:F2}");
Console.WriteLine((int)x);
Console.WriteLine(Math.Round(x));
Console.WriteLine(Convert.ToInt32(x));
```

Çıktı

```
2,68
2
3
3
```

> **Sınavda Çıkar: F2 yuvarlar, (int) keser**
>
> `{2.678:F2}` → `2,68` (yuvarlama), `(int)2.678` → `2` (kesme), `Math.Round(2.678)` → `3`. `Convert.ToInt32(double)` de yuvarlar, ama tam yarım değerlerde **çifte doğru** yuvarlar: `Convert.ToInt32(6.5)` → `6`, `Convert.ToInt32(7.5)` → `8`. Bu dörtlü sık sık aynı soruda karşılaştırılır.

Genişlik ve hizalama

Süslü parantezde değerden sonra **virgül ve bir sayı** yazarsan, değer **en az o kadar karakterlik** bir alana yerleştirilir. Varsayılan olarak **sağa** yaslanır; sayıyı eksi yaparsan **sola** yaslanır. Biçim belirteci genişlikten **sonra** gelir: `{değer,genişlik:biçim}`. Tablo yazdırmak için idealdir:

tablo.cs

```
Console.WriteLine($"{"Ürün",-10}{"Adet",6}{"Fiyat",8}");
Console.WriteLine($"{"Kalem",-10}{3,6}{12.5,8:F2}");
Console.WriteLine($"{"Defter",-10}{12,6}{45.0,8:F2}");
Console.WriteLine($"{"Silgi",-10}{125,6}{7.25,8:F2}");
Console.WriteLine("------------------------");
Console.WriteLine($"[{42,5}] [{42,-5}] [{42:D5}]");
Console.WriteLine($"[{1234567:N0}] [{Math.PI,8:F3}]");
```

Çıktı

```
Ürün        Adet   Fiyat
Kalem          3   12,50
Defter        12   45,00
Silgi        125    7,25
------------------------
[   42] [42   ] [00042]
[1.234.567] [   3,142]
```

Yazım	Anlamı	Değer 42 için
`{x,5}`	5 karakterlik alan, sağa yasla	`42`
`{x,-5}`	5 karakterlik alan, sola yasla	`42`
`{x:D5}`	Sıfırla doldur	`00042`
`{x,8:F3}`	8 karakterlik alan ve 3 ondalık	ondalıklı değerlerde

Metinleri de aynı şekilde hizalarsın (`{"Ürün",-10}`). Metnin kendisi süslü parantez içinde olduğu için tırnakları da yazman gerekir.

Birleşik biçimlendirme: {0} ve string.Format

`$` işareti gelmeden önce kullanılan eski yazım, hâlâ çok yaygındır ve sınavda sorulur. Değerleri sonda sıralarsın, metnin içinde **sıra numarasıyla** çağırırsın (numaralar **0'dan** başlar):

birlesikBicim.cs

```
string urun = "Defter";
int adet = 3;
double fiyat = 45.5;

Console.WriteLine("{0} x {1} = {2:F2} TL", urun, adet, adet * fiyat);
Console.WriteLine("{0} + {0} = {1}", 5, 10);

string satir = string.Format("{0,-8}|{1,4}", urun, adet);
Console.WriteLine(satir);
Console.WriteLine(satir.Length);
```

Çıktı

```
Defter x 3 = 136,50 TL
5 + 5 = 10
Defter  |   3
13
```

`Console.WriteLine("{0} {1}", a, b)` ilk parametresini biçim metni sayar; `{0}` birinci değeri, `{1}` ikinci değeri temsil eder. Aynı numarayı birden çok yerde kullanabilirsin. Genişlik ve belirteç yine aynı biçimdedir: `{2:F2}`, `{0,-8}`.

`string.Format` aynı kuralları kullanır ama sonucu ekrana yazmak yerine **bir `string` olarak döndürür**. Yukarıdaki `satir` değişkeni bunun örneğidir: biçimlendirilmiş metni bir değişkende saklamak ya da başka bir metinle birleştirmek istediğinde kullanılır.

> **Sık Yapılan Hata: Değer sayısı ile {n} sayısının uyuşmaması**
>
> `Console.WriteLine("{0} {1}", 5)` derlenir ama çalışırken `FormatException` verir: metin iki değer ister, sen bir tane verdin. Numaralar 0'dan başladığı için `{1}` yazıp yalnızca bir değer vermek de aynı hatayı üretir. Enterpolasyonda (`$"..."`) bu sorun yoktur, çünkü değerler zaten metnin içindedir.

3.5 Kaçış dizileri ve @"..." metinler

Normal bir metinde ters eğik çizgi (`\`) özel anlam taşır. Bu iki karakterlik dizilere **kaçış dizisi** (escape sequence) denir:

Dizi	Anlamı
`\n`	Alt satıra geç
`\t`	Sekme (tab) boşluğu
`\\`	Ters eğik çizginin kendisi
`\"`	Çift tırnağın kendisi

Dosya yolları çok sayıda `\` içerdiği için bunları ikişer kez yazmak yorucu olur. Metnin önüne `@` koyarsan metin **düz (verbatim)** olur: `\` artık kaçış başlatmaz. Düz metinde çift tırnağı yazmak için tırnağı iki kez yazarsın:

kacis.cs

```
Console.WriteLine("Satır 1\nSatır 2");
Console.WriteLine("Ad:\tDeniz");
Console.WriteLine("Yol: C:\\Users\\deniz");
Console.WriteLine(@"Yol: C:\Users\deniz");
Console.WriteLine("Dedi ki: \"merhaba\"");
Console.WriteLine(@"Dedi ki: ""merhaba""");

string kullanici = "deniz";
Console.WriteLine($@"Yol: C:\Users\{kullanici}\Belgeler");

string mesaj = @"Birinci satır
İkinci satır";
Console.WriteLine(mesaj);
```

Çıktı

```
Satır 1
Satır 2
Ad:     Deniz
Yol: C:\Users\deniz
Yol: C:\Users\deniz
Dedi ki: "merhaba"
Dedi ki: "merhaba"
Yol: C:\Users\deniz\Belgeler
Birinci satır
İkinci satır
```

- `@"..."` içinde satır sonuna basmak, metne gerçek bir satır sonu ekler; `\n` gerekmez.
- `$@"..."` (ya da `@$"..."`) hem düz metin hem enterpolasyondur.

> **Sınavda Çıkar**
>
> `"C:\temp\yeni"` yazarsan `\t` sekme olur, `\y` geçerli bir kaçış dizisi olmadığı için program derlenmez. Yol yazarken `@"C:\temp\yeni"` ya da `"C:\\temp\\yeni"` kullan.

3.6 Türkçe karakterler konsolda

C# kaynak dosyaları UTF-8 saklar ve `ç, ğ, ı, ö, ş, ü` gibi harfler metin içinde serbestçe yazılabilir. Çıktı tarafında genellikle sorun çıkmaz: Visual Studio'nun konsolu, Windows Terminal ve VS Code terminali Türkçe karakterleri doğru gösterir. Eski bir Komut İstemi penceresinde ya da farklı bir kod sayfasıyla çalışan konsolda `ş`, `ğ`, `İ` yerine `?` ya da anlamsız simgeler görürsen, programın **ilk satırına** çıktı kodlamasını UTF-8 yap:

turkceKarakter.cs

```
Console.OutputEncoding = System.Text.Encoding.UTF8;

Console.WriteLine("ç ğ ı i ö ş ü");
Console.WriteLine("Ç Ğ I İ Ö Ş Ü");
Console.WriteLine("Şişli'de çığ düştü, İğdır'a gidelim.");
```

Çıktı

```
ç ğ ı i ö ş ü
Ç Ğ I İ Ö Ş Ü
Şişli'de çığ düştü, İğdır'a gidelim.
```

> **İpucu**
>
> Kitaptaki programlarda bu satırı yazmıyoruz; modern ortamlarda gerekmez. Konsolunda Türkçe karakterler bozuk görünürse en üste ekle. Tek bir harf bozuksa önce terminali (Windows Terminal) değiştirmeyi dene.

3.7 Örnek uygulama: Vücut kitle indeksi

Öğrendiklerimizi birleştirelim. Kullanıcıdan adını, kilosunu ve boyunu alıp **vücut kitle indeksini** (VKİ = kilo / boy²) hesaplayan bir program. Ondalık girdilerin virgülle verildiğine dikkat et:

vki.cs

```
Console.Write("Adın: ");
string ad = Console.ReadLine() ?? "";
Console.Write("Kilon (kg): ");
double kilo = Convert.ToDouble(Console.ReadLine());
Console.Write("Boyun (m): ");
double boy = Convert.ToDouble(Console.ReadLine());

double vki = kilo / (boy * boy);
Console.WriteLine($"{ad}, VKİ değerin: {vki:F1}");
Console.WriteLine($"Boy: {boy,5:F2} m  Kilo: {kilo,6:F1} kg");
```

Çıktı

```
Adın: Mert Can
Kilon (kg): 72,5
Boyun (m): 1,80
Mert Can, VKİ değerin: 22,4
Boy:  1,80 m  Kilo:   72,5 kg
```

Sonucu “zayıf / normal / kilolu” diye yorumlamak için **koşul** gerekir; bu bir sonraki bölümün konusu.

Bölüm Özeti – Bilmen Gerekenler

- Klavyeden satır okumak için `Console.ReadLine()`; dönüş tipi **`string?`**, yani `null` olabilir.
- Uyarısız kalıp: `string ad = Console.ReadLine() ?? "";` (`??`: soldaki `null` ise sağdakini al).
- Sayı için: `Convert.ToInt32(Console.ReadLine())`, `Convert.ToDouble(Console.ReadLine())`; ya da `int.Parse(metin)`, `double.Parse(metin)`.
- Bozuk metin → `FormatException`. `int.TryParse(metin, out int sayi)` çökmez: başarılıysa `true`, değilse `false` ve `sayi` `0` olur.
- Satırı oku, gerekiyorsa çevir; sayıdan sonra metin okumak sorunsuzdur.
- Türkçe kültürde ondalık ayırıcı **virgül**, binlik nokta: `Convert.ToDouble("3.5")` → **35**; girdi `3,5` olmalı, çıktı `3,5` yazılır.
- Sabit kültür: `using System.Globalization;` ve `CultureInfo.InvariantCulture`.
- Enterpolasyon: `$"{x}"`, `{x:F2}` (yuvarlar), `{x:N0}` (binlik ayırıcı), `{x:D5}`, `{x:P0}`; `{x,8}` sağa, `{x,-8}` sola yaslar; süslü parantezin kendisi için parantez ikişer kez yazılır.
- Birleşik biçim: `Console.WriteLine("{0} {1}", a, b)`; `string.Format(...)` aynı işi yapıp `string` döndürür.
- `\n`, `\t`, `\\`, `\"` kaçış dizileridir; `@"C:\Users"` düz metindir (`""` ile tırnak yazılır).
- Para için `:C` yerine `{x:N2} TL`; Türkçe karakterler bozuksa `Console.OutputEncoding = System.Text.Encoding.UTF8;`.

Sorular

A) Çoktan Seçmeli

1 `Console.ReadLine()` metodunun dönüş türü nedir?

(A) `int` (B) `string[]` (C) `string?` (D) `char` (E) `object`

2 Türkçe (`tr-TR`) kültürlü bir bilgisayarda aşağıdaki kod çalıştırılırsa ekrana ne yazılır?

```
double x = Convert.ToDouble("3.5");
Console.WriteLine(x);
```

(A) `3,5`
(B) `3.5`
(C) `FormatException` oluşur, program çöker.
(D) `35`
(E) `0`

3 Aşağıdaki kod çalıştırılınca ekrana ne yazılır?

```
bool tamam = int.TryParse("12a", out int sayi);
Console.WriteLine($"{tamam} {sayi}");
```

(A) `False 0`
(B) `True 12`
(C) `False 12`
(D) `FormatException` oluşur, program çöker.
(E) Program derlenmez.

4 Aşağıdaki komutun Türkçe kültürdeki çıktısı nedir? (Boşluklar `␣` ile gösterilmiştir.)

```
Console.WriteLine($"[{3.14159,8:F2}]");
```

(A) `[3,14]` (B) `[3.14␣␣␣␣]` (C) `[␣␣␣␣3.14]` (D) `[3,14␣␣␣␣]` (E) `[␣␣␣␣3,14]`

5 Türkçe kültürlü bir bilgisayarda aşağıdaki ifadelerden hangisi `FormatException` fırlatır?

(A) `Convert.ToInt32("0042")`
(B) `int.Parse("3,5")`
(C) `Convert.ToDouble("3,5")`
(D) `int.Parse(" 7 ")`
(E) `int.TryParse("abc", out int k)`

B) Çıktı Tahmini

6 Aşağıdaki program ekrana ne yazdırır? (Satırlardaki boşluk sayısına dikkat et. Kültür: Türkçe.)

```
Console.WriteLine($"|{"ab",5}|{"cd",-5}|");
Console.WriteLine($"|{7,4}|{7,-4}|{7:D4}|");
Console.WriteLine($"|{2.71828:F3}|{2.6:F0}|{-3.14159,7:F2}|");
Console.WriteLine("{0}-{1}-{2}", 10 + 5, 'x', 3 > 2);
```

7 Kullanıcı klavyeden aşağıdaki girdiyi satır satır yazarsa program ne yazdırır?

```
12
5
Ahmet Yılmaz
```

```
int a = Convert.ToInt32(Console.ReadLine());
int b = Convert.ToInt32(Console.ReadLine());
string ad = Console.ReadLine() ?? "";

Console.WriteLine($"{a / b}|{a % b}|{ad}|");
Console.WriteLine(a % b * 2.0);
Console.WriteLine($"{(double)a / b:F2}");
```

8 Aşağıdaki program ekrana ne yazdırır? (Kültür: Türkçe.)

```
int ogr = 1234;
double ort = 72.456;
string s = string.Format("No:{0:D6} Ort:{1:F1}", ogr, ort);

Console.WriteLine(s);
Console.WriteLine(s.Length);
Console.WriteLine($"{2500000:N0} TL");
Console.WriteLine($"{7 * 100 / 8}%");
Console.WriteLine($"{Convert.ToInt32(6.5)} {Convert.ToInt32(7.5)} {(int)7.5}");
```

C) Kod Yazma

9 Kullanıcıdan iki ondalıklı sayı okuyan ve bu sayıların toplamını, farkını, çarpımını ve bölümünü **virgülden sonra 2 basamakla** aşağıdaki gibi yazdıran `dortIslem` programını yaz. Ondalık girdiler Türkçe kültüre uygun, virgüllü verilecek.

```
Birinci sayı: 7,5
İkinci sayı: 2
7,50 + 2,00 = 9,50
7,50 - 2,00 = 5,50
7,50 * 2,00 = 15,00
7,50 / 2,00 = 3,75
```

10 Kullanıcıdan önce **öğrenci numarasını** (tam sayı), sonra **ad soyadını** (boşluk içerebilir), sonra **vize** ve **final** notlarını (tam sayı) okuyan `notHesap` programını yaz. Ortalama = vize × 0.4 + final × 0.6. Çıktı örneği:

```
Öğrenci no: 2026104
Ad soyad: Elif Su Demir
Vize: 65
Final: 78
2026104 - Elif Su Demir - Ortalama: 72,80
```

Çözümler

Cevap anahtarı

1 C	2 D	3 A	4 E	5 B

1 Cevap: C

`ReadLine()` satırı `string` olarak verir ama girdi akışı bitmişse okunacak bir şey kalmaz ve sonuç `null` olur. Bu ihtimal yüzünden dönüş türü `string?` yazılır. A yanlış: sayı döndürmez, sayıya sen çevirirsin. B yanlış: tek satır döner, dizi değil. D yanlış: tek karakter değil, satırın tamamı gelir. E yanlış: dönüş türü `object` değil, `string?`'tir.

2 Cevap: D

Türkçe kültürde nokta **binlik ayırıcıdır**. `Convert.ToDouble("3.5")` hata vermez; noktayı yok sayıp `35` okur.

```
Çıktı
35
```

A yanlış: `3,5` yazdırmak için metin `"3,5"` olmalıydı. B yanlış: `3.5` yalnızca nokta ondalık ayırıcı olan kültürlerde (örneğin `en-US`) gelirdi. C yanlış: nokta binlik ayırıcı olarak kabul edildiği için hata oluşmaz. E yanlış: sonuç `0` değil.

3 Cevap: A

`TryParse`, çeviri başarısızsa `false` döndürür ve `out` değişkenine `0` yazar. `"12a"` geçerli bir sayı olmadığı için `tamam` `false`, `sayi` ise `0` olur. Bool değerler `True` ve `False` biçiminde, baş harfi büyük yazılır.

```
Çıktı
False 0
```

B ve C yanlış: baştaki `12` çevrilmez, değer `0` kalır. D yanlış: `TryParse` istisna fırlatmaz; bu `Parse` ve `Convert`'in davranışıdır. E yanlış: `out int sayi` yazımı geçerlidir.

4 Cevap: E

`F2` iki ondalık basamak ve Türkçe kültürde virgül demektir: `3,14` (dört karakter). Genişlik `8` olduğu için toplam alan 8 karakterdir ve değer **sağa** yaslanır; soluna 4 boşluk eklenir.

```
Çıktı
[    3,14]
```

A yanlış: genişlik yok sayılmış. B yanlış: nokta kullanmış ve sola yaslamış (sola yaslamak için genişlik `-8` olmalıydı). C yanlış: virgül yerine nokta var. D yanlış: sola yaslı.

5 Cevap: B

`int.Parse` tam sayı bekler; ondalık ayırıcı içeren `"3,5"` metni tam sayı sayılmaz ve `FormatException` fırlatır. Diğer seçenekleri aşağıdaki program doğruluyor.

A: başındaki sıfırlar atılır, sonuç `42`. C: Türkçe kültürde virgül ondalık ayırıcıdır, sonuç `3,5`. D: baştaki ve sondaki boşluklar yok sayılır, sonuç `7`. E: `TryParse` hata fırlatmaz, `false` döner.

6

- `{"ab",5}` → `ab`'nin soluna 3 boşluk; `{"cd",-5}` → `cd`'nin sağına 3 boşluk.
- `{7,4}` → `7`; `{7,-4}` → `7`; `{7:D4}` → `0007`.
- `{2.71828:F3}` → `2,718` (Türkçe kültürde virgül). `{2.6:F0}` → ondalık basamak kalmaz, `2,6` yuvarlanır: `3`. `{-3.14159,7:F2}` → `-3,14` (5 karakter), 7'ye tamamlamak için başına 2 boşluk.
- `10 + 5` → `15`, `'x'` → `x`, `3 > 2` → `True`.

```
Çıktı
|   ab|cd   |
|   7|7   |0007|
|2,718|3|  -3,14|
15-x-True
```

7

- Girdi satır satır okunur: `a = 12`, `b = 5`, `ad = "Ahmet Yılmaz"` (boşluk içeren satır tek parça gelir).
- `12 / 5` tam sayı bölmesi → `2`; `12 % 5` → `2`. İlk satır: `2|2|Ahmet Yılmaz|`.
- `a % b * 2.0` → `2 * 2.0` = `4`; `double` olarak yazıldığı için `4` görünür.
- `(double)a / b` → `2,4`; `F2` ile `2,40`.

```
Çıktı
12
5
Ahmet Yılmaz
2|2|Ahmet Yılmaz|
4
2,40
```

8

- `{0:D6}` → 1234 altı haneye sıfırla tamamlanır: `001234`. `{1:F1}` → `72,456` yuvarlanır: `72,5`.
- `s` = `No:001234 Ort:72,5` → 18 karakter.
- `{2500000:N0}` → Türkçe kültürde `2.500.000`.
- `7 * 100 / 8` = `700 / 8` = `87` (tam sayı bölmesi), ardından `%` karakteri.
- `Convert.ToInt32(6.5)` çifte yuvarlar → `6`; `Convert.ToInt32(7.5)` → `8`; `(int)7.5` keser → `7`.

```
Çıktı
No:001234 Ort:72,5
18
2.500.000 TL
87%
6 8 7
```

9

Ondalık girdiler virgülle verildiği için `Convert.ToDouble` doğrudan işe yarar. Her sayı `:F2` ile iki basamaklı yazdırılır; Türkçe kültürde ondalık ayırıcı kendiliğinden virgül olur.

dortIslem.cs

```
Console.Write("Birinci sayı: ");
double a = Convert.ToDouble(Console.ReadLine());
Console.Write("İkinci sayı: ");
double b = Convert.ToDouble(Console.ReadLine());

Console.WriteLine($"{a:F2} + {b:F2} = {a + b:F2}");
Console.WriteLine($"{a:F2} - {b:F2} = {a - b:F2}");
Console.WriteLine($"{a:F2} * {b:F2} = {a * b:F2}");
Console.WriteLine($"{a:F2} / {b:F2} = {a / b:F2}");
```

Çıktı

```
Birinci sayı: 7,5
İkinci sayı: 2
7,50 + 2,00 = 9,50
7,50 - 2,00 = 5,50
7,50 * 2,00 = 15,00
7,50 / 2,00 = 3,75
```

10

Her değer `ReadLine()` ile ayrı satır olarak okunur; sayıdan sonra metin gelmesi sorun değildir. Notlar `int` olsa da `0.4` ile çarpım `double` verir. Ortalamayı `:F2` ile yazdırırız.

notHesap.cs

```
Console.Write("Öğrenci no: ");
int no = Convert.ToInt32(Console.ReadLine());
Console.Write("Ad soyad: ");
string ad = Console.ReadLine() ?? "";
Console.Write("Vize: ");
int vize = Convert.ToInt32(Console.ReadLine());
Console.Write("Final: ");
int finalNotu = Convert.ToInt32(Console.ReadLine());

double ort = vize * 0.4 + finalNotu * 0.6;
Console.WriteLine($"{no} - {ad} - Ortalama: {ort:F2}");
```

Çıktı

```
Öğrenci no: 2026104
Ad soyad: Elif Su Demir
Vize: 65
Final: 78
2026104 - Elif Su Demir - Ortalama: 72,80
```

BÖLÜM 4

Koşullu İfadeler

- **Bu Bölümde Öğreneceklerin**
  - İlişkisel operatörlerle (`==`, `!=`, `<`, `>`, `<=`, `>=`) `bool` ifadeler kurmayı
  - `if`, `if-else` ve `else if` merdiveniyle programın akışını dallandırmayı
  - Süslü parantezsiz `if` ve sarkan `else` tuzaklarını; `if (x = 5)` ile `if (x)` neden derlenmez
  - `&&`, `||`, `!` operatörlerini, **kısa devre** davranışını ve De Morgan kurallarını
  - Operatör öncelik sırasını ve üçlü operatörü (`?:`)
  - `switch` deyimini (her bölüm `break` ile biter, CS0163), `switch` ifadesini (`=>`) ve ilişkisel desenleri (`>= 90`, `and`, `or`)
  - `string` ve `double` değerleri doğru karşılaştırmayı

Şimdiye kadar yazdığın programlar yukarıdan aşağı her satırı sırayla çalıştırdı. Gerçek programlar ise **karar verir**: "Not 50'den büyükse *geçti* yaz, değilse *kaldı* yaz." Bu bölümde programın akışını koşullara göre dallandırmayı öğreneceksin.

4.1 İlişkisel operatörler ve bool ifadeler

Karar vermek için önce bir soru sormak gerekir: "a, b'den büyük mü?" Bu sorunun cevabı her zaman `true` ya da `false`'tur, yani bir `bool` değerdir (Bölüm 2). Soru sormak için **ilişkisel operatörler** (relational operators) kullanılır:

Operatör	Anlamı	Örnek (`a = 7`, `b = 3`)	Sonuç
`==`	Eşit mi?	`a == b`	`false`
`!=`	Eşit değil mi?	`a != b`	`true`
`>`	Büyük mü?	`a > b`	`true`
`<`	Küçük mü?	`a < b`	`false`
`>=`	Büyük veya eşit mi?	`a >= 7`	`true`
`<=`	Küçük veya eşit mi?	`a <= 6`	`false`

Sonucu `bool` olan ifadelere **bool ifade** (boolean expression) ya da kısaca **koşul** (condition) denir. Sonucu bir değişkende saklayabilir veya ekrana yazdırabilirsin:

iliskisel.cs

```
int a = 7, b = 3;
Console.WriteLine(a == b);
Console.WriteLine(a != b);
Console.WriteLine(a <= 6);

bool gecti = a * 10 >= 50;        // önce 7 * 10, sonra >=
Console.WriteLine("Geçti mi? " + gecti);
Console.WriteLine('a' < 'b');     // 97 < 98
```

Çıktı

```
False
True
False
Geçti mi? True
True
```

Aritmetik işlemler karşılaştırmadan **önce** yapılır: `a * 10 >= 50` ifadesi `70 >= 50` olarak hesaplanır. Karakterler ise Unicode numaralarıyla karşılaştırılır (`'a'` = 97, `'b'` = 98).

> **İpucu**
>
> Kodda `true` ve `false` küçük harfle yazılır, ama `Console.WriteLine` bir `bool` değeri `True` ve `False` diye yazdırır. Çıktı sorularında bu büyük harfe dikkat et.

> **Sık Yapılan Hata: `=` ile `==` karıştırma**
>
> `=` **atama**, `==` **karşılaştırma** operatörüdür. `a = b` "b'nin değerini a'ya koy", `a == b` ise "a ile b eşit mi?" demektir. Bu karışıklığın `if` içinde neye yol açtığını birazdan göreceksin.

4.2 if deyimi

En temel karar yapısı `if`'tir (eğer):

```
if (koşul)
{
    // koşul true ise çalışacak komutlar
}
```

Koşul **parantez içinde** yazılır. Koşul `true` ise blok çalışır; `false` ise blok atlanır ve program bloktan sonraki satırdan devam eder. Bu kitapta süslü parantezleri C# stiline uygun olarak ayrı satıra yazıyoruz.

ifOrnek.cs

```
Console.Write("Sıcaklık: ");
int derece = Convert.ToInt32(Console.ReadLine());

if (derece > 30)
{
    Console.WriteLine("Hava çok sıcak, bol su iç!");
}
Console.WriteLine("İyi günler.");
```

```
Çıktı
Sıcaklık: 34
Hava çok sıcak, bol su iç!
İyi günler.
```

Sıcaklık 30 veya daha düşük girilseydi blok atlanır, sadece `İyi günler.` yazılırdı.

Süslü parantezsiz if

Blokta **tek bir komut** varsa süslü parantezler yazılmayabilir. Ama bu durumda `if`'e sadece **hemen ardından gelen tek komut** bağlıdır. Girinti (indentation) C# için hiçbir anlam taşımaz:

parantezsizIf.cs

```
int bakiye = 50;
if (bakiye >= 100)
    Console.WriteLine("Ödeme yapıldı.");
    Console.WriteLine("Bakiye düşüldü.");   // if'e dahil DEĞİL!
Console.WriteLine("Bitti.");
```

```
Çıktı
Bakiye düşüldü.
Bitti.
```

`Bakiye düşüldü.` satırı `if`'in içindeymiş gibi girintili yazılmış, ama `if`'e sadece ilk `WriteLine` bağlı. Koşul `false` olduğu halde ikinci satır çalıştı.

> **İpucu**
>
> Tek satır bile olsa **her zaman süslü parantez kullan**. Sonradan bloğa ikinci bir satır eklediğinde bu tuzağa düşme ihtimalin kalmaz. Visual Studio'da **Ctrl+K, Ctrl+D**, VS Code'da **Shift+Alt+F** (macOS: **Shift+Option+F**) kodu otomatik biçimlendirir ve girintiyi gerçek yapıya göre düzeltir; böyle hatalar hemen göze batar.

> **Sık Yapılan Hata: if'ten sonra noktalı virgül**
>
> `if (puan >= 50);` satırının sonundaki `;` **boş bir komuttur**: "Koşul doğruysa hiçbir şey yapma." Altına yazdığın `{ ... }` bloğu artık `if`'e bağlı değildir ve koşul ne olursa olsun **her zaman** çalışır. C# derleyicisi bunu CS0642 uyarısıyla yakalar; uyarıyı görmezden gelirsen program yine de derlenir ve yanlış çalışır:

bosIf.cs

```
int puan = 30;
if (puan >= 50);
{
    Console.WriteLine("Geçtin.");
}
```

```
Derleyici uyarısı (warning)
bosIf.cs(2,16): warning CS0642: Possible mistaken empty statement
```

```
Çıktı
Geçtin.
```

Puan 30 olduğu halde `Geçtin.` yazıldı. Bu kitapta derleyici uyarılarını hata gibi ele alıyoruz: uyarı çıktıysa kodu düzeltmeden ilerleme.

Koşul bool olmak zorunda

Bazı dillerde (C ve C++ gibi) `if (x = 5)` yazmak derlenir ve sıfırdan farklı sayı "doğru" sayılır; bu, yıllardır süren hataların kaynağıdır. C# koşulun **mutlaka** `bool` olmasını ister. `x = 5` bir atamadır ve değeri `int` (5) olduğu için program derlenmez:

atamaKosul.cs

```
int x = 3;
if (x = 5)
{
    Console.WriteLine("x beştir");
}
if (x)
{
    Console.WriteLine("x sıfırdan farklı");
}
```

Derleyici çıktısı – program derlenmez

```
atamaKosul.cs(2,5): error CS0029: Cannot implicitly convert type 'int' to 'bool'
atamaKosul.cs(6,5): error CS0029: Cannot implicitly convert type 'int' to 'bool'
```

İki hata da CS0029 koduyla gelir: ilki atamayı, ikincisi `if (x)` yazımını yakalar. C#'ta "sıfırdan farklı sayı doğrudur" diye bir kabul yoktur; sayının sıfır olup olmadığını açıkça sormalısın: `if (x != 0)`.

> **Sınavda Çıkar: `bool` değişkende atama tuzağı**
>
> `if (x = 5)` derlenmez, ama değişken `bool` ise atama **derlenir**! `bool bitti = false; if (bitti = true) {...}` yazarsan `bitti = true` atamasının değeri `true` olduğu için blok **her zaman** çalışır, üstelik `bitti` değişkeni de `true` olur. Derleyici bunu CS0665 uyarısıyla işaretler. `bool` bir değişkeni kontrol ederken `== true` yazmana gerek yok: `if (bitti)` ve `if (!bitti)` hem kısa hem güvenlidir.

4.3 if-else

Koşul yanlış olduğunda da bir şey yapmak istiyorsan `else` (değilse) kullanırsın. İki bloktan **tam olarak biri** çalışır:

tekCift.cs

```
int sayi = 17;

if (sayi % 2 == 0)
{
    Console.WriteLine($"{sayi} çift sayıdır.");
}
else
{
    Console.WriteLine($"{sayi} tek sayıdır.");
}
```

Çıktı

```
17 tek sayıdır.
```

> **● Sık Yapılan Hata: Negatif tek sayılar**
>
> Tek sayı kontrolünü `sayi % 2 == 1` diye yazma. Bölüm 2'de gördüğün gibi `%` sonucunun işareti soldaki sayının işaretiyle aynıdır: `-7 % 2` sonucu `-1`'dir. Yani `-7 % 2 == 1` ifadesi **false** olur ve -7 tek sayı sayılmaz. Güvenli yol: çiftlik için `sayi % 2 == 0`, teklik için `sayi % 2 != 0`.

4.4 else-if merdiveni

İkiden fazla seçenek varsa `else if` ile koşulları zincirlersin. Koşullar **yukarıdan aşağı** sırayla denenir; **ilk doğru olan** koşulun bloğu çalışır ve merdivenin geri kalanı atlanır. Hiçbiri doğru değilse en sondaki `else` çalışır:

harfNotu.cs

```
int puan = 78;
string harf;

if (puan >= 90)
{
    harf = "AA";
}
else if (puan >= 80)
{
    harf = "BB";
}
else if (puan >= 70)
{
    harf = "CC";
}
else if (puan >= 60)
{
    harf = "DD";
}
else
{
    harf = "FF";
}
Console.WriteLine($"Harf notun: {harf}");
```

Çıktı

```
Harf notun: CC
```

78 için `puan >= 90` ve `puan >= 80` yanlış, `puan >= 70` doğru: harf `CC` olur. Alttaki `puan >= 60` da doğrudur ama artık **denenmez**. Bu yüzden `else if (puan >= 70 && puan < 80)` yazmana gerek yok; o satıra gelindiyse puanın 80'den küçük olduğu zaten bellidir. (Gerçek harf aralıkları üniversiteden üniversiteye değişir.)

Merdiven yerine art arda **bağımsız** `if`'ler yazsaydık her koşul ayrı ayrı denenir ve 78 puan için `CC`, ardından `DD` ataması da yapılırdı. "Sadece biri çalışsın" istiyorsan `else if` kullan.

> **● Sınavda Çıkar: Koşulların sırası önemli**
>
> Merdiveni ters sırayla yazarsan (`if (puan >= 60) ... else if (puan >= 70) ... else if (puan >= 90)`) program derlenir ama yanlış çalışır: 95 puan ilk koşul olan `puan >= 60`'ı sağladığı için `DD` alır, alttaki koşullara hiç bakılmaz. Kural: `>=` ile kurulan merdivenlerde **en büyük eşikten başla**.

4.5 İç içe if ve sarkan else

Bir `if` bloğunun içine başka bir `if` yazılabilir; buna **iç içe if** (nested if) denir. İçteki `if`, ancak dıştaki koşul doğruysa denenir. Süslü parantezleri doğru kullanırsan iç içe yapı sorun çıkarmaz; asıl tuzak aşağıdaki sarkan `else`'tir.

> **İpucu: Üç sayının en büyüğü: "şampiyon" yöntemi**
>
> İç içe `if`'lerle de bulunabilir, ama kod hızla uzar. Bunun yerine ilk sayıyı şampiyon kabul et; her yeni sayı şampiyondan büyükse şampiyon o olsun: `int enBuyuk = a; if (b > enBuyuk) enBuyuk = b; if (c > enBuyuk) enBuyuk = c;` Bu yöntem sayı adedi arttıkça da işe yarar; Bölüm 5'te döngülerle tekrar kullanacağız. Hazır metot istersen: `Math.Max(a, Math.Max(b, c))`.

Sarkan else (dangling else)

Süslü parantez kullanmadan iç içe `if` yazarsan "Bu `else` hangi `if`'e ait?" sorusu ortaya çıkar. C#'ın kuralı: **`else`, kendisinden önceki en yakın ve henüz `else`'i olmayan `if`'e aittir.** Girintinin yine hiçbir önemi yoktur:

sarkanElse.cs

```
int x = 5, y = 15;

// Girinti yanıltıcı: else aslında içteki if'e ait
if (x > 10)
    if (y > 10)
        Console.WriteLine("1: İkisi de büyük");
else
    Console.WriteLine("1: x küçük");

// Süslü parantezle niyetimizi açıkça yazdık
if (x > 10)
{
    if (y > 10)
    {
        Console.WriteLine("2: İkisi de büyük");
    }
}
else
{
    Console.WriteLine("2: x küçük");
}
```

Çıktı

```
2: x küçük
```

İlk yapıda `else`, girintiye bakılırsa dıştaki `if`'e aitmiş gibi duruyor; ama C# onu içteki `if (y > 10)`'a bağlar. Dıştaki `x > 10` yanlış olduğu için içteki `if` ve onun `else`'i **tamamen** atlanır, ekrana hiçbir şey yazılmaz. İkinci yapıda süslü parantezler `else`'i dıştaki `if`'e bağladı.

> **Sınavda Çıkar**
>
> Sarkan `else` çıktı sorularının favorisidir. Çözerken **girintiyi tamamen görmezden gel**: Her `else`'i yukarıya doğru giderek en yakın, boşta kalan `if` ile eşleştir, sonra süslü parantezleri kafanda kendin yerleştir.

4.6 Mantıksal operatörler

Birden fazla koşulu birleştirmek için **mantıksal operatörler** (logical operators) kullanılır:

Operatör	Adı	Ne zaman `true`?	Örnek
`&&`	VE (AND)	İki koşul da doğruysa	`yas >= 18 && ehliyetVar`
`||`	VEYA (OR)	En az biri doğruysa	`gun == 6 || gun == 7`
`!`	DEĞİL (NOT)	Koşul yanlışsa (tersine çevirir)	`!bitti`

Doğruluk tablosu (truth table):

`a`	`b`	`a && b`	`a || b`	`!a`
`true`	`true`	`true`	`true`	`false`
`true`	`false`	`false`	`true`	`false`
`false`	`true`	`false`	`true`	`true`
`false`	`false`	`false`	`false`	`true`

Kısaca: `&&` sadece "ikisi de doğru" satırında doğru; `||` sadece "ikisi de yanlış" satırında yanlış.

Klasik bir örnek **artık yıl** (leap year) kontrolüdür. Bir yıl artık yıldır (Şubat 29 gün çeker) ancak şu durumda: **4'e bölünür ve 100'e bölünmez; ya da 400'e bölünür.**

artikYil.cs

```
Console.Write("Yıl: ");
int yil = Convert.ToInt32(Console.ReadLine());

bool artik = (yil % 4 == 0 && yil % 100 != 0) || yil % 400 == 0;

if (artik)
{
    Console.WriteLine($"{yil} artık yıldır, Şubat 29 gün.");
}
else
{
    Console.WriteLine($"{yil} artık yıl değildir, Şubat 28 gün.");
}
```

Çıktı

```
Yıl: 1900
1900 artık yıl değildir, Şubat 28 gün.
```

1900, 4'e bölünür ama 100'e de bölünür ve 400'e bölünmez; bu yüzden artık yıl değildir. 2000 ise 400'e bölündüğü için artık yıldır. Sınavda en çok bu iki yıl sorulur.

> **Sık Yapılan Hata: Matematikteki gibi aralık yazmak**
>
> Matematikte `0 ≤ puan ≤ 100` yazılır, ama C#'ta `if (0 <= puan <= 100)` **derlenmez** (CS0019). Önce `0 <= puan` hesaplanıp `true` ya da `false` olur, sonra `true <= 100` gibi anlamsız bir karşılaştırma denenir. Doğrusu iki koşulu `&&` ile bağlamaktır: `puan >= 0 && puan <= 100`. Aynı sebeple `a == b == c` de derlenmez; `a == b && b == c` yazmalısın. (C# 9'dan beri aralığı `puan is >= 0 and <= 100` diye de yazabilirsin; aşağıda göreceksin.)

Kısa devre değerlendirme

`&&` ve `||` **kısa devre** (short-circuit) operatörleridir: Sonuç sol taraftan belli oluyorsa sağ taraf **hiç hesaplanmaz**.

- `false && ...` → sağ taraf ne olursa olsun sonuç `false`; sağ taraf atlanır.
- `true || ...` → sağ taraf ne olursa olsun sonuç `true`; sağ taraf atlanır.

Bu özellik sayesinde tehlikeli bir işlemi bir koşulla "koruyabilirsin":

kisaDevre.cs

```
int x = 0;
if (x != 0 && 10 / x > 1)
{
    Console.WriteLine("10 / x, 1'den büyük");
}
else
{
    Console.WriteLine("Sıfıra bölme yapılmadı.");
}

int sayac = 0;
int a = 5;
bool s1 = (a > 3) || (++sayac > 0);   // sağ taraf atlanır
bool s2 = (a > 3) | (++sayac > 0);    // iki taraf da hesaplanır
Console.WriteLine($"{s1} {s2} sayac={sayac}");
```

Çıktı

```
Sıfıra bölme yapılmadı.
True True sayac=1
```

`x != 0` yanlış olduğu için `10 / x` hiç hesaplanmadı ve program çökmedi. Koşulların sırasını ters yazsaydın (`10 / x > 1 && x != 0`), önce bölme yapılır ve `DivideByZeroException` oluşurdu (tamsayıda sıfıra bölme, Bölüm 2). **Koruyucu koşul her zaman solda olmalı.**

Tek karakterli `&` ve `|` de `bool`'larla VE/VEYA yapar ama kısa devre **yapmaz**; iki tarafı da her zaman hesaplar. Yukarıda `sayac` sadece bir kez arttı: `||`'nin sağ tarafı atlandı, `|`'nün sağ tarafı çalıştı. Aynı korumayı `x != 0 & 10 / x > 1` diye tek `&` ile yazsaydın, bölme yine yapılır ve program `DivideByZeroException` ile çökerdi. (Bölüm 2'de tamsayılarda bit işlemi yapan `&` ve `|` ile aynı operatörlerdir; iki işlenen de `bool` olunca mantıksal çalışırlar. `^` ise `bool`'larda "tam biri doğru" demektir.)

> **Sınavda Çıkar**
>
> Kısa devre soruları genelde koşulun içine `++` gizler: `if (a > 5 && ++b > 0)`. Önce sol tarafı hesapla; sonuç belli oluyorsa sağdaki `++` **hiç çalışmaz** ve `b` artmaz. `&&` ve `||` kısa devre yapar; `&` ve `|` yapmaz.

De Morgan kuralları

Bir koşulun **tersini** yazman gerektiğinde **De Morgan kuralları** kullanılır:

- `!(a && b)` ≡ `!a || !b`
- `!(a || b)` ≡ `!a && !b`

Yani `!` parantezin içine girerken her koşulu tersine çevirir ve `&&` ile `||` **yer değiştirir**. Karşılaştırmaların tersleri: `<` ↔ `>=`, `>` ↔ `<=`, `==` ↔ `!=`. Örneğin "puan geçerli değil" koşulu `!(puan >= 0 && puan <= 100)` yerine `puan < 0 || puan > 100` diye yazılabilir.

> **Sık Yapılan Hata: Tersini alırken && kalırsa**
>
> "Puan geçersiz" koşulunu `puan < 0 && puan > 100` diye yazmak çok sık yapılan bir hatadır. Bir sayı aynı anda hem 0'dan küçük hem 100'den büyük olamayacağı için bu koşul **hiçbir zaman** doğru olmaz. Karşılaştırmaları çevirirken `&&`'yi de `||`'ye çevirmeyi unutma.

4.7 Operatör önceliği

Bölüm 2'de aritmetik operatörlerin önceliğini gördün. Karşılaştırma ve mantıksal operatörler eklenince tablo şöyle olur (yukarıdan aşağı, öncelik azalır):

Öncelik	Operatörler	Açıklama
1	`()`, `x++`, `x--`	Parantez, sonek artırma
2	`!`, `++x`, `--x`, tekli `-`, `(tip)`	Tekli (unary) operatörler
3	`*`, `/`, `%`	Çarpma, bölme, mod
4	`+`, `-`	Toplama, çıkarma, string birleştirme
5	`<`, `>`, `<=`, `>=`, `is`	Büyüklük karşılaştırma
6	`==`, `!=`	Eşitlik
7	`&&`	Mantıksal VE
8	`||`	Mantıksal VEYA
9	`??`	Boşsa-değer (Bölüm 3)
10	`?:`	Üçlü operatör
11	`=`, `+=`, `-=`, …	Atama (en son)

Tek karakterli `&`, `^`, `|` operatörleri (bu sırayla) bu tabloda `==` ile `&&` arasında yer alır. Ezber cümlesi: **"Tekli, aritmetik, karşılaştırma, mantık, atama."** En önemli ayrıntı: **`&&`, `||`'den önce gelir** (çarpmanın toplamadan önce gelmesi gibi).

oncelikMantik.cs

```
int a = 5, b = 10;
bool t = true, f = false;
Console.WriteLine(a + 5 > b - 1);        // 10 > 9
Console.WriteLine(t || f && f);          // && önce
Console.WriteLine((t || f) && f);        // parantez önce
Console.WriteLine(!(a > 3) || b == 10);
Console.WriteLine(a > 3 == b > 3);       // True == True
```

Çıktı

```
True
True
False
True
True
```

İkinci satırda önce `f && f` → `False` hesaplanır, sonra `t || False` → `True`. Parantez sırayı değiştirince sonuç `False` oldu.

> **Sık Yapılan Hata: String birleştirme ile karşılaştırma**
>
> `Console.WriteLine("Sonuç: " + a > b);` **derlenmez**. `+` operatörü `>`'den önce geldiği için önce `"Sonuç: " + a` hesaplanır ve bir `string` olur; sonra `string` ile sayı `>` ile karşılaştırılmaya çalışılır. Derleyici CS0019 hatası verir. Çözüm parantez: `"Sonuç: " + (a > b)`.

4.8 Üçlü operatör (?:)

Bir koşula göre iki değerden birini seçmenin kısa yolu **üçlü operatördür** (ternary / conditional operator). C#'ın üç işlenen alan tek operatörüdür:

```
koşul ? doğruysaDeğer : yanlışsaDeğer
```

uclu.cs

```
int a = 8, b = 13;
int buyuk = (a > b) ? a : b;
Console.WriteLine($"Büyük olan: {buyuk}");

int puan = 64;
Console.WriteLine($"Durum: {(puan >= 50 ? "Geçti" : "Kaldı")}");
Console.WriteLine((a < b ? 1 : 2.5) / 2);   // int mi, double mı?
```

Çıktı

```
Büyük olan: 13
Durum: Geçti
0,5
```

`int buyuk = (a > b) ? a : b;` satırı, dört satırlık bir `if-else`'in kısa halidir. Üçlü operatör bir **değer** ürettiği için doğrudan atamada veya yazdırma içinde kullanılabilir. `$"..."` içinde parantez şarttır, çünkü `{ }` içindeki `:` karakteri biçim belirteci (`{x:F2}`) sayılır.

> **Sınavda Çıkar: Üçlü operatörde tip yükseltme**
>
> İki seçenek farklı sayısal tipteyse sonuç **büyük olan tipe** dönüştürülür. Bu yüzden yukarıdaki programın son satırı `0` değil **`0,5`** yazdırdı: `1` (`int`) ile `2.5` (`double`) karşılaşınca ifade `double` oldu, bölme de ondalıklı yapıldı. Ayrıca iç içe üçlü operatör (`a > b ? (a > c ? a : c) : ...`) okunması zor koddur; birden fazla koşul varsa `if-else` tercih et.

4.9 switch deyimi

Bir değişkenin **belirli değerlerine** göre farklı işler yapacaksan uzun bir else-if merdiveni yerine `switch` deyimini (statement) kullanabilirsin:

menu.cs

```
int secim = 2;
switch (secim)
{
    case 1:
        Console.WriteLine("Bakiye görüntüleniyor");
        break;
    case 2:
        Console.WriteLine("Para yatırma ekranı");
        break;
    default:
        Console.WriteLine("Geçersiz seçim!");
        break;
}
Console.WriteLine("İşlem tamam.");
```

Çıktı

```
Para yatırma ekranı
İşlem tamam.
```

Çalışma mantığı:

1. Parantezdeki değer hesaplanır (`secim` → 2).
2. Değeri eşleşen `case` etiketine atlanır (`case 2:`).
3. Oradan başlayarak komutlar çalışır; **`break`** görülünce switch'ten çıkılır.
4. Hiçbir `case` eşleşmezse `default` çalışır. `default` zorunlu değildir ve genelde en sona yazılır.

break unutulursa: derleme hatası

Bazı dillerde `break` unutulunca program alttaki case'lere "düşer" (fall-through) ve sessizce yanlış çalışır. C# bu hatayı baştan engeller: **her `case` bölümü `break` ile (ya da metot içindeysen `return` ile) bitmek zorundadır**, `default` bölümü de dahil. Bitmezse program derlenmez:

dusme.cs

```
int secim = 2;
switch (secim)
{
    case 1:
        Console.WriteLine("Bakiye görüntüleniyor");
    case 2:
        Console.WriteLine("Para yatırma ekranı");
        break;
    default:
        Console.WriteLine("Geçersiz seçim!");
        break;
}
```

Derleyici çıktısı – program derlenmez

```
dusme.cs(4,5): error CS0163: Control cannot fall through from one case label ('case 1:') to another
```

> **Sınavda Çıkar: Boş case'ler birleşir, dolu case'ler birleşmez**
>
> `case 1:` ile `case 2:` arasında **hiç komut yoksa** iki etiket aynı bölümü paylaşır; bunun için `break` gerekmez. Etiketler arasında tek bir komut bile varsa, o bölüm `break` / `return` ile bitmek zorundadır; yoksa yukarıdaki CS0163 hatasını alırsın. Hata mesajındaki dil sana Türkçe gelebilir, `CS` kodu aynı kalır.

Birden fazla etiket, string ile switch

Aynı işi yapan birkaç case'i **alt alta** yazarak gruplayabilirsin. `switch` `string` ile de çalışır; karşılaştırma büyük/küçük harf duyarlıdır:

haftaSonu.cs

```
string gun = "Pazar";
switch (gun)
{
    case "Cumartesi":
    case "Pazar":
        Console.WriteLine("Hafta sonu, dinlen!");
        break;
    case "Cuma":
        Console.WriteLine("Bir gün kaldı.");
        break;
    default:
        Console.WriteLine("Hafta içi, çalış.");
        break;
}
```

Çıktı

```
Hafta sonu, dinlen!
```

`"Cumartesi"` ve `"Pazar"` etiketleri arasında komut olmadığı için aynı bölümü paylaşır. `"pazar"` yazsaydın hiçbir etiketle eşleşmez, `default` çalışırdı.

> **Not: switch'te case etiketleri**
>
> `case` etiketleri **sabit** olmalıdır (`1`, `'a'`, `"evet"` gibi değerler ya da `const` sabitler; sıradan bir değişken olamaz) ve aynı değer iki kez yazılamaz. `int`, `char`, `string`, `bool` ve `enum` (Bölüm 11) ile rahatça kullanılır. Aralık kontrolü (`puan >= 90`) gerekiyorsa klasik `case` yetmez; aşağıdaki ilişkisel desenleri ya da `if` kullan.

4.10 switch ifadesi ve desenler

C# 8'den beri `switch` bir **ifade** (expression) olarak da yazılabilir: Bir **değer üretir** ve bu değer bir değişkene atanır. Yazımı klasik `switch`'ten farklıdır:

- `case` ve `break` yoktur; her kol `desen => değer,` biçimindedir. Kollar virgülle ayrılır.
- Hiçbir kol birbirine düşmez; sadece **ilk eşleşen** kol çalışır.
- Hiçbir kol eşleşmezse çalışacak `_` (her şeyi yakalayan "atma" deseni, discard) en sona yazılır; `default`'un karşılığıdır.

İlişkisel desenler: and, or, not

Desenler sabit değerle sınırlı değildir; `>=`, `<` gibi ilişkisel operatörlerle **aralık** da yazabilirsin. Desenleri `and`, `or`, `not` ile birleştirirsin. Else-if merdiveniyle yazdığımız harf notu hesabı böylece kısalır. Bir `switch` ifadesi bir atama komutunun parçası olduğu için kapanan süslü parantezden sonra `;` gelir:

harfSwitch.cs

```
Console.Write("Puanın (0–100): ");
int puan = Convert.ToInt32(Console.ReadLine());

string harf = puan switch
{
    < 0 or > 100 => "Geçersiz",
    >= 90 => "AA",
    >= 80 => "BB",
    >= 70 => "CC",
    >= 60 => "DD",
    _ => "FF"
};

string durum = puan switch
{
    >= 0 and < 50 => "Kaldı",
    >= 50 and <= 100 => "Geçti",
    _ => "Geçersiz"
};
Console.WriteLine($"Harf: {harf}, durum: {durum}");
```

Çıktı

```
Puanın (0–100): 85
Harf: BB, durum: Geçti
```

İlk `switch`'te sıra önemlidir: kollar **yukarıdan aşağı** denenir, ilk eşleşen kazanır. `< 0 or > 100` kolunu en üste koyduk; yoksa 105 puan `>= 90` koluna takılıp `AA` alırdı. İkinci `switch`'te kollar `and` ile birbirinden ayrıldığı için sıra sonucu değiştirmez.

Derleyici sana yardım eder: Bir kol, yukarıdaki kolların zaten kapsadığı değerlerden başka bir şey yakalamıyorsa program derlenmez:

siraHata.cs

```
int puan = 85;
string harf = puan switch
{
    >= 60 => "DD",
    >= 90 => "AA",
    _ => "FF"
};
Console.WriteLine(harf);
```

Derleyici çıktısı – program derlenmez

```
siraHata.cs(5,5): error CS8510: The pattern is unreachable. It has already been handled by a
previous arm of the switch expression or it is impossible to match.
```

`>= 60` zaten 90 ve üstünü de kapsadığı için `>= 90` kolu hiçbir zaman çalışamaz; derleyici bunu CS8510 hatasıyla bildirir. Kollar **kısmen** örtüşüyorsa (örneğin `> 0 and < 20` ile `> 10`) derleme hatası olmaz; yine de ilk eşleşen kol kazanır. Aynı sıra kuralı `if-else if` merdiveninde derleyici uyarısı olmadan yanlış sonuç üretir, `switch` ifadesinde ise çoğu zaman yakalanır.

> **Sınavda Çıkar: switch ifadesi her durumu kapsamalı**
>
> Bir `switch` ifadesi **her olası değer** için bir sonuç vermek zorundadır. Son kola `_` yazmazsan derleyici CS8509 uyarısı verir (uyarı derlemeyi durdurmaz, ama ciddiye al); program çalışırken hiçbir kol eşleşmezse `SwitchExpressionException` ile çöker. Bir kolda birden fazla komut gerekiyorsa `switch` ifadesi uygun değildir; klasik `switch` ya da `if` kullan.

> **Yeni C# Notu: Desenler**
>
> `switch` ifadesi C# 8 ile, `>=`, `and`, `or`, `not` desenleri C# 9 ile geldi. Eski kodlarda aynı işin `case int n when n >= 90:` gibi daha uzun yazıldığını görebilirsin. `when` bir koşul ekler: `var k when k % 2 == 0 => "çift"`. Derleyici `when` içindeki koşulu çözümleyemez; bu yüzden sıra hatalarını `when` kollarında yakalayamaz.

is deseni

Aynı desenleri `if` içinde `is` ile de kullanabilirsin. `is` bir `bool` üretir; aralık kontrolünün en okunaklı hali budur:

isDeseni.cs

```
int sayi = 72;
Console.WriteLine(sayi is >= 0 and <= 100);   // aralık
Console.WriteLine(sayi is 0 or 100);          // iki değerden biri mi?
Console.WriteLine(sayi is not 50);            // 50 değil mi?
```

Çıktı

```
True
False
True
```

4.11 string ve double karşılaştırma

string'ler == ile karşılaştırılır

`string` bir başvuru tipi (Bölüm 7'de göreceksin) olsa da `==` operatörü C#'ta **içeriği** karşılaştırır. Yani `cevap == "evet"` kullanıcının yazdığı metin tam olarak `evet` ise `true` verir. Karşılaştırma büyük/küçük harf duyarlıdır. Harf farkını yok saymak istiyorsan `string.Equals` metoduna karşılaştırma türünü söylersin:

stringEsitlik.cs

```
Console.Write("Devam edilsin mi? ");
string cevap = Console.ReadLine() ?? "";

Console.WriteLine(cevap == "evet");
Console.WriteLine(cevap == "EVET");
Console.WriteLine(
    string.Equals(cevap, "EVET", StringComparison.OrdinalIgnoreCase));
```

Çıktı

```
Devam edilsin mi? evet
True
False
True
```

`StringComparison.OrdinalIgnoreCase`, harfleri kültürden bağımsız karşılaştırır ve büyük/küçük farkını yok sayar. `string`'in diğer karşılaştırma metotlarını Bölüm 8'de göreceksin.

> **Sık Yapılan Hata: ToLower() ile karşılaştırma**
>
> `cevap.ToLower() == "info"` gibi yazımlar Türkçe kültürde sürpriz yapar: Türkçe'de `"I".ToLower()` noktasız `ı` verir, `"i".ToUpper()` ise noktalı `İ` verir. Bu yüzden `"INFO".ToLower()` sonucu `"info"` olmaz. Büyük/küçük harfi yok sayan karşılaştırma için `StringComparison.OrdinalIgnoreCase` kullan.

Ondalıklı sayılar

Bölüm 2'de `0.1 + 0.2 == 0.3` ifadesinin `false` olduğunu gördün. Bu yüzden hesaplama sonucu olan `double`'ları `if` içinde `==` ile karşılaştırma; aralarındaki farkın çok küçük bir **tolerans** değerinden (epsilon) küçük olup olmadığına bak:

doubleEsitlik.cs

```
double toplam = 0.1 + 0.2;
Console.WriteLine(toplam == 0.3);
Console.WriteLine(Math.Abs(toplam - 0.3) < 1e-9);   // tolerans
Console.WriteLine(toplam);
```

Çıktı

```
False
True
0,30000000000000004
```

Üçüncü satır `toplam` değerini tam hâliyle gösterir: `0,30000000000000004`. `0,3` değildir; fark `1e-9` toleransından çok küçük olduğu için tolerans kontrolü `True` verir. Para gibi tam sonuç gereken yerlerde `decimal` kullanılır (Bölüm 2).

`a == 0` gibi bir sabitle karşılaştırma ise sorun değildir: Kullanıcının yazdığı `0` tam olarak `0.0` olarak okunur. Sorun, **hesaplama sonucunda** oluşan küçük yuvarlama farklarındadır.

4.12 Örnek uygulama: VKİ yorumu

Bölüm 3'teki vücut kitle indeksi programını tamamlayalım. Dünya Sağlık Örgütü'nün sınıflandırması: 18,5'in altı **zayıf**, 25'in altı **normal**, 30'un altı **fazla kilolu**, 30 ve üzeri **obez**. Okuma kısmı Bölüm 3'tekiyle aynı olduğu için değerleri doğrudan yazıyoruz. `double` üzerinde ilişkisel desen kullanmak da mümkündür:

vkiYorum.cs

```
double kilo = 84, boy = 1.75;
double vki = kilo / (boy * boy);

string durum = vki switch
{
    < 18.5 => "zayıf",
    < 25.0 => "normal",
    < 30.0 => "fazla kilolu",
    _ => "obez"
};
Console.WriteLine($"VKİ: {vki:F1} ({durum})");
```

Çıktı
```
VKİ: 27,4 (fazla kilolu)
```

Bölüm Özeti – Bilmen Gerekenler

- İlişkisel operatörler (`==`, `!=`, `<`, `>`, `<=`, `>=`) `bool` sonuç üretir. `=` atama, `==` karşılaştırmadır. Çıktıda `True` / `False` yazılır.
- `if` koşulu **mutlaka** `bool` olmalıdır: `if (x = 5)` ve `if (x)` (x `int` ise) CS0029 ile derlenmez. `bool` değişkende `if (b = true)` derlenir ve hep doğrudur (CS0665 uyarısı).
- Süslü parantez yoksa `if`'e sadece **bir** komut bağlıdır; girinti önemsizdir. `if (...);` boş komuttur (CS0642 uyarısı).
- else-if merdiveninde **ilk doğru** koşul çalışır; `>=` merdivenlerinde en büyük eşikten başla.
- **Sarkan else:** `else`, en yakın boşta kalan `if`'e aittir.
- `&&` (VE), `||` (VEYA), `!` (DEĞİL). `&&` ve `||` **kısa devre** yapar; `&` ve `|` iki tarafı da hesaplar. Koruyucu koşul solda: `x != 0 && 10 / x > 1`.
- De Morgan: `!(a && b)` = `!a || !b`, `!(a || b)` = `!a && !b`. `0 <= x <= 100` derlenmez; `x >= 0 && x <= 100` yaz.
- Öncelik: `!` > aritmetik > `< > <= >=` > `== !=` > `&&` > `||` > `?:` > `=`.
- `koşul ? a : b` bir değer üretir; farklı sayısal tiplerde büyük tipe yükseltilir.
- `switch` deyiminde her `case` bölümü `break` / `return` ile biter; aksi halde CS0163. Boş case'ler birleşir.
- `switch` ifadesi: `x switch { >= 90 => "AA", _ => "FF" }`; kollar yukarıdan aşağı denenir, ilk eşleşen kazanır; `_` şart; tamamen örtülen kol CS8510 hatasıdır. `and`, `or`, `not` ve `is` desenleri aralık kontrolünü kısaltır.
- `string` karşılaştırmada `==` içeriğe bakar; harf farkını yok saymak için `string.Equals(a, b, StringComparison.OrdinalIgnoreCase)`. `double`'lar toleransla karşılaştırılır (`Math.Abs(a - b) < 1e-9`).

Sorular

A) Çoktan Seçmeli

1 `!(x > 5 && y <= 10)` ifadesine **denk** olan ifade hangisidir?

(A) `x <= 5 && y > 10`
(B) `x < 5 || y >= 10`
(C) `x > 5 || y <= 10`
(D) `x <= 5 || y > 10`
(E) `x < 5 && y > 10`

2 Aşağıdaki kod parçası çalıştığında ekrana ne yazdırılır?

```
int a = 3, b = 0;
if (a > 5 && ++b > 0)
{
    a++;
}
if (a < 5 || ++b > 0)
{
    a--;
}
Console.WriteLine($"{a} {b}");
```

(A) `2 0` (B) `2 1` (C) `4 1` (D) `3 0` (E) `2 2`

3 `int n` değişkeninin **tek** sayı olup olmadığını, negatif sayılar dahil her zaman doğru kontrol eden koşul hangisidir?

(A) `n % 2 == 1` (B) `n / 2 == 1` (C) `n % 2 != 0` (D) `n % 2 == -1` (E) `n % 2 > 0`

4 Aşağıdaki kod parçası çalıştığında ekrana ne yazdırılır?

```
int n = 15;
string s = n switch
{
    > 0 and < 20 => "küçük",
    > 10 => "büyük",
    _ => "yok"
};
Console.WriteLine(s);
```

(A) `yok`
(B) `büyük`
(C) `küçük büyük`
(D) Program derlenmez
(E) `küçük`

5 `int x = 3;` tanımından sonra yazılan aşağıdaki satırlardan hangisi **derleme hatası** verir?

(A) `if (x == 5) x++;`
(B) `if (x = 5) x++;`
(C) `bool b = x > 3 && x < 9;`
(D) `string s = x > 0 ? "pozitif" : "değil";`
(E) `bool c = (x = 5) > 3;`

B) Çıktı Tahmini

6 Aşağıdaki program ekrana ne yazdırır? (Girintiye güvenme!)

```
int a = 4, b = 9;
if (a > b)
    if (a > 0)
        Console.WriteLine("bir");
else
    Console.WriteLine("iki");
if (b % a == 1)
    Console.WriteLine("üç");
    Console.WriteLine("dört");
int x = 8, y = 3;
if (x > 5)
    if (y > 5)
        Console.WriteLine("beş");
else
    Console.WriteLine("altı");
```

7 Aşağıdaki program ekrana ne yazdırır?

```
int x = 5, y = 10;
bool p = x > 3 || ++y > 10;
bool q = x > 3 | ++y > 10;
bool r = !(x == 5) && y++ > 0;
Console.WriteLine($"{p} {q} {r} {y}");
Console.WriteLine(x > y ? x : y * 2);
Console.WriteLine((x == 5 ? 1 : 2.0) / 2);
```

8 Aşağıdaki program ekrana ne yazdırır?

```
string s = "b";
switch (s)
{
    case "a":
    case "b":
        Console.Write("1");
        break;
    case "c":
        Console.Write("2");
        break;
    default:
        Console.Write("3");
        break;
}
Console.WriteLine();

int n = 7;
string t = (n % 3) switch
{
    0 => "sıfır",
    1 => "bir",
    _ => "iki"
};
Console.WriteLine(t);
```

C) Kod Yazma

9 **İkinci dereceden denklemin** ($ax^2 + bx + c = 0$) köklerini bulan bir program yaz. Kullanıcıdan `a`, `b`, `c` katsayılarını ondalıklı sayı olarak oku. $\Delta = b^2 - 4ac$ olmak üzere: $\Delta > 0$ ise iki kök, $\Delta = 0$ ise çift kat kök, $\Delta < 0$ ise "gerçek kök yok" yazdır. `a = 0` ise uyarı ver. Kökler 2 basamakla yazılsın:

```
a: 1
b: -5
c: 6
İki kök var: x1 = 3,00, x2 = 2,00
```

10 Kullanıcıdan bir **ay** (1–12) ve **yıl** okuyup o ayın kaç gün çektiğini yazdıran bir program yaz. Şubat için artık yıl kontrolü yap. Ay geçersizse uyarı ver. (İpucu: `switch` ifadesi ile çok kısa yazılır.)

```
Ay (1-12): 2
Yıl: 2024
2024 yılının 2. ayı 29 gün çeker.
```

Çözümler

Cevap anahtarı

1 D	2 A	3 C	4 E	5 B

1 Cevap: D

De Morgan: `!(A && B)` = `!A || !B`. `x > 5`'in tersi `x <= 5`, `y <= 10`'un tersi `y > 10`; `&&` ise `||` olur. A şıkkı `&&`'yi değiştirmeyi unutmuş. B ve E'de karşılaştırmaların tersi yanlış alınmış (`>`'nin tersi `<` değil `<=`'dir). C ise ifadeyi hiç tersine çevirmemiş.

2 Cevap: A

İlk `if`: `a > 5` yanlış → `&&` kısa devre yapar, `++b` **çalışmaz**; blok da çalışmaz. İkinci `if`: `a < 5` doğru → `||` kısa devre yapar, `++b` yine **çalışmaz**; blok çalışır ve `a` 2 olur. `b` hiç artmadı. B, C ve E şıkları `++b`'nin çalıştığını varsayar; D şıkkı ise ikinci bloğun çalıştığını görmemiştir.

Çıktı

```
2 0
```

3 Cevap: C

Negatif tek sayılarda kalan **-1** olur: `-7 % 2` = `-1`. Bu yüzden `n % 2 == 1` (A) ve `n % 2 > 0` (E) negatif tek sayılarda `false` verir; `n % 2 == -1` (D) ise pozitif tek sayılarda `false` verir. B şıkkı bölmeyle ilgilidir, teklikle ilgisi yoktur. `n % 2 != 0` ise kalan 1 de olsa -1 de olsa doğru çalışır. `n = -7` için ilk satır kalanı, ikinci satır A, C, E şıklarını; üçüncü satır `m = 7` için D ve C şıklarını gösteriyor:

Çıktı

```
-1
False True False
False True
```

4 Cevap: E

`switch` ifadesinde kollar yukarıdan aşağı denenir ve **ilk eşleşen** kol kazanır. 15, `> 0 and < 20` desenine uyar; `> 10` kolu da uyardı ama ona hiç sıra gelmez. Sonuç `küçük`. Switch ifadesi tek bir değer üretir, bu yüzden C yanlıştır. `> 10` kolu önceki kolun kapsamadığı değerleri (örneğin 25) yakalayabildiği için derleyici hata vermez (D yanlış); hata ancak bir kolun tamamı öncekilerce kapsanıyorsa verilir. A ve B şıkları kolların sırasını yanlış okur.

Çıktı

```
küçük
```

5 Cevap: B

`x = 5` bir atamadır ve değeri `int`'tir; `if` ise `bool` ister, bu yüzden B derlenmez (CS0029). E şıkkı garip görünse de derlenir: Parantez içindeki `x = 5` hem `x`'e 5 atar hem de değer olarak 5 üretir; `5 > 3` de `bool`'dur. A, C ve D sıradan, geçerli satırlardır.

```
Çıktı
5 False pozitif True
```

6

- İlk yapı: `else`, girintiye rağmen içteki `if (a > 0)`'a aittir. Dıştaki `a > b` (4 > 9) yanlış olduğu için hiçbir şey yazılmaz.
- `b % a` = `9 % 4` = 1 → koşul doğru, `üç` yazılır. `dört` satırı `if`'e bağlı değildir, her durumda yazılır.
- Üçüncü yapıda `x > 5` doğru, içteki `y > 5` (3 > 5) yanlış. `else` içteki `if`'e ait olduğu için çalışır ve `altı` yazılır; `beş` yazılmaz.

```
Çıktı
üç
dört
altı
```

7

- `p`: `x > 3` doğru → `||` kısa devre, `++y` çalışmaz. `p = True`, `y = 10`.
- `q`: `|` kısa devre yapmaz → `++y` çalışır, `y = 11`, `11 > 10` doğru. `q = True`.
- `r`: `!(x == 5)` = `False` → `&&` kısa devre, `y++` çalışmaz. `r = False`, `y = 11`.
- `x > y` → `5 > 11` yanlış → `y * 2` = 22.
- `x == 5 ? 1 : 2.0` → `int` ve `double` karışık olduğu için sonuç `double` (`1.0`); `1.0 / 2` = `0,5` (Türkçe kültürde virgülle yazılır).

```
Çıktı
True True False 11
22
0,5
```

8

- Klasik `switch`: `"b"`, `case "a":` ile aynı bölümü paylaşır (arada komut yok); bölüm `1` yazar ve `break` ile biter.
- `(n % 3) switch`: `7 % 3` = 1 → `"bir"`. Parantez şarttır; yoksa `switch` ifadesi `n % 3`'ün tamamına değil sadece `3`'e bağlanırdı.

```
Çıktı
1
bir
```

9

Önce `a = 0` durumunu ele, sonra deltayı hesapla. Kökler: x = (−b ± √Δ) / 2a. Δ ondalıklı sayılarla hesaplandığı için "Δ = 0" kontrolünü toleransla yapıyoruz. Girdi `Convert.ToDouble` ile okunduğu için Türkçe kültürde ondalıklı sayılar virgülle girilir; çıktı da virgüllü yazılır.

denklem.cs

```
Console.Write("a: ");
double a = Convert.ToDouble(Console.ReadLine());
Console.Write("b: ");
double b = Convert.ToDouble(Console.ReadLine());
Console.Write("c: ");
```

```
double c = Convert.ToDouble(Console.ReadLine());

if (a == 0)
{
    Console.WriteLine("a = 0 ise denklem ikinci dereceden değildir.");
}
else
{
    double delta = b * b - 4 * a * c;
    if (Math.Abs(delta) < 1e-9)
    {
        Console.WriteLine($"Çift kat kök: x = {-b / (2 * a):F2}");
    }
    else if (delta > 0)
    {
        double x1 = (-b + Math.Sqrt(delta)) / (2 * a);
        double x2 = (-b - Math.Sqrt(delta)) / (2 * a);
        Console.WriteLine($"İki kök var: x1 = {x1:F2}, x2 = {x2:F2}");
    }
    else
    {
        Console.WriteLine("Gerçek kök yok (delta < 0).");
    }
}
```

Çıktı

```
a: 1
b: -5
c: 6
İki kök var: x1 = 3,00, x2 = 2,00
```

10

Artık yıl koşulunu bölümdeki gibi tek bir `bool` ifadeyle hesapla. `switch` ifadesinde 31 ve 30 çeken ayları `or` ile grupla; Şubat için üçlü operatör kullan. Geçersiz ay için `_` kolu 0 üretsin, sonra bunu `if` ile kontrol et.

ayGun.cs

```
Console.Write("Ay (1-12): ");
int ay = Convert.ToInt32(Console.ReadLine());
Console.Write("Yıl: ");
int yil = Convert.ToInt32(Console.ReadLine());

bool artik = (yil % 4 == 0 && yil % 100 != 0) || yil % 400 == 0;
int gun = ay switch
{
    1 or 3 or 5 or 7 or 8 or 10 or 12 => 31,
    4 or 6 or 9 or 11 => 30,
    2 => artik ? 29 : 28,
    _ => 0
};

if (gun == 0)
{
    Console.WriteLine($"Geçersiz ay: {ay}");
}
else
{
    Console.WriteLine($"{yil} yılının {ay}. ayı {gun} gün çeker.");
}
```

Çıktı

```
Ay (1–12): 2
Yıl: 2024
2024 yılının 2. ayı 29 gün çeker.
```

BÖLÜM 5

Döngüler

- **Bu Bölümde Öğreneceklerin**
  - `while`, `do-while` ve `for` döngülerini ve hangisinin ne zaman kullanılacağını
  - Sonsuz döngüyü, `break` ve `continue` ile döngü akışını kontrol etmeyi
  - İç içe döngülerle tablo ve **yıldız deseni** yazdırmayı (sınav klasiği)
  - **İzleme tablosu** (trace table) ile çıktı tahmin etmeyi ve off-by-one hatasını
  - Klasik algoritmaları: basamak toplamı, asal sayı, faktöriyel, Fibonacci, EBOB, en büyük ve en küçük değer

5.1 Neden döngü?

Ekrana 1'den 1000'e kadar sayı yazdırmak için 1000 tane `Console.WriteLine` yazamazsın. Bir işi **tekrar tekrar** yaptırmak için **döngü** (loop) kullanılır; döngünün her bir turuna **iterasyon** (iteration) denir. C#'ta üç döngü vardır: `while`, `do-while` ve `for`. (Dizilerle kullanılan `foreach` Bölüm 7'nin konusudur.)

5.2 while döngüsü

`while` (... olduğu sürece), koşul `true` olduğu sürece gövdesini tekrar eder. Koşul her turdan **önce** kontrol edilir:

```
while (koşul)
{
    // koşul true olduğu sürece tekrar edilecek komutlar
}
```

Sayaçlı bir döngünün üç parçası vardır: **başlangıç**, **koşul** ve **güncelleme**. Biri eksikse döngü ya hiç çalışmaz ya da hiç bitmez:

whileOrnek.cs

```
int i = 1;                          // 1. başlangıç
while (i <= 5)                      // 2. koşul
{
    Console.Write($"{i} ");
    i++;                            // 3. güncelleme
}
Console.WriteLine();
Console.WriteLine($"Döngüden sonra i = {i}");

int toplam = 0;
int sayi = 1;
while (sayi <= 100)
{
    toplam += sayi;                 // toplama ekle
    sayi++;
}
Console.WriteLine($"1'den 100'e toplam: {toplam}");
```

```
Çıktı
1 2 3 4 5
Döngüden sonra i = 6
1'den 100'e toplam: 5050
```

Döngü bittiğinde `i` 5 değil **6**'dır: Koşulu bozan değer 6'dır. İkinci döngü **biriktirme** (accumulator) kalıbıdır: `toplam` döngüden **önce** 0 yapılır, her turda üzerine eklenir.

> **Sık Yapılan Hata: Toplamı döngünün içinde sıfırlamak**
>
> `int toplam = 0;` satırını döngünün **içine** yazarsan toplam her turda sıfırlanır ve sonunda sadece son sayıyı tutar. Biriktirilen değişken her zaman döngüden **önce** başlatılır. Çarpımda başlangıç değeri 0 değil **1** olmalıdır.

5.3 do-while döngüsü

`do-while`'da koşul gövdeden **sonra** kontrol edilir. Gövde, koşul baştan yanlış olsa bile **en az bir kez** çalışır:

```
do
{
    // en az bir kez çalışacak komutlar
} while (koşul);      ← noktalı virgül şart!
```

En tipik kullanımı **menüdür**: Menü önce bir kez gösterilir, kullanıcı çıkış (0) diyene kadar tekrar eder.

menuDongu.cs

```
int bakiye = 100;
int secim;                          // do'nun DIŞINDA tanımlanmalı
do
{
    Console.Write("1) Bakiye 2) Yatır 0) Çıkış > ");
    secim = Convert.ToInt32(Console.ReadLine());
    if (secim == 2)
    {
        Console.Write("Miktar: ");
        bakiye += Convert.ToInt32(Console.ReadLine());
    }
    else if (secim == 1)
    {
        Console.WriteLine($"Bakiye: {bakiye} TL");
    }
} while (secim != 0);
Console.WriteLine("Güle güle!");
```

```
Çıktı
1) Bakiye 2) Yatır 0) Çıkış > 2
Miktar: 50
1) Bakiye 2) Yatır 0) Çıkış > 1
Bakiye: 150 TL
1) Bakiye 2) Yatır 0) Çıkış > 0
Güle güle!
```

> **Sık Yapılan Hata: do-while'da değişkenin kapsamı**
>
> `secim`'i `do { int secim = ...; }` bloğunun **içinde** tanımlarsan `while (secim != 0)` koşulu onu göremez ve program derlenmez. Değişken, döngüden önce tanımlanmalıdır.

5.4 for döngüsü

Tekrar sayısı **belli** olduğunda en okunaklı döngü `for`'dur. Başlangıç, koşul ve güncelleme tek satırda, `for (başlangıç; koşul; güncelleme)` biçiminde yazılır.

Çalışma sırası: **1)** başlangıç (sadece bir kez) → **2)** koşul → `true` ise **3)** gövde → **4)** güncelleme → tekrar **2)**. Koşul `false` olunca döngü biter.

forOrnek.cs

```
for (int i = 1; i <= 5; i++)
{
    Console.Write($"{i} ");
}
Console.WriteLine();

for (int i = 10; i > 0; i -= 3)     // geriye doğru, 3'er 3'er
{
    Console.Write($"{i} ");
}
Console.WriteLine();

for (char c = 'a'; c <= 'e'; c++)   // char da sayaç olabilir
{
    Console.Write(c);
}
Console.WriteLine();
```

Çıktı

```
1 2 3 4 5
10 7 4 1
abcde
```

Sayaç kapsamı

`for` parantezindeki `int i` **sadece döngünün içinde** yaşar. Döngüden sonra `Console.WriteLine(i)` yazarsan derleyici `i`'yi tanımaz ve derleme hatası verir. Sayaca döngüden sonra da ihtiyacın varsa onu döngüden **önce** `int i;` ile tanımla.

Noktalı virgül tuzağı

`for (...)` ya da `while (...)` parantezinden hemen sonraki `;` gövdeyi **boşaltır**. Alttaki blok döngüye ait değildir ve yalnızca bir kez çalışır:

noktaliVirgul.cs

```
int sayac = 0;
for (int i = 0; i < 5; i++);
{
    sayac++;
}
Console.WriteLine($"sayac = {sayac}");
```

Derleyici uyarısı (warning)

```
noktaliVirgul.cs(2,28): warning CS0642: Possible mistaken empty statement
```

Çıktı
```
sayac = 1
```

Derleyici bu satır için CS0642 uyarısı verir (“Possible mistaken empty statement”). Uyarı derlemeyi durdurmaz, ama program yanlış çalışır: sayaç sadece bir kez artar.

Hangisini seçmeli?

	`while`	`do-while`	`for`
Koşul ne zaman kontrol edilir?	Turdan önce	Turdan sonra	Turdan önce
En az kaç kez çalışır?	0	**1**	0
Ne zaman tercih edilir?	Tekrar sayısı belli değil (“0 girilene kadar”)	Gövde en az bir kez çalışmalı (menü)	Tekrar sayısı belli (“10 kez”, “1’den n’e”)

5.5 Sonsuz döngü, break ve continue

Koşulu hiçbir zaman `false` olmayan döngüye **sonsuz döngü** (infinite loop) denir. En sık sebep, sayacı güncellemeyi unutmaktır:

sonsuzDongu.cs
```
int i = 1;
while (i <= 10)
{
    Console.WriteLine(i);           // i++ unutuldu: i hep 1
}

int x = 1;
while (x != 10)                     // x hep tek sayı: asla 10 olmaz
{
    x += 2;
}
```

İkinci döngüde `!=` yerine `x < 10` yazsaydın döngü `x` 11 olunca biterdi. Sayaçlı döngülerde `!=` yerine `<` veya `<=` tercih et. C#’ta sonsuz döngü için `while (true)` yazılır; `while (1)` derleme hatasıdır, çünkü `while` koşulu `bool` olmak zorundadır.

> **İpucu**
>
> Terminalde `dotnet run` çalışırken **Ctrl+C** ile programı durdurabilirsin. Visual Studio’da **F9** ile kesme noktası koyup **F5** ile çalıştırarak değişkenleri her turda izleyebilirsin.

Bazen sonsuz döngü **bilerek** yazılır: `while (true)` ile başlayıp, çıkış koşulu sağlanınca `break` ile çıkılır. `break` döngüyü **anında** bitirir. `continue` ise o turun geri kalanını atlayıp **sonraki tura** geçer (`for`’da önce güncelleme yapılır):

breakContinue.cs

```
for (int i = 1; i <= 10; i++)
{
    if (i % 2 == 0)
    {
        continue;                  // çiftleri atla
    }
    if (i > 7)
    {
        break;                     // 7'den büyük ilk tekte dur
    }
    Console.Write($"{i} ");
}
Console.WriteLine();
```

Çıktı

```
1 3 5 7
```

Sık Yapılan Hata: while'da continue tuzağı

`while (i < 10)` döngüsünde güncelleme (`i++`) gövdenin **sonundaysa** `continue` onu da atlar: `i` 0'da takılı kalır ve döngü sonsuz olur. `for`'da bu sorun yoktur; `continue`'dan sonra güncelleme yine çalışır.

5.6 İç içe döngüler

Dış döngünün **her turu** için iç döngü baştan sona çalışır. Dış döngü 5, iç döngü 5 tur atıyorsa iç gövde $5 \times 5 = 25$ kez çalışır. Çarpım tablosunda sütunları hizalamak için `{x,4}` biçimini kullanıyoruz:

carpimTablosu.cs

```
for (int i = 1; i <= 5; i++)           // satırlar
{
    for (int j = 1; j <= 5; j++)       // sütunlar
    {
        Console.Write($"{i * j,4}");
    }
    Console.WriteLine();               // satır bitti
}
```

Çıktı

```
   1   2   3   4   5
   2   4   6   8  10
   3   6   9  12  15
   4   8  12  16  20
   5  10  15  20  25
```

C#'ta etiketli break yoktur

İç içe döngüde `break` sadece en içteki döngüyü bitirir. İki döngüden birden çıkmak için **bayrak** (flag) kullanılır: Bayrak `false` başlar, aranan durum bulununca `true` olur ve dış döngünün koşuluna eklenir. `break dis;` gibi etiketli bir `break` derlenmez; `goto` ile etiket yazılabilir ama kodu okunmaz yapar.

5.7 Yıldız desenleri

"n verildiğinde şu deseni yazdır" soruları sınavların klasiğidir. Hepsinin tarifi aynıdır: **dış döngü satırları** sayar, **iç döngü** o satırdaki karakterleri basar. Kaç karakter basılacağı **satır numarası `i` cinsinden** yazılır; satır bitince `Console.WriteLine()` çağrılır.

desenler1.cs

```
int n = 4;
// Dik üçgen: i. satırda i yıldız
for (int i = 1; i <= n; i++)
{
    for (int j = 1; j <= i; j++)
    {
        Console.Write("*");
    }
    Console.WriteLine();
}
```

Çıktı

```
*
**
***
****
```

Piramit için her satırda önce `n - i` boşluk, sonra `2 * i - 1` yıldız basılır. n = 4 için satırlar: 3 boşluk + 1 yıldız, 2 + 3, 1 + 5, 0 + 7. Formülü kâğıtta bir tabloyla bul.

desenler2.cs

```
int n = 4;
// Piramit
for (int i = 1; i <= n; i++)
{
    for (int j = 1; j <= n - i; j++)
    {
        Console.Write(" ");
    }
    for (int j = 1; j <= 2 * i - 1; j++)
    {
        Console.Write("*");
    }
    Console.WriteLine();
}
Console.WriteLine();
// Sayı üçgeni: i. satırda 1'den i'ye kadar sayılar
for (int i = 1; i <= n; i++)
{
    for (int j = 1; j <= i; j++)
    {
        Console.Write($"{j} ");
    }
    Console.WriteLine();
}
```

```
Çıktı
    *
   ***
  *****
*******

1
1 2
1 2 3
1 2 3 4
```

> **Sınavda Çıkar: Desen sorusunu çözme yöntemi**
>
> Kodu yazmadan önce kâğıda **satır | boşluk | karakter** tablosunu çiz ve her sütunu `i` ile ifade et. Sayı üçgeninde `j` yerine `i` yazsaydın her satır kendi numarasını tekrar ederdi: `1`, `2 2`, `3 3 3`... Hocalar bu iki varyantı sık sık karşılaştırır.

5.8 Döngü izleme tablosu ve off-by-one

İzleme tablosu (trace table)

Döngülü bir kodun çıktısını tahmin etmenin en güvenli yolu, kâğıda **izleme tablosu** (trace table) çizmektir: Her değişkene bir sütun açılır, her turda değişen değerler yeni satıra yazılır.

izleme.cs

```
int a = 1, b = 0;
while (a < 20)
{
    b += a;
    a *= 3;
}
Console.WriteLine($"{a} {b}");
```

Tur	Koşul `a < 20`	`b += a`	`a *= 3`
başlangıç		`b = 0`	`a = 1`
1	`1 < 20` ✓	`b = 1`	`a = 3`
2	`3 < 20` ✓	`b = 4`	`a = 9`
3	`9 < 20` ✓	`b = 13`	`a = 27`
—	`27 < 20` ✗	döngü biter	

```
Çıktı
27 13
```

> **Sınavda Çıkar**
>
> Çıktı sorularında izleme tablosu çizmek zaman kaybı değildir, **puan kazancıdır**. Yukarıda `a`'nın son değeri 27'dir, 9 değil: Döngüden çıkıldığında değişkenlerin **son** değerleri yazdırılır.

Off-by-one hatası

Döngünün **bir tur fazla veya eksik** çalışmasına **off-by-one** (bir kayma) hatası denir. Sebebi genelde `<` ile `<=` karışıklığıdır. `i = a; i <= b` döngüsü `b - a + 1` tur, `i = a; i < b` döngüsü `b - a` tur atar: `i = 0; i <= 10` 11 tur, `i = 0; i < 10` 10 tur. Emin olamadığında küçük bir değerle (n = 1 veya 2) döngüyü kafanda çalıştır ve say.

5.9 Klasik döngü algoritmaları

Bu algoritmalar sınavlarda ve ödevlerde tekrar tekrar karşına çıkacak. Anlayarak **ezberle**.

Basamak toplamı ve ters çevirme

Bölüm 2'deki iki kural burada döngüye dönüşür: `n % 10` **son basamağı verir**, `n / 10` **son basamağı atar**:

basamaklar.cs

```
Console.Write("Bir sayı gir: ");
int sayi = Convert.ToInt32(Console.ReadLine());

int n = Math.Abs(sayi);             // kopya; eksi işaretini at
int adet = 0, toplam = 0, ters = 0;
do                                  // 0 da bir basamaklıdır: en az bir tur
{
    int basamak = n % 10;           // son basamak
    adet++;
    toplam += basamak;
    ters = ters * 10 + basamak;
    n /= 10;                        // son basamağı at
} while (n > 0);
Console.WriteLine($"Basamak sayısı: {adet}");
Console.WriteLine($"Basamak toplamı: {toplam}");
Console.WriteLine($"Tersi: {ters}");
string palindrom = sayi == ters ? "evet" : "hayır";
Console.WriteLine($"Palindrom mu? {palindrom}");
```

Çıktı

```
Bir sayı gir: 4728
Basamak sayısı: 4
Basamak toplamı: 21
Tersi: 8274
Palindrom mu? hayır
```

Döngü `do-while` çünkü `0` girilirse de bir tur dönmeli: `0` bir basamaklı bir sayıdır (`while (n > 0)` yazsaydık hiç dönmez ve "0 basamak" derdi). `Math.Abs` ile eksi işaretini atıyoruz; böylece `-472` için de basamaklar doğru sayılır. `ters = ters * 10 + basamak` satırı sayıyı bir basamak sola kaydırıp yeni basamağı ekler: 0 → 8 → 82 → 827 → 8274. Tersi kendisine eşit olan sayıya **palindrom** denir (12321 gibi). `bool` değeri `Console.WriteLine` ile yazılırsa `True`/`False` görünür; bu yüzden `evet`/`hayır` yazdırıyoruz.

Asal sayı kontrolü

Bir sayı **asal** ise 1 ve kendisinden başka böleni yoktur; en küçük asal 2'dir. 2'den başlayıp bölen ararız; bir bölen bulunca cevap kesinleşir ve `break` ile çıkılır:

asalMi.cs

```
Console.Write("Sayı: ");
int n = Convert.ToInt32(Console.ReadLine());

bool asal = n >= 2;                 // 0 ve 1 asal değil
for (int i = 2; i * i <= n; i++)    // karekökü kadar dene
{
    if (n % i == 0)
    {
        Console.WriteLine($"{i} bir böleni.");
        asal = false;
        break;
    }
}
string sonuc = asal ? "asaldır." : "asal değildir.";
Console.WriteLine($"{n} {sonuc}");
```

Çıktı

```
Sayı: 91
7 bir böleni.
91 asal değildir.
```

`i * i <= n` koşulu bir hızlandırmadır: Bir sayının karekökünden büyük bir böleni varsa, karekökünden küçük bir eşi de vardır ($91 = 7 \times 13$). `asal` değişkeni bir **bayrak** (flag) değişkenidir: önce `true` kabul edilir, aksini gösteren durum bulununca `false` yapılır.

Faktöriyel ve Fibonacci

Faktöriyel $n! = 1 \times 2 \times \ldots \times n$ bir **çarpım biriktirme** örneğidir. Sonuç çok hızlı büyür; `long` kullanılmalıdır:

faktoriyel.cs

```
int n = 20;
long f = 1;                         // çarpımda başlangıç 1!
int fInt = 1;
for (int i = 1; i <= n; i++)
{
    f *= i;
    fInt *= i;
}
Console.WriteLine($"{n}! (long) = {f}");
Console.WriteLine($"{n}! (int)  = {fInt}");
```

Çıktı

```
20! (long) = 2432902008176640000
20! (int)  = -2102132736
```

`int` versiyonu sessizce **taştı** (Bölüm 2): C#'ta tamsayı taşması hata vermez, değer sarılır. `int` en fazla 12!'yi, `long` en fazla 20!'yi doğru tutabilir.

Fibonacci dizisinde her terim kendinden önceki iki terimin toplamıdır: 0, 1, 1, 2, 3, 5, 8... Sadece son iki terimi tutan iki değişken yeter:

fibonacci.cs

```
int a = 0, b = 1;
for (int i = 1; i <= 12; i++)
{
    Console.Write($"{a} ");
    int sonraki = a + b;
    a = b;
    b = sonraki;
}
Console.WriteLine();
```

Çıktı

```
0 1 1 2 3 5 8 13 21 34 55 89
```

`a = b; b = a + b;` yazsaydın `a`'nın eski değeri kaybolurdu; bu yüzden toplam önce `sonraki`'ye alınır.

EBOB (Öklid algoritması)

İki sayının **en büyük ortak böleni** (EBOB, GCD) için **Öklid algoritması**: `(a, b)` çiftini `(b, a % b)` ile değiştir; `b` 0 olunca `a` EBOB'dur. **En küçük ortak kat** (EKOK) ise `a * b / EBOB`'dur.

ebob.cs

```
int a = 48, b = 18;
int x = a, y = b;
while (y != 0)
{
    int kalan = x % y;
    x = y;
    y = kalan;
}
Console.WriteLine($"EBOB({a}, {b}) = {x}");
Console.WriteLine($"EKOK({a}, {b}) = {a / x * b}");
```

Çıktı

```
EBOB(48, 18) = 6
EKOK(48, 18) = 144
```

İzleme: (48, 18) → (18, 12) → (12, 6) → (6, 0). `a / x * b` sırası bilinçlidir: önce bölmek, `a * b`'nin taşma ihtimalini azaltır.

En büyük ve en küçük değer

En büyüğü ve en küçüğü bulmak için ilk değer iki değişkene de atanır; her yeni sayı ikisiyle karşılaştırılır. Başlangıç olarak `0` kullanmak yanlıştır: tüm sayılar negatif olsaydı en büyük değer `0` kalırdı.

enBuyukKucuk.cs

```
Console.Write("1. sayı: ");
int enBuyuk = Convert.ToInt32(Console.ReadLine());
int enKucuk = enBuyuk;              // iki değişken de ilk değerle başlar
for (int i = 2; i <= 5; i++)
{
    Console.Write($"{i}. sayı: ");
    int sayi = Convert.ToInt32(Console.ReadLine());
    if (sayi > enBuyuk)
    {
        enBuyuk = sayi;
    }
    if (sayi < enKucuk)
    {
        enKucuk = sayi;
    }
}
Console.WriteLine($"En büyük: {enBuyuk}, en küçük: {enKucuk}");
```

Çıktı

```
1. sayı: 12
2. sayı: -7
3. sayı: 30
4. sayı: 5
5. sayı: -15
En büyük: 30, en küçük: -15
```

5.10 Kullanıcı girdisiyle döngüler

Nöbetçi değer (sentinel)

Kaç değer gireceği bilinmeyen girişlerde, girişin bittiğini haber veren özel bir değer belirlenir: **nöbetçi değer** (sentinel). Nöbetçi gerçek veri olamayacak bir değer olmalı; pozitif sayılar için `0` uygundur:

nobetci.cs

```
Console.Write("Pozitif bir sayı gir (bitirmek için 0): ");
int sayi = Convert.ToInt32(Console.ReadLine());
int toplam = 0, adet = 0;
while (sayi != 0)
{
    toplam += sayi;
    adet++;
    Console.Write("Pozitif bir sayı gir (bitirmek için 0): ");
    sayi = Convert.ToInt32(Console.ReadLine());
}

if (adet > 0)
{
    double ort = (double)toplam / adet;
    Console.WriteLine($"{adet} sayı, ortalama {ort:F2}");
}
else
{
    Console.WriteLine("Hiç sayı girilmedi.");
}
```

```
Çıktı
Pozitif bir sayı gir (bitirmek için 0): 70
Pozitif bir sayı gir (bitirmek için 0): 85
Pozitif bir sayı gir (bitirmek için 0): 92
Pozitif bir sayı gir (bitirmek için 0): 0
3 sayı, ortalama 82,33
```

İlk değer döngüden **önce** okunur (*priming read*), sonrakiler gövdenin **sonunda**; böylece `0` toplama girmez. `Convert.ToInt32` sayı olmayan bir metin alırsa `FormatException` fırlatır; kullanıcı yanlış yazabilecekse Bölüm 3'teki `int.TryParse` ile kontrol et.

Boş satırla bitirmek

Metin okurken nöbetçi olarak **boş satır** kullanılabilir. C#'ta `!=` metinlerde içerik karşılaştırır. Dosya sonunda `Console.ReadLine()` `null` döndürür; `?? ""` bunu boş metne çevirir ve döngü yine biter:

bosSatirDur.cs

```
Console.WriteLine("Satır yaz, bitirmek için boş satır bırak.");
int satirSayisi = 0;
int karakter = 0;
string satir = Console.ReadLine() ?? "";
while (satir != "")
{
    satirSayisi++;
    karakter += satir.Length;
    satir = Console.ReadLine() ?? "";
}
Console.WriteLine($"{satirSayisi} satır, {karakter} karakter.");
```

```
Çıktı
Satır yaz, bitirmek için boş satır bırak.
merhaba dünya
C#

2 satır, 15 karakter.
```

Boş satırdan sonraki `bu satır okunmaz` hiç işlenmedi: döngü boş satırda bitti.

Bölüm Özeti – Bilmen Gerekenler

- `while` koşulu turdan **önce**, `do-while` koşulu turdan **sonra** kontrol eder; `do-while` gövdeyi en az bir kez çalıştırır ve sonunda `;` ister.
- `for (başlangıç; koşul; güncelleme)` tekrar sayısı belliyken en okunaklıdır; parantezdeki sayaç yalnızca döngü içinde geçerlidir.
- Güncelleme unutulursa **sonsuz döngü** olur. `for (...);` ve `while (...);` gövdeyi boşaltır (CS0642).
- `break` döngüyü bitirir, `continue` sonraki tura geçer. İki döngüden çıkmak için bayrak kullan; C#'ta etiketli `break` yoktur.
- İç içe döngüde dış döngünün **her turu** için iç döngü baştan çalışır. Desenlerde dış döngü **satır**, iç döngü **sütun**dur.
- Basamaklar `n % 10` ile son basamağı verir. Faktöriyel `long` ile hesaplanır (int 12!'de, long 20!'de sınıra ulaşır). EBOB: `(a, b) → (b, a % b)`.
- Toplam, sayaç ve en büyük/en küçük değer döngüden **önce** başlatılır. Nöbetçi değer: ilk değeri döngüden önce oku, sonrakileri gövde sonunda.

Sorular

A) Çoktan Seçmeli

1 Aşağıdaki kod parçasında `*` karakteri toplam kaç kez yazdırılır?

```
for (int i = 1; i <= 5; i++)
{
    for (int j = i; j <= 5; j++)
    {
        Console.Write("*");
    }
}
Console.WriteLine();
```

(A) 25 (B) 15 (C) 5 (D) 10 (E) 20

2 Aşağıdaki kod parçası ekrana ne yazdırır?

```
int x = 10;
do
{
    Console.Write($"{x} ");
    x++;
} while (x < 5);
Console.WriteLine();
```

(A) Hiçbir şey yazdırmaz.
(B) `5 6 7 8 9`
(C) Sonsuz döngüye girer.
(D) `10`
(E) Derleme hatası verir.

3 Aşağıdaki döngülerden hangisi gövdesini **tam 5 kez** çalıştırır?

(A) `for (int i = 5; i > 0; i--)`
(B) `for (int i = 0; i <= 5; i++)`
(C) `for (int i = 1; i < 5; i++)`
(D) `for (int i = 0; i < 10; i += 3)`
(E) `for (int i = 10; i >= 0; i -= 2)`

4 Aşağıdaki kod parçası ekrana ne yazdırır?

```
int toplam = 0;
for (int i = 1; i <= 10; i++)
{
    if (i % 3 == 0)
    {
        continue;
    }
    if (i == 8)
    {
        break;
    }
    toplam += i;
}
Console.WriteLine(toplam);
```

(A) 27 (B) 37 (C) 22 (D) 12 (E) 19

5 Aşağıdakilerden hangisi **derleme hatası** verir?

(A) `for (;;) { break; }`

(B) `int i = 0; while (i < 3) i++;`

(C) `for (int i = 0; i < 3; i++) { } Console.WriteLine(i);`

(D) `do { } while (false);`

(E) `for (int i = 0, j = 10; i < j; i++, j--) { }`

B) Çıktı Tahmini

6 Aşağıdaki program ekrana ne yazdırır?

```
int i = 0;
while (i++ < 3)
{
    Console.Write($"{i} ");
}
Console.WriteLine($"| {i}");

int j = 0;
while (++j < 3)
{
    Console.Write($"{j} ");
}
Console.WriteLine($"| {j}");

int k;
for (k = 0; k < 3; k++);
{
    Console.WriteLine($"k = {k}");
}
```

7 Aşağıdaki program ekrana ne yazdırır?

```
int sayac = 0;
for (int i = 1; i <= 4; i++)
{
    for (int j = 1; j <= i; j++)
    {
        Console.Write((i + j) % 2 == 0 ? "1" : "0");
        sayac++;
    }
    Console.WriteLine();
}
Console.WriteLine($"sayac = {sayac}");
```

8 Aşağıdaki program ekrana ne yazdırır?

```
int n = 1203, sonuc = 0;
while (n > 0)
{
    int b = n % 10;
    n /= 10;
    if (b == 0)
    {
        continue;
    }
    sonuc = sonuc * 10 + b;
}
Console.WriteLine(sonuc);

bool dur = false;
for (int i = 1; i <= 3 && !dur; i++)
{
    for (int j = 1; j <= 3; j++)
    {
        if (i == 3)
        {
            dur = true;
            break;
        }
        if (j > i)
        {
            break;
        }
        Console.Write($"{i}{j} ");
    }
}
Console.WriteLine();
```

C) Kod Yazma

9 Kullanıcıdan bir `n` sayısı okuyup `2n − 1` satırlık **elmas** (baklava) desenini yazdıran `Elmas` programını yaz. `n = 4` için çıktı:

```
   *
  ***
 *****
*******
 *****
  ***
   *
```

10 **Sayı tahmin oyunu:** Program 1 ile 100 arasında bir sayı tutsun; kullanıcı doğru tahmin edene kadar “Daha büyük!” / “Daha küçük!” ipucu versin ve sonunda kaç denemede bildiğini yazsın. Sayıyı `Random` sınıfıyla üret: `new Random(42).Next(1, 101)` (1 dahil, 101 hariç; sabit tohum `42` her çalıştırmada aynı sayıyı verir). `TahminOyunu` adlı programı `do-while` ile yaz.

Çözümler

Cevap anahtarı

1 B	2 D	3 A	4 E	5 C

1 Cevap: B

İç döngü `j = i`'den 5'e kadar döner: `i = 1` için 5, `i = 2` için 4, … `i = 5` için 1 tur. Toplam 5 + 4 + 3 + 2 + 1 = **15**. A şıkkı (25), iç döngünün her seferinde 1'den başladığını sananların cevabıdır.

Çıktı
```
***************
```

2 Cevap: D

`do-while` koşulu **sonradan** kontrol eder; gövde bir kez çalışır ve `10` yazılır. Sonra `x = 11` olur, `11 < 5` yanlış olduğu için döngü biter. Aynı kod `while` ile yazılsaydı hiçbir şey yazdırmazdı (A).

Çıktı
```
10
```

3 Cevap: A

`i` 5, 4, 3, 2, 1 değerlerini alır: 5 tur. Diğerleri: B → 0'dan 5'e **6** tur (off-by-one); C → 1'den 4'e **4** tur; D → 0, 3, 6, 9: **4** tur; E → 10, 8, 6, 4, 2, 0: **6** tur. Beş döngünün tur sayıları sırasıyla:

Çıktı
```
5 6 4 4 6
```

4 Cevap: E

`i = 3` ve `i = 6` `continue` ile atlanır. `i = 8` olunca `break` ile döngü biter (`9` ve `10` hiç gelmez). Toplanan sayılar: 1 + 2 + 4 + 5 + 7 = **19**.

Çıktı
```
19
```

5 Cevap: C

`for` parantezinde tanımlanan `i` yalnızca döngünün içinde geçerlidir; döngüden sonra `Console.WriteLine(i)` derleme hatası verir (bkz. "Sayaç kapsamı"). Diğer dört şık derlenir: A geçerli bir sonsuz döngüdür ve `break` ile hemen çıkar; B gövdesi tek komut olan bir `while`'dır; D gövdesi boş bir `do-while`'dır; E virgülle iki değişkenli geçerli bir `for`'dur. Doğrulama:

Çıktı
```
A, B, D, E derlendi: i = 3
```

6

- `while (i++ < 3)`: Karşılaştırma `i`'nin **eski** değeriyle yapılır, sonra `i` artar. 0 < 3 → `i = 1`, yazılır; 1 < 3 → 2; 2 < 3 → 3; 3 < 3 yanlış, ama `i` yine artar → **4**.
- `while (++j < 3)`: Önce artırılır. `j = 1` ✓, `j = 2` ✓, `j = 3` → 3 < 3 yanlış. Sadece 1 ve 2 yazılır, `j = 3`.
- `for (...);` sonundaki `;` boş gövdedir: döngü hiçbir şey yapmadan `k`'yı 3'e çıkarır. Derleyici bu satırda CS0642 uyarısı verir:

```
Derleyici uyarısı (warning)
soru6.cs(16,24): warning CS0642: Possible mistaken empty statement
```

Alttaki blok döngüye ait değildir, **bir kez** çalışır.

```
Çıktı
1 2 3 | 4
1 2 | 3
k = 3
```

7

`i + j` çiftse `1`, tekse `0` yazılır. İzleme: `i = 1`: j=1 → 2 çift → `1`. `i = 2`: j=1 → 3 → `0`, j=2 → 4 → `1`. `i = 3`: `1`, `0`, `1`. `i = 4`: `0`, `1`, `0`, `1`. İç gövde 1 + 2 + 3 + 4 = 10 kez çalışır.

```
Çıktı
1
01
101
0101
sayac = 10
```

8

- İlk döngü basamakları **sağdan sola** gezer: 3, 0, 2, 1. `0` `continue` ile atlanır; diğerleri ters çevirme formülüyle eklenir: 3 → 32 → 321. `n /= 10` satırı `continue`'dan **önce** olduğu için sonsuz döngü olmaz.
- İkinci döngüde `break` sadece iç döngüyü bitirir: `i = 1` için `11`, `i = 2` için `21 22` yazılır. `i = 3` olunca `dur` `true` olur ve dış döngü de biter.

```
Çıktı
321
11 21 22
```

9

Elmas, bir piramit ile onun ters çevrilmiş halinin birleşimidir. Üst kısım `i = 1 ... n` için piramit formülüyle (`n − i` boşluk, `2i − 1` yıldız); alt kısım aynı formülle `i = n − 1`'den 1'e **azalarak** yazılır. Ortadaki en geniş satır iki kez yazılmasın diye alt kısım `n − 1`'den başlar.

elmas.cs

```
Console.WriteLine("n sayısını gir:");
int n = Convert.ToInt32(Console.ReadLine());

for (int i = 1; i <= n; i++)            // üst yarı
{
```

```
    for (int j = 1; j <= n - i; j++)
    {
        Console.Write(" ");
    }
    for (int j = 1; j <= 2 * i - 1; j++)
    {
        Console.Write("*");
    }
    Console.WriteLine();
}
for (int i = n - 1; i >= 1; i--)        // alt yarı
{
    for (int j = 1; j <= n - i; j++)
    {
        Console.Write(" ");
    }
    for (int j = 1; j <= 2 * i - 1; j++)
    {
        Console.Write("*");
    }
    Console.WriteLine();
}
```

Çıktı

```
n sayısını gir:
4
   *
  ***
 *****
*******
 *****
  ***
   *
```

10

Kullanıcı en az bir tahmin yapacağı için `do-while` en doğal seçimdir. `tahmin` ve `deneme` koşulda kullanılacağı için döngüden **önce** tanımlanır. Bu çalıştırmada tutulan sayı 67'dir; program dört denemede bulur. Her tahminle aralığı daraltmak (50 → 75 → 62 → 67) en hızlı stratejidir; buna ikili arama (binary search) denir.

tahminOyunu.cs

```
int tutulan = new Random(42).Next(1, 101);
int tahmin;
int deneme = 0;

Console.WriteLine("1-100 arası bir sayı tuttum.");
do
{
    Console.Write("Tahmin: ");
    tahmin = Convert.ToInt32(Console.ReadLine());
    deneme++;
    if (tahmin < tutulan)
    {
        Console.WriteLine("Daha büyük!");
    }
    else if (tahmin > tutulan)
    {
        Console.WriteLine("Daha küçük!");
```

```
    }
} while (tahmin != tutulan);

Console.WriteLine($"Bildin! {deneme} denemede buldun.");
```

Çıktı

```
1–100 arası bir sayı tuttum.
Tahmin: 50
Daha büyük!
Tahmin: 75
Daha küçük!
Tahmin: 62
Daha büyük!
Tahmin: 67
Bildin! 4 denemede buldun.
```

BÖLÜM 6

Metotlar

Bu Bölümde Öğreneceklerin

- Kodu neden metotlara böldüğümüzü (tekrarı önleme, okunabilirlik)
- Bir metodun parçalarını: `static`, dönüş tipi, ad, parametre listesi, gövde
- Metotların hem ana kodun altında hem klasik `class Program` biçiminde nasıl yazıldığını
- `void` ile değer döndüren metotların farkını, `return` kullanımını ve CS0161 hatasını
- Parametre ile argüman farkını ve çağrı yığınını (call stack)
- Değer ile geçirmeyi; `ref`, `out` ve `in` ile değişkenin kendisini geçirmeyi
- İsteğe bağlı ve adlandırılmış parametreleri, aşırı yüklemeyi (overloading), ifade gövdeli metotları
- Kapsamı (scope), `Math` sınıfını ve `Random` ile rastgele sayı üretmeyi

6.1 Neden metot?

Şimdiye kadar bütün kodu tek bir yere, dosyanın en üstüne yazdık. Program büyüdükçe iki sorun çıkar: Aynı kod parçasını **tekrar tekrar** yazarsın ve ana kod okunamayacak kadar uzar. **Metot** (method), bir işi yapan kod parçasına bir **ad** verip onu istediğin yerden, istediğin kadar **çağırmanı** (call) sağlar. Başka dillerde buna **fonksiyon** (function) denir.

cizgiMetodu.cs

```
CizgiCiz();
Console.WriteLine("  2026-2027 GÜZ DÖNEMİ");
CizgiCiz();
Console.WriteLine("  Programlamaya Giriş");
CizgiCiz();

static void CizgiCiz()
{
    for (int i = 0; i < 24; i++)
    {
        Console.Write("=");
    }
    Console.WriteLine();
}
```

Çıktı

```
========================
  2026-2027 GÜZ DÖNEMİ
========================
  Programlamaya Giriş
========================
```

Döngüyü bir kere yazdık, üç kere çağırdık. Çizginin uzunluğunu 30 yapmak istersen tek bir yeri değiştirmen yeter. Metot kullanmanın faydaları:

- **Tekrar yok:** Kod bir kere yazılır, çok kere kullanılır (*DRY: Don't Repeat Yourself*).

- **Okunabilirlik:** `CizgiCiz();` satırı ne yaptığını adıyla anlatır; içindeki döngüyü okumana gerek kalmaz.
- **Kolay test ve hata ayıklama:** Her metodu ayrı ayrı deneyebilirsin; hata varsa nerede arayacağını bilirsin.

Aslında metotları baştan beri kullanıyorsun: `Console.WriteLine`, `Math.Sqrt`, `Convert.ToInt32` ve `int.TryParse` hazır metotlardır. Bu bölümde **kendi** metotlarını yazacaksın.

> **Not: Ana kodun altındaki metot nedir?**
>
> Dosyanın başındaki satırları çalıştırırken derleyici onları görünmez bir `Main` metodunun içine koyar. `CizgiCiz` gibi ana kodun altına yazdığın metotlar da bu gizli `Main`'in içinde tanımlanmış **yerel fonksiyonlardır** (local function). Kitapta hepsine "metot" diyeceğiz. Aradaki farkı birkaç yerde hissedeceksin (`public` yazamamak, aynı ada iki metot yazamamak, yığın izindeki garip adlar); her birini yeri geldiğinde söyleyeceğim.

6.2 Metodun anatomisi

Bir metot tanımı (method declaration) beş parçadan oluşur:

```
static  int     Topla  (int a, int b)   {  return a + b;  }
------  ------  -----  ---------------   ------------------
static  dönüş   ad     parametre         gövde
        tipi           listesi
```

Parça	Örnekte	Anlamı
`static`	`static`	Ana kodun altındaki metotta: metot, ana koddaki değişkenlere dokunamaz, yalnız parametreleriyle çalışır. Klasik biçimde: metot nesne oluşturmadan çağrılabilir. Şimdilik hep yazacağız (nedeni sonraki başlıkta).
Dönüş tipi (return type)	`int`	Metodun geri verdiği değerin tipi. Hiçbir şey döndürmüyorsa `void`.
Metot adı	`Topla`	PascalCase, genellikle fiil: `OrtalamaHesapla`, `Yazdir`, `AsalMi`.
Parametre listesi	`(int a, int b)`	Metodun dışarıdan aldığı değerler; her biri **tip + ad**. Boş olabilir: `()`.
Gövde (body)	`{ return a + b; }`	Metot çağrıldığında çalışan komutlar.

Metot adı PascalCase, parametre ve yerel değişken adları camelCase yazılır; tanımlayıcılarda Türkçe karakter kullanma (`OrtalamaHesapla`). Klasik biçimde `static`'in önüne bir de erişim belirteci (access modifier) gelebilir; ayrıntısı Bölüm 9'da. Metodu ana kodun **altına** yazıyoruz; bu bir zorunluluktan çok düzendir, derleyici üstte mi altta mı olduğuna bakmaz.

ilkMetot.cs

```
int x = Topla(3, 4);
Console.WriteLine(x);
Console.WriteLine(Topla(10, 20) * 2);
Console.WriteLine(Topla(Topla(1, 2), 3));

static int Topla(int a, int b)
{
    int sonuc = a + b;
    return sonuc;
}
```

```
Çıktı
7
60
6
```

`Topla(3, 4)` çağrısında `a` = 3, `b` = 4 olur, gövde çalışır ve `return` ile 7 değeri çağrının yapıldığı yere döner. Yani `int x = Topla(3, 4);` satırı, metot bittikten sonra `int x = 7;` gibi davranır. Değer döndüren bir çağrıyı, o tipte bir değerin kullanılabileceği her yerde kullanabilirsin: bir ifadenin içinde (`Topla(10, 20) * 2`), `Console.WriteLine`'ın içinde, hatta başka bir çağrının argümanı olarak (`Topla(Topla(1, 2), 3)`: önce içteki çağrı 3 döndürür, sonra `Topla(3, 3)` çalışır).

6.3 Klasik biçimde metotlar

Bölüm 1'de gördüğün `class Program` biçiminde metotlar sınıfın içine, ama `Main`'in **dışına** yazılır; `Main`'den önce ya da sonra olması fark etmez. Sınavda çoğu zaman bu biçim verilir, o yüzden aynı metotları bir de burada yaz:

klasikBicim.cs

```
class Program
{
    static int Topla(int a, int b)
    {
        return a + b;
    }

    static double Ortalama(int a, int b)
    {
        return (a + b) / 2.0;
    }

    static void Main(string[] args)
    {
        int x = Topla(3, 4);
        Console.WriteLine(x);
        Console.WriteLine(Ortalama(7, 8));
    }
}
```

```
Çıktı
7
7,5
```

> **Not: `Main` metodunu artık okuyabilirsin**
>
> `static void Main(string[] args)`: nesne gerektirmeyen (`static`), değer döndürmeyen (`void`), adı `Main` olan ve parametre olarak bir `string` dizisi (`string[] args`, Bölüm 7) alan bir metot. Program çalışmaya buradan başlar. Üst düzey deyimlerde bu metodu derleyici senin yerine yazar.

İki biçim arasındaki farklar:

	Ana kodun altı (yerel fonksiyon)	Klasik `class Program`
Metot nerede?	Ana kodun altında	Sınıfın içinde, `Main`'in dışında
`public` / `private`	Yazılamaz (CS0106)	Yazılabilir; yazılmazsa `private`
`static`	Yazmak şart değil; yazarsan ana koddaki değişkenleri göremez	**Şart**: yoksa `Main` metodu çağıramaz
Aynı ada iki metot (aşırı yükleme)	Yazılamaz (CS0128)	Yazılabilir

`static` yazmayı unutursan klasik biçimde program derlenmez:

```
class Program
{
    int Kare(int n)
    {
        return n * n;
    }

    static void Main()
    {
        Console.WriteLine(Kare(6));
    }
}
```

Derleyici çıktısı – program derlenmez

```
staticHata.cs(10,27): error CS0120: An object reference is required for the non-static field,
method, or property 'Program.Kare(int)'
```

`static` olmayan metotlar **nesneye ait** metotlardır ve bir nesne üzerinden çağrılır; nesneleri ve `static`'in tam anlamını Bölüm 9'da öğreneceksin. O zamana kadar kural basit: **Yazdığın her metodun başına `static` koy.**

6.4 void ve değer döndüren metotlar

Metotlar iki gruba ayrılır:

- **`void` metotlar** bir **iş yapar** (ekrana yazmak gibi), ama geriye değer **vermez**. Çağrısı tek başına bir komuttur: `CizgiCiz();`
- **Değer döndüren metotlar** bir **hesap yapar** ve sonucu `return` ile çağırana **geri verir**. Dönüş tipi, döndürülen değerin tipidir.

donusTipleri.cs

```
Selamla("Ece");
int k = Kare(5);
Kare(9);                        // dönen 81 kullanılmadı, kayboldu
Console.WriteLine($"{k} {Kare(k)}");
Console.WriteLine(Ortalama(7, 8));

static void Selamla(string ad)
{
    Console.WriteLine($"Merhaba {ad}!");
}

static int Kare(int n)
{
    return n * n;
}

static double Ortalama(int a, int b)
{
    return (a + b) / 2.0;
}
```

Çıktı

```
Merhaba Ece!
25 625
7,5
```

- `Kare(9);` derlenir ve çalışır, ama dönen değer hiçbir yerde kullanılmadığı için **kaybolur**. C# bunu hata saymaz.
- `Ortalama`'da `2.0` yerine `2` yazsaydık tam sayı bölmesi yüzünden `7` dönerdi. Dönüş tipinin `double` olması bölmeyi ondalıklı **yapmaz**.
- `return` ifadesinin tipi dönüş tipine uymalıdır: Genişleyen dönüşüm otomatiktir (`double` metotta `return 5;` olur), daralan değildir (`int` metotta `return 2.5;` CS0266 verir).

Sık Yapılan Hata: void metodun "değerini" kullanmak

`void` metot değer üretmediği için bir değişkene atanamaz, ifadenin içinde kullanılamaz:

```
int x = Selamla("Ali");

static void Selamla(string ad)
{
    Console.WriteLine($"Selam {ad}");
}
```

Derleyici çıktısı – program derlenmez

```
voidHata.cs(1,9): error CS0029: Cannot implicitly convert type 'void' to 'int'
```

Sınavda Çıkar: Yazdırmak mı, döndürmek mi?

Soruda "... değerini **döndüren** bir metot yaz" deniyorsa metodun içinde `Console.WriteLine` değil `return` olmalı; yazdırma işini ana kod yapar. Değer döndüren metot daha esnektir: Sonucu yazdırabilir, saklayabilir ya da başka bir hesapta kullanabilirsin. Metot sadece yazdırırsa sonuç ekrana gider ve program onu bir daha kullanamaz.

6.5 return ile erken çıkış

`return` çalıştığı anda metot **biter**; altındaki satırlar çalışmaz ve kontrol çağırana döner. Bu yüzden bir metotta birden fazla `return` olabilir. `void` metottan da değersiz `return;` ile erken çıkılır:

erkenCikis.cs

```
Console.WriteLine($"{HarfNotu(92)} {HarfNotu(70)} {HarfNotu(150)}");
BolumuYaz(17, 5);
BolumuYaz(17, 0);

static string HarfNotu(int puan)
{
    if (puan < 0 || puan > 100)
    {
        return "Geçersiz";
    }
    if (puan >= 85)
    {
        return "AA";
    }
    if (puan >= 60)
    {
        return "CC";
    }
    return "FF";
}

static void BolumuYaz(int a, int b)
{
    if (b == 0)
    {
        Console.WriteLine("Sıfıra bölünemez!");
        return;                    // void metottan erken çıkış
    }
    Console.WriteLine($"{a} / {b} = {a / b}");
}
```

Çıktı

```
AA CC Geçersiz
17 / 5 = 3
Sıfıra bölünemez!
```

`HarfNotu(92)` çağrısında ikinci `if` doğru olur ve `"AA"` döner; `puan >= 60` kontrolü **hiç çalışmaz**. Her `return` metodu bitirdiği için `else` yazmaya gerek kalmadı. `BolumuYaz(17, 0)` çağrısında da `return;` sayesinde bölme satırına hiç gelinmedi. `return`'den hemen sonra, aynı blokta bir komut yazarsan derleyici ona asla ulaşılamayacağını fark eder ve CS0162 uyarısı verir.

"Not all code paths return a value" hatası

Dönüş tipi `void` olmayan bir metotta derleyici, metodun **her olası yoldan** bir değer döndürdüğünden emin olmak ister. Bir yol bile `return`'süz biterse program derlenmez (CS0161). İşin ilginç yanı, derleyici koşulların değerini **hesaplamaz**:

eksikReturn.cs

```
Console.WriteLine(Isaret(-3));

static string Isaret(int n)
{
    if (n >= 0)
    {
        return "pozitif veya sıfır";
    }
    if (n < 0)
    {
        return "negatif";
    }
}
```

Derleyici çıktısı – program derlenmez

```
eksikReturn.cs(3,15): error CS0161: 'Isaret(int)': not all code paths return a value
```

Sen biliyorsun ki iki koşuldan biri mutlaka doğrudur. Derleyici ise iki ayrı `if` görür ve "ikisi de yanlış olursa ne dönecek?" diye sorar. Çözüm: İkinci `if`'i kaldırıp metodu düz bir `return "negatif";` ile bitirmek ya da `if-else` kullanmak; `else` dalı olan bir `if-else`'te iki daldan birinin mutlaka çalışacağını derleyici bilir.

> **Sınavda Çıkar**
>
> "Aşağıdaki metot derlenir mi?" sorularında metodun **son satırına** bak. Son komut `return` değilse ve son `if`'in `else` dalı yoksa cevap büyük ihtimalle CS0161'dir. Döngünün içindeki `return` de yetmez: Döngü hiç dönmeyebilir, döngüden sonra da bir `return` gerekir (aşağıdaki `AsalMi`'ye bak).

6.6 İfade gövdeli metotlar

Gövdesi tek bir `return` ifadesinden oluşan metodu `=>` ile kısaca yazabilirsin. `return` ve süslü parantezler gider, ifade doğrudan `=>`'in sağına yazılır:

ifadeGovdeli.cs

```
Console.WriteLine(Carp(6, 7));
Console.WriteLine(CiftMi(10));
Selamla("Ece");

static int Carp(int a, int b) => a * b;

static bool CiftMi(int n) => n % 2 == 0;

static void Selamla(string ad) => Console.WriteLine($"Merhaba {ad}!");
```

Çıktı

```
42
True
Merhaba Ece!
```

Davranış tam olarak aynıdır; sadece yazım kısalır. `void` metotta `=>`'in sağında bir çağrı olur. Gövde birkaç satırsa süslü parantezli biçimi kullan. Bundan sonraki örneklerde kısa metotları çoğunlukla bu biçimde göreceksin.

6.7 Parametre ve argüman

Bu iki kelime sık karıştırılır, ama sınavda ayrımı sorulur:

	Parametre (parameter)	Argüman (argument)
Nerede?	Metot **tanımında**	Metot **çağrısında**
Nedir?	Değeri bekleyen bir değişken	Gönderilen gerçek değer
Örnek	`Cizgi(char karakter, int uzunluk)`	`Cizgi('*', 10)`

Çağrıda argümanlar **sırayla** parametrelere atanır: birinci argüman birinci parametreye, ikinci argüman ikinci parametreye... Bu yüzden argümanların **sayısı**, **sırası** ve **tipi** parametrelerle uyumlu olmalıdır. Argüman bir değişken, bir sabit ya da `n + 1` gibi bir ifade olabilir; önce değeri hesaplanır, sonra parametreye konur.

parametreler.cs

```
Cizgi('*', 10);
int n = 3;
Cizgi('#', n + 1);
Console.WriteLine(Math.Sqrt(16));       // int argüman, double parametre

static void Cizgi(char karakter, int uzunluk)
{
    for (int i = 0; i < uzunluk; i++)
    {
        Console.Write(karakter);
    }
    Console.WriteLine();
}
```

Çıktı

```
**********
####
4
```

`Cizgi('#', n + 1)` çağrısında ana koddaki değişkenin adı `n`, metottaki parametrenin adı `uzunluk`: Parametre adlarının argümanların adlarıyla hiçbir ilgisi yoktur. `Math.Sqrt(16)` çağrısında `int` argüman `double` parametreye **otomatik** dönüştürüldü (genişleyen dönüşüm). Tersi olmaz: `int` parametreli bir metoda `2.5` gönderirsen CS1503, eksik argüman gönderirsen CS7036, fazla argüman gönderirsen CS1501 derleme hatası alırsın.

6.8 Metot çağrıldığında ne olur?

Bir metot çağrıldığında sırasıyla şunlar olur:

1. Argümanların değerleri hesaplanır ve metodun parametrelerine **kopyalanır**.
2. Çağıran taraf olduğu yerde **bekler**; çağrılan metodun gövdesi çalışır.
3. `return` ile (ya da `void` metotta gövdenin sonuna gelince) kontrol, çağrının yapıldığı **noktaya** geri döner. Dönen değer varsa çağrı ifadesinin yerine geçer.

Metotlar başka metotları da çağırabilir. Akışı görmek için her metoda mesaj yazdıralım:

cagriAkisi.cs

```
Console.WriteLine("ana kod başladı");
int s = KareToplami(3, 4);
Console.WriteLine($"ana kod: sonuç = {s}");

static int Kare(int n)
{
    Console.WriteLine($"    Kare({n}) başladı");
    return n * n;
}

static int KareToplami(int a, int b)
{
    Console.WriteLine($"  KareToplami({a}, {b}) başladı");
    int t = Kare(a) + Kare(b);
    Console.WriteLine($"  KareToplami {t} döndürüyor");
    return t;
}
```

Çıktı

```
ana kod başladı
  KareToplami(3, 4) başladı
    Kare(3) başladı
    Kare(4) başladı
  KareToplami 25 döndürüyor
ana kod: sonuç = 25
```

Çağrı yığını

Program, hangi metodun hangisini çağırdığını **çağrı yığını** (call stack) denen bir yapıda tutar. Her çağrıda yığının en üstüne o metoda ait bir **çerçeve** (stack frame) eklenir; çerçevede metodun parametreleri ve yerel değişkenleri bulunur. Metot bitince çerçevesi atılır ve içindeki değişkenler **yok olur**. Çalışan metot her zaman en üsttekidir. `Kare(3)` çalışırken yığın şöyledir:

```
+---------------------------+
| Kare         n=3          |   <- en üst: şu an çalışan
+---------------------------+
| KareToplami  a=3, b=4     |   <- Kare'nin bitmesini bekliyor
+---------------------------+
| Main                      |   <- KareToplami'nin bitmesini bekliyor
+---------------------------+
```

Bir metot kendini de çağırabilir (özyineleme, Bölüm 15); o zaman her çağrı yığına yeni bir çerçeve ekler. Yığını aslında baştan beri görüyorsun: Yakalanmamış bir istisna (exception) oluştuğunda program o anki çağrı yığınını ekrana döker (*stack trace*). Bu kez klasik biçimle yazalım:

yiginIzi.cs

```
class Program
{
    static int Bol(int a, int b)
    {
        return a / b;
    }

    static int Ortalama(int toplam, int adet)
    {
        return Bol(toplam, adet);
    }

    static void Main()
    {
        Console.WriteLine(Ortalama(30, 3));
        Console.WriteLine(Ortalama(30, 0));
    }
}
```

Çıktı – program yakalanmamış istisna ile sonlanır

```
10
Unhandled exception. System.DivideByZeroException: Attempted to divide by zero.
   at Program.Bol(Int32 a, Int32 b) in yiginIzi.cs:line 5
   at Program.Ortalama(Int32 toplam, Int32 adet) in yiginIzi.cs:line 10
   at Program.Main() in yiginIzi.cs:line 16
```

İpucu: Yığın izini okumak

`at` satırlarını **yukarıdan aşağıya** oku: En üstteki satır hatanın **oluştuğu** yer (`Bol`), altındakiler sırayla onu **çağıranlar** (`Ortalama`, sonra `Main`). Satırın sonundaki `line N` numarası da dosya yolu da senin bilgisayarında kendi klasörünü gösterir. Aynı programı ana kodun altındaki metotlarla yazsaydın metot adları `<<Main>$>g__Bol|0_4` gibi görünürdü: Derleyicinin yerel fonksiyona verdiği addır, aradaki `Bol`'u oku.

6.9 Değer ile geçirme

Varsayılan olarak argümanlar **değer ile** geçirilir (*pass-by-value*): Metoda değişkenin kendisi değil, değerinin bir **kopyası** gider. Parametre, metodun kendi yerel değişkenidir. Metot içinde parametreyi değiştirmek çağırandaki değişkeni **etkilemez**:

degerIleGecirme.cs

```
int sayi = 5;
Artir(sayi);
Console.WriteLine($"ana kod: sayi = {sayi}");

int x = 1;
int y = 2;
YerDegistir(x, y);
Console.WriteLine($"ana kod: x = {x}, y = {y}");

static void Artir(int n)
{
    n = n + 10;
    Console.WriteLine($"Artir içinde n = {n}");
}

static void YerDegistir(int a, int b)
{
```

```
    int gecici = a;
    a = b;
    b = gecici;
    Console.WriteLine($"YerDegistir içinde a = {a}, b = {b}");
}
```

Çıktı

```
Artir içinde n = 15
ana kod: sayi = 5
YerDegistir içinde a = 2, b = 1
ana kod: x = 1, y = 2
```

`YerDegistir` (swap) kendi içinde değerleri değiştirdi, ama ana koddaki `x` ve `y` hiç etkilenmedi:

An	ana kod: x	ana kod: y	`YerDegistir`: a	`YerDegistir`: b
`YerDegistir(x, y)` çağrıldı, değerler kopyalandı	1	2	1	2
`a` ile `b` yer değiştirdi	1	2	2	1
Metot bitti, çerçevesi silindi	1	2	—	—

Sonucu çağırana iletmenin ilk yolu **`return`**'dür: `Artir` metodunu `static int Artir(int n) { return n + 10; }` biçiminde yazıp çağıran tarafta dönen değeri geri atarsın: `sayi = Artir(sayi);`. Bu kural `int`, `double`, `bool`, `char` gibi **değer tipleri** için geçerlidir. Dizi ve nesneler (Bölüm 7 ve 9) başvuru tipidir; onlarda kopyalanan şey bir başvurudur ve metottaki değişiklikler dışarıdan görünebilir.

> **Sık Yapılan Hata: Aynı isim, aynı değişken değildir**
>
> `Artir` metodunun parametresinin adını `sayi` yapsan bile sonuç değişmez: Metottaki `sayi` ile ana koddaki `sayi` **iki ayrı değişkendir**, sadece adları aynıdır. Metot, kendi kopyasını değiştirir.

ref: değişkenin kendisini geçirmek

Metodun çağıran tarafın değişkenini **gerçekten** değiştirmesini istiyorsan parametrenin önüne `ref` yaz, çağrıda da argümanın önüne `ref` ekle. Artık metoda kopya değil, değişkenin **kendisi** verilir; metottaki `a` ile ana koddaki `x` aynı bellek yerinin iki adıdır:

refSwap.cs

```
int x = 1;
int y = 2;
YerDegistir(ref x, ref y);
Console.WriteLine($"x = {x}, y = {y}");

static void YerDegistir(ref int a, ref int b)
{
    int gecici = a;
    a = b;
    b = gecici;
}
```

Çıktı

```
x = 2, y = 1
```

Kurallar: `ref` hem tanımda hem çağrıda yazılır; argüman bir **değişken** olmalıdır (`ref 5` olmaz) ve çağrıdan önce **değer almış** olmalıdır (aksi hâlde CS0165).

Sık Yapılan Hata: Çağrıda ref'i unutmak

Tanımda `ref` varsa çağrıda da olmak zorundadır. Unutursan program derlenmez:

```
int x = 1;
int y = 2;
YerDegistir(x, y);

static void YerDegistir(ref int a, ref int b)
{
    int gecici = a;
    a = b;
    b = gecici;
}
```

Derleyici çıktısı – program derlenmez

```
refUnutmak.cs(3,13): error CS1620: Argument 1 must be passed with the 'ref' keyword
refUnutmak.cs(3,16): error CS1620: Argument 2 must be passed with the 'ref' keyword
```

out: birden fazla değer döndürmek

`return` tek değer döndürür. Bölüm ve kalan gibi iki sonuç gerekiyorsa `out` parametre kullanılır. `out`, `ref` gibi değişkenin kendisini geçirir, ama yönü tersidir: Metot değişkeni **doldurur**, çağıranın önceden değer vermesi gerekmez.

outOrnek.cs

```
BolVeKalan(17, 5, out int b, out int k);
Console.WriteLine($"bölüm = {b}, kalan = {k}");

if (int.TryParse("123", out int sayi))
{
    Console.WriteLine($"sayi = {sayi}");
}
if (!int.TryParse("12a", out int hatali))
{
    Console.WriteLine($"çevrilemedi, hatali = {hatali}");
}

static void BolVeKalan(int bolunen, int bolen, out int bolum, out int kalan)
{
    bolum = bolunen / bolen;
    kalan = bolunen % bolen;
}
```

Çıktı

```
bölüm = 3, kalan = 2
sayi = 123
çevrilemedi, hatali = 0
```

- Metot bitmeden her `out` parametreye **mutlaka** atama yapmalıdır; yapmazsa CS0177.
- Çağrıda `out int b` yazarak değişkeni aynı anda tanımlayabilirsin. Değeri istemiyorsan `out _` yaz (atma değişkeni, *discard*).
- Bölüm 3'te kullandığın `int.TryParse(metin, out int sayi)` tam olarak bu kalıptır: Dönüş değeri `bool` (çevirme başarılı mı?), sonuç `out` ile gelir. Başarısız olursa `out` değişkeni 0 olur.

in: salt okunur başvuru

`in` parametre de değişkeni kopyalamadan geçirir, ama metot onu **değiştiremez** (atama yapmak CS8331 verir). Büyük veri yapılarını kopyalamamak için kullanılır; şimdilik tanıman yeter.

Parametre	Çağrıda yazılır mı?	Çağıran değişken önceden değer almalı mı?	Metot ne yapabilir?
(hiçbiri)	Hayır	Evet (kopyası gider)	Kopyayı değiştirir, dışarı yansımaz
`ref`	`ref x`	Evet	Okur ve yazar, dışarı yansır
`out`	`out x`	Hayır	Yazar (atama **zorunlu**), dışarı yansır
`in`	`x` (`in x` isteğe bağlı)	Evet	Yalnızca okur

> **Sınavda Çıkar: Anahtar sözcüğe bak**
>
> "Metot çağrıldıktan sonra değişkenin değeri nedir?" sorusunda önce parametrede `ref` ya da `out` var mı diye bak. Yoksa çağıran değişken **değişmez**, metot içinde ne yapılırsa yapılsın. Varsa değişir; ama çağrıda `ref`/`out` yazılmamışsa program zaten derlenmez. Bir metot hem `return` ile değer döndürüp hem `ref`/`out` ile değişken değiştirebilir: Her ikisini ayrı ayrı izle.

Demet (tuple): birden çok değeri tek `return` ile döndürmek

`out` iki sonuç döndürmeye yarar ama çağıran önce değişkenleri tanımlamak zorunda kalır. Daha rahat bir yol **demet**'tir (tuple): birkaç değeri parantez içinde tek pakete koyar, `(17, 5)` gibi. Metodun dönüş tipini parantezli bir liste olarak yazarsan metot tek `return` ile birden çok değer döndürür; her değere bir ad da verebilirsin. Çağıran taraf paketi `var (b, k) = ...` ile parçalara ayırır (*deconstruction*):

demet.cs

```
var (b, k) = Bol(17, 5);
Console.WriteLine($"bölüm = {b}, kalan = {k}");

(int bolum, int kalan) sonuc = Bol(20, 6);
Console.WriteLine(sonuc.bolum + " " + sonuc.kalan);
Console.WriteLine(sonuc);
Console.WriteLine(sonuc.Item1);

var (_, sadeceKalan) = Bol(9, 4);
Console.WriteLine(sadeceKalan);

var (kalan, bolum) = Bol(17, 5);        // ad değil SIRA önemlidir
Console.WriteLine($"kalan = {kalan}");

var (alan, cevre) = Dikdortgen(3.5, 2);
Console.WriteLine($"alan = {alan:F1}, çevre = {cevre:F1}");

static (int bolum, int kalan) Bol(int bolunen, int bolen)
{
    return (bolunen / bolen, bolunen % bolen);
}

static (double alan, double cevre) Dikdortgen(double en, double boy)
    => (en * boy, 2 * (en + boy));
```

```
Çıktı
bölüm = 3, kalan = 2
3 2
(3, 2)
3
1
kalan = 3
alan = 7,0, çevre = 11,0
```

- Demeti ekrana yazdırırsan `(3, 2)` gibi parantezli görünür. Adsız elemanlara `Item1`, `Item2` ile de ulaşılır; ad vermek yalnızca okunabilirlik içindir.
- Parçalarken eşleşme **sıraya** göredir. `var (kalan, bolum) = Bol(17, 5);` yazarsan `kalan` değişkeni demetin **birinci** elemanını, yani bölümü (3) alır.
- İstemediğin parçayı `_` ile atlarsın (`var (_, sadeceKalan) = ...`).

	`out` ile	Demet ile
Tanım	`void Bol(int a, int b, out int bolum, out int kalan)`	`(int bolum, int kalan) Bol(int a, int b)`
Çağrı	`Bol(17, 5, out int b, out int k);`	`var (b, k) = Bol(17, 5);`
Ne zaman?	`TryParse` gibi "başarılı mı?" sorusu + sonuç	Birden çok sonucu paket olarak döndürmek

> **Sınavda Çıkar: Demette ad değil sıra**
>
> Demet bir paket değerdir: elemanlar yan yana durur ve `var (x, y) = ...` ile **sırayla** eşleşir. Dönüş tipinde yazdığın adlar (`bolum`, `kalan`) çağıran taraftaki değişken adlarını belirlemez. Çıktı sorularında parçalama satırında adların sırasına bak.

6.10 İsteğe bağlı ve adlandırılmış parametreler

Parametreye tanım sırasında bir **varsayılan değer** verirsen o parametre **isteğe bağlı** (optional) olur: Çağrıda yazılmazsa varsayılan değer kullanılır. Argümanı parametre adıyla vermek de mümkündür (**adlandırılmış argüman**, named argument): `ad: "Ece"`.

istegeBagli.cs

```
Selamla("Ece");
Selamla("Ece", "Dr.");
Selamla("Ece", tekrar: 2);
Selamla(tekrar: 1, ad: "Ali", unvan: "Prof.");
Console.WriteLine(Kdv(200));
Console.WriteLine(Kdv(200, 0.10));

static void Selamla(string ad, string unvan = "Sayın", int tekrar = 1)
{
    for (int i = 0; i < tekrar; i++)
    {
        Console.WriteLine($"{unvan} {ad}, hoş geldiniz.");
    }
}

static double Kdv(double fiyat, double oran = 0.20)
{
    return fiyat * oran;
}
```

Çıktı

```
Sayın Ece, hoş geldiniz.
Dr. Ece, hoş geldiniz.
Sayın Ece, hoş geldiniz.
Sayın Ece, hoş geldiniz.
Prof. Ali, hoş geldiniz.
40
20
```

- `Selamla("Ece", tekrar: 2)` çağrısında `unvan` atlandı ve varsayılan `"Sayın"` kullandı. Ortadaki bir isteğe bağlı parametreyi atlayıp sonrakine değer vermenin yolu adlandırılmış argümandır.
- Adlandırılmış argümanlar **istediğin sırada** yazılabilir. `Kdv(fiyat: 200, oran: 0.10)` gibi çağrılar, hangi sayının ne olduğunu okuyana söyler.
- Varsayılan değer **derleme zamanında bilinen bir sabit** olmalıdır (`0.20`, `"Sayın"`, `true`); `DateTime.Now` gibi çağrılar yazılamaz.

Sık Yapılan Hata: İsteğe bağlı parametre sonda olmak zorunda

İsteğe bağlı parametreler, zorunlu parametrelerin **hepsinden sonra** gelmelidir. Zorunlu parametre isteğe bağlının arkasında kalırsa program derlenmez:

```
Selamla("Dr.", "Ece");

static void Selamla(string unvan = "Sayın", string ad)
{
    Console.WriteLine($"{unvan} {ad}");
}
```

Derleyici çıktısı – program derlenmez

```
istegeBagliSira.cs(3,54): error CS1737: Optional parameters must appear after all required
parameters
```

6.11 Aşırı yükleme (overloading)

Aynı sınıfta, **aynı adı** taşıyan ama **parametre listeleri farklı** olan birden fazla metot tanımlayabilirsin. Buna **aşırı yükleme** (overloading) denir. Derleyici, argümanlara bakarak hangi sürümün çalışacağına **derleme zamanında** karar verir.

Aşırı yükleme **gerçek metotlar** gerektirir. Ana kodun altındaki yerel fonksiyonlar aynı adı iki kez taşıyamaz:

```
Console.WriteLine(Topla(1, 2));

static int Topla(int a, int b)
{
    return a + b;
}

static double Topla(double a, double b)
{
    return a + b;
}
```

Derleyici çıktısı – program derlenmez

```
yerelYukleme.cs(8,15): error CS0128: A local variable or function named 'Topla' is already defined
in this scope
```

Bu yüzden aşırı yüklemeyi klasik biçimde yazacağız:

asiriYukleme.cs

```
class Program
{
    static int Topla(int a, int b)
    {
        Console.Write("[int,int] ");
        return a + b;
    }

    static int Topla(int a, int b, int c)
    {
        Console.Write("[int,int,int] ");
        return a + b + c;
    }

    static double Topla(double a, double b)
    {
        Console.Write("[double,double] ");
        return a + b;
    }

    static void Main()
    {
        Console.WriteLine(Topla(2, 3));
        Console.WriteLine(Topla(2, 3, 4));
        Console.WriteLine(Topla(2.5, 1.5));
        Console.WriteLine(Topla(2, 3.5));
    }
}
```

Çıktı

```
[int,int] 5
[int,int,int] 9
[double,double] 4
[double,double] 5,5
```

`Topla(2, 3.5)` için `(int, double)` alan bir sürüm yok. Derleyici önce **birebir uyan** sürümü arar; bulamazsa **genişleyen dönüşümle** uyanı seçer: `2` → `2.0` yapılır ve `Topla(double, double)` çağrılır.

Hangi metotların birlikte var olabileceğini **metot imzası** (method signature) belirler: İmza = **metodun adı + parametrelerin tipleri** (sayısı ve sırasıyla). **Dönüş tipi ve parametre adları imzaya dahil değildir.** Bir sınıfta aynı imzalı iki metot olamaz.

Birinci metot	İkinci metot	Geçerli aşırı yükleme mi?
`int F(int a)`	`int F(int a, int b)`	Evet: parametre **sayısı** farklı
`int F(int a)`	`int F(double a)`	Evet: parametre **tipi** farklı
`void F(int a, string s)`	`void F(string s, int a)`	Evet: parametre **sırası** farklı
`void F(int a)`	`void F(ref int a)`	Evet: `ref` de imzanın parçası
`int F(int a)`	`int F(int b)`	**Hayır**: sadece parametre adı farklı
`int F(int a)`	`double F(int a)`	**Hayır**: sadece dönüş tipi farklı
`void F(ref int a)`	`void F(out int a)`	**Hayır** (CS0663): `ref` ile `out` farkı yetmez

```
class Program
{
    static int Hesapla(int x) => x * 2;
    static double Hesapla(int x) => x * 2.5;

    static void Main()
    {
        Hesapla(4);
    }
}
```

Derleyici çıktısı – program derlenmez

```
imzaHata.cs(4,19): error CS0111: Type 'Program' already defines a member called 'Hesapla' with the
same parameter types
```

Dönüş tipinin imzaya dahil olmamasının sebebi `Main`'deki satırda: `Hesapla(4);` çağrısında dönen değer kullanılmıyor, dolayısıyla hangi metodun kastedildiği anlaşılamaz.

İki sürümün ikisi de uygunsa ve hiçbiri diğerinden belirgin biçimde iyi değilse çağrı **belirsiz** (ambiguous) olur:

```
class Program
{
    static void Alan(int a, double b) => Console.WriteLine("int,double");
    static void Alan(double a, int b) => Console.WriteLine("double,int");

    static void Main()
    {
        Alan(3, 2.5);
        Alan(3, 2);
    }
}
```

Derleyici çıktısı – program derlenmez

```
belirsizCagri.cs(9,9): error CS0121: The call is ambiguous between the following methods or
properties: 'Program.Alan(int, double)' and 'Program.Alan(double, int)'
```

`Alan(3, 2.5)` birebir uyar. `Alan(3, 2)` ise iki sürüme de genişleyen dönüşümle uyar: Birincisi ikinci argümanı, ikincisi birinci argümanı `double` yapar. Hiçbiri açıkça daha iyi değildir, derleyici seçemez (CS0121). Çağrıyı `Alan(3, 2.0)` yaparak kendin belirt.

Sınavda Çıkar

- **"Hangisi geçerli bir aşırı yüklemedir?"** Sadece parametre listesine bak: sayı, tip veya sıra farklıysa geçerli. Dönüş tipi ya da parametre adı farkı **yetmez**.
- **"Hangi sürüm çağrılır?"** Önce birebir uyan; yoksa genişleyen dönüşümle uyan (`int` → `long` → `double`). `char` argüman, `int` parametreli sürüme gider. İki sürüm de aynı derecede uyuyorsa derleme hatası vardır.

`Console.WriteLine` her tip için ayrı yazılmış (`WriteLine(int)`, `WriteLine(double)`, `WriteLine(string)`...) aşırı yüklenmiş bir metottur. Bölüm 11'deki **override** (geçersiz kılma) ile karıştırma; o bambaşka bir kavram.

6.12 Kapsam (scope) ve yerel değişkenler

Bir değişkenin programın neresinden kullanılabildiğine **kapsam** (scope) denir. Metodun içinde tanımlanan değişkenlere **yerel değişken** (local variable) denir. Kurallar:

1. Yerel değişken, tanımlandığı satırdan itibaren içinde bulunduğu `{ }` bloğunun **sonuna kadar** geçerlidir. Parametreler, metodun tamamında geçerli yerel değişkenlerdir.
2. `for (int i = ...)` başlığında tanımlanan `i` sadece döngünün içinde yaşar.
3. Farklı metotlardaki aynı isimli değişkenler birbirinden **tamamen bağımsızdır**.
4. Bir değişken hâlâ kapsamdayken, onun adıyla iç blokta yeni bir değişken tanımlanamaz.

kapsam.cs

```
int x = 1;
Yazdir();
Console.WriteLine($"ana kod: x = {x}");
for (int i = 1; i <= 3; i++)
{
    int kare = i * i;              // her turda yeniden oluşur
    Console.Write(kare + " ");
}
Console.WriteLine();               // burada i ve kare artık yok

static void Yazdir()
{
    int x = 100;                   // ana koddaki x ile ilgisi yok
    Console.WriteLine($"Yazdir: x = {x}");
}
```

Çıktı

```
Yazdir: x = 100
ana kod: x = 1
1 4 9
```

Kapsam dışındaki bir değişkeni kullanmak ya da kapsamdaki bir değişkeni yeniden tanımlamak derleme hatasıdır:

```
int x = 5;
for (int i = 0; i < 3; i++)
{
    int x = i;
}
Console.WriteLine(x + i);
```

Derleyici çıktısı – program derlenmez

```
kapsamHata.cs(4,9): error CS0136: A local or parameter named 'x' cannot be declared in this scope
because that name is used in an enclosing local scope to define a local or parameter
kapsamHata.cs(6,23): error CS0103: The name 'i' does not exist in the current context
```

Ana kodun değişkenlerini metodun içinde **kullanmaya** çalışırsan ne olur? Metot `static` değilse derleyici buna izin verir, ama bu kötü bir alışkanlıktır: Metot, yalnız parametrelerine bakarak çalışmalı ki tek başına anlaşılsın ve denensin. `static` yazdığında derleyici bunu engeller:

```
int sayac = 10;
Console.WriteLine(Arttir());

static int Arttir()
{
    return sayac + 1;
}
```

Derleyici çıktısı – program derlenmez

```
statikYakalama.cs(6,12): error CS8421: A static local function cannot contain a reference to
'sayac'.
```

Kapsam kuralı, metotlar arasında veri aktarmanın yolunun parametre ve `return` olduğu anlamına gelir. (Sınıf düzeyindeki değişkenleri, yani alanları, Bölüm 9'da göreceksin.)

> ● **Sık Yapılan Hata: Döngüde hesaplanan değeri döngüden sonra kullanmak**
>
> `for` içinde `int toplam = 0;` tanımlayıp döngüden sonra `toplam`'ı yazdırmaya çalışırsan CS0103 ("name does not exist") alırsın. Üstelik değişken her turda yeniden 0 yapılır. Döngüden sonra kullanacağın değişkeni **döngüden önce** tanımla.

6.13 params: değişken sayıda argüman

Argüman sayısı önceden belli olmayan metotlar için parametrenin önüne `params` yazılır. Böyle bir metoda istediğin kadar argüman verirsin:

```
static int Topla(params int[] sayilar)    // gövdesini Bölüm 7'de yazacaksın

Topla();              // argümansız
Topla(5);             // tek argüman
Topla(1, 2, 3, 4);    // istediğin kadar
```

`params` parametresi bir dizidir (`int[]`, Bölüm 7) ve metodun **son** parametresi olmak zorundadır. Hazır metotlardan `string.Format` ve `Console.WriteLine("{0} {1}", a, b)` bu yolla çalışır. Şimdilik yalnızca tanımanı istiyoruz.

> ● **Yeni C# Notu: `params` artık yalnız dizi değil**
>
> C# 13'ten itibaren `params` parametresi dizinin yanında `List<int>`, `ReadOnlySpan<int>` gibi koleksiyon tipleriyle de yazılabilir. Çağrı biçimi aynı kalır: `Topla(1, 2, 3)`.

6.14 Algoritmaları metotlara dönüştürmek

Bölüm 5'te döngülerle yazdığın asal sayı, faktöriyel, EBOB ve basamak toplamı algoritmaları metot olmaya biçilmiş kaftandır: Her biri bir girdi alır, bir sonuç üretir.

algoritmalar.cs

```
Console.Write("40'a kadar asallar: ");
for (int i = 1; i <= 40; i++)
{
    if (AsalMi(i))
    {
        Console.Write(i + " ");
    }
}
Console.WriteLine();
Console.WriteLine($"20! = {Faktoriyel(20)}");
int x = 48;
int y = 36;
int e = Ebob(x, y);
Console.WriteLine($"ebob = {e}, ekok = {x / e * y}");
Console.WriteLine($"x = {x}, y = {y}");
Console.WriteLine($"2026 basamak toplamı: {BasamakToplami(2026)}");
Console.WriteLine($"{ArtikYilMi(2024)} {ArtikYilMi(1900)}");

static bool AsalMi(int n)
{
    if (n < 2)
    {
        return false;
    }
    for (int i = 2; i * i <= n; i++)
    {
        if (n % i == 0)
        {
            return false;          // bölen bulundu, hemen çık
        }
    }
    return true;
}

static long Faktoriyel(int n)
{
    long sonuc = 1;
    for (int i = 2; i <= n; i++)
    {
        sonuc *= i;
    }
    return sonuc;
}

static int Ebob(int a, int b)
{
    while (b != 0)                 // Öklid algoritması
    {
        int kalan = a % b;
        a = b;
        b = kalan;
    }
    return a;
}

static int BasamakToplami(int n)
{
```

```
    int toplam = 0;
    while (n > 0)
    {
        toplam += n % 10;
        n /= 10;
    }
    return toplam;
}

static bool ArtikYilMi(int yil) =>
    (yil % 4 == 0 && yil % 100 != 0) || yil % 400 == 0;
```

Çıktı

```
40'a kadar asallar: 2 3 5 7 11 13 17 19 23 29 31 37
20! = 2432902008176640000
ebob = 12, ekok = 144
x = 48, y = 36
2026 basamak toplamı: 10
True False
```

- `AsalMi`'de `return false;` döngünün içinde: Bölen bulunur bulunmaz metot biter; `break`'e ve bayrak değişkenine gerek kalmaz. Döngü bölen bulamadan biterse sondaki `return true;` çalışır.
- `Faktoriyel` `long` döndürür, çünkü 13! `int`'e sığmaz. 20! `long`'a sığan en büyük faktöriyeldir; 21! sessizce taşar (Bölüm 2).
- `Ebob` parametreleri `a` ve `b`'yi değiştiriyor; ama değer ile geçirme sayesinde ana koddaki `x` ve `y` aynen kaldı.
- `ArtikYilMi` gibi "evet/hayır" sorusuna cevap veren metotlar `bool` döndürür. Adlarını bir soru gibi koymak gelenektir: `AsalMi`, `CiftMi`, `GecerliMi`. 1900, 100'e bölünüp 400'e bölünmediği için artık yıl değildir.

İpucu: Acemi bool kalıpları

`if (n % 2 == 0) { return true; } else { return false; }` yerine doğrudan `return n % 2 == 0;` yaz; karşılaştırmanın kendisi zaten `true` ya da `false`'tur. Aynı şekilde `if (AsalMi(n) == true)` yerine `if (AsalMi(n))`, `if (AsalMi(n) == false)` yerine `if (!AsalMi(n))` yaz.

İpucu: Metodu belgelemek

Metodun hemen üstüne `///` yazınca Visual Studio bir `<summary>` iskeleti oluşturur; sen de açıklamayı doldurursun. Metodun adının üzerine fareyi getirince bu açıklama görünür:

```
/// <summary>Bir sayının karesini döndürür.</summary>
/// <param name="n">Karesi alınacak sayı</param>
/// <returns>n * n</returns>
static int Kare(int n) => n * n;
```

6.15 Hazır metotlar: Math ve Random

Bölüm 2'de kullandığın `Math.Sqrt`, `Math.Pow`, `Math.Max` gibi metotlar senin yazdıklarından farksızdır: `Math` adlı sınıfın içinde tanımlanmış `static` metotlardır. Başka bir sınıftaki `static` metodu `SinifAdi.MetotAdi(...)` biçiminde çağırırsın; bu yüzden `Console.WriteLine` de öyle yazılır. Bir metodu doğru kullanmak için imzasını, özellikle **dönüş tipini** bilmelisin:

Metodun imzası (sadeleştirilmiş)	Dikkat
`static double Sqrt(double d)`	`Math.Sqrt(16)` → `4`; `int` versen de `double` döner
`static double Pow(double x, double y)`	`Math.Pow(2, 3)` → `8` (`double`)
`static double Round(double d)`	`int` değil `double` döner; `int`'e atamak için `(int)` gerekir
`int Abs(int)`, `double Abs(double)` …	Aşırı yüklenmiş: `Math.Abs(-7)` → `7`, `Math.Abs(-7.5)` → `7,5`
`int Max(int, int)`, `double Max(double, double)` …	`Math.Max(3, 8.0)` → `8` (genişleyen dönüşüm, `double`)

mathKullanimi.cs

```
double kok = Math.Sqrt(16);
double us = Math.Pow(2, 10);
int yuvarla = (int)Math.Round(2.7);
Console.WriteLine($"{kok} {us} {yuvarla}");
Console.WriteLine($"{Math.Abs(-7)} {Math.Abs(-7.5)}");
Console.WriteLine($"{Math.Max(3, 8.0)} {Math.Min(3, 8)}");
Console.WriteLine($"{Math.Round(2.5)} {Math.Round(3.5)}");
```

Çıktı

```
4 1024 3
7 7,5
8 3
2 4
```

`Math.Round(2.5)` sonucu 2'dir: Varsayılan yuvarlama, .5'i en yakın **çift** sayıya götürür (banker yuvarlaması).

> **Sınavda Çıkar: Dönüş tipini bilmek zorundasın**
>
> `int k = Math.Sqrt(16);` ve `int r = Math.Round(2.7);` satırları **derlenmez** (CS0266): İkisi de `double` döndürür ve `double` `int`'e otomatik daralmaz. Doğrusu `int k = (int)Math.Sqrt(16);` ve `int r = (int)Math.Round(2.7);`.

Rastgele sayı üretmek için de hazır bir sınıf var: `Random`. Onu bir nesne olarak oluşturup `Next` metodunu çağırırsın (nesneleri Bölüm 9'da öğreneceksin, şimdilik kalıp olarak kullan). `Next(1, 7)` 1 ile 7 arasında bir tam sayı verir; **alt sınır dahil, üst sınır hariçtir**, yani 1, 2, 3, 4, 5, 6 çıkar. `new Random(42)` ile verdiğin sayı **tohumdur** (seed): Aynı tohum her çalıştırmada ve her bilgisayarda aynı diziyi üretir. Hata ayıklarken ve sınav sorularında bu çok işe yarar.

zarAt.cs

```
Random zar = new Random(42);
for (int i = 0; i < 8; i++)
{
    Console.Write(ZarAt(zar) + " ");
}
Console.WriteLine();

static int ZarAt(Random r)
{
    return r.Next(1, 7);
}
```

Çıktı
```
5 1 1 4 2 2 5 4
```

Tohum vermezsen (`new Random()`) her çalıştırmada farklı bir dizi gelir:

zarRastgele.cs
```
Random zar = new Random();
Console.WriteLine($"Zar: {zar.Next(1, 7)}");
```

Çıktı
```
Zar: 5
```

Bu programın çıktısı her çalıştırmada değişir; **senin çıktın farklı olur**.

Sık Yapılan Hata: Next'in üst sınırı dahil değildir

Zar için `Next(1, 6)` yazarsan 6 hiç çıkmaz, çünkü üst sınır hariçtir. Zarın yüzleri 1'den 6'ya kadarsa doğrusu `Next(1, 7)`'dir.

Bölüm Özeti – Bilmen Gerekenler

- **Metot**, bir işi yapan ve adıyla çağrılan kod bloğudur. Tekrarı önler, kodu okunur ve test edilebilir yapar.
- Anatomi: `static dönüşTipi Ad(parametreler) { gövde }`. Ana kodun altına yazılan metot gizli `Main`'in yerel fonksiyonudur; klasik biçimde metot `class Program` içinde, `Main`'in dışında durur ve `static` olmalıdır.
- `void` metot değer döndürmez ve ifade içinde kullanılamaz. Değer döndüren metot `return ifade;` ile sonucu çağırana verir; kullanılmayan dönüş değeri kaybolur. Tek ifadelik metot `=> ifade;` ile yazılabilir.
- `return` metodu **hemen** bitirir. `void` metotta `return;` erken çıkış sağlar.
- Değer döndüren metodun **her yolu** `return` ile bitmeli; yoksa CS0161. Derleyici `if` koşullarının değerini hesaplamaz.
- **Parametre** tanımdaki değişken, **argüman** çağrıdaki değerdir. Sayı, sıra ve tip uyumlu olmalı; genişleyen dönüşüm otomatik.
- Her çağrı **çağrı yığınına** bir çerçeve ekler; metot bitince yerel değişkenleri yok olur. Yığın izinde en üst satır hatanın oluştuğu metottur.
- Varsayılan geçirme **değer iledir**: Parametreyi değiştirmek çağırandaki değişkeni etkilemez. Kendisini geçirmek için `ref` (okur-yazar), iki değer döndürmek için `out` (metot doldurur, `int.TryParse` gibi), salt okunur için `in`; `ref` ve `out` çağrıda da yazılır.
- **Demet** (`(int bolum, int kalan) Bol(...)`) birden çok değeri tek `return` ile döndürür; `var (b, k) = Bol(17, 5);` ile parçalanır ve eşleşme adla değil **sıra** iledir.
- **İsteğe bağlı parametre** (`oran = 0.20`) zorunlu parametrelerden sonra gelir; **adlandırılmış argüman** (`oran: 0.10`) sırayı ve atlamayı kolaylaştırır.
- **Aşırı yükleme:** aynı ad, farklı parametre listesi; yalnız klasik metotlarda mümkündür. **İmza** = ad + parametre tipleri (`ref` dahil); dönüş tipi ve parametre adları imzaya dahil **değil**. Eşit derecede uyan iki sürüm belirsiz çağrı (CS0121) olur.
- Yerel değişken tanımlandığı bloğun sonuna kadar yaşar. Farklı metotlardaki aynı isimli değişkenler bağımsızdır.

- `params` değişken sayıda argüman almayı sağlar (Bölüm 7). Başka sınıftaki static metot `SinifAdi.Metot()` ile çağrılır (`Math.Sqrt`); `Math.Round` ve `Math.Sqrt` `double` döndürür. `new Random(42)` her seferinde aynı diziyi verir, `Next(1, 7)` üst sınırı **içermez**.

Sorular

A) Çoktan Seçmeli

1 Aşağıdakilerden hangisi bir metodun **imzasına** (signature) dahil **değildir**?

(A) Metodun adı
(B) Parametrelerin sayısı
(C) Metodun dönüş tipi
(D) Parametrelerin tipleri
(E) Parametrelerin sırası

2 Aşağıdaki program ekrana ne yazdırır?

```
int a = 5;
Dene(a);
Yap(ref a);
Dene(a);
Console.WriteLine(a);

static void Dene(int n)
{
    n = n * 2;
}

static void Yap(ref int n)
{
    n = n * 2;
}
```

(A) `5`
(B) `10`
(C) `20`
(D) `40`
(E) Derleme hatası verir

3 Aşağıdaki metot için hangisi doğrudur?

```
static int EnBuyuk(int a, int b)
{
    if (a >= b)
    {
        return a;
    }
    if (a < b)
    {
        return b;
    }
}
```

(A) Her zaman büyük olan sayıyı döndürür.

(B) `a` ile `b` eşitse 0 döndürür.

(C) Derlenir, ama çalışırken `InvalidOperationException` fırlatır.

(D) CS0161 hatası verir, derlenmez.

(E) Her zaman `b`'yi döndürür.

4 Aşağıdaki metot başlıklarından hangisi **derlenmez**?

(A) `static void Yaz(int adet = 1, string ad)`

(B) `static void Yaz(string ad, int adet = 1)`

(C) `static void Yaz(string ad = "?", int adet = 1)`

(D) `static void Yaz(string ad, int adet = 1, bool buyuk = false)`

(E) `static void Yaz(string ad = "?")`

5 Aynı sınıfta şu iki metot tanımlıdır:

```
static void F(int a, double b) => Console.WriteLine("int,double");
static void F(double a, int b) => Console.WriteLine("double,int");
```

Aşağıdaki çağrılardan hangisi **belirsiz çağrı** (CS0121) hatası verir?

(A) `F(1, 2.5)` (B) `F(1.5, 2)` (C) `F('a', 2.5)` (D) `F(2, 3L)` (E) `F(1, 2)`

B) Çıktı Tahmini

6 Aşağıdaki program ekrana ne yazdırır?

```
int a = F(2) + F(3);
Console.WriteLine();
Console.WriteLine(a);
Console.WriteLine(F(F(4)));

static int F(int x)
{
    Console.Write($"f{x} ");
    return x * 2;
}
```

7 Aşağıdaki program ekrana ne yazdırır?

```
int a = 3;
int b = 4;
int c = Hesapla(b, a);
Console.WriteLine($"{a} {b} {c}");
Ekle(ref a, b);
Console.WriteLine($"{a} {b}");
Console.WriteLine(Hesapla(a, a) + a);

static int Hesapla(int a, int b)
{
    a = a + b;
    b = a - b;
    return a * b;
}

static void Ekle(ref int hedef, int deger)
{
    hedef = hedef + deger;
    deger = 100;
}
```

8 Aşağıdaki program ekrana ne yazdırır?

```
class Program
{
    static void Yaz(int x) => Console.WriteLine("int: " + x);
    static void Yaz(double x) => Console.WriteLine("double: " + x);
    static void Yaz(string s) => Console.WriteLine("string: " + s);
    static void Yaz(int a, int b) => Console.WriteLine("iki: " + (a + b));

    static void Main()
    {
        Yaz(5 / 2);
        Yaz(5 / 2.0);
        Yaz("5" + 2);
        Yaz('A');
        Yaz(5L);
        Yaz(2, 3);
    }
}
```

C) Kod Yazma

9 Bir sayının **kendisi hariç** pozitif bölenlerinin toplamını döndüren `BolenToplami(int n)` ve bu toplam sayının kendisine eşitse `true` döndüren `MukemmelMi(int n)` metotlarını yaz. Program bu metotları kullanarak 1 ile 10000 arasındaki **mükemmel sayıları** aynı satıra yazdırsın. (Örnek: 28'in bölenleri 1 + 2 + 4 + 7 + 14 = 28 olduğu için 28 mükemmel sayıdır.)

```
1–10000 arası mükemmel sayılar: 6 28 496 8128
```

10 Bir tam sayının basamaklarını ters çevirip döndüren `TersCevir(int n)` ve sayı tersine eşitse `true` döndüren `PalindromMu(int n)` metotlarını yaz. Program kullanıcı **0 girene kadar** sayı okusun ve her sayı için sonucu yazdırsın:

```
Sayı (çıkış için 0): 12321
12321 -> tersi 12321, palindrom
Sayı (çıkış için 0): 1230
1230 -> tersi 321, palindrom değil
Sayı (çıkış için 0): 0
```

Çözümler

Cevap anahtarı

1 C	2 B	3 D	4 A	5 E

1 Cevap: C

Metot imzası **ad + parametre tipleri** (sayısı, sırası ve `ref`/`out` türüyle birlikte) demektir. Dönüş tipi imzaya dahil değildir; bu yüzden yalnızca dönüş tipi farklı olan iki metot aynı sınıfta bulunamaz (CS0111). A, B, D ve E imzanın parçalarıdır.

2 Cevap: B

`Dene` her çağrıda `a`'nın **kopyasını** alır; kendi `n`'ini değiştirir ve bu hiçbir yere yansımaz, iki çağrı da etkisizdir. `Yap(ref a)` ise `a`'nın kendisini alır: `5 * 2 = 10`. A, `ref`'in de etkisiz sanılmasıdır. C, `Dene` çağrılarının etkili sanılmasıdır ($5 \cdot 2 \cdot 2$). D, üç çağrının da etkili sanılmasıdır. Çağrılar doğru yazıldığı için derleme hatası da yoktur.

Çıktı

```
10
```

3 Cevap: D

Derleyici `if` koşullarının değerini hesaplamaz; "iki `if` de yanlış olursa ne döner?" diye bakar ve metodun sonunda `return` olmadığı için derlemez (CS0161). Mantıken bu yol imkânsız olsa da kural böyledir. İkinci `if` yerine düz `return b;` yazılmalıydı. Derlenmeyen bir programın çalışma zamanı davranışı (A, B, C, E) olmaz.

Derleyici çıktısı – program derlenmez

```
test3.cs(3,12): error CS0161: 'EnBuyuk(int, int)': not all code paths return a value
```

4 Cevap: A

İsteğe bağlı parametre, zorunlu parametrelerin hepsinden sonra gelmelidir. A'da zorunlu `ad`, isteğe bağlı `adet`'in arkasında kaldığı için derlenmez (CS1737). B, D'de isteğe bağlılar sonda; C ve E'de bütün parametreler isteğe bağlıdır, bu da geçerlidir.

Derleyici çıktısı – program derlenmez

```
test4.cs(3,44): error CS1737: Optional parameters must appear after all required parameters
```

5 Cevap: E

`F(1, 2)` iki sürüme de uyar: `(int, double)` ikinci argümanı, `(double, int)` birinci argümanı `double` yapar. Hiçbiri diğerinden iyi değildir, çağrı belirsizdir. A ve B birebir uyar. C'de `char` → `int` genişlemesiyle `(int, double)` seçilir, `(double, int)` ise `2.5`'i `int` yapamaz. D'de `long` → `double` ile yine yalnız `(int, double)` uygundur.

Derleyici çıktısı – program derlenmez

```
test5.cs(8,9): error CS0121: The call is ambiguous between the following methods or properties:
'Program.F(int, double)' and 'Program.F(double, int)'
```

6

- `F(2) + F(3)`: Soldan sağa önce `F(2)` çalışır, `f2` yazar ve 4 döndürür; sonra `F(3)`, `f3` yazar ve 6 döndürür. `a = 10`. Bu satırda `WriteLine` olmadığı için imleç aynı satırda kalır; ardından gelen `Console.WriteLine()` alt satıra geçirir.
- `F(F(4))`: Önce içteki `F(4)`: `f4` yazar, 8 döndürür. Sonra `F(8)`: `f8` yazar, 16 döndürür. `Console.WriteLine` en son, argümanı hesaplandıktan **sonra** 16'yı aynı satıra yazar.

```
Çıktı
f2 f3
10
f4 f8 16
```

7

- `Hesapla(b, a)` çağrısında argümanların **sırası** ters: metodun `a`'sı 4, `b`'si 3 olur. `a = 4 + 3 = 7`, `b = 7 − 3 = 4`, dönen değer `7 * 4 = 28`. Metot kendi kopyalarını değiştirdi; ana koddaki `a` = 3 ve `b` = 4 aynen kaldı, ilk satır `3 4 28`.
- `Ekle(ref a, b)`: `hedef`, ana koddaki `a`'nın **kendisidir**; `3 + 4 = 7` olur ve dışarıya yansır. `deger` ise `b`'nin kopyasıdır, 100 yapılması `b`'yi etkilemez. İkinci satır `7 4`.
- `Hesapla(7, 7)`: `a = 14`, `b = 14 − 7 = 7`, dönen `98`. Ana koddaki `a` (şimdi 7) eklenir: `105`.

```
Çıktı
3 4 28
7 4
105
```

8

- `5 / 2` tam sayı bölmesi → `2` (`int`) → `int` sürümü. `5 / 2.0` → `2,5` → `double` sürümü (Türkçe ayarlarda ondalık ayracı virgüldür). `"5" + 2` → `"52"` → `string` sürümü.
- `'A'` bir `char`'dır ve `char` alan sürüm yok. En yakın genişleyen dönüşüm `char` → `int` olduğu için `int` sürümü çağrılır ve `'A'`'nın kodu **65** yazılır.
- `5L` bir `long`'dur; `long` → `int` daraltma olduğundan `int` sürümü uygun değil. `long` → `double` genişlemesiyle `double` sürümü seçilir. Değer `5` olduğu için `5,0` değil `5` yazılır.
- `Yaz(2, 3)` iki parametreli tek sürüme gider: `2 + 3 = 5`.

```
Çıktı
int: 2
double: 2,5
string: 52
int: 65
double: 5
iki: 5
```

9

`BolenToplami` 1'den `n / 2`'ye kadar dolaşır (bir sayının kendisi hariç hiçbir böleni yarısından büyük olamaz). `MukemmelMi` tek satırdır; `n > 1` koşulu, bölen toplamı 0 olan 1 sayısını eler. Ana kod sadece 1–10000 arasını dolaşıp `MukemmelMi`'ye sorar.

mukemmelSayilar.cs

```
Console.Write("1–10000 arası mükemmel sayılar:");
for (int i = 1; i <= 10000; i++)
{
    if (MukemmelMi(i))
    {
        Console.Write(" " + i);
    }
}
Console.WriteLine();

static int BolenToplami(int n)
{
    int toplam = 0;
    for (int i = 1; i <= n / 2; i++)
    {
        if (n % i == 0)
        {
            toplam += i;
        }
    }
    return toplam;
}

static bool MukemmelMi(int n) => n > 1 && BolenToplami(n) == n;
```

Çıktı

```
1–10000 arası mükemmel sayılar: 6 28 496 8128
```

10

`TersCevir`, Bölüm 2'deki `% 10` (son basamak) ve `/ 10` (son basamağı at) ikilisini kullanır: Her turda `ters = ters * 10 + son basamak`. `n` parametresi bir kopya olduğu için metodun içinde değiştirmek ana koddaki sayıyı bozmaz. `PalindromMu` ise `TersCevir`'i çağırır. Ana koddaki `while (true)` döngüsü 0 gelince `break` ile biter.

palindrom.cs

```
while (true)
{
    Console.Write("Sayı (çıkış için 0): ");
    int sayi = Convert.ToInt32(Console.ReadLine());
    if (sayi == 0)
    {
        break;
    }
    Console.Write($"{sayi} -> tersi {TersCevir(sayi)}, ");
    if (PalindromMu(sayi))
    {
        Console.WriteLine("palindrom");
    }
    else
    {
        Console.WriteLine("palindrom değil");
    }
}

static int TersCevir(int n)
{
    int ters = 0;
    while (n > 0)
```

```
    {
        ters = ters * 10 + n % 10;
        n /= 10;
    }
    return ters;
}

static bool PalindromMu(int n) => n == TersCevir(n);
```

Çıktı

```
Sayı (çıkış için 0): 12321
12321 -> tersi 12321, palindrom
Sayı (çıkış için 0): 1230
1230 -> tersi 321, palindrom değil
Sayı (çıkış için 0): 0
```

`TersCevir(1230)` sonucu `0321` değil `321`'dir; tam sayının başındaki sıfır yazılmaz.

BÖLÜM 7

Diziler

- **Bu Bölümde Öğreneceklerin**
  - Dizi tanımlamayı, oluşturmayı ve ilk değer vermeyi; varsayılan eleman değerlerini
  - İndeks mantığını (`0 … Length – 1`) ve `IndexOutOfRangeException` hatasını
  - Dizileri `for` ve `foreach` ile dolaşmayı; toplam, ortalama, en büyük, arama, sayma algoritmalarını
  - Dizilerin **referans** tipi olduğunu: `b = a`, `==` tuzağı ve kopyalama
  - `Array` sınıfının hazır metotlarını (`Sort`, `Reverse`, `IndexOf`, `Copy`, `Fill`)
  - Diziyi metoda göndermeyi, metottan dizi döndürmeyi ve `params` ile değişken sayıda parametreyi
  - İki boyutlu (`int[,]`) ve düzensiz (`int[][]`) dizileri

7.1 Neden dizi?

Bir sınıftaki 40 öğrencinin notunu saklamak istediğini düşün. `not1`, `not2`, …, `not40` diye 40 değişken tanımlamak hem zahmetli hem de işe yaramaz: Bu değişkenleri bir döngüyle dolaşamazsın, çünkü `not + i` diye bir değişken adı üretemezsin.

Dizi (array), **aynı tipteki** sabit sayıda değeri **tek bir ad** altında, yan yana tutan yapıdır. Her elemana bir **indeks** (index) numarasıyla ulaşılır ve indeksler **0'dan** başlar:

```
indeks:     0    1    2    3    4
          +----+----+----+----+----+
notlar:   | 85 | 70 | 92 | 64 | 78 |
          +----+----+----+----+----+
```

`notlar[2]` → 92. İndeks bir değişken de olabilir: `notlar[i]`. Dizileri güçlü yapan da budur: `i`'yi döngüyle değiştirerek bütün elemanlara sırayla ulaşırsın.

7.2 Dizi tanımlama ve oluşturma

```
tip[]  ad     = new tip[boyut];
int[]  notlar = new int[5];
```

- `int[]` "int dizisi" anlamına gelen bir tiptir. `int notlar[]` yazımı C# dilinde geçersizdir; ayraç her zaman tipin yanındadır: `int[] notlar`.
- `new int[5]` bellekte 5 `int`'lik yer ayırır. Boyut bir kere belirlenince **değişmez**. Boyut bir değişken de olabilir: `new int[n]`.
- Elemanları baştan biliyorsan süslü parantezle **ilk değer** verebilirsin: `int[] a = { 85, 70, 92 };` Boyutu C# sayar (burada 3).

DiziOlusturma.cs

```
int[] notlar = new int[5];          // 5 elemanlı, hepsi 0
notlar[0] = 85;
notlar[1] = 70;
notlar[4] = notlar[0] + 5;
Console.WriteLine($"{notlar[0]} {notlar[1]} {notlar[2]}");
Console.WriteLine($"Son eleman: {notlar[4]}");

string[] gunler = { "Pzt", "Sal", "Çar", "Per", "Cum" };
double[] fiyatlar = { 12.5, 8.0, 99.9 };
Console.WriteLine($"{gunler[2]} {fiyatlar[1]:F1}");
Console.Write($"{notlar.Length} {gunler.Length}");
Console.WriteLine($" {fiyatlar.Length}");
```

Çıktı

```
85 70 0
Son eleman: 90
Çar 8,0
5 5 3
```

`{ ... }` ile ilk değer verme **sadece tanımlama satırında** yapılabilir. Önceden tanımlanmış bir diziye `a = { 1, 2, 3 };` yazamazsın (derlenmez); bunun yerine `a = new int[] { 1, 2, 3 };` yazılır.

> **Yeni C# Notu: Koleksiyon ifadesi: `int[] a = [1, 2, 3];`**
>
> C# 12 ile gelen **koleksiyon ifadesi** (collection expression) köşeli parantezle yazılır. Tip, sol taraftan anlaşılır; sağ tarafa `new int[]` yazmak gerekmez. `..` ile başka bir diziyi araya yayabilirsin:

KoleksiyonIfadesi.cs

```
int[] c = [1, 2, 3];
int[] d = [.. c, 4, 5];
Console.WriteLine(string.Join(", ", c));
Console.WriteLine(string.Join(", ", d));
```

Çıktı

```
1, 2, 3
1, 2, 3, 4, 5
```

`string.Join(", ", c)` dizinin elemanlarını aralarına `, ` koyarak tek bir metinde birleştirir; dizileri yazdırmanın kısa yolu budur (ayrıntısı Bölüm 8'de).

Varsayılan değerler

`new` ile oluşturulan bir dizinin elemanları, sen değer vermesen bile otomatik olarak **varsayılan değer** (default value) alır. Bu, Bölüm 2'deki yerel değişkenlerden farklıdır: Yerel değişkeni değer vermeden okuyamazsın, ama dizi elemanını okuyabilirsin.

Eleman tipi	Varsayılan değer
`int`, `long`, `short`, `byte`	`0`
`double`, `float`	`0.0`
`decimal`	`0m`
`bool`	`false`
`char`	`'\0'` (kodu 0 olan boş karakter)
`string` ve diğer bütün sınıflar	`null` (hiçbir nesneyi göstermiyor)

Varsayilan.cs

```
int[] a = new int[3];
double[] b = new double[3];
bool[] c = new bool[3];
char[] d = new char[3];
string[] e = new string[3];
Console.WriteLine($"{a[0]} {b[1]:F1} {c[2]} {e[0] == null}");
Console.WriteLine((int)d[0]);
```

Çıktı

```
0 0,0 False True
0
```

`null` bir `string` değildir; "burada henüz nesne yok" demektir. `e[0].Length` gibi bir `null` üzerinden özellik okumak `NullReferenceException` ile çöker (Bölüm 12'de ele alınacak).

7.3 Length ve indeksler

Her dizinin eleman sayısını tutan bir **`Length`** özelliği (property) vardır. Geçerli indeksler **0'dan `Length - 1`'e** kadardır; son elemana `a[a.Length - 1]` ile ulaşılır.

Sınavda Çıkar: `Length` mi, `Length()` mi?

Dizide `Length` bir **özelliktir**, parantez almaz: `a.Length`. `Length()` diye bir metot yoktur. Aynı kural `string` için de geçerlidir: `s.Length`. Parantez koymak derleme hatasıdır (CS1955):

```
int[] a = { 1, 2, 3 };
string s = "C#";
Console.WriteLine(a.Length());
Console.WriteLine(s.Length);
```

Derleyici çıktısı – program derlenmez

```
LengthHata.cs(3,21): error CS1955: Non-invocable member 'Array.Length' cannot be used like a method.
```

IndexOutOfRangeException

Geçersiz bir indeks kullanırsan (negatif ya da `Length`'e eşit veya büyük) program **derlenir**, ama çalışırken `IndexOutOfRangeException` ile çöker. Derleyici indeksin değerini bilemez; kontrol çalışma zamanında yapılır:

SinirDisi.cs

```
int[] a = { 10, 20, 30 };
Console.WriteLine($"{a[0]} {a[a.Length - 1]}");
for (int i = 0; i <= a.Length; i++)      // <= : HATA!
{
    Console.WriteLine($"{i}: {a[i]}");
}
```

Çıktı – program yakalanmamış istisna ile sonlanır

```
10 30
0: 10
1: 20
2: 30
Unhandled exception. System.IndexOutOfRangeException: Index was outside the bounds of the array.
   at Program.<Main>$(String[] args) in SinirDisi.cs:line 5
```

Sık Yapılan Hata: `<=` ile bir fazla dönmek

3 elemanlı dizinin indeksleri 0, 1, 2'dir; `a[3]` yoktur. Döngü koşulu **`i < a.Length`** olmalı, `i <= a.Length` değil. Bu "bir fazla/bir eksik" hatasına *off-by-one* denir. Hata mesajındaki *Index was outside the bounds of the array.* ifadesi neyin yanlış gittiğini söyler: indeks dizinin sınırının dışında.

Sondan indeks ^ ve aralık ..

Son elemana şimdiye kadar `a[a.Length - 1]` ile ulaştın. C# 8'den beri bunun kısa bir yazımı var: **sondan indeks** (index from end) `^` ile yazılır; `a[^1]` son eleman, `a[^2]` sondan ikinci elemandır. **Aralık** (range) ise iki nokta ile yazılır: `a[b..e]`, `b` indeksinden başlar ve `e`'ye kadar gider; `e` **dahil değildir**. Başlangıç ya da bitiş yazmazsan baştan ya da sona kadar alınır:

Yazım	Anlamı
`a[^1]`	Son eleman (`a[a.Length - 1]` ile aynı)
`a[1..4]`	1., 2. ve 3. indeksler (4 hariç): 3 elemanlı **yeni dizi**
`a[..2]`	Baştan 2. indekse kadar (2 hariç): ilk 2 eleman
`a[2..]`	2. indeksten sona kadar
`a[^3..]`	Son 3 eleman

dizideAralik.cs

```
int[] a = { 10, 20, 30, 40, 50 };
Console.WriteLine($"{a[a.Length - 1]} {a[^1]} {a[^2]}");

int[] orta = a[1..4];
Console.WriteLine(string.Join(", ", orta));
Console.WriteLine(string.Join(", ", a[..2]));
Console.WriteLine(string.Join(", ", a[2..]));
Console.WriteLine(string.Join(", ", a[^3..]));

orta[0] = 99;                 // orta ayrı bir kopya: a değişmez
Console.WriteLine($"{a[1]} {orta[0]}");
```

Çıktı

```
50 50 40
20, 30, 40
10, 20
30, 40, 50
30, 40, 50
20 99
```

Aralık, dizinin seçilen kısmının **yeni bir kopyasını** üretir; kopyayı değiştirmek asıl diziyi etkilemez. Aralığın eleman sayısı `e - b`'dir (`1..4` → 3 eleman). Bölüm 8'de aynı yazımın `string` üzerinde de çalıştığını göreceksin.

> **Sınavda Çıkar: `^1` son eleman, `^0` hata**
>
> `^1` son elemandır; `^0` ise `a[a.Length]` demektir ve geçersiz bir indekstir. Aralıkta bu kural yoktur: `a[2..^0]` sona kadar alır. Ayrıca bu `^`, Bölüm 2'deki bit operatörü `a ^ b` değildir: Köşeli parantezin içinde ve sayının önünde duran `^` sondan sayar.

```
int[] a = { 10, 20, 30 };
Console.WriteLine(a[2..^0].Length);
Console.WriteLine(a[^0]);
```

Çıktı – program yakalanmamış istisna ile sonlanır

```
1
Unhandled exception. System.IndexOutOfRangeException: Index was outside the bounds of the
array.
   at Program.<Main>$(String[] args) in sondanHata.cs:line 3
```

7.4 Diziyi dolaşmak

Diziyi dolaşmanın iki yolu vardır: indeksli klasik `for` ve `foreach`. `foreach` döngüsü `foreach (tip eleman in dizi)` biçiminde yazılır ve "dizideki her eleman için" diye okunur:

Dolasma.cs

```
int[] notlar = { 85, 70, 92, 64, 78 };
for (int i = 0; i < notlar.Length; i++)
{
    Console.Write($"[{i}]={notlar[i]} ");
}
Console.WriteLine();
for (int i = notlar.Length - 1; i >= 0; i--)     // tersten
{
    Console.Write($"{notlar[i]} ");
}
Console.WriteLine();
foreach (int nt in notlar)                        // foreach
{
    Console.Write($"{nt} ");
}
Console.WriteLine();
Console.WriteLine(notlar);                        // elemanları değil, tipi
```

Çıktı
```
[0]=85 [1]=70 [2]=92 [3]=64 [4]=78
78 64 92 70 85
85 70 92 64 78
System.Int32[]
```

Sık Yapılan Hata: Diziyi doğrudan yazdırmak

`Console.WriteLine(notlar)` yazarsan elemanları değil `System.Int32[]` görürsün: bu, dizinin **tipini** söyler (`int` dizisi), içindekileri değil. Elemanları görmek için ya döngü kur ya da `string.Join(", ", notlar)` kullan. Bundan sonra bu yöntemi sık kullanacağız.

foreach ile dizi değiştirilemez

`foreach` döngüsündeki değişken **salt okunurdur** (read-only). Ona değer atamaya kalkarsan derleyici durur; dizi değişmez, hata verilir:

ForEachTuzak.cs
```
int[] a = { 1, 2, 3 };
foreach (int x in a)
{
    x = x * 10;                 // derleme hatası
}
Console.WriteLine(string.Join(", ", a));
```

Derleyici çıktısı – program derlenmez
```
ForEachTuzak.cs(4,5): error CS1656: Cannot assign to 'x' because it is a 'foreach iteration
variable'
```

Dizinin elemanını değiştirmek istiyorsan indeksli `for` kullanırsın:

DiziyiDegistir.cs
```
int[] a = { 1, 2, 3 };
for (int i = 0; i < a.Length; i++)
{
    a[i] = a[i] * 10;           // dizinin kendisini değiştirir
}
Console.WriteLine(string.Join(", ", a));
```

Çıktı
```
10, 20, 30
```

	Klasik `for`	`foreach`
İndekse erişim	Var (`i`)	Yok
Elemanı değiştirme	Olur: `a[i] = ...`	**Olmaz** (CS1656)
Tersten / atlayarak dolaşma	Olur	Olmaz, hep baştan sona
Ne zaman?	İndeks gerekiyorsa veya dizi değişecekse	Sadece okumak yetiyorsa (toplam, arama)

7.5 Temel dizi algoritmaları

Toplam, ortalama, en büyük, en küçük

İstatistik.cs

```
int[] d = { 12, -5, 47, 8, 47, -20, 3 };
int toplam = 0;
foreach (int x in d)
{
    toplam += x;
}
double ortalama = (double)toplam / d.Length;

int enBuyuk = d[0], enKucuk = d[0], enBuyukIndeks = 0;
for (int i = 1; i < d.Length; i++)
{
    if (d[i] > enBuyuk)
    {
        enBuyuk = d[i];
        enBuyukIndeks = i;
    }
    if (d[i] < enKucuk)
    {
        enKucuk = d[i];
    }
}
Console.WriteLine($"Toplam: {toplam}");
Console.WriteLine($"Ortalama: {ortalama:F2}");
Console.WriteLine($"En büyük: {enBuyuk} (indeks {enBuyukIndeks})");
Console.WriteLine($"En küçük: {enKucuk}");
```

Çıktı

```
Toplam: 92
Ortalama: 13,14
En büyük: 47 (indeks 2)
En küçük: -20
```

- Ortalamada `(double)` dönüşümü şart; yoksa `92 / 7` tam sayı bölmesiyle `13` olurdu.
- 47 dizide iki kere var (indeks 2 ve 4). `>` kullandığımız için **ilk** bulunanın indeksi kaldı. `>=` yazsaydık sonuncusu (4) kalırdı. Sınavda bu fark sorulur.

> **Sık Yapılan Hata: En büyüğe 0 ile başlamak**
>
> `int enBuyuk = 0;` ile başlarsan, elemanların hepsi negatif olan bir dizide (ör. `{ -7, -3, -9 }`) sonuç `-3` yerine **0** çıkar; 0 dizide yok bile! Aynı şekilde `enKucuk = 0` pozitif dizilerde yanlış sonuç verir. Başlangıç değeri her zaman dizinin **kendi elemanı** olmalı: `d[0]`.

Arama ve sayma

Bir değeri dizide baştan sona tek tek bakarak aramaya **doğrusal arama** (linear search) denir. Bulunamazsa geçersiz bir indeks olan **-1** döndürmek gelenektir. Hazır `Array.IndexOf` de aynı işi yapar; onu aşağıda göreceksin. Daha hızlı olan ikili aramayı (binary search) ileride göreceksin.

AramaSayma.cs

```
int[] d = { 4, 9, 2, 9, 7, 9, 1 };
int aranan = 9;

int konum = -1;                    // -1: bulunamadı
for (int i = 0; i < d.Length; i++)
{
    if (d[i] == aranan)
    {
        konum = i;
        break;                     // ilkini bulunca dur
    }
}
Console.WriteLine($"{aranan} ilk olarak {konum}. indekste");

int adet = 0;
foreach (int x in d)
{
    if (x == aranan)
    {
        adet++;
    }
}
Console.WriteLine($"{aranan} dizide {adet} kez geçiyor");
```

Çıktı

```
9 ilk olarak 1. indekste
9 dizide 3 kez geçiyor
```

7.6 Girdi ile dizi doldurma

Dizinin boyutunu ve elemanlarını kullanıcıdan alabilirsin. Aşağıdaki program dizisiz yazılamayacak bir iş yapıyor: Ortalamanın üstündeki notları saymak için notları **iki kez** dolaşmak gerekir; ilk turda ortalama henüz bilinmez.

NotGirisi.cs

```
Console.Write("Kaç öğrenci? ");
int n = Convert.ToInt32(Console.ReadLine());
int[] notlar = new int[n];          // boyut çalışırken belli oldu

for (int i = 0; i < n; i++)
{
    Console.Write($"{i + 1}. öğrencinin notu: ");
    notlar[i] = Convert.ToInt32(Console.ReadLine());
}

int toplam = 0;
foreach (int nt in notlar)
{
    toplam += nt;
}
double ort = (double)toplam / n;

int ustunde = 0;
foreach (int nt in notlar)
{
    if (nt > ort)
    {
        ustunde++;
```

```
    }
}
Console.WriteLine($"Ortalama: {ort:F1}");
Console.WriteLine($"Ortalamanın üstünde: {ustunde} öğrenci");
```

Çıktı

```
Kaç öğrenci? 5
1. öğrencinin notu: 85
2. öğrencinin notu: 70
3. öğrencinin notu: 92
4. öğrencinin notu: 64
5. öğrencinin notu: 78
Ortalama: 77,8
Ortalamanın üstünde: 3 öğrenci
```

Kullanıcıya gösterilen sıra numarası `i + 1`, dizideki indeks ise `i`. İnsanlar saymaya 1'den, diziler 0'dan başlar.

7.7 Diziler başvuru tipidir

Bölüm 2'deki temel tipler (`int`, `double`...) değişkenin içinde **değerin kendisini** tutar. Dizi değişkeni ise dizinin kendisini değil, bellekteki dizinin **adresini** tutar; buna **başvuru** ya da **referans** (reference) denir. Böyle davranan tiplere **başvuru tipi** (reference type), değeri doğrudan tutanlara **değer tipi** (value type) denir; Bölüm 4 ve 6'da adını andığımız ayrım budur. Dizinin kendisi belleğin *heap* denen bölgesinde durur.

TakmaAd.cs

```
int x = 5;
int y = x;          // değer kopyalandı
y = 99;
Console.WriteLine($"x = {x}");

int[] a = { 1, 2, 3 };
int[] b = a;        // referans kopyalandı: aynı dizi!
b[0] = 99;
Console.WriteLine($"a = {string.Join(", ", a)}");

int[] c = { 99, 2, 3 };
Console.WriteLine(a == b);
Console.WriteLine(a == c);
Console.WriteLine(IcerikAyni(a, c));

static bool IcerikAyni(int[] p, int[] q)
{
    if (p.Length != q.Length)
    {
        return false;
    }
    for (int i = 0; i < p.Length; i++)
    {
        if (p[i] != q[i])
        {
            return false;
        }
    }
    return true;
}
```

```
Çıktı
x = 5
a = 99, 2, 3
True
False
True
```

`int[] b = a;` satırı yeni bir dizi **oluşturmaz**; sadece aynı diziyi gösteren ikinci bir ad (takma ad, *alias*) oluşturur. `b` üzerinden yapılan değişiklik `a`'dan da görünür, çünkü ortada **tek bir dizi** vardır:

```
a ---+
     +--->  [ 99 | 2 | 3 ]     tek dizi, iki ad
b ---+

c ------->  [ 99 | 2 | 3 ]     ayrı bir dizi, içeriği aynı
```

> **Sınavda Çıkar: `==` içeriği değil, adresi karşılaştırır**
>
> Dizilerde `==` "bu iki değişken **aynı diziyi** mi gösteriyor?" sorusunu sorar. `a == c` içerikleri aynı olsa bile `false`'tur, çünkü ikisi ayrı dizilerdir. İçerik karşılaştırmak için kendi yazdığın bir metot (`IcerikAyni`) ya da Bölüm 13'te göreceğin LINQ'un `SequenceEqual` metodu kullanılır.
>
> Bu, `string` ile farklıdır: `string` tipinde `==` **içeriği** karşılaştırır (`"ali" == "ali"` → `true`). Dizide ise adres karşılaştırılır. Sınavda bu iki tipi karıştırmak sık yapılan hatadır.

7.8 Diziyi kopyalama

Madem `b = a` kopyalamıyor, gerçek bir kopya (ayrı bir dizi) nasıl oluşturulur? Üç yolu var:

Kopyalama.cs

```
int[] a = { 5, 10, 15 };

int[] k1 = new int[a.Length];             // 1) döngü ile
for (int i = 0; i < a.Length; i++)
{
    k1[i] = a[i];
}
int[] k2 = new int[a.Length];             // 2) Array.Copy ile
Array.Copy(a, k2, a.Length);
int[] k3 = (int[])a.Clone();              // 3) Clone() ile

k1[0] = -1;
k2[0] = -2;
k3[0] = -3;
Console.WriteLine(string.Join(", ", a));  // a etkilenmedi
Console.WriteLine($"{k1[0]} {k2[0]} {k3[0]}");

int[] buyuk = new int[5];                 // fazlası 0 ile dolar
Array.Copy(a, buyuk, a.Length);
int[] kucuk = new int[2];                 // hedef küçük: sadece 2 eleman
Array.Copy(a, kucuk, kucuk.Length);
Console.WriteLine($"{string.Join(", ", buyuk)} | {string.Join(", ", kucuk)}");
```

```
Çıktı
5, 10, 15
-1 -2 -3
5, 10, 15, 0, 0 | 5, 10
```

`Clone()` `object` döndürür; bu yüzden `(int[])` dönüşümü gerekir. Dizinin boyutu değişmediği için bir diziyi "büyütmenin" yolu, daha büyük yeni bir dizi oluşturup elemanları ona kopyalamaktır. Bunu `Array.Resize` yapar, ama o `ref` ile çağrılır ve dizi değişkenine yeni bir dizi atar:

```
Array.Resize(ref a, a.Length * 2);
```

7.9 Array sınıfı

`Array` sınıfı, dizilerle ilgili hazır `static` metotlar sunar. `using` yazmana gerek yoktur; `System` ad alanı otomatik gelir:

Metot	Ne yapar?
`Array.Sort(a)`	Diziyi küçükten büyüğe **yerinde** sıralar, bir şey döndürmez (`void`)
`Array.Reverse(a)`	Diziyi **yerinde** ters çevirir (`void`)
`Array.IndexOf(a, x)`	`x`'in ilk indeksini döndürür; yoksa `-1`
`Array.Fill(a, x)`	Bütün elemanlara `x` değerini atar
`Array.Copy(kaynak, hedef, n)`	`kaynak`'ın ilk `n` elemanını `hedef`'e kopyalar
`a.Clone()`	Yeni bir kopya döndürür (`object`, `(int[])` ile dönüştürülür)

ArrayOrnek.cs

```
int[] a = { 42, 7, 19, 3, 25 };
Array.Sort(a);
Console.WriteLine(string.Join(", ", a));
Console.WriteLine($"25 indeksi: {Array.IndexOf(a, 25)}");
Console.WriteLine($"100 indeksi: {Array.IndexOf(a, 100)}");

Array.Reverse(a);
Console.WriteLine(string.Join(", ", a));

string[] isimler = { "Zeynep", "ali", "Can", "Ayşe" };
Array.Sort(isimler);
Console.WriteLine(string.Join(", ", isimler));

int[] b = new int[4];
Array.Fill(b, 7);
Console.WriteLine(string.Join(", ", b));
```

```
Çıktı
3, 7, 19, 25, 42
25 indeksi: 3
100 indeksi: -1
42, 25, 19, 7, 3
ali, Ayşe, Can, Zeynep
7, 7, 7, 7
```

`Array.Sort` string'leri **kültüre göre** sıralar: `a` ile `A` aynı sırada sayılır ve ilk fark `l` ile `y` arasında olduğu için `ali`, `Ayşe`'den önce gelir. Karakter koduna göre sıralamak istersen `Array.Sort(isimler, StringComparer.Ordinal)` yazarsın.

> **İpucu**
>
> `Array.Sort(a)` diziyi **yerinde** değiştirir ve `void`'dir; `int[] s = Array.Sort(a);` derlenmez. Orijinal sırayı korumak istiyorsan önce kopyala, sonra kopyayı sırala: `int[] s = (int[])a.Clone(); Array.Sort(s);`

7.10 Diziler ve metotlar

Diziyi metoda geçirmek

Bölüm 6'da `int` parametrelerle yazılan `Swap(int a, int b)` metodunun çağıranın değişkenlerini değiştirmediğini gördün. Diziyi metoda gönderdiğinde kural aynıdır; ama kopyalanan değer bu sefer bir **referanstır**. Metot ile çağıran, **aynı diziyi** gösteren iki ayrı değişkene sahip olur:

DiziMetot.cs

```
int[] a = { 1, 2, 3 };
IkiyleCarp(a);
Console.WriteLine(string.Join(", ", a));
YeniDiziAta(a);
Console.WriteLine(string.Join(", ", a));
YerDegistir(a, 0, 2);
Console.WriteLine(string.Join(", ", a));

static void IkiyleCarp(int[] d)
{
    for (int i = 0; i < d.Length; i++)
    {
        d[i] *= 2;
    }
}

static void YeniDiziAta(int[] d)
{
    d = new int[] { 0, 0, 0 };     // d artık başka bir diziyi gösteriyor
    d[0] = 100;
}

static void YerDegistir(int[] d, int i, int j)
{
    int gecici = d[i];
    d[i] = d[j];
    d[j] = gecici;
}
```

Çıktı

```
2, 4, 6
2, 4, 6
6, 4, 2
```

- `IkiyleCarp`: `d` ile `a` aynı diziyi gösterir; `d[i]` üzerinden yapılan değişiklik `Main`'deki `a`'dan görünür.
- `YeniDiziAta`: `d = new int[] ...` satırı sadece metodun **kendi kopyası** olan `d`'yi yeni bir diziye yönlendirir; çağıran taraftaki `a` hâlâ eski diziyi gösterir. Yeni dizi metot bitince kaybolur.
- `YerDegistir`: Bölüm 6'da `int`'lerle yazılamayan yer değiştirme, dizinin elemanlarıyla yazılabilir.

```
IkiyleCarp(a) çalışırken:           YeniDiziAta içinde d = new ... sonrası:

Main:       a ---+                  Main:        a ---->  [ 2 | 4 | 6 ]
                 +-->  [ 1 | 2 | 3 ]
IkiyleCarp: d ---+                  YeniDiziAta: d ---->  [ 100 | 0 | 0 ]
```

Sınavda Çıkar: Dizi parametresi: ne dışarıdan görünür?

Metodun içinde yapılan	Çağıranda görünür mü?
Elemanı değiştirmek: `d[0] = 50;`	**Evet**: aynı dizi
Parametreye yeni dizi atamak: `d = new int[3];`	**Hayır**: sadece yerel kopya değişti
`int` parametreyi değiştirmek: `n = 50;` (`F(a[0])` ile çağrılsa bile)	**Hayır**: `a[0]`'ın değeri kopyalandı

"C#'ta diziler referans ile geçirilir" demek **yanlıştır**. Doğrusu: C# parametreleri değer ile geçirir; dizi değişkeninin değeri bir referans olduğu için metot aynı diziye erişebilir. Parametreye yeni dizi atamanın dışarıdan görünmesini istiyorsan `ref` kullanmak gerekir (Bölüm 6).

Metottan dizi döndürmek

Dönüş tipi `int[]` olan bir metot, içinde oluşturduğu diziyi `return` ile döndürebilir. Metodun yerel değişkeni (`sonuc`) metot bitince yok olur, ama dizinin kendisi yaşamaya devam eder; çünkü artık çağıran taraftaki değişken onu göstermektedir.

DiziDondurme.cs

```
int[] k = Kareler(5);
int[] t = TersKopya(k);
Console.WriteLine($"{string.Join(", ", k)} | {string.Join(", ", t)}");

static int[] Kareler(int n)
{
    int[] sonuc = new int[n];
    for (int i = 0; i < n; i++)
    {
        sonuc[i] = (i + 1) * (i + 1);
    }
    return sonuc;
}

static int[] TersKopya(int[] d)
{
    int[] ters = new int[d.Length];
    for (int i = 0; i < d.Length; i++)
    {
        ters[i] = d[d.Length - 1 - i];
    }
    return ters;
}
```

Çıktı

```
1, 4, 9, 16, 25 | 25, 16, 9, 4, 1
```

`TersKopya` orijinal diziye dokunmadan **yeni** bir dizi döndürür; `k` olduğu gibi kaldı.

7.11 Ters çevirme ve sıralama

Diziyi yeni bir dizi oluşturmadan **yerinde** (in-place) ters çevirmek için baştaki ve sondaki elemanlar yer değiştirilerek ortaya doğru ilerlenir:

TersCevir.cs

```
int[] a = { 1, 2, 3, 4, 5 };
TersCevir(a);
Console.WriteLine(string.Join(", ", a));

static void TersCevir(int[] d)
{
    for (int i = 0; i < d.Length / 2; i++)
    {
        int gecici = d[i];
        d[i] = d[d.Length - 1 - i];
        d[d.Length - 1 - i] = gecici;
    }
}
```

Çıktı

```
5, 4, 3, 2, 1
```

> **Sınavda Çıkar: Ters çevirme döngüsünün sınırı**
>
> Döngü `d.Length / 2`'ye kadar gitmeli. Sonuna kadar giderse her çift **iki kez** yer değiştirir ve dizi eski haline döner. Hazır `Array.Reverse(a)` aynı işi tek satırda yapar.

Elle sıralama: kabarcık ve seçmeli

Kabarcık sıralamada komşu iki eleman yanlış sıradaysa yer değiştirilir; her geçişte kalan kısmın en büyüğü sona yerleşir. Seçmeli sıralamada her turda kalan kısmın en küçüğü bulunup başa konur. İkisi de `Array.Sort`'un işini elle yapar; büyük dizilerde yavaştır, ama algoritmayı anlamak için iyidir.

ElleSiralama.cs

```
int[] a = { 5, 1, 4, 2, 8 };
int[] b = { 5, 1, 4, 2, 8 };
KabarcikSirala(a);
SeciliSirala(b);
Console.WriteLine($"Kabarcık: {string.Join(", ", a)}");
Console.WriteLine($"Seçmeli:  {string.Join(", ", b)}");

static void KabarcikSirala(int[] d)
{
    for (int i = 0; i < d.Length - 1; i++)
    {
        for (int j = 0; j < d.Length - 1 - i; j++)
        {
            if (d[j] > d[j + 1])
            {
                int gecici = d[j];
                d[j] = d[j + 1];
                d[j + 1] = gecici;
            }
        }
    }
}
```

```
static void SeciliSirala(int[] d)
{
    for (int i = 0; i < d.Length - 1; i++)
    {
        int enKucuk = i;
        for (int j = i + 1; j < d.Length; j++)
        {
            if (d[j] < d[enKucuk])
            {
                enKucuk = j;
            }
        }
        int gecici = d[i];
        d[i] = d[enKucuk];
        d[enKucuk] = gecici;
    }
}
```

Çıktı

```
Kabarcık: 1, 2, 4, 5, 8
Seçmeli:  1, 2, 4, 5, 8
```

7.12 Değişken sayıda parametre: params

Bir metodun son parametresi `params` ile işaretlenirse, çağıran taraf bu tipten istediği kadar değer verebilir. Derleyici bunları bir diziye toplar:

ParamsOrnek.cs

```
Console.WriteLine(Topla(1, 2, 3));
Console.WriteLine(Topla());
int[] dizi = { 4, 5 };
Console.WriteLine(Topla(dizi));

static int Topla(params int[] sayilar)
{
    int toplam = 0;
    foreach (int sayi in sayilar)
    {
        toplam += sayi;
    }
    return toplam;
}
```

Çıktı

```
6
0
9
```

`params` yalnızca son parametrede olabilir. Hazır bir diziyi de doğrudan geçebilirsin (`Topla(dizi)`); bu durumda metot o diziyi aynen alır.

7.13 İki boyutlu diziler

Tablo ya da **matris** biçimindeki veriler (satır × sütun) için **iki boyutlu dizi** kullanılır. C#'ta iki boyutlu dizinin en yaygın biçimi **dikdörtgen** dizidir: tek bir virgüllü tip `int[,]` ile yazılır ve bütün satırlar aynı uzunluktadır:

```
int[,] m = new int[3, 4];      // 3 satır, 4 sütun, hepsi 0
```

```
           sütun 0  sütun 1  sütun 2  sütun 3
satır 0  [     3        8        1        6    ]
satır 1  [     4        0        7        2    ]     m[1, 2] = 7
satır 2  [     9        5        2        4    ]
```

- `m[i, j]` : i. satırın j. sütunundaki eleman. İndeksler virgülle ayrılır.
- `m.GetLength(0)` : satır sayısı. `m.GetLength(1)` : sütun sayısı.
- `m.Length` : satır ve sütunların çarpımı, yani **toplam eleman sayısı**. Satır sayısını vermez.
- İç içe döngüyle dolaşılır: dıştaki döngü satırları, içteki sütunları gezer.

Matris.cs

```
int[,] m =
{
    { 3, 8, 1, 6 },
    { 4, 0, 7, 2 },
    { 9, 5, 2, 4 }
};
int satir = m.GetLength(0), sutun = m.GetLength(1);
Console.WriteLine($"Satır: {satir}, sütun: {sutun}, toplam: {m.Length}");

for (int i = 0; i < m.GetLength(0); i++)
{
    int satirToplami = 0;
    for (int j = 0; j < m.GetLength(1); j++)
    {
        Console.Write($"{m[i, j],4}");
        satirToplami += m[i, j];
    }
    Console.WriteLine($"  | {satirToplami}");
}
Console.WriteLine("----------------");
for (int j = 0; j < m.GetLength(1); j++)     // sütun toplamları
{
    int sutunToplami = 0;
    for (int i = 0; i < m.GetLength(0); i++)
    {
        sutunToplami += m[i, j];
    }
    Console.Write($"{sutunToplami,4}");
}
Console.WriteLine();
```

Çıktı

```
Satır: 3, sütun: 4, toplam: 12
   3   8   1   6  | 18
   4   0   7   2  | 13
   9   5   2   4  | 20
----------------
  16  13  10  12
```

Satır toplamında **dış** döngü satırı sabit tutar, iç döngü sütunları gezer. Sütun toplamında ise tersi: Dış döngü sütunu (`j`) sabit tutar, iç döngü satırları (`i`) gezer.

> **● Sık Yapılan Hata: Satır ve sütun sayısını karıştırmak**
>
> Dikdörtgen bir dizide iç döngüyü `j < m.Length` diye yazarsan, bu değer satır sayısı değil toplam eleman sayısıdır (3×4 matriste 12). Bu, `IndexOutOfRangeException` ile çöker. Kural: Dış döngü **`m.GetLength(0)`**, iç döngü **`m.GetLength(1)`**.

7.14 Düzensiz (jagged) diziler

C#'ta bir de **düzensiz** dizi vardır: `int[][]`. Bu, "dizilerin dizisi"dir. Her satır ayrı bir dizidir ve satırların uzunluğu farklı olabilir. Önce sadece satır sayısı verilir (`new int[5][]`), her satır sonra ayrı ayrı oluşturulur. Pascal üçgeni bunun güzel bir örneğidir:

Duzensiz.cs

```
int[][] pascal = new int[5][];           // sadece satır sayısı belli
for (int i = 0; i < pascal.Length; i++)
{
    pascal[i] = new int[i + 1];          // her satır farklı uzunlukta
    pascal[i][0] = 1;
    pascal[i][i] = 1;
    for (int j = 1; j < i; j++)
    {
        pascal[i][j] = pascal[i - 1][j - 1] + pascal[i - 1][j];
    }
}
foreach (int[] satir in pascal)
{
    foreach (int x in satir)
    {
        Console.Write($"{x} ");
    }
    Console.WriteLine($"({satir.Length} eleman)");
}
```

Çıktı

```
1 (1 eleman)
1 1 (2 eleman)
1 2 1 (3 eleman)
1 3 3 1 (4 eleman)
1 4 6 4 1 (5 eleman)
```

Dıştaki `foreach`'in değişkeni `satir` bir `int[]`'dir; içteki `foreach` onun elemanlarını gezer. Düzensiz dizilerde `pascal[i].Length` kullanmak **zorunludur**, çünkü her satırın uzunluğu farklıdır. Hazır değerle yazmak istersen her satırı `new[]` ile vermelisin: `int[][] j = { new[] { 1, 2 }, new[] { 3, 4, 5 } };`

	Dikdörtgen `int[,]`	Düzensiz `int[][]`
Erişim	`m[i, j]`	`j[i][k]`
Satır sayısı	`m.GetLength(0)`	`j.Length`
i. satırın sütun sayısı	`m.GetLength(1)` (hepsi aynı)	`j[i].Length` (değişebilir)
Toplam eleman	`m.Length`	Satır satır toplanır

Bölüm Özeti – Bilmen Gerekenler

- Dizi, **aynı tipte** sabit sayıda elemanı tek ad altında tutar: `int[] a = new int[5];` veya `int[] a = { 1, 2, 3 };`. Boyut sonradan değişmez.
- `new` ile oluşan dizide varsayılan değerler: sayılar `0`, `bool` `false`, `char` `'\0'`, `string` ve sınıflar `null`.
- İndeksler `0` … `a.Length - 1`. Geçersiz indeks derlenir ama çalışırken `IndexOutOfRangeException` verir. Döngü koşulu `i < a.Length`.
- `Length` dizide bir özellik (parantezsiz). `foreach` sadece okumak içindir; döngü değişkeni salt okunurdur.
- Sondan indeks: `a[^1]` son eleman (`^0` geçersiz). Aralık: `a[1..4]` 4 hariç **yeni dizi** verir; `a[..2]`, `a[2..]`, `a[^3..]` da geçerlidir.
- En büyük/en küçük aramaya `d[0]` ile başla, 0 ile değil. Doğrusal arama bulamazsa `-1` döndürür.
- Dizi değişkeni bir **referans** tutar: `b = a` kopya değil takma addır. `==` adres, içerik için kendi metodun ya da `SequenceEqual` gerekir.
- Gerçek kopya: döngü, `Array.Copy`, `Clone()`. `Array.Sort`, `Reverse`, `IndexOf`, `Fill` hazır metotlardır.
- Diziyi metoda göndermek referansın kopyasını gönderir: Elemanlardaki değişiklik **dışarıdan görünür**, parametreye yeni dizi atamak görünmez.
- `params` son parametreye değişken sayıda değer alır.
- `int[,]` dikdörtgen dizidir: `GetLength(0)` satır, `GetLength(1)` sütun, `Length` toplam eleman. `int[][]` düzensizdir: her satır ayrı dizidir.

Sorular

A) Çoktan Seçmeli

1 `double[] d = new double[4];` satırından sonra `d[2]` ve `d.Length` değerleri nedir?

(A) `0.0` ve `4`

(B) `null` ve `4`

(C) `0` ve `3`

(D) Değer verilmediği için derleme hatası

(E) `0.0` ve `3`

2 `int[] a` bir dizi, `string s` bir string ise uzunluğu veren **doğru** ifade çifti hangisidir?

(A) `a.Length()` ve `s.Length()`

(B) `a.Count` ve `s.Length()`

(C) `a.Size` ve `s.Count`

(D) `a.Length()` ve `s.Length`

(E) `a.Length` ve `s.Length`

3 Aşağıdaki kod parçası ekrana ne yazdırır?

```
int[] a = { 1, 2, 3 };
int[] b = a;
b[1] = 20;
a = new int[] { 7, 8, 9 };
Console.WriteLine(b[1] + a[1]);
```

A) `10` B) `28` C) `16` D) `22` E) `40`

4 Aşağıdaki metot ve çağrı için ekrana ne yazılır?

```
static void Degistir(int[] d, int n)
{
    d[0] = 50;
    n = 50;
}
// Main içinde:
int[] a = { 1, 2 };
int x = 1;
Degistir(a, x);
Console.WriteLine($"{a[0]} {x}");
```

A) `1 1` B) `50 50` C) `1 50` D) `50 1` E) Derleme hatası

5 Aşağıdaki tanıma göre `m.Length + m[1].Length + m[2][0]` ifadesinin değeri nedir?

```
int[][] m = { new[] { 1, 2 }, new[] { 3, 4, 5 }, new[] { 6 } };
```

A) `9` B) `11` C) `12` D) `6` E) `IndexOutOfRangeException`

B) Çıktı Tahmini

6 Aşağıdaki program ekrana ne yazdırır?

```
int[] a = { 3, 8, 1, 6, 4 };
int t = 0;
for (int i = 1; i < a.Length; i += 2)
{
    t += a[i];
}
Console.WriteLine(t);
for (int i = a.Length - 1; i > 0; i--)
{
    a[i] = a[i - 1];
}
Console.WriteLine(string.Join(", ", a));
int k = 0;
foreach (int x in a)
{
    if (x > a[1])
    {
        k++;
    }
}
Console.WriteLine(k);
```

7 Aşağıdaki program ekrana ne yazdırır?

```
static void Degistir(int[] d)
{
    d[0] = d[d.Length - 1];
    d = new int[] { 9, 9, 9 };
    d[1] = 0;
}

static int[] IkiKat(int[] d)
{
    int[] s = (int[])d.Clone();
    for (int i = 0; i < s.Length; i++)
    {
        s[i] *= 2;
    }
    return s;
}

int[] a = { 1, 2, 3 };
Degistir(a);
int[] b = IkiKat(a);
int[] c = a;
c[1] = 7;
Console.WriteLine(string.Join(", ", a));
Console.WriteLine(string.Join(", ", b));
Console.WriteLine($"{a == c} {a == b}");
```

8 Aşağıdaki program ekrana ne yazdırır?

```
int[][] m = { new[] { 2, 0, 1 }, new[] { 4, 5, 3 }, new[] { 1, 7, 6 } };
int kosegen = 0, ters = 0;
for (int i = 0; i < m.Length; i++)
{
    kosegen += m[i][i];
    ters += m[i][m.Length - 1 - i];
}
Console.WriteLine($"{kosegen} {ters}");
foreach (int[] satir in m)
{
    Console.Write($"{satir[satir.Length - 1]} ");
}
Console.WriteLine();
int[] ikinci = m[1];
ikinci[0] = 0;
Console.WriteLine(m[1][0] + m.Length * m[2][0]);
```

C) Kod Yazma

9 Bir `int` dizisindeki **en büyük ikinci farklı** değeri döndüren `IkinciEnBuyuk(int[] d)` metodunu yaz. Diziyi sıralamadan, **tek bir döngüyle** çöz. Dizide en az iki farklı değer olduğunu varsayabilirsin. Program metodu iki diziyle denesin. Beklenen çıktı:

```
12, 45, 7, 45, 30 -> 30
-3, -8, -1 -> -3
```

10 Kullanıcıdan önce matrisin boyutu `n`'yi, sonra her satırı bir satır olarak yazılmış `n × n` bir matrisin elemanlarını okuyan `Simetrik` programını yaz. Matrisin **simetrik** olup olmadığını (her `i`, `j` için `m[i, j] == m[j, i]`) `static bool SimetrikMi(int[,] m)` metoduyla kontrol etsin. (İpucu: Boşlukla ayrılmış bir satırı parçalamak için `satir.Split(' ')` kullan; parçaları bir `string` dizisi olarak verir. Bölüm 3'te "Bölüm 8'de öğreneceksin" dediğimiz bu metodu şimdilik kalıp olarak al.)

```
Matris boyutu: 3
Elemanları satır satır gir:
1 2 3
2 5 4
3 4 9
Matris simetrik.
```

Çözümler

Cevap anahtarı

1	2	3	4	5
A	E	B	D	C

1 Cevap: A

`new` ile oluşturulan `double` dizisinin her elemanı `0.0` olur. `Length` eleman sayısıdır (4), son indeks değil (3). `null` sadece sınıf tipli dizilerin varsayılanıdır; `double` dizisi için `null` söz konusu değildir ve değer verilmediği için derleme hatası da olmaz.

2 Cevap: E

Dizide `Length` bir **özelliktir** (parantezsiz), `string`'de de `Length` bir özelliktir. `Length()` diye bir metot yoktur; `a.Count` ve `a.Size` de dizilerde tanımlı değildir. `Count` ailesi Bölüm 13'teki koleksiyonlara aittir.

3 Cevap: B

`b = a` ile iki değişken **aynı diziyi** gösterir; `b[1] = 20` bu diziyi `{ 1, 20, 3 }` yapar. Sonra `a` yeni bir diziye yönlendirilir, ama `b` hâlâ eski diziyi gösterir. `b[1] + a[1] = 20 + 8 = 28`.

Çıktı

```
28
```

4 Cevap: D

Dizi parametresi `d`, `a` ile aynı diziyi gösterir; `d[0] = 50` dışarıdan görünür. `n` ise `x`'in değerinin kopyasıdır; onu değiştirmek `x`'i etkilemez.

Çıktı

```
50 1
```

5 Cevap: C

`m.Length` = 3 (satır sayısı), `m[1].Length` = 3 (`{ 3, 4, 5 }`), `m[2][0]` = 6. Toplam `3 + 3 + 6 = 12`. Düzensiz dizide satır uzunlukları farklıdır, ama kullanılan bütün indeksler geçerli olduğu için istisna oluşmaz.

Çıktı

```
12
```

6

- İlk döngü `i = 1, 3` için çalışır: `t = a[1] + a[3] = 8 + 6 = 14`.
- İkinci döngü sondan başa her elemanı bir sağa taşır: `a[4] = 6`, `a[3] = 1`, `a[2] = 8`, `a[1] = 3`. `a[0]`'a dokunulmadığı için 3 olarak kalır ve iki kez görünür. Bu bir kaydırma değil, son elemanın kaybolduğu bir **öteleme**dir.
- `a[1]` artık 3: 3'ten büyük elemanlar 8 ve 6 → `k = 2`.

```
Çıktı
14
3, 3, 8, 1, 6
2
```

7

- `Degistir(a)`: `d[0] = d[2]` → `a` = `{ 3, 2, 3 }`. Sonraki `d = new int[] ...` sadece yerel `d`'yi değiştirir; `d[1] = 0` yeni diziye yazılır, `a` etkilenmez.
- `IkiKat(a)`: `Clone()` ile ayrı bir kopya alınıp ikiyle çarpılır → `b` = `{ 6, 4, 6 }`; `a` değişmez.
- `c = a` takma addır: `c[1] = 7` → `a` = `{ 3, 7, 3 }`.
- `a == c` aynı dizi olduğu için `True`; `a == b` ayrı dizilerdir, `False`.

```
Çıktı
3, 7, 3
6, 4, 6
True False
```

8

- Ana köşegen: `m[0][0] + m[1][1] + m[2][2] = 2 + 5 + 6 = 13`. Ters köşegen: `m[0][2] + m[1][1] + m[2][0] = 1 + 5 + 1 = 7`.
- Her satırın son elemanı: `1 3 6` (sonda boşluk var).
- `ikinci = m[1]` yeni bir dizi değil, ikinci **satırın kendisini** gösteren bir referanstır. `ikinci[0] = 0` aslında `m[1][0] = 0` demektir. Sonuç: `0 + 3 * 1 = 3`.

```
Çıktı
13 7
1 3 6
3
```

9

İki değişken tut: `birinci` (en büyük) ve `ikinci`. Yeni eleman `birinci`'den büyükse eski `birinci` ikinciliğe düşer. Değilse ve `ikinci`'den büyükse (ve `birinci`'ye eşit değilse) yeni `ikinci` olur. Eşitlik kontrolü, 45 gibi tekrar eden en büyük değerin ikinci sayılmasını engeller. Başlangıç değeri, bu bölümdeki kurala uyarak dizinin kendi elemanıdır (`d[0]`); `int.MinValue` gibi uydurma bir değer seçseydik dizide gerçekten `int.MinValue` bulunduğunda yanılırdık. `ikinci` için henüz değer bulunmadığını `bulundu` bayrağı söyler.

IkinciEnBuyuk.cs

```
int[] a = { 12, 45, 7, 45, 30 };
int[] b = { -3, -8, -1 };
Console.WriteLine($"{string.Join(", ", a)} -> {IkinciEnBuyuk(a)}");
Console.WriteLine($"{string.Join(", ", b)} -> {IkinciEnBuyuk(b)}");

static int IkinciEnBuyuk(int[] d)
{
    int birinci = d[0];             // başlangıç: dizinin kendi elemanı
    int ikinci = d[0];
    bool bulundu = false;           // ikinci henüz bulunmadı
    foreach (int x in d)
    {
        if (x > birinci)
        {
```

```
            ikinci = birinci;
            birinci = x;
            bulundu = true;
        }
        else if (x < birinci && (!bulundu || x > ikinci))
        {
            ikinci = x;
            bulundu = true;
        }
    }
    return ikinci;
}
```

Çıktı

```
12, 45, 7, 45, 30 -> 30
-3, -8, -1 -> -3
```

10

Her satır `Split(' ')` ile parçalara bölünür ve parçalar sayıya çevrilerek iç içe iki döngüyle matrise yerleştirilir. `SimetrikMi` her `m[i, j]`'yi `m[j, i]` ile karşılaştırır; ilk uyumsuzlukta `false` döndürüp çıkar. İç döngünün `j = i + 1`'den başlaması, köşegeni ve her çifti iki kez kontrol etmeyi önler.

Simetrik.cs

```
static bool SimetrikMi(int[,] m)
{
    for (int i = 0; i < m.GetLength(0); i++)
    {
        for (int j = i + 1; j < m.GetLength(0); j++)
        {
            if (m[i, j] != m[j, i])
            {
                return false;
            }
        }
    }
    return true;
}

Console.Write("Matris boyutu: ");
int n = Convert.ToInt32(Console.ReadLine());
int[,] m = new int[n, n];
Console.WriteLine("Elemanları satır satır gir:");
for (int i = 0; i < n; i++)
{
    string[] parcalar = (Console.ReadLine() ?? "").Split(' ');
    for (int j = 0; j < n; j++)
    {
        m[i, j] = Convert.ToInt32(parcalar[j]);
    }
}
if (SimetrikMi(m))
{
    Console.WriteLine("Matris simetrik.");
}
else
{
    Console.WriteLine("Matris simetrik değil.");
}
```

Çıktı

```
Matris boyutu: 3
Elemanları satır satır gir:
1 2 3
2 5 4
3 4 9
Matris simetrik.
```

BÖLÜM 8

String ve Karakter İşlemleri

Bu Bölümde Öğreneceklerin

- `string`'in **değiştirilemez** (immutable) olmasının ne demek olduğunu
- `Length`, `s[i]`, `IndexOf`, `Substring`, `Split`, `Replace` gibi temel üyeleri
- `Substring`'in ikinci parametresinin **uzunluk** olduğunu (bitiş indeksi değil!)
- `==`, `Equals` ve `Compare` farkını; `StringComparison` seçeneklerini
- Türkçe `i/İ` ve `ı/I` sorununu; `ToUpper` ile `ToUpperInvariant` farkını
- `char` türünü, `char` metotlarını ve `'a' + 1` tuzağını
- `StringBuilder` ile verimli metin oluşturmayı; string ↔ sayı dönüşümlerini
- Palindrom, sesli harf sayma, Sezar şifreleme, harf frekansı gibi klasik algoritmaları

8.1 string nedir?

string, karakterlerden oluşan bir dizidir: bir isim, bir cümle, bir e-posta adresi... Bölüm 2'de `string`'i bir veri tipi olarak tanıdın. Aslında `string`, .NET'in `System.String` sınıfının kısa adıdır (`String` yazsan da aynı şeydir). `string` bir **başvuru tipidir** (reference type): değişken metnin kendisini değil, bellekteki string nesnesine giden bir **başvuruyu** tutar. Metotlarını, değişken adından sonra nokta koyarak çağırırsın: `ad.ToUpper()`.

string genellikle **sabit (literal)** ile oluşturulur: `string ad = "C#";`. Boş metin `""` ya da `string.Empty`'dir (hiç karakteri yoktur, uzunluğu 0). Aynı karakteri tekrarlamak için `new string('-', 20)` yazılır (yirmi tire).

Bölüm 7'deki dizilerde olduğu gibi, string'in karakterleri de **0'dan başlayarak** numaralanır. Bu numaraya **indeks** (index) denir. `"C# 14"` için:

Karakter	C	#	␣	1	4
İndeks	0	1	2	3	4

Uzunluk 5'tir, ama son indeks **4**'tür. Kural: **son indeks = `Length - 1`**.

stringGiris.cs

```
string ad = "C# 14";
Console.WriteLine(ad.Length);
Console.WriteLine(ad[0] + "|" + ad[2] + "|" + ad[4]);
Console.WriteLine(ad[ad.Length - 1]);
```

Çıktı

```
5
C| |4
4
```

`ad[i]` ifadesi `i`. indeksteki karakteri bir `char` olarak verir. Boşluk da bir karakterdir; `ad[2]` boşluğu verdiği için iki `|` arasında boşluk görünüyor.

Sık Yapılan Hata: `Length` bir özelliktir, metot değil

Dizilerde de string'lerde de uzunluk bir **özelliktir** (property): `dizi.Length`, `s.Length`. Parantez yazılmaz. Metot sanıp `s.Length()` yazarsan program derlenmez:

```
string s = "C#";
Console.WriteLine(s.Length());
```

Derleyici çıktısı – program derlenmez

```
lengthHata.cs(2,21): error CS1955: Non-invocable member 'string.Length' cannot be used like a method.
```

Ham metin sabitleri (raw string literal)

Bölüm 3'te `@"..."` ile `\` karakterinin kaçış sayılmadığını, çift tırnağın ise `""` diye yazıldığını gördün. Tırnak ve süslü parantez dolu uzun metinlerde (JSON, HTML gibi) bu da yorucu olur. **Ham metin sabiti** metni **üç çift tırnak** arasına alır; içinde `\`, `"` ve satır sonu olduğu gibi yazılır, hiçbir şeyin kaçırılması gerekmez.

Yazım	`\` karakteri	Çift tırnak	Birden çok satır
`"..."`	Kaçış başlatır (`\\` yazılır)	`\"`	`\n` ile
`@"..."`	Olduğu gibi	`""`	Doğrudan
`"""..."""`	Olduğu gibi	Olduğu gibi	Doğrudan; girinti silinir

Yeni C# Notu: Ham metin sabiti: `"""..."""`

C# 11 ile geldi. Kurallar: - Tek satırlıkta metin üç tırnağın arasındadır: `"""C:\temp"""`. - Çok satırlıkta açan `"""` kendi satırında durur, metin sonraki satırda başlar, kapanan `"""` da kendi satırındadır. İlk ve son satır sonları metne **girmez**. - **Kapanan `"""` işaretinin solundaki boşluk kadar girinti, her satırdan silinir.** Böylece metni kodun girintisine uydurarak yazarsın, çıktıya o girinti geçmez.

hamMetin.cs

```
string yol = """C:\Users\deniz\notlar.txt""";
Console.WriteLine(yol);

string soz = """Dedi ki: "Merhaba" ve gitti.""";
Console.WriteLine(soz);

string siir = """
    Bir iki
      üç dört
    beş
    """;
Console.WriteLine("[" + siir + "]");
```

Çıktı

```
C:\Users\deniz\notlar.txt
Dedi ki: "Merhaba" ve gitti.
[Bir iki
  üç dört
beş]
```

`siir` içinde ikinci satır dört değil **altı** boşlukla başlıyordu; kapanan `"""` dört boşluk girintili olduğu için baştaki dördü silindi, geriye iki boşluk kaldı. Metnin sonuna yeni satır girmediği için çıktıdaki `]` son harfin hemen yanındadır.

`$"""..."""` ile enterpolasyon da yapılır: `{ad}` yine değerle yer değiştirir. Metnin içinde süslü parantez de varsa (JSON gibi) `$` işaretini iki kez yaz: `$$"""..."""`. Bu durumda enterpolasyon için süslü parantezi **iki kez** yazarsın, tek süslü parantez düz karakter olur:

hamEnterpolasyon.cs

```
string ad = "Deniz";
int puan = 85;

string rapor = $"""
    Öğrenci: {ad}
    Puan: "{puan}" (geçti)
    """;
Console.WriteLine(rapor);

string json = $$"""
    {
      "ad": "{{ad}}",
      "puan": {{puan}}
    }
    """;
Console.WriteLine(json);
```

Çıktı

```
Öğrenci: Deniz
Puan: "85" (geçti)
{
  "ad": "Deniz",
  "puan": 85
}
```

Sık Yapılan Hata: Satır, kapanan işaretten az girintili olamaz

Çok satırlı ham metinde herhangi bir satırın girintisi kapanan `"""` satırından **azsa** program derlenmez (CS8999). Silinecek girinti o satırda yoktur:

```
string s = """
    Birinci
  İkinci
    """;
Console.WriteLine(s);
```

Derleyici çıktısı – program derlenmez

```
hamGirinti.cs(3,1): error CS8999: Line does not start with the same whitespace as the closing
line of the raw string literal.
```

8.2 string değiştirilemez (immutable)

string'in en önemli özelliği şudur: Bir string nesnesi bir kere oluşturulduktan sonra **içeriği asla değişmez**. Buna **değiştirilemezlik** (immutability) denir. `ToUpper()`, `Replace()`, `Trim()` gibi metotlar mevcut string'i değiştirmez; **yeni bir string üretip döndürür**. Sonucu bir değişkene atamazsan, üretilen yeni string kaybolur:

degismez.cs

```
string s = "merhaba";
s.ToUpper();                 // yeni string üretildi ama atılmadı!
Console.WriteLine(s);

string buyuk = s.ToUpper();  // yeni string'i sakla
Console.WriteLine(buyuk + " / " + s);

s = s.Replace('a', 'e');     // s artık YENİ nesneyi gösteriyor
Console.WriteLine(s);

string t = s;
s = s + "!";
Console.WriteLine(s + " / " + t);
```

Çıktı

```
merhaba
MERHABA / merhaba
merhebe
merhebe! / merhebe
```

Peki `s = s.Replace(...)` satırında `s` değişmedi mi? Değişen **string nesnesi değil, `s` değişkeninin gösterdiği yerdir**:

```
string s = "abc";     s ──► "abc"
string t = s;         t ──► "abc"   (aynı nesne)
s = s + "d";          s ──► "abcd"  (yeni nesne)
                      t ──► "abc"   (eski nesne hiç değişmedi)
```

`string` başvuru tipi olduğu halde çoğu zaman bir `int` gibi davranır: `t = s` yaptıktan sonra `s`'ye yeni bir değer atarsan `t` etkilenmez, çünkü nesne zaten hiç değişmez. Son satırda `t`'nin hâlâ `merhebe` yazmasının nedeni budur.

● Sık Yapılan Hata: Sonucu atanmayan string metodu

`s.ToUpper();`, `s.Trim();`, `s.Replace("a", "b");` satırları tek başına **hiçbir şey yapmaz**. Derleyici seni uyarmaz, program çalışır, ama `s` aynı kalır. Doğrusu: `s = s.ToUpper();`. Çıktı sorularında bu tuzak çok sık kullanılır.

● Sık Yapılan Hata: Karakteri yerinde değiştirmek

`s[0] = 'M';` yazmak isteyebilirsin. string değiştirilemez; indeksleyici **salt okunurdur** ve derleyici buna izin vermez:

```
string s = "merhaba";
s[0] = 'M';
Console.WriteLine(s);
```

Derleyici çıktısı – program derlenmez

```
indeksYaz.cs(2,1): error CS0200: Property or indexer 'string.this[int]' cannot be assigned to
-- it is read only
```

Çözüm: yeni string üret (`"M" + s.Substring(1)`) ya da aşağıda göreceğin `StringBuilder`'ı kullan.

8.3 Temel string üyeleri

Aşağıdaki tablo bu bölümün "kopya kâğıdı"dır. Örneklerde `s = "Merhaba"` kabul edilmiştir (M0 e1 r2 h3 a4 b5 a6):

Üye	Ne yapar?	Örnek	Sonuç
`Length`	Karakter sayısı (özellik)	`s.Length`	`7`
`s[i]`	`i`. karakter (`char`, salt okunur)	`s[1]`	`'e'`
`IndexOf(x)`	`x`'in ilk konumu, yoksa `-1`	`s.IndexOf('a')`	`4`
`LastIndexOf(x)`	`x`'in son konumu	`s.LastIndexOf('a')`	`6`
`Substring(b)`	`b`'den sona kadar	`s.Substring(3)`	`"haba"`
`Substring(b, n)`	`b`'den başlayıp **`n` karakter**	`s.Substring(1, 3)`	`"erh"`
`ToUpper()`	Büyük harfe çevir	`s.ToUpper()`	`"MERHABA"`
`ToLower()`	Küçük harfe çevir	`s.ToLower()`	`"merhaba"`
`Trim()`	Baştaki ve sondaki boşlukları sil	`" a b ".Trim()`	`"a b"`
`Replace(a, b)`	Bütün `a`'ları `b` yap	`s.Replace('a', 'e')`	`"Merhebe"`
`Contains(x)`	`x` içinde geçiyor mu?	`s.Contains("hab")`	`true`
`StartsWith(x)`	`x` ile başlıyor mu?	`s.StartsWith("Me")`	`true`
`EndsWith(x)`	`x` ile bitiyor mu?	`s.EndsWith("ba")`	`true`
`Split(ayraç)`	Parçalara böl → `string[]`	`"a,b,c".Split(',')`	`{"a", "b", "c"}`
`string.Join(ayraç, dizi)`	Parçaları birleştir	`string.Join("-", dizi)`	`"a-b-c"`
`PadLeft(n, c)`	Soldan `c` ile `n` genişliğe tamamla	`"7".PadLeft(3, '0')`	`"007"`
`Insert(i, x)`	`i`. konuma `x`'i ekle	`"ac".Insert(1, "b")`	`"abc"`
`Remove(b, n)`	`b`'den başlayıp `n` karakter sil	`"abcd".Remove(1, 2)`	`"ad"`
`string.IsNullOrEmpty(x)`	`null` ya da `""` mu?	`string.IsNullOrEmpty("")`	`true`
`string.IsNullOrWhiteSpace(x)`	Boş ya da yalnız boşluk mu?	`string.IsNullOrWhiteSpace(" ")`	`true`

Arama: IndexOf, LastIndexOf, Contains

`IndexOf` aranan karakterin ya da metnin **ilk** geçtiği indeksi verir; bulamazsa **`-1`** verir. İkinci parametre verirsen aramaya o indeksten başlar. Bu sayede bir kelimenin kaç kez geçtiğini bir döngüyle sayabilirsin:

arama.cs

```
string cumle = "C# zor değil, C# eğlenceli";
Console.WriteLine(cumle.IndexOf("C#"));
Console.WriteLine(cumle.LastIndexOf("C#"));
Console.WriteLine(cumle.IndexOf("Python"));
Console.WriteLine(cumle.Contains("değil"));

// "C#" kaç kez geçiyor?
int sayac = 0;
int konum = cumle.IndexOf("C#");
while (konum != -1)
{
    sayac++;
    konum = cumle.IndexOf("C#", konum + 1);
}
Console.WriteLine("C# " + sayac + " kez geçiyor.");
```

Çıktı

```
0
14
-1
True
C# 2 kez geçiyor.
```

Sınavda tuzak: `IndexOf` bulamazsa hata vermez, **`-1`** verir; `0` ise "ilk karakterde buldum" demektir. "Var mı?" sorusu için `Contains(x)` ya da `IndexOf(x) != -1` yaz; `IndexOf(x) > 0` başta bulunan eşleşmeyi kaçırır.

Substring: başlangıç ve uzunluk

`Substring(b, n)` metni **`b` indeksinden başlayıp `n` karakter** alır. İkinci sayı **bitiş indeksi değil, uzunluktur**. Sonucun uzunluğu her zaman `n`'dir. `Substring(b)` ise `b`'den sonuna kadar her şeyi alır. `"Bilgisayar"` üzerinde deneyelim:

Karakter	B	i	l	g	i	s	a	y	a	r
İndeks	0	1	2	3	4	5	6	7	8	9

altMetin.cs

```
string s = "Bilgisayar";
Console.WriteLine(s.Substring(3));
Console.WriteLine(s.Substring(0, 3));
Console.WriteLine(s.Substring(3, 4));
Console.WriteLine(s.Substring(3, 4).Length);
Console.WriteLine("[" + s.Substring(10) + "]");

string eposta = "elif.demir@ogr.edu.tr";
int at = eposta.IndexOf('@');
string kullanici = eposta.Substring(0, at);
string alan = eposta.Substring(at + 1);
Console.WriteLine(kullanici + " | " + alan);
```

```
Çıktı
gisayar
Bil
gisa
4
[]
elif.demir | ogr.edu.tr
```

`Substring(10)` hata vermedi, boş string döndü: Başlangıç indeksi `Length`'e **eşit** olabilir, ama ondan büyük olamaz. E-posta örneği gerçek hayatta çok kullanılan bir kalıptır: önce `IndexOf` ile ayırıcıyı bul, sonra `Substring` ile parçala.

Sınavda Çıkar: Substring kuralları

- `s.Substring(b, n)` → `b`'den başlayıp `n` **karakter**. `Substring(3, 4)` → 3, 4, 5, 6. indeksler → 4 karakter.
- Bitiş indeksini (`e`, dahil değil) biliyorsan uzunluk `e - b`'dir: `s.Substring(b, e - b)`.
- İlk 3 karakter: `s.Substring(0, 3)`. Son 3 karakter: `s.Substring(s.Length - 3)`.
- `s.Substring(i, 1)` → `i`. karakter, ama `char` değil **string** olarak. `char` istiyorsan `s[i]`.
- `s.Substring(s.Length)` → `""` (hata değil). `b + n > Length` ya da `b > Length` → hata.

Yeni C# Notu: Aralık (`..`) ve sondan indeks (`^`)

Bölüm 7'de dizilerde gördüğün `^` ve `..` yazımı `string` üzerinde de çalışır: `s[^1]` son karakterdir; `s[b..e]` ise `Substring(b, e - b)` ile aynı sonucu verir. Dikkat: `..` ile ikinci sayı *bitiş indeksi* (hariç), `Substring` ile ikinci sayı *uzunluk*tur.

```
Console.WriteLine(s[0] + " " + s[^1]);
Console.WriteLine(s[3..7]);
Console.WriteLine(s[..3]);
Console.WriteLine(s[^3..]);
```

```
Çıktı
B r
gisa
Bil
yar
```

String'in sınırları dışına çıkarsan `s[i]` ile `IndexOutOfRangeException`, `Substring` ile `ArgumentOutOfRangeException` alırsın. İlki Bölüm 7'de dizilerde gördüğün hatanın aynısıdır:

indeksHatasi.cs

```
string s = "Veri";
Console.WriteLine(s[3]);
Console.WriteLine("[" + s.Substring(4) + "]");
Console.WriteLine(s.Substring(2, 3));
```

Çıktı – program yakalanmamış istisna ile sonlanır

```
i
[]
Unhandled exception. System.ArgumentOutOfRangeException: Index and length must refer to a location
within the string. (Parameter 'length')
   at Program.<Main>$(String[] args) in indeksHatasi.cs:line 4
```

Mesaj her şeyi söylüyor: `"Veri"`'de 2. indeksten başlayıp 3 karakter alınamaz, çünkü orada yalnız 2 karakter var. Bu hatanın en sık sebepleri döngü koşulunda `i < s.Length` yerine **`i <= s.Length`** yazmak ve `Substring`'in ikinci parametresini bitiş indeksi sanmaktır.

Temizleme ve değiştirme: Trim, Replace, IsNullOrWhiteSpace

Kullanıcıdan gelen metinlerin başında ve sonunda çoğu zaman istenmeyen boşluklar olur. `Trim()` bunları (sekme ve satır sonu dahil) siler, **aradaki** boşluklara dokunmaz: `" Ali Veli ".Trim()` → `"Ali Veli"`. Yalnız bir tarafı temizlemek için `TrimStart()` ve `TrimEnd()` vardır. `Replace` hem `char` hem string ile çalışır ve geçtiği **her** yeri değiştirir; bütün boşlukları silmek için `s.Replace(" ", "")` yazılır.

Bir girdinin boş olup olmadığını kontrol ederken dikkat: `string.IsNullOrEmpty(s)` yalnız `null` ve `""` için `true` verir, `" "` için `false` verir. `string.IsNullOrWhiteSpace(s)` ise yalnız boşluktan oluşan metni de yakalar; kullanıcının gerçekten bir şey yazıp yazmadığını anlamak için o daha güvenlidir. `PadLeft` ve `PadRight` metni belirli genişliğe tamamlar (`"7".PadLeft(3, '0')` → `"007"`); tablo çıktılarını hizalamak için işine yarar.

Bölme ve birleştirme: Split, string.Join

`Split` bir metni verilen ayırıcıdan böler ve parçaları bir **`string` dizisi** olarak verir. Virgülle ya da noktalı virgülle ayrılmış verileri okumanın standart yoludur. Ayırıcı bir `char` ya da string olabilir; birden fazla ayırıcı için `Split(',', ';')` yazılır. `string.Join` bunun tersini yapar: parçaları aralarına ayırıcı koyarak birleştirir.

bolme.cs

```
string satir = "Ayşe;Yılmaz;70;85";
string[] p = satir.Split(';');
Console.WriteLine(p.Length + " parça: " + p[0] + " " + p[1]);
Console.WriteLine(int.Parse(p[2]) + int.Parse(p[3]));

string cumle = "  C#   ile    kod  ";
string[] kelimeler = cumle.Split(' ', StringSplitOptions.RemoveEmptyEntries);
Console.WriteLine(kelimeler.Length + " kelime");
Console.WriteLine(string.Join("-", kelimeler));
Console.WriteLine(cumle.Split(' ').Length + " parça (seçeneksiz)");
```

Çıktı

```
4 parça: Ayşe Yılmaz
155
3 kelime
C#-ile-kod
12 parça (seçeneksiz)
```

Kelimeler arasında birden fazla boşluk varsa `Split(' ')` araya **boş string'ler** koyar (yukarıda 12 parça çıktı). `StringSplitOptions.RemoveEmptyEntries` seçeneği bu boş parçaları atar ve sadece kelimeleri bırakır.

8.4 Büyük/küçük harf ve Türkçe I sorunu

`ToUpper()` ve `ToLower()` parametresiz çağrıldığında, harf dönüşümünü **bilgisayarın kültür ayarına** (culture, Bölüm 3) göre yapar. Çoğu dilde bu fark etmez; ama Türkçede `i` harfinin **dört** biçimi vardır (`i`, `ı`, `İ`, `I`) ve kurallar İngilizceden farklıdır:

İngilizce kuralda `i` büyüyünce `I`, `I` küçülünce `i` olur. Türkçe kuralda `i` büyüyünce `İ`, `I` küçülünce `ı` olur. Bu yüzden aynı program Türkçe bir bilgisayarda farklı davranabilir. Kültürden bağımsız çevirmek için `ToUpperInvariant()` ve `ToLowerInvariant()` vardır; bunlar her bilgisayarda İngilizce kurallarla çalışır:

turkcel.cs

```
string sehir = "izmir";
Console.WriteLine(sehir.ToUpper());
Console.WriteLine(sehir.ToUpperInvariant());

string komut = "EXIT";
string kucuk = komut.ToLower();
Console.WriteLine(kucuk + " " + (kucuk == "exit"));
Console.WriteLine(komut.ToLowerInvariant() == "exit");
Console.WriteLine("i".ToUpper() + " " + "I".ToLower());
```

Çıktı

```
İZMİR
IZMIR
exıt False
True
İ ı
```

Türkçe kurallarla `"EXIT"` küçültülünce `"exıt"` olur ve `"exit"` ile eşleşmez. Türkçe Windows'ta çalışan pek çok programın "anlamsız" hatalarının sebebi budur.

> **İpucu: Hangisini kullanmalıyım?**
>
> - Kullanıcıya **gösterilecek Türkçe metin** (isim, şehir) → `ToUpper()` / `ToLower()`.
> - **Program içi karşılaştırma** (komut, dosya uzantısı) → `ToUpperInvariant()` / `ToLowerInvariant()` ya da `StringComparison.OrdinalIgnoreCase` (sonraki başlıkta).
>
> Bu kitabın çıktıları Türkçe (`tr-TR`) kültürle üretildi; İngilizce bir bilgisayarda `"izmir".ToUpper()` sonucu `IZMIR` olur.

8.5 string karşılaştırma: ==, Equals ve StringComparison

C#'ta `==` ve `!=` iki string'in **içeriğini** karşılaştırır (Bölüm 4'te kullandın). `string` sınıfı bu operatörü içerik karşılaştırması için yeniden tanımlar; yani `a == b` ile `a.Equals(b)` aynı sonucu verir. Başvuru tipi olması, adresleri karşılaştırdığın anlamına gelmez. Adres karşılaştırması ancak string'leri `object` gibi ele alırsan yapılır:

karsilastirma.cs

```
string a = "kod";
string parca = "ko";
string b = parca + "d";      // çalışırken oluşan yeni nesne

Console.WriteLine(a == b);
Console.WriteLine(a.Equals(b));
Console.WriteLine(object.ReferenceEquals(a, b));
Console.WriteLine((object)a == (object)b);

string c = "KOD";
Console.WriteLine(a == c);
Console.WriteLine(a.Equals(c, StringComparison.OrdinalIgnoreCase));
Console.WriteLine(string.Equals(a, c, StringComparison.OrdinalIgnoreCase));
```

Çıktı

```
True
True
False
False
False
True
True
```

`a` ve `b` ayrı nesnelerdir, ama içerikleri aynı olduğu için `==` ve `Equals` `True` verir. `object.ReferenceEquals` ile `(object)a == (object)b` ise "aynı nesne mi?" sorusunu sorar ve burada `False` verir. Bunları gerçek programlarda neredeyse hiç kullanmazsın: **string'leri `==` ya da `Equals` ile karşılaştır.**

Büyük/küçük harfi önemsemeden karşılaştırmak için karşılaştırma türünü açıkça söylersin: **`StringComparison.OrdinalIgnoreCase`**. Başlıca seçenekler:

- `Ordinal` / `OrdinalIgnoreCase` → karakter kodlarına göre, büyük/küçük harf **duyarlı** / **önemsiz**.
- `CurrentCulture` / `CurrentCultureIgnoreCase` → bilgisayarın kültür kurallarına göre.

Program içi karşılaştırmalarda (komut, uzantı, anahtar kelime) doğru tercih `OrdinalIgnoreCase`'dir; çünkü kültüre bağlı değildir ve Türkçe `I` sorunundan etkilenmez.

> **Sık Yapılan Hata: Karşılaştırmak için ToLower() kullanmak**
>
> `if (komut.ToLower() == "exit")` İngilizce bilgisayarda çalışır, Türkçe bilgisayarda **çalışmaz**: Kullanıcı `EXIT` yazsa bile `ToLower()` sonucu `"exıt"` olur (yukarıdaki çıktıya bak). Kodun doğru göründüğü için bu hatayı bulmak zordur. Büyük/küçük harf önemsizse `string.Equals(komut, "exit", StringComparison.OrdinalIgnoreCase)` yaz. Bonus: bu statik biçim `komut` boş bir başvuru (`null`, Bölüm 9) olsa bile çökmez; `komut.Equals(...)` ise çöker.

8.6 Sıralama karşılaştırması: CompareTo ve Compare

`Equals` yalnız "aynı mı?" sorusunu cevaplar. "Sözlükte hangisi önce gelir?" için `a.CompareTo(b)` ya da `string.Compare(a, b)` kullanılır. İkisi de bir **`int`** döndürür:

- **Negatif** → `a`, `b`'den **önce** gelir.
- **0** → ikisi eşit.
- **Pozitif** → `a`, `b`'den **sonra** gelir.

Dönen sayının kendisine (`-1`, `4`...) güvenme; yalnız **işaretine** bak. Karşılaştırma varsayılan olarak **kültüre duyarlıdır**, yani gerçek bir sözlük gibi çalışır. Karakter kodlarına göre sıralamak için `string.CompareOrdinal` ya da `StringComparison.Ordinal` kullanılır. Farkı görmek için işaretleri yazdıralım (`Math.Sign` sayıyı `-1`, `0` ya da `1` yapar):

siralama.cs

```
Console.WriteLine(Math.Sign("elma".CompareTo("armut")));
Console.WriteLine(Math.Sign("kale".CompareTo("kalem")));
Console.WriteLine(Math.Sign("kale".CompareTo("KALE")));
Console.WriteLine(Math.Sign(string.CompareOrdinal("kale", "KALE")));
Console.WriteLine(Math.Sign("çilek".CompareTo("zeytin")));
Console.WriteLine(Math.Sign(string.CompareOrdinal("çilek", "zeytin")));

string[] isimler = { "Zeynep", "ali", "Çağla", "Ece" };
Array.Sort(isimler);
Console.WriteLine(string.Join(" ", isimler));
Array.Sort(isimler, StringComparer.Ordinal);
Console.WriteLine(string.Join(" ", isimler));
```

Çıktı

```
1
-1
-1
1
-1
1
ali Çağla Ece Zeynep
Ece Zeynep ali Çağla
```

- `"kale"` ile `"kalem"`: İlk 4 karakter aynı, `"kale"` daha kısa → önce gelir (`-1`).
- **Kültüre duyarlı** (varsayılan): aynı harfte küçük harf önce gelir (`"kale"` < `"KALE"`), Türkçe harfler alfabedeki yerindedir (`"çilek"` < `"zeytin"`). `Array.Sort(isimler)` de böyle sıralar.
- **Ordinal** (karakter kodu): `'K'` = 75, `'k'` = 107 olduğu için büyük harfler küçüklerden önce gelir; `'ç'` = 231, `'z'` = 122'den büyük olduğu için Türkçe harfler sona düşer.

> **Sınavda Çıkar: CompareTo'nun değeri değil işareti**
>
> Sınavda "`"Ankara".CompareTo("Antalya")` negatif mi pozitif mi?" diye sorulursa: İlk fark 2. indekste, `'k'` ile `'t'`; `'k'` alfabede önce geldiği için sonuç **negatif**. Koşullarda yalnız işarete bak: `a.CompareTo(b) < 0` → "a önce gelir". Bir dizideki alfabetik ilk ismi bulmak için Bölüm 7'deki "en küçüğü bul" algoritmasında `dizi[i] < enKucuk` yerine `dizi[i].CompareTo(enKucuk) < 0` yazman yeterlidir. Sayılardaki `<` ve `>` string'lerde **kullanılamaz** (`a < b` yazarsan `CS0019` alırsın); `a.CompareTo(b) < 0` yaz.

8.7 Karakter karakter dolaşma ve char

`char` tek bir karakteri tutar (`'a'`, `'7'`, `'ş'`). Bellekte 2 bayt yer kaplar ve Unicode'un **UTF-16** kodlamasını kullanır; bu yüzden Türkçe harflerle sorunsuz çalışır. Pek çok string sorusu aynı kalıpla çözülür: string'i **karakter karakter dolaş**, her karakteri kontrol et. İndeksle işin yoksa en rahat yol `foreach`'tir; indeks gerekiyorsa `for` ve `s[i]` kullanılır. Karakterleri sınıflandırmak için `char` türünün hazır metotları vardır:

Metot	Ne yapar?	Örnek → Sonuç
`char.IsDigit(c)`	Rakam mı?	`IsDigit('7')` → `true`
`char.IsLetter(c)`	Harf mi? (Türkçe harfler dahil)	`IsLetter('ş')` → `true`
`char.IsLetterOrDigit(c)`	Harf ya da rakam mı?	`IsLetterOrDigit('_')` → `false`
`char.IsUpper(c)`	Büyük harf mi?	`IsUpper('Ç')` → `true`
`char.IsLower(c)`	Küçük harf mi?	`IsLower('a')` → `true`
`char.ToUpper(c)`	Büyük harfe çevir (`char` verir)	`ToUpper('a')` → `'A'`
`char.GetNumericValue(c)`	Rakamın sayısal değeri (`double`), rakam değilse `-1`	`GetNumericValue('7')` → `7`

`char.ToUpper` da kültüre duyarlıdır (`'i'` → `'İ'`); kültürden bağımsızı `char.ToUpperInvariant`'tır. Örnek: Güçlü bir şifrede büyük harf, küçük harf ve rakam bulunmalıdır. Bir şifredeki her türden karakteri sayalım:

sifreKontrol.cs

```
string sifre = "CSharp25!Güçlü";
int buyuk = 0, kucuk = 0, rakam = 0;

foreach (char c in sifre)
{
    if (char.IsUpper(c))
    {
        buyuk++;
    }
    else if (char.IsLower(c))
    {
        kucuk++;
    }
    else if (char.IsDigit(c))
    {
        rakam++;
    }
}
Console.WriteLine($"Büyük: {buyuk}, küçük: {kucuk}, rakam: {rakam}");
```

Çıktı

```
Büyük: 3, küçük: 8, rakam: 2
```

`ü` ve `ç` de küçük harf olarak sayıldı: `char` metotları Unicode'u tanır.

> **Sık Yapılan Hata: Rakam karakterini sayı sanmak**
>
> `'7'` karakterinin değeri 7 değil, karakter kodu olan **55**'tir (Bölüm 2). Bir string'deki rakamları toplarken `toplam += c;` yazarsan kodları toplarsın: `"2026"` için 50 + 48 + 50 + 54 = **202** çıkar. Doğrusu `toplam += c - '0';` (sonuç 10). `c - '0'` karakterin rakam olup olmadığına bakmaz; emin değilsen önce `char.IsDigit(c)` ile kontrol et. `char.GetNumericValue('a')` ise rakam olmayan karakter için `-1` verir.

char aritmetiği: 'a' + 1 tuzağı

`char` aslında bir sayıdır (karakter kodu), ama `+` ile işleme girince sonuç `int` olur (Bölüm 2). Bu yüzden `'a' + 1`, `'b'` değil **98**'dir. Karakter istiyorsan sonucu `(char)` ile geri çevirmen gerekir:

İfade	Türü	Sonuç
`'a' + 1`	`int`	`98`
`(char)('a' + 1)`	`char`	`b`
`"" + 'a' + 1`	`string`	`a1`

`"" + 'a' + 1` başta string olduğu için **birleştirme** yapar. İki kuralı ezberle: `char + int` → `int`; `string + char` → `string`. Bir `char` değişkenine `c++` yazarsan doğrudan çalışır ve bir sonraki karaktere geçer. Ama aritmetik sonucunu `char` değişkenine atamak derlenmez: `char d = c + 1;` satırı `CS0266` hatası verir (`int` örtük olarak `char`'a çevrilemez). Doğrusu `char d = (char)(c + 1);`.

8.8 StringBuilder

string değiştirilemez olduğu için, bir metni parça parça oluştururken her `+` işlemi **yeni bir string** üretir. Bunun yerine **değiştirilebilir** (mutable) bir metin kutusu olan `StringBuilder` kullanılır. `StringBuilder`'ın metotları **nesnenin kendisini değiştirir**; sonucu atamana gerek yoktur. `System.Text` ad alanında olduğu için dosyanın başına `using System.Text;` yazmalısın; yazmazsan `CS0246` hatası alırsın.

Üye	Ne yapar?
`Append(x)`	Sona ekler (her tip olabilir: `int`, `char`, `double`, string...)
`Insert(i, x)`	`i`. konuma ekler
`Remove(b, n)`	`b`'den başlayıp `n` karakter siler
`sb[i]`	`i`. karakteri okur **ve değiştirir** (`sb[0] = 'x'`)
`Length`	Karakter sayısı
`ToString()`	İçeriği bir string olarak verir

sbOrnek.cs

```
using System.Text;

StringBuilder sb = new StringBuilder("C#");
sb.Append(" 14");
sb.Insert(0, "Merhaba ");
sb[0] = 'm';
sb.Remove(sb.Length - 1, 1);
sb.Append('!').Append(2026);
Console.WriteLine(sb);

char[] harfler = sb.ToString().ToCharArray();
Array.Reverse(harfler);
string ters = new string(harfler);
Console.WriteLine(ters + " (" + ters.Length + ")");
```

Çıktı

```
merhaba C# 1!2026
6202!1 #C abahrem (17)
```

`Append`, `Insert`, `Remove` gibi metotlar değişikliği yaptıktan sonra **aynı nesneyi** döndürür; bu yüzden `sb.Append('!').Append(2026)` gibi art arda **zincirleme** (chaining) çağrı yapılabilir. `StringBuilder`'da `Reverse` metodu yoktur. Ters çevirmek için yukarıdaki kalıbı kullanırsın: `ToCharArray()` ile `char` dizisi al, Bölüm 7'deki `Array.Reverse` ile çevir, `new string(dizi)` ile string'e dön.

Döngüde string birleştirmenin maliyeti

Bir döngüde `s += ...` yazdığında her adımda yeni bir string oluşturulur ve eski metnin **bütün karakterleri** oraya kopyalanır. 1'er karakterlik `n` ekleme için toplam kopya sayısı $1 + 2 + \ldots + n \approx n^2/2$'dir: 1 000 ekleme için yaklaşık 500 bin, 100 000 ekleme için yaklaşık **5 milyar** karakter kopyalanır. `StringBuilder` ise ekleri doğrudan kendi tamponuna yazar; toplam iş `n` ile orantılı kalır. Döngüde metin biriktirirken kalıp hep aynıdır: döngüden önce `StringBuilder sb = new StringBuilder();`, döngünün içinde `sb.Append(...)`, döngüden sonra `sb.ToString()`. Birkaç parçayı bir kerelik birleştirmek için `+` ve `$"..."` yeterlidir.

> **Sınavda Çıkar: string ve StringBuilder**
>
> "string ile StringBuilder arasındaki fark nedir?" klasik bir sözlü/yazılı sorudur:
>
> - **string** → değiştirilemez (immutable). Her değişiklik yeni nesne üretir.
> - **StringBuilder** → değiştirilebilir (mutable), döngüde metin oluştururken hızlıdır.
>
> Tuzak: İki `StringBuilder`'ı `==` ile karşılaştırmak **içeriği değil adresi** karşılaştırır; iki farklı nesne aynı metni tutsa bile `False` çıkar. İçerik karşılaştırmak için `sb1.ToString() == sb2.ToString()` yaz.

8.9 Dönüşümler: string, sayı ve char

Kullanıcıdan gelen veriler, dosyadan okunan satırlar hep string'dir; hesap yapmak için sayıya çevirmen gerekir. Tersine, bir sayının basamaklarıyla uğraşmak için onu string'e çevirmek işini kolaylaştırır:

Dönüşüm	Yöntem	Örnek
sayı → string	`x.ToString()` ya da `$"{x}"`	`42.ToString()` → `"42"`
string → `int`	`int.Parse(s)` ya da `Convert.ToInt32(s)`	`int.Parse("42")` → `42`
string → `double`	`double.Parse(s)` ya da `Convert.ToDouble(s)`	`double.Parse("3,75")` → `3,75`
güvenli dönüşüm	`int.TryParse(s, out int sonuc)`	başarısızsa `false` verir
`char` → string	`c.ToString()` ya da `"" + c`	`'a'.ToString()` → `"a"`
rakam `char` → `int`	`c - '0'`	`'7' - '0'` → `7`
string ↔ `char[]`	`s.ToCharArray()` / `new string(dizi)`	

donusumler.cs

```
int yil = 2026;
string metin = yil.ToString();
Console.WriteLine(metin.Length + " basamaklı");
Console.WriteLine(metin + 1);
Console.WriteLine(int.Parse("2026") + 1);
Console.WriteLine(double.Parse("3,75") * 2);

bool tamSayiMi = int.TryParse("4.2", out int deger);
Console.WriteLine(tamSayiMi + " " + deger);

int basamakToplami = 0;
foreach (char c in metin)
{
    basamakToplami += c - '0';
}
Console.WriteLine("Basamak toplamı: " + basamakToplami);
```

```
Çıktı
4 basamaklı
20261
2027
7,5
False 0
Basamak toplamı: 10
```

Bölüm 5'te bir sayının basamaklarını `% 10` ve `/ 10` ile ayırmıştın; string'e çevirip karakterleri dolaşmak da aynı işi yapar. `metin + 1` ise toplama değil birleştirme yaptı: `"20261"`. `TryParse` hata fırlatmaz; çevirme başarısızsa `false` verir ve `deger` `0` kalır.

`Parse`'a sayıya çevrilemeyen bir metin verirsen program `FormatException` ile çöker:

sayiHatasi.cs

```
Console.WriteLine(int.Parse("42"));
Console.WriteLine(int.Parse(" 42 "));
Console.WriteLine(int.Parse("4.2"));
```

```
Çıktı – program yakalanmamış istisna ile sonlanır
42
42
Unhandled exception. System.FormatException: The input string '4.2' was not in a correct format.
   at Program.<Main>$(String[] args) in sayiHatasi.cs:line 3
```

`"4.2"` bir tam sayı değildir. `"12a"` ve `""` de aynı hatayı verir. `int.Parse` baştaki ve sondaki boşluklara toleranslıdır (ikinci satır `42` yazdı), ama metnin içindeki harf, ondalık ayıracı ya da boşluk hata sebebidir. Bu hatayı `try-catch` ile yakalamayı Bölüm 12'de öğreneceksin; o zamana kadar kullanıcıdan gelen metin için `TryParse` kullan.

8.10 Klasik string algoritmaları

Sınavların "kod yazma" kısmında string soruları çok sevilir. Aşağıdaki algoritmalar, karşına çıkacak soruların büyük kısmının iskeletidir. Hepsi aynı fikre dayanır: **karakter karakter dolaş, karar ver, sonucu biriktir.**

Ters çevirme ve palindrom

Palindrom, tersten okunuşu da aynı olan metindir: `kayak`, `ey edip adana'da pide ye`. Ters çevirmeyi döngüyle yapmak için string'i **sondan başa** dolaşırız. Palindrom kontrolünde ise önce metni temizleriz (küçük harf, yalnız harf ve rakamlar), sonra **iki uçtan ortaya doğru** karşılaştırırız:

palindrom.cs

```
using System.Text;

Console.WriteLine(TersCevir("Bilgisayar"));
Console.WriteLine(PalindromMu("kayak") + " " + PalindromMu("Kitap"));
Console.WriteLine(PalindromMu("Ey Edip Adana'da pide ye"));

static string TersCevir(string s)
{
    StringBuilder sb = new StringBuilder();
    for (int i = s.Length - 1; i >= 0; i--)
    {
        sb.Append(s[i]);
    }
    return sb.ToString();
```

```
}

static bool PalindromMu(string s)
{
    StringBuilder temiz = new StringBuilder();   // sadece harf ve rakamlar
    foreach (char c in s.ToLowerInvariant())
    {
        if (char.IsLetterOrDigit(c))
        {
            temiz.Append(c);
        }
    }
    int i = 0, j = temiz.Length - 1;             // iki uçtan ortaya
    while (i < j)
    {
        if (temiz[i] != temiz[j])
        {
            return false;
        }
        i++;
        j--;
    }
    return true;
}
```

Çıktı

```
rayasigliB
True False
True
```

İki indeksli kontrol ilk farklı karakterde `false` döndürüp durur ve ters çevrilmiş bir kopya oluşturmaz. Karşılaştırma için `ToLowerInvariant` kullandık: iç karşılaştırmada kültür kurallarına ihtiyacımız yok.

Sesli harf ve kelime sayma

Bir karakterin bir kümede olup olmadığını anlamanın pratik yolu: kümedeki karakterleri bir string'e yaz ve `Contains(c)` ile ara (`IndexOf(c) != -1` de aynı işi yapar). Kelime saymak için ise `Split` ve `RemoveEmptyEntries` yeterlidir:

metinAnaliz.cs

```
string sesliler = "aeıioöuü";

Console.Write("Bir cümle yaz: ");
string cumle = (Console.ReadLine() ?? "").ToLower();
int sesli = 0, sessiz = 0;
foreach (char c in cumle)
{
    if (sesliler.Contains(c))
    {
        sesli++;
    }
    else if (char.IsLetter(c))
    {
        sessiz++;
    }
}
Console.WriteLine("Kelime: " + KelimeSayisi(cumle));
Console.WriteLine("Sesli: " + sesli + ", sessiz: " + sessiz);
Console.WriteLine("Boş metin: " + KelimeSayisi("   ") + " kelime");
```

```
static int KelimeSayisi(string metin)
{
    return metin.Split(' ', StringSplitOptions.RemoveEmptyEntries).Length;
}
```

Çıktı

```
Bir cümle yaz:   İzmir'de   Bilgisayar Mühendisliği okuyorum
Kelime: 4
Sesli: 16, sessiz: 21
Boş metin: 0 kelime
```

- Türkçe metin olduğu için küçük harfe çevirirken kültüre duyarlı `ToLower()` kullandık; böylece `İ` → `i`, `I` → `ı` olur ve ikisi de sesli harf sayılır. Boşluk ve kesme işareti harf olmadığı için hiçbir sayaca girmez.
- Yalnız boşluktan oluşan metinde `RemoveEmptyEntries` sıfır parça bırakır, yani `0` kelime çıkar. Seçeneksiz `"".Split(' ')` ise **1 elemanlı** bir dizi (`{""}`) verir ve kelime sayısı yanlışlıkla 1 olur. Sınırdaki durumları (boş girdi, tek kelime, fazla boşluk) düşünmek, tam puan ile yarım puan arasındaki farktır.

Sezar şifreleme

Sezar şifresi, her harfi alfabede sabit bir miktar (anahtar) kaydırır: anahtar 3 ise A → D, B → E, ..., X → A, Y → B, Z → C. Alfabenin sonunda başa dönmek için `%` kullanılır. Harfin alfabedeki sırası `c - 'A'` ile bulunur (A=0, B=1, ..., Z=25):

sezar.cs

```
using System.Text;

string mesaj = "Vize Pazartesi, saat 10:00!";
string gizli = Sifrele(mesaj, 3);
Console.WriteLine(gizli);
Console.WriteLine(Sifrele(gizli, 26 - 3));
Console.WriteLine(Sifrele(gizli, -3));
Console.WriteLine(Sifrele("xyz XYZ", 3));

static string Sifrele(string metin, int anahtar)
{
    int k = (anahtar % 26 + 26) % 26;   // eksi anahtarı 0–25 arasına çevir
    StringBuilder sb = new StringBuilder();
    for (int i = 0; i < metin.Length; i++)
    {
        char c = metin[i];
        if (c >= 'A' && c <= 'Z')
        {
            c = (char)((c - 'A' + k) % 26 + 'A');
        }
        else if (c >= 'a' && c <= 'z')
        {
            c = (char)((c - 'a' + k) % 26 + 'a');
        }
        sb.Append(c);
    }
    return sb.ToString();
}
```

```
Çıktı
Ylch Sdcduwhvl, vddw 10:00!
Vize Pazartesi, saat 10:00!
Vize Pazartesi, saat 10:00!
abc ABC
```

Şifreyi çözmek için ayrı bir metoda gerek yok: 3 ileri kaydırmanın tersi, 26 − 3 = 23 ileri kaydırmaktır; doğrudan `-3` de verilebilir. Bunu `k = (anahtar % 26 + 26) % 26` satırı sağlar: C#'ta `%` işleci bölünenin işaretini korur (`-3 % 26` = `-3`), `+ 26` ve tekrar `% 26` sonucu her zaman 0–25 aralığına getirir. Bu satır olmasaydı eksi anahtarda harf `'A'`'dan önceki karakterlere kayardı. `(char)` dönüşümü şart, çünkü `char + int` işleminin sonucu `int`'tir. Bu basit sürüm yalnız İngilizce harfleri kaydırır; rakamlar, noktalama ve Türkçe harfler olduğu gibi kalır.

Harf frekansı: int[26]

"Bir metinde hangi harf kaç kez geçiyor?" sorusunun klasik çözümü 26 elemanlı bir sayaç dizisidir. Püf noktası, harfi doğrudan **indekse** çevirmektir: `'a' - 'a'` = 0, `'b' - 'a'` = 1, ..., `'z' - 'a'` = 25:

harfFrekansi.cs

```
string metin = "Programlama dili";
int[] sayac = new int[26];

foreach (char c in metin.ToLowerInvariant())
{
    if (c >= 'a' && c <= 'z')
    {
        sayac[c - 'a']++;
    }
}

int enCok = 0;
for (int i = 0; i < 26; i++)
{
    if (sayac[i] > 0)
    {
        Console.Write($"{(char)('a' + i)}={sayac[i]} ");
    }
    if (sayac[i] > sayac[enCok])
    {
        enCok = i;
    }
}
Console.WriteLine();
Console.WriteLine($"En sık: {(char)('a' + enCok)} ({sayac[enCok]})");
```

```
Çıktı
a=3 d=1 g=1 i=2 l=2 m=2 o=1 p=1 r=2
En sık: a (3)
```

Burada `ToLower()` yerine `ToLowerInvariant()` kullandık: Türkçe kurallarla `I` harfi `ı` olurdu ve `'a'..'z'` aralığının dışına düşüp sayılmazdı. Aynı fikirle **anagram** kontrolü de yapılır: İki kelimenin sayaç dizileri birebir aynıysa, biri diğerinin harflerinin yer değiştirmesiyle elde edilmiştir (`elma` ↔ `alem`).

Bölüm Özeti – Bilmen Gerekenler

- `string` bir **başvuru tipidir**, ama **değiştirilemez**; indeksler 0'dan başlar, son indeks `Length - 1`'dir. `Length` bir özelliktir (parantezsiz).

- `ToUpper`, `Replace`, `Trim` yeni string döndürür. `s.ToUpper();` tek başına bir şey yapmaz; `s = s.ToUpper();` yaz. `s[0] = 'x'` derlenmez.
- `IndexOf` bulamazsa **-1** verir. `Substring(b, n)` → `b`'den başlayıp `n` **karakter**; ikinci sayı uzunluktur. `s[b..e]` aralığında ise `e` bitiş indeksidir (hariç).
- Ham metin sabiti `"""..."""` içinde `\`, `"` ve satır sonu olduğu gibi yazılır; kapanan `"""` işaretinin girintisi her satırdan silinir, daha az girintili satır derlenmez (CS8999). `$"""..."""` ile enterpolasyon yapılır.
- Sınır dışı `s[i]` → `IndexOutOfRangeException`; sınır dışı `Substring` → `ArgumentOutOfRangeException`. `Split(ayraç, StringSplitOptions.RemoveEmptyEntries)` boş parçaları atar; `string.Join` birleştirir.
- `ToUpper()` / `ToLower()` kültüre duyarlıdır: Türkçede `"i".ToUpper()` → `"İ"`, `"I".ToLower()` → `"ı"`. Program içi karşılaştırmada `ToUpperInvariant()` ya da `StringComparison.OrdinalIgnoreCase`.
- `==` ve `Equals` string'lerin **içeriğini** karşılaştırır. `CompareTo` / `string.Compare` yalnız işaretiyle kullanılır ve kültüre duyarlıdır; `Ordinal` karakter koduna bakar.
- `char.IsDigit/IsLetter/IsUpper/ToUpper`; rakam karakteri → sayı: `c - '0'`. `'a' + 1` bir `int`'tir (98); karakter için `(char)('a' + 1)`.
- Döngüde metin oluştururken **`StringBuilder`** kullan (`using System.Text;`). `Reverse` yoktur: `char[]` + `Array.Reverse`.
- `int.Parse` / `double.Parse` geçersiz metinde `FormatException` fırlatır; güvenli yol `int.TryParse`.

Sorular

A) Çoktan Seçmeli

1 `"Programlama".Substring(2, 5)` ifadesinin sonucu nedir?

A) `"ogr"` B) `"gram"` C) `"ogram"` D) `"ogramla"` E) `"gramlam"`

2 Aşağıdaki program Türkçe (`tr-TR`) kültür ayarlı bir bilgisayarda çalışıyor. Ne yazdırır?

```
string s = "istanbul";
Console.WriteLine(s.ToUpper() + " " + s.ToUpperInvariant());
```

A) `İSTANBUL ISTANBUL`
B) `ISTANBUL ISTANBUL`
C) `İSTANBUL İSTANBUL`
D) `ISTANBUL İSTANBUL`
E) `istanbul ISTANBUL`

3 Aşağıdaki program ne yazdırır?

```
string s = "merhaba";
s.ToUpper();
s.Replace('a', 'e');
string t = s;
t = t + "!";
Console.WriteLine(s + t.Length);
```

A) `MERHABA7` B) `merhebe8` C) `MERHEBE8` D) `merhaba7` E) `merhaba8`

4 Aşağıdaki program ne yazdırır?

```
char c = 'a';
int x = "merhaba".IndexOf('z');
Console.WriteLine(x + " " + (c + 1) + " " + (char)(c + 1));
```

A) `-1 b b` B) `-1 98 b` C) `0 98 b` D) `-1 a1 b` E) `-1 98 98`

5 `string s = "Veri";` tanımından sonra aşağıdaki ifadelerden hangisi çalışırken `ArgumentOutOfRangeException` hatasına yol açar?

A) `s[3]` B) `s.Substring(4)` C) `s.Substring(1, 3)` D) `s.Substring(2, 3)` E) `s.IndexOf('z')`

B) Çıktı Tahmini

6 Aşağıdaki program ekrana ne yazdırır? (Program Türkçe kültür ayarıyla çalışıyor.)

```
string s = "Programlama Dili";
Console.WriteLine(s.IndexOf('a') + " " + s.LastIndexOf('a'));
Console.WriteLine(s.IndexOf("gram") + " " + s.IndexOf('x'));
Console.WriteLine(s.Substring(12, 4).ToUpper());
Console.WriteLine(s[s.Length - 1] + s[0]);
Console.WriteLine("" + s[1] + s[2]);
```

7 Aşağıdaki program ekrana ne yazdırır?

```
using System.Text;

string s = "abc";
string t = s;
s = s + "d";
s.ToUpper();
Console.WriteLine(s + " " + t);

StringBuilder sb = new StringBuilder("abc");
StringBuilder sb2 = sb;
sb.Append("d").Insert(0, 'x');
Console.WriteLine(sb + " " + sb2);
sb2.Remove(1, 2);
Console.WriteLine(sb);
```

8 Aşağıdaki program ekrana ne yazdırır?

```
string s = "a1B2c3";
int toplam = 0;
string harfler = "";
foreach (char c in s)
{
    if (char.IsDigit(c))
    {
        toplam += c;
    }
    else
    {
        harfler = char.ToUpper(c) + harfler;
    }
}
Console.WriteLine(harfler);
Console.WriteLine(toplam);
Console.WriteLine('a' + 1 + "|" + (char)('a' + 1));
Console.WriteLine(string.Compare("elma", "ekmek", StringComparison.Ordinal) > 0);
```

C) Kod Yazma

9 Kullanıcıdan bir kurum adı okuyan bir program yaz. Program her kelimenin ilk harfini büyük, kalanını küçük yazmalı ve kelimelerin baş harflerinden bir kısaltma üretmeli. Kelimeler arasında birden fazla boşluk olabilir; Türkçe harfler doğru çevrilmeli (`i` → `İ`).

```
Kurum adı:   istanbul   TEKNİK üniversitesi
Düzgün yazım: İstanbul Teknik Üniversitesi
Kısaltma    : İTÜ
```

10 Bir kelimedeki **art arda tekrar eden** harfleri “harf + tekrar sayısı” biçiminde sıkıştıran `static string Sikistir(string s)` metodunu ve bu metodu kullanıcıdan okunan bir kelimeyle deneyen programı yaz. Örnekler: `aaabccdddd` → `a3b1c2d4`, `xyz` → `x1y1z1`.

Çözümler

Cevap anahtarı

1 C	2 A	3 E	4 B	5 D

1 Cevap: C

İndeksler: P0 r1 o2 g3 r4 a5 m6 l7 a8 m9 a10. `Substring(2, 5)` → 2. indeksten başlayıp **5 karakter**: `o`, `g`, `r`, `a`, `m` = `"ogram"`. A (3 karakter) ikinci sayıyı bitiş indeksi sanmaktan çıkar. B, 3. indeksten 4 karakter alır. D 7 karakter alır. E ise 3. indeksten 7 karakterdir.

2 Cevap: A

Türkçe kültürde `ToUpper()` küçük `i` harfini `İ` yapar → `İSTANBUL`. `ToUpperInvariant()` kültürden bağımsızdır, `i` → `I` → `ISTANBUL`. B İngilizce kültürü varsayıyor. C ve D iki metodun davranışını aynı ya da ters sanıyor. E, `ToUpper()`'ın hiçbir şey yapmadığını varsayıyor.

Çıktı

```
İSTANBUL ISTANBUL
```

3 Cevap: E

string değiştirilemez. `s.ToUpper()` ve `s.Replace(...)` yeni string'ler üretir ama sonuçları hiçbir değişkene atanmadığı için kaybolur; `s` hâlâ `"merhaba"`'dır. `t = s` ikisini aynı nesneye bağlar, ama `t + "!"` **yeni** bir string (`"merhaba!"`, 8 karakter) üretir ve `t` ona bağlanır; `s` etkilenmez. `s + t.Length` → `"merhaba" + 8` → `merhaba8`. A, B, C `s`'nin değiştiğini varsayıyor. D, `t.Length`'i 7 sanıyor.

Çıktı

```
merhaba8
```

4 Cevap: B

`IndexOf` bulamayınca **-1** verir (0 değil; 0 "ilk karakter" demektir). `c + 1` işleminde `char` ile `int` toplanır ve sonuç `int` olur: 98 (parantezler yüzünden birleştirme de olmaz). `(char)(c + 1)` sonucu tekrar `char` yapar: `b`. A, `c + 1`'in `char` olduğunu sanıyor. C `IndexOf`'u 0 sanıyor. D, `c + 1`'i birleştirme sanıyor. E, `(char)` dönüşümünü unutmuş.

Çıktı

```
-1 98 b
```

5 Cevap: D

`"Veri"`'nin uzunluğu 4'tür. `Substring(2, 3)` → 2. indeksten başlayıp 3 karakter ister, ama sonda yalnız 2 karakter (`ri`) vardır → `ArgumentOutOfRangeException`. A geçerlidir (`'i'`). B hata vermez: başlangıç = uzunluk olduğu için boş string verir. C `"eri"` verir (1, 2, 3. indeksler). E bulamadığı için `-1` verir; hata vermez.

6

`s` = `"Programlama Dili"`: P0 r1 o2 g3 r4 a5 m6 l7 a8 m9 a10 ␣11 D12 i13 l14 i15, uzunluk 16.

- İlk `'a'` 5. indekste, son `'a'` 10. indekste.
- `"gram"` 3. indeksten başlar; `'x'` yok → `-1`.
- `Substring(12, 4)` → 12. indeksten **4 karakter** → `"Dili"`. Türkçe kültürde `ToUpper()` bunu `DİLİ` yapar (noktalı büyük `İ`).
- **Tuzak:** `s[15] + s[0]` iki `char`'ın toplamıdır, birleştirme değil! `'i'` (105) + `'P'` (80) = **185**.
- `"" + 'r' + 'o'` → başta string olduğu için birleştirme yapılır: `ro`.

Çıktı

```
5 10
3 -1
DİLİ
185
ro
```

7

- `t = s` → ikisi de `"abc"`'yi gösterir. `s = s + "d"` → `s` yeni nesne `"abcd"`'yi gösterir, `t` hâlâ `"abc"`. `s.ToUpper()` sonucu atanmadığı için etkisizdir → `abcd abc`.
- `sb2 = sb` → **aynı** `StringBuilder` nesnesi. `Append("d")` → `"abcd"`, `Insert(0, 'x')` → `"xabcd"`. İki değişken aynı nesneyi gösterdiği için ikisi de `xabcd` yazar.
- `sb2.Remove(1, 2)` → 1. indeksten başlayıp **2 karakter** siler (`a` ve `b`) → `"xcd"`. `sb` de aynı nesne olduğu için `xcd` yazılır.

Çıktı

```
abcd abc
xabcd xabcd
xcd
```

8

- Harfler: `char.ToUpper(c) + harfler` her yeni harfi **başa** ekler (`char + string` → string). `a` → `"A"`, `B` → `"BA"`, `c` → `"CBA"`.
- Rakamlar: `toplam += c` rakamın değerini değil **karakter kodunu** ekler: `'1'` + `'2'` + `'3'` = 49 + 50 + 51 = **150** (6 değil!).
- `'a' + 1 + "|" + (char)('a' + 1)`: soldan sağa, önce `'a' + 1` = **98** (`int`), sonra `"98|"`, sonra `'b'` eklenir → `98|b`.
- Ordinal karşılaştırmada `"elma"` ile `"ekmek"`'in ilk farkı 1. indekste: `'l'` (108) > `'k'` (107) → sonuç pozitif → `True`.

Çıktı

```
CBA
150
98|b
True
```

9

Satırı `Split(' ', StringSplitOptions.RemoveEmptyEntries)` ile kelimelere ayırıyoruz; böylece fazla boşluklar sorun olmuyor. Her kelimenin ilk harfini `Substring(0, 1).ToUpper()`, kalanını `Substring(1).ToLower()` ile çeviriyoruz. Parametresiz `ToUpper()` Türkçe kültürü kullandığı için `istanbul` → `İstanbul` olur, kısaltma `İTÜ` çıkar; `TEKNİK` küçültülürken de `İ` harfi doğru çevrilir. Sonuçları `StringBuilder` ile biriktiriyoruz.

kisaltma.cs

```
using System.Text;

Console.Write("Kurum adı: ");
string[] kelimeler = (Console.ReadLine() ?? "")
    .Split(' ', StringSplitOptions.RemoveEmptyEntries);

StringBuilder duzgun = new StringBuilder();
StringBuilder kisa = new StringBuilder();
foreach (string k in kelimeler)
{
    string ilk = k.Substring(0, 1).ToUpper();
    duzgun.Append(ilk).Append(k.Substring(1).ToLower()).Append(' ');
    kisa.Append(ilk);
}
Console.WriteLine("Düzgün yazım: " + duzgun.ToString().Trim());
Console.WriteLine("Kısaltma    : " + kisa);
```

Çıktı

```
Kurum adı:   istanbul   TEKNİK üniversitesi
Düzgün yazım: İstanbul Teknik Üniversitesi
Kısaltma    : İTÜ
```

10

Dış döngü her yeni harf grubunun başında durur. İç döngü, aynı harf devam ettikçe sayacı ve indeksi ilerletir. İç döngü koşulundaki `i < s.Length` kontrolünün **önce** yazılması şarttır: Kısa devre sayesinde `i` sona ulaştığında `s[i]` hiç çalışmaz, böylece `IndexOutOfRangeException` oluşmaz.

sikistir.cs

```
using System.Text;

Console.Write("Kelime: ");
string kelime = Console.ReadLine() ?? "";
Console.WriteLine("Sıkıştırılmış: " + Sikistir(kelime));
Console.WriteLine(Sikistir("xyz") + " " + Sikistir("kkkkkkkkkkk"));

static string Sikistir(string s)
{
    StringBuilder sb = new StringBuilder();
    int i = 0;
    while (i < s.Length)
    {
        char c = s[i];
        int adet = 0;
        while (i < s.Length && s[i] == c)
        {
            adet++;
            i++;
        }
```

```
            sb.Append(c).Append(adet);
        }
        return sb.ToString();
    }
```

Çıktı

```
Kelime: aaabccdddd
Sıkıştırılmış: a3b1c2d4
x1y1z1 k12
```

PROJE 1

Öğrenci Not Sistemi

İlk sekiz bölümde öğrendiklerini ilk kez **tek bir gerçek programda** birleştiriyoruz: `Console.ReadLine` ile güvenli girdi okuma (Bölüm 3), koşullar ve `switch` (Bölüm 4), döngüler (Bölüm 5), metotlar (Bölüm 6), diziler (Bölüm 7) ve String işlemleri (Bölüm 8). Kodu okumadan önce kendin yazmayı dene; takıldığın yerde "Tasarım" ve "Kod" başlıklarındaki açıklamalara bak.

> **Bu Bölümde Öğreneceklerin**
>
> Bu projeyi bitirdiğinde kullanıcı ne yazarsa yazsın çökmeyen, metotlara bölünmüş ve birden çok diziyi birlikte yöneten bir konsol programını kendi başına yazabileceksin.

Amaç

Bir dersin hocasının işini kolaylaştıran bir konsol programı yazacağız. Hoca öğrencilerin adlarını ve notlarını girecek; program her öğrencinin ortalamasını ve harf notunu hesaplayacak, sınıfın durumunu özetleyecek ve bir menü üzerinden istenen bilgiyi gösterecek. Kullanıcı ne yazarsa yazsın program **çökmemeli**.

Gereksinimler

- Program önce öğrenci sayısını sorar (1–50 arası).
- Her öğrenci için **ad soyad**, **vize** ve **final** notu girilir. Notlar 0–100 arası tam sayıdır. Geçersiz girişte (harf, boş satır, aralık dışı sayı) program uyarır ve **aynı soruyu tekrar sorar**.
- Ortalama = vize × 0.4 + final × 0.6.
- Harf notu aşağıdaki tabloya göre verilir (ortalama, harfin altındaki değere eşit veya ondan büyükse o harf alınır):

Harf	AA	BA	BB	CB	CC	DC	DD	FD	FF
En düşük ortalama	90	85	80	75	70	65	60	50	0

- Veriler girildikten sonra bir **menü** gösterilir; kullanıcı `0` seçene kadar menü tekrar eder:
- `1` → Bütün öğrencileri tablo halinde listele (no, ad, vize, final, ortalama, harf).
- `2` → İstatistik: sınıf ortalaması, en yüksek ve en düşük ortalamalı öğrenci.
- `3` → Sınıf ortalamasının üstündeki öğrenciler.
- `4` → Harf notu dağılımı: her harf için o harfi alan öğrenci sayısı kadar `*`.
- `5` → İsimle arama: aranan metin ismin herhangi bir yerinde geçerse bulunur; büyük/küçük harf önemsizdir (`ay` ile `AY` aynı sonucu verir).
- `6` → Öğrencileri ortalamaya göre büyükten küçüğe sırala ve listele.
- `0` → Çıkış.

Tasarım

Veriler: paralel diziler

Her öğrencinin dört bilgisi var: ad, vize, final, ortalama. Bunları dört ayrı dizide tutacağız. Dizilerin **aynı indeksi aynı öğrenciye** aittir; bu yapıya **paralel diziler** (parallel arrays) denir:

```
indeks         0              1             2             3
adlar       Ayşe Yılmaz    Mehmet Can    Zeynep Kaya   İbrahim Demir
vizeler        70             50            95            60
finaller       85             58            88            40
ortalamalar    79,0           54,8          90,8          48,0
```

Dizileri en fazla 50 kutuluk açıyoruz (`kapasite`). Gerçek öğrenci sayısını `sayi` adlı bir **sayaç** tutar; sayaç hangi kutuların dolu olduğunu söyler. (Bölüm 13'te boyutu sonradan değişebilen `List<T>`'yi öğreneceksin.)

> **Sınavda Çıkar: Dolu kutu sayısı**
>
> Diziler 50 kutuluk, ama sadece ilk `sayi` kutusu doludur. `foreach` ile bütün diziyi gezersen boş kutuların `0` değerlerini de sayarsın. "İlk n öğrenci" içeren bir soruda `for` döngüsü ve sayaç kullan.

Harf notu: switch ifadesi

Harf notunu `if-else` zinciri yerine bir **switch ifadesiyle** buluyoruz (Bölüm 4). Harf listesi iki yerde görünür: `HarfNotu` metodunda ve `Histogram` metodunda. Harf tablosu değişirse ikisini de birlikte düzeltmelisin.

Metotlar

Her işi tek bir metoda veriyoruz. Ana kod sadece verileri okur ve menü döngüsünü çalıştırır; asıl işi metotlar yapar. Metotlar ana koddan sonra yazılır:

Metot	Görevi	Dönüş tipi
`SayiOku(mesaj, min, max)`	`min`–`max` arası geçerli bir tam sayı girilene kadar sorar	`int`
`MetinOku(mesaj)`	Boş olmayan bir satır okur	`string`
`OrtalamaHesapla(vize, fin)`	%40 vize + %60 final	`double`
`HarfNotu(ort)`	Ortalamanın harf notu (switch ifadesi)	`string`
`SinifOrtalamasi(ortalamalar, sayi)`	İlk `sayi` ortalamanın ortalaması	`double`
`MenuYazdir()`	Menüyü ekrana yazar	`void`
`Listele(...)`	Öğrenci tablosunu yazar	`void`
`Istatistik(...)`	Sınıf ortalaması, en yüksek, en düşük	`void`
`OrtalamaUstu(...)`	Ortalamanın üstündekileri yazar	`void`
`Histogram(...)`	Harf notu dağılımını yıldızlarla çizer	`void`
`Ara(aranan, ...)`	İsimde geçen metne göre arar	`void`
`OrtalamayaGoreSirala(...)`	Dört diziyi birlikte büyükten küçüğe sıralar	`void`

Kod

Tam programı aşağıda tek parça halinde bulacaksın. Önce önemli kısımların neden öyle yazıldığına bakalım.

> **İpucu**
>
> Programı parça parça yaz. Önce yalnızca `SayiOku` metodunu yazıp farklı girdilerle dene; sonra menüyü ekle. Her parçayı çalıştırıp görmeden bir sonrakine geçme.

1. Çökmeyen girdi okuma

Bütün girdileri `Console.ReadLine()` ile satır olarak okuyoruz. Bu metot boş gelirse `null` döndürebilir; `?? ""` bu durumda boş metin verir. Ardından `Trim()` baştaki ve sondaki boşlukları atar.

Sayı okurken `int.TryParse` kullanıyoruz. `abc`, boş satır, `7.5` ya da `int` sınırını aşan bir sayı için `TryParse` `false` döner ve program çökmez. Sayı geçerliyse `min`–`max` aralığı ayrıca kontrol edilir.

Bir adım başarısız olursa uyarı yazılır ve `while (true)` döngüsü soruyu tekrarlar. Geçerli bir sayı bulunduğunda `return` hem metottan hem döngüden çıkar. `MetinOku` da kullanıcı boş satır girdiği sürece soruyu tekrarlar (`do–while`).

2. Ortalama ve harf notu

`OrtalamaHesapla` formülü `vize * 0.4 + fin * 0.6` yerine `(vize * 4 + fin * 6) / 10.0` olarak hesaplar. İki formül matematikte aynıdır; ama Bölüm 2'de gördüğün gibi `0.4` ve `0.6` bilgisayarda **tam olarak** saklanamaz ve harf sınırında yanlış not verilebilir:

> **Sık Yapılan Hata: Sınırda yanlış harf notu**

```
int vize = 6, fin = 96;
Console.WriteLine(vize * 0.4 + fin * 0.6);
Console.WriteLine((vize * 4 + fin * 6) / 10.0);
```

```
Çıktı
59,99999999999999
60
```

Gerçek ortalama tam 60, yani **DD**. Ama ilk formülle 60'tan küçük bir sayı çıkıyor ve öğrenci **FD** alıyor! İkinci formülde çarpma ve toplama tam sayılarla yapılır, en sonda tek bir bölme kalır; sonuç her zaman doğru çıkar.

`HarfNotu` bir switch ifadesidir (Bölüm 4). Kollar yukarıdan aşağıya denenir ve ilk doğru olan kolun değeri alınır. Bu yüzden büyük sınırlar önce yazılır; `_` ise her şeyi yakalayan son koldur. Ortalaması 79,0 olan bir öğrenci önce `>= 90`, `>= 85` ve `>= 80` kollarında eşleşmez; `>= 75` kolunda eşleşir ve `CB` alır.

> **Sınavda Çıkar: Sınır değeri**
>
> `ort >= 60` ile `ort > 60` farklı sonuç verir. `>` kullanılırsa tam 60 alan öğrenci `DD` yerine `FD` alır. Bu tür sorularda sınır değerinin hangi kola girdiğine dikkat et.

3. Menü

Menü bir `do–while` döngüsüdür: menü **en az bir kere** gösterilir ve `0` seçilene kadar tekrar eder. Seçim `SayiOku` ile 0–6 aralığında okunduğu için `switch` geçersiz bir değer almaz. Her `case`'in sonunda `break` olmalıdır; yoksa derleyici hata verir (CS0163).

4. Tablo, istatistik ve histogram

- `Listele` metni `{0,-3}` ve `{1,-16}` ile sola, sayıları `{2,5}` ile sağa yaslar. `{4,6:F1}` ortalamayı 6 kutuya ve bir ondalık basamakla yazar. Çizgi için `new string('-', 46)` kullanılır.
- `Istatistik` en yüksek ve en düşük ortalamanın **değerini değil indeksini** tutar (Bölüm 7). Adı `adlar[enYuksek]` ile alırız.
- `Histogram` harf sayaçlarını, harfin listedeki yerini `Array.IndexOf` ile bularak tutar. Yıldızları iç içe bir `for` döngüsüyle yazar (Bölüm 5).

5. Sıralama: seçilim sıralaması

`OrtalamayaGoreSirala` her turda kalan kısımdaki en büyük ortalamanın indeksini bulur ve onu sıradaki kutuyla yer değiştirir. Yer değiştirirken **dört diziyi de aynı indekste** değiştiririz. Böylece bir öğrencinin adı, vizesi, finali ve ortalaması hep aynı kutuda kalır.

> **Sık Yapılan Hata: Yalnız bir diziyi sıralamak**
>
> Sadece `ortalamalar` dizisini sıralarsan adlar eski yerinde kalır. Sonuçta bir öğrencinin notu başka öğrencinin adının karşısında görünür. Dört diziyi aynı anda değiştirmelisin.

6. İsimle arama

`Contains(aranan, StringComparison.OrdinalIgnoreCase)` büyük ve küçük harfi aynı sayar: `AY` ile `Ay` eşleşir. Ama bu kural Türkçe harfler için yeterli değildir:

> **Sık Yapılan Hata: Türkçe I tuzağı**
>
> `OrdinalIgnoreCase` harfleri karakter koduna göre büyütüp küçültür. Bu yüzden `i` ile `I` eşleşir, ama noktalı `İ` ile `i` eşleşmez. Sonuç: `ibra` araması `İbrahim Demir`'i bulamaz (bkz. örnek çalışma). Türkçe kuralını Kendin Geliştir kısmında deneyeceksin.

Tam program

notSistemi.cs

```
const int kapasite = 50;

Console.WriteLine("===== ÖĞRENCİ NOT SİSTEMİ =====");
int sayi = SayiOku("Öğrenci sayısı: ", 1, 50);

string[] adlar = new string[kapasite];
int[] vizeler = new int[kapasite];
int[] finaller = new int[kapasite];
double[] ortalamalar = new double[kapasite];

for (int i = 0; i < sayi; i++)
{
    Console.WriteLine($"--- {i + 1}. öğrenci ---");
    adlar[i] = MetinOku("Ad soyad: ");
    vizeler[i] = SayiOku("Vize: ", 0, 100);
    finaller[i] = SayiOku("Final: ", 0, 100);
    ortalamalar[i] = OrtalamaHesapla(vizeler[i], finaller[i]);
}

int secim;
do
{
    MenuYazdir();
```

```
    secim = SayiOku("Seçimin: ", 0, 6);
    switch (secim)
    {
        case 1:
            Listele(adlar, vizeler, finaller, ortalamalar, sayi);
            break;
        case 2:
            Istatistik(adlar, ortalamalar, sayi);
            break;
        case 3:
            OrtalamaUstu(adlar, ortalamalar, sayi);
            break;
        case 4:
            Histogram(ortalamalar, sayi);
            break;
        case 5:
            Ara(MetinOku("Aranacak isim: "), adlar, ortalamalar, sayi);
            break;
        case 6:
            OrtalamayaGoreSirala(adlar, vizeler, finaller, ortalamalar, sayi);
            Listele(adlar, vizeler, finaller, ortalamalar, sayi);
            break;
        case 0:
            Console.WriteLine("Program kapatıldı. İyi çalışmalar!");
            break;
    }
} while (secim != 0);

// ---------- Girdi okuma ----------

// min-max arasında geçerli bir tam sayı girilene kadar sorar
static int SayiOku(string mesaj, int min, int max)
{
    while (true)
    {
        Console.Write(mesaj);
        string satir = (Console.ReadLine() ?? "").Trim();
        if (int.TryParse(satir, out int deger) && deger >= min && deger <= max)
        {
            return deger;
        }
        Console.WriteLine($"  Geçersiz! {min}-{max} arası bir tam sayı gir.");
    }
}

// Boş bırakılamayan bir satır okur
static string MetinOku(string mesaj)
{
    string satir;
    do
    {
        Console.Write(mesaj);
        satir = (Console.ReadLine() ?? "").Trim();
    } while (satir == "");
    return satir;
}

// ---------- Hesaplama ----------

// vize * 0.4 + fin * 0.6; ondalık hatası olmasın diye tam sayıyla hesaplanır
static double OrtalamaHesapla(int vize, int fin)
{
    return (vize * 4 + fin * 6) / 10.0;
}
```

```
// Ortalamanın harf notu; kollar yukarıdan aşağıya denenir
static string HarfNotu(double ort) => ort switch
{
    >= 90 => "AA",
    >= 85 => "BA",
    >= 80 => "BB",
    >= 75 => "CB",
    >= 70 => "CC",
    >= 65 => "DC",
    >= 60 => "DD",
    >= 50 => "FD",
    _ => "FF",
};

// İlk sayi tane ortalamanın ortalaması
static double SinifOrtalamasi(double[] ortalamalar, int sayi)
{
    double toplam = 0;
    for (int i = 0; i < sayi; i++)
    {
        toplam += ortalamalar[i];
    }
    return toplam / sayi;
}

// ---------- Menü işlemleri ----------

static void MenuYazdir()
{
    Console.WriteLine();
    Console.WriteLine("1) Liste  2) İstatistik  3) Ortalama üstü");
    Console.WriteLine("4) Harf dağılımı  5) İsimle ara  6) Sırala  0) Çıkış");
}

static void Listele(string[] adlar, int[] vizeler, int[] finaller,
    double[] ortalamalar, int sayi)
{
    Console.WriteLine("{0,-3} {1,-16} {2,5} {3,6} {4,6} {5,5}",
        "No", "Ad Soyad", "Vize", "Final", "Ort.", "Harf");
    Console.WriteLine(new string('-', 46));
    for (int i = 0; i < sayi; i++)
    {
        Console.WriteLine("{0,-3} {1,-16} {2,5} {3,6} {4,6:F1} {5,5}",
            i + 1, adlar[i], vizeler[i], finaller[i], ortalamalar[i],
            HarfNotu(ortalamalar[i]));
    }
}

static void Istatistik(string[] adlar, double[] ortalamalar, int sayi)
{
    int enYuksek = 0, enDusuk = 0;
    for (int i = 1; i < sayi; i++)
    {
        if (ortalamalar[i] > ortalamalar[enYuksek])
        {
            enYuksek = i;
        }
        if (ortalamalar[i] < ortalamalar[enDusuk])
        {
            enDusuk = i;
        }
    }
    Console.WriteLine($"Sınıf ortalaması: {SinifOrtalamasi(ortalamalar, sayi):F2}");
    Console.WriteLine("En yüksek       : {0} ({1:F1})",
        adlar[enYuksek], ortalamalar[enYuksek]);
```

```
    Console.WriteLine("En düşük        : {0} ({1:F1})",
        adlar[enDusuk], ortalamalar[enDusuk]);
}

static void OrtalamaUstu(string[] adlar, double[] ortalamalar, int sayi)
{
    double sinifOrt = SinifOrtalamasi(ortalamalar, sayi);
    Console.WriteLine($"Sınıf ortalamasının ({sinifOrt:F2}) üstündekiler:");
    int sayac = 0;
    for (int i = 0; i < sayi; i++)
    {
        if (ortalamalar[i] > sinifOrt)
        {
            Console.WriteLine($"  {adlar[i],-16} {ortalamalar[i],5:F1}");
            sayac++;
        }
    }
    Console.WriteLine($"Toplam {sayac} öğrenci.");
}

static void Histogram(double[] ortalamalar, int sayi)
{
    string[] harfler = { "AA", "BA", "BB", "CB", "CC", "DC", "DD", "FD", "FF" };
    int[] sayac = new int[harfler.Length];
    for (int i = 0; i < sayi; i++)
    {
        int indeks = Array.IndexOf(harfler, HarfNotu(ortalamalar[i]));
        sayac[indeks]++;
    }
    Console.WriteLine("Harf notu dağılımı:");
    for (int i = 0; i < harfler.Length; i++)
    {
        Console.Write($"  {harfler[i]} | ");
        for (int j = 0; j < sayac[i]; j++)
        {
            Console.Write("*");
        }
        Console.WriteLine();
    }
}

// Büyük/küçük harf duyarsız, isim parçasıyla arama
static void Ara(string aranan, string[] adlar, double[] ortalamalar, int sayi)
{
    bool bulundu = false;
    for (int i = 0; i < sayi; i++)
    {
        if (adlar[i].Contains(aranan, StringComparison.OrdinalIgnoreCase))
        {
            double ort = ortalamalar[i];
            string harf = HarfNotu(ort);
            Console.WriteLine($"  {adlar[i]} -> ortalama {ort:F1} ({harf})");
            bulundu = true;
        }
    }
    if (!bulundu)
    {
        Console.WriteLine($"  \"{aranan}\" bulunamadı.");
    }
}

// Seçilim sıralaması: büyükten küçüğe; dört dizi birlikte değişir
static void OrtalamayaGoreSirala(string[] adlar, int[] vizeler, int[] finaller,
    double[] ortalamalar, int sayi)
{
```

```
    for (int i = 0; i < sayi - 1; i++)
    {
        int enBuyuk = i;
        for (int j = i + 1; j < sayi; j++)
        {
            if (ortalamalar[j] > ortalamalar[enBuyuk])
            {
                enBuyuk = j;
            }
        }

        string geciciAd = adlar[i];
        adlar[i] = adlar[enBuyuk];
        adlar[enBuyuk] = geciciAd;

        int geciciVize = vizeler[i];
        vizeler[i] = vizeler[enBuyuk];
        vizeler[enBuyuk] = geciciVize;

        int geciciFinal = finaller[i];
        finaller[i] = finaller[enBuyuk];
        finaller[enBuyuk] = geciciFinal;

        double geciciOrt = ortalamalar[i];
        ortalamalar[i] = ortalamalar[enBuyuk];
        ortalamalar[enBuyuk] = geciciOrt;
    }
    Console.WriteLine("Öğrenciler ortalamaya göre sıralandı.");
}
```

Örnek çalışma

Aşağıdaki çalışmada 4 öğrenci giriliyor. Üçüncü öğrencinin vizesine önce yanlışlıkla `105` yazılıyor. Ardından menünün bütün seçenekleri sırayla deneniyor: iki arama (`AY` ve `ibra`), sıralama, iki geçersiz seçim (`abc` ve `7`) ve çıkış.

Çıktı

```
===== ÖĞRENCİ NOT SİSTEMİ =====
Öğrenci sayısı: 4
--- 1. öğrenci ---
Ad soyad: Ayşe Yılmaz
Vize: 70
Final: 85
--- 2. öğrenci ---
Ad soyad: Mehmet Can
Vize: 50
Final: 58
--- 3. öğrenci ---
Ad soyad: Zeynep Kaya
Vize: 105
  Geçersiz! 0-100 arası bir tam sayı gir.
Vize: 95
Final: 88
--- 4. öğrenci ---
Ad soyad: İbrahim Demir
Vize: 60
Final: 40

1) Liste  2) İstatistik  3) Ortalama üstü
4) Harf dağılımı  5) İsimle ara  6) Sırala  0) Çıkış
Seçimin: 1
No  Ad Soyad          Vize  Final   Ort.  Harf
```

```
---------------------------------------------
1   Ayşe Yılmaz         70     85   79,0    CB
2   Mehmet Can          50     58   54,8    FD
3   Zeynep Kaya         95     88   90,8    AA
4   İbrahim Demir       60     40   48,0    FF

1) Liste  2) İstatistik  3) Ortalama üstü
4) Harf dağılımı  5) İsimle ara  6) Sırala  0) Çıkış
Seçimin: 2
Sınıf ortalaması: 68,15
En yüksek       : Zeynep Kaya (90,8)
En düşük        : İbrahim Demir (48,0)

1) Liste  2) İstatistik  3) Ortalama üstü
4) Harf dağılımı  5) İsimle ara  6) Sırala  0) Çıkış
Seçimin: 3
Sınıf ortalamasının (68,15) üstündekiler:
  Ayşe Yılmaz       79,0
  Zeynep Kaya       90,8
Toplam 2 öğrenci.

1) Liste  2) İstatistik  3) Ortalama üstü
4) Harf dağılımı  5) İsimle ara  6) Sırala  0) Çıkış
Seçimin: 4
Harf notu dağılımı:
  AA | *
  BA |
  BB |
  CB | *
  CC |
  DC |
  DD |
  FD | *
  FF | *

1) Liste  2) İstatistik  3) Ortalama üstü
4) Harf dağılımı  5) İsimle ara  6) Sırala  0) Çıkış
Seçimin: 5
Aranacak isim: AY
  Ayşe Yılmaz -> ortalama 79,0 (CB)
  Zeynep Kaya -> ortalama 90,8 (AA)

1) Liste  2) İstatistik  3) Ortalama üstü
4) Harf dağılımı  5) İsimle ara  6) Sırala  0) Çıkış
Seçimin: 5
Aranacak isim: ibra
  "ibra" bulunamadı.

1) Liste  2) İstatistik  3) Ortalama üstü
4) Harf dağılımı  5) İsimle ara  6) Sırala  0) Çıkış
Seçimin: 6
Öğrenciler ortalamaya göre sıralandı.
No  Ad Soyad          Vize  Final   Ort.  Harf
---------------------------------------------
1   Zeynep Kaya         95     88   90,8    AA
2   Ayşe Yılmaz         70     85   79,0    CB
3   Mehmet Can          50     58   54,8    FD
4   İbrahim Demir       60     40   48,0    FF

1) Liste  2) İstatistik  3) Ortalama üstü
4) Harf dağılımı  5) İsimle ara  6) Sırala  0) Çıkış
Seçimin: abc
  Geçersiz! 0-6 arası bir tam sayı gir.
```

```
Seçimin: 7
  Geçersiz! 0–6 arası bir tam sayı gir.
Seçimin: 0
Program kapatıldı. İyi çalışmalar!
```

`AY` araması hem **Ay**şe Yılmaz'ı hem de Zeynep K**ay**a'yı buldu. `ibra` araması ise İbrahim Demir'i bulamadı; sebebi yukarıdaki Türkçe I tuzağıdır. Altıncı seçenekten sonra liste ortalamaya göre büyükten küçüğe dizildi; dört dizi birlikte yer değiştirdi.

Bölüm Özeti – Bilmen Gerekenler

- Girdi: `(Console.ReadLine() ?? "").Trim()` ile metin, `int.TryParse` ile sayı okunur. Geçersiz girişte aynı soru tekrar sorulur.
- Paralel diziler: aynı indeks aynı öğrenciye aittir. Sayaç (`sayi`) yalnızca dolu kısmı gösterir.
- Harf notu bir switch ifadesiyle, menü bir `switch` deyimiyle seçilir.
- Sıralarken dört diziyi birlikte değiştir.

Kendin Geliştir

- **Final barajı:** Finalden 50'nin altında alan öğrenci, ortalaması ne olursa olsun `FF` alsın.
- **Geçti/kaldı:** İstatistik ekranına geçen (`DD` ve üstü) ve kalan öğrenci sayısını ekle.
- **Not güncelleme:** İsmi verilen öğrencinin vize veya final notunu değiştirip ortalamasını yeniden hesaplayan bir seçenek ekle. Yeni notu `SayiOku` ile al.
- **Standart sapma:** Sınıf ortalamasının yanında notların standart sapmasını da göster: `Math.Sqrt(Σ(xᵢ − ort)² / n)`.
- **Bütünleme:** `FD` veya `FF` alan öğrenciler için bütünleme notu iste; bütünleme notu final yerine geçsin.
- **Türkçe arama:** `ibra` yazınca `İbrahim`'i bulacak şekilde aramayı düzelt. İpucu: iki metni de `ToLower(new System.Globalization.CultureInfo("tr-TR"))` ile küçült, sonra `Contains` kullan.

DENEME SINAVI

Deneme Vize Sınavı

Bu deneme sınavı kitabın ilk sekiz bölümünü kapsar: C#'a giriş, değişkenler ve operatörler, girdi-çıktı ve biçimlendirme, koşullar, döngüler, metotlar, diziler ve string. Soru türleri ve zorluk, bilgisayar mühendisliği 1. sınıf vize sınavlarına göre ayarlandı; tuzaklar **bilerek** kondu. Kolay görünen sorulara özellikle dikkat et.

Bölüm	Soru sayısı	Soru başına puan	Toplam
A) Çoktan seçmeli	10	3	30
B) Çıktı tahmini	4	10	40
C) Kod yazma	3	10	30
Toplam	**17**		**100**

Süre: 90 dakika. Bilgisayar, kitap ve not kullanmak yok; sadece kâğıt ve kalem.

> **İpucu: Önce süre tutarak çöz**
>
> Sınavı gerçek sınav gibi çöz: 90 dakikalık bir sayaç kur ve süre bitene kadar çözümlere bakma. Kabaca **20 dakikayı** testlere, **35 dakikayı** çıktı sorularına, **35 dakikayı** kod yazmaya ayır. Bir soruda takılırsan işaretle ve geç; sona dönersin. Süre bitince kodlarını Visual Studio'da çalıştırmadan önce çözümlerle karşılaştırıp kendini puanla.

> **Not: Çıktılar Türkçe Windows'a göre**
>
> Sınavdaki bütün çıktılar Türkçe Windows'ta (tr-TR kültürü) çalışan programa göredir: `3.5` ekrana `3,5` yazılır, `"i".ToUpper()` sonucu `"İ"` olur. `bool` değerleri `True` ve `False` olarak yazılır.

> **Not: Kendini nasıl puanlarsın?**
>
> - **Çıktı sorularında** çıktının her satırı eşit puan değerindedir. Eksik boşluk, fazladan satır veya `3,5` yerine `3.5` yazmak o satırı yanlış yapar; gerçek sınavda da böyledir.
> - **Kod sorularında** çözümdeki program tek doğru cevap değildir. Senin kodun aynı işi yapıyorsa tam puan al. Derleme hatası olan her satır için 1 puan, yanlış sonuç veren her mantık hatası için 2–3 puan kır.
> - Her cevabın sonunda konunun anlatıldığı bölüm yazıyor. Yanlış yaptığın soruların bölümlerindeki **Sınavda Çıkar** ve **Sık Yapılan Hata** kutularını tekrar oku.

Sorular

A) Çoktan Seçmeli

1 Aşağıdaki satırlardan hangisi **derlenmez**?

(A) `double d = 7 / 2;`

(B) `byte b = 200; b += 100;`

(C) `long n = 'x';`

(D) `decimal m = 5;`

(E) `char c = 'a' + 1;`

2 Aşağıdaki program ne yazdırır?

```
int x = 5;
int y = x-- - --x * 2 + x++;
Console.WriteLine($"x = {x}, y = {y}");
```

(A) `x = 3, y = 2` (B) `x = 4, y = 2` (C) `x = 4, y = 7` (D) `x = 5, y = 4` (E) `x = 4, y = 3`

3 Aşağıdaki program Türkçe Windows'ta çalıştırılınca ne yazdırır? (Boşluklar `␣` ile gösterilmiştir.)

```
Console.WriteLine($"[{9.996,7:F2}][{42,-4}][{7 / 2.0}]");
```

(A) `[␣␣9,99][42␣␣][3,5]`

(B) `[␣␣10,00][␣␣42][3,5]`

(C) `[␣␣10,00][42␣␣][3]`

(D) `[␣␣10,00][42␣␣][3,5]`

(E) `[␣␣10.00][42␣␣][3.5]`

4 Aşağıdaki program ne yazdırır?

```
int a = 3, b = 5;
bool r = a > b && ++a > 0 || b++ > 4 && a++ > 2;
Console.WriteLine($"r = {r}, a = {a}, b = {b}");
```

(A) `r = True, a = 4, b = 6`

(B) `r = False, a = 3, b = 5`

(C) `r = True, a = 3, b = 6`

(D) `r = True, a = 4, b = 5`

(E) `r = False, a = 4, b = 6`

5 Aşağıdaki program derlenip çalıştırılırsa ne olur?

```
int k = 1, t = 0;
switch (k)
{
    case 1:
        t += 1;
    case 2:
        t += 2;
        break;
    default:
        t += 10;
        break;
}
Console.WriteLine(t);
```

(A) `1` yazar.
(B) `3` yazar.
(C) Derleme hatası verir.
(D) `2` yazar.
(E) `13` yazar.

6 Aşağıdaki program ne yazdırır?

```
int i = 10, s = 0;
do
{
    s += i;
} while (i-- > 8);
Console.WriteLine($"{s} {i}");
```

(A) `19 8` (B) `27 8` (C) `19 7` (D) `10 9` (E) `27 7`

7 Aşağıdaki program ne yazdırır?

```
int x = 1, y = 2;
int r1 = Islem(x, ref y);
int r2 = Islem(c: 1, a: x, b: ref y);
Console.WriteLine($"{x} {y} {r1} {r2}");

static int Islem(int a, ref int b, int c = 10)
{
    a += 1;
    b += 2;
    c += a + b;
    return c;
}
```

(A) `1 6 16 9`
(B) `2 6 16 9`
(C) `1 4 16 9`
(D) `1 6 15 8`
(E) Derleme hatası: adlandırılmış argümanlar sırayı bozamaz.

8 Aşağıdaki program ne yazdırır?

```
int[] a = { 1, 2, 3 };
int[] b = a;
int[] c = (int[])a.Clone();
b[0] = 10;
c[1] = 20;
Console.WriteLine(a[0] + b[1] + " " + c[0] + c[1]);
```

(A) `3 120` (B) `12 1020` (C) `12 120` (D) `102 120` (E) `12 21`

9 Aşağıdaki program ne yazdırır?

```
string s = "Bilgisayar";
s.ToUpper();
s.Replace('a', 'e');
Console.WriteLine(s.Substring(3, 6) + s.IndexOf('a') + s.LastIndexOf("a"));
```

(A) `GISAYA68` (B) `gis68` (C) `gisaya14` (D) `gisaya68` (E) `giseye-1-1`

10 Aşağıdaki program Türkçe Windows'ta çalıştırılınca ne yazdırır?

```
string a = "ilk";
string b = "IRMAK";
Console.WriteLine(a.ToUpper() + " " + b.ToLower());
```

(A) `ILK irmak` (B) `İLK ırmak` (C) `ILK ırmak` (D) `İLK irmak` (E) `ilk IRMAK`

B) Çıktı Tahmini

11 Aşağıdaki program Türkçe Windows'ta ekrana ne yazdırır?

```
int m = int.MaxValue;
m++;
Console.WriteLine(m);
Console.WriteLine(7 / 2 + 7 / 2.0);

double x = 0.1, y = 0.2;
decimal p = 0.1m, q = 0.2m;
Console.WriteLine(x + y == 0.3);
Console.WriteLine(p + q == 0.3m);

Console.WriteLine(Convert.ToInt32(2.5) + Convert.ToInt32(3.5) + " " + (int)-2.9);
Console.WriteLine(-7 / 2 + " " + -7 % 3);
Console.WriteLine((6 & 3) + " " + (6 | 3) + " " + (6 ^ 3) + " " + (1 << 4));

bool ok1 = int.TryParse("12a", out int n1);
bool ok2 = double.TryParse("2,5", out double d2);
Console.WriteLine($"{ok1} {n1} {ok2} {d2 * 2}");
Console.WriteLine($"{10 / 4 * 4.0} {Math.Round(2.5)} {Math.Round(-1.5)}");
```

12 Aşağıdaki program klasik `Main` biçiminde yazılmıştır. Ekrana ne yazdırır?

```
class Program
{
    static void Main(string[] args)
    {
        int toplam = 0;
        for (int i = 1; i <= 10; i++)
        {
            if (i % 3 == 0)
            {
                continue;
            }
            if (i * i > 50)
            {
                break;
            }
            toplam += i;
            Console.Write(i + " ");
        }
        Console.WriteLine("| toplam=" + toplam);

        int[] puanlar = { 95, 72, 50, 49, 88 };
        foreach (int p in puanlar)
        {
            string harf = p switch
            {
                >= 90 => "A",
                >= 70 => "B",
                >= 50 => "C",
                _ => "F"
            };
            Console.Write(harf);
        }
        Console.WriteLine();

        int sayi = 17;
        int sonuc = Artir(ref sayi);
        Console.WriteLine($"{sayi} {sonuc}");
        Console.WriteLine(Topla(2, 3) + " " + Topla(2, 3.5) + " " + Topla('a', 1));
    }

    static int Artir(ref int x, int adim = 2)
    {
        x += adim;
        return x * 2;
    }

    static int Topla(int a, int b)
    {
        return a + b;
    }

    static double Topla(double a, double b)
    {
        return a + b;
    }
}
```

13 Aşağıdaki program Türkçe Windows'ta ekrana ne yazdırır?

```
int[] a = { 5, 2, 8 };
int[] b = a;
int[] c = (int[])a.Clone();
b[0] = 1;
Array.Sort(c);
a = new int[] { 7, 7 };
string ya = string.Join(" ", a);
string yb = string.Join(" ", b);
string yc = string.Join(" ", c);
Console.WriteLine(ya + " | " + yb + " | " + yc);

int[][] m = { new[] { 1, 2, 3 }, new[] { 4, 5, 6 }, new[] { 7, 8, 9 } };
int[] satir = m[1];
satir[2] = 0;
m[2] = m[0];
m[2][0] = 99;

foreach (int[] s in m)
{
    int t = 0;
    foreach (int v in s)
    {
        t += v;
    }
    Console.Write(t + " ");
}
Console.WriteLine();

int[,] g = new int[3, 4];
for (int i = 0; i < g.GetLength(0); i++)
{
    for (int j = 0; j < g.GetLength(1); j++)
    {
        g[i, j] = i * j;
    }
}
Console.WriteLine($"{g.Length} {g.GetLength(0)} {g.GetLength(1)} {g[2, 3]}");
Console.WriteLine(m[0][0] + m[1][2] + "" + m[2].Length);
```

14 Aşağıdaki program Türkçe Windows'ta ekrana ne yazdırır?

```
using System.Text;

string s = "Programlama";
s.ToUpper();
string t = s.Replace("am", "_");
Console.WriteLine(s.Length + " " + t + " " + t.Length);

string u = s.Substring(3, 4) + s[s.Length - 1];
Console.WriteLine(u + s.IndexOf("ram") + s.LastIndexOf('a'));

StringBuilder sb = new StringBuilder("abc");
sb.Append(12).Insert(0, 'x').Replace("b", "BB");
Console.WriteLine(sb + " " + sb.Length);

char ch = s[0];
Console.WriteLine(ch + 1);
Console.WriteLine((char)(ch + 1));
Console.WriteLine(ch + "" + 1);
Console.WriteLine(s.IndexOf("xyz") + s.Substring(8).Length);
```

C) Kod Yazma

15 Kullanıcıdan satır satır girdi okuyan `SayiAnaliz` programını yaz. Girdi **`bitir`** olunca okuma durur ve `bitir` sayılmaz. Her satır `int.TryParse` ile tamsayıya çevrilebiliyorsa **geçerli sayı** sayılır; çevrilemiyorsa (örneğin `abc` ya da `3,5`) **geçersiz giriş** sayılır ve atlanır. Program sonunda şunları yazdırsın:

- geçerli sayı adedi ve geçersiz giriş adedi,
- **pozitif** sayıların ortalaması (virgülden sonra 2 basamak; pozitif sayı yoksa `Pozitif sayı yok.`),
- en büyük sayı (sayıların hepsi negatif olabilir!),
- çift sayıların adedi.

Hiç geçerli sayı girilmeden `bitir` yazılırsa sadece `Geçerli sayı girilmedi.` yazsın. Örnek çalışma:

```
Sayı (çıkmak için bitir): 12
Sayı (çıkmak için bitir): -5
Sayı (çıkmak için bitir): abc
Sayı (çıkmak için bitir): 7
Sayı (çıkmak için bitir): 30
Sayı (çıkmak için bitir): -8
Sayı (çıkmak için bitir): 3,5
Sayı (çıkmak için bitir): bitir
Geçerli sayı: 5
Geçersiz giriş: 2
Pozitif ortalaması: 16,33
En büyük: 30
Çift sayı adedi: 3
```

16 Aşağıdaki iki metodu yaz ve ana kodda `{7, 3, 9, 1, 9, 4}` ile `{5, 5, 5}` dizileriyle test et:

- `static void TersCevir(int[] d)`: Diziyi **yeni bir dizi oluşturmadan** (yerinde) ters çevirir.
- `static bool IkinciEnBuyuk(int[] d, out int deger)`: Dizide birbirinden farklı en az iki değer varsa `true` döndürür ve ikinci en büyük **farklı** değeri `deger`'e yazar; yoksa `false` döndürür. Diziyi sıralama; tek döngü kullan.

Dizileri yazdırırken `string.Join(", ", dizi)` işine yarar. Beklenen çıktı:

```
Önce : [7, 3, 9, 1, 9, 4]
Sonra: [4, 9, 1, 9, 3, 7]
İkinci en büyük: 7
İkinci en büyük yok.
```

17 Kullanıcıdan bir cümle okuyan `CumleAnaliz` programını yaz. Program cümlenin **kelime sayısını**, **sesli harf sayısını** (Türkçe sesliler: a e ı i o ö u ü, büyük ve küçük) ve boşluklar ile büyük/küçük harf farkı yok sayıldığında cümlenin **palindrom** (tersten okunuşu aynı) olup olmadığını yazdırsın. Kelimeler arasında birden fazla boşluk olabilir. Örnek çalışma:

```
Cümle: Anastas mum satsana
Kelime sayısı: 3
Sesli harf sayısı: 7
Palindrom: evet
```

Çözümler

Cevap anahtarı

1 E	2 B	3 D	4 A	5 C	6 E	7 A	8 C
9 D	10 B						

1 Cevap: E

`'a' + 1` bir `int` sabitidir (98). `int` sabitler, değer sığıyorsa `byte`, `short` gibi türlere kendiliğinden dönüşür ama `char`'a **dönüşmez**: CS0266. Düzeltmek için `(char)('a' + 1)` ya da doğrudan `'b'` yazılır. Diğerleri derlenir:

- A) `7 / 2` tamsayı bölmesidir, sonuç `3` olur ve `double`'a genişler: `3`.
- B) Bileşik atama gizli cast içerir: `b = (byte)(b + 100)`. `100` byte aralığına sığar, bu yüzden derlenir; ama `300` bayta sığmayıp taşar ve `b` 44 olur.
- C) `char` → `long` genişleyen dönüşümdür: `120`.
- D) `int` → `decimal` dönüşümü kendiliğinden yapılır: `5`.

A–D şıklarındaki satırlar bir programda çalıştırılıp değişkenler yazdırıldığında:

Çıktı
```
3 44 120 5
```

E şıkkındaki satırın derleyici çıktısı:

Derleyici çıktısı – program derlenmez
```
test1e.cs(1,10): error CS0266: Cannot implicitly convert type 'int' to 'char'. An explicit
conversion exists (are you missing a cast?)
```

Konu: Bölüm 2.

2 Cevap: B

Değişken tablosuyla soldan sağa git; çarpma toplama ve çıkarmadan önce yapılır ama işlenenler yine soldan sağa hesaplanır:

1. `x--` → değer **5**, sonra `x` = 4.
2. `--x` → önce `x` = 3, değer **3**. Çarpma: `3 * 2 = 6`.
3. `x++` → değer **3**, sonra `x` = 4.
4. `y = 5 - 6 + 3 = 2`, `x` = 4.

A son `x++` etkisini unutuyor (`x` 3 kalır). C işlemleri soldan sağa grupluyor: `(5 - 3) * 2 + 3 = 7`; oysa `*` önce yapılır. D ve E bu iki hatanın karışımıdır.

Çıktı
```
x = 4, y = 2
```

Konu: Bölüm 2.

3 Cevap: D

- `{9.996,7:F2}`: `F2` iki basamağa **yuvarlar**: `10,00` (5 karakter, virgül Türkçe kültürün ondalık ayracı). `,7` alan genişliğidir; sağa yaslanır, başa 2 boşluk gelir. A şıkkı yuvarlamayı kesme sanıyor, E şıkkı noktayı kullanıyor.
- `{42,-4}`: Negatif genişlik **sola** yaslar, sağa 2 boşluk gelir. B şıkkı sağa yaslamış.
- `{7 / 2.0}`: Bölenlerden biri `double` olduğu için sonuç `3,5` olur. C şıkkı tamsayı bölmesi sanıyor.

```
Çıktı
[  10,00][42  ][3,5]
```

Konu: Bölüm 2 ve 3.

4 Cevap: A

`&&`, `||`'dan önceliklidir; ifade `(a > b && ++a > 0) || (b++ > 4 && a++ > 2)` olarak okunur.

1. `a > b` → `3 > 5` → `false`. `&&`'nin sol tarafı `false` olduğu için **kısa devre** olur, `++a` **hiç çalışmaz**: `a` 3 kalır.
2. `||`'nin sol tarafı `false`, sağ tarafa bakılır. `b++ > 4` → `5 > 4` → `true`, sonra `b` = 6.
3. Bu `&&`'nin sağ tarafı da çalışır: `a++ > 2` → `3 > 2` → `true`, sonra `a` = 4.
4. Sonuç `True`.

C şıkkı son `a++`'ı unutuyor, D şıkkı `b++`'ı. B ve E şıkları sonucun `False` olduğunu sanıyor.

```
Çıktı
r = True, a = 4, b = 6
```

Konu: Bölüm 4.

5 Cevap: C

C#'ta bir `case` bloğunun sonu `break` (ya da `return`, `continue`, `goto`) ile bitmek **zorundadır**; bir sonraki `case`'e düşmek (*fall-through*) derleme hatasıdır. Burada `case 1:` bloğu `break`'siz bittiği için CS0163 verilir. Derleyici `k`'nin değerine bakmaz: `k` 1 olsun ya da olmasın hata aynıdır, program hiç çalışmaz.

`3` yazan B şıkkı, düşmenin serbest olduğu dillerdeki davranıştır (`1 + 2`). Diğer sayılar da programın çalıştığını varsayıyor.

```
Derleyici çıktısı – program derlenmez
test5.cs(4,5): error CS0163: Control cannot fall through from one case label ('case 1:') to another
```

Konu: Bölüm 4.

6 Cevap: E

`do-while`'da gövde koşuldan **önce** en az bir kez çalışır. `i-- > 8` karşılaştırmada `i`'nin **eski** değerini kullanır, ama azaltma karşılaştırma yanlış çıksa bile yapılır:

Tur	`s` (eklemeden sonra)	Karşılaştırma	`i` (sonra)
1	10	`10 > 8`, doğru	9
2	19	`9 > 8`, doğru	8
3	27	`8 > 8`, yanlış	**7**

Döngü biter: `s` = 27, `i` = 7. A ve C şıkları gövdeyi iki kez çalıştırıyor, B şıkkı `i`'yi 8 sanıyor, D şıkkı döngüyü bir turda bitiriyor.

```
Çıktı
27 7
```

Konu: Bölüm 5.

7 Cevap: A

`x` değer parametresidir: `a` onun kopyasıdır, `x` hep 1 kalır (B yanlış). `y` ise `ref` ile geçer: `b`, `y`'nin kendisidir.

- Birinci çağrı: `a` = 2, `y` = 2 + 2 = 4, `c` = 10 (varsayılan). `c` = 10 + 2 + 4 = **16**.
- İkinci çağrı: Adlandırılmış argümanlar **istenen sırada** verilebilir (E yanlış). `a` = 2, `y` = 4 + 2 = 6, `c` = 1. `c` = 1 + 2 + 6 = **9**.

C şıkkı `y`'nin ikinci çağrıda değişmediğini, D şıkkı ise `a += 1` satırının etkisini unutuyor.

```
Çıktı
1 6 16 9
```

Konu: Bölüm 6.

8 Cevap: C

`int[] b = a;` dizinin kopyasını değil, **aynı diziyi** gösteren ikinci bir başvuru oluşturur; `b[0] = 10` aynı zamanda `a[0]`'ı değiştirir. `Clone()` ise gerçek bir kopya üretir: `c[1] = 20` sadece `c`'yi etkiler (B yanlış). `a` ve `b` ayrı diziler sanılırsa A şıkkı çıkar.

Yazdırma satırında soldan sağa gidilir:

1. `a[0] + b[1]` iki `int` toplamıdır: `10 + 2 = 12`.
2. `+ " "` ile ifade metne dönüşür: `"12 "`.
3. Bundan sonraki `c[0]` ve `c[1]` metne **eklenir**, toplanmaz: `"12 " + 1 + 20` → `"12 120"`.

D şıkkı ilk toplamayı birleştirme sanıyor (`"102"`), E şıkkı `c[0] + c[1]`'i toplanıyor sanıyor (`21`).

```
Çıktı
12 120
```

Konu: Bölüm 7.

9 Cevap: D

`string` **değiştirilemez** (*immutable*): `ToUpper()` ve `Replace()` yeni metin döndürür, ama sonuç bir değişkene atanmadığı için kaybolur; `s` hâlâ `"Bilgisayar"`'dır. A ve E bu tuzağa düşüyor.

`B0 i1 l2 g3 i4 s5 a6 y7 a8 r9`. `Substring(3, 6)` ikinci parametreyi **uzunluk** olarak alır: 3. indeksten başlayıp 6 karakter, `"gisaya"`. B şıkkı ikinci sayıyı bitiş indeksi sanıyor (`"gis"`). `IndexOf('a')` = 6, `LastIndexOf("a")` = 8. Soldaki ifade metin olduğu için `+` birleştirme yapar: `"gisaya68"`. C şıkkı `6 + 8`'i toplayıp `14` yapıyor.

```
Çıktı
gisaya68
```

Konu: Bölüm 8.

10 Cevap: B

Türkçe kültürde büyük/küçük harf dönüşümü Türkçe kurallara göre yapılır: `i` ↔ `İ` ve `ı` ↔ `I`. Bu yüzden:

- `"ilk".ToUpper()` → `"İLK"` (noktalı büyük İ). A ve C bunu `I` sanıyor.
- `"IRMAK".ToLower()` → `"ırmak"` (noktasız küçük ı). A ve D bunu `i` sanıyor.

E şıkkı metinlerin değişmediğini sanıyor, ama burada dönüşümün sonucu atanmadan doğrudan kullanılıyor; yani değişmezlik tuzağı yok.

```
Çıktı
İLK ırmak
```

Konu: Bölüm 8.

11

Satır satır:

1. `m++` sınırı aşar; `checked` olmadığı için sessizce **taşar**: `-2147483648`.
2. `7 / 2` tamsayı bölmesi → `3`; `7 / 2.0` → `3,5`; toplam `6,5` (virgül).
3. `0.1 + 0.2` ikilik sistemde tam gösterilemez (`0.30000000000000004`): `False`.
4. `decimal` onluk sistemde tutar, tam eşittir: `True`.
5. `Convert.ToInt32` yarım değeri en yakın **çift** sayıya yuvarlar: `2.5` → 2, `3.5` → 4. Bunlar henüz sayıdır, toplanır: `6`. Sonra `" "` ve `(int)-2.9`: cast **keser** (sıfıra doğru): `-2`. Satır: `6 -2`.
6. `-7 / 2` → `-3` (sıfıra doğru kesilir); `-7 % 3` → `-1` (kalanın işareti bölünenin işaretidir). Satır: `-3 -1`.
7. `6` = 110, `3` = 011. `&` → 010 = 2, `|` → 111 = 7, `^` → 101 = 5, `1 << 4` = 16.
8. `"12a"` sayı değil: `False`, `n1` = 0. Türkçe kültürde `"2,5"` ondalık sayıdır: `True`, `d2 * 2` = `5`.
9. `10 / 4` → `2` (tamsayı), `2 * 4.0` → `8`. `Math.Round(2.5)` banker yuvarlamasıyla `2`; `Math.Round(-1.5)` → `-2`.

```
Çıktı
-2147483648
6,5
False
True
6 -2
-3 -1
2 7 5 16
False 0 True 5
8 2 -2
```

12

Üst düzey deyimlerde yazdığın metotlar aslında yerel işlevdir ve **aşırı yüklenemez** (CS0128); bu yüzden aşırı yükleme klasik `Main` biçiminde sorulur.

- **İlk döngü:** 3'ün katları (`3`, `6`, `9`) `continue` ile atlanır. `i = 7`'de `49 > 50` olmadığı için yazılır; `i = 8`'de `64 > 50` olduğu için `break` ile çıkılır. Yazılanlar `1 2 4 5 7`, toplam `19`.
- **switch ifadesi:** Kollar yukarıdan aşağı denenir, ilk uyan alınır. `95` → A; `72` → B; `50` → C (`>= 70` değil ama `>= 50`); `49` → F; `88` → B. Çıktı: `ABCFB`.
- **`ref`:** `Artir(ref sayi)` `sayi`'nın kendisini değiştirir: `17 + 2 = 19`. Dönüş `19 * 2 = 38`.

- **Aşırı yükleme:** `Topla(2, 3)` iki `int`'e tam uyar: `5`. `Topla(2, 3.5)` için ikinci argüman `double` olduğundan `double` sürümü seçilir: `5,5`. `Topla('a', 1)`: `char` → `int` en yakın dönüşümdür; `'a'` 97 sayılır: `98`.

```
Çıktı
1 2 4 5 7 | toplam=19
ABCFB
19 38
5 5,5 98
```

13

Bir boyutlu diziler: `b = a` aynı diziyi gösterir, `Clone()` kopya üretir. `b[0] = 1` ile `a` ve `b` `{1, 2, 8}` olur. `Array.Sort(c)` sadece kopyayı sıralar: `{2, 5, 8}`. Sonra `a` **yeni** bir diziye yönlendirilir: `{7, 7}`; `b` hâlâ eski diziyi gösterir.

Düzensiz dizi:

- `satir` ile `m[1]` aynı diziyi gösterir: `satir[2] = 0` → `m[1] = {4, 5, 0}`.
- `m[2] = m[0]`: Artık `m[2]` ve `m[0]` **aynı** satırdır. Eski `{7, 8, 9}` satırına kimse erişemez.
- `m[2][0] = 99` → `m[0][0]` de 99 olur: `m[0] = m[2] = {99, 2, 3}`.

Satır toplamları: `104`, `9`, `104`.

İki boyutlu dizi: `g.Length` **bütün** elemanların sayısıdır: `3 * 4 = 12`. `GetLength(0)` = 3 satır, `GetLength(1)` = 4 sütun. `g[2, 3] = 2 * 3` = 6.

Son satır: `m[0][0] + m[1][2] = 99 + 0` = 99 (tamsayı toplamı), sonra `""` ile metne dönüşür ve `m[2].Length` = 3 eklenir: `"993"`.

```
Çıktı
7 7 | 1 2 8 | 2 5 8
104 9 104
12 3 4 6
993
```

14

`"Programlama"` → `P0 r1 o2 g3 r4 a5 m6 l7 a8 m9 a10`, uzunluk 11. `s.ToUpper()` sonucu atanmadığı için `s` değişmez.

- `Replace("am", "_")`: İki `am` de değişir: `"Progr_l_a"` (9 karakter).
- `Substring(3, 4)` = `"gram"` (3. indeksten 4 karakter), `s[10]` = `'a'` → `u = "grama"`. Sonra `IndexOf("ram")` = 4 ve `LastIndexOf('a')` = 10 **metin olarak** eklenir: `"grama410"`.
- `StringBuilder` kendisi değişir: `"abc"` → `"abc12"` → `"xabc12"` → `b` yerine `BB`: `"xaBBc12"`, uzunluk 7.
- `ch` = `'P'` (80). `ch + 1` → `int` toplamı: `81`. `(char)(ch + 1)` → `Q`. `ch + "" + 1` metin birleştirmedir: `P1`.
- `IndexOf("xyz")` bulunamayınca **-1** döner; `Substring(8)` = `"ama"`, uzunluk 3. Toplam `-1 + 3` = `2`.

```
Çıktı
11 Progr_l_a 9
grama410
xaBBc12 7
81
Q
P1
2
```

15

Klasik **nöbetçi değerli** (*sentinel*) döngü: Önce bir satır oku, satır `bitir` olmadığı sürece işle ve bir sonrakini oku. Satır sayı olabilir de olmayabilir de; bunu `int.TryParse` söyler. Üç tuzak var:

- En büyüğün başlangıç değeri `0` olursa, sayıların hepsi negatifken yanlış sonuç çıkar. `int.MinValue` ile başla.
- Çiftlik kontrolü `% 2 == 0` ile yapılmalı. `% 2 == 1` ile tek sayı kontrolü negatiflerde çalışmaz (`-5 % 2` = −1).
- Ortalamada `(double)` cast'ini unutursan `49 / 3` tamsayı bölmesi olur ve `16,00` yazar. `3,5` tamsayı olmadığı için `TryParse` onu da geçersiz sayar.

sayiAnaliz.cs

```
int adet = 0, gecersiz = 0, ciftAdet = 0, pozitifAdet = 0, pozitifToplam = 0;
int enBuyuk = int.MinValue;

Console.Write("Sayı (çıkmak için bitir): ");
string giris = Console.ReadLine() ?? "";
while (giris != "bitir")
{
    if (int.TryParse(giris, out int sayi))
    {
        adet++;
        if (sayi > 0)
        {
            pozitifToplam += sayi;
            pozitifAdet++;
        }
        if (sayi % 2 == 0)
        {
            ciftAdet++;
        }
        if (sayi > enBuyuk)
        {
            enBuyuk = sayi;
        }
    }
    else
    {
        gecersiz++;
    }
    Console.Write("Sayı (çıkmak için bitir): ");
    giris = Console.ReadLine() ?? "";
}

if (adet == 0)
{
    Console.WriteLine("Geçerli sayı girilmedi.");
}
else
{
    Console.WriteLine($"Geçerli sayı: {adet}");
    Console.WriteLine($"Geçersiz giriş: {gecersiz}");
    if (pozitifAdet > 0)
    {
        double ort = (double)pozitifToplam / pozitifAdet;
        Console.WriteLine($"Pozitif ortalaması: {ort:F2}");
    }
    else
    {
        Console.WriteLine("Pozitif sayı yok.");
```

```
    }
    Console.WriteLine($"En büyük: {enBuyuk}");
    Console.WriteLine($"Çift sayı adedi: {ciftAdet}");
}
```

Çıktı

```
Sayı (çıkmak için bitir): 12
Sayı (çıkmak için bitir): -5
Sayı (çıkmak için bitir): abc
Sayı (çıkmak için bitir): 7
Sayı (çıkmak için bitir): 30
Sayı (çıkmak için bitir): -8
Sayı (çıkmak için bitir): 3,5
Sayı (çıkmak için bitir): bitir
Geçerli sayı: 5
Geçersiz giriş: 2
Pozitif ortalaması: 16,33
En büyük: 30
Çift sayı adedi: 3
```

Çıktıda mor görünen kısımlar klavyeden girilen satırlardır; kendi çözümünde de her `ReadLine` bir satır tüketir.

16

Ters çevirme: Baştaki ve sondaki elemanı yer değiştir (*swap*), ortaya doğru ilerle. Döngü `d.Length / 2` kere döner; sonuna kadar gidersen dizi iki kere ters çevrilip eski haline döner.

İkinci en büyük: `TryParse` gibi `bool` döndür, değeri `out` ile ver. Tek döngüde en büyüğü (`birinci`) ve ikinciyi (`deger`) tut. Yeni bir en büyük gelirse eski en büyük ikinciye kayar. Eşit değerleri (`9`, `9`) ikinciye almamak için `x < birinci` şartı gerekir. `bulundu` bayrağı, ikinci değerin gerçekten bulunup bulunmadığını söyler; `{5, 5, 5}` dizisinde hiç bulunmaz. Boş dizide `d[0]` olmadığı için önce `d.Length == 0` kontrol edilir; yoksa `IndexOutOfRangeException` alırsın.

dizilslemleri.cs

```
int[] dizi = { 7, 3, 9, 1, 9, 4 };
Console.WriteLine("Önce : [" + string.Join(", ", dizi) + "]");
TersCevir(dizi);
Console.WriteLine("Sonra: [" + string.Join(", ", dizi) + "]");
if (IkinciEnBuyuk(dizi, out int ikinci))
{
    Console.WriteLine($"İkinci en büyük: {ikinci}");
}
else
{
    Console.WriteLine("İkinci en büyük yok.");
}

int[] ayni = { 5, 5, 5 };
if (IkinciEnBuyuk(ayni, out ikinci))
{
    Console.WriteLine($"İkinci en büyük: {ikinci}");
}
else
{
    Console.WriteLine("İkinci en büyük yok.");
}

static void TersCevir(int[] d)
{
    for (int i = 0; i < d.Length / 2; i++)
```

```
    {
        int gecici = d[i];
        d[i] = d[d.Length - 1 - i];
        d[d.Length - 1 - i] = gecici;
    }
}

static bool IkinciEnBuyuk(int[] d, out int deger)
{
    deger = 0;
    if (d.Length == 0)          // boş dizide d[0] yok
    {
        return false;
    }
    int birinci = d[0];
    bool bulundu = false;
    foreach (int x in d)
    {
        if (x > birinci)
        {
            deger = birinci;
            bulundu = true;
            birinci = x;
        }
        else if (x < birinci && (!bulundu || x > deger))
        {
            deger = x;
            bulundu = true;
        }
    }
    return bulundu;
}
```

Çıktı

```
Önce : [7, 3, 9, 1, 9, 4]
Sonra: [4, 9, 1, 9, 3, 7]
İkinci en büyük: 7
İkinci en büyük yok.
```

`TersCevir`'in `void` olduğuna dikkat et: Dizi başvuruyla geldiği için metodun yaptığı değişiklik ana koddaki diziye yansır (Bölüm 7). `out` parametresine metodun her çıkış yolunda değer atanmalıdır; bu yüzden `deger = 0;` satırı en başta duruyor (Bölüm 6).

17

- **Kelime sayısı:** `Trim()` ile baştaki ve sondaki boşlukları at, `Split(' ', StringSplitOptions.RemoveEmptyEntries)` ile böl. `RemoveEmptyEntries` art arda gelen boşlukların ürettiği boş parçaları atar.
- **Sesli harf:** Bütün sesli harfleri (büyük ve küçük) bir metne koy; cümledeki her karakter için `IndexOf` ile bu metinde var mı diye bak. `-1` değilse sesli harftir. Türkçe `ı`, `İ`, `ö`, `ü` metne ayrıca yazılmalıdır.
- **Palindrom:** Boşlukları sil, `ToLower()` ile küçük harfe çevir. Baştaki ve sondaki karakteri karşılaştırarak ortaya doğru ilerle; biri bile farklıysa palindrom değildir. Metni ters çevirip `==` ile karşılaştırmak da doğru olurdu, çünkü `string` için `==` içeriği karşılaştırır.

cumleAnaliz.cs

```
Console.Write("Cümle: ");
string cumle = (Console.ReadLine() ?? "").Trim();

string[] kelimeler = cumle.Split(' ', StringSplitOptions.RemoveEmptyEntries);
```

```
Console.WriteLine($"Kelime sayısı: {kelimeler.Length}");

string sesliler = "aeıioöuüAEIİOÖUÜ";
int sesli = 0;
foreach (char c in cumle)
{
    if (sesliler.IndexOf(c) != -1)
    {
        sesli++;
    }
}
Console.WriteLine($"Sesli harf sayısı: {sesli}");

string sade = cumle.Replace(" ", "").ToLower();
bool palindrom = true;
for (int i = 0, j = sade.Length - 1; i < j; i++, j--)
{
    if (sade[i] != sade[j])
    {
        palindrom = false;
        break;
    }
}
Console.WriteLine("Palindrom: " + (palindrom ? "evet" : "hayır"));
```

Çıktı

```
Cümle:   Anastas  mum satsana
Kelime sayısı: 3
Sesli harf sayısı: 7
Palindrom: evet
```

Girdide baştaki, sondaki ve kelimeler arasındaki fazla boşluklar bilerek bırakıldı; `Trim` ve `RemoveEmptyEntries` olmasaydı kelime sayısı yanlış çıkardı.

BÖLÜM 9

Sınıflar ve Nesneler

Bu Bölümde Öğreneceklerin

- Nesneye yönelik programlamanın (OOP) temel fikrini; sınıf ile nesne farkını, `new` ve nokta operatörünü
- `public` / `private` erişim belirleyicilerini ve kapsüllemeyi
- Özellikleri (property): `get`, `set`, otomatik özellik, `init`, hesaplanan özellik
- Kurucuları (constructor), `this` ve `: this(...)` zincirlemeyi, nesne başlatıcıyı, `ToString` metodunu
- Referans davranışını: `b = a`, `==`, `null` ve `NullReferenceException`
- `static` üyeleri, `const` ile `static readonly` farkını
- Nesne dizilerini, `struct` ile `class` farkını ve `record` fikrini

9.1 Nesneye yönelik programlama fikri

Şimdiye kadar veriler (değişkenler, diziler) ve onlar üzerinde çalışan metotlar birbirinden ayrıydı. Öğrenci Not Sistemi projesinde olduğu gibi öğrencilerin adını, numarasını ve ortalamasını tutmak için üç **paralel dizi** kullanırdın:

```
string[] adlar       = { "Ali", "Ayşe", "Can" };
int[]    numaralar   = { 101,   102,    103 };
double[] ortalamalar = { 72.5,  88.0,   64.0 };
```

Bu yaklaşım kırılgandır: Adları sıralayıp diğer iki diziyi sıralamayı unutursan Ali'nin notu Ayşe'ye geçer; yeni bir bilgi eklemek yeni bir dizi demektir. **Nesneye yönelik programlama** (Object-Oriented Programming, **OOP**), birbirine ait verileri ve işlemleri **tek bir pakette** toplar: Her öğrenci adını, numarasını ve ortalamasını içinde taşır, "bilgilerini yazdır" işini de kendisi yapar. Üç dizi yerine tek bir `Ogrenci[]` olur.

Her nesnenin iki yönü vardır: **durum** (state: ad, numara, ortalama) ve **davranış** (behavior: bilgilerini yazdır, ders seç). C#'ta durum **alanlarla** (field) ve **özelliklerle** (property), davranış **metotlarla** (method) ifade edilir.

Sınavda Çıkar: OOP'nin dört temel ilkesi

"OOP'nin temel ilkelerini yazınız" klasik bir sorudur: **Kapsülleme** (encapsulation, bu bölüm), **Kalıtım** (inheritance, Bölüm 10), **Çok biçimlilik** (polymorphism, Bölüm 11) ve **Soyutlama** (abstraction, Bölüm 11). Cevabında İngilizce karşılıklarını da yaz.

9.2 Sınıf ve nesne

Sınıf (class), nesnelerin nasıl olacağını tarif eden bir **şablondur**; **nesne** (object) ise bu şablondan üretilmiş somut bir **örnektir** (instance). Kurabiye kalıbı tek, kurabiye çok: Her nesnenin **yapısı aynı, değerleri farklıdır**. `Ogrenci` sınıfı şablondur; Ali, Ayşe, Can onun nesneleridir. İlk sınıfımızı yazalım. Sınıf, üst düzey deyimlerin **altında**, aynı dosyada durur:

sinifGiris.cs

```
Ogrenci a = new Ogrenci();      // 1. nesne
a.Ad = "Ali";
a.No = 101;
a.Ortalama = 72.5;

Ogrenci b = new Ogrenci();      // 2. nesne
b.Ad = "Ayşe";
b.No = 102;
b.Ortalama = 88.0;

a.BilgiYazdir();
b.BilgiYazdir();

class Ogrenci
{
    public string Ad = "";      // alanlar: her nesnenin kendi kopyası olur
    public int No;
    public double Ortalama;

    public void BilgiYazdir()   // metot: nesnenin davranışı
    {
        Console.WriteLine($"{No} - {Ad} - {Ortalama}");
    }
}
```

Çıktı

```
101 - Ali - 72,5
102 - Ayşe - 88
```

- `class Ogrenci { ... }` yeni bir **tip** tanımlar; artık `int` gibi `Ogrenci` tipinde de değişken tanımlayabiliriz.
- `new Ogrenci()` bellekte yeni bir nesne **oluşturur** (instantiation). Her `new`, yepyeni ve bağımsız bir nesnedir.
- **Nokta operatörü** (`.`) ile nesnenin alanına (`a.Ad`) ve metoduna (`a.BilgiYazdir()`) ulaşılır. Metot, **hangi nesne üzerinden çağrıldıysa** onun alanlarını kullanır.
- `72.5` çıktıda `72,5` göründü: Bölüm 2'deki Türkçe kültür ayarı. `= ""` ve `public` şimdilik ezber; `""` nullable uyarısını önler, `public` birazdan açıklanıyor.
- Bölüm 6'daki metotların başında `static` vardı, `BilgiYazdir`'da yok: `static` olmayan metot mutlaka bir **nesne üzerinden** çağrılır.

> **Not: Sınıf nerede yazılır?**
>
> Kitaptaki her örnek tek bir `.cs` dosyasıdır: önce üst düzey deyimler, **altında** sınıflar; sınıfı deyimlerin üstüne yazarsan CS8803 hatası alırsın. Gerçek projelerde her sınıf **kendi dosyasındadır**: Visual Studio'da projeye sağ tıkla → **Add** → **Class** → `Ogrenci.cs`. `using` gerekmez; aynı projedeki sınıflar birbirini görür ve `dotnet run` bütün `.cs` dosyalarını birlikte derler.

9.3 Alanlar ve metotlar

Sınıfın içinde ama metotların dışında tanımlanan değişkenlere **alan** denir; nesnenin durumunu oluşturur ve nesne yaşadığı sürece yaşar. Sınıfın bütün metotları alanları okuyabilir ve değiştirebilir. **Yerel değişken** (local variable) ise bir metodun içinde tanımlanır, yalnızca o blokta geçerlidir ve metot bitince yok olur. Sınavlık fark: Alanlar varsayılan değer alır, yerel değişkenler **almaz** (değer verilmeden okunamaz).

`new` ile nesne oluşturulduğunda C# bütün alanlara **otomatik olarak varsayılan değer** verir (Bölüm 7'de dizi elemanlarında gördüğün kuralın aynısı):

Alanın tipi	Varsayılan değer
`int`, `double`, `decimal`	`0`
`bool`	`false`
`char`	Kodu 0 olan görünmez karakter
`string`, diziler, sınıflar (**başvuru tipleri**)	`null`

`null`, "bu değişken henüz hiçbir nesneyi göstermiyor" demektir. Nullable açık projede `string?` yazımındaki `?`, "bu alan bilerek `null` olabilir" demektir; `string` yazarsan alanı kendin doldurmalısın (`= ""` gibi).

9.4 Erişim belirleyiciler ve kapsülleme

Alanlara dışarıdan doğrudan erişirsek biri `a.Ortalama = 250;` veya `hesap.Bakiye = -5000;` yazabilir ve nesne **geçersiz** bir duruma düşer. **Kapsülleme** (encapsulation), alanları dış dünyadan gizleyip onlara yalnızca sınıfın izin verdiği yoldan erişilmesini sağlamaktır. Bankamatik gibi: Kasayı açıp parayı kendin alamazsın, ama düğmeler aracılığıyla kurallara uygun işlem yapabilirsin. Gizlemeyi **erişim belirleyiciler** (access modifiers) sağlar:

Belirleyici	Aynı sınıf	Aynı proje	Başka proje
`public`	Evet	Evet	Evet
`internal`	Evet	Evet	Hayır
`private`	Evet	Hayır	Hayır

- **Üye** (alan, metot, kurucu) için belirleyici yazılmazsa varsayılan **`private`**'tır. Sınıfın kendisi için varsayılan `internal`'dır.
- `protected` ve kardeşleri kalıtımla ilgilidir; Bölüm 10'da.

Belirleyici yazmayı unutmak, ilk sınıf yazan herkesin yaşadığı hatadır:

privateErisim.cs

```
Hesap h = new Hesap();
h.Bakiye = -5000;

class Hesap
{
    double Bakiye = 100;        // belirleyici yok: private sayılır

    public void Yaz() => Console.WriteLine(Bakiye);
}
```

Derleyici çıktısı – program derlenmez

```
privateErisim.cs(2,3): error CS0122: 'Hesap.Bakiye' is inaccessible due to its protection level
```

`Hesap` ve ana kod aynı dosyada olmasına rağmen erişim engellendi: `private`, dosyaya değil **sınıfa** özeldir. Genel kural: **Alanlar `private`**, dışarıya sunulan üyeler `public`, yalnızca sınıfın kendi içinde kullanılan yardımcı metotlar `private`.

> **Sınavda Çıkar: private neye özeldir**
>
> "Belirleyici yazılmazsa ne olur?" → üye için `private`, sınıf için `internal`. Klasik tuzak: `private` **sınıfa** özeldir, **nesneye** değil. `Ogrenci` sınıfının bir metodu, parametre olarak gelen başka bir `Ogrenci` nesnesinin `private` alanını (`diger._ortalama`) okuyabilir.

9.5 Özellikler (property)

Alanı `private` yapınca dışarıdan okuma ve yazma için bir kapı lazım. C#'ta bunun için dile yerleşik bir yapı vardır: **özellik** (property). Dışarıdan alan gibi okunur ve yazılır (`o.Ortalama = 78.5;`) ama içinde kod çalışır. İki parçası vardır: `get` değeri döndürür; `set` atanan değeri özel bir isimle, `value` olarak alır ve doğrulamayı burada yaparız. Değeri tutan gizli alana **destek alanı** (backing field) denir. Bu kitapta `private` alan adları `_` ile başlar (`_ortalama`); böylece özellikten (`Ortalama`) ve parametreden (`ortalama`) ayırt edilir.

kapsulleme.cs

```
Ogrenci o = new Ogrenci();
o.Ortalama = 78.5;
o.Ortalama = 250;
Console.WriteLine($"{o.Ortalama} {o.Gecti}");
o.Ortalama = 30;
Console.WriteLine($"{o.Ortalama} {o.Gecti}");

class Ogrenci
{
    private double _ortalama;       // destek alanı

    public double Ortalama
    {
        get { return _ortalama; }
        set
        {
            if (value < 0 || value > 100)
            {
                Console.WriteLine($"Geçersiz ortalama: {value}");
                return;             // alanı değiştirmeden çık
            }
            _ortalama = value;
        }
    }

    public bool Gecti => _ortalama >= 50;       // hesaplanan özellik
}
```

Çıktı

```
Geçersiz ortalama: 250
78,5 True
30 False
```

`set` içindeki doğrulama, geçersiz `250` değerinin alana hiç yazılmasını engelledi. `Gecti` özelliğinin `set`'i ve destek alanı yoktur: `=>` ile yazılan bu biçime ifade gövdeli üye (expression-bodied member) denir ve değer her istendiğinde **hesaplanır** (**hesaplanan özellik**). Metodun parantezi vardır (`BilgiYazdir()`), özelliğinki yoktur (`o.Gecti`).

Otomatik özellik

Çoğu özellikte `get/set` alanı olduğu gibi okuyup yazar. Bu durumda gövdeyi derleyiciye bırakırsın: `{ get; set; }`. Destek alanını derleyici üretir ve gizler. Erişimi daraltarak dört biçim elde edersin:

Yazım	Dışarıdan okuma	Dışarıdan yazma	Değer nerede atanır?
`{ get; set; }`	Evet	Evet	Her yerde
`{ get; private set; }`	Evet	Hayır	Sınıfın içinde
`{ get; }`	Evet	Hayır	Kurucuda ve başlangıç değerinde
`{ get; init; }`	Evet	Hayır	Kurucuda ve nesne başlatıcıda

`set`'i kapalı olan bir özelliğe sınıfın dışından atama yapmaya kalkarsan program derlenmez:

ozellikTurleri.cs

```
Hesap h = new Hesap("Ali", 1001);
h.Sahip = "Veli";           // { get; set; }         : olur
h.No = 2002;                // { get; }              : set hiç yok
h.Bakiye = 5;               // { get; private set; } : set dışarıya kapalı

class Hesap
{
    public string Sahip { get; set; }
    public int No { get; }
    public decimal Bakiye { get; private set; }

    public Hesap(string sahip, int no)
    {
        Sahip = sahip;
        No = no;            // { get; }: kurucuda ya da başlangıç değerinde atanır
    }
}
```

Derleyici çıktısı – program derlenmez

```
ozellikTurleri.cs(3,1): error CS0200: Property or indexer 'Hesap.No' cannot be assigned to -- it is
read only
ozellikTurleri.cs(4,1): error CS0272: The property or indexer 'Hesap.Bakiye' cannot be used in this
context because the set accessor is inaccessible
```

`set` hiç yoksa **CS0200** ("... read only"), `set` var ama `private` ise **CS0272** ("set accessor is inaccessible") alırsın. `Hesap` sınıfının kendi metotları `Bakiye`'yi (`private set`) serbestçe değiştirebilir.

İpucu: Özelliği elle yazma

Visual Studio'da sınıfın içinde `prop` yazıp **Tab**'a iki kez bas: otomatik özellik iskeleti gelir. `propfull` alan + `get`/`set` iskeletini, `ctor` kurucu iskeletini üretir. Var olan bir alanın üstünde **Ctrl+.** → *Encapsulate field* onu özelliğe çevirir.

Yeni C# Notu: C# 14: `field` anahtar sözcüğü

Doğrulamalı bir özellik için ayrı bir `_ortalama` alanı yazmak şart değil. **`field`**, derleyicinin senin yerine ürettiği gizli destek alanının adıdır: `get` ya da `set` içinde `field` yazarsın, alanı derleyici açar. Gövdesiz bırakılan `get;` otomatik çalışır, `set` ise doğrulamayı yapar:

fieldAnahtar.cs

```
Ogrenci o = new Ogrenci();
o.Ortalama = 78.5;
o.Ortalama = 250;
o.Ad = "  Ece  ";
Console.WriteLine($"{o.Ortalama} [{o.Ad}]");
```

```
class Ogrenci
{
    public double Ortalama
    {
        get;
        set
        {
            if (value < 0 || value > 100)
            {
                Console.WriteLine($"Geçersiz ortalama: {value}");
                return;
            }
            field = value;
        }
    }

    public string Ad
    {
        get => field;
        set => field = value.Trim();
    } = "";
}
```

Çıktı

```
Geçersiz ortalama: 250
78,5 [Ece]
```

`Ad` özelliğinde `get => field;` ile `set => field = value.Trim();` aynı gizli alanı kullanır; sondaki `= ""` başlangıç değeridir. Bu kitapta `_ortalama` gibi açık destek alanı yazımını sınavlarda hâlâ sık görürsün; `field` yalnızca C# 14 ve sonrasında derlenir.

9.6 Kurucular (constructor)

Nesneyi oluşturduktan sonra alanları tek tek doldurmak hem uzun hem tehlikelidir: Birini unutursan nesne yarım kalır. **Kurucu** (constructor), `new` ile nesne oluşturulurken **otomatik çalışan** ve nesneye ilk değerlerini veren özel bir metottur. Kuralları:

1. Adı, sınıfın adıyla **birebir aynıdır**.
2. **Dönüş tipi yoktur**; `void` bile yazılmaz.
3. Normal metot gibi çağrılmaz; `new` ile kendiliğinden çalışır. Sınıfın dışından kullanılacaksa `public` olmalıdır.

Varsayılan kurucu ve kaybolması

İlk örneklerde hiç kurucu yazmadığımız halde `new Ogrenci()` çalışıyordu. Çünkü sınıfta **hiç kurucu yoksa** derleyici kendisi parametresiz, boş bir kurucu ekler: **varsayılan kurucu** (default constructor). Dikkat: Bunu **sadece hiç kurucu yazmadıysan** yapar. Parametreli bir kurucu yazdığın anda varsayılan kurucu ortadan kalkar:

varsayilanKayip.cs

```
Ogrenci x = new Ogrenci("Ali");     // tamam
Ogrenci y = new Ogrenci();          // HATA

class Ogrenci
{
    public string Ad { get; set; }

    public Ogrenci(string ad)
    {
        Ad = ad;
    }
}
```

Derleyici çıktısı – program derlenmez

```
varsayilanKayip.cs(2,17): error CS7036: There is no argument given that corresponds to the required parameter 'ad' of 'Ogrenci.Ogrenci(string)'
```

Hem `new Ogrenci()` hem `new Ogrenci("Ali")` kullanmak istiyorsan parametresiz kurucuyu **kendin yazmalısın**. Hata kodu **CS7036**: derleyici, elindeki tek kurucunun bir argüman beklediğini söylüyor.

> **● Sık Yapılan Hata: Kurucuya void yazmak**
>
> Kurucuya dönüş tipi yazarsan o artık kurucu değil, sıradan bir metot olur. C# bunu zaten kabul etmez: `class Kitap` içine `void Kitap()` yazdığında derleyici "member names cannot be the same as their enclosing type" der (**CS0542**). Kurucuda dönüş tipi **hiç** yazılmaz.

Aşırı yükleme ve : this(...) ile zincirleme

Bölüm 6'daki **aşırı yükleme** (overloading) kuruculara da geçerlidir: aynı ad, farklı parametre listesi. Derleyici, `new` ile verilen argümanların sayısına ve tiplerine bakarak hangi kurucunun çalışacağına karar verir. Birden fazla kurucu genellikle aynı işi tekrar eder; tekrarı önlemek için kurucu, parametre listesinden sonra `: this(...)` yazarak aynı sınıftaki başka bir kurucuyu çağırır:

zincirleme.cs

```
Saat s = new Saat(14);
s.Yaz();

class Saat
{
    private int _saat, _dakika;

    public Saat(int saat, int dakika)
    {
        _saat = saat;
        _dakika = dakika;
        Console.WriteLine("2 parametreli kurucu");
    }

    public Saat(int saat) : this(saat, 0)
    {
        Console.WriteLine("1 parametreli kurucu");
    }

    public void Yaz() => Console.WriteLine($"{_saat:D2}:{_dakika:D2}");
}
```

```
Çıktı
2 parametreli kurucu
1 parametreli kurucu
14:00
```

`new Saat(14)` 1 parametreli kurucuyu başlatır, ama `: this(saat, 0)` yüzünden kontrol hemen 2 parametreliye geçer. **Önce zincirin sonundaki kurucunun gövdesi çalışır**, sonra kontrol geri dönerek kalan gövde çalışır; mesajlar bu yüzden "2 → 1" sırasıyla yazıldı. `: this(...)` gövdeden **önce** yazılır. Bölüm 6'daki isteğe bağlı parametreler (`Saat(int saat, int dakika = 0)`) çoğu zaman iki kurucuyu tek kurucuya indirir; sınavda ise zincirleme sorulur, ikisini de bil.

Nesne başlatıcı

Özellikleri dışarıdan atanabilen bir sınıfta, nesneyi oluştururken değerleri süslü parantez içinde verebilirsin: **nesne başlatıcı** (object initializer).

nesneBaslatici.cs

```
Ogrenci a = new Ogrenci { Ad = "Ali", Yas = 20 };
Console.WriteLine($"{a.Ad} {a.Yas}");

class Ogrenci
{
    public string Ad { get; set; } = "";
    public int Yas { get; set; }
}
```

```
Çıktı
Ali 20
```

- `new Ogrenci { ... }` önce **parametresiz kurucuyu** çalıştırır, sonra özellikleri sırayla atar. Sınıfa parametreli bir kurucu yazarsan parametresiz kurucuyu da elle yazman gerekir; yoksa bu yazım derlenmez. Kurucuyla başlatıcı birlikte de olur: `new Ogrenci("Ayşe") { Yas = 21 }`.
- `set` yerine `init` yazılmış özellik **yalnızca** kurucuda ve nesne başlatıcıda atanabilir. `Ad`'ı `{ get; init; }` yapsaydık, nesne oluştuktan sonra yazılan `a.Ad = "Veli";` **CS8852** verirdi. Başına `required` koyarsan başlatıcıda yazılması zorunlu olur.

9.7 this anahtar sözcüğü

`this`, "**şu an üzerinde çalışılan nesne**" demektir: `a.BilgiYazdir()` çağrısında metodun içindeki `this`, `a`'nın gösterdiği nesnedir. En sık, parametre adının alan adıyla aynı olduğu durumda gerekir. Aynı adlı bir parametre **alanı gizler**; buna **gölgeleme** (shadowing) denir. Aşağıda alan adları bilerek `_` öneksiz yazıldı, çünkü tuzak ancak böyle oluşur:

golgeleme.cs

```
Ogrenci o = new Ogrenci("Ali", 101);
Console.WriteLine(o);

class Ogrenci
{
    private string ad = "";
    private int no;

    public Ogrenci(string ad, int no)
    {
        ad = ad;                // parametre kendine atanıyor!
        this.no = no;           // doğrusu: alan = parametre
    }

    public override string ToString() => $"[{ad}] {no}";
}
```

Derleyici uyarısı (warning)

```
golgeleme.cs(11,9): warning CS1717: Assignment made to same variable; did you mean to assign
something else?
```

Çıktı

```
[] 101
```

Kurucuda `ad` denildiğinde C# en yakın tanıma, yani **parametreye** bakar; `ad = ad;` parametreyi kendisine atar, alan `""` kalır (çıktıda `[]`). Derleyici bu satırı fark edip uyarır (CS1717) ama uyarı programı durdurmaz. `no` için `this.no` yazdığımızdan o alan doğru dolar.

> **Sık Yapılan Hata: Gölgeleme tuzağı**
>
> Uyarıyı kaçırırsan program çalışır ama alan boş kalır. Parametreyle alan aynı adı taşıyorsa her zaman `this.ad = ad;` yaz. Çıktı sorusunda `[]` gibi boş bir sonuç görürsen ilk bakacağın yer burasıdır. `_ad` gibi önekli alan adları tuzağı baştan ortadan kaldırır; sınavda ise iki yazım da çıkar.

`this` ayrıca `: this(...)` ile aynı sınıfın başka bir kurucusunu çağırır ve `return this;` ile nesnenin kendisini döndürür.

9.8 ToString metodu

Bir nesneyi doğrudan `Console.WriteLine` ile yazdırırsan işe yaramaz bir çıktı alırsın. Sınıfa `ToString` yazarsan C# nesneyi metne çevirmesi gerektiğinde **otomatik olarak** onu kullanır:

toStringOrnek.cs

```
NoktaYalin p = new NoktaYalin { X = 3, Y = 4 };
Nokta q = new Nokta { X = 3, Y = 4 };
Console.WriteLine(p);
Console.WriteLine(q);
Console.WriteLine("Konum: " + q);
Console.WriteLine($"Konum: {q}");

class NoktaYalin
{
    public int X { get; set; }
    public int Y { get; set; }
}

class Nokta
{
    public int X { get; set; }
    public int Y { get; set; }

    public override string ToString() => $"({X}, {Y})";
}
```

Çıktı

```
NoktaYalin
(3, 4)
Konum: (3, 4)
Konum: (3, 4)
```

- `ToString` yazılmamışsa çıktı sadece **tipin adıdır** (`NoktaYalin`).
- `Console.WriteLine(q)`, `"Konum: " + q` ve `$"Konum: {q}"` arka planda `q.ToString()`'i çağırır.
- İmza tam olarak `public override string ToString()` olmalıdır. `override` ("bilerek yeniden tanımlıyorum") yazılmazsa derleyici CS0114 uyarısı verir ve metodun `object`'teki asıl `ToString`'in yerine geçmez: `q.ToString()` diye doğrudan çağırırsan senin metodun çalışır, ama `Console.WriteLine(q)` gibi nesneyi `object` olarak alan metotlar eski `ToString`'i çağırır. Nedenini Bölüm 10'da göreceksin.

9.9 Referans değişkenler

`int x = 5;` yazdığında `x`'in içinde doğrudan **5 değeri** durur. `Ogrenci a = new Ogrenci();` yazdığında ise `a`'nın içinde nesnenin kendisi değil, nesnenin bellekte **nerede olduğu bilgisi** (adresi) durur; bu yüzden sınıf tipindeki değişkenlere **referans değişken** denir. Bellek iki bölge olarak kullanılır: **yığında** (stack) metotların yerel değişkenleri yaşar, `new` ile oluşturulan **sınıf nesneleri** ise **öbekte** (heap) durur. Bir referansı diğerine atadığında **nesne kopyalanmaz**, sadece adres kopyalanır; iki değişken **aynı nesneyi** gösterir:

referans.cs

```
int x = 5;
int y = x;          // değerin kopyası
y = 10;
Console.WriteLine($"x = {x}, y = {y}");

Kutu k1 = new Kutu();
k1.Deger = 5;
Kutu k2 = k1;       // adresin kopyası: AYNI nesne!
k2.Deger = 10;
Console.WriteLine($"k1.Deger = {k1.Deger}, k2.Deger = {k2.Deger}");

k2 = new Kutu();    // k2 artık başka bir nesneyi gösteriyor
k2.Deger = 99;
```

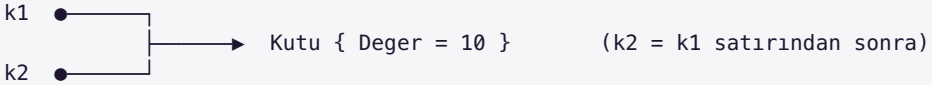
```
Console.WriteLine($"k1.Deger = {k1.Deger}, k2.Deger = {k2.Deger}");

Kutu k3 = new Kutu();
k3.Deger = 10;
Console.WriteLine(k1 == k3);        // içerik aynı (10) ama ayrı nesne
Console.WriteLine(k1.Equals(k3));

class Kutu
{
    public int Deger;
}
```

Çıktı

```
x = 5, y = 10
k1.Deger = 10, k2.Deger = 10
k1.Deger = 10, k2.Deger = 99
False
False
```



`k2.Deger = 10;` nesnenin içini değiştirdiği için `k1.Deger` da 10 görünür. Ama `k2 = new Kutu();` satırı nesneyi değil, **`k2`'nin gösterdiği yeri** değiştirir; `k1` eski nesneyi göstermeye devam eder. Hiçbir referansın göstermediği nesneyi **çöp toplayıcı** (garbage collector) kendiliğinden siler; nesneyi elle silen komut yoktur.

Nesneyi metoda göndermek

Parametreler varsayılan olarak **değerle** aktarılır: Metoda değişkenin bir **kopyası** gider. Nesnelerde kopyalanan şey **referanstır**; yani metot, çağıranla **aynı nesneyi** görür. Bölüm 6'daki `ref` ise referansın kendisini paylaşır:

metodaNesne.cs

```
Kutu kutu = new Kutu();
kutu.Deger = 1;

KutuArtir(kutu);
Console.WriteLine(kutu.Deger);
Yenile(kutu);
Console.WriteLine(kutu.Deger);
YenileRef(ref kutu);
Console.WriteLine(kutu.Deger);

static void KutuArtir(Kutu k) => k.Deger++;             // ortak nesnenin İÇİ
static void Yenile(Kutu k) => k = new Kutu { Deger = 100 };     // yerel k döner
static void YenileRef(ref Kutu k) => k = new Kutu { Deger = 100 };

class Kutu
{
    public int Deger;
}
```

Çıktı

```
2
2
100
```

> **Sınavda Çıkar: Nesne parametresi**
>
> Nesneler metoda **referansın kopyası** olarak gider. Metot nesnenin **içini değiştirebilir** (`k.Deger++` kalıcıdır), ama parametreye yeni bir nesne atayarak çağıranın değişkenini **başka nesneye yönlendiremez** (`Yenile` hiçbir şey değiştirmedi). Yönlendirmek için `ref` gerekir (`YenileRef`). Bölüm 7'de dizi parametrelerinde gördüğün davranışın aynısıdır.

Nesneleri == ile karşılaştırmak

Kendi yazdığın sınıflarda `==` nesnelerin **içeriğini değil, adreslerini** karşılaştırır: "İkisi **aynı nesneyi** mi gösteriyor?" Yukarıdaki `k1 == k3` bu yüzden `False`: İçerikleri aynı (10) ama iki **ayrı** nesne. `k1.Equals(k3)` de `False`: Kendi sınıfımızda `Equals` yazmadığımız için hazır `Equals` kullanıldı ve o da adres karşılaştırır. İçerik karşılaştıran `Equals`'ı Bölüm 10'da yazacaksın. Bölüm 8'deki `string` istisnadır: `string` içerik karşılaştırmasını kendisi tanımlar.

9.10 null ve NullReferenceException

`null`, bir referans değişkenin **hiçbir nesneyi göstermediği** anlamına gelir. `null` bir referans üzerinden alan okumaya veya metot çağırmaya çalışırsan program `NullReferenceException` ile çöker.

Nullable açık projelerde (bu kitaptaki gibi) derleyici de yardım eder: `Ogrenci` yazarsan "bu değişken `null` olmaz", `Ogrenci?` yazarsan "bu değişken **bilerek** `null` olabilir" demiş olursun. Güvenli kalıplar `is null`, `?.` (null ise zinciri keser) ve `??` (null ise yedek değer verir):

nullKontrol.cs

```
Ogrenci? bulunan = Bul("Ali");
Ogrenci? yok = Bul("Zeynep");

Console.WriteLine(bulunan?.Ad);
Console.WriteLine(yok?.Ad ?? "bulunamadı");
Console.WriteLine(yok is null);

static Ogrenci? Bul(string ad) => ad == "Ali" ? new Ogrenci { Ad = ad } : null;

class Ogrenci
{
    public string Ad = "";
}
```

Çıktı

```
Ali
bulunamadı
True
```

Kontrolü unutursan derleyici uyarır: `string? ad` parametresinde `ad.Length` yazarsan "Dereference of a possibly null reference" (**CS8602**) uyarısı çıkar; bu kitapta uyarı çıkan kod teslim edilmez. Derleyiciyi `!` ile susturabilirsin (`null!` = "bundan eminim, `null` değil"); ama yanılırsan program çalışırken çöker:

nullHata.cs

```
Ogrenci o = null!;          // derleyiciyi susturduk
Console.WriteLine("Bu satır çalışır");
Console.WriteLine(o.Ad);

class Ogrenci
{
    public string Ad = "";
}
```

Çıktı – program yakalanmamış istisna ile sonlanır

```
Bu satır çalışır
Unhandled exception. System.NullReferenceException: Object reference not set to an instance of an
object.
   at Program.<Main>$(String[] args) in nullHata.cs:line 3
```

Mesajdaki `Object reference not set to an instance of an object` cümlesi, "referans bir nesneye bağlanmadı" demektir; hata, `null` referansın üzerinden `.` kullandığın satırdadır. `o == null` ve `o is null` karşılaştırmaları her zaman güvenlidir; tehlikeli olan `null` üzerinden **nokta** kullanmaktır.

● **Yeni C# Notu: C# 14: null koşullu atama `?.=`**

`?.` şimdiye kadar yalnız **okumada** kullanılırdı. C# 14 ile **atamanın** solunda da yazılabilir: `m?.Ad = "Ali";` yalnız `m` `null` değilse atama yapar. `m` `null` ise atama atlanır, hata da çıkmaz; **sağ taraf bile hesaplanmaz**:

nullAtama.cs

```
Musteri? m1 = new Musteri();
Musteri? m2 = null;

m1?.Ad = Uret("m1");        // m1 dolu: atama yapılır
m2?.Ad = Uret("m2");        // m2 null: Uret çağrılmaz bile
Console.WriteLine(m1?.Ad);
Console.WriteLine(m2?.Ad ?? "m2 boş");

static string Uret(string ad)
{
    Console.WriteLine($"Uret çağrıldı: {ad}");
    return ad.ToUpper();
}

class Musteri
{
    public string Ad { get; set; } = "";
}
```

Çıktı

```
Uret çağrıldı: m1
M1
m2 boş
```

`Uret` yalnızca bir kez çağrıldı: `m2` `null` olduğu için ikinci satırın sağ tarafı hiç çalışmadı.

Nullable değer tipleri: int?, double?, bool?

`string` ve sınıf tipleri `null` olabilir, ama `int`, `double`, `bool` gibi **değer tipleri asla `null` olamaz**: `int yas = null;` derlenmez (CS0037). Oysa bazen "değer yok" demek gerekir: kullanıcı yaşını yazmadı, ölçüm yapılmadı, çevirme başarısız oldu... Tipin sonuna `?` koyarsan **nullable değer tipi** (nullable value type) elde edersin: `int?`, `double?`, `bool?`. Bu tipler normal değerin yanında `null` da tutar. (`string?` yalnız derleyiciye verilen bir ipucudur; `int?` ise gerçekten ayrı bir tiptir ve aşağıdaki üyeleri taşır.)

Yazım	Anlamı
`x.HasValue`	Değer var mı? (`bool`)
`x.Value`	Değeri verir; `x` `null` ise `InvalidOperationException`
`x ?? yedek`	Doluysa `x`'in değeri, `null` ise `yedek` (Bölüm 3'te gördüğün `??`)
`x == null`	`null` mı?

nullableDeger.cs

```
int? yas = null;
Console.WriteLine(yas.HasValue);
Console.WriteLine(yas ?? -1);
yas = 19;
Console.WriteLine(yas.HasValue + " " + yas.Value);

int? a = SayiyaCevir("42");
int? b = SayiyaCevir("kırk");
Console.WriteLine($"{a} / [{b}]");
Console.WriteLine((b ?? 0) + 1);

bool? cevap = null;                  // üç durum: true, false, null
Console.WriteLine($"{cevap == true} {cevap == false} {cevap == null}");

int? x = 5;
int? y = null;
Console.WriteLine("[" + (x + y) + "]");     // null ile işlem: null
Console.WriteLine(new Ogrenci().Numara.HasValue);

static int? SayiyaCevir(string metin)
{
    if (int.TryParse(metin, out int sayi))
    {
        return sayi;
    }
    return null;
}

class Ogrenci
{
    public int? Numara { get; set; }     // varsayılan değeri: null
}
```

Çıktı

```
False
-1
True 19
42 / []
1
False False True
[]
False
```

`SayiyaCevir` çevirme başarısızsa `null` döndürüyor; çağıran taraf `HasValue` ya da `??` ile durumu ele alır. `null` ekrana boş metin olarak yazılır. `cevap` örneği `bool?`'nin üç durumu olduğunu gösterir: `null`, `true` da değildir `false` da. İki nullable sayıdan biri `null` ise toplam da `null` olur.

Sık Yapılan Hata: `.Value` ile `null` değeri okumak

`HasValue` kontrolü yapmadan `.Value` yazarsan ve değişken `null` ise program çalışırken `InvalidOperationException` ile çöker:

```
int? yas = null;
Console.WriteLine(yas.HasValue);
Console.WriteLine(yas.Value);
```

Derleyici uyarısı (warning)

```
nullableHata.cs(3,19): warning CS8629: Nullable value type may be null.
```

Çıktı – program yakalanmamış istisna ile sonlanır

```
False
Unhandled exception. System.InvalidOperationException: Nullable object must have a value.
   at Program.<Main>$(String[] args) in nullableHata.cs:line 3
```

Derleyici `null` olduğunu sezip uyarır (CS8629), ama program yine de derlenir ve çalışırken çöker.

Güvenli yol: `yas ?? 0` ya da `if (yas.HasValue) { ... }`.

9.11 static üyeler

Normal bir alanın **her nesnede ayrı bir kopyası** vardır (**örnek alanı**, instance field). `static` alan ise **sınıfa aittir**: Kaç nesne oluşturulursa oluşturulsun tek kopyası vardır ve bütün nesneler onu **paylaşır**. Klasik örnek, nesneleri sayan sayaçtır:

staticSayac.cs

```
Console.WriteLine($"Başlangıç: {Ogrenci.Sayac}");
Ogrenci a = new Ogrenci();
Ogrenci b = new Ogrenci();
Ogrenci c = new Ogrenci();
Console.WriteLine($"{a.No} {b.No} {c.No}");
Console.WriteLine($"Toplam öğrenci: {Ogrenci.Sayac}");

class Ogrenci
{
    private static int _sayac;          // sınıfa ait: tek kopya
    private readonly int _no;           // nesneye ait: her nesnede ayrı

    public Ogrenci()
    {
```

```
        _sayac++;
        _no = 1000 + _sayac;            // her öğrenciye sıradaki numara
    }

    public int No => _no;
    public static int Sayac => _sayac;
}
```

Çıktı

```
Başlangıç: 0
1001 1002 1003
Toplam öğrenci: 3
```

`static` üyelere **sınıf adıyla** erişilir (`Ogrenci.Sayac`); hiç nesne yokken bile kullanılabilirler. Tanıdık örnekler: `Math.Sqrt(2)`, `Math.PI`, `Convert.ToInt32("5")`. Alanın önündeki `readonly`, değerin yalnızca kurucuda (ya da tanımlandığı yerde) atanabileceğini söyler.

`static` bir üye belirli bir nesne üzerinden çağrılmaz; bu yüzden içinde `this` yoktur ve **hangi nesnenin** alanını kullanacağını bilemez. Ters yön de yasaktır: C#'ta `static` üyeye **nesne üzerinden** erişilemez, sınıf adı gerekir:

staticHata.cs

```
Sayac s = new Sayac();
Console.WriteLine(s.Toplam);        // nesne üzerinden static

class Sayac
{
    public static int Toplam = 0;
    public int Adet = 1;

    public static void Yaz() => Console.WriteLine(Adet);    // static içinden örnek
}
```

Derleyici çıktısı – program derlenmez

```
staticHata.cs(2,19): error CS0176: Member 'Sayac.Toplam' cannot be accessed with an instance
reference; qualify it with a type name instead
staticHata.cs(9,51): error CS0120: An object reference is required for the non-static field, method,
or property 'Sayac.Adet'
```

CS0176: `static` üyeye nesne üzerinden erişim. **CS0120** ("bir nesne başvurusu gerekli"): `static` içinden örnek üyeye erişim. Bölüm 6'da `Main`'den çağırdığımız bütün metotlara `static` yazmamızın sebebi de buydu.

Özetle: örnek üye **nesneye** ait olduğundan `nesne.Uye` ile (`o.Ad`, `metin.Length`), `static` üye **sınıfa** ait olduğundan `SinifAdi.Uye` ile (`Math.Sqrt(2)`, `Ogrenci.Sayac`) kullanılır. `static` metot yalnızca `static` üyelere doğrudan erişebilir; örnek metot ikisine de erişebilir. Değişken tablosunda `static` alanı nesnelerin dışında, tek bir sütunda takip et; `s3 = s2;` gibi atamalar yeni nesne **oluşturmaz**, sayaç artmaz.

static sınıf, const ve static readonly

Yalnızca `static` üyelerden oluşan sınıf `static class` olarak işaretlenir ve ondan nesne üretilemez; `Math` tam olarak budur. Bütün nesneler için aynı olan değerler **sabit** olarak tanımlanır; C#'ta iki yolu vardır:

staticSinif.cs

```
Console.WriteLine($"{Ayarlar.GecmeNotu} {Ayarlar.Okul} {Ayarlar.Misafir.Ad}");

static class Ayarlar
{
    public const double GecmeNotu = 50;            // derleme zamanı sabiti
    public static readonly string Okul = "Teknik Üniversite";
    public static readonly Ogrenci Misafir = new Ogrenci { Ad = "Misafir" };
}

class Ogrenci
{
    public string Ad { get; set; } = "";
}
```

Çıktı

```
50 Teknik Üniversite Misafir
```

	`const`	`static readonly`
Değer ne zaman belirlenir?	Derleme sırasında; yazdığın yere gömülür	Çalışma sırasında, bir kez
Hangi tipler olabilir?	Sayılar, `bool`, `char`, `string`	Her tip (`Ogrenci` gibi nesneler dahil)
Örnek	`Math.PI`	`string.Empty`

Kural: Değer bir sayı ya da metin sabitiyse `const`; `new` ile üretilen ya da çalışma anında belli olan bir değerse `static readonly`. İkisine de sonradan atama yapılamaz.

9.12 Nesne dizileri

`new Ogrenci[4]` **dört öğrenci oluşturmaz**; sadece dört tane **boş referans** (`null`) içeren bir dizi oluşturur. Her elemana nesneyi ayrıca `new` ile koyman gerekir:

nesneDizisi.cs

```
Ogrenci[] sinif = new Ogrenci[4];       // 4 tane null
Console.WriteLine(sinif[0] is null);
sinif[0] = new Ogrenci { Ad = "Ali", Ortalama = 72.5 };
sinif[1] = new Ogrenci { Ad = "Ayşe", Ortalama = 88.0 };
sinif[2] = new Ogrenci { Ad = "Can", Ortalama = 64.0 };
sinif[3] = new Ogrenci { Ad = "Deniz", Ortalama = 91.5 };

double toplam = 0;
foreach (Ogrenci o in sinif)
{
    toplam += o.Ortalama;
}
Console.WriteLine($"Sınıf ortalaması: {toplam / sinif.Length:F2}");

class Ogrenci
{
    public string Ad { get; set; } = "";
    public double Ortalama { get; set; }
}
```

```
Çıktı
True
Sınıf ortalaması: 79,00
```

Kısa yoldan dizi oluşturulurken de doldurabilirsin: `Ogrenci[] sinif = [new Ogrenci { Ad = "Ali" }, new Ogrenci { Ad = "Can" }];`

> **Sık Yapılan Hata: Dizinin elemanlarını oluşturmayı unutmak**
>
> Diziyi oluşturup nesneleri oluşturmayı unutursan elemanlar `null` kalır ve `sinif[0].Ad = "Ali";` gibi ilk erişimde `NullReferenceException` alırsın. Derleyici bunu yakalamaz; `Ogrenci[]` tanımı elemanları "`null` olmayan" sayar. Bilerek boş eleman istiyorsan `Ogrenci?[]` yaz ve kullanmadan önce kontrol et.

9.13 struct ile class farkı

`struct` (yapı), sözdizimi sınıfa benzeyen ama bir **değer tipi** olan yapıdır: Atamada nesnenin kendisi kopyalanır. `int`, `double`, `bool` ve `DateTime` birer `struct`'tır. Farkı yan yana görelim:

structSinif.cs

```
NoktaS s1 = new NoktaS { X = 1 };
NoktaS s2 = s1;             // değer kopyalanır
s2.X = 99;

NoktaC c1 = new NoktaC { X = 1 };
NoktaC c2 = c1;             // adres kopyalanır
c2.X = 99;

Console.WriteLine($"struct: {s1.X} {s2.X}");
Console.WriteLine($"class : {c1.X} {c2.X}");

struct NoktaS
{
    public int X;
}

class NoktaC
{
    public int X;
}
```

```
Çıktı
struct: 1 99
class : 99 99
```

	`class`	`struct`
Tip	Başvuru tipi (nesne öbekte)	Değer tipi (yerel değişkense yığında; bir sınıfın alanı ya da dizi elemanıysa onunla birlikte öbekte)
`b = a`	Aynı nesne, iki referans	**Bağımsız kopya**
Metoda parametre	Referansın kopyası; içi değişir	Nesnenin kopyası; çağıran etkilenmez
Dizi elemanları / `null`	`null`; `null` olabilir	Varsayılan değerli nesne; `null` olamaz
Ne için?	Çoğu nesne (öğrenci, hesap…)	Küçük, basit değerler (nokta, renk, tarih)

Şüphedeyken `class` yaz. `struct`'ın tuzağı, kopya üzerindeki değişikliğin asıl nesneyi **değiştirmemesidir** (`s2.X = 99` → `s1.X` hâlâ 1).

Yeni C# Notu: C# 12: birincil kurucu ve record

Sınıf adının yanındaki parantezle **birincil kurucu** (primary constructor) tanımlanır; parametreler sınıfın içinde kullanılır. **`record`** ise veri taşımak için yazılmış bir sınıftır: özellikleri, `ToString`'i ve **içerik karşılaştıran** `==`'i derleyici üretir; `with` değiştirilmiş kopya verir:

recordOrnek.cs

```
Nokta p = new Nokta(3, 4);
Nokta q = new Nokta(3, 4);
Console.WriteLine(p);
Console.WriteLine(p == q);              // içeriği karşılaştırır
Console.WriteLine(p with { X = 10 });   // kopya + değişiklik
Console.WriteLine(new Kitap("Sefiller", 1500));

record Nokta(int X, int Y);

class Kitap(string ad, int sayfa)
{
    public string Ad { get; } = ad;
    public int Sayfa { get; } = sayfa;

    public override string ToString() => $"{Ad} ({Sayfa} sayfa)";
}
```

Çıktı

```
Nokta { X = 3, Y = 4 }
True
Nokta { X = 10, Y = 4 }
Sefiller (1500 sayfa)
```

Sınavda önce klasik sınıf yazımı beklenir; `record` ve birincil kurucuyu tanı, gördüğünde şaşırma.

Bölüm Özeti – Bilmen Gerekenler

- **Sınıf** şablondur, **nesne** o şablondan `new` ile üretilen örnektir. Durum **alan ve özelliklerde**, davranış **metotlarda** tutulur; üyelere **nokta** ile erişilir. Sınıf, üst düzey deyimlerin **altına** yazılır.
- Alanlar otomatik varsayılan değer alır (`0`, `false`, `null`); yerel değişkenler almaz. Üyede belirleyici yazılmazsa **`private`**'tır (CS0122); `private` sınıfa özeldir.

- **Kapsülleme**: alan `private` (`_ad`), erişim `public` özellikle; `set` içinde `value` doğrulanır. Biçimler: `{ get; set; }`, `{ get; private set; }`, `{ get; }`, `{ get; init; }`, hesaplanan `=> ...`. `set`'siz özelliğe atama: CS0200.
- **Kurucu**: sınıfla aynı ad, dönüş tipi yok. Kendi kurucunu yazınca **varsayılan kurucu** kaybolur (CS7036). `: this(...)` zincirinde önce en içteki gövde çalışır. Nesne başlatıcı kurucudan **sonra** atar.
- **Gölgeleme**: `ad = ad;` alanı değiştirmez; doğrusu `this.ad = ad;`. `public override string ToString()` yazılırsa `WriteLine(nesne)` ve `$"{nesne}"` onu kullanır.
- Sınıf tipli değişken **referans** tutar; `b = a;` nesneyi kopyalamaz. Metot nesnenin içini değiştirebilir, değişkeni yönlendiremez (`ref` hariç). `==` kendi sınıflarında adres karşılaştırır; `string` ve `record` içerik karşılaştırır.
- `null` referanstan erişim → `NullReferenceException`; `Ogrenci?`, `?.`, `??`, `is null`. `new Ogrenci[3]` **sıfır** nesne oluşturur.
- `int?`, `double?`, `bool?` **nullable değer tipidir**: `HasValue`, `Value` (`null` iken `InvalidOperationException`), `??`. `null` ile aritmetik `null` verir; `int?` alan ya da özelliğin varsayılanı `null`'dır.
- C# 14: `field` anahtar sözcüğü gizli destek alanıdır (`set { field = value; }`); `m?.Ad = "Ali";` yalnız `m` `null` değilse atar (sağ taraf da hesaplanmaz).
- `static` üye sınıfa aittir, tek kopyadır (nesne üzerinden: CS0176; `static` içinden örnek üyeye: CS0120). `const` derleme zamanı sabiti, `static readonly` çalışma zamanında bir kez atanır.
- `struct` değer tipidir: `b = a` bağımsız kopya üretir. `class` başvuru tipidir.

Sorular

A) Çoktan Seçmeli

1 Kurucular (constructor) ile ilgili aşağıdakilerden hangisi **yanlıştır**?

(A) Kurucunun adı sınıfın adıyla aynıdır.

(B) Bir sınıfta birden fazla kurucu bulunabilir.

(C) Sınıfta hiç kurucu yazılmazsa derleyici parametresiz bir kurucu ekler.

(D) Kurucu, `new` ile nesne oluşturulurken otomatik olarak çalışır.

(E) Kurucunun dönüş tipi olarak `void` yazılmalıdır.

2 `Hesap h = new Hesap();` ile bir nesne oluşturuldu. Aşağıdaki özellik tanımlarından hangisi için, sınıfın **dışından** yazılan atama derlenir?

(A) `public int No { get; }` için `h.No = 5;`

(B) `public decimal Bakiye { get; private set; }` için `h.Bakiye = 5;`

(C) `public string Sahip { get; set; }` için `h.Sahip = "Ali";`

(D) `public string Unvan { get; init; }` için `h.Unvan = "Dr.";`

(E) `public bool Gecti => true;` için `h.Gecti = false;`

3 `Kutu` sınıfında sadece `public int Deger;` alanı vardır. Aşağıdaki kod parçası ne yazdırır?

```
Kutu[] dizi = new Kutu[2];
Kutu a = new Kutu();
a.Deger = 4;
dizi[0] = a;
dizi[1] = a;
dizi[0].Deger += 2;
a = new Kutu();
a.Deger = 10;
Console.WriteLine($"{a.Deger} {dizi[0].Deger} {dizi[1].Deger}");
```

A) `10 6 6` B) `10 4 4` C) `10 6 4` D) `6 6 6` E) `10 10 10`

4 Aşağıdaki `Kitap` sınıfı için `new` ifadelerinden hangisi **derlenir**?

```
class Kitap
{
    public string Ad;
    public int Sayfa;

    public Kitap(string ad)
    {
        Ad = ad;
    }

    public Kitap(string ad, int sayfa) : this(ad)
    {
        Sayfa = sayfa;
    }
}
```

A) `new Kitap()`
B) `new Kitap(250)`
C) `new Kitap(250, "Sefiller")`
D) `new Kitap("Sefiller", 250)`
E) `new Kitap { Ad = "Sefiller" }`

5 `NoktaS` bir `struct`, `NoktaC` bir `class`'tır; ikisinde de `public int X;` alanı vardır. `SifirlaS` ve `SifirlaC` metotları aldıkları nesnenin `X`'ini 0 yapar. Kod ne yazdırır?

```
NoktaS p = new NoktaS { X = 1 };
NoktaS q = p;
q.X = 5;
SifirlaS(p);

NoktaC c = new NoktaC { X = 1 };
NoktaC d = c;
d.X = 5;
SifirlaC(c);

Console.WriteLine($"{p.X} {q.X} {c.X} {d.X}");
```

A) `0 5 0 0` B) `1 5 0 0` C) `1 5 5 5` D) `1 1 0 0` E) `0 0 0 0`

B) Çıktı Tahmini

6 Aşağıdaki program ekrana ne yazdırır?

```
Sayac s1 = new Sayac();
Sayac s2 = new Sayac();
s1.Artir();
s1.Artir();
s2.Artir();
Sayac s3 = s2;
s3.Artir();
Console.WriteLine($"{s1.Adet} {s2.Adet} {s3.Adet}");
Console.WriteLine(Sayac.Toplam);
s1 = new Sayac();
s1.Artir();
Console.WriteLine($"{s1.Adet} {Sayac.Toplam} {Sayac.Olusan}");

class Sayac
{
    public static int Toplam = 0;
    public static int Olusan = 0;
    public int Adet = 0;

    public Sayac() => Olusan++;

    public void Artir()
    {
        Adet++;
        Toplam++;
    }
}
```

7 Aşağıdaki program ekrana ne yazdırır?

```
Urun u = new Urun();
Console.WriteLine($"{u.Ad} {u.Fiyat}");
Urun v = new Urun { Ad = "Silgi", Fiyat = 7 };
Console.WriteLine($"{v.Ad} {v.Fiyat}");

class Urun
{
    public string Ad { get; set; } = "?";
    public int Fiyat { get; set; }

    public Urun() : this("Kalem")
    {
        Console.WriteLine("A");
    }

    public Urun(string ad) : this(ad, 10)
    {
        Console.WriteLine($"B {ad}");
    }

    public Urun(string ad, int fiyat)
    {
        Ad = ad;
        Fiyat = fiyat * 2;
        Console.WriteLine($"C {Ad}");
    }
}
```

8 Aşağıdaki program ekrana ne yazdırır?

```
Nokta a = new Nokta(1);
Nokta b = new Nokta(1);
int s = 1;
Degistir(a, s);
Console.WriteLine($"{a} {s}");
Yenile(ref a);
Console.WriteLine($"{a} {b}");
Console.WriteLine(a == b);
b = a;
b.X = 7;
Console.WriteLine($"{a} {a == b}");

static void Degistir(Nokta p, int sayi)
{
    p.X = p.X + 10;
    sayi = sayi + 10;
    p = new Nokta(99);
    p.X++;
}

static void Yenile(ref Nokta p)
{
    p = new Nokta(50);
    p.X++;
}

class Nokta
{
    public int X;

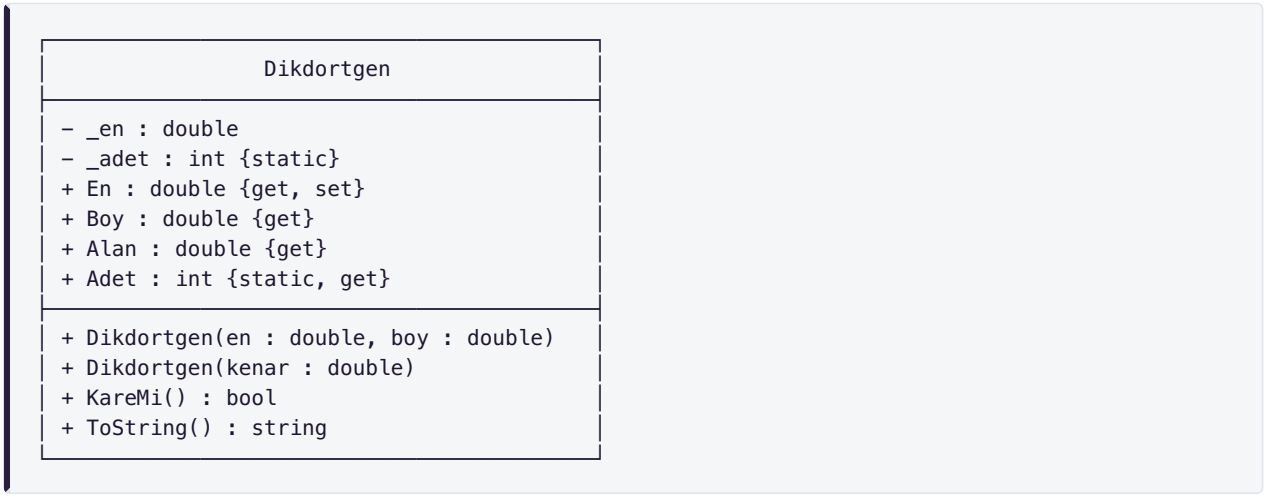
    public Nokta(int x)
    {
        X = x;
    }

    public override string ToString() => $"N{X}";
}
```

C) Kod Yazma

9 Aşağıdaki UML diyagramına göre `Dikdortgen` sınıfını yaz. Kurallar:

- Tek parametreli kurucu bir **kare** oluşturur ve işini `: this(...)` ile iki parametreli kurucuya yaptırır.
- `Adet`, oluşturulan dikdörtgen sayısını verir; dışarıdan değiştirilemez.
- `En` özelliğinin `set`'i sadece pozitif değerleri kabul eder; aksi halde `Geçersiz en: ...` yazar ve alanı değiştirmez.
- `Boy` yalnızca kurucuda atanır. `Alan` hesaplanan bir özelliktir.
- `ToString` → `Dikdortgen[3 x 4]` biçiminde metin döndürür.



UML'de `+` public, `-` private, `#` protected, `~` internal demektir; `{static}` static üyeyi, `{get}` yalnızca okunur, `{get, set}` okunur ve yazılır özelliği gösterir. Tip, adın **sonuna** iki noktayla yazılır.

Üst düzey deyimler şunları yapıp aşağıdaki çıktıyı vermelidir: `(3, 4)` ve `(5)` ile iki nesne oluştur, birincinin alanını, ikincinin kare olup olmadığını yazdır; birincinin enini önce `-2`, sonra `4` yap ve tekrar kare mi diye sor; son olarak oluşturulan dikdörtgen sayısını yazdır.

```
Dikdortgen[3 x 4] alan=12
Dikdortgen[5 x 5] kare mi? True
Geçersiz en: -2
Dikdortgen[4 x 4] kare mi? True
Oluşturulan: 2
```

10 Kullanıcıdan önce öğrenci sayısını, sonra her öğrencinin **adını** (tek kelime) ve **notunu** (tam sayı) aynı satırda okuyan bir program yaz. Öğrenciler, alanları `private` olan bir `Ogrenci` sınıfının nesneleri olarak bir **dizide** saklansın (`ToString` → `Ayşe (95)`). Program sınıf ortalamasını, en yüksek notu alan öğrenciyi ve ortalamanın üstündeki öğrencileri yazdırsın:

```
Öğrenci sayısı: 4
1. öğrenci (ad not): Ali 70
2. öğrenci (ad not): Ayşe 95
3. öğrenci (ad not): Can 55
4. öğrenci (ad not): Deniz 84
Ortalama: 76,00
En yüksek: Ayşe (95)
Ortalamanın üstündekiler:
Ayşe (95)
Deniz (84)
```

Çözümler

Cevap anahtarı

1	2	3	4	5
E	C	A	D	B

1 Cevap: E

Kurucunun **dönüş tipi yoktur**; `void` bile yazılmaz. Yazılırsa o üye kurucu olmaz ve C# zaten "üye adı, kapsayan sınıfın adıyla aynı olamaz" diye hata verir (CS0542). A, B (aşırı yükleme), C (varsayılan kurucu) ve D doğrudur.

2 Cevap: C

Dışarıdan sonradan atamaya yalnızca `{ get; set; }` izin verir.

- A) `set` hiç yok → **CS0200** (salt okunur).
- B) `set` var ama `private` → **CS0272** (erişilemez `set`). A ve B'nin derleyici mesajlarını bölümdeki `ozellikTurleri` programında görmüştün.
- D) `init`, yalnızca kurucuda ve nesne başlatıcıda atamaya izin verir; `h.Unvan = ...` → **CS8852**.
- E) `=>` ile yazılan hesaplanan özellik `set` içermez → **CS0200**.

3 Cevap: A

`dizi[0]` ve `dizi[1]`, `a` ile **aynı** nesneyi gösterir; `dizi[0].Deger += 2` bu ortak nesneyi 6 yapar, bu yüzden `dizi[1].Deger` de 6 olur. Sonra `a` **yeni** bir nesneye yönlendirilir ve onun değeri 10 olur; dizideki iki referans hâlâ eski nesneyi (6) göstermektedir. B ve C, dizi elemanlarının ayrı kopya olduğunu varsayar; D ve E, `a`'nın yeni nesnesinin diziyi etkilediğini varsayar.

Çıktı

```
10 6 6
```

4 Cevap: D

İki kurucu var: `Kitap(string)` ve `Kitap(string, int)`. **Parametresiz kurucu yok**: Kendi kurucularını yazdığın için derleyici varsayılan kurucuyu eklemedi.

- A) parametresiz kurucu aranır, yok (CS7036). B) `int` argüman `string` parametresine verilemez. C) argüman sırası ters.
- E) nesne başlatıcı önce **parametresiz** kurucuyu çağırır; o yok (CS7036). Klasik tuzak.
- D) ikinci kurucuyla birebir uyar. Doğru cevap.

5 Cevap: B

`struct` kopyalanır, `class` paylaşılır.

- `q = p`: `q` bağımsız kopyadır. `q.X = 5` yalnızca `q`'yu değiştirir. `SifirlaS(p)` de `p`'nin **kopyasını** sıfırlar; `p.X` hâlâ 1.
- `d = c`: aynı nesne. `d.X = 5` → `c.X` de 5. `SifirlaC(c)` ortak nesneyi 0 yapar; hem `c.X` hem `d.X` 0.

Sonuç: `1 5 0 0`.

```
Çıktı
1 5 0 0
```

6

`Toplam` ve `Olusan` static olduğu için tek kopyadır; `Adet` her nesnede ayrıdır.

İşlem	s1.Adet	s2.Adet (= s3.Adet)	Toplam	Olusan
iki `new Sayac()`	0	0	0	2
`s1.Artir()` × 2	2	0	2	2
`s2.Artir()`	2	1	3	2
`s3 = s2; s3.Artir()`	2	2	4	2
`s1 = new Sayac(); s1.Artir()`	1 (yeni nesne)	2	5	3

`s3 = s2;` yeni nesne üretmez, `Olusan` artmaz; `s3.Artir()` ise `s2` ile ortak nesnenin `Adet`'ini artırır. Toplam üç `new` vardır.

```
Çıktı
2 2 2
4
1 5 3
```

7

- `new Urun()` → `: this("Kalem")` → `: this("Kalem", 10)`. **En içteki kurucunun gövdesi önce** çalışır: `Ad = "Kalem"`, `Fiyat = 20` ve `C Kalem` yazılır.
- Kontrol geri döner: `B Kalem`, sonra `A`. İlk satırın sonucu `Kalem 20`.
- `new Urun { Ad = "Silgi", Fiyat = 7 }` önce **parametresiz kurucuyu**, yani `C Kalem`, `B Kalem`, `A` zincirini çalıştırır; atamalar ancak ondan **sonra** yapılır. Başlatıcıdaki `Fiyat = 7` kurucudaki `* 2` işleminden geçmez: `Silgi 7`.

```
Çıktı
C Kalem
B Kalem
A
Kalem 20
C Kalem
B Kalem
A
Silgi 7
```

8

- `Degistir(a, s)`: `p` ile `a` aynı nesneyi gösterir; `p.X = 1 + 10 = 11` kalıcıdır. `sayi` bir kopyadır, `s` 1 kalır. `p = new Nokta(99)` sadece yerel `p`'yi başka nesneye yönlendirir; `p.X++` o yeni nesneyi (100) değiştirir, `a`'yı etkilemez. → `N11 1`.
- `Yenile(ref a)`: `ref` olduğu için `p`, `a` değişkeninin **kendisidir**. `p = new Nokta(50)` ile `a` yeni nesneye yönlenir, `p.X++` onu 51 yapar. `b`'ye dokunulmadı: → `N51 N1`.
- `a == b`: iki **ayrı** nesne → `False`. (`==` içeriğe bakmaz.)
- `b = a;` artık ikisi aynı nesne; `b.X = 7` → `a` da `N7`, `a == b` → `True`.

Çıktı

```
N11 1
N51 N1
False
N7 True
```

9

UML'yi satır satır çeviriyoruz: `- _en : double` → `private double _en;`, `{static}` olanlar `static`. `{get}` yazan üyeler `set`'siz özelliktir: `Boy` otomatik özellik (`{ get; }`, kurucuda atanır), `Alan` ve `Adet` ifade gövdelidir. `En` için ayrı bir `_en` alanı ve doğrulamalı `set` gerekir. Kurucuda da `_en = en;` değil `En = en;` yazıyoruz: alana doğrudan atasaydık doğrulama atlanır ve `new Dikdortgen(-5, 4)` eksi enli bir nesne oluştururdu. Önce alanlar ve özellikler, sonra kurucular, en son metotlar yazılır; diyagramda olmayan üyeyi ekleme, hocalar fazlalığı da hata sayabilir. Sayacı yalnızca iki parametreli kurucuda artırıyoruz; tek parametreli kurucu oraya `: this(...)` ile yönlendiği için her nesne **bir kez** sayılır.

dikdortgenTest.cs

```
Dikdortgen d1 = new Dikdortgen(3, 4);
Dikdortgen d2 = new Dikdortgen(5);
Console.WriteLine($"{d1} alan={d1.Alan}");
Console.WriteLine($"{d2} kare mi? {d2.KareMi()}");
d1.En = -2;
d1.En = 4;
Console.WriteLine($"{d1} kare mi? {d1.KareMi()}");
Console.WriteLine($"Oluşturulan: {Dikdortgen.Adet}");

class Dikdortgen
{
    private double _en;
    private static int _adet;

    public double En
    {
        get { return _en; }
        set
        {
            if (value <= 0)
            {
                Console.WriteLine($"Geçersiz en: {value}");
                return;
            }
            _en = value;
        }
    }

    public double Boy { get; }
    public double Alan => _en * Boy;
    public static int Adet => _adet;

    public Dikdortgen(double en, double boy)
    {
        En = en;            // özellik üzerinden: doğrulama çalışsın
        Boy = boy;
        _adet++;
    }

    public Dikdortgen(double kenar) : this(kenar, kenar)
    {
    }
```

```
    public bool KareMi() => _en == Boy;

    public override string ToString() => $"Dikdortgen[{_en} x {Boy}]";
}
```

Çıktı

```
Dikdortgen[3 x 4] alan=12
Dikdortgen[5 x 5] kare mi? True
Geçersiz en: -2
Dikdortgen[4 x 4] kare mi? True
Oluşturulan: 2
```

`_adet++` satırını tek parametreli kurucuya da yazsaydık kare oluşturulurken sayaç iki kez artardı. Çıktıda `3,0` değil `3` yazıldığına dikkat: `double` değerler sondaki sıfırsız yazılır.

10

Önce `n` okunur ve `new Ogrenci[n]` ile **boş** dizi oluşturulur; döngüde her eleman için ayrıca `new Ogrenci(...)` yapılır. Her satır `Split(' ')` ile ad ve nota ayrılır, not `Convert.ToInt32` ile sayıya çevrilir. Ortalama ve en yüksek not ilk döngüde, ortalamanın üstündekiler ikinci döngüde bulunur. `{ortalama:F2}` Türkçe kültürde virgüllü yazılır (`76,00`).

sinifListesi.cs

```
Console.Write("Öğrenci sayısı: ");
int n = Convert.ToInt32(Console.ReadLine());
Ogrenci[] sinif = new Ogrenci[n];

for (int i = 0; i < n; i++)
{
    Console.Write($"{i + 1}. öğrenci (ad not): ");
    string[] parca = (Console.ReadLine() ?? "").Split(' ');
    sinif[i] = new Ogrenci(parca[0], Convert.ToInt32(parca[1]));
}

double toplam = 0;
Ogrenci enYuksek = sinif[0];
foreach (Ogrenci o in sinif)
{
    toplam += o.Not;
    if (o.Not > enYuksek.Not)
    {
        enYuksek = o;
    }
}
double ortalama = toplam / n;
Console.WriteLine($"Ortalama: {ortalama:F2}");
Console.WriteLine($"En yüksek: {enYuksek}");

Console.WriteLine("Ortalamanın üstündekiler:");
foreach (Ogrenci o in sinif)
{
    if (o.Not > ortalama)
    {
        Console.WriteLine(o);
    }
}

class Ogrenci
{
    private string _ad;
```

```
    private int _not;

    public Ogrenci(string ad, int puan)
    {
        _ad = ad;
        _not = puan;
    }

    public int Not => _not;

    public override string ToString() => $"{_ad} ({_not})";
}
```

Çıktı

```
Öğrenci sayısı: 4
1. öğrenci (ad not): Ali 70
2. öğrenci (ad not): Ayşe 95
3. öğrenci (ad not): Can 55
4. öğrenci (ad not): Deniz 84
Ortalama: 76,00
En yüksek: Ayşe (95)
Ortalamanın üstündekiler:
Ayşe (95)
Deniz (84)
```

`toplam`'ı `double` tanımladığımız için `toplam / n` tam sayı bölmesi yapmaz. `Console.WriteLine(o)` ve `{enYuksek}` ifadeleri `ToString`'i otomatik çağırır.

BÖLÜM 10

Kalıtım

- **Bu Bölümde Öğreneceklerin**
  - Kod tekrarını kalıtımla önlemeyi; `:` ile türetmeyi, taban sınıf / alt sınıf kavramlarını ve "is-a" ilişkisini
  - Alt sınıfa neyin geçip neyin geçmediğini (`private`, `protected`, kurucular)
  - `: base(...)` ile taban sınıf kurucusunu çağırmayı ve kurucuların **çalışma sırasını**
  - Metot ezmeyi (`virtual` + `override`), `new` ile gizlemeyi ve ikisinin **farkını** (sınav favorisi)
  - `base.Metot()` çağrısını ve `sealed` anahtar kelimesini
  - Yukarı ve aşağı dönüşümü: `is`, `as` ve `InvalidCastException`
  - `object` sınıfını: `ToString`, `Equals`, `GetHashCode`, `GetType()`
  - Sınıflarda neden tekli kalıtım olduğunu; "is-a" ile "has-a" farkını

10.1 Kod tekrarı problemi

Bir okul uygulaması yazdığını düşün. Öğrenci ve öğretmen için iki sınıf gerekiyor:

```
class Ogrenci
{
    public string Ad { get; set; } = "";
    public int Yas { get; set; }
    public int No { get; set; }
    public void Tanit() => Console.WriteLine($"Ben {Ad}, {Yas} yaşındayım.");
}

class Ogretmen
{
    public string Ad { get; set; } = "";
    public int Yas { get; set; }
    public string Brans { get; set; } = "";
    public void Tanit() => Console.WriteLine($"Ben {Ad}, {Yas} yaşındayım.");
}
```

(Tek ifadeli metotları `=>` ile kısa yazıyoruz; bu bölümde yer kazanmak için sık kullanacağız.) `Ad`, `Yas` ve `Tanit()` iki sınıfta **birebir aynı**. Yarın "e-posta" özelliği eklemek istersen iki yeri değiştirmen, `Tanit()`'teki bir hatayı düzeltirken ikisini birden düzeltmeyi hatırlaman gerekir. Memur, veli, rektör sınıfları da gelince iş içinden çıkılmaz hale gelir.

Çözüm: Ortak kısımları **tek bir sınıfta** (`Kisi`) topla, diğer sınıflar bu sınıftan **kalıtım** (inheritance) yoluyla bu kısımları devralsın ve yalnızca kendilerine özgü olanları eklesin.

10.2 Kalıtım sözdizimi

Bir sınıfın başka bir sınıftan türediğini sınıf adından sonra gelen `:` ile belirtiriz. `class Ogrenci : Kisi` şöyle okunur: "Ogrenci, Kisi'den türer."

kalitimGiris.cs

```
Ogrenci o = new Ogrenci { Ad = "Ali", Yas = 19, No = 101 };
o.Tanit();                          // Kisi'den gelen metot
o.DersCalis();

Ogretmen t = new Ogretmen { Ad = "Selin Hoca", Yas = 41, Brans = "C#" };
t.Tanit();
t.DersAnlat();

class Kisi
{
    public string Ad { get; set; } = "";
    public int Yas { get; set; }
    public void Tanit() => Console.WriteLine($"Ben {Ad}, {Yas} yaşındayım.");
}

class Ogrenci : Kisi
{
    public int No { get; set; }         // yalnız öğrencide olan özellik
    public void DersCalis() => Console.WriteLine($"{Ad} ders çalışıyor.");
}

class Ogretmen : Kisi
{
    public string Brans { get; set; } = "";
    public void DersAnlat() => Console.WriteLine($"{Ad} {Brans} anlatıyor.");
}
```

Çıktı

```
Ben Ali, 19 yaşındayım.
Ali ders çalışıyor.
Ben Selin Hoca, 41 yaşındayım.
Selin Hoca C# anlatıyor.
```

`Ogrenci` sınıfında `Ad`, `Yas` ve `Tanit()` yazmadığımız halde `o.Ad` ve `o.Tanit()` çalıştı: Bunlar `Kisi`'den **kalıtıldı**. `DersCalis` metodu da kalıtılan `Ad`'ı kendi özelliği gibi okuyor. (Bölüm 9'daki gibi bütün sınıflar tek dosyada, ana kodun altında.)

Kalıtım veren	Kalıtım alan
Taban sınıf (base class)	**Türetilmiş sınıf** (derived class)
Üst sınıf (superclass), ebeveyn (parent)	Alt sınıf (subclass), çocuk (child)

Bu kitapta "taban sınıf" ve "alt sınıf" diyeceğiz; sınavda hepsi birbirinin yerine kullanılır. Kalıtım bir **"is-a"** (*bir … dır*) ilişkisidir: "Öğrenci **bir** kişidir" doğrudur, tersi doğru değildir. Alt sınıf, taban sınıfın **özelleşmiş** halidir: Her şeyine sahiptir, üstüne kendi üyelerini ekler. UML'de kalıtım, alt sınıftan taban sınıfa giden ve ucu **içi boş üçgen** olan bir okla gösterilir (`Ogrenci → Kisi`).

> **İpucu: "is-a" testi**
>
> Kalıtım kurmadan önce cümleyi yüksek sesle söyle: "**Kedi bir Hayvandır**" (doğru → `class Kedi : Hayvan`), "**Kare bir Şekildir**" (doğru). "Araba bir Motordur" saçmaysa kalıtım da yanlıştır (bölümün sonunda "has-a" ilişkisini göreceksin).

10.3 Tek taban sınıf kuralı

C#'ta bir sınıfın `:` işaretinden sonra **yalnızca bir** taban sınıfı olabilir. İki sınıftan birden türetmeye çalışırsan derleyici durdurur:

cokluKalitim.cs

```
Console.WriteLine("Hiyerarşi kurulamadı");

class Telefon { }
class Kamera { }
class AkilliTelefon : Telefon, Kamera { }       // iki taban sınıf: yasak
```

Derleyici çıktısı – program derlenmez

```
cokluKalitim.cs(5,32): error CS1721: Class 'AkilliTelefon' cannot have multiple base classes:
'Telefon' and 'Kamera'
```

Neden? **Elmas problemi** (diamond problem) yüzünden: `Telefon` ve `Kamera` ikisi de ortak bir `Cihaz` sınıfındaki `Calis()` metodunu farklı biçimde ezmiş olsaydı, `AkilliTelefon` nesnesinde `Calis()` çağrısının hangisini çalıştıracağı belirsiz kalırdı. C# bu belirsizliği kökten engeller. Bir sınıfın birden fazla "tür" olması gerektiğinde **arayüzler** (interface) kullanılır: `class AkilliTelefon : Cihaz, ITelefon, IKamera` gibi bir taban sınıf ve istediğin kadar arayüz yazabilirsin (Bölüm 11).

Zincir biçimindeki **çok seviyeli** kalıtım (`B : A`, `C : B`) ve bir tabandan çok alt sınıf türeten **hiyerarşik** kalıtım (`B : A`, `C : A`) serbesttir. Yasak olan yalnız **çoklu** kalıtımdır: `C : A, B` (iki sınıf) → CS1721.

10.4 Neler kalıtılır?

Alt sınıf, taban sınıfın `public` ve `protected` üyelerini (aynı projede olduğumuz için `internal` üyelerini de) kullanabilir. `private` üyeler için durum biraz incedir: Alt sınıf nesnesinin içinde taban sınıfın `private` alanları **vardır** ve bellekte yer kaplar; ama alt sınıfın kodu onlara **adıyla erişemez**. Bunlara taban sınıfın `public` (veya `protected`) üyeleri üzerinden ulaşılır:

privateKalitim.cs

```
new Ogrenci().Yaz();

class Kisi
{
    private string _ad = "Ali";
    public string Ad => _ad;
}

class Ogrenci : Kisi
{
    public void Yaz()
    {
        Console.WriteLine(Ad);          // tamam: public özellik
        Console.WriteLine(_ad);         // HATA: private alan
    }
}
```

Derleyici çıktısı – program derlenmez

```
privateKalitim.cs(14,27): error CS0122: 'Kisi._ad' is inaccessible due to its protection level
```

Özet: `public`, `protected` ve `internal` üyelere alt sınıftan adıyla erişilir; `private` üyelere erişilmez; kurucular **kalıtılmaz**, yalnız `: base(...)` ile çağrılır.

Sınavda "`private` üyeler alt sınıfa geçer mi?" diye sorulursa iki cümleyi birlikte yaz: Alan alt sınıf nesnesinin içinde **vardır**, ama alt sınıfın kodu ona adıyla **erişemez**; kurucular ise hiç kalıtılmaz.

10.5 protected erişim

`protected` üyelere yalnız **kendi sınıfı ve alt sınıfları** erişebilir. Taban sınıf, alt sınıfların doğrudan kullanmasını istediği alanları `protected` yapar. İki sınır çizgisi sınavda sorulur: Sınıfın **dışından** erişilemez; ve alt sınıf, protected üyeye yalnız **kendi türünden** nesne üzerinden erişir, taban türünden değişken üzerinden erişemez:

protectedHata.cs

```
VadeliHesap h = new VadeliHesap { FaizOrani = 0.05 };
h.Yatir(1000);
Console.WriteLine(h._bakiye);              // HATA 1: sınıfın dışı

class Hesap
{
    protected double _bakiye;              // alt sınıflar kullanabilir
    private int _islemSayisi;              // alt sınıf dahil kimse göremez

    public void Yatir(double miktar)
    {
        _bakiye += miktar;
        _islemSayisi++;
    }
}

class VadeliHesap : Hesap
{
    public double FaizOrani { get; set; }

    public void Karsilastir(VadeliHesap benzer, Hesap diger)
    {
        _bakiye += _bakiye * FaizOrani;    // tamam: kendi nesnem
        Console.WriteLine(benzer._bakiye); // tamam: VadeliHesap tipinde
        Console.WriteLine(diger._bakiye);  // HATA 2: Hesap tipinde başvuru
        Console.WriteLine(_islemSayisi);   // HATA 3: private
    }
}
```

Derleyici çıktısı – program derlenmez

```
protectedHata.cs(3,21): error CS0122: 'Hesap._bakiye' is inaccessible due to its protection level
protectedHata.cs(25,33): error CS1540: Cannot access protected member 'Hesap._bakiye' via a
qualifier of type 'Hesap'; the qualifier must be of type 'VadeliHesap' (or derived from it)
protectedHata.cs(26,27): error CS0122: 'Hesap._islemSayisi' is inaccessible due to its protection
level
```

Sınavda Çıkar: protected erişim tablosu

`private`: yalnız kendi sınıfı. `protected`: kendi sınıfı + alt sınıflar (alt sınıfta yalnız alt sınıf tipli nesneyle). `internal`: aynı proje. `public`: herkes. İki birleşik belirteç de vardır: `protected internal` (alt sınıf **veya** aynı proje), `private protected` (alt sınıf **ve** aynı proje). Tek dosyalık projede bile ana koddan `protected` alana erişmek derleme hatasıdır. Alan yerine `public double Bakiye { get; protected set; }` yazarsan dışarıdan herkes okur, değeri yalnız sınıf ve alt sınıfları değiştirir.

10.6 Kurucular ve base(...)

Kurucular kalıtılmaz. Ama bir `Ogrenci` nesnesinin içinde bir de `Kisi` kısmı vardır ve o kısmın ilk değerlerini **`Kisi`'nin kurucusu** vermelidir. Alt sınıfın kurucusu, taban sınıfın kurucusunu `: base(...)` ile çağırır:

baseYapici.cs

```
Ogrenci o = new Ogrenci("Ali", 19, 101);
o.Yaz();

class Kisi
{
    public string Ad { get; }
    public int Yas { get; }

    public Kisi(string ad, int yas)
    {
        Ad = ad;
        Yas = yas;
        Console.WriteLine("Kisi kurucusu çalıştı");
    }
}

class Ogrenci : Kisi
{
    public int No { get; }

    public Ogrenci(string ad, int yas, int no) : base(ad, yas)
    {
        No = no;
        Console.WriteLine("Ogrenci kurucusu çalıştı");
    }

    public void Yaz() => Console.WriteLine($"{No} {Ad} ({Yas})");
}
```

Çıktı

```
Kisi kurucusu çalıştı
Ogrenci kurucusu çalıştı
101 Ali (19)
```

`: base(ad, yas)` kurucunun parametre listesinden sonra, gövdeden **önce** yazılır ve gövdeden önce çalışır: Önce taban kısım kurulur, sonra alt sınıfın kendi satırları gelir. Kurallar:

1. `: base(...)` yazmazsan derleyici gövdeden önce **örtük** olarak `: base()` ekler; yani taban sınıfın **parametresiz** kurucusunu çağırır.
2. Bir kurucunun `:` işaretinden sonra yalnız **tek bir** çağrısı olabilir: ya `: this(...)` (Bölüm 9'daki zincir) ya `: base(...)`. İkisi birlikte yazılamaz; zincirin sonundaki kurucu mutlaka bir `base`'e ulaşır.

İlk kuralın sonucu sınavların en sevilen derleme hatasıdır: Taban sınıfta parametresiz kurucu **yoksa** ve alt sınıf `: base(...)` yazmazsa, derleyicinin eklediği `: base()` çağrısının karşılığı bulunamaz:

baseEksik.cs

```
Ogrenci o = new Ogrenci(101);

class Kisi
{
    public string Ad { get; }
    public Kisi(string ad) => Ad = ad;      // tek kurucu: parametreli
}

class Ogrenci : Kisi
{
    public int No { get; }
    public Ogrenci(int no) => No = no;      // derleyici buraya : base() ekler
}
```

Derleyici çıktısı – program derlenmez

```
baseEksik.cs(12,12): error CS7036: There is no argument given that corresponds to the required
parameter 'ad' of 'Kisi.Kisi(string)'
```

Hata (CS7036), `Kisi` kurucusunun `ad` parametresi için argüman verilmediğini söylüyor. Çözüm: Ya `Ogrenci` kurucusuna `: base("...")` ekle, ya da `Kisi`'ye parametresiz bir kurucu yaz. Dikkat: `Ogrenci`'de **hiç kurucu yazmasan** da aynı hatayı alırsın, çünkü derleyicinin eklediği varsayılan kurucu `Ogrenci() : base()` biçimindedir.

> ● **Yeni C# Notu: Birincil kurucuyla kalıtım**
>
> C# 12'den beri sınıf adının yanına parametre yazabilirsin (birincil kurucu, *primary constructor*): `class Ogrenci(string ad, int no) : Kisi(ad)`. Taban kurucusu aynı satırda çağrılır. Sınavlarda hâlâ klasik `: base(...)` yazımı sorulur; bu bir kısayoldur.

10.7 Kurucuların çalışma sırası

Kalıtım zincirinde her kurucunun ilk işi (açıkça veya örtük olarak) **taban sınıfın kurucusunu çağırmaktır**. Bu yüzden `new Kopek()` dediğinde kurucu gövdeleri **en üstten aşağıya doğru** çalışır:

kurucuSirasi.cs

```
new Kopek();
Console.WriteLine("---");
new Hayvan();

class Canli
{
    public Canli() => Console.WriteLine("1. Canli kurucusu");
}

class Hayvan : Canli
{
    public Hayvan() => Console.WriteLine("2. Hayvan kurucusu");
}

class Kopek : Hayvan
{
    public Kopek() => Console.WriteLine("3. Kopek kurucusu");
}
```

```
Çıktı
1. Canli kurucusu
2. Hayvan kurucusu
3. Kopek kurucusu
---
1. Canli kurucusu
2. Hayvan kurucusu
```

Hiçbir kurucuda `: base()` yazmadık; derleyici hepsine ekledi. (Tek ifadeli kurucuları da `=>` ile kısa yazdık.) Çağrı ve çalışma sırası:

```
new Kopek()
 └ Kopek()          ilk iş: base()
     └ Hayvan()        ilk iş: base()
         └ Canli()        ilk iş: base()
             └ object()      (her sınıfın en üst atası)
             Canli gövdesi çalışır      → 1. satır
         Hayvan gövdesi çalışır         → 2. satır
     Kopek gövdesi çalışır              → 3. satır
```

> **● Sınavda Çıkar: Kurucu sırası sorusu**
>
> Kural: Kurucular **alttan yukarı çağrılır**, gövdeleri **yukarıdan aşağı biter**. Önce temel atılır, sonra katlar çıkılır. Soruda `: this(...)` da varsa, önce aynı sınıftaki hedef kurucuya git, zincirin `: base(...)`'e ulaştığı yerden yukarı çık. Ezbere gitme; adım adım bir **çağrı ağacı** çiz.

10.8 Metot ezme: virtual ve override

Alt sınıf, taban sınıftan gelen bir metodu **aynı imzayla yeniden yazarak** kendi davranışını tanımlayabilir. Buna **metot ezme** (overriding, *geçersiz kılma*) denir. C#'ta iki anahtar kelime birlikte çalışır: Taban sınıftaki metot `virtual` (ezilebilir) olmalı, alt sınıftaki metot `override` ile yazılmalı.

ezme.cs

```
Kedi k = new Kedi { Ad = "Tekir" };
Kopek d = new Kopek { Ad = "Karabaş" };
Balik b = new Balik { Ad = "Nemo" };
k.SesCikar();
d.SesCikar();
b.SesCikar();
Console.WriteLine("---");
k.Tanit();
d.Tanit();
b.Tanit();

class Hayvan
{
    public string Ad { get; set; } = "";
    public virtual void SesCikar() => Console.WriteLine("...");     // ezilebilir

    public void Tanit()
    {
        Console.Write($"Ben {Ad}. Sesim: ");
        SesCikar();                         // hangi SesCikar çalışacak?
    }
}

class Kedi : Hayvan
{
```

```
    public override void SesCikar() => Console.WriteLine("Miyav!");
}

class Kopek : Hayvan
{
    public override void SesCikar() => Console.WriteLine("Hav hav!");
}

class Balik : Hayvan { }                    // SesCikar ezilmedi
```

Çıktı

```
Miyav!
Hav hav!
...
---
Ben Tekir. Sesim: Miyav!
Ben Karabaş. Sesim: Hav hav!
Ben Nemo. Sesim: ...
```

İkinci kısma dikkat: `Tanit()` metodu **`Hayvan`'ın kodudur** ve içinde `SesCikar()` çağrısı vardır. Yine de `k.Tanit()` "Miyav!" yazdı. Ezilmiş (`virtual`) bir metot çağrıldığında C#, **nesnenin gerçek sınıfına** bakar: Nesne `Kedi` ise `Kedi`'nin `SesCikar`'ı çalışır, çağrı taban sınıfın içinden yapılmış olsa bile. Buna **dinamik bağlama** (dynamic binding) denir; Bölüm 11'de çok biçimliliğin temeli olarak tekrar göreceksin.

Kural	Açıklama
Taban metot `virtual` olmalı	`virtual` olmayan metot ezilemez (CS0506). `abstract` ve `override` metotlar da ezilebilir.
Alt sınıfta `override` yazılmalı	Yazmazsan ezme olmaz: **gizleme** olur (aşağıda).
İmza aynı olmalı	Ad ve parametre listesi birebir aynı. Liste farklıysa bu **aşırı yüklemedir**; ezilecek metot yoksa CS0115. Parametre **adlarının** önemi yoktur.
Dönüş tipi ve erişim aynı olmalı	`public` metot `protected` yapılamaz (CS0507).
`static` ve `private` metotlar `virtual` olamaz	Ezilemezler.

Yanlış yazılan ezmeler derleme hatasıdır: Ezilecek metot yoksa (yazım hatalı `override void SesCikart()`) CS0115, `virtual` olmayan metodu `override` etmeye çalışırsan CS0506, `public` metodu `protected override` yazarsan CS0507 alırsın. C#'ta `override` yazmak **zorunludur**; yazım hatalı bir ezme sessizce yeni bir metoda dönüşmez, derleyici durdurur. Ama `override` yazmayı **unutursan** başka bir şey olur: gizleme. Bunu birazdan göreceğiz.

10.9 base.Metot() ile taban sınıfın metodunu çağırma

Ezme, taban sınıfın metodunu tamamen değiştirir. Çoğu zaman ise taban sınıfın yaptığını **tekrar yazmadan genişletmek** isteriz. Alt sınıfın içinde `base.MetotAdi(...)` yazarak taban sınıfın versiyonunu çağırabilirsin:

baseMetot.cs

```
Kisi k = new Kisi { Ad = "Ali", Yas = 19 };
Ogrenci o = new Ogrenci { Ad = "Ayşe", Yas = 20, No = 102 };
k.Tanit();
Console.WriteLine("---");
o.Tanit();

class Kisi
{
```

```
    public string Ad { get; set; } = "";
    public int Yas { get; set; }
    public virtual void Tanit() => Console.WriteLine($"Ad: {Ad}, yaş: {Yas}");
}

class Ogrenci : Kisi
{
    public int No { get; set; }

    public override void Tanit()
    {
        base.Tanit();                       // önce ortak bilgiler
        Console.WriteLine($"No: {No}");
    }
}
```

Çıktı

```
Ad: Ali, yaş: 19
---
Ad: Ayşe, yaş: 20
No: 102
```

Sık Yapılan Hata: base. yazmayı unutmak

`Ogrenci.Tanit()` içinde `base.Tanit();` yerine `Tanit();` yazarsan metot **kendini** çağırır, o da yine kendini... Sonu gelmeyen bu çağrı zinciri çağrı yığınını doldurur ve program `StackOverflow` ile çöker; C#'ta bu hata yakalanamaz (özyinelemeyi Bölüm 15'te göreceksin). Taban sınıfın versiyonunu istiyorsan `base.` şarttır. `base.base.Metot()` diye bir yazım **yoktur**.

10.10 Yukarı dönüşüm: taban tipli değişken

"Öğrenci bir kişidir" olduğuna göre, `Ogrenci` nesnesini `Kisi` tipli bir değişkende tutabilirsin. Buna **yukarı dönüşüm** (upcasting) denir; otomatiktir, cast yazmak gerekmez:

```
Ogrenci o = new Ogrenci { Ad = "Ali", No = 101 };
Kisi k = o;                 // yukarı dönüşüm: Ogrenci → Kisi
```

`o` ve `k` **aynı nesneyi** gösterir. Fark değişkenin tipidir; tip bir **gözlük** gibidir: `k` üzerinden yalnız `Kisi`'de tanımlı üyeleri görürsün (`k.No` derlenmez). Ama `virtual`/`override` çifti olan bir metodu `k` üzerinden çağırırsan yine `Ogrenci`'nin ezilmiş versiyonu çalışır. Bunu bir sonraki başlıkta, `override` yazılmadığında ne olduğuyla karşılaştırarak göreceğiz.

10.11 Gizleme (hiding) ve new

Taban sınıfta `virtual` bir metot var, alt sınıfta aynı imzalı metodu `override` **yazmadan** yazdın. C# hata vermez, ama bir **uyarı** verir:

gizlemeUyari.cs

```
Kedi kedi = new Kedi();
Hayvan h = kedi;
kedi.SesCikar();
h.SesCikar();

class Hayvan
{
    public virtual void SesCikar() => Console.WriteLine("...");
}

class Kedi : Hayvan
{
    public void SesCikar() => Console.WriteLine("Miyav!");     // override yok
}
```

Çıktı

```
Miyav!
...
```

Derleyici alt sınıftaki metodu **ezme** olarak değil, taban metodu **gizleyen** (hiding), ayrı bir metot olarak aldı. Uyarı iki çözüm sunar: Niyetin ezmekse `override`, gizlemekse `new` ekle. Taban metot `virtual` değilse aynı durumun uyarı kodu CS0108'dir; `virtual`, `abstract` ya da `override` ise CS0114.

> **● Sık Yapılan Hata: override'ı unutmak**
>
> Yukarıdaki program derlenir ve çalışır; bu yüzden tehlikelidir. `h.SesCikar()` Kedi nesnesi için bile taban sınıfın metodunu çalıştırır ve çok biçimlilik sessizce bozulur. Uyarıyı **asla** görmezden gelme; metodu `virtual` yaptıysan alt sınıfta `override` yaz. `new`'ü de uyarıyı susturmak için yazma: "bu gizlemeyi bilerek yapıyorum" demektir.

Gizlemeyi bilerek yapmak istersen `new` yazarsın. Şimdi aynı nesneyi iki değişkenle (`Kedi` ve `Hayvan` tipinde) çağıralım; bir metot `override`, öteki `new` olsun:

newKarsilastirma.cs

```
Kedi kedi = new Kedi();
Hayvan hayvan = kedi;                  // aynı nesne, taban tipli değişken

Console.WriteLine("Kedi değişkeni:");
kedi.SesCikar();
kedi.Hareket();
Console.WriteLine("Hayvan değişkeni:");
hayvan.SesCikar();
hayvan.Hareket();

class Hayvan
{
    public virtual void SesCikar() => Console.WriteLine("Hayvan sesi: ...");
    public void Hareket() => Console.WriteLine("Hayvan yürüyor");
}

class Kedi : Hayvan
{
    public override void SesCikar() => Console.WriteLine("Kedi sesi: Miyav!");
    public new void Hareket() => Console.WriteLine("Kedi sinsice yürüyor");
}
```

```
Çıktı
Kedi değişkeni:
Kedi sesi: Miyav!
Kedi sinsice yürüyor
Hayvan değişkeni:
Kedi sesi: Miyav!
Hayvan yürüyor
```

Aynı nesne, aynı çağrı biçimi, farklı sonuç:

Çağrı	Değişkenin tipi	Nesne	Çalışan metot
`kedi.SesCikar()`	Kedi	Kedi	Kedi (`override`)
`hayvan.SesCikar()`	Hayvan	Kedi	**Kedi** (`override` → nesneye bakılır)
`kedi.Hareket()`	Kedi	Kedi	Kedi (`new`)
`hayvan.Hareket()`	Hayvan	Kedi	**Hayvan** (`new` → değişkenin tipine bakılır)

> ● **Sınavda Çıkar: new mi, override mı? (en sık tuzak)**
>
> Taban tipli bir değişken (`Hayvan h = new Kedi();`) görürsen şunu sor: Metot `virtual`/`override` çifti mi? Öyleyse **nesne** (Kedi) kazanır, çok biçimlilik vardır. `new` ya da sıradan (virtual olmayan) bir metot mu? Öyleyse **değişkenin tipi** (Hayvan) kazanır. Bu kural taban sınıfın **kendi içinden** yapılan çağrılar için de geçerlidir: Taban sınıfın kodundaki `Hareket()` çağrısı her zaman taban sınıfın `Hareket`'ine gider.

Aşırı yükleme ve ezme farkı

İki terim birbirine çok benzer ama bambaşka şeylerdir; hocaların en sevdiği karşılaştırma sorusudur:

	Aşırı yükleme (overloading)	Ezme (overriding)
Nerede?	Aynı sınıfta (veya alt sınıfta)	Alt sınıfta; kalıtım şart
Anahtar kelime	Yok	Taban `virtual`, alt `override`
Parametre listesi	**Farklı** olmalı	**Aynı** olmalı
Dönüş tipi ve erişim	Serbest	Aynı olmalı (C# 9'dan beri dönüş tipi, tabandakinin bir alt sınıfı da olabilir)
`static`, `private` metotlar	Aşırı yüklenebilir	Ezilemez
Hangi metot çalışır?	**Derleme** zamanında, argüman tiplerine göre	**Çalışma** zamanında, nesnenin gerçek sınıfına göre

Bölüm 6'daki aynı adlı, farklı parametreli metotlar (ör. `int` ve `double` alan iki `Topla`) aşırı yüklemeydi; `Kedi.SesCikar()` ezmedir. İkisi bir arada da olabilir (Soru 2).

10.12 sealed: ezilemez, türetilemez

`sealed` (*mühürlü*) iki yerde kullanılır: **Sınıfa** yazılırsa o sınıf türetilemez (`string` sınıfı da `sealed`'dır). **`override` bir metoda** yazılırsa (`sealed override`) daha aşağıdaki alt sınıflar onu tekrar ezemez. `sealed` tek başına bir metoda yazılamaz.

sealedHatalari.cs

```
Console.WriteLine("...");

sealed class Muhur { }
class Deneme : Muhur { }                    // HATA 1: sealed sınıftan türetme

class Ust
{
    public virtual void Kritik() => Console.WriteLine("Ust");
    public sealed void Tek() => Console.WriteLine("Tek");   // HATA 2
}

class Orta : Ust
{
    public sealed override void Kritik() => Console.WriteLine("Orta");
}

class Alt : Orta
{
    public override void Kritik() => Console.WriteLine("Alt");  // HATA 3
}
```

Derleyici çıktısı – program derlenmez

```
sealedHatalari.cs(4,16): error CS0509: 'Deneme': cannot derive from sealed type 'Muhur'
sealedHatalari.cs(9,24): error CS0238: 'Ust.Tek()' cannot be sealed because it is not an override
sealedHatalari.cs(19,26): error CS0239: 'Alt.Kritik()': cannot override inherited member
'Orta.Kritik()' because it is sealed
```

Bir sınıfın ya da metodun davranışının alt sınıflarca bozulmasını istemediğinde (güvenlik, doğruluk) `sealed` kullanırsın. `struct`'lar da başka bir sınıftan türeyemez.

10.13 Aşağı dönüşüm: is ve as

Yukarı dönüşümden sonra elinde `Kisi` tipli bir değişken var, ama nesne aslında bir `Ogrenci`. `k.No` yazarsan derleyici itiraz eder (CS1061: `Kisi` tipinde `No` yok); çünkü derleyici yalnız **değişkenin tipine** bakar. Çözüm, gerçek tipe geri dönmektir: **aşağı dönüşüm** (downcasting). Her `Kisi` bir `Ogrenci` olmadığı için önce **sormak** gerekir. İki güvenli yol vardır: `is` deseni ve `as`. Üçüncü yol cast'tir ve uymayan nesnede istisna fırlatır. (Örnekte `Kisi` ve `Ogrenci` sade sınıflardır: `Ad` ve `No` özellikleri; yalnız ana kodu gösteriyoruz.)

```
Kisi k1 = new Ogrenci { Ad = "Ali", No = 101 };
Kisi k2 = new Kisi { Ad = "Veli" };

if (k1 is Ogrenci o1)                       // is deseni: uyuyorsa o1'e ata
{
    Console.WriteLine($"k1 öğrenci, no: {o1.No}");
}

Ogrenci? s1 = k1 as Ogrenci;                // as: uymuyorsa null
Ogrenci? s2 = k2 as Ogrenci;
Console.WriteLine($"{s1 != null} {s2 == null}");
Console.WriteLine($"{k1 is Ogrenci} {k2 is Ogrenci}");

Ogrenci o2 = (Ogrenci)k2;                   // cast: uymuyorsa istisna
Console.WriteLine(o2.No);
```

Çıktı – program yakalanmamış istisna ile sonlanır

```
k1 öğrenci, no: 101
True True
True False
Unhandled exception. System.InvalidCastException: Unable to cast object of type 'Kisi' to type
'Ogrenci'.
   at Program.<Main>$(String[] args) in donusumler.cs:line 15
```

`k1 is Ogrenci o1` tek ifadede hem soru hem dönüşümdür: Nesne gerçekten `Ogrenci` ise `true` olur ve `o1` hazırdır; değilse `false` olur. `as` uymayan durumda `null` döndürür (bu yüzden sonuç tipi `Ogrenci?` yazıldı; nullable uyarılarını hatırla). Cast ise `Veli` bir `Kisi` olduğu için `InvalidCastException` fırlattı.

Yazım	Nesne uyuyorsa	Nesne uymuyorsa
`(Ogrenci)k`	`Ogrenci` döner	`InvalidCastException`
`k as Ogrenci`	`Ogrenci` döner	`null` döner
`k is Ogrenci o`	`true`, `o` dolu	`false`

- **Sınavda Çıkar: is, as ve cast**
  - `as` yalnız başvuru tiplerinde (sınıflar, nullable) kullanılır; sonucu `null` olabilir. Cast hata fırlatır, `as` ve `is` fırlatmaz.
  - `null is Ogrenci` her zaman `false`'tur; `null as Ogrenci` ise `null`'dır.
  - Yukarı dönüşüm (`Kisi k = ogrenci;`) cast gerektirmez ve hata vermez. Aşağı dönüşüm hep **sormayı** gerektirir.
  - Aşağı dönüşüm nesneyi değiştirmez; yalnız onu görme biçimini (değişkenin tipini) değiştirir.

10.14 object sınıfı: her sınıfın atası

Bir sınıfa `: Taban` yazmadığında, o sınıf örtük olarak C#'ın **`object`** sınıfından türer: `class Kisi { }` ile `class Kisi : object { }` aynıdır. Kurucu zincirinin en tepesinde `object()` olmasının sebebi budur. Böylece her nesnede `object`'ten gelen metotlar hazır bulunur:

Metot	Varsayılan davranış	Genelde ezilir mi?
`ToString()`	Sınıfın **tür adını** döndürür	Evet
`Equals(object? obj)`	Sınıflarda **referans eşitliği**: aynı nesne mi?	Evet; içerik karşılaştırması için
`GetHashCode()`	Nesneye özgü bir tam sayı	`Equals` ezildiyse **mutlaka**
`GetType()`	Nesnenin gerçek türü (`.Name` ile adı)	Hayır

Bölüm 9'da tarif olarak yazdığın `override ToString()`, aslında `object`'in `virtual` metodunu **ezmekti**. `public` olması da buradan gelir: Ezerken erişim değişemez. `Console.WriteLine(nesne)` ve `$"{nesne}"` bu metodu otomatik çağırır. Ezmezsen `object`'in versiyonu çalışır ve yalnız sınıfın adını yazar. Alt sınıf, taban sınıfın metnini `base.ToString()` ile genişletebilir (bölümün sonundaki `Yonetici` sınıfına bak).

`Equals` sınıflarda yalnız **aynı nesne mi?** sorusunu sorar (`==` ile aynı sonuç). İçerik karşılaştırması istiyorsan ezmelisin. İki dersi, kodları aynıysa eşit sayalım:

equalsOrnek.cs

```
Ders d1 = new Ders { Kod = "BLM101", Ad = "Programlama I" };
Ders d2 = new Ders { Kod = "BLM101", Ad = "Programlamaya Giriş" };
Ders d3 = new Ders { Kod = "MAT101", Ad = "Matematik I" };

Console.WriteLine(d1);                      // ToString ezilmedi: tür adı
Console.WriteLine(d1 == d2);
Console.WriteLine(d1.Equals(d2));
Console.WriteLine(d1.Equals(d3));
Console.WriteLine(d1.GetHashCode() == d2.GetHashCode());
Console.WriteLine(d1.Equals(null));

class Ders
{
    public string Kod { get; set; } = "";
    public string Ad { get; set; } = "";

    public override bool Equals(object? obj)
        => obj is Ders diger && Kod == diger.Kod;

    public override int GetHashCode() => Kod.GetHashCode();
}
```

Çıktı

```
Ders
False
True
False
True
False
```

- `Console.WriteLine(d1)` `ToString`'i ezmediğimiz için yalnız sınıf adını yazar. `d1 == d2` sınıflarda referansları karşılaştırır: iki ayrı nesne, `False`. `Equals` ise bizim yazdığımız kuralı çalıştırır.
- `obj is Ders diger` hem tür sorusu hem dönüşümdür (az önceki `is` deseni). `null` hiçbir türden olmadığı için `Equals(null)` `False` döner; başka türden nesne de `False` döner.
- Doğru imza `public override bool Equals(object? obj)` biçimindedir. `Equals(Ders d)` ezme değil aşırı yüklemedir.
- **Kural:** İki nesne `Equals`'a göre eşitse `GetHashCode` değerleri de aynı olmalıdır (tersi gerekmez). `HashSet` ve `Dictionary` gibi yapılar (Bölüm 13) nesneleri önce `GetHashCode`'a göre gruplar. Bu yüzden `Equals`'ta kullandığın alanı `GetHashCode`'ta da kullanırsın. İkisinden yalnız birini ezersen derleyici CS0659 uyarısı verir; görmezden gelme. Hash değerinin kendisi her çalıştırmada değişebilir; bu yüzden yalnız iki değerin eşit olup olmadığını yazdırdık, çıktı bu sayede sabit.

`record`'lar (Bölüm 9'daki kutu) bu ikisini içeriğe göre derleyiciye yazdırır; sınıflarda elle yazarsın.

10.15 is-a ve has-a: kalıtım mı, kompozisyon mu?

İki sınıf arasındaki her ilişki kalıtım değildir. **is-a** (*bir … dır*): "Araba **bir** taşıttır" → kalıtım, `class Araba : Tasit`. **has-a** (*… sahiptir*): "Arabanın **bir** motoru **vardır**" → **kompozisyon** (composition): `Araba` sınıfında `Motor` tipinde bir **alan** bulunur. `class Araba : Motor` teknik olarak derlenir ama tasarım olarak yanlıştır: Araba bir motor değildir ve motorun bütün metotlarını dışarıya açmak istemezsin.

kompozisyon.cs

```
Araba a = new Araba { Marka = "Togg" };
a.YolaCik();

class Motor
{
    public int Guc { get; set; }
    public void Calistir() => Console.WriteLine($"Motor çalıştı: {Guc} BG");
}

class Tasit
{
    public string Marka { get; set; } = "";
}

class Araba : Tasit                          // is-a: Araba bir Taşıttır
{
    private Motor _motor = new Motor { Guc = 218 };     // has-a: bir Motoru var

    public void YolaCik()
    {
        _motor.Calistir();                   // işi motora devreder
        Console.WriteLine($"{Marka} yola çıktı.");
    }
}
```

Çıktı

```
Motor çalıştı: 218 BG
Togg yola çıktı.
```

	is-a (kalıtım)	has-a (kompozisyon)
C#'ta	`class B : A`	Sınıf tipinde alan
Cümle testi	"X bir Y'dir"	"X'in bir Y'si vardır"
Örnek	Kedi – Hayvan, Kare – Şekil	Araba – Motor, Okul – Öğrenci
Bağ	Sıkı: Taban sınıftaki değişiklik alt sınıfları etkiler	Gevşek: Parça kolayca değiştirilebilir

> **Sınavda Çıkar: Kalıtım mı, kompozisyon mu?**
>
> Cümle testini uygula. Kalıtım her zaman cazip görünür ama "has-a" ilişkisini `:` ile kurmak klasik bir tasarım hatasıdır. Profesyonellerin kuralı: **"Kalıtım yerine kompozisyonu tercih et"**; kalıtımı yalnız gerçek bir "is-a" ilişkisi varsa kullan.

10.16 Sınıf hiyerarşisi örneği

Bölümdeki her şeyi bir arada görelim: `Calisan` taban sınıfı ve iki alt sınıf (`Muhendis`, `Yonetici`). Her alt sınıf `: base(...)` ile değer gönderiyor, `MaasHesapla()`'yı eziyor ve taban sınıfın hesabını `base.` ile kullanıyor. Taban sınıftaki `ToString`, `GetType().Name` sayesinde her çalışan için kendi sınıf adını yazıyor:

calisanHiyerarsisi.cs

```
Calisan c = new Calisan("Ayşe", 40000m);
Muhendis m = new Muhendis("Mehmet", 50000m, 3);
Yonetici y = new Yonetici("Burak", 60000m, 5);

Console.WriteLine(c);
```

```
Console.WriteLine(m);
Console.WriteLine(y);
decimal toplam = c.MaasHesapla() + m.MaasHesapla() + y.MaasHesapla();
Console.WriteLine($"Toplam: {toplam:N2} TL");

class Calisan
{
    private decimal _temelMaas;
    public string Ad { get; }

    public Calisan(string ad, decimal temelMaas)
    {
        Ad = ad;
        _temelMaas = temelMaas;
    }

    public virtual decimal MaasHesapla() => _temelMaas;

    public override string ToString()
        => $"{GetType().Name} {Ad}: {MaasHesapla():N2} TL";
}

class Muhendis : Calisan
{
    private int _projeSayisi;

    public Muhendis(string ad, decimal maas, int projeSayisi) : base(ad, maas)
    {
        _projeSayisi = projeSayisi;
    }

    public override decimal MaasHesapla()
        => base.MaasHesapla() + _projeSayisi * 2500m;       // proje başına
}

class Yonetici : Calisan
{
    private int _ekipSayisi;

    public Yonetici(string ad, decimal maas, int ekipSayisi) : base(ad, maas)
    {
        _ekipSayisi = ekipSayisi;
    }

    public override decimal MaasHesapla()
        => base.MaasHesapla() + _ekipSayisi * 1000m;        // kişi başına

    public override string ToString()
        => base.ToString() + $" (ekip: {_ekipSayisi} kişi)";
}
```

Çıktı

```
Calisan Ayşe: 40.000,00 TL
Muhendis Mehmet: 57.500,00 TL
Yonetici Burak: 65.000,00 TL (ekip: 5 kişi)
Toplam: 162.500,00 TL
```

`Console.WriteLine(m)` çağrısında `Calisan.ToString` çalışır (`Muhendis` ezmedi); içindeki `MaasHesapla()` ise nesne `Muhendis` olduğu için `Muhendis`'inkidir. `Yonetici.ToString` ise `base.ToString()` ile `Calisan`'ınkini alıp üstüne ekler.

`_temelMaas` `private` kaldı; alt sınıflar ona hiç dokunmadı, taban sınıfın `MaasHesapla()` metodunu `base.` ile kullandılar. Kapsülleme ile kalıtım böyle birlikte çalışır.

> **Not: Bir sonraki adım**
>
> Üç çalışanı tek bir dizide (`Calisan[] ekip = { c, m, y };`) tutup bir döngüyle hepsinin `MaasHesapla()`'sını çağırmak, yukarı dönüşümün ve `virtual`/`override` çiftinin asıl faydasıdır. Bunu, `abstract` sınıflar ve arayüzlerle birlikte Bölüm 11'de göreceksin.

Bölüm Özeti – Bilmen Gerekenler

- Kalıtım: `class Alt : Taban`. Alt sınıf taban sınıfın üyelerini devralır ve kendi üyelerini ekler. İlişki **is-a**'dır. Sınıflarda **tek** taban sınıf olur (`class C : A, B` → CS1721); çoklu tür için arayüzler (Bölüm 11).
- `private` alanlar alt sınıf nesnesinde vardır ama alt sınıfın kodu onlara **adıyla erişemez**. `protected`: kendi sınıfı + alt sınıflar, alt sınıfta yalnız alt sınıf tipli nesneyle (aksi CS1540). **Kurucular kalıtılmaz.**
- Alt sınıf kurucusu taban kurucuyu `: base(...)` ile çağırır. Yazılmazsa örtük `: base()` eklenir; taban sınıfta parametresiz kurucu yoksa **derleme hatası** (CS7036).
- Kurucular alttan yukarı çağrılır, gövdeleri **yukarıdan aşağı** çalışır: `object` → `Canli` → `Hayvan` → `Kopek`.
- **Ezme:** taban `virtual`, alt `override`; imza, dönüş tipi ve erişim aynı. `override` yazmayı unutursan **gizleme** olur (CS0108/CS0114 uyarısı); bilerek gizlemek için `new`.
- `virtual`/`override` çiftinde **nesnenin gerçek sınıfı** kazanır (taban sınıfın kendi içinden yapılan çağrılarda bile). `new` ve sıradan metotlarda **değişkenin tipi** kazanır.
- `base.Metot()` taban sınıfın versiyonunu çağırır; `base.` unutulursa metot kendini çağırır (`StackOverflow`). `sealed` sınıf türetilemez, `sealed override` metot tekrar ezilemez.
- **Yukarı dönüşüm** otomatiktir. **Aşağı dönüşüm** önce sorulur: `k is Ogrenci o`, `k as Ogrenci` (uymazsa `null`); cast `(Ogrenci)k` uymazsa `InvalidCastException` fırlatır.
- Her sınıf `object`'ten türer: `ToString()` → tür adı, `Equals` → referans eşitliği, `GetType().Name` → gerçek sınıf adı. `Equals` ezilirse `GetHashCode` da ezilir.
- **has-a** ilişkisi kalıtımla değil **kompozisyonla** (alan olarak) kurulur: Araba–Motor.

Sorular

A) Çoktan Seçmeli

1 C#'ta kalıtım ile ilgili aşağıdakilerden hangisi **doğrudur**?

(A) Bir sınıf `:` ile birden fazla sınıftan türeyebilir (`class C : A, B`).

(B) Alt sınıf, taban sınıfın `private` alanlarına adıyla doğrudan erişebilir.

(C) Taban sınıfın kurucuları alt sınıfa kalıtılır.

(D) `sealed` olarak tanımlanmış bir sınıftan başka bir sınıf türetilemez.

(E) Ezilen bir metodun erişimi `public`'ten `protected`'a daraltılabilir.

2 Aşağıdaki `B` sınıfında işaretli üç metot için hangisi doğrudur?

```
class A
{
    public virtual void Yaz(int x) => Console.WriteLine("A: " + x);
}

class B : A
{
    public override void Yaz(int y) => Console.WriteLine("B: " + y);   // (I)
    public void Yaz(string s) => Console.WriteLine("B: " + s);         // (II)
    public int Yaz(int x, int y) => x + y;                             // (III)
}
```

(A) (I) ezmedir (overriding); (II) ve (III) aşırı yüklemedir (overloading).

(B) Üçü de ezmedir.

(C) (I) aşırı yüklemedir; (II) ve (III) ezmedir.

(D) (III) derleme hatası verir, çünkü dönüş tipi `void` değildir.

(E) (I) derleme hatası verir, çünkü parametre adı taban sınıftakinden farklıdır.

3 Aşağıdaki program için hangisi doğrudur?

```
Otobus o = new Otobus(45);
Console.WriteLine(o.Koltuk);

class Arac
{
    public string Plaka { get; }
    public Arac(string plaka) => Plaka = plaka;
}

class Otobus : Arac
{
    public int Koltuk { get; }
    public Otobus(int koltuk) => Koltuk = koltuk;
}
```

(A) Derlenir ve `45` yazdırır; `Plaka` özelliği `null` kalır.

(B) Derlenmez; `Otobus` kurucusuna derleyicinin eklediği `: base()` çağrısı için `Arac`'ta parametresiz kurucu yoktur.

(C) Derlenir, ama çalışırken `NullReferenceException` oluşur.

(D) Derlenmez; çünkü `Otobus` sınıfında `Plaka` özelliği tanımlanmamıştır.

(E) Derlenmez; çünkü alt sınıfın kurucusunda özellik atanamaz.

4 Aşağıdaki programda numaralı satırlardan hangileri **derleme hatası** verir?

```
Vadeli v = new Vadeli();
Console.WriteLine(v._bakiye);              // (4)

class Hesap
{
    protected decimal _bakiye = 100m;
}

class Vadeli : Hesap
{
    public void Kontrol(Vadeli v, Hesap h)
    {
        Console.WriteLine(_bakiye);            // (1)
        Console.WriteLine(v._bakiye);          // (2)
        Console.WriteLine(h._bakiye);          // (3)
    }
}
```

A) Hiçbiri; hepsi derlenir.

B) Yalnız (4).

C) Yalnız (3).

D) (2), (3) ve (4).

E) (3) ve (4).

5 `Ogrenci`, `Kisi`'den türeyen bir sınıftır (`No` özelliği vardır). Aşağıdaki program için hangisi doğrudur?

```
Kisi k = new Kisi { Ad = "Veli" };
Ogrenci? a = k as Ogrenci;                 // (1)
bool b = k is Ogrenci;                     // (2)
Console.WriteLine($"{a == null} {b}");
Ogrenci c = (Ogrenci)k;                    // (3)
Console.WriteLine(c.No);
```

A) (1) `InvalidCastException` fırlatır; (2) `true` verir.

B) (1) `null` verir; (2) `true` verir; (3) başarılı olur.

C) (1) `null` verir; (2) `false` verir; (3) `InvalidCastException` fırlatır.

D) (1) `null` verir; (2) `false` verir; (3) `null` verir.

E) Üç satır da derleme hatası verir.

B) Çıktı Tahmini

6 Aşağıdaki program ekrana ne yazdırır?

```
new Taksi();
Console.WriteLine("---");
new Otomobil();

class Arac
{
    public Arac() => Console.WriteLine("Arac()");
    public Arac(string tur) : this() => Console.WriteLine($"Arac({tur})");
}

class Otomobil : Arac
{
    public Otomobil() : base("oto") => Console.WriteLine("Otomobil()");
    public Otomobil(int kapi) => Console.WriteLine($"Otomobil({kapi})");
}

class Taksi : Otomobil
{
    public Taksi() : this(4) => Console.WriteLine("Taksi()");
    public Taksi(int kapi) : base(kapi) => Console.WriteLine($"Taksi({kapi})");
}
```

7 Aşağıdaki program ekrana ne yazdırır?

```
Tasit t = new Tasit();
Bisiklet b = new Bisiklet();
Tasit tb = b;

Console.WriteLine(t.Bilgi());
Console.WriteLine(b.Bilgi());
Console.WriteLine(tb.Tur());
Console.WriteLine(tb.Tekerlek());
Console.WriteLine(b.Tekerlek() + tb.Tekerlek());

class Tasit
{
    public virtual string Tur() => "Taşıt";
    public int Tekerlek() => 4;
    public string Bilgi() => Tur() + "/" + Tekerlek();
}

class Bisiklet : Tasit
{
    public override string Tur() => "Bisiklet";
    public new int Tekerlek() => 2;
}
```

8 Aşağıdaki program ekrana ne yazdırır?

```
Hayvan h = new Kedi { Ad = "Pamuk" };
Console.WriteLine(h);
Console.WriteLine(h.GetType().Name);
Console.WriteLine(h is Kedi);
Console.WriteLine(h is Kopek);
Kopek? d = h as Kopek;
Console.WriteLine(d == null);
if (h is Kedi k && k.Ad.Length > 4)
{
    Console.WriteLine(k.Mirla());
}

Hayvan g = new Hayvan { Ad = "Boncuk" };
Console.WriteLine(g);
Console.WriteLine(g is Kedi);

class Hayvan
{
    public string Ad { get; set; } = "";
    public override string ToString() => $"Hayvan[{Ad}]";
}

class Kedi : Hayvan
{
    public string Mirla() => Ad + " mırlıyor";
    public override string ToString() => "Kedi:" + base.ToString();
}

class Kopek : Hayvan { }
```

C) Kod Yazma

9 Bir mağaza için ürün sınıflarını yaz:

- `Urun`: `private` alanlar `_ad` (`string`) ve `_fiyat` (`decimal`); kurucu; `Fiyat` salt okunur özelliği; KDV'li fiyatı döndüren `virtual decimal KdvliFiyat()` (KDV %20); `ToString` → `Kalem - 24,00 TL` (KDV'li fiyat, 2 basamak).
- `Kitap : Urun`: ek `Yazar` özelliği. Kitaplarda KDV %10 olduğu için `KdvliFiyat()`'ı ezer. `ToString`, taban sınıfın metnine `(yazar: ...)` ekler.
- `Elektronik : Urun`: ek `GarantiYil` (`int`) özelliği. `ToString`, taban sınıfın metnine `(garanti: ... yıl)` ekler.

`Kitap` ve `Elektronik` için yalnız `(ad, fiyat)` alan bir kurucu yaz; ek özellikleri object initializer ile ver. Ana kodda 20 TL'lik kalem, 150 TL'lik "Sefiller" (Victor Hugo) ve 1200 TL'lik, 2 yıl garantili kulaklık oluştur; hepsini ve toplam KDV'li tutarı yazdır (ondalık ayracı virgül):

```
Kalem - 24,00 TL
Sefiller - 165,00 TL (yazar: Victor Hugo)
Kulaklık - 1440,00 TL (garanti: 2 yıl)
Toplam: 1629,00 TL
```

10 Bir sınav hiyerarşisi yaz (kurucu yazma; nesneleri object initializer ile oluştur):

- `Sinav`: `Ders` (`string`) ve `Puan` (`int`) özellikleri; `virtual double Etki()` (varsayılan olarak puanın kendisi); `ToString` → `Programlama: 70`.
- `Vize : Sinav`: `Etki()` puanın **%40**'ını döndürür.
- `Final : Sinav`: ek `Butunleme` (`bool`) özelliği; `Etki()` puanın **%60**'ını döndürür; `ToString`, bütünleme ise taban sınıfın metnine `(bütünleme)` ekler.

Ana kodda `Sinav` tipli iki değişken kullan: bir vize (Programlama, 70) ve bütünleme olan bir final (Programlama, 80). Değişkenler `Sinav` tipinde olduğu için `Butunleme`'ye `is` deseniyle ulaş; bir de `as` ile finalin `Vize` olup olmadığını sına. Beklenen çıktı:

```
Programlama: 70
Programlama: 80 (bütünleme)
Vize katkısı : 28,00
Final katkısı: 48,00
Ders notu    : 76,00
Bu final bir bütünleme sınavıdır.
f bir Vize değil
```

Çözümler

Cevap anahtarı

1 D	2 A	3 B	4 E	5 C

1 Cevap: D

`sealed` bir sınıf türetilemez (`string` buna örnektir). A yanlış: Sınıflarda tekli kalıtım vardır (CS1721). B yanlış: `private` alanlara alt sınıftan adıyla erişilemez. C yanlış: Kurucular kalıtılmaz, `: base(...)` ile çağrılır. E yanlış: Ezerken erişim belirteci değiştirilemez (CS0507).

2 Cevap: A

(I), taban sınıftaki `virtual Yaz(int)` ile **aynı imzaya** sahiptir ve `override` ile yazılmıştır; parametre adının `x` yerine `y` olması önemsizdir → ezme. (II) ve (III)'ün parametre listeleri farklıdır → aşırı yükleme; aşırı yüklemede dönüş tipi serbesttir. Program hatasız derlenir. Sınıfları denemek için ana koda `B b = new B(); b.Yaz(1); b.Yaz("x"); Console.WriteLine(b.Yaz(2, 3));` yazarsan çıktı şöyle olur (soruda bu satırlar yoktu; yalnız doğrulama içindir):

Çıktı

```
B: 1
B: x
5
```

3 Cevap: B

`Otobus` kurucusunda `: base(...)` yazılmadığı için derleyici `: base()` ekler. `Arac`'ın tek kurucusu `string` parametre istediğinden bu çağrının karşılığı yoktur (CS7036):

Derleyici çıktısı – program derlenmez

```
test3.cs(13,12): error CS7036: There is no argument given that corresponds to the required parameter
'plaka' of 'Arac.Arac(string)'
```

Düzeltmek için `Otobus(int koltuk) : base("34 ABC 01")` gibi bir çağrı yazılmalı ya da `Arac`'a parametresiz kurucu eklenmelidir. A ve C yanlış: Program hiç derlenmez. D yanlış: `Plaka` `Arac`'tan gelir. E yanlış: Kurucuda özellik atamak serbesttir.

4 Cevap: E

`Kontrol` metodunda (1) kendi nesnemiz, (2) `Vadeli` tipli bir değişken üzerinden erişimdir; ikisi de serbesttir. (3)'te erişim `Hesap` tipli `h` üzerinden yapılıyor; alt sınıf protected üyeye yalnız **kendi tipinden** nesneyle erişebilir (CS1540). (4) sınıfın dışından erişimdir (CS0122):

Derleyici çıktısı – program derlenmez

```
test4.cs(2,21): error CS0122: 'Hesap._bakiye' is inaccessible due to its protection level
test4.cs(15,29): error CS1540: Cannot access protected member 'Hesap._bakiye' via a qualifier of
type 'Hesap'; the qualifier must be of type 'Vadeli' (or derived from it)
```

Hata satırlarına bak: yalnız (3) ve (4) hatalı. A, B, C ve D bu ikiliyi yanlış verir.

5 Cevap: C

Nesne gerçekte bir `Kisi`'dir, `Ogrenci` değil. (1) `as` uymayınca `null` döndürür, hata fırlatmaz. (2) `is` `false` verir. (3) cast uymayınca `InvalidCastException` fırlatır. Konsolda ilk satır `True False`, ardından yakalanmamış istisna görünür:

Çıktı – program yakalanmamış istisna ile sonlanır

```
True False
Unhandled exception. System.InvalidCastException: Unable to cast object of type 'Kisi' to type
'Ogrenci'.
   at Program.<Main>$(String[] args) in test5.cs:line 6
```

A yanlış: İstisnayı `as` değil cast fırlatır. B yanlış: Nesne `Ogrenci` değil. D yanlış: Cast `null` döndürmez. E yanlış: Üç satır da derlenir.

6

Çağrı ağacını çizelim:

```
new Taksi()
 └ Taksi()          ilk iş: this(4)
     └ Taksi(4)        ilk iş: base(4)
         └ Otomobil(4)    : base() yazılmamış → örtük base()
             └ Arac()        → "Arac()"
             → "Otomobil(4)"
         → "Taksi(4)"
     → "Taksi()"

new Otomobil()
 └ Otomobil()       ilk iş: base("oto")
     └ Arac("oto")     ilk iş: this()
         └ Arac()         → "Arac()"
         → "Arac(oto)"
     → "Otomobil()"
```

Tuzak: `Otomobil(int)` kurucusu `: base(...)` yazmadığı için `Arac(string)` değil **parametresiz** `Arac()` çağrılır.

Çıktı

```
Arac()
Otomobil(4)
Taksi(4)
Taksi()
---
Arac()
Arac(oto)
Otomobil()
```

7

- `t.Bilgi()` → `Taşıt/4`.
- `b.Bilgi()`: `Bilgi` metodu `Tasit`'te yazılı. İçindeki `Tur()` çağrısı `virtual`/`override` çifti olduğu için nesneye bakar ve `"Bisiklet"` gelir. `Tekerlek()` ise `new` ile gizlenmiş, sıradan bir metottur; `Tasit` içindeki çağrı derleme zamanında `Tasit.Tekerlek`'e bağlanır → `4`. Sonuç `Bisiklet/4`.
- `tb.Tur()`: değişken `Tasit`, ama `Tur` ezilmiş → `Bisiklet`.
- `tb.Tekerlek()`: değişken `Tasit` ve metot `new` ile gizlenmiş → değişkenin tipine bakılır → `4`.
- `b.Tekerlek() + tb.Tekerlek()` → `2 + 4 = 6`.

```
Çıktı
Taşıt/4
Bisiklet/4
Bisiklet
4
6
```

8

- `Console.WriteLine(h)`: `h` bir `Kedi` nesnesi, `ToString` ezilmiş → `Kedi:` + `base.ToString()` = `Kedi:Hayvan[Pamuk]`.
- `h.GetType().Name` değişkenin değil nesnenin gerçek sınıfını verir → `Kedi`.
- `h is Kedi` → `True`; `h is Kopek` → `False` (kardeş sınıf).
- `h as Kopek` uymadığı için `null`; `d == null` → `True`.
- `h is Kedi k` başarılı ve `"Pamuk".Length` = 5 > 4 → `k.Mirla()` çalışır: `Pamuk mırlıyor`.
- `g` düz bir `Hayvan`: `ToString` taban sınıfınki → `Hayvan[Boncuk]`; `g is Kedi` → `False` (taban nesnesi alt sınıf değildir).

```
Çıktı
Kedi:Hayvan[Pamuk]
Kedi
True
False
True
Pamuk mırlıyor
Hayvan[Boncuk]
False
```

9

`Kitap`, `_fiyat` alanına (`private`) doğrudan erişemediği için `Fiyat` özelliğini kullanır; kurucusu ise yalnız `: base(ad, fiyat)` çağrısından ibarettir (kurucular kalıtılmadığı için yine de yazılmalıdır). `ToString`'i `Urun`'de bir kez yazıp alt sınıflarda `base.ToString()` ile genişletiyoruz. `Urun.ToString` içindeki `KdvliFiyat()` çağrısı, nesne `Kitap` ise **`Kitap`'ın** versiyonuna gider; bu yüzden kitabın fiyatı %10 ile hesaplanır.

urunTest.cs

```
Urun u = new Urun("Kalem", 20m);
Kitap k = new Kitap("Sefiller", 150m) { Yazar = "Victor Hugo" };
Elektronik e = new Elektronik("Kulaklık", 1200m) { GarantiYil = 2 };
Console.WriteLine(u);
Console.WriteLine(k);
Console.WriteLine(e);
decimal toplam = u.KdvliFiyat() + k.KdvliFiyat() + e.KdvliFiyat();
Console.WriteLine($"Toplam: {toplam:F2} TL");

class Urun
{
    private string _ad;
    private decimal _fiyat;

    public Urun(string ad, decimal fiyat)
    {
        _ad = ad;
        _fiyat = fiyat;
    }

    public decimal Fiyat => _fiyat;
```

```
    public virtual decimal KdvliFiyat() => _fiyat * 1.20m;
    public override string ToString() => $"{_ad} — {KdvliFiyat():F2} TL";
}

class Kitap : Urun
{
    public string Yazar { get; set; } = "";

    public Kitap(string ad, decimal fiyat) : base(ad, fiyat) { }

    public override decimal KdvliFiyat() => Fiyat * 1.10m;
    public override string ToString() => base.ToString() + $" (yazar: {Yazar})";
}

class Elektronik : Urun
{
    public int GarantiYil { get; set; }

    public Elektronik(string ad, decimal fiyat) : base(ad, fiyat) { }

    public override string ToString()
        => base.ToString() + $" (garanti: {GarantiYil} yıl)";
}
```

Çıktı

```
Kalem — 24,00 TL
Sefiller — 165,00 TL (yazar: Victor Hugo)
Kulaklık — 1440,00 TL (garanti: 2 yıl)
Toplam: 1629,00 TL
```

10

`Sinav` değişkenleri taban tipli olduğu için `Butunleme`'ye doğrudan ulaşılamaz; `f is Final fn` hem soruyu sorar hem dönüşümü yapar. `Etki()` `virtual`/`override` çifti olduğundan, değişken `Sinav` tipinde olsa da her nesnenin kendi hesabı çalışır. `Final.ToString` taban sınıfın metnini `base.ToString()` ile alıp gerekirse üzerine ekler.

sinavTest.cs

```
Sinav v = new Vize { Ders = "Programlama", Puan = 70 };
Sinav f = new Final { Ders = "Programlama", Puan = 80, Butunleme = true };
Console.WriteLine(v);
Console.WriteLine(f);
Console.WriteLine($"Vize katkısı : {v.Etki():F2}");
Console.WriteLine($"Final katkısı: {f.Etki():F2}");
Console.WriteLine($"Ders notu    : {v.Etki() + f.Etki():F2}");

if (f is Final fn && fn.Butunleme)
{
    Console.WriteLine("Bu final bir bütünleme sınavıdır.");
}

Vize? vz = f as Vize;
Console.WriteLine(vz == null ? "f bir Vize değil" : "f bir Vize");

class Sinav
{
    public string Ders { get; set; } = "";
    public int Puan { get; set; }

    public virtual double Etki() => Puan;
    public override string ToString() => $"{Ders}: {Puan}";
}
```

```
class Vize : Sinav
{
    public override double Etki() => Puan * 0.40;
}

class Final : Sinav
{
    public bool Butunleme { get; set; }

    public override double Etki() => Puan * 0.60;

    public override string ToString()
        => base.ToString() + (Butunleme ? " (bütünleme)" : "");
}
```

Çıktı

```
Programlama: 70
Programlama: 80 (bütünleme)
Vize katkısı : 28,00
Final katkısı: 48,00
Ders notu    : 76,00
Bu final bir bütünleme sınavıdır.
f bir Vize değil
```

BÖLÜM 11

Polimorfizm, Soyut Sınıflar ve Arayüzler

- **Bu Bölümde Öğreneceklerin**
  - Polimorfizmin (çok biçimlilik) ne olduğunu ve neden nesneye yönelik programlamanın kalbi sayıldığını
  - Üst tip referansıyla (`Sekil`) alt tip nesnesi (`Daire`) tutmayı ve çağrının çalışma zamanında nesneye göre seçilmesini (dinamik bağlama)
  - `virtual`/`override` ile `new` arasındaki farkı: hangi metodun çalışacağını bulmayı
  - Soyut sınıf (abstract class) ve arayüz (interface) yazmayı, ikisinin farkını
  - `IComparable<T>` ile `Array.Sort`'un kendi nesnelerini sıralamasını, `is` ile arayüz sormayı

11.1 Polimorfizm nedir?

Bölüm 10'da `class Kedi : Hayvan` yazarak "her kedi bir hayvandır" (**is-a**) ilişkisini kurduk, `virtual` ve `override` ile alt sınıfın üst sınıftaki metodu **ezmesini** (override) gördük. Bu bölüm o ilişkinin en güçlü sonucunu anlatıyor: Bir `Hayvan` değişkeni bir `Kedi` nesnesini tutabilir ve ona `SesCikar()` dediğinde hayvan **kedi gibi** ses çıkarır.

Polimorfizm (polymorphism, çok biçimlilik): Aynı metot çağrısının, nesnenin **gerçek tipine** göre farklı davranmasıdır. Tek bir `Hayvan` değişkeni bazen kedi, bazen köpek "biçimine" girer.

Bu bölümde çok sayıda kısa metot var. Tek ifadelik metotları `=>` ile (ifade gövdeli üye, expression-bodied member) yazacağız: `public override double Alan() => _en * _boy;` satırı `{ return _en * _boy; }` ile aynı işi yapar. `=>` özelliklerde de kullanılır.

polimorfizm.cs

```
Hayvan h1 = new Kedi("Tekir");
Hayvan h2 = new Kopek("Karabaş");
Hayvan h3 = new Hayvan("Bilinmeyen");

h1.SesCikar();
h2.SesCikar();
h3.SesCikar();
Konustur(new Kedi("Pamuk"));

static void Konustur(Hayvan h)    // tek metot, her türlü hayvanla çalışır
{
    h.SesCikar();
}

class Hayvan
{
    protected string _ad;

    public Hayvan(string ad) { _ad = ad; }

    public virtual void SesCikar() => Console.WriteLine(_ad + ": ...");
```

```
}

class Kedi : Hayvan
{
    public Kedi(string ad) : base(ad) { }

    public override void SesCikar() => Console.WriteLine(_ad + ": Miyav");
}

class Kopek : Hayvan
{
    public Kopek(string ad) : base(ad) { }

    public override void SesCikar() => Console.WriteLine(_ad + ": Hav hav");
}
```

Çıktı

```
Tekir: Miyav
Karabaş: Hav hav
Bilinmeyen: ...
Pamuk: Miyav
```

Üç değişkenin tipi de `Hayvan`, üç satırda da aynı `SesCikar()` çağrısı var; ama üç farklı sonuç çıktı. `Konustur` metodu ise parametre olarak `Hayvan` alıyor ve yarın yazacağın `Kus`, `At` gibi yeni alt sınıflarla da **hiç değiştirilmeden** çalışacak. Polimorfizmin asıl kazancı budur: Kodu üst sınıfa göre yazarsın, her alt sınıf kendi davranışını getirir.

11.2 İki tip: referans tipi ve nesne tipi

`Hayvan h = new Kedi("Tekir");` satırında iki ayrı tip vardır. Bu ayrımı oturtmadan bu bölümdeki hiçbir çıktı sorusunu çözemezsin:

Kavram	Nerede yazar?	Örnekte	Neyi belirler?
Referans tipi (değişkenin tipi)	`=` işaretinin solunda	`Hayvan`	Hangi üyeleri **çağırabileceğini** (derleme zamanı)
Nesne tipi (gerçek tip)	`new`'den sonra	`Kedi`	Sanal bir metodun **hangi sürümünün** çalışacağını (çalışma zamanı)

Alt sınıf nesnesini üst sınıf referansına atamak (yukarı dönüşüm, upcasting) güvenlidir ve cast istemez; Bölüm 10'da gördün. Tersi için cast gerekir ve nesne uymuyorsa `InvalidCastException` alırsın. Bu bölümde asıl soru şu: Çağrı yapıldığında **hangi sürüm** çalışır?

11.3 Dinamik bağlama: hangi metot çalışır?

`h.SesCikar()` gibi bir çağrıda iki ayrı karar verilir:

1. **Derleme zamanında** (compile time): Derleyici sadece **referans tipine** bakar: "`Hayvan` sınıfında `SesCikar()` diye bir üye var mı?" Yoksa program derlenmez.
2. **Çalışma zamanında** (runtime): Metot `virtual`, `abstract` ya da `override` ise çalışma ortamı nesnenin **gerçek tipine** bakar ve o sınıftaki sürümü çalıştırır. O sınıf metodu ezmemişse bir üst sınıfa çıkar, bulana kadar devam eder.

Çağrılacak metodun program çalışırken, nesneye bakılarak seçilmesine **dinamik bağlama** (dynamic binding, geç bağlama) denir. Karar derleme sırasında, referans tipine göre verilirse buna **statik bağlama** (static binding, erken bağlama) denir.

	Statik bağlama	Dinamik bağlama
Karar ne zaman?	Derleme zamanı	Çalışma zamanı
Neye bakılır?	**Referans tipine**	**Nesne tipine**
Hangi üyeler?	Aşırı yüklenmiş (overload) metotlar, `new` ile gizlenenler, `static` üyeler, `virtual` olmayan metotlar, alanlar	`virtual`, `abstract` ve `override` ile yazılan metotlar ve özellikler

Dinamik bağlama, metot **üst sınıfın içinden** çağrıldığında bile geçerlidir. Sınavların en sevdiği tuzak budur:

dinamikBaglama.cs

```
Calisan c1 = new Calisan("Ali");
Calisan c2 = new Mudur("Ayşe");
Calisan c3 = new Stajyer("Can");
c1.Bordro();
c2.Bordro();
c3.Bordro();

class Calisan
{
    protected string _ad;
    protected double _taban = 30000;

    public Calisan(string ad) { _ad = ad; }

    public virtual double Maas() => _taban;
    public virtual string Unvan() => "Çalışan";

    public void Bordro()          // yalnız burada; alt sınıflar ezmiyor
    {
        Console.WriteLine($"{_ad,-5} {Unvan(),-8} {Maas(),9:F2}");
    }
}

class Mudur : Calisan
{
    public Mudur(string ad) : base(ad) { }

    public override double Maas() => base.Maas() * 2;
    public override string Unvan() => "Müdür";
}

class Stajyer : Calisan
{
    public Stajyer(string ad) : base(ad) { }

    public override double Maas() => _taban / 3;
}
```

Çıktı

```
Ali   Çalışan   30000,00
Ayşe  Müdür     60000,00
Can   Çalışan   10000,00
```

`Bordro()` metodu sadece `Calisan` sınıfında var. Ama `c2.Bordro()` çalışırken içindeki `Unvan()` ve `Maas()` çağrıları aslında `this.Unvan()` ve `this.Maas()` demektir; `this` ise bir `Mudur` nesnesidir. Bu yüzden müdürün sürümleri çalışır. `Stajyer` sadece `Maas()`'ı ezdiği için unvanı `Calisan`'dan gelir. `base.Maas()` ise dinamik bağlamayı atlar ve **doğrudan üst sınıfın** sürümünü çağırır. Ondalık ayracın virgül çıkması Bölüm 2'de anlattığımız Türkçe kültür ayarındandır.

> **Sınavda Çıkar: İki soru, iki tip**
>
> Polimorfizm sorularında her satır için iki soru sor:
>
> - **"Derlenir mi?"** → **Referans tipine** bak. Üye o sınıfta (veya üstlerinde) yoksa derleme hatası.
> - **"Ne yazdırır?"** → Metot `virtual` / `abstract` ise ve alt sınıf `override` ettiyse **nesne tipine** bak: `new` ile hangi sınıf oluşturulduysa, metot aramaya o sınıftan başlanır. Metot sanal değilse ya da `new` ile gizlenmişse **referans tipine** bak.
>
> Üst sınıftaki bir metodun içinden çağrılan sanal metotlar da nesne tipine göre seçilir. Bunu unutan öğrenci `Çalışan` yazar, doğrusu `Müdür`'dür.

Referans tipinde olmayan üye çağrılamaz

Nesne gerçekten bir kedi olsa bile, değişkenin tipi `Hayvan` ise sadece `Hayvan`'da bulunan üyeleri çağırabilirsin:

eksikMetot.cs

```
Hayvan h = new Kedi();
h.SesCikar();
h.Tirmala();

class Hayvan { public virtual void SesCikar() { } }

class Kedi : Hayvan { public void Tirmala() { } }
```

Derleyici çıktısı – program derlenmez

```
eksikMetot.cs(3,3): error CS1061: 'Hayvan' does not contain a definition for 'Tirmala' and no
accessible extension method 'Tirmala' accepting a first argument of type 'Hayvan' could be found
(are you missing a using directive or an assembly reference?)
```

Derleyici `h`'nin içinde gerçekte ne olduğuna bakmaz (çoğu zaman bilemez de; nesne bir metoda parametre olarak gelmiş olabilir). Sadece `h`'nin tipi olan `Hayvan` sınıfına bakar ve orada `Tirmala` bulamaz. Çözüm Bölüm 10'daki aşağı dönüşümdür: `if (h is Kedi k) { k.Tirmala(); }`.

Aşırı yükleme derleme zamanında seçilir

Override (ezme) ile overload (aşırı yükleme: aynı isimli, farklı parametreli metotlar) sık karıştırılır. Aralarındaki fark burada ortaya çıkar: **Hangi overload'un çağrılacağına derleyici, argümanın referans tipine bakarak karar verir.** Nesne tipine bakılmaz. Aynı adlı metotları bir sınıfın içine koyduk:

overloadTuzak.cs

```
Hayvan h = new Kedi();
Kedi k = new Kedi();
Tanitici.Tanit(h);
Tanitici.Tanit(k);
Tanitici.Tanit((Kedi)h);

class Hayvan { }

class Kedi : Hayvan { }

class Tanitici
{
    public static void Tanit(Hayvan h) => Console.WriteLine("Bir hayvan");
    public static void Tanit(Kedi k) => Console.WriteLine("Bir kedi");
}
```

Çıktı

```
Bir hayvan
Bir kedi
Bir kedi
```

`h` bir kedi nesnesini tuttuğu hâlde ilk satır `Bir hayvan` yazdırdı, çünkü `h`'nin referans tipi `Hayvan`'dır. Özetle: **override → çalışma zamanı, nesne tipi; overload → derleme zamanı, referans tipi.**

`override` mı, `new` mi?

Bölüm 10'da `new` ile gizlemeyi gördün. Şimdi aynı iki üyeyi bir `Ust` referansı üzerinden çağıralım: biri `override` ile ezilmiş, diğeri `new` ile gizlenmiş:

newTuzak.cs

```
Ust u = new Alt();
Alt a = new Alt();
Console.WriteLine(u.Ezilen());
Console.WriteLine(u.Gizlenen());
Console.WriteLine(a.Gizlenen());
Console.WriteLine(((Ust)a).Gizlenen());

class Ust
{
    public virtual string Ezilen() => "Ust.Ezilen";
    public virtual string Gizlenen() => "Ust.Gizlenen";
}

class Alt : Ust
{
    public override string Ezilen() => "Alt.Ezilen";
    public new string Gizlenen() => "Alt.Gizlenen";
}
```

Çıktı

```
Alt.Ezilen
Ust.Gizlenen
Alt.Gizlenen
Ust.Gizlenen
```

`new` yeni bir metot tanımlar; üst sınıftaki metodun yerine geçmez. `u.Gizlenen()` çağrısında derleyici `u`'nun tipine (`Ust`) bakar ve `Ust`'ın sürümünü seçer. Son satırdaki `(Ust)a` cast'i nesneyi değiştirmez, sadece referans tipini değiştirir; bu yüzden `Ust`'ın sürümü çalışır.

	`virtual` + `override`	`new` (gizleme)
Üst sınıfta	`virtual` (ya da `abstract`) şart	`virtual` olması gerekmez
Referans `Ust`, nesne `Alt`	`Alt`'ın sürümü çalışır	`Ust`'ın sürümü çalışır
Referans `Alt`, nesne `Alt`	`Alt`'ın sürümü çalışır	`Alt`'ın sürümü çalışır
Ne için?	Polimorfizm	Genelde yanlışlıkla olur; kaçın

Alanlar (fields) ve `static` üyeler hiçbir zaman sanal olmaz; onlar da her zaman referans tipine göre seçilir. Polimorfizm istiyorsan `virtual`/`override` kullan.

11.4 Polimorfik diziler

Polimorfizmin en çok işe yaradığı yer dizilerdir. Bir `Sekil[]` dizisinin her elemanı daire, dikdörtgen veya kare olabilir; döngü hangisinin ne olduğunu bilmeden hepsinin alanını hesaplar:

polimorfikDizi.cs

```
Sekil[] sekiller =
{
    new Daire(1), new Dikdortgen(3, 4), new Kare(2), new Daire(2.5)
};
double toplam = 0;
foreach (Sekil s in sekiller)
{
    Console.WriteLine($"{s.Ad(),-11} {s.Alan(),7:F2}");
    toplam += s.Alan();
}
Console.WriteLine($"Toplam alan: {toplam:F2}");

class Sekil
{
    public virtual double Alan() => 0;
    public virtual string Ad() => "Şekil";
}

class Daire : Sekil
{
    private double _r;

    public Daire(double r) { _r = r; }

    public override double Alan() => Math.PI * _r * _r;
    public override string Ad() => "Daire";
}

class Dikdortgen : Sekil
{
    private double _en, _boy;

    public Dikdortgen(double en, double boy) { _en = en; _boy = boy; }

    public override double Alan() => _en * _boy;
    public override string Ad() => "Dikdörtgen";
}

class Kare : Dikdortgen
```

```
{
    public Kare(double kenar) : base(kenar, kenar) { }

    public override string Ad() => "Kare";
}
```

Çıktı

```
Daire          3,14
Dikdörtgen    12,00
Kare           4,00
Daire         19,63
Toplam alan: 38,78
```

`Kare` alanı hesaplamayı `Dikdortgen`'den miras aldı, sadece adını değiştirdi. Programa yarın `Ucgen` sınıfı eklersen `foreach` döngüsünde **tek bir harf** değiştirmen gerekmez.

Yalnız bu tasarımda bir tuhaflık var: `Sekil` sınıfının `Alan()` metodu `0` döndürüyor. "Şekil" soyut bir kavramdır; alanı sıfır değil, **tanımsızdır**. Ayrıca kimse `new Sekil()` yazıp anlamsız bir nesne oluşturmamalı. İşte bu ihtiyaç, soyut sınıfları doğurur.

11.5 Soyut sınıflar (abstract class)

Soyut sınıf (abstract class), kendisinden nesne üretilmeyen, sadece **kalıtılmak için** var olan sınıftır. İçinde **soyut üyeler** (abstract member) bulunabilir: Sadece imzası yazılan, gövdesi olmayan metotlar ve özellikler. Soyut metot, "her alt sınıf bu metodu **yazmak zorunda**" demektir. `Sekil`'i soyut yapıp bir de `Cevre()` metodu ekleyelim (`Dikdortgen` sınıfı `Daire` ile aynı kalıpta yazıldığı için gösterilmedi):

```
Sekil[] sekiller = { new Daire(1.5), new Dikdortgen(2, 5) };
foreach (Sekil s in sekiller)
{
    s.Bilgi();
}

// ...

abstract class Sekil
{
    public string Ad { get; }

    protected Sekil(string ad)             // soyut sınıfın da kurucusu olur
    {
        Ad = ad;
    }

    public abstract double Alan();         // gövde yok, sonunda ; var
    public abstract double Cevre();

    public void Bilgi()                    // somut (gövdeli) metot
    {
        Console.WriteLine($"{Ad,-10} alan={Alan(),6:F2}  çevre={Cevre(),6:F2}");
    }
}

class Daire : Sekil
{
    private double _r;

    public Daire(double r) : base("Daire") { _r = r; }
```

```
    public override double Alan() => Math.PI * _r * _r;
    public override double Cevre() => 2 * Math.PI * _r;
}
```

Çıktı

```
Daire      alan=  7,07  çevre=  9,42
Dikdörtgen alan= 10,00  çevre= 14,00
```

Soyut sınıfın kuralları:

- `abstract` üyenin gövdesi olmaz: `public abstract double Alan();` (süslü parantez yok, noktalı virgül var). Soyut üye zaten sanaldır; ayrıca `virtual` yazılmaz.
- İçinde **en az bir** soyut üye olan sınıf `abstract` olmak **zorundadır**. (Tersi gerekmez: hiç soyut üyesi olmayan bir sınıf da `abstract` yapılabilir.)
- Soyut sınıfın **alanları, kurucusu ve gövdeli metotları** olabilir. Kurucu, alt sınıfların `: base(...)` çağrısıyla çalışır. Nesne üretilemeyeceği için kurucusunu genelde `protected` yazarız.
- Soyut olmayan bir alt sınıf, üst sınıftaki **bütün** soyut üyeleri `override` ile yazmak zorundadır. Yazmayacaksa kendisi de `abstract` olmalıdır.
- Özellik de soyut olabilir: `public abstract string Tur { get; }`. Alt sınıf `public override string Tur => "Daire";` diye yazar.
- Soyut sınıf türünde **referans** ve **dizi** tanımlanabilir (`Sekil s`, `new Sekil[3]`), ama soyut sınıftan **nesne** üretilemez:

soyutNesne.cs

```
Sekil s = new Sekil();

abstract class Sekil
{
    public abstract double Alan();
}
```

Derleyici çıktısı – program derlenmez

```
soyutNesne.cs(1,11): error CS0144: Cannot create an instance of the abstract type or interface
'Sekil'
```

● Sık Yapılan Hata: Soyut metodu ezmeyi unutmak

Alt sınıf, üst sınıftaki soyut üyelerden birini bile yazmayı unutursa program derlenmez. `Sekil`'in `Alan()` ve `Cevre()` metotları varken `Kare` sadece `Alan()`'ı yazarsa derleyici `Kare`'nin miras alınan soyut üyeyi uygulamadığını söyler (CS0534). Ya `Cevre()`'yi yaz ya da `Kare`'yi `abstract` yap.

```
Sekil s = new Kare();

abstract class Sekil
{
    public abstract double Alan();
    public abstract double Cevre();
}

class Kare : Sekil
{
    private double _a = 2;

    public override double Alan() => _a * _a;
}
```

Derleyici çıktısı – program derlenmez

```
eksikOverride.cs(9,7): error CS0534: 'Kare' does not implement inherited abstract member
'Sekil.Cevre()'
```

● Sınavda Çıkar: Soyut üyeyle birlikte kullanılamayanlar

Soyut üye alt sınıfta **ezilmek** için vardır. Bu yüzden ezilmeyi engelleyen belirteçlerle birlikte yazılamaz: `abstract` üye `private`, `static` veya `sealed` **olamaz**. Aynı mantıkla `abstract sealed class` da yazılamaz: mühürlü sınıf kalıtılamaz, soyut sınıf ise kalıtılmak zorundadır.

11.6 Arayüzler (interface)

Soyut sınıf "**ne olduğunu**" anlatır: Daire bir şekildir. Bazen ise ortak olan şey kimlik değil, **yetenektir**: Ördek uçabilir, uçak da uçabilir; ama uçak bir hayvan değildir. Böyle bir "yapabilir" sözleşmesi için **arayüz** (interface) kullanılır.

Arayüz, bir sınıfın **hangi üyeleri sağlayacağını** söyleyen bir sözleşmedir. Adı geleneksel olarak `I` **harfiyle** başlar (`ISekil`, `IUcabilir`). Bir sınıf arayüzü sınıf adından sonra `:` ile yazarak uygular ve içindeki bütün üyeleri yazmak zorundadır. Bir sınıf sadece **bir** sınıftan türeyebilir ama **istediği kadar** arayüzü uygulayabilir; yazarken önce sınıf, sonra arayüzler gelir:

arayuzOrnek.cs

```
IUcabilir[] ucanlar = { new Ordek("Donald"), new Ucak("TC-JAN") };
foreach (IUcabilir u in ucanlar)
{
    u.Uc();                          // polimorfizm: nesnenin sürümü çalışır
    if (u is IYuzebilir yz)          // arayüze de is ile sorulur
    {
        yz.Yuz();
    }
}
IYuzebilir y = new Ordek("Vakvak"); // ördek aynı zamanda IYuzebilir
y.Yuz();
Console.WriteLine(ucanlar[1] is IYuzebilir);
Console.WriteLine(ucanlar[0] is Hayvan);
```

```
interface IUcabilir { void Uc(); }     // üyeler otomatik olarak public

interface IYuzebilir { void Yuz(); }

class Hayvan
{
    protected string _ad;

    public Hayvan(string ad) { _ad = ad; }
}

class Ordek : Hayvan, IUcabilir, IYuzebilir   // önce sınıf, sonra arayüzler
{
    public Ordek(string ad) : base(ad) { }

    public void Uc() => Console.WriteLine(_ad + " kanat çırparak uçuyor");
    public void Yuz() => Console.WriteLine(_ad + " gölde yüzüyor");
}

class Ucak : IUcabilir          // hayvan değil, ama uçabilir
{
    private string _kod;

    public Ucak(string kod) { _kod = kod; }

    public void Uc() => Console.WriteLine(_kod + " 10.000 metrede uçuyor");
}
```

Çıktı

```
Donald kanat çırparak uçuyor
Donald gölde yüzüyor
TC-JAN 10.000 metrede uçuyor
Vakvak gölde yüzüyor
False
True
```

Arayüz de bir **tiptir**: `IUcabilir` türünde bir değişken veya dizi tanımlayabilir, içine onu uygulayan her sınıfın nesnesini koyabilirsin. Ama arayüzden de nesne üretilemez: `new IUcabilir()` derlenmez (CS0144, soyut sınıftakiyle aynı hata). Polimorfizm burada da geçerlidir: `u.Uc()` çağrısında çalışan sürüm, nesnenin sınıfındaki uygulamadır.

Üç noktaya dikkat:

- Arayüzü uygularken `override` **yazılmaz**; metodu doğrudan `public void Uc()` diye yazarsın.
- Kontrol için `is` kullanılır: `u is IYuzebilir yz` hem "bu nesne `IYuzebilir` uyguluyor mu?" diye sorar hem de doğruysa `yz` değişkenini hazır verir. Çıktıdaki sondan bir önceki satır (`ucanlar[1] is IYuzebilir`) `Ucak` yüzemediği için `False`, son satır ise ördek bir hayvan olduğu için `True` oldu.
- Arayüz tipli bir değişkenden yalnız **arayüzde tanımlı** üyeleri çağırabilirsin. `u.Yuz()` yazamazsın; önce `is` ile sormalısın.

Sık Yapılan Hata: Arayüz üyesini public yazmamak

Arayüzdeki üyeler otomatik olarak `public`'tir. Sınıfta bu metodu yazarken `public`'i unutursan, sınıftaki metot `private` (varsayılan) olur ve arayüzü uygulayamaz:

```
IUcabilir k = new Kus();
k.Uc();

interface IUcabilir { void Uc(); }

class Kus : IUcabilir
{
    void Uc() => Console.WriteLine("Kuş uçuyor");
}
```

Derleyici çıktısı – program derlenmez

```
zayifErisim.cs(6,13): error CS0737: 'Kus' does not implement interface member 'IUcabilir.Uc()'.
'Kus.Uc()' cannot implement an interface member because it is not public.
```

Sık Yapılan Hata: Birden çok sınıftan türemek

Bir sınıfın tek bir üst **sınıfı** olabilir. `class C : A, B` (iki sınıf) yazarsan CS1721, arayüzü sınıftan önce yazıp `class D : IX, A` dersen CS1722 ile derlenmez.

Çözüm: `class C : A, IX` biçiminde, **önce tek sınıf, sonra istediğin kadar arayüz**.

Özellik içeren arayüz

Arayüz yalnız metot değil, **özellik** de (property) isteyebilir. Arayüzdeki `string Ad { get; }` bir alan değildir, bir sözleşmedir: "Bu sınıf okunabilir bir `Ad` özelliği sağlayacak." Nasıl saklayacağı sınıfa kalmıştır. Arayüzde nesne alanı (`private int _x;` gibi) ve kurucu **bulunamaz**.

arayuzOzellik.cs

```
ISekil s = new Daire(2);       // arayüz tipinde değişken, nesne Daire
Console.WriteLine($"{s.Ad}: {s.Alan():F2}");

ISekil[] sekiller = { new Daire(0.5), new Kare(3), s };
double toplam = 0;
foreach (ISekil sekil in sekiller)
{
    toplam += sekil.Alan();
}
Console.WriteLine($"Toplam: {toplam:F2}");

interface ISekil
{
    string Ad { get; }             // özellik sözleşmesi
    double Alan();
}

class Daire : ISekil
{
    private double _r;

    public Daire(double r) { _r = r; }

    public string Ad => "Daire";
    public double Alan() => Math.PI * _r * _r;
```

```
}

class Kare : ISekil
{
    private double _a;

    public Kare(double a) { _a = a; }

    public string Ad { get; } = "Kare";
    public double Alan() => _a * _a;
}
```

Çıktı

```
Daire: 12,57
Toplam: 22,35
```

`Daire` özelliği `=>` ile, `Kare` ise otomatik özellik (`{ get; } = "Kare"`) ile sağladı. Arayüz ikisini de kabul eder: sözleşme sadece "okunabilir bir `Ad` var" der.

> **Not: Arayüzde varsayılan metot (C# 8)**
>
> Arayüzün bir metoduna **gövde** de yazılabilir (default interface method), örneğin `string Durum() => Seviye < 20 ? "şarj et!" : "iyi";`. Uygulayan sınıf bu metodu yazmazsa gövde kullanılır. Yalnız şuna dikkat: Bu metot sınıfın üyesi olmaz; yalnız **arayüz tipli** değişkenle çağrılabilir (`ISarjli t = new Telefon(600); t.Durum();`). `Telefon t` tipindeki bir değişkenle `t.Durum()` yazarsan CS1061 ile derlenmez. Bu bölümün sınav konusu değildir, tanıman yeter.

Arayüz referansı ve `new` tuzağı

Arayüz üyesinin hangi metoda bağlanacağı, arayüzü **uygulayan sınıf** tarafından belirlenir. Alt sınıf aynı adlı bir metodu `new` ile yazarsa arayüz bundan haberdar olmaz:

arayuzGizleme.cs

```
IMesaj m = new Alt();
Ust u = new Alt();
Alt a = new Alt();
Console.WriteLine(m.Yaz() + " " + u.Yaz() + " " + a.Yaz());

interface IMesaj { string Yaz(); }

class Ust : IMesaj { public string Yaz() => "Ust"; }

class Alt : Ust { public new string Yaz() => "Alt"; }
```

Çıktı

```
Ust Ust Alt
```

`IMesaj`'ı `Ust` uyguladı; `Alt` arayüzü yeniden listelemedi ve `new` ile yazdığı `Yaz()` ayrı bir üyedir. Bu yüzden arayüz referansı da `Ust` referansı da `Ust`'ın metoduna gider; yalnız `Alt` referansı `Alt`'ınkini görür. Hepsinin `Alt` yazmasını istiyorsan `Ust.Yaz()`'ı `virtual`, `Alt.Yaz()`'ı `override` yap.

11.7 Soyut sınıf mı, arayüz mü?

Bu karşılaştırma, vize ve finallerin klasik "açıklayınız" sorusudur:

Özellik	Soyut sınıf	Arayüz
Tanım / kullanım	`abstract class A` / `class B : A`	`interface IA` / `class B : IA`
Kaç tane?	Bir sınıf sadece **bir** sınıftan türeyebilir	Bir sınıf **birden çok** arayüz uygulayabilir
Nesne üretilebilir mi?	Hayır	Hayır
Kurucu (constructor)	Olabilir	**Olamaz**
Alanlar	Her türlü (nesne alanı, `private`, `protected`...)	Nesne alanı **yok**; özellik sözleşmesi olabilir
Üyeler	Soyut + gövdeli, her erişim belirteci	Soyut (gövdeli üye yalnız ikincil bir özellik)
Üyelerin erişimi	Sen belirlersin	Varsayılan olarak `public`
Uygulama	Soyut üye `override` ile yazılır	`override` yazılmaz, `public` yazılır
İlişki	"**is-a**" (bir ...dır): Daire bir şekildir	"**can-do**" (...yapabilir): Ördek uçabilir
Ne zaman?	Ortak **durum** (alan) ve kod paylaşan yakın akraba sınıflar	İlgisiz sınıflara ortak bir **yetenek** kazandırmak

> ● **Sınavda Çıkar: Neden sınıflarda çoklu kalıtım yok?**
>
> C#'ta `class C : A, B` (iki sınıf) yazılamaz. Sebep "**elmas problemi**" (diamond problem): `A` ve `B`'de aynı imzalı iki farklı metot ve aynı isimli alanlar olsaydı, `C` hangisini miras alacaktı? Arayüzlerde **durum** (nesne alanı) olmadığı için çoklu uygulama güvenlidir. Sınavda "`class Ordek : Hayvan, IUcabilir, IYuzebilir` geçerli mi?" diye sorulursa cevap: Evet, bir sınıf + iki arayüz.

11.8 `IComparable<T>`: kendi nesnelerini sıralamak

.NET'in kendi kütüphanesi de arayüzlerle doludur. En meşhurlarından biri `IComparable<T>`'dir: "Bu sınıfın nesneleri birbiriyle **karşılaştırılabilir**" sözleşmesi. Bölüm 7'de `Array.Sort` ile sayı dizilerini sıraladın; kendi sınıfının nesnelerini sıralamak istiyorsan, sınıf `IComparable<T>` arayüzünü uygulamalı ve `CompareTo` metodunu yazmalıdır:

comparableOrnek.cs

```
Ogrenci[] sinif =
{
    new Ogrenci("Elif", 85), new Ogrenci("Can", 62), new Ogrenci("Deniz", 91)
};
Array.Sort(sinif);              // CompareTo'yu kullanarak sıralar
Console.WriteLine(string.Join(", ", sinif));
Console.WriteLine(sinif[0].CompareTo(sinif[2]));
Console.WriteLine(sinif[2].CompareTo(sinif[0]));
Console.WriteLine(sinif[1].CompareTo(sinif[1]));

class Ogrenci : IComparable<Ogrenci>
{
    public string Ad { get; }
    public int Puan { get; }

    public Ogrenci(string ad, int puan)
    {
        Ad = ad;
        Puan = puan;
    }
```

```
    public int CompareTo(Ogrenci? diger)
    {
        if (diger is null)
        {
            return 1;                  // null her nesneden "küçük" sayılır
        }
        return Puan.CompareTo(diger.Puan);
    }

    public override string ToString() => $"{Ad}({Puan})";
}
```

Çıktı

```
Can(62), Elif(85), Deniz(91)
-1
1
0
```

`a.CompareTo(b)` sözleşmesi: `a` önce gelmeliyse **negatif**, eşitse **0**, sonra gelmeliyse **pozitif** bir sayı döndür. Biz işi `int`'in kendi `CompareTo` metoduna bıraktık: Can'ın 62'si Deniz'in 91'inden küçük olduğu için sonuç negatif çıktı. Küçükten büyüğe sıralamak için `Puan.CompareTo(diger.Puan)`, büyükten küçüğe sıralamak için `diger.Puan.CompareTo(Puan)` yazılır.

`Ogrenci?` yazımı, `null` gelebileceğini söyleyen nullable işaretidir; arayüzün imzası böyle olduğu için uyarı almamak adına aynen yazdık. `IComparable<Ogrenci>` yazımındaki `<...>` **generics**'tir; ayrıntısını ve farklı sıralamaları Bölüm 13'te göreceksin. `string`'in `CompareTo` metodu da aslında bu arayüzden gelir. Sınıf `IComparable<T>` uygulamıyorsa `Array.Sort` onu sıralayamaz ve çalışırken `InvalidOperationException` fırlatır (istisnaları Bölüm 12'de göreceksin).

> **İpucu: is zinciri bir uyarı işaretidir**
>
> Kodunda `if (x is A) ... else if (x is B) ... else if (x is C)` gibi uzun zincirler varsa, polimorfizmi kullanmıyorsun demektir. Ortak davranışı üst sınıfa (ya da bir arayüze) bir metot olarak koy, her alt sınıf kendi sürümünü yazsın; zincir tek bir `x.Metot()` çağrısına dönüşür. `is`'i sadece **alt sınıfa özgü** bir şey yapman gerektiğinde kullan.

11.9 `object`'ten gelen sanal metotlar

Bölüm 10'da her sınıfın `object`'ten türediğini gördün. `ToString`, `Equals` ve `GetHashCode` orada `virtual` olarak tanımlıdır; yani en yaygın polimorfizm aslında budur: `Console.WriteLine(nesne)` çağrısı nesnenin kendi `ToString()` sürümünü çalıştırır.

objeVirtual.cs

```
object[] nesneler = { new Nokta(3, 4), new Nokta(1, 2), "metin", 42 };
foreach (object o in nesneler)
{
    Console.WriteLine(o);          // her biri kendi ToString'ini çalıştırır
}
Nokta a = new Nokta(3, 4);
Nokta b = new Nokta(3, 4);
Console.WriteLine(a == b);         // aynı nesne mi?
Console.WriteLine(a.Equals(b));    // ezilmediği için bu da referansa bakar

class Nokta
{
    public int X { get; }
```

```
    public int Y { get; }

    public Nokta(int x, int y)
    {
        X = x;
        Y = y;
    }

    public override string ToString() => $"({X}, {Y})";
}
```

Çıktı

```
(3, 4)
(1, 2)
metin
42
False
False
```

`Equals`'ı ezmezsen o da `==` gibi **aynı nesne mi** diye bakar; içeriği eşit iki ayrı nesne `False` verir. Değerlere göre karşılaştırmak için `Equals` ezilir ve `GetHashCode` da birlikte ezilmelidir (yoksa derleyici uyarı verir, CS0659). `Nokta` sınıfına şunları eklersen `a.Equals(b)` `True` olur; `a == b` ise yine `False` kalır:

```
public override bool Equals(object? obj)
    => obj is Nokta p && p.X == X && p.Y == Y;

public override int GetHashCode() => HashCode.Combine(X, Y);
```

> **Not: Operatör aşırı yükleme (isteğe bağlı)**
>
> C#'ta `+`, `==`, `<` gibi operatörleri kendi sınıfın için de tanımlayabilirsin (operator overloading). Örneğin `Nokta` için `public static Nokta operator +(Nokta a, Nokta b)` yazarsan `new Nokta(1, 2) + new Nokta(3, 4)` ifadesi geçerli olur. `==` tanımlarsan `!=`'i, `Equals`'ı ve `GetHashCode`'u da birlikte tanımlaman gerekir. Bu kitabın sınav soruları için şart değildir.

11.10 Tam örnek: ödeme sistemi

Şimdiye kadar öğrendiklerini bir arada kullanalım. `Odeme` soyut bir sınıftır: ortak alanları ve `Ozet()` metodu var, komisyonu ise her ödeme türü kendisi hesaplar. `IIadeEdilebilir` arayüzü ise yalnız iade edilebilen ödemelere verilen bir yetenektir: Nakit ödeme iade edilemez, o yüzden `Nakit` arayüzü uygulamıyor.

odemeSistemi.cs

```
Odeme[] odemeler =
{
    new KrediKarti("Ayşe", 1200, 3),
    new Havale("Burak", 800),
    new KrediKarti("Ceren", 450, 1),
    new Nakit("Deniz", 150)
};
double toplamKomisyon = 0;
foreach (Odeme o in odemeler)
{
    o.Ozet();
    toplamKomisyon += o.Komisyon();
    if (o is IIadeEdilebilir iade)       // yalnız yeteneği olanlar
    {
        iade.IadeEt();
```

```
    }
}
Console.WriteLine($"Toplam komisyon: {toplamKomisyon:F2} TL");

interface IIadeEdilebilir
{
    void IadeEt();
}

abstract class Odeme
{
    public string Sahip { get; }
    public double Tutar { get; }

    protected Odeme(string sahip, double tutar)
    {
        Sahip = sahip;
        Tutar = tutar;
    }

    public abstract string Tur { get; }
    public abstract double Komisyon();

    public void Ozet() => Console.WriteLine(
        $"{Sahip,-6} {Tur,-11} {Tutar,8:F2}  komisyon {Komisyon(),6:F2}");
}

class KrediKarti : Odeme, IIadeEdilebilir
{
    private int _taksit;

    public KrediKarti(string sahip, double tutar, int taksit)
        : base(sahip, tutar) { _taksit = taksit; }

    public override string Tur => "Kredi kartı";
    public override double Komisyon() => Tutar * (0.015 + 0.005 * _taksit);
    public void IadeEt() => Console.WriteLine($"  -> {Sahip}: karta iade");
}

class Havale : Odeme, IIadeEdilebilir
{
    public Havale(string sahip, double tutar) : base(sahip, tutar) { }

    public override string Tur => "Havale";
    public override double Komisyon() => 5;
    public void IadeEt() => Console.WriteLine($"  -> {Sahip}: hesaba iade");
}

class Nakit : Odeme
{
    public Nakit(string sahip, double tutar) : base(sahip, tutar) { }

    public override string Tur => "Nakit";
    public override double Komisyon() => 0;
}
```

Çıktı

```
Ayşe   Kredi kartı  1200,00  komisyon  36,00
  -> Ayşe: karta iade
Burak  Havale        800,00  komisyon   5,00
  -> Burak: hesaba iade
Ceren  Kredi kartı   450,00  komisyon   9,00
  -> Ceren: karta iade
Deniz  Nakit         150,00  komisyon   0,00
Toplam komisyon: 50,00 TL
```

Programı baştan sonra izle: Döngü `Odeme` tipinde çalışıyor ve `Tur`, `Komisyon()` her nesnede kendi sürümüne gidiyor (soyut sınıf + dinamik bağlama). `IadeEt()` yalnız `KrediKarti` ve `Havale` için çalıştı, çünkü `o is IIadeEdilebilir iade` sorusunun cevabı `Nakit` için `false`. Yarın `Cek` adında yeni bir ödeme türü eklemek, döngüye dokunmadan yeni bir sınıf yazmak demektir.

Bölüm Özeti – Bilmen Gerekenler

- **Polimorfizm:** Aynı çağrı, nesnenin gerçek tipine göre farklı davranır. `Hayvan h = new Kedi();` → referans tipi `Hayvan`, nesne tipi `Kedi`.
- **Derleyici referans tipine bakar** (üye var mı?), **çalışma ortamı nesne tipine bakar** (hangi sürüm çalışır?). Üst sınıf metodunun içinden yapılan sanal çağrılar da nesne tipine göre seçilir.
- **Override** çalışma zamanında nesne tipine göre, **overload** ve **`new` ile gizleme** derleme zamanında referans tipine göre seçilir. Alanlar ve `static` üyeler de referans tipine göredir.
- `x is T t`: `x` bir `T` (veya alt tipi) ise `true` ve `t` hazır; `null` için hep `false`. Arayüz tipine de sorulabilir.
- **Soyut sınıf:** `abstract`, nesnesi üretilemez (CS0144), kurucusu ve alanları olabilir. Soyut üyenin gövdesi yoktur; somut alt sınıf hepsini `override` etmek zorundadır (CS0534).
- **Arayüz:** `interface IAd`, bir sınıf birden çok arayüz uygulayabilir ama tek sınıftan türeyebilir (`class C : Sinif, IA, IB`). Üyeler varsayılan `public`'tir; sınıfta `public` yazılır, `override` yazılmaz. Kurucu ve nesne alanı yoktur; özellik sözleşmesi olabilir.
- Arayüz referansı, arayüzü uygulayan sınıfın metoduna gider; alt sınıftaki `new` metot görünmez.
- Soyut sınıf "is-a" ve ortak durum, arayüz "can-do" ve ortak yetenek içindir.
- **`IComparable<T>`:** `CompareTo` negatif/0/pozitif döndürür; `Array.Sort` kendi nesnelerini bununla sıralar.
- `ToString` / `Equals` / `GetHashCode` `object`'ten gelen sanal metotlardır; `Equals` ezilirse `GetHashCode` da ezilir.

Sorular

A) Çoktan Seçmeli

1 `abstract class Sekil { public abstract double Alan(); }` tanımlı olsun ve bu sınıfı doğru biçimde uygulayan, kurucusu `Daire(double r)` olan `class Daire : Sekil` bulunsun. Aşağıdaki satırlardan hangisi **derlenmez**?

(A) `Sekil s = new Daire(2);`
(B) `Sekil[] dizi = new Sekil[3];`
(C) `Sekil s = new Sekil();`
(D) `object o = new Daire(2);`
(E) `Sekil? s = null;`

2 `A` ve `B` birer **sınıf**, `IX` ve `IY` birer **arayüz** olsun. Aşağıdaki sınıf başlıklarından hangisi **derlenir**?

(A) `class C : A, B`
(B) `class C : IX, A`
(C) `class C : IX, IY, A`
(D) `class C : A, IX, IY`
(E) `class C : A, B, IX`

3 Aşağıdaki kod parçası ne yazdırır?

```
interface IKonus { string Konus(); }
class Ana : IKonus { public string Konus() => "Ana"; }
class Cocuk : Ana { public new string Konus() => "Cocuk"; }
// ana kodda:
IKonus k = new Cocuk();
Ana a = new Cocuk();
Cocuk c = new Cocuk();
Console.WriteLine(k.Konus() + a.Konus() + c.Konus());
```

(A) `AnaAnaCocuk`
(B) `CocukCocukCocuk`
(C) `AnaCocukCocuk`
(D) `CocukAnaCocuk`
(E) Derleme hatası verir, çünkü `Cocuk`, `Konus()` metodunu `override` etmeden yazmıştır.

4 Arayüzlerle ilgili aşağıdakilerden hangisi **yanlıştır**?

(A) Arayüz türünde değişken veya dizi tanımlanabilir.
(B) Arayüzden `new` ile doğrudan nesne üretilemez.
(C) Bir sınıf birden çok arayüz uygulayabilir.
(D) Arayüz üyeleri `public` olduğundan, sınıftaki uygulamaları da `public` yazılır.
(E) Arayüzde kurucu (constructor) tanımlanabilir.

5 Aşağıdaki sınıf ve kod parçası verilmiştir:

```
class Ogrenci : IComparable<Ogrenci>
{
    public int Puan { get; }

    public Ogrenci(int puan) { Puan = puan; }

    public int CompareTo(Ogrenci? diger)
    {
        return diger is null ? 1 : diger.Puan.CompareTo(Puan);
    }
}
// ana kodda:
Ogrenci[] d = { new Ogrenci(70), new Ogrenci(95), new Ogrenci(40) };
Array.Sort(d);
Console.WriteLine(d[0].Puan + " " + d[2].Puan);
```

Çıktı ne olur?

A) `40 95`

B) `95 40`

C) `70 40`

D) `70 95`

E) Derleme hatası verir, çünkü sınıf `Equals` metodunu ezmemiştir.

B) Çıktı Tahmini

6 Aşağıdaki program ekrana ne yazdırır?

```
Arac[] dizi = { new Arac(), new Motor(), new Kamyon() };
foreach (Arac a in dizi)
{
    a.Yaz();
}
Arac x = dizi[2];
Console.WriteLine(x is Motor);
Console.WriteLine(x.Ad().Length + x.Teker());

class Arac
{
    public virtual string Ad() => "Araç";
    public virtual int Teker() => 4;
    public virtual void Yaz() => Console.WriteLine(Ad() + " " + Teker());
}

class Motor : Arac
{
    public override string Ad() => "Motor";
    public override int Teker() => 2;
}

class Kamyon : Arac
{
    public override int Teker() => base.Teker() + 2;

    public override void Yaz()
    {
        Console.Write("[Yük] ");
        base.Yaz();
    }
}
```

7 Aşağıdaki program ekrana ne yazdırır? (Override, `new` ve overload'a dikkat!)

```
Hayvan a = new Kedi();
Kedi b = new Kedi();
Hayvan c = new Hayvan();
Console.WriteLine(Doktor.Sor(a));
Console.WriteLine(Doktor.Sor(b));
Console.WriteLine(Doktor.Sor(c));
Console.WriteLine(Doktor.Sor((Kedi)a));
Console.WriteLine(Doktor.Sor((Hayvan)b));

class Hayvan
{
    public virtual string Ses() => "...";
    public string Tur() => "Hayvan";
}

class Kedi : Hayvan
{
    public override string Ses() => "Miyav";
    public new string Tur() => "Kedi";
}

class Doktor
{
    public static string Sor(Hayvan h) => "H:" + h.Ses() + "/" + h.Tur();
    public static string Sor(Kedi k) => "K:" + k.Ses() + "/" + k.Tur();
}
```

8 Aşağıdaki program ekrana ne yazdırır?

```
ISelam[] liste = { new Ogrenci("Ece"), new Hoca("Kaya"), new Ogrenci("Mert") };
foreach (ISelam s in liste)
{
    Console.WriteLine(s.Selam());
}
Console.WriteLine(Kisi.Sayac);

interface ISelam { string Selam(); }

abstract class Kisi : ISelam
{
    public static int Sayac = 0;
    protected string _isim;

    protected Kisi(string isim) { _isim = isim; Sayac++; }

    public abstract string Ad();
    public virtual string Selam() => "Merhaba " + Ad();
}

class Ogrenci : Kisi
{
    public Ogrenci(string isim) : base(isim) { }

    public override string Ad() => _isim;
    public override string Selam() => "Selam, ben " + Ad();
}

class Hoca : Kisi
{
    public Hoca(string isim) : base(isim) { }

    public override string Ad() => "Dr. " + _isim;
}
```

C) Kod Yazma

9 Bir şirketin bordrosunu hesaplayan programı yaz:

- Soyut `Calisan` sınıfı: `Ad` özelliği, kurucu, soyut `double Maas()` ve soyut `string Tur()` metotları.
- `TamZamanli`: Aylık sabit maaş alır.
- `SaatUcretli`: Maaş = çalışılan saat × saatlik ücret. Ancak **160 saatin üzerindeki** her saat için ücretin **1,5 katı** ödenir.
- Ana kod: Şu dört çalışanı bir `Calisan[]` dizisine koysun: Ayşe (tam zamanlı, 45000), Burak (150 saat, saati 200 TL), Ceren (180 saat, saati 200 TL), Deniz (tam zamanlı, 38500). Her çalışanı ve toplam bordroyu yazdırsın (ad 6, tür 14, maaş 9 karakter genişliğinde, maaş iki ondalıklı):

```
Ayşe   Tam zamanlı     45000,00
Burak  Saat ücretli    30000,00
Ceren  Saat ücretli    38000,00
Deniz  Tam zamanlı     38500,00
Toplam bordro: 151500,00 TL
```

10 `Urun` sınıfını yaz ve ürünleri fiyata göre sıralayan programı tamamla:

- `Ad` (string) ve `Fiyat` (double) özellikleri olsun; sınıf `IComparable<Urun>` uygulasın ve küçük fiyat önce gelsin.
- `ToString()` ezilsin: `Ad — Fiyat TL` biçiminde (fiyat iki ondalıklı).
- Ana kod: Kalem 12, Çanta 310, Silgi 8,25, Defter 45,5 ürünlerini bir diziye koysun, `Array.Sort` ile sıralasın, her ürünü yazdırsın, en sonda en ucuz ve en pahalı ürünün adını versin:

```
Silgi — 8,25 TL
Kalem — 12,00 TL
Defter — 45,50 TL
Çanta — 310,00 TL
En ucuz: Silgi, en pahalı: Çanta
```

Çözümler

Cevap anahtarı

1 C	2 D	3 A	4 E	5 B

1 Cevap: C

Soyut sınıftan `new` ile nesne üretilemez; `new Sekil()` derleme hatası verir (CS0144). A ve D yukarı dönüşümdür, geçerlidir. E geçerlidir: soyut sınıf türünde (boş olabilen) bir referans tanımlamak serbesttir. B tuzaktır: `new Sekil[3]` **nesne** değil, `Sekil` referansları taşıyacak bir **dizi** üretir; elemanları başlangıçta `null`'dır ve derlenir.

2 Cevap: D

Önce tek sınıf, sonra arayüzler gelir. D (`A, IX, IY`) tam olarak bu kalıptır. A ve E iki sınıftan türemeye çalışır (CS1721). B ve C arayüzleri sınıftan önce yazar; sınıf, arayüzlerden önce gelmelidir (CS1722).

3 Cevap: A

`IKonus` arayüzünü `Ana` uyguladı. `Cocuk` arayüzü yeniden listelemediği için `IKonus` eşlemesi `Ana.Konus()`'ta kalır; `Cocuk`'un `new` ile yazdığı metot ayrı bir üyedir. `k.Konus()` → `Ana`. `a.Konus()`: referans tipi `Ana`, metot sanal değil → `Ana`. `c.Konus()`: referans tipi `Cocuk` → `Cocuk`. Sonuç `AnaAnaCocuk`. B, `new`'ü `override` sanan; C ve D, arayüz çağrısını nesne tipine göre çözmeye çalışan öğrencinin cevabıdır. E yanlıştır: `new` yazılmıştır; yazılmasaydı bile bu bir uyarı olurdu, hata değil.

4 Cevap: E

Arayüzde kurucu bulunamaz; çünkü arayüzden nesne üretilmez ve arayüzde nesne alanı yoktur, ilk değer verilecek bir şey de yok. A, B ve C doğrudur. D de doğrudur: Arayüz üyeleri varsayılan olarak `public` olduğu için, sınıfta `public` yazmazsan program derlenmez (CS0737).

5 Cevap: B

`CompareTo` içinde karşılaştırma **ters** yazılmış: `diger.Puan.CompareTo(Puan)`. Yani `Puan` büyük olan nesne "önce gelir" sayılır. `Array.Sort` bu sözleşmeyle büyükten küçüğe sıralar: `95, 70, 40`. `d[0].Puan` → 95, `d[2].Puan` → 40; `int + string` birleştirmesi `95 40` verir. A, `CompareTo`'nun doğal (artan) halini sanan öğrencinin cevabıdır. C ve D hiç sıralanmadığını sanar. E yanlıştır: `IComparable<T>` için `Equals` ezmek zorunlu değildir.

6

- `new Arac()`: Kendi metotları → `Araç 4`.
- `new Motor()`: `Yaz()` `Arac`'tan gelir ama içindeki `Ad()` ve `Teker()` `Motor`'unkilerdir → `Motor 2`.
- `new Kamyon()`: Kendi `Yaz()`'ı önce `[Yük]` yazar, sonra `base.Yaz()` ile `Arac`'ın `Yaz()`'ı çalışır. Orada `Ad()` ezilmediği için `Araç`; `Teker()` ise `Kamyon`'unki: `base.Teker() + 2 = 6` → `[Yük] Araç 6`.
- `x` bir `Kamyon`'dur, `Motor` değildir → `False` (C#'ta `bool` değeri büyük harfle yazılır).
- `"Araç".Length` = 4, `x.Teker()` = 6 → iki `int` toplanır: `10`.

```
Çıktı
Araç 4
Motor 2
[Yük] Araç 6
False
10
```

7

Her satırda üç karar var: **Hangi `Sor`?** (overload, derleme zamanı, argümanın **referans tipi**), **hangi `Ses`?** (`override`, çalışma zamanı, **nesne tipi**) ve **hangi `Tur`?** (`new` ile gizlenmiş, derleme zamanı, **referans tipi**; `Sor` içinde `h` her zaman `Hayvan` tipindedir).

Çağrı	`Sor`	`Ses` (nesne tipi)	`Tur` (`Sor` içindeki parametre tipi)	Sonuç
`Sor(a)`	`Hayvan` → `H:`	`Kedi` → `Miyav`	`Hayvan` → `Hayvan`	`H:Miyav/Hayvan`
`Sor(b)`	`Kedi` → `K:`	`Kedi` → `Miyav`	`Kedi` → `Kedi`	`K:Miyav/Kedi`
`Sor(c)`	`Hayvan` → `H:`	`Hayvan` → `...`	`Hayvan` → `Hayvan`	`H:.../Hayvan`
`Sor((Kedi)a)`	`Kedi` → `K:`	`Kedi` → `Miyav`	`Kedi` → `Kedi`	`K:Miyav/Kedi`
`Sor((Hayvan)b)`	`Hayvan` → `H:`	`Kedi` → `Miyav`	`Hayvan` → `Hayvan`	`H:Miyav/Hayvan`

Son satır öğreticidir: `(Hayvan)b` cast'i nesneyi değiştirmez, sadece referans tipini değiştirir. Bu yüzden `H:` seçilir, `Ses` hâlâ `Miyav`'dır ama `Tur` `Hayvan`'a döner.

```
Çıktı
H:Miyav/Hayvan
K:Miyav/Kedi
H:.../Hayvan
K:Miyav/Kedi
H:Miyav/Hayvan
```

8

- `Ogrenci("Ece")`: `Selam()`'ı ezmiş → `Selam, ben Ece`.
- `Hoca("Kaya")`: `Selam()`'ı ezmemiş, `Kisi`'deki sanal sürüm çalışır. Arayüz referansından çağrılsa bile bu sürüm seçilir, çünkü `Kisi` arayüzü bu metotla uyguluyor. İçindeki `Ad()` çağrısı dinamik bağlanır ve `Hoca`'nın `Ad()`'ı çalışır → `Merhaba Dr. Kaya`.
- `Ogrenci("Mert")` → `Selam, ben Mert`.
- Her nesne oluşturulurken `Kisi` kurucusu çalıştı ve `static` sayaç 3 kez arttı. `static` alan bütün nesnelerce **ortak** olduğu için sonuç `3`.

```
Çıktı
Selam, ben Ece
Merhaba Dr. Kaya
Selam, ben Mert
3
```

9

`Calisan` sınıfı `Maas()` ve `Tur()`'ü soyut bırakır; ortak yazdırma metodu `Satir()` bu iki metodu çağırır ve dinamik bağlama doğru sürümü seçer. Fazla mesai için 160 saati aşan kısım ayrıca hesaplanır: Ceren için `160 × 200 + 20 × 200 × 1,5 = 32000 + 6000 = 38000`.

bordro.cs

```
Calisan[] ekip =
{
    new TamZamanli("Ayşe", 45000),
    new SaatUcretli("Burak", 150, 200),
    new SaatUcretli("Ceren", 180, 200),
    new TamZamanli("Deniz", 38500)
};
double toplam = 0;
foreach (Calisan c in ekip)
{
    Console.WriteLine(c.Satir());
    toplam += c.Maas();
}
Console.WriteLine($"Toplam bordro: {toplam:F2} TL");

abstract class Calisan
{
    public string Ad { get; }

    protected Calisan(string ad) { Ad = ad; }

    public abstract double Maas();
    public abstract string Tur();

    public string Satir() => $"{Ad,-6} {Tur(),-14} {Maas(),9:F2}";
}

class TamZamanli : Calisan
{
    private double _aylik;

    public TamZamanli(string ad, double aylik) : base(ad) { _aylik = aylik; }

    public override double Maas() => _aylik;
    public override string Tur() => "Tam zamanlı";
}

class SaatUcretli : Calisan
{
    private int _saat;
    private double _ucret;

    public SaatUcretli(string ad, int saat, double ucret) : base(ad)
    {
        _saat = saat;
        _ucret = ucret;
    }

    public override double Maas()
    {
        if (_saat <= 160) { return _saat * _ucret; }
        return 160 * _ucret + (_saat - 160) * _ucret * 1.5;
    }

    public override string Tur() => "Saat ücretli";
}
```

Çıktı

```
Ayşe   Tam zamanlı     45000,00
Burak  Saat ücretli    30000,00
Ceren  Saat ücretli    38000,00
Deniz  Tam zamanlı     38500,00
Toplam bordro: 151500,00 TL
```

10

`Urun`, `IComparable<Urun>` uygular ve `CompareTo`'da fiyatları `double`'ın kendi `CompareTo` metoduyla karşılaştırır: küçük fiyat negatif sonuç verir, yani önce gelir. `Console.WriteLine(u)` çağrısı `ToString()`'in ezilmiş sürümünü kullanır. Sıralanmış dizide en ucuz ürün ilk, en pahalı son elemandır.

urunSirala.cs

```
Urun[] urunler =
{
    new Urun("Kalem", 12), new Urun("Çanta", 310),
    new Urun("Silgi", 8.25), new Urun("Defter", 45.5)
};
Array.Sort(urunler);
foreach (Urun u in urunler)
{
    Console.WriteLine(u);
}
Urun enUcuz = urunler[0];
Urun enPahali = urunler[urunler.Length - 1];
Console.WriteLine($"En ucuz: {enUcuz.Ad}, en pahalı: {enPahali.Ad}");

class Urun : IComparable<Urun>
{
    public string Ad { get; }
    public double Fiyat { get; }

    public Urun(string ad, double fiyat)
    {
        Ad = ad;
        Fiyat = fiyat;
    }

    public int CompareTo(Urun? diger)
    {
        if (diger is null)
        {
            return 1;
        }
        return Fiyat.CompareTo(diger.Fiyat);
    }

    public override string ToString() => $"{Ad} - {Fiyat:F2} TL";
}
```

Çıktı

```
Silgi - 8,25 TL
Kalem - 12,00 TL
Defter - 45,50 TL
Çanta - 310,00 TL
En ucuz: Silgi, en pahalı: Çanta
```

BÖLÜM 12

İstisna Yönetimi

Bu Bölümde Öğreneceklerin

- İstisnanın (exception) ne olduğunu ve programın **çökmesi** ile hatanın **ele alınması** arasındaki farkı
- `try`, `catch`, `finally` yazmayı; birden çok `catch`'i doğru sırayla dizmeyi
- İstisna hiyerarşisini ve en sık karşılaşılan istisna türlerini
- `throw` ile istisna fırlatmayı, `throw;` ile `throw ex;` farkını, `when` süzgecini
- Kendi istisna sınıfını yazmayı
- `using` ile kaynakları otomatik bırakmayı (`IDisposable`)
- `TryParse` ile istisna arasında doğru seçimi yapmayı

12.1 İstisna nedir?

Önceki bölümlerde sıfıra bölünce ya da harf içeren metni sayıya çevirmeye çalışınca programın çöktüğünü gördün. **İstisna** (exception), program çalışırken oluşan ve normal akışı bozan olaydır: olmayan dizi elemanına erişmek, sayı beklenirken harf okumak, `null` bir değişkenin üyesini kullanmak... C#'ta her istisna bir **nesnedir**; içinde hatanın türü, mesajı ve nerede oluştuğu bilgisi bulunur.

İstisna ele alınmazsa program durur ve ekrana bir hata raporu basılır. Aşağıdaki not listesinde yanlışlıkla bir metin var:

cokme.cs

```
string[] notlar = { "85", "70", "doksan", "60" };
Console.WriteLine($"Ortalama: {Hesap.Ortalama(notlar)}");

class Hesap
{
    public static int Ortalama(string[] notlar)
    {
        int toplam = 0;
        foreach (string s in notlar)
        {
            toplam += int.Parse(s);
        }
        return toplam / notlar.Length;
    }
}
```

Çıktı – program yakalanmamış istisna ile sonlanır

```
Unhandled exception. System.FormatException: The input string 'doksan' was not in a correct format.
   at Hesap.Ortalama(String[] notlar) in cokme.cs:line 11
   at Program.<Main>$(String[] args) in cokme.cs:line 2
```

Raporu yukarıdan aşağıya oku:

- **İlk satır:** istisnanın **türü** (`System.FormatException`) ve **mesajı**. Mesajlar İngilizcedir; ezberlemene gerek yok, tanıman yeterli.
- **`at` satırları:** hatanın oluştuğu andaki çağrı zinciri. En üstteki satır hatanın fırlatıldığı yerdir; her alt satır, bir üstündeki metodu **çağıran** metottur.
- Kendi ekranında bunların arasında `at System.` ile başlayan satırlar da görürsün: onlar .NET'in kendi kodudur ve sayıları Windows'ta, Linux'ta farklıdır; bu kitaptaki çıktılarda yer kazanmak için gösterilmez. **Kendi dosya adının geçtiği ilk satırı** ara: `in cokme.cs:line 11` hatanın senin kodundaki yeridir. Windows'ta dosya adının önünde tam klasör yolu da görünür (`C:\Projeler\Notlar\cokme.cs:line 11` gibi).
- `Program.<Main>$`, üst düzey deyimlerin içinde çalıştığı gizli metodun adıdır.

Tek bir hatalı veri yüzünden ortalama hiç yazılmadı. Oysa hatayı **yakalayıp** (catch) programı sürdürebiliriz:

yakalama.cs

```
string[] notlar = { "85", "70", "doksan", "60" };
int toplam = 0;
int adet = 0;
foreach (string s in notlar)
{
    try
    {
        toplam += int.Parse(s);
        adet++;
    }
    catch (FormatException)
    {
        Console.WriteLine($"Geçersiz not atlandı: {s}");
    }
}
Console.WriteLine($"Ortalama: {toplam / adet}");
```

Çıktı

```
Geçersiz not atlandı: doksan
Ortalama: 71
```

`(85 + 70 + 60) / 3 = 71`. `doksan` için `int.Parse` istisna fırlatınca `try`'ın geri kalanı (`adet++`) atlandı, `catch` çalıştı, döngü sonraki nota geçti. İstisna yönetiminin amacı budur: **beklenmedik durumda programı çökertmek yerine kontrollü tepki vermek.**

12.2 İstisna hiyerarşisi

İstisnalar da sınıftır ve Bölüm 10'daki kalıtım ağacına göre düzenlenmiştir. Hepsinin atası `Exception`'dır; çalışma zamanının fırlattıklarının çoğu onun alt sınıfı `SystemException`'dan türer. En sık karşılaşacakların:

İstisna	Üst sınıfı	Ne zaman oluşur?	Örnek
`FormatException`	`SystemException`	Metin sayıya çevrilemez	`int.Parse("12a")`
`DivideByZeroException`	`ArithmeticException`	Tamsayı sıfıra bölünür	`10 / b` (`b` sıfır)
`OverflowException`	`ArithmeticException`	`checked` taşma, sığmayan sayı metni	`Convert.ToInt32("99999999999")`
`NullReferenceException`	`SystemException`	`null` değişkenin üyesine erişilir	`s.Length` (`s` null)
`IndexOutOfRangeException`	`SystemException`	Dizi sınırı dışına erişilir	3 elemanlı dizide `a[3]`
`InvalidCastException`	`SystemException`	Geçersiz tür dönüşümü (Bölüm 10)	`(Kedi)hayvan` (nesne köpek)
`ArgumentException`	`SystemException`	Metoda geçersiz argüman verilir	Negatif yaş (genelde **sen** fırlatırsın)
`ArgumentNullException`	`ArgumentException`	Argüman `null` olmamalıydı	`null` bir ad
`ArgumentOutOfRangeException`	`ArgumentException`	Argüman geçerli aralığın dışında	`"abc".Substring(5)`
`InvalidOperationException`	`SystemException`	Nesne o işlem için uygun durumda değil	Kapalı hesaptan para çekmek
`KeyNotFoundException`	`SystemException`	Sözlükte olmayan anahtar (Bölüm 13)	`sozluk["yok"]`
`FileNotFoundException`	`IOException`	Olmayan dosya açılır (Bölüm 14)	`File.ReadAllText("yok.txt")`

Zincirleri sondan başa oku: `DivideByZeroException` → `ArithmeticException` → `SystemException` → `Exception`. `FileNotFoundException` → `IOException` → `SystemException` → `Exception`. Bir `catch (X)` bloğu `X` türünü ve onun **bütün alt sınıflarını** yakalar; sıradaki başlıkta bunun sonucunu göreceksin.

12.3 try ve catch

İstisna çıkabilecek kod `try` bloğuna, hata olursa çalışacak kod `catch` bloğuna yazılır:

```
try
{
    // istisna fırlatabilecek kod
}
catch (IstisnaTuru ex)
{
    // o türde istisna olursa çalışacak kod
}
```

Kurallar (`yakalama` programına bak):

1. `try` içinde istisna oluştuğu anda `try`'ın **geri kalanı atlanır** (`adet++` çalışmadı).
2. Çalışma zamanı, istisnanın türüne **uyan ilk** `catch`'i bulur ve çalıştırır.
3. `catch` bitince program `try-catch`'in **altından** devam eder; istisnanın oluştuğu yere geri dönülmez.
4. `try` hatasız biterse `catch` bloklarına hiç uğranmaz.
5. Uyan `catch` yoksa istisna metodu terk edip **çağıran metoda** geçer; ana koddan da çıkarsa program çöker.

Parantezdeki `ex` yakalanan istisna nesnesidir. Nesneye ihtiyacın yoksa adı yazmazsın: `catch (FormatException)`.

> **Sık Yapılan Hata: `try` içinde tanımlanan değişken**
>
> `try { int x = int.Parse(s); } catch (FormatException) { }` yazdıktan sonra `x`'i `try`'ın dışında kullanamazsın; her blok gibi `try` da kendi **kapsamını** (scope) oluşturur. Değişkeni dışarıda kullanacaksan `try`'dan **önce** tanımla: `int x = 0; try { x = ...; }`.

12.4 Birden çok catch

Bir `try` bloğundan farklı türde istisnalar çıkabiliyorsa her biri için ayrı `catch` yazabilirsin. **Uyan ilk** `catch` çalışır, diğerleri atlanır:

cokluCatch.cs

```
string[] girdiler = { "0", "abc", "1", "7", "2" };
foreach (string g in girdiler)
{
    Dene(g);
}

static void Dene(string girdi)
{
    int[] bolenler = { 4, 0, 25 };
    try
    {
        int i = int.Parse(girdi);
        Console.WriteLine($"{girdi} -> {100 / bolenler[i]}");
    }
    catch (FormatException ex)
    {
        Console.WriteLine($"{girdi} -> sayı değil: {ex.Message}");
    }
    catch (DivideByZeroException)
    {
        Console.WriteLine($"{girdi} -> sıfıra bölme");
    }
    catch (IndexOutOfRangeException)
    {
        Console.WriteLine($"{girdi} -> geçersiz indeks");
    }
}
```

Çıktı

```
0 -> 25
abc -> sayı değil: The input string 'abc' was not in a correct format.
1 -> sıfıra bölme
7 -> geçersiz indeks
2 -> 4
```

`ex.Message` hatanın açıklamasıdır. `ex.GetType().Name` türün adını verir. `ex.ToString()` (ya da `Console.WriteLine(ex)`) tür, mesaj ve çağrı zincirini, yani az önce gördüğün raporu verir.

Üst sınıf türündeki `catch` bütün alt sınıfları da yakaladığı için **sıra** önemlidir: **önce özel (alt sınıf), sonra genel (üst sınıf).** Genel olanı önce yazarsan alttakine asla sıra gelmez ve derleyici bunu hata sayar:

catchSirasi.cs

```
try
{
    int b = 0;
    Console.WriteLine(10 / b);
}
catch (Exception)
{
    Console.WriteLine("Genel hata");
}
catch (DivideByZeroException)
{
    Console.WriteLine("Sıfıra bölme");
}
```

Derleyici çıktısı – program derlenmez

```
catchSirasi.cs(10,8): error CS0160: A previous catch clause already catches all exceptions of this
or of a super type ('Exception')
```

> **Sınavda Çıkar: catch sırası kuralı**
>
> "Hangi sıralama derlenmez?" sorusunda her `catch` için şuna bak: **Kendinden önce, kendi üst sınıfını yakalayan bir `catch` var mı?** Varsa derleme hatası (`CS0160`). `catch (Exception)` her zaman **en sona** yazılır. Kardeş türler (`FormatException` ile `NullReferenceException` gibi) her sırada yazılabilir. Dikkat: `FormatException`, `ArgumentException`'ın alt sınıfı **değildir**; ikisi `SystemException`'ın kardeşidir.

12.5 finally: her durumda çalışan blok

`try-catch`'ten sonra isteğe bağlı bir **`finally`** bloğu yazılabilir. `finally` **her durumda** çalışır: istisna olsa da olmasa da, yakalansa da yakalanmasa da, hatta `try` içinde **`return`** olsa bile. Açılan dosyayı kapatmak gibi "ne olursa olsun yapılmalı" işler için kullanılır.

finallyOrnek.cs

```
Console.WriteLine($"Sonuç: {Bol(10, 2)}");
Console.WriteLine($"Sonuç: {Bol(10, 0)}");

static int Bol(int a, int b)
{
    try
    {
        Console.WriteLine($"  try: {a}/{b}");
        return a / b;
    }
    catch (DivideByZeroException ex)
    {
        Console.WriteLine($"  catch: {ex.Message}");
        return -1;
    }
    finally
    {
        Console.WriteLine("  finally çalıştı");
    }
}
```

Çıktı

```
  try: 10/2
  finally çalıştı
Sonuç: 5
  try: 10/0
  catch: Attempted to divide by zero.
  finally çalıştı
Sonuç: -1
```

İlk çağrıda `return a / b;` çalıştı ama metot hemen dönmedi: dönüş değeri (5) bir kenara alındı, **önce `finally` çalıştı**, sonra metot 5'i döndürdü. `Sonuç:` satırının `finally`'den **sonra** çıkması bu yüzden.

> **Sınavda Çıkar: finally soruları**
>
> - `finally`'nin çalışmadığı klasik durum, `try` içinde `Environment.Exit(0)` ile programın kapatılmasıdır. Programın tamamen çöktüğü durumlarda da çalışmaz: `Environment.FailFast` ya da yakalanamayan `StackOverflowException` (Bölüm 15).
> - `try` içinde `return x;` varsa `x`'in değeri **`finally`'den önce** alınır. `finally` içinde `x`'i değiştirmek dönen değeri değiştirmez (Soru 7).
> - `finally` içinden `return`, `break` ya da `continue` ile **çıkamazsın**; bu derleme hatasıdır (`CS0157`).
> - `catch` olmadan `try-finally` yazılabilir; tek başına `try` yazılamaz.

12.6 throw: istisna fırlatmak

Şimdiye kadar istisnaları .NET fırlattı. **`throw`** ile sen de fırlatabilirsin. Genellikle bir metoda ya da özelliğe geçersiz değer geldiğinde, yanlış veriyle devam etmek yerine işi hemen durdurmak için kullanılır. Örnekte not özelliğinin `set` bloğu (Bölüm 9) geçersiz notu reddediyor:

throwOrnek.cs

```
Ogrenci o = new Ogrenci("Elif");
foreach (int n in new int[] { 85, 120, -5, 70 })
{
    try
    {
```

```
        o.Not = n;
        Console.WriteLine($"{o.Ad} notu {o.Not} olarak kaydedildi.");
    }
    catch (ArgumentException ex)
    {
        Console.WriteLine($"Reddedildi -> {ex.Message}");
    }
}

class Ogrenci
{
    private int _notu;
    public string Ad { get; }

    public Ogrenci(string ad) { Ad = ad; }

    public int Not
    {
        get { return _notu; }
        set
        {
            if (value < 0 || value > 100)
            {
                throw new ArgumentException($"Not 0-100 arası olmalı: {value}");
            }
            _notu = value;
        }
    }
}
```

Çıktı

```
Elif notu 85 olarak kaydedildi.
Reddedildi -> Not 0-100 arası olmalı: 120
Reddedildi -> Not 0-100 arası olmalı: -5
Elif notu 70 olarak kaydedildi.
```

`throw new ArgumentException("...")` satırında önce `new` ile bir istisna **nesnesi** oluşturulur, sonra `throw` ile fırlatılır. `throw`'dan sonraki satırlar çalışmaz (`notu = value;` atlanır); tıpkı `return` gibi blok o anda terk edilir. Böylece `notu` alanına geçersiz değer hiçbir zaman yazılmaz. Bu, kapsülleme fikrinin devamıdır: `set` yalnız doğrulamakla kalmaz, hatayı çağırana da **bildirir**. Geçersiz argüman için `ArgumentException` (ve alt sınıfları), nesnenin durumu o işleme uygun değilse `InvalidOperationException` kullan.

C#'ta `throws` yoktur

Bir metodun hangi istisnaları fırlatabileceğini imzada bildirmen gerekmez; derleyici seni bir istisnayı yakalamaya **zorlamaz**. Bu yüzden C#'ta "checked exception" kavramı yoktur; başka kaynaklarda `throws` anahtar sözcüğünü görürsen C# için geçerli değildir. Fırlatılabilecek istisnaları yalnız **belgelersin**; XML belge yorumundaki `<exception>` etiketi bunun içindir:

```
/// <summary>Notu kaydeder.</summary>
/// <exception cref="ArgumentException">Not 0-100 dışındaysa.</exception>
public void NotKaydet(int deger) { ... }
```

Bu yalnız bir yorumdur; derleyici denetlemez.

12.7 throw; ile throw ex; farkı

Bir `catch` içinde istisnayı işleyip yukarıya **yeniden fırlatmak** isteyebilirsin. İki yazım vardır ve **aynı değildir**:

- `throw;` istisnayı olduğu gibi, **çağrı zinciri (stack trace) korunarak** yeniden fırlatır.
- `throw ex;` yakalanan nesneyi yeni bir istisna gibi fırlatır ve çağrı zincirini bu satırdan başlatır: hatanın asıl yeri **kaybolur**.

yenidenFirlat.cs

```
foreach (bool koruMu in new bool[] { true, false })
{
    try
    {
        Aktar(koruMu);
    }
    catch (DivideByZeroException ex)
    {
        bool izde = (ex.StackTrace ?? "").Contains("Bol");
        Console.WriteLine($"koru={koruMu}  izde Bol var mı: {izde}");
    }
}

static int Bol(int a, int b)
{
    return a / b;
}

static int Aktar(bool koru)
{
    try
    {
        return Bol(1, 0);
    }
    catch (DivideByZeroException ex)
    {
        if (koru)
        {
            throw;
        }
        throw ex;
    }
}
```

Çıktı

```
koru=True  izde Bol var mı: True
koru=False  izde Bol var mı: False
```

İstisna aslında `Bol` metodunda oluştu. `throw;` ile iz `Bol`'u içerir (`True`); `throw ex;` ile iz `Aktar`'da başlar (`False`) ve hata ayıklayan kişi sorunun gerçek yerini göremez.

> **Sık Yapılan Hata: `throw ex;` yazmak**
>
> Yeniden fırlatırken `throw ex;` yazma, `throw;` yaz. Derleyici ikisini de kabul eder; fark yalnız hata ayıklarken ortaya çıkar. İstisnayı başka bir türe **çevirmek** istiyorsan yenisini fırlat ve eskisini ikinci argüman olarak ver: `throw new InvalidOperationException("Hesap başarısız", ex);` (`ex`, yenisinin `InnerException` özelliğinde saklanır).

12.8 Kendi istisna sınıfını yazmak

.NET'in hazır istisnaları her durumu anlatmaya yetmez. "Bakiye yetersiz", "stok bitti" gibi programına özgü hatalar için **kendi istisna sınıfını** yazarsın. Kural basit: `Exception`'dan türet, mesajı `: base(mesaj)` ile üst sınıfa ilet, sınıf adı `Exception` ile bitsin. İkinci `catch`'e dikkat: `when (koşul)` eklenen `catch` yalnız koşul **doğruysa** yakalar.

ozellistisna.cs

```
decimal kasa = 500;
foreach (decimal tutar in new decimal[] { 200, 400, 900, 300 })
{
    try
    {
        kasa = Cek(kasa, tutar);
        Console.WriteLine($"{tutar} TL çekildi, kalan: {kasa} TL");
    }
    catch (YetersizBakiyeException ex) when (ex.Eksik <= 100)
    {
        Console.WriteLine($"Az eksik ({ex.Eksik} TL): {ex.Message}");
    }
    catch (YetersizBakiyeException ex)
    {
        Console.WriteLine($"Çok eksik: {ex.Message}");
    }
}

static decimal Cek(decimal bakiye, decimal tutar)
{
    if (tutar > bakiye)
    {
        throw new YetersizBakiyeException(bakiye, tutar);
    }
    return bakiye - tutar;
}

class YetersizBakiyeException : Exception
{
    public decimal Eksik { get; }

    public YetersizBakiyeException(decimal bakiye, decimal istenen)
        : base($"Yetersiz bakiye: {bakiye} TL var, {istenen} TL istendi")
    {
        Eksik = istenen - bakiye;
    }
}
```

Çıktı

```
200 TL çekildi, kalan: 300 TL
Az eksik (100 TL): Yetersiz bakiye: 300 TL var, 400 TL istendi
Çok eksik: Yetersiz bakiye: 300 TL var, 900 TL istendi
300 TL çekildi, kalan: 0 TL
```

Kendi istisnan sıradan bir sınıftır: özellikleri (`Eksik`) ve metotları olabilir; böylece `catch` yalnız mesajı değil hatayla ilgili **veriyi** de kullanır. `: base(...)` Bölüm 10'daki kurucu zincirinin aynısıdır.

`400 TL` çekiminde eksik 100 olduğu için `when` koşulu doğru çıktı ve ilk `catch` çalıştı. `900 TL` çekiminde eksik 600'dü; koşul yanlış olunca ilk `catch` sanki hiç uymamış gibi atlandı ve ikincisi çalıştı.

> **İpucu: Ne zaman kendi istisnanı yazmalı?**
>
> Hazır tür durumu zaten anlatıyorsa onu kullan: geçersiz argüman için `ArgumentException`, uygunsuz durum için `InvalidOperationException`. Çağıranın **türe bakarak** farklı davranması gerekiyorsa (yetersiz bakiyede tutar sor, kilitli hesapta uyar) ya da hatayla birlikte veri taşıman gerekiyorsa kendi sınıfını yaz.

12.9 using: kaynakları otomatik bırakmak

Dosya, ağ bağlantısı gibi **kaynaklar** işin sonunda mutlaka serbest bırakılmalıdır. Bunu `finally` ile yapmak uzun ve hataya açıktır. `using` ile kaynak açılır ve blok bitince, istisna olsa bile, **otomatik** bırakılır. Bunun için sınıfın `IDisposable` arayüzünü (Bölüm 11) uygulaması gerekir; tek metodu `Dispose()`'tur. Ne zaman çağrıldığını görmek için kendi sınıfımızı yazalım:

kaynakOrnek.cs

```
try
{
    using (Kaynak a = new Kaynak("Dosya"))
    using (Kaynak b = new Kaynak("Bağlantı"))
    {
        a.Kullan();
        b.Kullan();
        throw new InvalidOperationException("işlem yarıda kaldı");
    }
}
catch (InvalidOperationException ex)
{
    Console.WriteLine($"catch: {ex.Message}");
}
finally
{
    Console.WriteLine("finally");
}
Rapor();

static void Rapor()
{
    using var r = new Kaynak("Rapor");
    r.Kullan();
    Console.WriteLine("Rapor metodu bitiyor");
}

class Kaynak : IDisposable
{
    private readonly string _ad;

    public Kaynak(string ad)
    {
        _ad = ad;
        Console.WriteLine($"{ad} açıldı");
    }

    public void Kullan() { Console.WriteLine($"{_ad} kullanılıyor"); }
    public void Dispose() { Console.WriteLine($"{_ad} kapandı"); }
}
```

Çıktı

```
Dosya açıldı
Bağlantı açıldı
Dosya kullanılıyor
Bağlantı kullanılıyor
Bağlantı kapandı
Dosya kapandı
catch: işlem yarıda kaldı
finally
Rapor açıldı
Rapor kullanılıyor
Rapor metodu bitiyor
Rapor kapandı
```

Çıktıdan üç kural çıkar:

1. Kaynaklar açıldıkları sıranın **tersine** bırakılır: önce `Bağlantı`, sonra `Dosya`.
2. İstisna olsa bile `Dispose` çalışır ve `catch`'ten **önce** çalışır.
3. `using var r = ...;` biçimi (`Rapor` metodundaki gibi, süslü parantezsiz) kaynağı **içinde bulunduğu bloğun sonunda** bırakır.

`StreamReader` gibi dosya sınıfları da `IDisposable`'dır; Bölüm 14'te bu kalıbı sürekli kullanacaksın.

12.10 Kullanıcı girdisini doğrulamak

Kullanıcı sayı yerine harf yazınca `Convert.ToInt32`'nin `FormatException` fırlattığını gördün. Artık bunu yakalayıp girdiyi **tekrar** isteyebiliriz. Çok büyük bir sayı (`99999999999`) `OverflowException` üretir; ikisini ayrı mesajlarla karşılıyoruz:

guvenliOkuma.cs

```
int yas = 0;
while (true)
{
    Console.Write("Yaşın: ");
    try
    {
        yas = Convert.ToInt32(Console.ReadLine());
        if (yas >= 0 && yas <= 120)
        {
            break;
        }
        Console.WriteLine("Yaş 0 ile 120 arasında olmalı.");
    }
    catch (FormatException)
    {
        Console.WriteLine("Sayı girmelisin!");
    }
    catch (OverflowException)
    {
        Console.WriteLine("Bu sayı çok büyük!");
    }
}
Console.WriteLine($"Kaydedildi: {yas}");
```

```
Çıktı
Yaşın: yirmi
Sayı girmelisin!
Yaşın: 99999999999
Bu sayı çok büyük!
Yaşın: -5
Yaş 0 ile 120 arasında olmalı.
Yaşın: 19
Kaydedildi: 19
```

`break` yalnız geçerli girdide çalışır; hata olduğunda `catch` mesajı yazar ve döngü başa dönüp soruyu tekrarlar.

12.11 TryParse mi, istisna mı?

Aynı doğrulamayı istisnasız da yapabilirdin: `int.TryParse` çevirme başarısız olunca istisna fırlatmaz, `false` döndürür. İki araç farklı durumlar içindir:

- Hatanın **sık ve beklenen** olduğu yerde (kullanıcı girdisi, metin ayrıştırma) `TryParse` kullan. Kod daha kısa ve okunurdur; üstelik istisna fırlatıp yakalamak, aynı kontrolü `if` ile yapmaktan **kat kat yavaştır**. Tek seferde fark etmezsin ama döngüde yüz binlerce kez olursa fark edersin.
- **Gerçekten istisnai** durumda (dosya yok, nesne uygunsuz durumda, metoda yanlış argüman verildi) istisna kullan. Çağırana "bu iş yapılamadı" demenin en güvenli yoludur: dönüş değerini kontrol etmeyi unutsan bile program sessizce yanlış çalışmaz.

İstisnalar güçlü bir araçtır ama yanlış kullanılınca hataları **gizler**:

1. **Boş `catch` (istisna yutma):** hatayı halının altına süpürmektir; program çökmez ama yanlış sonuç üretir. Aşağıda `20` (sıfır değil, **O harfi**) sessizce yutuluyor.
2. **Gereksiz genel `catch (Exception)`:** beklemediğin bir `NullReferenceException`'ı da sessizce "geçersiz girdi" sanırsın. Beklediğin **özel** türü yakala.
3. **İstisnayı `if` yerine kullanmak:** `if (b != 0)` yazmak `DivideByZeroException` yakalamaktan hem daha okunur hem daha hızlıdır. `IndexOutOfRangeException` yakalıyorsan muhtemelen döngünde hata vardır; düzelt.
4. **Anlamlı mesaj ver:** `"Hata"` yerine `$"Tutar pozitif olmalı: {tutar}"` yaz.

yutma.cs

```cs
string[] veriler = { "10", "20", "30" };
int toplam = 0;
foreach (string v in veriler)
{
    try
    {
        toplam += int.Parse(v);
    }
    catch (Exception)
    {
        // boş catch: hata sessizce yutuldu
    }
}
Console.WriteLine($"Toplam: {toplam}");
```

```
Çıktı
Toplam: 40
```

Program `60` beklenirken `40` yazdı ve hiçbir uyarı vermedi. Bu tür hatalar haftalarca fark edilmeyebilir. Doğrusu, bozuk veriyi görünür kılmaktır:

```
if (int.TryParse(v, out int n))
{
    toplam += n;
}
else
{
    Console.WriteLine($"Geçersiz veri: {v}");
}
```

Bölüm Özeti – Bilmen Gerekenler

- **İstisna**, çalışma zamanında normal akışı bozan olaydır ve bir **nesnedir**. Yakalanmazsa program `Unhandled exception. ...` raporu basıp çöker; dosya adının geçtiği ilk `at` satırı hatanın yeridir.
- Hiyerarşi: `Exception` → `SystemException` → `ArithmeticException`, `ArgumentException`, `FormatException`, `IOException` ... C#'ta **`throws` ve checked exception yoktur**.
- `try`'da istisna olunca geri kalanı atlanır, **uyan ilk** `catch` çalışır, program try-catch'in altından devam eder.
- `catch` sırası **özelden genele**; üst sınıf önce yazılırsa `CS0160`. `catch (Exception)` en sona. `FormatException`, `ArgumentException`'ın alt sınıfı değildir.
- **`finally`** her durumda çalışır (`return` olsa bile); yalnız `Environment.Exit` ve programı anında çökerten durumlar (`StackOverflowException`, `Environment.FailFast`) engeller. `return x` değeri `finally`'den önce alınır; `finally` içinden `return` yazılamaz.
- `throw new X("mesaj")` fırlatır. Yeniden fırlatırken **`throw;`** yaz (`throw ex;` çağrı zincirini sıfırlar). `catch (X ex) when (koşul)` yalnız koşul doğruysa yakalar.
- Kendi istisnan: `class XException : Exception`, mesaj için `: base(mesaj)`.
- **`using`**, `IDisposable` kaynağı blok bitince, istisna olsa da, ters sırayla ve `catch`'ten önce bırakır; `using var` kaynağı bloğun sonunda bırakır.
- Beklenen hatalarda (kullanıcı girdisi) `TryParse`, gerçekten istisnai durumda istisna. Boş `catch` yazma; genel `Exception` yakalamaktan kaçın.

Sorular

A) Çoktan Seçmeli

1 Aşağıdakilerden hangisi `ArithmeticException`'ın alt sınıfıdır?

(A) `FormatException`
(B) `DivideByZeroException`
(C) `IndexOutOfRangeException`
(D) `NullReferenceException`
(E) `InvalidCastException`

2 Aşağıdaki `catch` sıralamalarından hangisi **derleme hatası** verir? (Soldaki önce yazılmıştır.)

A) `FormatException` → `Exception`

B) `DivideByZeroException` → `ArithmeticException`

C) `FormatException` → `NullReferenceException`

D) `ArithmeticException` → `OverflowException`

E) `ArgumentNullException` → `ArgumentException`

3 Aşağıdaki durumların hangisinde `finally` bloğu **çalışmaz**?

A) `try` bloğu hiç istisna oluşmadan bittiğinde

B) `try` içinde oluşan istisna bir `catch` tarafından yakalandığında

C) `try` bloğu içinde bir `return` çalıştığında

D) `catch` bloğu içinde yeni bir istisna fırlatıldığında

E) `try` bloğu içinde `Environment.Exit(0)` çağrıldığında

4 Bir `try` bloğunda `int.Parse("x")` satırı `FormatException` fırlatıyor. `catch` blokları sırasıyla `catch (ArgumentException)`, `catch (SystemException)`, `catch (Exception)` olarak yazılmış. Hangisi çalışır?

A) Yalnızca `catch (SystemException)` bloğu

B) Yalnızca `catch (ArgumentException)` bloğu

C) Üçü de sırayla

D) Yalnızca `catch (Exception)` bloğu

E) Hiçbiri; program çöker

5 Aşağıdakilerden hangisi doğrudur?

A) C#'ta metot imzasına `throws` yazmak zorunludur.

B) `finally` içinde `return` yazılabilir ve istisnayı yutar.

C) `catch` içinde `throw;` çağrı zincirini korur, `throw ex;` ise sıfırlar.

D) `using` bloğundaki kaynak, `catch` bloğundan **sonra** bırakılır.

E) `int.TryParse`, geçersiz girdide `FormatException` fırlatır.

B) Çıktı Tahmini

6 Aşağıdaki program ekrana ne yazdırır?

```
try
{
    B(0);
    B(5);
    Console.WriteLine("Ana kod: B'den döndü");
}
catch (ArgumentException ex)
{
    Console.WriteLine($"Ana kod yakaladı: {ex.Message}");
}
Console.WriteLine("son");

static void B(int x)
{
    try
    {
        Console.WriteLine("B başladı");
        if (x > 0)
        {
            throw new ArgumentException("x pozitif");
        }
        Console.WriteLine("B bitti");
    }
    finally
    {
        Console.WriteLine("B: finally");
    }
}
```

7 Aşağıdaki program ekrana ne yazdırır? (`finally` ve `return` ilişkisine dikkat!)

```
Console.WriteLine(F());
Console.WriteLine(H("7") + H("x"));

static int F()
{
    int x = 1;
    try
    {
        return x;
    }
    finally
    {
        x = 2;
        Console.WriteLine($"F: finally, x={x}");
    }
}

static int H(string s)
{
    try
    {
        return int.Parse(s);
    }
    catch (FormatException)
    {
        return -1;
    }
    finally
    {
        Console.WriteLine("H: finally");
    }
}
```

8 Aşağıdaki program ekrana ne yazdırır?

```
Gir("salon");
try
{
    Gir("kasa");
}
catch (InvalidOperationException ex)
{
    Console.WriteLine($"Hata: {ex.Message}");
}

static void Gir(string oda)
{
    using Kapi dis = new Kapi("Dış");
    using Kapi ic = new Kapi("İç");
    if (oda == "kasa")
    {
        throw new InvalidOperationException("Kasa kilitli");
    }
    Console.WriteLine($"{oda} odasına girildi");
}

class Kapi : IDisposable
{
    private readonly string _ad;

    public Kapi(string ad) { _ad = ad; Console.WriteLine($"{ad} açık"); }
    public void Dispose() { Console.WriteLine($"{_ad} kapalı"); }
}
```

C) Kod Yazma

9 Bir mağazanın stok takibini yapan `stokTakip` programını yaz:

- `Exception`'dan türeyen `StokYetersizException` sınıfı: kurucusu eksik miktarı alsın, mesajı `Stok yetersiz, X adet eksik` olsun, `Eksik` özelliği bu miktarı tutsun.
- `static int Sat(string ad, int stok, int adet)` metodu: `adet <= 0` ise `ArgumentException` (mesaj: `Adet pozitif olmalı: X`), `adet > stok` ise `StokYetersizException` fırlatsın; ikisi de değilse satışı yazdırıp **yeni stoğu** döndürsün.
- Ana kod: stoğu 10 olan "Kalem" için sırasıyla 4, 0, 9, 6 adetlik satış denesin; her denemeyi ayrı yakalasın. Beklenen çıktı:

```
4 adet Kalem satıldı, kalan stok: 6
Geçersiz istek: Adet pozitif olmalı: 0
Satılamadı: Stok yetersiz, 3 adet eksik
6 adet Kalem satıldı, kalan stok: 0
```

10 Kullanıcıdan `sayı işlem sayı` biçiminde satırlar okuyan (örn. `12 + 30`) ve sonucu yazdıran `hesapMakinesi` programını yaz. `q` yazılınca program `Güle güle!` yazıp bitsin. Sayılar tam sayıdır; işlemler `+ - * /`. Program **hiçbir girdide çökmemeli**:

- Sayı olmayan değer → `Hata: geçersiz sayı`
- `int`'e sığmayan sayı → `Hata: sayı int sınırlarının dışında`
- Sıfıra bölme → `Hata: sıfıra bölünemez`
- Bilinmeyen işlem (örn. `x`) → `ArgumentException` fırlat; ekrana `Hata: bilinmeyen işlem: x`
- Üç parçadan oluşmayan satır → `ArgumentException` fırlat; ekrana `Hata: biçim 'sayı işlem sayı' olmalı`

Örnek: `12 + 30` → `= 42`; `7 / 0` → `Hata: sıfıra bölünemez`; `abc * 2` → `Hata: geçersiz sayı`.

Çözümler

Cevap anahtarı

1 B	2 D	3 E	4 A	5 C

1 Cevap: B

`DivideByZeroException`, `ArithmeticException`'dan türer (`OverflowException` de öyle). Diğer dört şık `SystemException`'ın doğrudan alt sınıflarıdır, yani `ArithmeticException`'ın kardeşidir.

2 Cevap: D

`OverflowException`, `ArithmeticException`'ın alt sınıfıdır. `ArithmeticException` önce yazıldığı için taşma hatalarını da o yakalar; ikinci `catch`'e asla sıra gelmez ve derleyici `CS0160` verir. A, B ve E'de özel olan önce, genel olan sonra yazılmıştır (doğru sıra). C'deki iki tür kardeştir; sıraları önemsizdir.

3 Cevap: E

`Environment.Exit(0)` programı hemen kapatır; `finally` dahil hiçbir kod çalışmaz. Diğer durumların hepsinde `finally` çalışır: hatasız bitişte (A), yakalanan istisnada (B), `return`'den hemen önce (C) ve `catch` yeni bir istisna fırlattığında, istisna yukarı iletilmeden önce (D).

4 Cevap: A

`FormatException`'ın üst sınıfı `SystemException`'dır; `ArgumentException` ile **kardeştir**. Bu yüzden ilk `catch` uymaz. İkinci `catch` (`SystemException`) uyar ve çalışır; **uyan ilk** `catch` çalıştığı için üçüncüye sıra gelmez (D ve C yanlış). İstisna yakalandığı için program çökmez (E yanlış). B yaygın bir yanlıştır: metin sayıya çevrilemeyince "argüman hatası" sanılır, oysa öyle değildir.

5 Cevap: C

`throw;` istisnayı olduğu gibi yeniden fırlatır ve iz korunur; `throw ex;` izi o satırdan başlatır. A yanlış: C#'ta `throws` yoktur. B yanlış: `finally` içinden `return` yazmak derleme hatasıdır (`CS0157`). D yanlış: `using` kaynağı, istisna yukarı çıkarken `catch`'ten **önce** bırakır. E yanlış: `TryParse` istisna fırlatmaz, `false` döndürür.

6

- `B(0)`: istisna yok; `B başladı`, `B bitti`, sonra `finally` → `B: finally`.
- `B(5)`: `B başladı` yazılır, `throw` istisnayı fırlatır. `throw`'dan sonraki `B bitti` **atlanır**. `B` istisnayı yakalamaz ama `finally` çalışır → `B: finally`; istisna ana koda geçer.
- Ana kodda `B(5)`'ten sonraki `Ana kod: B'den döndü` atlanır, `catch` çalışır. Sonra program devam eder: `son`.

Çıktı

```
B başladı
B bitti
B: finally
B başladı
B: finally
Ana kod yakaladı: x pozitif
son
```

7

- `F()`: `return x;` çalışırken dönecek değer (**1**) alınır. `finally` `x`'i 2 yapıp yazdırır ama alınmış değer değişmez → önce `F: finally, x=2`, sonra `1`.
- `H("7")`: `try` 7 döndürecek, önce `finally` → `H: finally`. `H("x")`: `FormatException` olur, `catch` -1 döndürecek, önce `finally` → `H: finally`. Toplam `7 + (-1) = 6`. İki `H: finally` satırı, toplam yazdırılmadan **önce** çıkar.

Çıktı

```
F: finally, x=2
1
H: finally
H: finally
6
```

8

- `Gir("salon")`: iki kapı açılır (`Dış açık`, `İç açık`), odaya girilir, metot bitince kapılar **ters sırayla** kapanır (`İç kapalı`, `Dış kapalı`).
- `Gir("kasa")`: kapılar açılır, istisna fırlatılır. `Dispose`'lar istisna ana koda ulaşmadan **önce** ters sırayla çalışır (`İç kapalı`, `Dış kapalı`); sonra `catch` → `Hata: Kasa kilitli`.

Çıktı

```
Dış açık
İç açık
salon odasına girildi
İç kapalı
Dış kapalı
Dış açık
İç açık
İç kapalı
Dış kapalı
Hata: Kasa kilitli
```

9

Yetersiz stok ve geçersiz adet farklı durumlardır; ikisini ayrı `catch` ile, ayrı mesajlarla karşılıyoruz. `StokYetersizException` ve `ArgumentException` birbirinin alt sınıfı olmadığı için `catch` sırası serbesttir. `Sat` istisna fırlatırsa `kalan` güncellenmez; bu istediğimiz davranıştır.

stokTakip.cs

```
int kalan = 10;
foreach (int adet in new int[] { 4, 0, 9, 6 })
{
    try
    {
        kalan = Sat("Kalem", kalan, adet);
    }
    catch (StokYetersizException ex)
    {
        Console.WriteLine($"Satılamadı: {ex.Message}");
    }
    catch (ArgumentException ex)
    {
        Console.WriteLine($"Geçersiz istek: {ex.Message}");
    }
}
```

```
static int Sat(string ad, int stok, int adet)
{
    if (adet <= 0)
    {
        throw new ArgumentException($"Adet pozitif olmalı: {adet}");
    }
    if (adet > stok)
    {
        throw new StokYetersizException(adet - stok);
    }
    Console.WriteLine($"{adet} adet {ad} satıldı, kalan stok: {stok - adet}");
    return stok - adet;
}

class StokYetersizException : Exception
{
    public int Eksik { get; }

    public StokYetersizException(int eksik)
        : base($"Stok yetersiz, {eksik} adet eksik")
    {
        Eksik = eksik;
    }
}
```

Çıktı

```
4 adet Kalem satıldı, kalan stok: 6
Geçersiz istek: Adet pozitif olmalı: 0
Satılamadı: Stok yetersiz, 3 adet eksik
6 adet Kalem satıldı, kalan stok: 0
```

10

Satırı `Split(' ', StringSplitOptions.RemoveEmptyEntries)` ile parçalara ayırıyoruz (Bölüm 8); fazladan boşluklar boş parça üretmez. Hatalardan bir kısmını .NET kendisi fırlatır (`int.Parse` → `FormatException`, sayı `int`'e sığmazsa `OverflowException`, `a / b` → `DivideByZeroException`), bir kısmını biz `throw` ile fırlatırız. Switch ifadesinde `throw` da kullanılabilir. **Dikkat:** `FormatException`, `ArgumentException`'ın alt sınıfı değildir; bu yüzden `ArgumentException` yakalayan `catch` onu yakalamaz ve bu dört `catch` her sırada yazılabilir. `OverflowException`'ı unutursan `999999999999 + 1` gibi bir girdi programı çökertir.

hesapMakinesi.cs

```
while (true)
{
    Console.Write("Hesap> ");
    string satir = Console.ReadLine() ?? "q";
    if (satir == "q")
    {
        break;
    }
    try
    {
        Console.WriteLine($"= {Hesapla(satir)}");
    }
    catch (FormatException)
    {
        Console.WriteLine("Hata: geçersiz sayı");
    }
    catch (OverflowException)
    {
```

```
        Console.WriteLine("Hata: sayı int sınırlarının dışında");
    }
    catch (DivideByZeroException)
    {
        Console.WriteLine("Hata: sıfıra bölünemez");
    }
    catch (ArgumentException ex)
    {
        Console.WriteLine($"Hata: {ex.Message}");
    }
}
Console.WriteLine("Güle güle!");

static int Hesapla(string satir)
{
    string[] p = satir.Split(' ', StringSplitOptions.RemoveEmptyEntries);
    if (p.Length != 3)
    {
        throw new ArgumentException("biçim 'sayı işlem sayı' olmalı");
    }
    int a = int.Parse(p[0]);
    int b = int.Parse(p[2]);
    return p[1] switch
    {
        "+" => a + b,
        "-" => a - b,
        "*" => a * b,
        "/" => a / b,
        _ => throw new ArgumentException($"bilinmeyen işlem: {p[1]}")
    };
}
```

Çıktı

```
Hesap> 12 + 30
= 42
Hesap> 7 / 0
Hata: sıfıra bölünemez
Hesap> abc * 2
Hata: geçersiz sayı
Hesap> 999999999999 + 1
Hata: sayı int sınırlarının dışında
Hesap> 9 x 3
Hata: bilinmeyen işlem: x
Hesap> 8 *
Hata: biçim 'sayı işlem sayı' olmalı
Hesap> q
Güle güle!
```

PROJE 2

Banka Hesabı Simülasyonu

Amaç

Bu projede küçük bir bankanın hesap sistemini yazacaksın. Kullanıcı hesap açacak, para yatırıp çekecek, transfer yapacak, ay sonunda faizleri işletecek ve hesap dökümlerini görecek. Proje, bu bölümde gördüğün konuları tek programda birleştiriyor:

Bölüm	Projede nerede?
9 · Sınıflar, kapsülleme	`private set` özellikler, `Banka` ve `Hesap` sınıfları
10 · Kalıtım	`VadesizHesap` ve `VadeliHesap`, `Hesap`'tan türer, `base(...)`
11 · Soyut sınıf, polimorfizm	`abstract Hesap`; `AySonu()` ve `Cekilebilir()` her türde farklı çalışır
12 · İstisnalar	`YetersizBakiyeException`, `ArgumentException`, menüde `try/catch`

Programın en önemli özelliği **hiçbir girdide çökmemesi**: yetersiz bakiye, olmayan hesap numarası, negatif tutar, harf yazılmış sayı... Her hatalı durum anlaşılır bir mesajla bildirilir ve menü çalışmaya devam eder.

> **Bu Bölümde Öğreneceklerin**
>
> Bu projeyi bitirince soyut bir sınıftan türeyen sınıflar yazabilir, `decimal` ile para hesaplayabilir ve kendi istisna sınıfını fırlatıp yakalayabilirsin.

Gereksinimler

İşlevler (menü seçenekleri):

1. **Hesap aç:** Tür (1 vadesiz, 2 vadeli) ve ad soyad alınır; vadeli hesapta yıllık faiz de istenir. Banka sırayla `1001`, `1002`, ... numaraları verir.
2. **Para yatır / Para çek:** Çekilebilir tutarı aşan istek reddedilir. Vadesiz hesap 1000 TL'ye kadar eksiye düşebilir; vadeli hesap düşemez.
3. **Transfer:** Gönderen hesapta para yetmezse **hiçbir hesap değişmez**.
4. **Hesap dökümü:** İşlem geçmişi, her işlemden sonraki bakiyeyle.
5. **Ay sonu:** Vadeli hesaba aylık faiz (yıllık oran / 12) eklenir; eksideki vadesiz hesaptan borcun %5'i kesilir.
6. **Listele:** Bütün hesaplar ve bankadaki toplam para.
7. **Çıkış.**

Teknik kurallar:

- `Hesap` **soyut** olmalı; hesap türleri ondan türemeli. Hesaplar `Hesap[]` dizisinde, bir sayaçla (`_adet`) tutulur; işlem geçmişi hesabın içinde `string[]`'de.
- Para `decimal` ile tutulur. Sıfır/negatif tutar ve olmayan hesap için `ArgumentException`; yetersiz bakiye için kendi `YetersizBakiyeException`'ımız fırlatılır.
- Girdi `Console.ReadLine()` ile okunup `int.TryParse` / `decimal.TryParse` ile çevrilir. Menü `switch` ile yazılır, her işlem `try/catch` içinde olur.

Tasarım

Sınıflar ve sorumlulukları

Sınıf	Tür	Önemli üyeler	Sorumluluğu
`YetersizBakiyeException`	`Exception`'dan türer	`Eksik`	Reddedilen çekimin eksik tutarını taşır
`Hesap`	**Soyut** sınıf	`No`, `Sahip`, `Bakiye`, `_gecmis`; `Yatir`, `Cek`, `Dokum`; soyut `Tur`, `Cekilebilir`, `AySonu`	Ortak davranış: tutar kontrolü, bakiye değişimi, kayıt
`VadesizHesap`	`Hesap`'ın alt sınıfı	`EkHesapLimiti`, `EkHesapFaizi`	Ek hesap kuralı, eksiye faiz
`VadeliHesap`	`Hesap`'ın alt sınıfı	`_yillikOran`	Eksiye düşmeme kuralı, ay sonu faizi
`Banka`	Sınıf	`_hesaplar`, `_adet`, `_sonNo`; `HesapAc`, `Bul`, `Transfer`, `AySonu`, `Listele`	Hesapları saklar, numara verir, transferi yönetir
Üst düzey kod	Ana program	`MenuYaz`, `SayiOku`, `OndalikOku`	Kullanıcıyla konuşur: menü, girdi, hata mesajı

Yazım sırası: `YetersizBakiyeException` → `Hesap` → `VadesizHesap` ve `VadeliHesap` → `Banka` → üst düzey kod. Menü en son gelir.

Hata durumları

Durum	Ne fırlatılır?	Kim fırlatır?
Çekilecek tutar, çekilebilir tutardan fazla	`YetersizBakiyeException`	`Hesap.Cek`
Tutar sıfır veya negatif	`ArgumentException`	`Hesap.TutarKontrol`
Olmayan hesap numarası	`ArgumentException`	`Banka.Bul`
Aynı hesaba transfer, geçersiz tür veya faiz oranı	`ArgumentException`	`Banka`, `VadeliHesap`
Banka dolu	`InvalidOperationException`	`Banka.HesapAc`
Sayı yerine harf girildi	`FormatException`	`SayiOku`, `OndalikOku`

Bütün bu istisnalar menü döngüsündeki `try/catch`'te yakalanır. İş kuralları ekrana bir şey yazmaz; yalnızca hatayı fırlatır. Mesajı göstermek menünün işidir.

> **İpucu: Önce sınıfları, sonra menüyü yaz**
>
> Dosyada ana kod sınıflardan önce gelir, ama yazarken sırayı izle: önce `Hesap` ve `VadesizHesap`'ı yaz, üst düzey kodda birkaç hesap açıp `Yatir` ve `Cek` çağırarak dene. Çalışınca `VadeliHesap`'ı ve `Banka`'yı ekle. Her adımdan sonra derlendiğinden emin ol.

Kod

BankaUygulamasi.cs

```
Banka banka = new Banka(10);
MenuYaz();
bool devam = true;
while (devam)
{
    string secim = MetinOku("\nSeçim: ");
    try
    {
        switch (secim)
        {
            case "1": HesapAcEkrani(banka); break;
            case "2":
                banka.Bul(SayiOku("Hesap no: "))
                    .Yatir(OndalikOku("Tutar (TL): "), "Para yatırma");
                break;
            case "3":
                banka.Bul(SayiOku("Hesap no: "))
                    .Cek(OndalikOku("Tutar (TL): "), "Para çekme");
                break;
            case "4":
                banka.Transfer(SayiOku("Gönderen hesap: "),
                    SayiOku("Alıcı hesap: "), OndalikOku("Tutar (TL): "));
                Console.WriteLine("Transfer tamamlandı.");
                break;
            case "5": banka.Bul(SayiOku("Hesap no: ")).Dokum(); break;
            case "6":
                banka.AySonu();
                Console.WriteLine("Ay sonu işlemleri yapıldı.");
                break;
            case "7": banka.Listele(); break;
            case "0": devam = false; break;
            default: Console.WriteLine($"Geçersiz seçim: {secim}"); break;
        }
    }
    catch (YetersizBakiyeException e)
    {
        Console.WriteLine($"HATA: {e.Message}");
        Console.WriteLine($"Eksik tutar: {e.Eksik:N2} TL");
    }
    catch (Exception e)                    // sayı, tutar, hesap no, banka dolu
    {
        Console.WriteLine($"HATA: {e.Message}");
    }
}
Console.WriteLine("İyi günler!");

static void MenuYaz()
{
    Console.WriteLine("======== BANKA ========");
    Console.WriteLine("1) Hesap aç  2) Yatır  3) Çek  4) Transfer");
    Console.WriteLine("5) Döküm  6) Ay sonu  7) Listele  0) Çıkış");
}

static void HesapAcEkrani(Banka banka)
{
    int tur = SayiOku("Tür (1=Vadesiz, 2=Vadeli): ");
    string ad = MetinOku("Ad soyad: ");
    decimal yuzde = tur == 2 ? OndalikOku("Yıllık faiz (%): ") : 0;
    Console.WriteLine($"Açıldı: {banka.HesapAc(tur, ad, yuzde)}");
}
```

```
static string MetinOku(string soru)
{
    Console.Write(soru);
    return (Console.ReadLine() ?? "").Trim();
}

static int SayiOku(string soru)
{
    return int.TryParse(MetinOku(soru), out int s)
        ? s : throw new FormatException("Sayı bekleniyordu.");
}

static decimal OndalikOku(string soru)
{
    return decimal.TryParse(MetinOku(soru), out decimal s)
        ? s : throw new FormatException("Sayı bekleniyordu.");
}

class YetersizBakiyeException : Exception
{
    public decimal Eksik { get; }

    public YetersizBakiyeException(int hesapNo, decimal istenen, decimal limit)
        : base($"Hesap {hesapNo}: {istenen:N2} TL istendi, en fazla {limit:N2} TL")
    {
        Eksik = istenen - limit;
    }
}

abstract class Hesap
{
    private readonly string[] _gecmis = new string[20];  // son 20 işlem
    private int _islemSayisi = 0;

    public int No { get; }
    public string Sahip { get; }
    public decimal Bakiye { get; private set; }

    protected Hesap(int no, string sahip)
    {
        No = no;
        Sahip = sahip;
        Kaydet("Hesap açıldı");
    }

    public abstract string Tur();
    public abstract decimal Cekilebilir();   // en yüksek çekilebilir tutar
    public abstract void AySonu();           // her türün kendi kuralı

    public void Yatir(decimal tutar, string aciklama)
    {
        TutarKontrol(tutar);
        Degistir(tutar, aciklama);
    }

    public void Cek(decimal tutar, string aciklama)
    {
        TutarKontrol(tutar);
        decimal limit = Cekilebilir();
        if (tutar > limit)
        {
            throw new YetersizBakiyeException(No, tutar, limit);
        }
        Degistir(-tutar, aciklama);
    }
```

```
    // Bakiyeyi değiştirir ve kaydeder; türetilmiş sınıflar da kullanır
    protected void Degistir(decimal fark, string aciklama)
    {
        Bakiye += fark;
        string isaret = fark < 0 ? "-" : "+";
        Kaydet($"{aciklama,-14} {isaret}{Math.Abs(fark),10:N2}");
    }

    protected void Kaydet(string islem)
    {
        if (_islemSayisi == _gecmis.Length)    // dizi doldu: en eskiyi at
        {
            for (int i = 1; i < _gecmis.Length; i++)
            {
                _gecmis[i - 1] = _gecmis[i];
            }
            _islemSayisi--;
        }
        _gecmis[_islemSayisi++] = $"{islem,-26} | bakiye {Bakiye,10:N2}";
    }

    public void Dokum()
    {
        Console.WriteLine($"Döküm: {this}");
        for (int i = 0; i < _islemSayisi; i++)
        {
            Console.WriteLine($"  {_gecmis[i]}");
        }
    }

    private static void TutarKontrol(decimal tutar)
    {
        if (tutar <= 0)
        {
            throw new ArgumentException($"Tutar pozitif olmalı: {tutar:N2}");
        }
    }

    public override string ToString() =>
        $"{No} {Tur(),-8} {Sahip,-12} {Bakiye,10:N2} TL";
}

class VadesizHesap : Hesap
{
    public const decimal EkHesapLimiti = 1000m;
    public const decimal EkHesapFaizi = 0.05m;     // aylık, borç üzerinden

    public VadesizHesap(int no, string sahip) : base(no, sahip) { }

    public override string Tur() => "Vadesiz";
    public override decimal Cekilebilir() => Math.Max(0m, Bakiye + EkHesapLimiti);

    public override void AySonu()
    {
        if (Bakiye < 0)
        {
            decimal faiz = Math.Round(-Bakiye * EkHesapFaizi, 2);
            Degistir(-faiz, "Ek hesap faizi");
        }
    }
}

class VadeliHesap : Hesap
{
```

```
    private readonly decimal _yillikOran;          // 0.36 = yıllık %36

    public VadeliHesap(int no, string sahip, decimal yuzde) : base(no, sahip)
    {
        if (yuzde <= 0 || yuzde > 100)
        {
            throw new ArgumentException("Faiz oranı 0-100 arası olmalı");
        }
        _yillikOran = yuzde / 100;
    }

    public override string Tur() => "Vadeli";
    public override decimal Cekilebilir() => Bakiye;     // eksiye düşemez

    public override void AySonu()
    {
        decimal faiz = Math.Round(Bakiye * _yillikOran / 12, 2);
        if (faiz > 0)
        {
            Degistir(faiz, "Aylık faiz");
        }
    }
}

class Banka
{
    private readonly Hesap[] _hesaplar;
    private int _adet = 0;
    private int _sonNo = 1000;

    public Banka(int kapasite)
    {
        _hesaplar = new Hesap[kapasite];
    }

    public Hesap HesapAc(int tur, string sahip, decimal yuzde)
    {
        if (_adet == _hesaplar.Length)
        {
            throw new InvalidOperationException("Banka dolu, yeni hesap açılamaz");
        }
        Hesap h = tur switch
        {
            1 => new VadesizHesap(_sonNo + 1, sahip),
            2 => new VadeliHesap(_sonNo + 1, sahip, yuzde),
            _ => throw new ArgumentException($"Geçersiz hesap türü: {tur}")
        };
        _sonNo++;                                  // numara, hesap oluşunca artar
        _hesaplar[_adet++] = h;
        return h;
    }

    public Hesap Bul(int no)
    {
        for (int i = 0; i < _adet; i++)
        {
            if (_hesaplar[i].No == no)
            {
                return _hesaplar[i];
            }
        }
        throw new ArgumentException($"Hesap bulunamadı: {no}");
    }

    public void Transfer(int kaynakNo, int hedefNo, decimal tutar)
```

```
    {
        Hesap kaynak = Bul(kaynakNo);
        Hesap hedef = Bul(hedefNo);
        if (kaynak == hedef)
        {
            throw new ArgumentException("Aynı hesaba transfer yapılamaz");
        }
        kaynak.Cek(tutar, "Giden transfer");    // para yetmezse burada durur
        hedef.Yatir(tutar, "Gelen transfer");
    }

    public void AySonu()
    {
        for (int i = 0; i < _adet; i++)
        {
            _hesaplar[i].AySonu();              // polimorfizm
        }
    }

    public void Listele()
    {
        decimal toplam = 0;
        for (int i = 0; i < _adet; i++)
        {
            Console.WriteLine(_hesaplar[i]);
            toplam += _hesaplar[i].Bakiye;
        }
        Console.WriteLine($"Toplam: {toplam:N2} TL, {_adet} hesap");
    }
}
```

Kodun önemli noktaları

- **Polimorfizm:** `Banka.AySonu` yalnızca `_hesaplar[i].AySonu()` çağırır; hangi kuralın işleyeceğine nesnenin türü karar verir. `Cek` içindeki `Cekilebilir()` da türe göre `Math.Max(0m, Bakiye + 1000)` ya da `Bakiye` döndürür. `Math.Max` sayesinde çekilebilir tutar hiç eksi olmaz: ek hesap faizi bakiyeyi `-1050`'ye indirse bile "en fazla -50 TL" gibi anlamsız bir mesaj çıkmaz. Yeni hesap türü için `HesapAc`'a bir `switch` kolu yeter.
- **İstisna akışı:** Metot imzasında istisna bildirmek gerekmez. `Cek` fırlatır, `Transfer` yakalamadan geçirir, menü yakalar. Yakalanmayan istisna programı durdurur.
- **Transferin güvenliği:** `Transfer` önce `Cek`, sonra `Yatir` çağırır. Para yetmezse `Yatir` hiç çalışmaz; iki hesap da değişmeden kalır.
- **Numara sayacı:** `_sonNo` yalnızca hesap oluştuktan sonra artar. Yapıcı istisna fırlatırsa numara boşa gitmez.
- **Yuvarlama:** Faiz `Math.Round(x, 2)` ile kuruşa yuvarlanır. Varsayılan yuvarlama bankerdir: `Math.Round(2.5m)` 2 olur.

> **Sık Yapılan Hata: Ondalık sabitte `m` eki**
>
> `EkHesapFaizi = 0.05;` yazarsan `0.05` bir `double` olur ve `decimal` alana atanamaz (CS0664). Sonuna `m` koy: `0.05m`.

> **Sık Yapılan Hata: `Bakiye` dışarıdan değişmez**
>
> `private set` yüzünden `h.Bakiye = 5000;` derlenmez. Bakiye yalnızca `Degistir` ile değişir; böylece her hareket kayda geçer.

Sınavda Çıkar: Hangi istisna?

Çekilebilir tutar 300 TL iken `Cek(500)` hangi istisnayı fırlatır? `YetersizBakiyeException`. `Cek(0)` ise `ArgumentException`.

Sınavda Çıkar: Yakalama sırası

Genel `catch (Exception e)` en sona yazılır. Önce yazılırsa altındaki `catch (YetersizBakiyeException e)` derlenmez (CS0160).

Örnek çalışma

Aşağıdaki oturumda iki hesap açılıp para yatırılıyor. Sonra sırayla şu hatalar deneniyor: ek hesap limitini aşan çekim, harf, negatif tutar ve olmayan hesap. Ardından transfer, ek hesaba düşen bir çekim, ay sonu, döküm ve listeleme geliyor:

Çıktı

```
======== BANKA ========
1) Hesap aç  2) Yatır  3) Çek  4) Transfer
5) Döküm  6) Ay sonu  7) Listele  0) Çıkış

Seçim: 1
Tür (1=Vadesiz, 2=Vadeli): 1
Ad soyad: Ayşe Demir
Açıldı: 1001 Vadesiz  Ayşe Demir         0,00 TL

Seçim: 1
Tür (1=Vadesiz, 2=Vadeli): 2
Ad soyad: Mehmet Kaya
Yıllık faiz (%): 36
Açıldı: 1002 Vadeli   Mehmet Kaya        0,00 TL

Seçim: 2
Hesap no: 1001
Tutar (TL): 2000

Seçim: 2
Hesap no: 1002
Tutar (TL): 10000

Seçim: 3
Hesap no: 1001
Tutar (TL): 3500
HATA: Hesap 1001: 3.500,00 TL istendi, en fazla 3.000,00 TL
Eksik tutar: 500,00 TL

Seçim: 3
Hesap no: 1001
Tutar (TL): abc
HATA: Sayı bekleniyordu.

Seçim: 2
Hesap no: 1001
Tutar (TL): -50
HATA: Tutar pozitif olmalı: -50,00

Seçim: 2
Hesap no: 1005
HATA: Hesap bulunamadı: 1005
```

```
Seçim: 4
Gönderen hesap: 1002
Alıcı hesap: 1001
Tutar (TL): 1500
Transfer tamamlandı.

Seçim: 3
Hesap no: 1001
Tutar (TL): 3800

Seçim: 6
Ay sonu işlemleri yapıldı.

Seçim: 5
Hesap no: 1001
Döküm: 1001 Vadesiz  Ayşe Demir      −315,00 TL
  Hesap açıldı               | bakiye       0,00
  Para yatırma   +  2.000,00 | bakiye   2.000,00
  Gelen transfer +  1.500,00 | bakiye   3.500,00
  Para çekme     −  3.800,00 | bakiye    −300,00
  Ek hesap faizi −     15,00 | bakiye    −315,00

Seçim: 7
1001 Vadesiz  Ayşe Demir      −315,00 TL
1002 Vadeli   Mehmet Kaya    8.755,00 TL
Toplam: 8.440,00 TL, 2 hesap

Seçim: 0
İyi günler!
```

Sonuçları elle kontrol edelim:

- Ayşe'nin vadesiz hesabında 2.000,00 TL varken çekilebilir tutar 3.000,00 TL'dir; 3.500,00 TL isteği 500,00 TL eksikle reddedildi.
- Transferden sonra Mehmet'te 10.000 − 1.500 = 8.500, Ayşe'de 3.500 TL kaldı. Ayşe 3.800 TL çekince bakiyesi −300 TL oldu.
- Ay sonunda Ayşe'den 300 × 0,05 = 15 TL ek hesap faizi kesildi (−315). Mehmet'e 8.500 × 0,36 / 12 = 255 TL faiz eklendi (8.755). Toplam: 8.440 TL.
- Hatalı işlemlerin hiçbiri bakiyeleri değiştirmedi ve döküme girmedi.

Bölüm Özeti – Bilmen Gerekenler

- Hesap türleri `Hesap`'tan türer; `Cekilebilir` ve `AySonu` her türde `override` ile farklı yazılır.
- Para `decimal` ile tutulur; iş kuralları istisna fırlatır, menü `try/catch` ile yakalar.
- Girdi `TryParse` ile okunur; hesaplar bir dizide, bir sayaçla takip edilir.

Kendin Geliştir

- **List'e geçir (Bölüm 13):** `Hesap[]` yerine `List<Hesap>` kullan; "banka dolu" sınırı kalksın. `Bul` için `Dictionary<int, Hesap>` ile aramayı döngüsüz yaz.
- **Yeni hesap türü:** Günlük en fazla 2000 TL çekim izni olan, faiz almayan bir `OgrenciHesabi` ekle. Sadece yeni bir alt sınıf ve `HesapAc`'ta bir kol gerekir.
- **PIN kontrolü:** Üç yanlış denemede hesabı kilitleyen bir `HesapKilitliException` yaz.

BÖLÜM 13

Koleksiyonlar, Generics ve LINQ

Bu Bölümde Öğreneceklerin

- Dizilerin neden yetmediğini ve **koleksiyon** (collection) kavramını
- `List<T>` metotlarını, `Remove` / `RemoveAt` farkını ve `Count` ile `Length` karışıklığını
- Dolaşırken listeyi değiştirmenin neden istisna verdiğini ve doğru yolları
- Kendi sınıfını `IComparable<T>` ve lambda ile sıralamayı
- `Dictionary<TKey, TValue>` ile anahtar–değer saklamayı, `TryGetValue`'yu, kelime saymayı
- `HashSet<T>`, `Queue<T>` ve `Stack<T>` koleksiyonlarını
- Generics: tip güvenliğini, `Kutu<T>` gibi generic sınıf ve generic metot yazmayı
- LINQ'e giriş: lambda, `Where`, `Select`, `OrderBy`, `Sum`, `First` ve ertelenmiş çalışma

13.1 Dizilerin sınırı: koleksiyonlar

Bölüm 7'de gördüğün dizilerin büyük bir kısıtı var: **boyutu oluşturulurken belirlenir ve bir daha değişmez.** Sınıfa yeni bir öğrenci geldiğinde diziye "bir kutu daha" ekleyemezsin; daha büyük yeni bir dizi oluşturup eskisini kopyalaman gerekir (`Array.Resize` bunu arka planda yapar). .NET'in hazır **koleksiyon** (collection) sınıfları bu işi senin yerine yapar:

diziVsListe.cs

```
string[] dizi = { "Ali", "Ece", "Can" };
Array.Resize(ref dizi, dizi.Length + 1);   // yeni, daha büyük dizi + kopyalama
dizi[3] = "Naz";
Console.WriteLine(string.Join(", ", dizi));

List<string> liste = new();                // büyümeyi kendisi yapar
liste.Add("Ali");
liste.Add("Ece");
liste.Add("Can");
liste.Add("Naz");
Console.WriteLine(string.Join(", ", liste));
Console.WriteLine("Eleman sayısı: " + liste.Count);
```

Çıktı

```
Ali, Ece, Can, Naz
Ali, Ece, Can, Naz
Eleman sayısı: 4
```

`new()` tipi soldan çıkarır (`new List<string>()` ile aynı). Koleksiyonlar `System.Collections.Generic` ad alanındadır; bu kitaptaki projelerde hazır geldiği için `using` yazmadık. Sınavda `using System.Collections.Generic;` satırını görürsen nedeni budur. Bu bölümde beş koleksiyon göreceksin:

Tip	Ne saklar?	Örnek kullanım
`List<T>`	Sıralı elemanlar, tekrar olabilir, indeksle erişilir	Sınıf listesi
`Dictionary<TKey, TValue>`	**Anahtar → değer** çiftleri	Öğrenci no → ad
`HashSet<T>`	**Tekrarsız** elemanlar	Farklı kelimeler
`Queue<T>`	İlk giren ilk çıkar (FIFO)	Yazıcı kuyruğu
`Stack<T>`	Son giren ilk çıkar (LIFO)	Geri al (undo)

13.2 List: büyüyen dizi

`List<T>`, boyutu otomatik büyüyen bir dizi gibidir. `<` ve `>` arasına **eleman tipini** yazarsın: `List<int>`, `List<string>`, `List<Ogrenci>` (bu yazımın adı **generics**, 13.7'de). Elemanlara dizi gibi indeksle erişirsin ama eleman sayısı `Length` değil **`Count`**'tur.

listeTemel.cs

```
List<string> dersler = new();
dersler.Add("Fizik");                  // sona ekle
dersler.Add("Kimya");
dersler.Add("Programlama");
dersler.Insert(0, "Matematik");        // 0. indekse ekle, diğerleri sağa kayar
Console.WriteLine(string.Join(", ", dersler) + " | Count: " + dersler.Count);

Console.WriteLine("1. indeks: " + dersler[1]);
dersler[3] = "Biyoloji";               // indeksleyiciyle değiştir
Console.WriteLine(string.Join(", ", dersler));

Console.WriteLine(dersler.Contains("Kimya") + " " + dersler.Contains("Tarih"));
Console.WriteLine(dersler.IndexOf("Kimya") + " " + dersler.IndexOf("Tarih"));

dersler.Remove("Fizik");               // değere göre sil
dersler.RemoveAt(0);                   // indekse göre sil
Console.WriteLine(string.Join(", ", dersler));
dersler.Clear();
Console.WriteLine("Count: " + dersler.Count);
```

Çıktı

```
Matematik, Fizik, Kimya, Programlama | Count: 4
1. indeks: Fizik
Matematik, Fizik, Kimya, Biyoloji
True False
2 -1
Kimya, Biyoloji
Count: 0
```

Üye	Ne yapar?	Dönüş
`Add(e)`	Sona ekler	—
`Insert(i, e)`	`i`. konuma ekler; sonrakiler **sağa kayar**	—
`liste[i]`	Okur / değiştirir (indeksleyici)	eleman
`Remove(e)`	Değeri `e` olan **ilk** elemanı siler	`bool` (bulundu mu?)
`RemoveAt(i)`	`i`. elemanı siler; sonrakiler **sola kayar**	—
`Contains(e)`	Var mı?	`bool`
`IndexOf(e)`	İlk konumu; yoksa **−1**	`int`
`Count`	Eleman sayısı (özellik, parantezsiz)	`int`
`Clear()`	Hepsini siler	—

Sık Yapılan Hata: `Length` mı, `Count` mı?

Dizi ve `string` için `Length`, `List<T>` ve öteki koleksiyonlar için `Count` kullanılır; ikisi de **özelliktir**, parantez almaz. `liste.Length` yazarsan derleme hatası alırsın (CS1061); `dizi.Count` da derlenmez. `liste.Count()` (parantezli) ise bir LINQ metodudur (13.8); çalışır ama özellik varken gereksizdir.

Olmayan bir indekse erişirsen (son indeks `Count - 1`'dir) `ArgumentOutOfRangeException` oluşur:

indeksTasma.cs

```
List<int> notlar = new() { 70, 85, 90 };
Console.WriteLine("Count: " + notlar.Count);
Console.WriteLine(notlar[2]);
Console.WriteLine(notlar[notlar.Count]);   // 3. indeks yok
```

Çıktı – program yakalanmamış istisna ile sonlanır

```
Count: 3
90
Unhandled exception. System.ArgumentOutOfRangeException: Index was out of range. Must be non-
negative and less than the size of the collection. (Parameter 'index')
   at Program.<Main>$(String[] args) in indeksTasma.cs:line 4
```

Yeni C# Notu: Koleksiyon başlatıcı ve koleksiyon ifadesi (C# 12)

Listeyi elemanlarıyla birlikte oluşturmanın iki yolu vardır. Köşeli parantezli yazım **koleksiyon ifadesidir** (collection expression); `..` başka bir koleksiyonun elemanlarını yerine açar:

koleksiyonIfadesi.cs

```
List<int> ilk = new List<int> { 3, 8, 5 };   // koleksiyon başlatıcı
List<int> ikinci = [3, 8, 5];                 // koleksiyon ifadesi
List<int> birlesik = [..ilk, 9, ..ikinci];
List<int> bos = [];
Console.WriteLine(string.Join(" ", birlesik) + " | " + bos.Count);
```

Çıktı

```
3 8 5 9 3 8 5 | 0
```

`Remove` ile `RemoveAt` sık karıştırılır: biri **değere**, öbürü **indekse** bakar.

removeTuzak.cs

```
List<int> sayilar = new() { 5, 1, 3, 1 };

sayilar.Remove(1);        // değeri 1 olan İLK elemanı siler
Console.WriteLine(string.Join(" ", sayilar));

sayilar.RemoveAt(1);      // 1. İNDEKSTEKİ elemanı siler
Console.WriteLine(string.Join(" ", sayilar));

bool silindi = sayilar.Remove(99);   // yok: istisna yok, false döner
Console.WriteLine(silindi);
```

Çıktı

```
5 3 1
5 1
False
```

- **Sınavda Çıkar: `Remove` ve `RemoveAt`**
  - `Remove(değer)` **yalnız ilk** eşleşmeyi siler (hepsini değil); bulamazsa istisna atmaz, `false` döndürür.
  - `RemoveAt(indeks)` siler ve sonraki elemanların indeksleri **bir azalır**; art arda `RemoveAt(2)` iki farklı elemanı siler. Var olmayan indeks → `ArgumentOutOfRangeException`.

13.3 Dolaşırken listeden eleman silmek

"Çift sayıları sil" görevinde ilk akla gelen, `foreach` ile dolaşıp silmektir. Deneyelim:

foreachSilme.cs

```
List<int> sayilar = new() { 3, 8, 5, 12, 7 };
foreach (int s in sayilar)
{
    if (s % 2 == 0)
    {
        sayilar.Remove(s);   // dolaşırken listeyi değiştirme!
    }
}
Console.WriteLine(string.Join(" ", sayilar));
```

Çıktı – program yakalanmamış istisna ile sonlanır

```
Unhandled exception. System.InvalidOperationException: Collection was modified; enumeration
operation may not execute.
   at Program.<Main>$(String[] args) in foreachSilme.cs:line 2
```

`foreach`, arkada listeyi gezen bir **gezici (enumerator)** kullanır. Liste dolaşma sürerken değiştirilirse gezici bunu fark eder ve `InvalidOperationException` fırlatır: 8'i sildikten sonra döngü bir sonraki elemana geçmeye çalışırken program çöker. İki güvenli yol:

dogruSilme.cs

```
// Yol 1: sondan başa indeksli döngü
List<int> sayilar1 = new() { 3, 8, 5, 12, 7 };
for (int i = sayilar1.Count - 1; i >= 0; i--)
{
    if (sayilar1[i] % 2 == 0)
    {
        sayilar1.RemoveAt(i);
    }
}
Console.WriteLine(string.Join(" ", sayilar1));

// Yol 2: RemoveAll - koşulu sağlayan hepsini siler, silinen sayısını döndürür
List<int> sayilar2 = new() { 3, 8, 5, 12, 7 };
int silinen = sayilar2.RemoveAll(s => s % 2 == 0);
Console.WriteLine(string.Join(" ", sayilar2) + " (silinen: " + silinen + ")");
```

Çıktı

```
3 5 7
3 5 7 (silinen: 2)
```

`s => s % 2 == 0` bir **lambda ifadesidir**: "her `s` için: `s` çift mi?" diye oku (ayrıntısı 13.8'de). Sondan başa dolaşmak işe yarar, çünkü silme henüz bakılmamış elemanları kaydırmaz.

> **Sık Yapılan Hata: İndeksle baştan sona dolaşırken silmek**
>
> `for (int i = 0; i < liste.Count; i++)` içinde `liste.RemoveAt(i)` yaparsan istisna oluşmaz ama **eleman atlanır**: silinen elemanın yerine sağdaki kayar, `i++` ise onun üzerinden geçer. Yan yana iki çift sayıdan ikincisi silinmeden kalır. Soru 6 tam bu tuzağı soruyor.

13.4 Sıralama: IComparable ve lambda

`Sort()` sayıları ve string'leri nasıl sıralayacağını bilir. `Ogrenci` nesnelerini sıralamak için iki yol var. Birincisi, Bölüm 11'de gördüğün `IComparable<T>`: sınıfa `CompareTo` yazarsan sınıfın bir **doğal sırası** (natural ordering) olur ve parametresiz `Sort()` onu kullanır. İkincisi, `Sort`'a bir **karşılaştırma lambdası** vermek: `(a, b) => a.Puan.CompareTo(b.Puan)`. Lambda, adı olmayan kısa bir metottur (13.8); `a` ve `b` listeden gelen iki elemandır. Sonuç `a` önce gelmeliyse **negatif**, eşitse **0**, sonra gelmeliyse **pozitif** olmalıdır. Aşağıda `Ogrenci`'yi kısaca `record` ile yazdık (Bölüm 9); numaraya göre doğal sırası var:

siralama.cs

```
List<Ogrenci> sinif = new()
{
    new Ogrenci("Zeynep", 1203, 88),
    new Ogrenci("Burak", 1187, 72),
    new Ogrenci("Ayşe", 1250, 95),
    new Ogrenci("Mert", 1199, 64),
};

sinif.Sort();                                   // doğal sıra: CompareTo
Console.WriteLine(string.Join(", ", sinif));

sinif.Sort((a, b) => a.Puan.CompareTo(b.Puan));  // puana göre artan
Console.WriteLine(string.Join(", ", sinif));

sinif.Sort((a, b) => b.Puan.CompareTo(a.Puan));  // a ile b yer değişti: azalan
Console.WriteLine(string.Join(", ", sinif));
```

```
record Ogrenci(string Ad, int No, int Puan) : IComparable<Ogrenci>
{
    public int CompareTo(Ogrenci? diger)
    {
        return diger is null ? 1 : No.CompareTo(diger.No);
    }

    public override string ToString() => $"{Ad}({No}, {Puan})";
}
```

Çıktı

```
Burak(1187, 72), Mert(1199, 64), Zeynep(1203, 88), Ayşe(1250, 95)
Mert(1199, 64), Burak(1187, 72), Zeynep(1203, 88), Ayşe(1250, 95)
Ayşe(1250, 95), Zeynep(1203, 88), Burak(1187, 72), Mert(1199, 64)
```

İlk `Sort()` numaraya göre, ikincisi puana göre artan sıraladı. Üçüncüde `a` ile `b`'nin yerini değiştirmek sıralamayı **azalan** yaptı.

- **Sınavda Çıkar: `Sort()` hakkında**
  - `Sort()` listeyi **yerinde** değiştirir ve bir şey döndürmez; `liste = liste.Sort();` derlenmez.
  - `a.No - b.No` kısa görünür ama büyük sayılarda taşabilir (Bölüm 2); `CompareTo` kullan.
  - `IComparable` uygulamayan bir sınıfın listesinde `Sort()` **derlenir** ama çalışırken `InvalidOperationException` atar. `Sort` kararlı değildir: eşit elemanların sırası korunmayabilir (kararlı sıralama için LINQ `OrderBy`, 13.8).

13.5 Dictionary: anahtar-değer çiftleri

Telefon rehberi gibi düşün: isme (anahtar) bakarsın, numarayı (değer) bulursun. `Dictionary`'de her **anahtar** (key) bir **değere** (value) eşlenir. Anahtarlar **benzersizdir**, değerler tekrar edebilir. Tanımlarken iki tip yazılır: `Dictionary<AnahtarTipi, DegerTipi>`.

sozlukTemel.cs

```
Dictionary<string, int> stok = new();
stok.Add("kalem", 40);                  // yeni çift ekle
stok.Add("defter", 25);
stok["silgi"] = 60;                     // indeksleyiciyle ekle (anahtar yoksa)
Console.WriteLine("Defter: " + stok["defter"]);

stok["defter"] = 30;                    // anahtar varsa değeri DEĞİŞTİRİR
Console.WriteLine("Yeni: " + stok["defter"] + ", Count: " + stok.Count);

Console.WriteLine(stok.ContainsKey("silgi") + " " + stok.ContainsKey("cetvel"));

if (stok.TryGetValue("kalem", out int kalemAdet))
{
    Console.WriteLine("Kalem: " + kalemAdet);
}
bool bulundu = stok.TryGetValue("cetvel", out int cetvelAdet);
Console.WriteLine($"{bulundu}, adet = {cetvelAdet}");

stok.Remove("silgi");
Console.WriteLine(stok.ContainsKey("silgi") + " " + stok.Count);
```

Çıktı

```
Defter: 25
Yeni: 30, Count: 3
True False
Kalem: 40
False, adet = 0
False 2
```

`TryGetValue` anahtar varsa `true` döndürür ve değeri `out` değişkenine yazar; yoksa `false` döndürür ve `out` değişkeni türün varsayılanı olur (`int` için 0). İstisna riski yoktur; sözlükten okumanın güvenli yolu budur (Bölüm 3 ve 12'deki `TryParse` ile aynı kalıp).

Üye	Ne yapar?
`Add(k, v)`	Ekler; `k` zaten varsa `ArgumentException`
`d[k]` (okuma)	Değeri verir; `k` yoksa `KeyNotFoundException`
`d[k] = v`	`k` yoksa ekler, **varsa değeri değiştirir**
`ContainsKey(k)` / `ContainsValue(v)`	Anahtar / değer var mı?
`TryGetValue(k, out v)`	Varsa `true` ve değeri `out`'a yazar; yoksa `false`
`Remove(k)`	Çifti siler; `bool` döndürür
`Count`, `Keys`, `Values`	Çift sayısı, anahtarlar, değerler

Olmayan bir anahtarı okumanın sonucu:

sozlukHata.cs

```
Dictionary<string, int> puan = new();
puan.Add("Ece", 10);
puan["Can"] = 20;                  // indeksleyiciyle ekleme: sorun yok
Console.WriteLine(puan["Can"]);
Console.WriteLine(puan["Bora"]);   // olmayan anahtarı okumak
```

Çıktı – program yakalanmamış istisna ile sonlanır

```
20
Unhandled exception. System.Collections.Generic.KeyNotFoundException: The given key 'Bora' was not
present in the dictionary.
   at Program.<Main>$(String[] args) in sozlukHata.cs:line 5
```

> **Sık Yapılan Hata: Olmayan anahtarı okumak, var olanı `Add` ile eklemek**
>
> `d["x"]` okurken `"x"` yoksa `null` ya da `0` dönmez, `KeyNotFoundException` fırlar. `d.Add("x", 1)` ise `"x"` zaten varsa `ArgumentException` fırlatır. Emin değilsen `ContainsKey` ya da `TryGetValue` kullan; "ekle ya da değiştir" istiyorsan `d["x"] = 1` yaz.

Dolaşmak ve kelime saymak

`foreach` sözlükte her adımda bir `KeyValuePair<TKey, TValue>` verir (`Key` ve `Value` özellikleri). C# bunu iki değişkene açmana da izin verir (deconstruction): `foreach (var (anahtar, deger) in sozluk)`. Klasik uygulama **kelime saymaktır**: her kelime bir anahtar, geçme sayısı değerdir.

kelimeSayaci.cs

```
string metin = "bir elma bir armut iki elma bir de kiraz";
Dictionary<string, int> sayac = new();

foreach (string kelime in metin.Split(' '))
{
    sayac.TryGetValue(kelime, out int eski);   // yoksa eski = 0
    sayac[kelime] = eski + 1;
}

// Sıra garanti olmadığı için anahtarları sıralayıp öyle yazdır
List<string> anahtarlar = new(sayac.Keys);
anahtarlar.Sort();
foreach (string k in anahtarlar)
{
    Console.WriteLine($"{k,-6} {sayac[k]}");
}

string enSik = "";
int enCok = 0;
foreach (var (anahtar, adet) in sayac)
{
    if (adet > enCok)
    {
        enCok = adet;
        enSik = anahtar;
    }
}
Console.WriteLine($"En sık: {enSik} ({enCok}), farklı kelime: {sayac.Count}");
```

Çıktı

```
armut  1
bir    3
de     1
elma   2
iki    1
kiraz  1
En sık: bir (3), farklı kelime: 6
```

> **Not: `Dictionary`'de sıra**
>
> `Dictionary` elemanları dolaşırken pratikte ekleme sırasıyla verir, ama bu **garanti değildir**: silmeden sonra yeni eklenen eleman boşalan yere girip sırayı bozabilir. Çıktısı sıraya bağlı kod yazma: anahtarları listeye alıp `Sort()` ile sırala ya da en büyük/toplam gibi sıradan bağımsız sonuçlar üret. Yukarıda `enSik`, tek bir en sık kelime olduğu için sıradan etkilenmez. (Anahtarları hep sıralı tutan `SortedDictionary` de vardır.)

13.6 HashSet, Queue ve Stack

HashSet: tekrarsız küme

`HashSet<T>` (küme) **tekrarsız** eleman tutar. Zaten olan bir elemanı eklemeye çalışırsan hata vermez; `Add` sadece `false` döndürür ve hiçbir şey yapmaz. İndeks yoktur (`kume[0]` yazılamaz); "var mı?" sorusunu `Contains` çok hızlı yanıtlar.

hashSetOrnek.cs

```
HashSet<string> dersler = new();
Console.WriteLine(dersler.Add("Programlama") + " " + dersler.Add("Fizik"));
Console.WriteLine(dersler.Add("Programlama"));       // zaten var: eklenmez
Console.WriteLine(dersler.Count + " " + dersler.Contains("Fizik"));

// Listedeki tekrarları silmenin kısa yolu: kümeye kopyala
List<string> yoklama = new() { "Can", "Ece", "Ali", "Can", "Ece", "Can" };
HashSet<string> tekil = new(yoklama);
List<string> sirali = new(tekil);
sirali.Sort();                                       // küme sırasına güvenme
Console.WriteLine(string.Join(", ", sirali));
Console.WriteLine($"{yoklama.Count} kayıt, {tekil.Count} kişi");
```

Çıktı

```
True True
False
2 True
Ali, Can, Ece
6 kayıt, 3 kişi
```

`HashSet` de dolaşma sırası garantisi vermez (pratikte ekleme sırasıdır ama güvenme); bu yüzden yukarıda sırayı listeye alıp `Sort()` ile kurduk.

Queue ve Stack

`Queue<T>` (kuyruk) **ilk giren ilk çıkar** (FIFO) çalışır: `Enqueue` sona ekler, `Dequeue` baştakini alıp çıkarır, `Peek` baştakine çıkarmadan bakar. `Stack<T>` (yığın) **son giren ilk çıkar** (LIFO) çalışır: `Push` üste koyar, `Pop` üsttekini alıp çıkarır, `Peek` üsttekine bakar.

kuyrukYigin.cs

```
Queue<string> kuyruk = new();
kuyruk.Enqueue("Ayşe");
kuyruk.Enqueue("Burak");
kuyruk.Enqueue("Cem");
Console.WriteLine("Sıradaki: " + kuyruk.Peek());      // çıkarmaz
Console.WriteLine("Çağrılan: " + kuyruk.Dequeue());   // çıkarır
Console.WriteLine(string.Join(", ", kuyruk) + " | Count: " + kuyruk.Count);

Stack<string> yigin = new();
yigin.Push("a");
yigin.Push("b");
yigin.Push("c");
Console.WriteLine("Tepe: " + yigin.Peek());
Console.WriteLine("Çıkan: " + yigin.Pop());
Console.WriteLine(string.Join(", ", yigin) + " | Count: " + yigin.Count);
```

Çıktı

```
Sıradaki: Ayşe
Çağrılan: Ayşe
Burak, Cem | Count: 2
Tepe: c
Çıkan: c
b, a | Count: 2
```

`Queue` dolaşılırken baştan sona, `Stack` üstten alta verir. **Boş** bir koleksiyonda `Dequeue`, `Pop` ve `Peek` `InvalidOperationException` atar; istisnasız sürümleri `TryDequeue`, `TryPop` ve `TryPeek`'tir (`bool` döndürür, değeri `out` ile verir).

> **İpucu: Hangi koleksiyon?**
>
> Sıra önemli ve indeksle erişiyorsun → `List<T>`. Anahtarla bulacaksın → `Dictionary`. Tekrarsız olsun ya da "var mı?" diye sık soracaksın → `HashSet`. İlk gelen önce işlensin → `Queue`. Son gelen önce işlensin (geri al) → `Stack`.

13.7 Generics

Neden generics?

`List<T>`'deki `T`, "eleman tipi sonradan söylenecek" demektir; bu yapıya **generics** (tip parametreli programlama) denir. Değerini görmek için eleman tipi `object` olan bir liste dene; `object` her şeyi tutar:

genericNeden.cs

```
List<object> karisik = new();
karisik.Add("Ali");
karisik.Add(42);                      // derleyici itiraz etmez
string ilk = (string)karisik[0];      // her okumada dönüşüm (cast) gerekir
Console.WriteLine(ilk);
string ikinci = (string)karisik[1];   // çalışırken patlar
Console.WriteLine(ikinci);
```

Çıktı – program yakalanmamış istisna ile sonlanır

```
Ali
Unhandled exception. System.InvalidCastException: Unable to cast object of type 'System.Int32' to
type 'System.String'.
   at Program.<Main>$(String[] args) in genericNeden.cs:line 6
```

Aynı kodu eleman tipini belirterek yazınca hata **derleme anına** gelir; program hiç çalışmadan sorunu görürsün:

genericDerlenmez.cs

```
List<string> liste = new();
liste.Add("Ali");
liste.Add(42);
string ilk = liste[0];                // dönüşüm gerekmez
Console.WriteLine(ilk);
```

Derleyici çıktısı – program derlenmez

```
genericDerlenmez.cs(3,11): error CS1503: Argument 1: cannot convert from 'int' to 'string'
```

Generics'in iki kazancı: **tip güvenliği** (yanlış tipte eleman derlenmez) ve **dönüşümsüz okuma**. (`System.Collections.ArrayList` gibi eski, tipsiz koleksiyonlar `object` tutar; yeni kodda kullanılmaz.) Değer tiplerinde de kazanç var: `List<int>` sayıları kutulamadan (boxing) doğrudan saklar; `List<object>`'e eklenen her `int` ise bir nesneye kutulanır, yani hem yavaş hem tipsizdir.

Generic sınıf ve generic metot

Kendi sınıfına da bir **tip parametresi** verebilirsin: sınıf adından sonra `<T>` yazılır, `T` kullanılırken verilecek gerçek tipin yerini tutar. Tek bir metodu generic yapmak için `<T>` metot adından sonra yazılır ve **kısıtlanabilir**: `where T : IComparable<T>` "`T` herhangi bir tip olabilir, yeter ki karşılaştırılabilsin" demektir; kısıt olmasa metot içinde `CompareTo` derlenmezdi.

genericOrnek.cs

```
Kutu<string> mektup = new Kutu<string>("Merhaba");
string s = mektup.Icerik;                // dönüşüm (cast) yok
Console.WriteLine(s.ToUpper());

Kutu<int> sayi = new(7);
int k = sayi.Icerik * 6;
Console.WriteLine(k);

int[] sayilar = { 3, 17, 4 };
string[] adlar = { "elma", "armut", "muz" };
Console.WriteLine(EnBuyuk(sayilar) + " " + EnBuyuk(adlar));

static T EnBuyuk<T>(T[] dizi) where T : IComparable<T>
{
    T enBuyuk = dizi[0];
    foreach (T eleman in dizi)
    {
        if (eleman.CompareTo(enBuyuk) > 0)
        {
            enBuyuk = eleman;
        }
    }
    return enBuyuk;
}

class Kutu<T>
{
    public T Icerik { get; set; }

    public Kutu(T icerik)
    {
        Icerik = icerik;
    }
}
```

Çıktı

```
MERHABA
42
17 muz
```

`Kutu<string>` için derleyici sınıftaki her `T`'yi `string` gibi düşünür. Tip parametresinin adı .NET geleneğinde `T` (type) ya da `T` ile başlayan açıklayıcı bir addır: `Dictionary<TKey, TValue>` gibi. Birden fazla olunca virgülle yazılır: `class Ikili<TIlk, TIkinci>`. `EnBuyuk(sayilar)` çağrısında `T`'yi derleyici dizinin tipinden (`int`) çıkarır; `int` ve `string` ikisi de `IComparable<T>` uyguladığı için kısıtı sağlar.

13.8 LINQ'e giriş

Lambda ifadesi

`RemoveAll` ve `Sort`'ta gördüğün `=>` yazımı bir **lambda ifadesidir** (lambda expression): adı olmayan kısa bir metot. `=>` işaretinin solunda parametreler, sağında döndürülen ifade bulunur; parametre tiplerini derleyici bağlamdan çıkarır. Koleksiyonlarla asıl kullanımı LINQ'tedir.

Lambda	Anlamı
`x => x * 2`	Bir parametre; ifadenin değeri döner
`(a, b) => a + b`	İki parametre: parantez şart
`() => 42`	Parametresiz
`s => s.Length > 3`	`bool` döndüren bir **koşul**
`x => { int y = x * x; return y + 1; }`	Çok satırlı: süslü parantez ve `return` gerekir

LINQ metotları

LINQ (Language Integrated Query), her `IEnumerable<T>` üzerinde (liste, dizi, küme, sözlük) çalışan sorgu metotlarıdır; `System.Linq` bu kitapta hazır gelir. Metotlar zincirlenir: `liste.Where(...).OrderBy(...).Select(...)`. Bir lambda'yı değişkende de saklayabilirsin: `Func<girdi, çıktı>` tipinde son tip dönüş tipidir.

linqSayilar.cs

```
Func<int, int> ikiKati = x => x * 2;
Func<int, int, int> topla = (a, b) => a + b;
Console.WriteLine(ikiKati(21) + " " + topla(3, 4));

List<int> sayilar = [4, 15, 8, 23, 16, 45];
Console.WriteLine(string.Join(" ", sayilar.Where(x => x % 2 == 0)));
Console.WriteLine(string.Join(" ", sayilar.Select(x => x * x)));
Console.WriteLine(string.Join(" ", sayilar.OrderBy(x => x)));
Console.WriteLine(string.Join(" ", sayilar.OrderByDescending(x => x)));
Console.WriteLine(sayilar.Count(x => x > 10));
Console.WriteLine(sayilar.Sum() + " " + sayilar.Max() + " " + sayilar.Min());
Console.WriteLine(sayilar.Average());
Console.WriteLine(sayilar.Any(x => x > 40) + " " + sayilar.All(x => x > 5));
List<int> ciftler = sayilar.Where(x => x % 2 == 0).ToList();
Console.WriteLine(ciftler.Count);
```

Çıktı

```
42 7
4 8 16
16 225 64 529 256 2025
4 8 15 16 23 45
45 23 16 15 8 4
4
111 45 4
18,5
True False
3
```

Metot	Ne yapar?	Döndürür
`Where(koşul)`	Koşulu sağlayanları süzer	`IEnumerable<T>`
`Select(dönüşüm)`	Her elemanı dönüştürür	`IEnumerable<U>`
`OrderBy(anahtar)` / `OrderByDescending`	Artan / azalan sıralar; kaynağı **bozmaz**	`IEnumerable<T>`
`Count(koşul)`, `Sum`, `Average`, `Max`, `Min`	Say, topla, ortalama, en büyük, en küçük	`int`, `double`…
`First`, `FirstOrDefault`	İlk eleman (boşsa istisna / varsayılan değer)	`T` / `T?`
`Any(koşul)`, `All(koşul)`	Biri / hepsi koşulu sağlıyor mu?	`bool`
`ToList()`	Sonucu `List<T>` yapar	`List<T>`

`Average()` tamsayı listede bile `double` döndürür. Sorgu metotları nesne listelerinde asıl işini görür. Aynı sorgunun **sorgu sözdizimi** (query syntax) ile yazılışı da aşağıda; SQL'e benzer, ama sınavda esas olan metot sözdizimidir.

linqOgrenci.cs

```
List<Ogrenci> sinif = new()
{
    new Ogrenci("Zeynep", 88),
    new Ogrenci("Burak", 72),
    new Ogrenci("Ayşe", 95),
    new Ogrenci("Mert", 64),
    new Ogrenci("Ece", 82),
};

// Metot sözdizimi: puanı 80 ve üstü olanların adları, puana göre azalan
IEnumerable<string> adlar = sinif.Where(o => o.Puan >= 80)
                                 .OrderByDescending(o => o.Puan)
                                 .Select(o => o.Ad);
Console.WriteLine(string.Join(", ", adlar));

// Aynı sorgu, sorgu sözdizimiyle
IEnumerable<string> adlar2 = from ogr in sinif
                             where ogr.Puan >= 80
                             orderby ogr.Puan descending
                             select ogr.Ad;
Console.WriteLine(string.Join(", ", adlar2));

Console.WriteLine(sinif.Average(o => o.Puan));
Console.WriteLine(sinif.Count(o => o.Puan < 70));
Ogrenci enIyi = sinif.OrderByDescending(o => o.Puan).First();
Console.WriteLine(enIyi.Ad);

record Ogrenci(string Ad, int Puan);
```

Çıktı

```
Ayşe, Zeynep, Ece
Ayşe, Zeynep, Ece
80,2
1
Ayşe
```

`First()` koşulu sağlayan eleman (ya da hiç eleman) yoksa `InvalidOperationException` atar. `FirstOrDefault()` ise istisna yerine tipin varsayılan değerini döndürür: sayı için `0`, `string` ve nesne için `null`. Eleman olmayabilirse `FirstOrDefault` kullan ve sonucu `null`'a karşı kontrol et.

firstBos.cs

```
List<int> bos = new();
Console.WriteLine(bos.FirstOrDefault());                // int'in varsayılanı: 0
List<string> adlar = new();
Console.WriteLine(adlar.FirstOrDefault() is null);      // string'inki: null
Console.WriteLine(bos.First());                         // boş: istisna
```

Çıktı – program yakalanmamış istisna ile sonlanır

```
0
True
Unhandled exception. System.InvalidOperationException: Sequence contains no elements
   at Program.<Main>$(String[] args) in firstBos.cs:line 5
```

Not: Ertelenmiş çalışma (deferred execution)

`Where`, `Select`, `OrderBy` gibi metotlar sorguyu **hemen çalıştırmaz**, yalnızca tarifini saklar. Sorgu, sonuç gerçekten istendiğinde (`foreach`, `string.Join`, `Count()`, `ToList()` ...) ve **her istendiğinde yeniden** çalışır. Kaynak liste sonradan değişirse sorgu yeni hâlini görür; `ToList()` ise anlık görüntü alır:

ertelenmis.cs

```
List<int> sayilar = [1, 2, 3];
IEnumerable<int> sorgu = sayilar.Where(x => x > 1);    // henüz çalışmadı
List<int> kopya = sayilar.Where(x => x > 1).ToList();  // ToList: şimdi çalıştı
sayilar.Add(10);
Console.WriteLine(string.Join(" ", sorgu) + " | " + string.Join(" ", kopya));
```

Çıktı

```
2 3 10 | 2 3
```

Sınavda Çıkar: LINQ'te sık çıkanlar

- `Sum()` boş listede 0 verir; ama `Average()`, `Max()` ve `Min()` boş **sayı** listesinde `InvalidOperationException` atar.
- `OrderBy` kaynak listeyi **değiştirmez**, sıralı bir **sorgu** (`IOrderedEnumerable<T>`) döndürür; elemanlar ancak dolaşılınca ya da `.ToList()` ile listeye çevrilince sıralanır; `Sort()` ise listeyi yerinde değiştirir ve bir şey döndürmez.
- `liste.Count` (özellik) ile `liste.Count(x => ...)` (LINQ metodu, koşullu sayım) farklı şeylerdir.
- `Average()` tamsayı listede bile `double` döndürür: ortalama 18,5 çıkabilir.

13.9 Temsilciler, Func, Action ve event'e giriş

Lambda ifadesi aslında bir **değerdir**: Bir metodu değişkende saklayabilir, başka bir metoda parametre olarak verebilirsin. Bu değerin tipine **temsilci** (delegate) denir. Temsilci, "şu parametreleri alan ve şu tipte değer döndüren her metot" anlamına gelen bir **metot tipidir**.

delegate: metot tipi

`delegate` anahtar sözcüğü yeni bir metot tipi tanımlar. Aşağıda `Islem`, "iki `int` alıp `int` döndüren metot" demektir. Bu tipteki bir değişkene hem adı verilen bir metot hem de bir lambda atanabilir:

temsilci.cs

```
Islem topla = Topla;               // metot adı: parantezsiz!
Islem carp = (a, b) => a * b;      // lambda da aynı tipe atanır

Console.WriteLine(topla(3, 4));
Console.WriteLine(carp(3, 4));
Console.WriteLine(Uygula(10, 5, topla));
Console.WriteLine(Uygula(10, 5, (a, b) => a - b));

static int Topla(int a, int b)
{
    return a + b;
}

static int Uygula(int x, int y, Islem islem)
{
    return islem(x, y);
}

delegate int Islem(int a, int b);
```

Çıktı

```
7
12
15
5
```

`Islem topla = Topla;` satırında metodu **çağırmıyorsun**: Parantez yazmadığın için metodun kendisi değişkene gidiyor. `topla(3, 4)` yazınca metot çalışır. `Uygula` metodu ne yapacağını parametre olarak alır; her çağrıda farklı bir davranış verebilirsin.

Func, Action ve Predicate

Her tür için `delegate` tanımlamak gerekmez; .NET hazır temsilciler sunar. Bunlardan `Func`'ı LINQ metotlarında gördün:

Tip	Anlamı	Örnek
`Func<T1, T2, TSonuc>`	Parametreler ve **son tip dönüş tipi**	`Func<int, int, int>`: iki `int` al, `int` döndür
`Action<T1, T2>`	Değer **döndürmez** (`void`)	`Action<string>`: bir `string` al, bir şey yap
`Predicate<T>`	Tek parametre, `bool` döndürür	`Predicate<int>`: bu sayı koşulu sağlıyor mu?

hazirTemsilci.cs

```
Func<int, int, int> topla = (a, b) => a + b;
Action<string> yaz = mesaj => Console.WriteLine("> " + mesaj);
Predicate<int> ciftMi = x => x % 2 == 0;

yaz("Merhaba");
Console.WriteLine(topla(3, 4));
Console.WriteLine(ciftMi(10) + " " + ciftMi(7));

List<int> sayilar = [5, 12, 8, 3, 20];
List<int> ciftler = sayilar.FindAll(ciftMi);
Console.WriteLine(string.Join(" ", ciftler));
Console.WriteLine(string.Join(" ", sayilar.FindAll(x => x > 6)));
```

```
Çıktı
> Merhaba
7
True False
12 8 20
12 8 20
```

`List<T>.FindAll` bir `Predicate<T>` ister ve koşulu sağlayanlardan yeni bir `List<T>` döndürür. LINQ'in `Where` metodu aynı işi `Func<T, bool>` ile yapar. Yani `Where(x => x > 6)` yazarken verdiğin lambda aslında bir `Func<int, bool>` değeridir.

event: olay

Bir nesne "bir şey oldu" diye başkalarına haber vermek isteyebilir: indirme bitti, sayaç eşiği geçti, düğmeye basıldı. Bu, **olay** (event) ile yapılır. Olayı yayınlayan sınıfta `event` anahtar sözcüğüyle bir temsilci tanımlanır. Dinlemek isteyen metotlar `+=` ile **abone olur** (subscribe), `-=` ile aboneliği bırakır. Olay tetiklenince abone olan bütün metotlar, eklendikleri sırayla çalışır:

olay.cs

```
Sayac s = new Sayac();
s.EsikAsildi += Duyur;                          // abone ol
s.EsikAsildi += d => Console.WriteLine($"İkinci abone: {d}");

for (int i = 0; i < 4; i++)
{
    s.Artir();
}

s.EsikAsildi -= Duyur;                          // aboneliği bırak
s.Artir();

static void Duyur(int deger)
{
    Console.WriteLine($"Eşik aşıldı: {deger}");
}

class Sayac
{
    private int _deger;
    public event Action<int>? EsikAsildi;

    public void Artir()
    {
        _deger++;
        if (_deger >= 3)
        {
            EsikAsildi?.Invoke(_deger);         // olayı tetikle
        }
    }
}
```

```
Çıktı
Eşik aşıldı: 3
İkinci abone: 3
Eşik aşıldı: 4
İkinci abone: 4
İkinci abone: 5
```

Hiç abone yoksa olay `null`'dır; bu yüzden tipi `Action<int>?` yazılır ve tetiklerken `?.Invoke(...)` kullanılır. Son `Artir()` çağrısında yalnız ikinci abone çalıştı, çünkü `Duyur` aboneliği bırakmıştı.

Sınavda Çıkar: `event` ile sıradan temsilci farkı

`event` olayı sınıfın **dışından** yalnızca `+=` ve `-=` ile değiştirebilirsin. Dışarıdan tetikleme (`s.EsikAsildi?.Invoke(5)`) ya da `s.EsikAsildi = null;` yazmak derlenmez (CS0070). Tetikleme yetkisi yalnız olayı tanımlayan sınıftadır.

```
Sayac s = new Sayac();
s.EsikAsildi = null;
s.EsikAsildi?.Invoke(5);

class Sayac
{
    public event Action<int>? EsikAsildi;
}
```

Derleyici çıktısı – program derlenmez

```
olayDisari.cs(2,3): error CS0070: The event 'Sayac.EsikAsildi' can only appear on the left hand
side of += or -= (except when used from within the type 'Sayac')
olayDisari.cs(3,3): error CS0070: The event 'Sayac.EsikAsildi' can only appear on the left hand
side of += or -= (except when used from within the type 'Sayac')
```

Bölüm Özeti – Bilmen Gerekenler

- Dizi boyutu sabittir; koleksiyonlar dinamiktir: `List<T>`, `Dictionary<TKey, TValue>`, `HashSet<T>`, `Queue<T>`, `Stack<T>` (`System.Collections.Generic`).
- Eleman sayısı: dizi ve string → `Length`, koleksiyon → `Count` (ikisi de parantezsiz özellik).
- `List<T>`: `Add`, `Insert`, `Remove` (değerin **ilk** eşleşmesi, `bool` döner), `RemoveAt` (indeks; sonrakiler kayar), `Contains`, `IndexOf` (yoksa −1), indeksleyici. Geçersiz indeks → `ArgumentOutOfRangeException`.
- `foreach` içinde koleksiyonu değiştirmek → `InvalidOperationException`. Doğrusu: sondan başa `for` ya da `RemoveAll`.
- `Sort()` doğal sıra için `IComparable<T>`, başka ölçüt için `Sort((a, b) => ...)` ister; liste yerinde değişir.
- `Dictionary`: `d[k] = v` ekler ya da değiştirir; `Add` var olan anahtarda `ArgumentException`; olmayan anahtarı okumak `KeyNotFoundException`; güvenli okuma `TryGetValue`. Sıra garanti değildir.
- `HashSet.Add` tekrar için `false` döner. `Queue` FIFO (`Enqueue`/`Dequeue`/`Peek`), `Stack` LIFO (`Push`/`Pop`/`Peek`); boşken `Dequeue`/`Pop`/`Peek` → `InvalidOperationException`.
- Generics: tip güvenliği + dönüşümsüz okuma; `class Kutu<T>`, `static T EnBuyuk<T>(T[] dizi) where T : IComparable<T>`; değer tipleri kutulanmaz.
- LINQ: `Where`, `Select`, `OrderBy`, `Count`, `Sum`, `Average`, `Max`, `Min`, `Any`, `All`, `First`/`FirstOrDefault`, `ToList`. Sorgular **ertelenmiş** çalışır; `First` boşta istisna atar, `FirstOrDefault` varsayılan değer döndürür.
- **Temsilci** (delegate) bir metot tipidir: `delegate int Islem(int a, int b);`. Hazır olanlar: `Func<..., TSonuc>` (son tip dönüş), `Action<...>` (`void`), `Predicate<T>` (`bool`); `FindAll` bir `Predicate<T>` ister. `event` ile abonelik `+=`/`-=`, tetikleme yalnız tanımlayan sınıftan (`olay?.Invoke(...)`).

Sorular

A) Çoktan Seçmeli

1 `int[] dizi = new int[3];`, `List<int> liste = new() { 1, 2, 3 };` ve `string metin = "abc";` tanımları verilmiştir. Aşağıdaki ifadelerden hangisi **derlenir** ve `3` değerini verir?

(A) `liste.Length` (B) `dizi.Count` (C) `metin.Count` (D) `liste.Count` (E) `dizi.Length()`

2 Aşağıdaki kod parçası ne yazdırır?

```
List<int> l = new() { 4, 2, 0, 2 };
l.Remove(2);
l.RemoveAt(2);
Console.WriteLine(string.Join(" ", l));
```

(A) `4 0`
(B) `4 2`
(C) `4 0 2`
(D) `4 2 0`
(E) `ArgumentOutOfRangeException` fırlatır.

3 Aşağıdaki kod parçasında ne olur?

```
Dictionary<string, int> d = new();
d["a"] = 1;
d["a"] = 2;
d.Add("b", 3);
d.Add("b", 4);
Console.WriteLine(d.Count);
```

(A) `d["a"] = 2;` satırında `ArgumentException` fırlatılır.
(B) `d["a"] = 2;` satırında `KeyNotFoundException` fırlatılır.
(C) `d.Add("b", 3);` satırında `ArgumentException` fırlatılır.
(D) Hata olmaz, ekrana `2` yazılır.
(E) `d.Add("b", 4);` satırında `ArgumentException` fırlatılır ve program çöker.

4 Bir web tarayıcısında “Geri” düğmesi, en son ziyaret edilen sayfaya dönmeyi sağlar. Ziyaret edilen sayfaları tutmak için en uygun koleksiyon hangisidir?

(A) `Queue<string>`
(B) `Stack<string>`
(C) `HashSet<string>`
(D) `Dictionary<string, int>`
(E) `string[]`

5 Aşağıdaki kod parçası ne yazdırır?

```
List<int> l = new() { 1, 2, 3 };
var sorgu = l.Where(x => x > 1);
var kopya = l.Where(x => x > 1).ToList();
l.Add(4);
Console.WriteLine(sorgu.Count() + " " + kopya.Count);
```

A) `2 2`

B) `3 3`

C) `3 2`

D) `2 3`

E) Derlenmez: `kopya.Count` yazımı hatalıdır.

B) Çıktı Tahmini

6 Aşağıdaki program ekrana ne yazdırır?

```
List<int> l = new() { 2, 4, 4, 5, 6, 8 };
for (int i = 0; i < l.Count; i++)
{
    if (l[i] % 2 == 0)
    {
        l.RemoveAt(i);
    }
}
Console.WriteLine(string.Join(" ", l) + " | " + l.Count);
l.Insert(1, 9);
Console.WriteLine(l.IndexOf(8) + " " + l.Contains(4) + " " + string.Join(" ", l));
```

7 Aşağıdaki program ekrana ne yazdırır?

```
string[] kazananlar = { "Ece", "Can", "Ece", "Bora", "Ece", "Can" };
Dictionary<string, int> puan = new();
foreach (string k in kazananlar)
{
    puan.TryGetValue(k, out int eski);
    puan[k] = eski + 10;
}
puan["Can"] -= 5;
puan.Remove("Bora");
Console.WriteLine(puan.Count + " " + puan.ContainsKey("Bora"));

int toplam = 0;
foreach (int p in puan.Values)
{
    toplam += p;
}
Console.WriteLine(toplam);
Console.WriteLine(puan["Ece"] + " " + puan["Can"]);
```

8 Aşağıdaki program ekrana ne yazdırır?

```
List<int> a = new() { 3, 1, 2 };
List<int> b = a;
List<int> c = new(a);
b.Add(9);
c.Sort();
Console.WriteLine(string.Join(" ", a) + " | " + string.Join(" ", c));

HashSet<int> kume = new(a);
Console.WriteLine(kume.Add(1) + " " + kume.Add(5) + " " + kume.Count);

try
{
    Console.WriteLine(a.Where(x => x > 5).First());
    Console.WriteLine(a.Where(x => x > 10).First());
}
catch (InvalidOperationException)
{
    Console.WriteLine("boş");
}
Console.WriteLine(a.Where(x => x > 10).FirstOrDefault());
```

C) Kod Yazma

9 Kullanıcıdan **bir satır** metin okuyan (kelimeler tek boşlukla ayrılmış) bir program yaz. Program kelimeleri **küçük harfe** çevirip saysın, kelimeleri **alfabetik sırayla** sayılarıyla birlikte yazdırsın ve en sık geçen kelimeyi göstersin. Örnek (`Kod kolay kod eğlenceli Kod zor değil` girilmiş):

```
Metin: Kod kolay kod eğlenceli Kod zor değil
değil: 1
eğlenceli: 1
kod: 3
kolay: 1
zor: 1
En sık geçen: kod (3)
```

10 `Ad` (string), `Fiyat` (decimal) ve `Stok` (int) özellikleri olan bir `Urun` tipi tanımla (kısaca `record` yazabilirsin). 6 ürünlük bir `List<Urun>` oluştur ve sırasıyla: (1) stoğu 0 olan ürünleri `RemoveAll` ile sil, (2) kalanları **fiyata göre artan**, fiyatı eşit olanları **ada göre** sıralayıp tablo halinde yazdır, (3) stoğu en fazla olan ürünü bul.

Çözümler

Cevap anahtarı

1 D	2 A	3 E	4 B	5 C

1 Cevap: D

`Count`, `List<T>` gibi koleksiyonların eleman sayısı **özelliğidir** (parantezsiz): `liste.Count` → 3. A: `List<T>`'de `Length` yoktur. B: dizinin eleman sayısı `Count` değil `Length`'tir. C: `string`'in uzunluğu `Length`'tir; `Count` ancak parantezli bir LINQ metodu olarak vardır, parantezsiz yazılamaz. E: `Length` bir özelliktir, `Length()` diye çağrılamaz.

2 Cevap: A

`Remove(2)` değeri 2 olan **ilk** elemanı siler: `4 0 2`. `RemoveAt(2)` ise **2. indeksteki** elemanı (sondaki 2) siler: `4 0`. B, C ve D, `Remove`'un hangi 2'yi sildiği ya da `RemoveAt`'in hangi indeksi sildiği yanlış hesaplanınca çıkan sonuçlardır. E yanlış: `Remove` tüm 2'leri silmez ve `RemoveAt(2)` çalışırken listede 3 eleman vardır, indeks geçerlidir.

Çıktı

```
4 0
```

3 Cevap: E

İndeksleyiciyle atama (`d["a"] = ...`) anahtar yoksa ekler, varsa değeri değiştirir; hata vermez (A ve B yanlış). `d.Add("b", 3)` yeni bir anahtar ekler (C yanlış). İkinci `d.Add("b", 4)` var olan bir anahtarı eklemeye çalıştığı için `ArgumentException` fırlatır ve `Count` satırına hiç gelinmez (D yanlış). Çıktıdaki `at ...` satırlarında dosya yolu ve satır numarası görünür.

Çıktı – program yakalanmamış istisna ile sonlanır

```
Unhandled exception. System.ArgumentException: An item with the same key has already been added.
Key: b
   at Program.<Main>$(String[] args) in test3.cs:line 5
```

4 Cevap: B

"Geri" için **son giren ilk çıkar** (LIFO) gerekir: sayfa açılınca `Push`, Geri'ye basılınca `Pop`. `Queue` ilk girenden başlar (A yanlış). `HashSet` sıra tutmaz, `Dictionary` anahtar–değer içindir (C, D yanlış). `string[]` boyutu sabittir ve sayfalar eklenip çıkarılamaz (E yanlış).

5 Cevap: C

`sorgu` ertelenmiş çalışır: `Count()` çağrıldığında, yani `Add(4)`'ten **sonra** çalışır ve 2, 3, 4 → 3 sayar. `kopya` ise `ToList()` ile `Add`'ten **önce** hesaplandı: 2, 3 → `Count` 2. A, sorgunun da erken çalıştığını; B, ikisinin de yeni hâli gördüğünü; D ise sonuçların ters olduğunu sanar. E yanlış: `kopya` bir `List<int>`'tir ve `Count` özelliği geçerlidir.

Çıktı

```
3 2
```

6

Baştan sona indeksle silmek eleman atlatır:

i	Liste (kontrol öncesi)	l[i]	İşlem
0	`[2, 4, 4, 5, 6, 8]`	2	çift → sil
1	`[4, 4, 5, 6, 8]`	4	çift → sil (0. indeksteki 4 atlandı!)
2	`[4, 5, 6, 8]`	6	çift → sil
3	`[4, 5, 8]`	—	`3 < 3` yanlış, döngü biter

Sonuç `[4, 5, 8]`, `Count` 3. `Insert(1, 9)` → `[4, 9, 5, 8]`; `8`'in indeksi 3, `4` hâlâ listede.

Çıktı
```
4 5 8 | 3
3 True 4 9 5 8
```

7

- Döngü: Ece 3 kez (30), Can 2 kez (20), Bora 1 kez (10). `TryGetValue` olmayan anahtar için `eski`'yi 0 yapar.
- `puan["Can"] -= 5` → 15. `Remove("Bora")` sonrası sözlükte Ece ve Can kalır: `Count` 2, `ContainsKey("Bora")` `False`.
- `Values` toplamı 30 + 15 = 45. Sıraya bağlı bir şey yazdırılmadığı için çıktı sözlük sırasından etkilenmez.

Çıktı
```
2 False
45
30 15
```

8

- `b = a` kopya değil, **aynı listeye ikinci başvurudur**: `b.Add(9)` `a`'yı da `[3, 1, 2, 9]` yapar. `c` ise `new(a)` ile 9 eklenmeden **önce** alınmış bağımsız bir kopyadır; sıralanınca `[1, 2, 3]`.
- `kume` {3, 1, 2, 9} ile başlar: `Add(1)` zaten var → `False`; `Add(5)` yeni → `True`; `Count` 5.
- `Where(x => x > 5).First()` → 9. 10'dan büyük eleman yok: `First()` istisna atar, `catch` "boş" yazdırır. `FirstOrDefault()` ise istisna atmaz, `int`'in varsayılanı `0` döner.

Çıktı
```
3 1 2 9 | 1 2 3
False True 5
9
boş
0
```

9

Kelimeleri bir `Dictionary<string, int>` içinde sayarız. Sözlük sırası garanti olmadığı için anahtarları bir listeye alıp `Sort()` ile alfabetik sıralarız; en sık kelimeyi de bu sıralı listeyi gezerken buluruz.

frekans.cs

```
Console.Write("Metin: ");
string satir = Console.ReadLine() ?? "";
string[] kelimeler = satir.ToLower().Split(' ');

Dictionary<string, int> sayac = new();
foreach (string kelime in kelimeler)
{
    sayac.TryGetValue(kelime, out int eski);
    sayac[kelime] = eski + 1;
}

List<string> anahtarlar = new(sayac.Keys);
anahtarlar.Sort();

string enSik = "";
int enCok = 0;
foreach (string k in anahtarlar)
{
    Console.WriteLine(k + ": " + sayac[k]);
    if (sayac[k] > enCok)
    {
        enCok = sayac[k];
        enSik = k;
    }
}
Console.WriteLine($"En sık geçen: {enSik} ({enCok})");
```

Çıktı

```
Metin: Kod kolay kod eğlenceli Kod zor değil
değil: 1
eğlenceli: 1
kod: 3
kolay: 1
zor: 1
En sık geçen: kod (3)
```

10

`RemoveAll` ile stoğu 0 olanları siler, `OrderBy(...).ThenBy(...)` ile iki ölçütlü sıralarız; en çok stoklu ürün için stoğa göre azalan sıralayıp `First()` alırız.

urunListesi.cs

```
List<Urun> urunler = new()
{
    new Urun("Defter", 45.0m, 12),
    new Urun("Kalem", 12.5m, 0),
    new Urun("Silgi", 7.25m, 125),
    new Urun("Cetvel", 45.0m, 8),
    new Urun("Boya", 89.9m, 0),
    new Urun("Makas", 30.0m, 15),
};

urunler.RemoveAll(u => u.Stok == 0);
List<Urun> sirali = urunler.OrderBy(u => u.Fiyat).ThenBy(u => u.Ad).ToList();

Console.WriteLine($"{"Ürün",-8}{"Fiyat",8}{"Stok",6}");
foreach (Urun u in sirali)
{
    Console.WriteLine($"{u.Ad,-8}{u.Fiyat,8:F2}{u.Stok,6}");
```

```
}

Urun enCok = urunler.OrderByDescending(u => u.Stok).First();
Console.WriteLine("En çok stok: " + enCok.Ad);

record Urun(string Ad, decimal Fiyat, int Stok);
```

Çıktı

```
Ürün       Fiyat  Stok
Silgi       7,25   125
Makas      30,00    15
Cetvel     45,00     8
Defter     45,00    12
En çok stok: Silgi
```

`Cetvel` ve `Defter`'in fiyatı eşit (45) olduğu için `ThenBy` devreye girdi ve ada göre sıralandılar.

BÖLÜM 14

Dosya İşlemleri

Bu Bölümde Öğreneceklerin

- Verinin program bitince de kalması için dosyaya yazmayı
- `Path` ile dosya yollarını, göreli ve mutlak yol farkını, **çalışma klasörünü**
- `File` sınıfıyla tek çağrıda okuma, yazma, ekleme ve silme işlemlerini
- `FileNotFoundException`'ı yakalamayı ve `IOException` ile ilişkisini
- `StreamReader` ve `StreamWriter` ile satır satır çalışmayı; `using` ile dosyayı kapatmayı
- UTF-8 kodlamasını ve Türkçe karakter sorunlarını
- CSV dosyasını `;` ayırıcıyla yazıp okumayı ve `Ogrenci` nesnelerini kaydetmeyi

14.1 Neden dosya?

Şimdiye kadar yazdığın programlarda veriler (değişkenler, diziler, `List<T>` nesneleri) **RAM**'de duruyordu. Program bittiğinde hepsi silinir. Verinin **kalıcı** olması için diske, bir **dosyaya** yazman gerekir: not listesi, oyun skorları, bir kayıt defteri...

Bu bölümde **metin dosyaları** (`.txt`, `.csv`) ile çalışacağız; bunları Not Defteri, VS Code ya da Excel ile açıp okuyabilirsin. Dosya işlemleri `System.IO` ad alanındadır ve projede hazır gelir:

Sınıf	Ne için kullanılır?
`File`	Tek çağrıyla okuma, yazma, ekleme, silme
`Directory`	Klasör oluşturma ve listeleme
`Path`	Dosya yolunu birleştirme ve parçalama
`StreamReader`, `StreamWriter`	Satır satır okuma ve yazma

İlk programımız bir dosyaya iki satır yazıp geri okuyor:

ilkDosya.cs

```
File.WriteAllText("selam.txt", "Merhaba dosya!\nBu satır diskte duruyor.\n");
Console.WriteLine("selam.txt yazıldı.");

string icerik = File.ReadAllText("selam.txt");
Console.Write(icerik);
```

Çıktı

```
selam.txt yazıldı.
Merhaba dosya!
Bu satır diskte duruyor.
```

Program bittikten sonra çalışma klasöründe `selam.txt` dosyasını görürsün. `File.WriteAllText` dosyayı kendisi açıp kapattığı için bu programda `using` gerekmez.

14.2 Dosya yolları ve çalışma klasörü

Dosyanın diskteki adresine **yol** (path) denir. Yolu elle birleştirmek yerine `Path.Combine` kullan: ayırıcıyı (`/` ya da `\`) işletim sistemine göre kendisi seçer.

yolOrnek.cs

```
string yol = Path.Combine("veriler", "2026", "notlar.csv");
Console.WriteLine("Yol       : " + yol);
Console.WriteLine("Dosya adı : " + Path.GetFileName(yol));
Console.WriteLine("Uzantı    : " + Path.GetExtension(yol));
Console.WriteLine("Üst klasör: " + Path.GetDirectoryName(yol));
Console.WriteLine("Mutlak mı?: " + Path.IsPathRooted(yol));
```

Çıktı

```
Yol       : veriler/2026/notlar.csv
Dosya adı : notlar.csv
Uzantı    : .csv
Üst klasör: veriler/2026
Mutlak mı?: False
```

Bu çıktı macOS'ta üretildi; Windows'ta ayırıcı `\` olur: `veriler\2026\notlar.csv`. İki tür yol vardır:

Tür	Örnek	Nereden başlar?
Mutlak (absolute)	`@"C:\Veriler\notlar.txt"`, `/home/elif/notlar.csv`	Diskin kökünden; her yerden aynı dosya
Göreli (relative)	`notlar.csv`, `veriler/notlar.csv`	Programın **çalışma klasöründen**

Göreli yol, programın çalışma klasörüne göre çözülür. Visual Studio'da (Windows) bu klasör `bin\Debug\net10.0` klasörüdür; veri dosyasını proje klasörüne değil, buraya koymalısın. Komut satırında `dotnet run` çalıştırıyorsan çalışma klasörü, komutu yazdığın klasördür.

İpucu: "Dosya bulunamadı" diyorsa çalışma klasörüne bak

Programın dosyayı nerede aradığını görmek için `Path.GetFullPath("notlar.csv")` sonucunu yazdır. Dosyayı o klasöre taşı ya da yolu mutlak olarak yaz.

Sık Yapılan Hata: Windows yolunda ters bölü

`\V` tanınmayan bir kaçış dizisidir; derleyici aşağıdaki kodu reddeder:

```
string yol = "C:\Veriler\notlar.txt";
Console.WriteLine(yol);
```

Derleyici çıktısı – program derlenmez

```
tersBolu.cs(1,17): error CS1009: Unrecognized escape sequence
```

Çözüm iki tanedir: ters bölüleri ikile (`"C:\\Veriler\\notlar.txt"`) yazmak ya da metnin başına `@` koyup **verbatim** (olduğu gibi) metin yapmak: `@"C:\Veriler\notlar.txt"`.

14.3 File sınıfıyla yazma ve okuma

`File` sınıfının statik metotları küçük ve orta boy dosyalar için en kısa yoldur:

Metot	Ne yapar?
`File.WriteAllText(yol, metin)`	Yazar; dosya yoksa oluşturur, varsa **eski içeriği siler**
`File.AppendAllText(yol, metin)`	Metni dosyanın **sonuna** ekler; dosya yoksa oluşturur
`File.ReadAllText(yol)`	Bütün dosyayı tek bir `string` olarak okur
`File.WriteAllLines(yol, satirlar)`	Dizinin ya da listenin her elemanını ayrı satıra yazar
`File.ReadAllLines(yol)`	Satırları `string[]` olarak verir; satır sonu karakteri yoktur
`File.AppendAllLines(yol, satirlar)`	Satırları dosyanın sonuna ekler
`File.Exists(yol)`	Dosya var mı? (`bool` döndürür)
`File.Delete(yol)`	Dosyayı siler

dosyalslemleri.cs

```
string[] urunler = { "ekmek", "süt", "yumurta" };
File.WriteAllLines("alisveris.txt", urunler);
Console.WriteLine("Var mı? " + File.Exists("alisveris.txt"));

List<string> ekler = ["peynir", "zeytin"];
File.AppendAllLines("alisveris.txt", ekler);

string[] satirlar = File.ReadAllLines("alisveris.txt");
Console.WriteLine(satirlar.Length + " satır");
for (int i = 0; i < satirlar.Length; i++)
{
    Console.WriteLine((i + 1) + ". " + satirlar[i]);
}

File.WriteAllText("alisveris.txt", "sadece bu kaldı\n");
Console.Write(File.ReadAllText("alisveris.txt"));

File.Delete("alisveris.txt");
Console.WriteLine("Silindi mi? " + !File.Exists("alisveris.txt"));
```

Çıktı

```
Var mı? True
5 satır
1. ekmek
2. süt
3. yumurta
4. peynir
5. zeytin
sadece bu kaldı
Silindi mi? True
```

> **Yeni C# Notu: Koleksiyon ifadesi**
>
> `List<string> ekler = ["peynir", "zeytin"];` satırındaki `[...]` yazımı C# 12 ile gelen **koleksiyon ifadesidir** (collection expression). Dizi ya da `List<T>` oluşturmanın kısa yoludur; `new List<string> { ... }` ile aynı işi yapar.

Sık Yapılan Hata: `WriteAllText` eski içeriği siler

`File.WriteAllText` ve `File.WriteAllLines` dosya varsa içeriğini sorgusuz siler. Her çalıştırmada yeni bir kayıt eklemek istiyorsan `File.AppendAllText` kullan. Yukarıdaki programda dosyayı bilerek `WriteAllText` ile baştan yazdık.

Sınavda Çıkar: Yoksa oluşturma farkı

`AppendAllText` ve `AppendAllLines` dosya yoksa **oluşturur**; ekleme için ek bir seçenek gerekmez. `File.Delete` ise var olmayan bir dosyayı silmeye çalışınca hata **vermez**, sessizce döner.

14.4 Olmayan dosya: FileNotFoundException

Dosya yoksa `File.ReadAllText` `System.IO.FileNotFoundException` fırlatır. Bu sınıf `IOException`'ın alt sınıfıdır. C#'ta derleyici istisna yakalamanı zorunlu tutmaz; yakalamazsan program çöker:

olmayanDosya.cs

```
Console.WriteLine("Okuma başlıyor.");
string icerik = File.ReadAllText("yok.txt");
Console.WriteLine(icerik);
```

Çıktı – program yakalanmamış istisna ile sonlanır

```
Okuma başlıyor.
Unhandled exception. System.IO.FileNotFoundException: Could not find file '...\yok.txt'.
File name: '...\yok.txt'
   at Program.<Main>$(String[] args) in olmayanDosya.cs:line 2
```

Çıktıda önce `Okuma başlıyor.` görünür, sonra `Unhandled exception.` ile başlayan hata yazılır. Mesajdaki tam yol bilgisayarına göre değişir; sondaki `at` satırları hatanın yerini gösterir.

Programın çökmesini istemiyorsan `try-catch` ile yakala. Hatayı yakalayan bir `OkuVeYaz` metodu yazalım:

guvenliOku.cs

```
File.WriteAllText("var.txt", "C#");
OkuVeYaz("var.txt");
OkuVeYaz("yok.txt");
Console.WriteLine("Program çökmedi, devam ediyor.");

static void OkuVeYaz(string ad)
{
    try
    {
        string icerik = File.ReadAllText(ad);
        Console.WriteLine(ad + " okundu: " + icerik.Length + " karakter");
    }
    catch (FileNotFoundException)
    {
        Console.WriteLine("Dosya bulunamadı: " + ad);
    }
    catch (IOException e)
    {
        Console.WriteLine("Okuma hatası: " + e.Message);
    }
}
```

```
Çıktı
var.txt okundu: 2 karakter
Dosya bulunamadı: yok.txt
Program çökmedi, devam ediyor.
```

> **Sınavda Çıkar: `catch` sırası**
>
> `FileNotFoundException`, `IOException`'dan türediği için önce yazılır. Ters yazarsan `IOException` her hatayı yakalar ve alttaki `catch` hiç çalışamaz; derleyici bunu hata olarak bildirir.

14.5 StreamWriter ve StreamReader

`File.ReadAllText` küçük dosyalar için idealdir. Büyük bir dosyayı ya da her satırı okur okumaz işlemek istiyorsan **akış** (stream) kullan: yazmak için `StreamWriter`, okumak için `StreamReader`. İkisi de dosyaya bağlı bir kaynaktır ve işin bitince **kapatılmalıdır**. `using` bloğu bunu otomatik yapar (Bölüm 12):

tamponluYazOku.cs

```
using (StreamWriter yazici = new StreamWriter("program.txt"))
{
    yazici.WriteLine("Pazartesi: C# dersi");
    yazici.WriteLine("Salı: Fizik laboratuvarı");
    yazici.WriteLine("Çarşamba: Matematik quiz");
}

using (StreamReader okuyucu = new StreamReader("program.txt"))
{
    string? satir;
    int no = 0;
    while ((satir = okuyucu.ReadLine()) != null)
    {
        no++;
        Console.WriteLine(no + ") " + satir);
    }
}
```

```
Çıktı
1) Pazartesi: C# dersi
2) Salı: Fizik laboratuvarı
3) Çarşamba: Matematik quiz
```

`WriteLine` satır sonunu kendisi ekler. `ReadLine` satır sonu karakterini döndürmez ve dosya bittiğinde **`null`** verir; okuma döngüsünün `!= null` koşulu bu yüzdendir. `string?` yazımı "bu metin `null` olabilir" demektir. Dosyanın tamamını tek seferde istersen `okuyucu.ReadToEnd()` çağrısı yeterlidir.

> **Sınavda Çıkar: `using var` nerede kapanır?**
>
> `using var` ile açılan kaynak **bulunduğu bloğun sonunda** kapanır. Üst düzey deyimlerde bu, programın sonudur. Yazıcıyı `using var` ile açıp hemen ardından aynı dosyayı okursan veri henüz diske ulaşmamış olabilir; okuyucu eksik ya da boş bir dosya görür. Windows'ta durum daha da kötüdür: dosya hâlâ yazma için açık olduğundan okuma girişimi `IOException` ("dosya başka bir işlem tarafından kullanılıyor") ile çöker. Bu yüzden yukarıda `using (...) { }` bloğunu kullandık.

Sık Yapılan Hata: Kapatmayı unutmak

Yazıcı, yazdığını bir **tampon**da (buffer) biriktirir. Veri diske ancak tampon dolduğunda ya da `Dispose()` / `Close()` çağrıldığında yazılır. Kapatmadan önce ve sonra dosyanın diskteki boyutuna bakalım. Dosyayı okumaya çalışmıyoruz, çünkü Windows'ta hâlâ yazma için açık olan bir dosyayı okumak `IOException` verir; `new FileInfo("sonuc.txt").Length` ise dosyayı açmadan boyutunu bayt olarak söyler:

```
StreamWriter yazici = new StreamWriter("sonuc.txt");
yazici.WriteLine("Önemli sonuç: 42");
long once = new FileInfo("sonuc.txt").Length;
Console.WriteLine($"Kapatmadan önce: {once} bayt");
yazici.Dispose();
long sonra = new FileInfo("sonuc.txt").Length;
Console.WriteLine($"Kapattıktan sonra: {sonra} bayt");
```

Çıktı

```
Kapatmadan önce: 0 bayt
Kapattıktan sonra: 19 bayt
```

Kapatmadan önce veri hâlâ tampondaydı, dosya 0 bayttı. (Windows'ta ikinci satır 20 bayt gösterir: `WriteLine` orada satır sonunu `\r\n` diye iki baytla yazar.)

`using` bloğu bittiğinde yazıcı her zaman kapanır ve tampon boşalır. Bu yüzden kapatmayı elle değil, `using` ile yap.

14.6 Kodlama: UTF-8 ve Türkçe karakterler

Dosyada karakterler değil **baytlar** saklanır. Hangi karakterin hangi baytlarla yazılacağını **karakter kodlaması** belirler. .NET'te `File` sınıfı, `StreamReader` ve `StreamWriter` varsayılan olarak **UTF-8** kullanır. UTF-8'de `ç ğ ı ö ş ü` gibi harfler 2 bayt yer kaplar; Türkçe karakterler dosyaya yazılıp geri okunurken bozulmaz.

Sorun, dosyayı başka bir kodlamayla okursan çıkar:

kodlama.cs

```
using System.Text;

string metin = "Ilık süt ve çörek";
File.WriteAllText("turkce.txt", metin);
Console.WriteLine("Karakter sayısı: " + metin.Length);
Console.WriteLine("Dosya boyutu: " + new FileInfo("turkce.txt").Length + " bayt");

string dogru = File.ReadAllText("turkce.txt", Encoding.UTF8);
string yanlis = File.ReadAllText("turkce.txt", Encoding.Latin1);
Console.WriteLine("UTF-8 ile oku: " + dogru);
Console.WriteLine("Latin-1 ile oku: " + yanlis);
```

Çıktı

```
Karakter sayısı: 17
Dosya boyutu: 21 bayt
UTF-8 ile oku: Ilık süt ve çörek
Latin-1 ile oku: IlÄ±k sÃ¼t ve Ã§Ã¶rek
```

Metinde 4 Türkçe harf (`ı`, `ü`, `ç`, `ö`) var; her biri 2 bayt tuttuğu için 17 karakterlik metin 21 bayt yer kapladı. Latin-1 ile okuyunca her Türkçe harf iki anlamsız karaktere dönüştü. Ekranda `Ã§` gibi işaretler görürsen sebep budur: dosya UTF-8 ile yazılmış, okuma başka bir kodlamayla yapılmıştır. Eski bir programın Windows-1254 ile kaydettiği dosyayı okurken kodlamayı açıkça belirtmen gerekir.

> **Not: Satır sonu**
>
> Windows'ta satır sonu iki karakterdir (`\r\n`), Linux ve macOS'ta bir karakterdir (`\n`). `WriteLine` ve `WriteAllLines` işletim sisteminin satır sonunu kullanır; `ReadLine` ve `ReadAllLines` ikisini de tanır.

14.7 CSV dosyası ve Ogrenci sınıfı

CSV (Comma-Separated Values) tablo verisini metin olarak tutar: her satır bir kayıt, `;` ile ayrılmış her parça bir alandır. Ayırıcı olarak **noktalı virgül** kullanacağız. Türkçe Excel'de ondalık ayırıcı virgüldür; `72,5` gibi bir sayı ile `,` ayırıcı karışırdı. Sayılar tr-TR biçiminde (`72,5`) yazılır ve `Convert.ToDouble` ile okunur.

Aşağıdaki program bir `Ogrenci` listesini CSV'ye yazar. Önce dosyanın ham hâlini, sonra okunmuş verinin ortalamalarını yazdırır:

ogrenciCsv.cs

```
List<Ogrenci> liste =
[
    new Ogrenci("Ayşe Kaya", 72.5, 85),
    new Ogrenci("Burak Demir", 45, 38.5),
    new Ogrenci("Cem Yıldız", 90, 95),
];

CsvYaz("ogrenciKayit.csv", liste);
Console.Write(File.ReadAllText("ogrenciKayit.csv"));

List<Ogrenci> okunan = CsvOku("ogrenciKayit.csv");
foreach (Ogrenci o in okunan)
{
    Console.WriteLine($"{o.Ad,-12}{o.Ortalama,8:F2}");
}
Console.WriteLine($"Sınıf ortalaması: {okunan.Average(o => o.Ortalama):F2}");

static void CsvYaz(string yol, List<Ogrenci> liste)
{
    List<string> satirlar = ["ad;vize;final"];
    foreach (Ogrenci o in liste)
    {
        satirlar.Add($"{o.Ad};{o.Vize:F1};{o.Final:F1}");
    }
    File.WriteAllLines(yol, satirlar);
}

static List<Ogrenci> CsvOku(string yol)
{
    List<Ogrenci> sonuc = [];
    string[] satirlar = File.ReadAllLines(yol);
    for (int i = 1; i < satirlar.Length; i++)   // 0. satır başlık
    {
        string[] p = satirlar[i].Split(';');
        double vize = Convert.ToDouble(p[1]);
        double fin = Convert.ToDouble(p[2]);
        sonuc.Add(new Ogrenci(p[0], vize, fin));
    }
    return sonuc;
}
```

```
class Ogrenci
{
    public string Ad { get; }
    public double Vize { get; }
    public double Final { get; }
    public double Ortalama => Vize * 0.4 + Final * 0.6;

    public Ogrenci(string ad, double vize, double fin)
    {
        Ad = ad;
        Vize = vize;
        Final = fin;
    }
}
```

Çıktı

```
ad;vize;final
Ayşe Kaya;72,5;85,0
Burak Demir;45,0;38,5
Cem Yıldız;90,0;95,0
Ayşe Kaya      80,00
Burak Demir    41,10
Cem Yıldız     93,00
Sınıf ortalaması: 71,37
```

`CsvYaz` ilk satıra başlığı yazar. `CsvOku` başlığı (0. satır) `i = 1` ile baştan atlar; her satırı `Split(';')` ile parçalar, sayı alanlarını `Convert.ToDouble` ile çevirir ve `new Ogrenci(...)` ile listeye ekler. `Ortalama` bir **hesaplanan özellik** olduğu için dosyaya yazılmaz; her seferinde vizeden ve finalden hesaplanır.

> **Sık Yapılan Hata: Sayı biçimi kültüre bağlıdır**
>
> `Convert.ToDouble("72,5")` yalnızca virgülü ondalık ayırıcı kabul eden bir kültürde (tr-TR) doğru çalışır. Aynı dosyayı noktalı bir ayarda (`en-US`) okursan yanlış değer alabilirsin ya da hata alırsın. Kültürden bağımsız okumak için `double.Parse(p[1], CultureInfo.InvariantCulture)` yazılır (`using System.Globalization;` gerekir); ama o zaman dosyadaki sayılar da nokta ile yazılmalıdır.

> **Sınavda Çıkar: `Split` boş parçayı atmaz**
>
> `"a;b;".Split(';')` **üç** parça verir; son parça boş metindir. Boş son alan kaybolmaz. Eksik alanlı bir satırda `p[2]` erişimi `IndexOutOfRangeException` fırlatır; erişmeden önce `p.Length` kontrol et.

14.8 Satır ve kelime sayma

Metin dosyasının satır ve kelime sayısını bulmak için `ReadLine` döngüsü yeterlidir. Boş satır da bir satırdır, ama kelime sayısını artırmaz:

metin.txt

```
Dosyalar verileri kalıcı kılar.
Her satır bir kayıttır.

Kelimeleri boşlukla sayarız.
```

satirKelime.cs

```
int satirSayisi = 0;
int kelimeSayisi = 0;
using (StreamReader okuyucu = new StreamReader("metin.txt"))
{
    string? satir;
    while ((satir = okuyucu.ReadLine()) != null)
    {
        satirSayisi++;
        var kelimeler = satir.Split(' ', StringSplitOptions.RemoveEmptyEntries);
        kelimeSayisi += kelimeler.Length;
    }
}
Console.WriteLine($"Satır: {satirSayisi}, kelime: {kelimeSayisi}");
```

Çıktı

```
Satır: 4, kelime: 11
```

14.9 Klasörler

`Directory` sınıfı klasör işlerini yapar. `Directory.CreateDirectory` klasör yoksa oluşturur, varsa hata vermez. `Directory.GetFiles` bir klasördeki dosyaların **yollarını** verir; sırası garanti değildir, bu yüzden sıralamayı kendin yaparsın. Çıktıda tam yol yerine `Path.GetFileName` ile yalnızca dosya adını yazdırıyoruz:

klasorIslemleri.cs

```
Directory.CreateDirectory("veriler");
File.WriteAllText(Path.Combine("veriler", "b.txt"), "B");
File.WriteAllText(Path.Combine("veriler", "a.txt"), "A");
File.WriteAllText(Path.Combine("veriler", "c.csv"), "C");

string[] dosyalar = Directory.GetFiles("veriler", "*.txt");
foreach (string dosya in dosyalar.OrderBy(d => d))
{
    Console.WriteLine(Path.GetFileName(dosya));
}
```

Çıktı

```
a.txt
b.txt
```

14.10 async/await ile dosya işlemleri

Büyük bir dosyayı ya da yavaş bir diski okurken program o satırda bekler. Konsol programında bu önemsizdir, ama pencereli bir uygulamada bekleme **arayüzün donması** demektir. **Asenkron** (async) çalışmada program beklerken başka işlere devam edebilir; iş bitince kaldığı yerden sürer. C#'ta üç parça vardır:

- **`Task<T>`**: "işlem sürüyor, sonuç sonra gelecek" sözüdür. `Task<string>` bitince bir `string` verir; sonuç dönmeyen işler için `Task` kullanılır.
- **`await`**: Görevin bitmesini bekler (programı kilitlemeden) ve sonucunu verir.
- **`async`**: İçinde `await` kullanan metoda yazılır; dönüş tipi `Task` ya da `Task<T>` olur.

`File` sınıfının asenkron karşılıkları adın sonuna `Async` ekler: `WriteAllTextAsync`, `ReadAllTextAsync`, `ReadAllLinesAsync`, `AppendAllTextAsync`. Üst düzey deyimlerde `await` doğrudan yazılabilir; ayrıca bir `async Main` yazman gerekmez.

asenkronDosya.cs

```
await File.WriteAllTextAsync("gunluk.txt", "Birinci satır\nİkinci satır\n");
Console.WriteLine("Yazıldı");
string icerik = await File.ReadAllTextAsync("gunluk.txt");
Console.Write(icerik);

int sayi = await SatirSayAsync("gunluk.txt");
Console.WriteLine($"{sayi} satır");

await File.WriteAllTextAsync("a.txt", "A dosyası");
await File.WriteAllTextAsync("b.txt", "B dosyası");
Task<string> ta = File.ReadAllTextAsync("a.txt");   // iki okuma birlikte başlar
Task<string> tb = File.ReadAllTextAsync("b.txt");
string[] hepsi = await Task.WhenAll(ta, tb);
Console.WriteLine(string.Join(" | ", hepsi));

static async Task<int> SatirSayAsync(string yol)
{
    string[] satirlar = await File.ReadAllLinesAsync(yol);
    return satirlar.Length;
}
```

Çıktı

```
Yazıldı
Birinci satır
İkinci satır
2 satır
A dosyası | B dosyası
```

`SatirSayAsync` metodu `async` olduğu için dönüş tipi `Task<int>`'tir; `return satirlar.Length;` satırında yine düz bir `int` döndürürsün, `Task` sarmalamasını derleyici yapar. Metodun adının sonuna `Async` yazmak bir gelenektir. `Task.WhenAll` birden çok görevin hepsinin bitmesini bekler ve sonuçları, verdiğin **sırayla** bir dizide getirir.

● Sık Yapılan Hata: await unutmak

`ReadAllTextAsync` bir `string` değil, `Task<string>` döndürür. `await` yazmazsan elinde sonuç değil "söz" kalır ve `string` değişkene atama derlenmez (CS0029):

```
string icerik = File.ReadAllTextAsync("gunluk.txt");
Console.WriteLine(icerik);
```

Derleyici çıktısı – program derlenmez

```
awaitUnutuldu.cs(1,17): error CS0029: Cannot implicitly convert type
'System.Threading.Tasks.Task<string>' to 'string'
```

● Sınavda Çıkar: async neyi hızlandırır?

Art arda yazılmış `await` satırları yine **sırayla** çalışır; asenkron yazmak tek bir işi hızlandırmaz. Kazanç, beklerken arayüzün (ya da başka işlerin) kilitlenmemesi ve bağımsız işlerin `Task.WhenAll` ile birlikte yürümesidir.

14.11 JSON ile kaydetme: System.Text.Json

CSV yalnızca düz tabloları taşır; her alanın tipini ve iç içe verileri sen çözersin. **JSON** (JavaScript Object Notation) nesneleri `{"Ad":"Çay","Fiyat":12.5}` gibi okunaklı bir metinle saklayan, programlar arası veri alışverişinin ortak biçimidir. Nesneyi JSON metnine çevirmeye **serileştirme** (serialization), geri nesneye çevirmeye **seri durumdan çıkarma** (deserialization) denir. İki metot yeter:

- `JsonSerializer.Serialize(nesne)` → JSON `string`
- `JsonSerializer.Deserialize<T>(json)` → `T` nesnesi (JSON metni `null` ise sonuç `null` olur; bu yüzden dönüş tipi `T?`'dir)

`JsonSerializer`, `System.Text.Json` ad alanındadır; bu ad alanı otomatik gelmez, dosyanın en üstüne `using System.Text.Json;` yazarsın. Aşağıdaki program bir `record` listesini dosyaya kaydedip geri okur. İlk satır, ayar vermediğimizde Türkçe harflerin nasıl yazıldığını gösterir:

jsonKayit.cs

```
using System.Text.Encodings.Web;
using System.Text.Json;
using System.Text.Unicode;

List<Urun> urunler =
[
    new Urun("Çay", 12.5m, 40),
    new Urun("Şeker", 30m, 15),
];

Console.WriteLine(JsonSerializer.Serialize(urunler[0]));

var secenek = new JsonSerializerOptions
{
    WriteIndented = true,
    Encoder = JavaScriptEncoder.Create(UnicodeRanges.All),
};
string json = JsonSerializer.Serialize(urunler, secenek);
await File.WriteAllTextAsync("urunler.json", json);
Console.WriteLine(json);

string okunan = await File.ReadAllTextAsync("urunler.json");
List<Urun> geri = JsonSerializer.Deserialize<List<Urun>>(okunan) ?? [];
foreach (Urun u in geri)
{
    Console.WriteLine($"{u.Ad,-8}{u.Fiyat,8:F2}{u.Stok,5}");
}

record Urun(string Ad, decimal Fiyat, int Stok);
```

Çıktı

```
{"Ad":"\u00C7ay","Fiyat":12.5,"Stok":40}
[
  {
    "Ad": "Çay",
    "Fiyat": 12.5,
    "Stok": 40
  },
  {
    "Ad": "Şeker",
    "Fiyat": 30,
    "Stok": 15
  }
]
Çay        12,50   40
Şeker      30,00   15
```

- Ayarsız `Serialize` Türkçe harfleri `\u00C7ay` gibi kaçış dizisine çevirir. Bu geçerli JSON'dur ve geri okununca `Çay` olur, ama dosyayı gözle okumak zordur. `Encoder = JavaScriptEncoder.Create(UnicodeRanges.All)` harflerin olduğu gibi yazılmasını sağlar; `WriteIndented = true` ise çıktıyı girintili ve satır satır yapar.
- JSON sayıları **her kültürde nokta** ile yazar (`12.5`). Böylece CSV'deki "virgül mü nokta mı" sorunu burada yoktur; ekranda gördüğün `12,50` ise `F2` biçiminin tr-TR kültüründe yazılmasından gelir.
- Bozuk bir JSON metnini okumaya çalışırsan `JsonException` fırlar.

Sık Yapılan Hata: Alan değil, özellik yaz

`JsonSerializer` varsayılan olarak yalnızca **public özellikleri** (`{ get; set; }`) yazar; public **alanlar** sessizce atlanır. Aşağıda `Renk` alandır, kayboldu:

```
using System.Text.Json;

Kalem k = new Kalem();
Console.WriteLine(JsonSerializer.Serialize(k));

class Kalem
{
    public string Renk = "mavi";
    public int Uzunluk { get; set; } = 15;
}
```

Çıktı

```
{"Uzunluk":15}
```

`record Urun(string Ad, ...)` gibi konumsal bir kayıtta parametreler zaten özellik olduğu için bu sorun çıkmaz.

Bölüm Özeti – Bilmen Gerekenler

- RAM'deki veri program bitince kaybolur; kalıcılık için dosyaya yaz. Dosya işleri `System.IO` içindedir: `File`, `Directory`, `Path`, `StreamReader`, `StreamWriter`.
- `"notlar.txt"` göreli yoldur ve **çalışma klasörüne** göre çözülür (Visual Studio'da `bin\Debug\net10.0`). Windows yolunu `@"C:\..."` (verbatim) ya da `"C:\\..."` olarak yaz.
- `File.WriteAllText` ve `WriteAllLines` eski içeriği siler; eklemek için `AppendAllText` ve `AppendAllLines` kullan.
- `ReadAllText` tek `string`, `ReadAllLines` `string[]` verir. Olmayan dosya `FileNotFoundException` fırlatır; bu bir `IOException`'dır.
- Satır satır okuma: `while ((satir = okuyucu.ReadLine()) != null)`. `ReadLine` dosya sonunda `null` döndürür.
- `StreamReader` ve `StreamWriter`'ı `using` ile aç; kapanmayan yazıcının tamponu diske ulaşmayabilir.
- `new StreamWriter(yol)` dosyayı baştan yazar; eklemek için `new StreamWriter(yol, true)`.
- Dosyaya yazdıktan sonra geri okuyup doğrula.
- UTF-8'de `ç ğ ı ö ş ü` 2 bayt yer kaplar. .NET varsayılan olarak UTF-8 kullanır.
- CSV: `;` ile yaz, `Split(';')` ile ayır, `Convert.ToDouble` ile sayıya çevir. `Split` sondaki boş parçayı atmaz.
- Asenkron: `await File.WriteAllTextAsync/ReadAllTextAsync(...)` (üst düzey deyimlerde doğrudan); `await` yoksa elinde `Task<string>` kalır (CS0029). JSON: `JsonSerializer.Serialize` / `Deserialize<T>` (`using System.Text.Json;`), yalnız public **özellikleri** yazar; sayılar her kültürde noktalıdır.

Sorular

A) Çoktan Seçmeli

1 C# programında `string s = File.ReadAllText("a.txt");` satırı hiçbir `try-catch` olmadan yazılmış. `a.txt` yoksa ne olur?

(A) Derleme hatası: `IOException` yakalanmalı ya da bildirilmelidir.

(B) `s` değişkenine `null` atanır.

(C) Program derlenir; çalışırken `FileNotFoundException` ile çöker.

(D) `s` değişkenine boş metin (`""`) atanır.

(E) Derleme hatası: `File` sınıfı bu yerde kullanılamaz.

2 `log.txt` dosyası **yoksa oluşturulsun, varsa sonuna** `giriş` satırı eklensin. Hangisi doğrudur?

(A) `File.WriteAllText("log.txt", "giriş\n");`

(B) `File.ReadAllText("log.txt", "giriş\n");`

(C) `new StreamWriter("log.txt").WriteLine("giriş");`

(D) `File.AppendAllLines("log.txt");`

(E) `File.AppendAllText("log.txt", "giriş\n");`

3 `StreamReader` nesnesinin `ReadLine()` metodu dosyanın sonuna gelindiğinde ne döndürür?

A) `null`
B) Boş metin (`""`)
C) `-1`
D) `EndOfStreamException` fırlatır.
E) `"\n"`

4 `Console.WriteLine("x,,y,".Split(',').Length);` satırı ne yazdırır?

A) 2
B) 3
C) 5
D) 4
E) Çalışma zamanında `IndexOutOfRangeException` fırlatır.

5 `using` ile ilgili aşağıdakilerden hangisi **doğrudur**?

A) Kaynak yalnızca `using` bloğu hatasız biterse kapatılır.
B) `using` bloğu normal bitse de exception fırlatılsa da kaynağın `Dispose()` metodunu çağırır.
C) `using` yalnızca `FileStream` ile kullanılabilir.
D) Bir `using` içinde açılan kaynaklar, açılış sırasıyla kapatılır.
E) Bir `using` içinde yalnızca bir kaynak açılabilir.

B) Çıktı Tahmini

6 Aşağıdaki program ekrana ne yazdırır? (Çalışma klasöründe `s6.txt` dosyası yoktur.)

```
File.WriteAllText("s6.txt", "bir\niki\n");
File.AppendAllText("s6.txt", "üç\n");
File.AppendAllLines("s6.txt", new[] { "dört", "beş" });
string[] satirlar = File.ReadAllLines("s6.txt");
Console.WriteLine(satirlar.Length + " " + string.Join("|", satirlar));
File.WriteAllText("s6.txt", "altı");
Console.WriteLine(File.ReadAllLines("s6.txt").Length);
Console.WriteLine(File.ReadAllText("s6.txt").Length);
```

7 Aşağıdaki program ekrana ne yazdırır?

```
string[] satirlar = { "a;10", "b; 20", "c;x", "d", "e;5;" };
int toplam = 0;
int hata = 0;
foreach (string satir in satirlar)
{
    string[] parca = satir.Split(';');
    try
    {
        toplam += Convert.ToInt32(parca[1].Trim());
    }
    catch (FormatException)
    {
        hata++;
    }
    catch (IndexOutOfRangeException)
    {
        hata += 10;
    }
}
Console.WriteLine(toplam + " " + hata);
Console.WriteLine("e;5;".Split(';').Length);
Console.WriteLine("b; 20".Split(';')[1].Trim().Length);
```

8 Aşağıdaki program ekrana ne yazdırır?

```
try
{
    using var a = new Kaynak("dosya");
    using var b = new Kaynak("tampon");
    Console.WriteLine("okunuyor");
    throw new InvalidOperationException("bozuk satır");
}
catch (InvalidOperationException e)
{
    Console.WriteLine("yakalandı: " + e.Message);
}
finally
{
    Console.WriteLine("finally");
}

class Kaynak : IDisposable
{
    private readonly string _ad;

    public Kaynak(string ad)
    {
        _ad = ad;
        Console.WriteLine(ad + " açıldı");
    }

    public void Dispose()
    {
        Console.WriteLine(_ad + " kapandı");
    }
}
```

C) Kod Yazma

9 Çalışma klasöründeki `sayilar.txt` dosyasında her satırda bir tam sayı olması gerekiyor; ama bazı satırlar boş ya da bozuk. `StreamReader` ile dosyayı satır satır okuyan programı yaz. Program:

- boş satırları sessizce atlasın,
- geçersiz satırları **satır numarasıyla** bildirsin,
- sonunda geçerli sayıların adedini, toplamını, en büyüğünü ve ortalamasını (2 basamak) yazdırsın.

Sayı ayrıştırmak için `int.TryParse` kullan (Bölüm 12).

sayilar.txt

```
15
8
abc
42

-3
7.5
20
```

10 `ogrenciler.csv` dosyasında öğrencilerin adı, vizesi ve finali `;` ile ayrılmış. Final notlarından biri virgüllü yazılmış (`50,5`). `sonuc.csv` dosyasını yazan programı yaz:

- Her öğrencinin ortalamasını hesaplasın (vize %40 + final %60),
- `sonuc.csv` dosyasına `ad;ortalama;durum` başlığıyla yazsın (ortalama 2 basamak; 50 ve üstü `GEÇTİ`, altı `KALDI`),
- sonra `sonuc.csv` dosyasını geri okuyup ekrana basarak doğrulasın.

ogrenciler.csv

```
ad;vize;final
Ali Yılmaz;60;70
Buse Er;30;40
Can Tan;80;45
Derya Ak;55;50,5
```

Çözümler

Cevap anahtarı

1 C	2 E	3 A	4 D	5 B

1 Cevap: C

C#'ta derleyici istisna yakalamanı zorunlu tutmaz; metot imzasında bildirim de gerekmez. Kod derlenir. `a.txt` yoksa `File.ReadAllText` çalışma anında `FileNotFoundException` fırlatır ve program çöker. A yanlıştır. B ve D yanlıştır: hata durumunda değişkene değer atanmaz. E yanlıştır; `File` sınıfı `System.IO` içindedir ve projede hazır gelir.

2 Cevap: E

`File.AppendAllText` dosya yoksa oluşturur, varsa sonuna ekler; istenen davranış budur. A dosyanın içeriğini siler. B derlenmez: `ReadAllText`'in ikinci parametresi kodlamadır, metin değil. C dosyayı baştan yazar ve akışı kapatmadığı için satır diske ulaşmayabilir. D derlenmez: `AppendAllLines` iki parametre ister, burada satır listesi verilmemiş.

3 Cevap: A

`ReadLine()` dosya sonunda `null` döndürür; okuma döngüsünün `!= null` koşulu bu yüzdendir. Boş metin (B) dosyadaki **boş bir satırı** temsil eder. `-1` (C), karakter karakter okuyan `Read()` metodunun dosya sonu değeridir. D yanlıştır; `ReadLine` istisna fırlatmaz. E yanlıştır; satır sonu karakteri döndürülmez.

4 Cevap: D

`"x,,y,".Split(',')` dört parça üretir: `"x"`, `""`, `"y"` ve `""`. C#'ta `Split` boş parçaları **silmez**; sondaki boş parça da sayılır. Boş parçaları atmak için ikinci parametre olarak `StringSplitOptions.RemoveEmptyEntries` verilir. A (2) ve B (3) yanlıştır; C (5) de yanlıştır. E yanlıştır; boş parça dizin hatası üretmez.

Çıktı

```
4
```

5 Cevap: B

`using` bloğu, içindeki kod normal bitse de exception ile çıksa da `Dispose()` çağrılmasını garanti eder. A yanlıştır: exception olsa bile kapatılır. C yanlıştır: `using`, `IDisposable` uygulayan her nesneyle kullanılabilir. D yanlıştır: kaynaklar açılış sırasının **tersine** kapatılır. E yanlıştır: bir `using` içinde birden fazla kaynak açılabilir (`using var a = ...; using var b = ...;`).

6

- `WriteAllText` dosyayı oluşturur: `bir`, `iki`.
- `AppendAllText` ile `üç` eklenir; `AppendAllLines` iki satır daha ekler, toplam 5 satır olur.
- İlk satır `5 bir|iki|üç|dört|beş` olarak yazılır.
- `WriteAllText` eski içeriği **siler**; dosyada yalnızca `altı` kalır (satır sonu yok). `ReadAllLines` tek satır döndürür: `1`.
- `ReadAllText(...).Length` karakter sayısını verir: `altı` 4 karakterdir.

```
Çıktı
5 bir|iki|üç|dört|beş
1
4
```

7

Satır satır:

Satır	`Split(';')` sonucu	Ne olur?
`a;10`	`"a"`, `"10"`	toplam 10
`b; 20`	`"b"`, `" 20"`; `Trim()` ile 20	toplam 30
`c;x`	`"c"`, `"x"`	`FormatException`, hata 1
`d`	`"d"` tek parça; `parca[1]` yok	`IndexOutOfRangeException`, hata 11
`e;5;`	`"e"`, `"5"`, `""`	toplam 35

Çıktının ikinci satırı: `"e;5;"` üç parçaya bölünür, sondaki boş parça kalır. Üçüncü satır: `b; 20` satırının ikinci parçası boşluklu `" 20"` metnidir; `Trim()` sonrasında 2 karakter kalır.

```
Çıktı
35 11
3
2
```

8

1. `try` bloğuna girilir. `dosya` açılır, sonra `tampon` açılır.
2. `okunuyor` yazılır ve exception fırlatılır.
3. `catch`'e geçmeden **önce** kaynaklar açılış sırasının tersiyle kapanır: önce `tampon`, sonra `dosya`.
4. `catch` çalışır, en son `finally`.

```
Çıktı
dosya açıldı
tampon açıldı
okunuyor
tampon kapandı
dosya kapandı
yakalandı: bozuk satır
finally
```

9

Kalıp, `while ((satir = okuyucu.ReadLine()) != null)` döngüsüdür. Boş satır `Trim()` sonrası boş metin olduğu için `continue` ile atlanır. `int.TryParse` başarısız olursa satır numarasıyla bildirilir. Ortalamayı yalnızca en az bir geçerli sayı varsa hesaplıyoruz (`adet == 0` kontrolü); böylece sıfıra bölme olmaz. Ortalama tr-TR ayarında virgülle yazılır.

sayiDosyasi.cs

```
int adet = 0;
int toplam = 0;
int enBuyuk = int.MinValue;
int no = 0;

using (StreamReader okuyucu = new StreamReader("sayilar.txt"))
{
    string? satir;
    while ((satir = okuyucu.ReadLine()) != null)
    {
        no++;
        satir = satir.Trim();
        if (satir == "")
        {
            continue;
        }
        if (!int.TryParse(satir, out int sayi))
        {
            Console.WriteLine("Satır " + no + " geçersiz: " + satir);
            continue;
        }
        adet++;
        toplam += sayi;
        enBuyuk = Math.Max(enBuyuk, sayi);
    }
}

if (adet == 0)
{
    Console.WriteLine("Geçerli sayı yok.");
}
else
{
    Console.WriteLine($"Geçerli sayı: {adet}");
    Console.WriteLine($"Toplam: {toplam}, en büyük: {enBuyuk}");
    Console.WriteLine($"Ortalama: {(double)toplam / adet:F2}");
}
```

Çıktı

```
Satır 3 geçersiz: abc
Satır 7 geçersiz: 7.5
Geçerli sayı: 5
Toplam: 82, en büyük: 42
Ortalama: 16,40
```

10

Girdiyi `File.ReadAllLines` ile okuyoruz; `i = 0` başlık olduğu için döngü 1'den başlıyor. Final notu `Convert.ToDouble` ile okunuyor (`50,5`). Ortalama `F2` ile yazılıyor; tr-TR ayarında ondalık ayırıcı virgül olur ve Türkçe Excel, `;` ayırıcılı bir dosyada bunu bekler. Son olarak `sonuc.csv` geri okunup ekrana basılıyor.

sonucDosyasi.cs

```
List<string> satirlar = ["ad;ortalama;durum"];

string[] girdi = File.ReadAllLines("ogrenciler.csv");
for (int i = 1; i < girdi.Length; i++)
{
    string[] p = girdi[i].Split(';');
    double vize = Convert.ToDouble(p[1]);
    double fin = Convert.ToDouble(p[2]);
    double ort = vize * 0.4 + fin * 0.6;
    string durum = ort >= 50 ? "GEÇTİ" : "KALDI";
    satirlar.Add($"{p[0]};{ort:F2};{durum}");
}

File.WriteAllLines("sonuc.csv", satirlar);
Console.Write(File.ReadAllText("sonuc.csv"));
```

Çıktı

```
ad;ortalama;durum
Ali Yılmaz;66,00;GEÇTİ
Buse Er;36,00;KALDI
Can Tan;59,00;GEÇTİ
Derya Ak;52,30;GEÇTİ
```

BÖLÜM 15

Özyineleme (Recursion)

Bu Bölümde Öğreneceklerin

- Özyinelemenin ne olduğunu ve iki parçasını: **temel durum** (base case), **özyinelemeli durum**
- Çağrı yığınını (call stack) iniş ve çıkış adımlarıyla izlemeyi
- Faktöriyel, üs alma, basamak toplamı, EBOB ve ikilik sayıya çevirme örneklerini
- Fibonacci'nin saf halinin neden yavaş olduğunu ve `Dictionary` ile hafızalamayı
- String ve dizilerde indeksli özyinelemeyi; Hanoi ile özyinelemeli ikili aramayı
- Temel durum unutulunca oluşan `Stack overflow.` hatasını ve çıktı sorularını çözmeyi

15.1 Özyineleme nedir?

Özyineleme (recursion), bir metodun **kendi kendini çağırmasıdır**. Kulağa garip gelir: "Metot daha bitmeden kendini nasıl çağırır?" Cevap: Her çağrı problemin **daha küçük** bir halini çözer ve en sonunda cevabı doğrudan bilinen çok küçük bir duruma ulaşılır.

Günlük hayattan bir örnek: Sinemada uzun bir kuyrukta kaçıncı sırada olduğunu merak ediyorsun. Herkesi tek tek saymak yerine önündekine soruyorsun: "Sen kaçıncısın?" O da önündekine soruyor, o da önündekine... En öndeki kişinin önünde kimse yok; o hemen **"1"** diyor. Sonra cevaplar geri dönüyor: Arkasındaki "2", onun arkasındaki "3"... Sana cevap gelince sen de 1 ekliyorsun.

Bu küçük hikâyede özyinelemenin bütün parçaları var:

- **Aynı soru, daha küçük problem:** Herkes kendinden bir kişi kısa bir kuyruk için aynı soruyu soruyor.
- **Durma noktası:** En öndeki kişi kimseye sormadan "1" diyor.
- **İniş ve çıkış:** Sorular öne gidiyor (iniş), cevaplar arkaya dönüyor (çıkış).

İlk özyinelemeli metodumuz geriye sayıyor:

geriSayim.cs

```
GeriSay(3);

static void GeriSay(int n)
{
    if (n == 0)
    {
        Console.WriteLine("Ateş!");     // durma noktası
        return;
    }
    Console.WriteLine(n);
    GeriSay(n - 1);                     // kendini daha küçük n ile çağırır
}
```

Çıktı
```
3
2
1
Ateş!
```

`GeriSay(3)` önce `3` yazar, sonra `GeriSay(2)`'yi çağırır; o `2` yazıp `GeriSay(1)`'i çağırır... `GeriSay(0)` ise kendini çağırmaz, `Ateş!` yazıp döner. Döngü kullanmadan tekrar elde ettik.

15.2 Temel durum ve özyinelemeli durum

Doğru yazılmış her özyinelemeli metot iki parçadan oluşur:

1. **Temel durum** (base case): Cevabı **doğrudan** bilinen, metodun kendini **çağırmadığı** en küçük durum. Özyinelemeyi durduran fren budur.
2. **Özyinelemeli durum** (recursive case): Problemi **bir adım küçültüp** metodu tekrar çağıran, gelen cevabı kullanarak kendi cevabını kuran kısım. Her çağrı temel duruma **yaklaşmalıdır**.

Klasik örnek **faktöriyel**: `n! = n × (n−1) × ... × 1` ve `0! = 1`. `5! = 5 × 4!`, `4! = 4 × 3!`... Yani faktöriyel kendisi cinsinden tanımlanabilir:

Matematik	C#
`n ≤ 1` ise `n! = 1`	`if (n <= 1) return 1;` → temel durum
`n > 1` ise `n! = n × (n−1)!`	`return n * Faktoriyel(n - 1);` → özyinelemeli durum

faktoriyel.cs
```
for (int i = 0; i <= 5; i++)
{
    Console.WriteLine($"{i}! = {Faktoriyel(i)}");
}
Console.WriteLine($"20! = {Faktoriyel(20)}");
Console.WriteLine($"21! = {Faktoriyel(21)}");

static long Faktoriyel(int n)
{
    if (n <= 1)
    {
        return 1;                       // temel durum
    }
    return n * Faktoriyel(n - 1);       // özyinelemeli durum
}
```

Çıktı
```
0! = 1
1! = 1
2! = 2
3! = 6
4! = 24
5! = 120
20! = 2432902008176640000
21! = -4249290049419214848
```

Dönüş tipini `long` seçtik, çünkü faktöriyel çok hızlı büyür: `13!` zaten `int`'e sığmaz. `20!` `long`'a sığan en büyük faktöriyeldir. `21!` ise `long`'un sınırını aşar; C#'ta taşma **sessizdir**. Program hata vermez, yanlış bir sayı yazar (Bölüm 2).

> **İpucu: Özyinelemeli metot yazarken üç soru**
>
> 1. **En küçük durum ne ve cevabı ne?** (0, 1, boş string, dizinin sonu...) → temel durum.
> 2. **Problemi bir adım nasıl küçültürüm?** (`n - 1`, `n / 10`, `s.Substring(1)`, `i + 1`...)
> 3. **Küçük problemin cevabı elimdeyse, büyüğün cevabını nasıl kurarım?** (`n * ...`, `... + n % 10`...)

15.3 Çağrı yığını (call stack)

Bir metot çağrıldığında .NET, o çağrı için bellekte bir **çerçeve** (stack frame) ayırır. Çerçevede metodun parametreleri, yerel değişkenleri ve "iş bitince nereye dönüleceği" bilgisi tutulur. Çerçeveler üst üste bir **yığın** (stack) halinde dizilir: Yeni çağrı en üste konur, biten çağrı en üstten kaldırılır. Bu düzene **LIFO** (Last In, First Out: son giren ilk çıkar) denir; üst üste konmuş tabaklar gibi.

Özyinelemede aynı metodun **birçok çerçevesi** aynı anda yığında bulunur ve her birinin kendi `n`'i vardır. `Faktoriyel(4)` çağrısını adım adım izleyelim (kısaca `f` yazıyoruz):

Adım	Olay	Yığındaki çerçeveler (alttan üste)	Bekleyen işlem
1	İniş: `f(4)` çağrıldı	Main, f(4)	`4 * f(3)`
2	İniş: `f(3)` çağrıldı	Main, f(4), f(3)	`3 * f(2)`
3	İniş: `f(2)` çağrıldı	Main, f(4), f(3), f(2)	`2 * f(1)`
4	Dip: `f(1)` temel durum	Main, f(4), f(3), f(2), f(1)	`1` döner
5	Çıkış: `f(2)` devam eder	Main, f(4), f(3), f(2)	`2 * 1 = 2` döner
6	Çıkış: `f(3)` devam eder	Main, f(4), f(3)	`3 * 2 = 6` döner
7	Çıkış: `f(4)` devam eder	Main, f(4)	`4 * 6 = 24` döner

İnişte hiçbir çarpma yapılmaz; her çerçeve bir alttakinin cevabını **bekler**. Hesaplama çıkışta, temel durumdan geriye doğru yapılır.

Aynı şeyi programın kendisine anlatalım. Her çağrı, derinliği kadar girintiyle ne yaptığını yazsın:

faktoriyelIz.cs

```
long f = FaktoriyelIz(4, 0);
Console.WriteLine($"Sonuç: {f}");

static long FaktoriyelIz(int n, int derinlik)
{
    string girinti = new string(' ', 2 * derinlik);
    Console.WriteLine($"{girinti}faktoriyel({n}) çağrıldı");
    if (n <= 1)
    {
        Console.WriteLine($"{girinti}temel durum -> 1 döner");
        return 1;
    }
    long sonuc = n * FaktoriyelIz(n - 1, derinlik + 1);
    Console.WriteLine($"{girinti}faktoriyel({n}) = {sonuc} döner");
    return sonuc;
}
```

Çıktı

```
faktoriyel(4) çağrıldı
  faktoriyel(3) çağrıldı
    faktoriyel(2) çağrıldı
      faktoriyel(1) çağrıldı
      temel durum -> 1 döner
    faktoriyel(2) = 2 döner
  faktoriyel(3) = 6 döner
faktoriyel(4) = 24 döner
Sonuç: 24
```

Çıktının şekline dikkat et: Girinti önce artıyor (iniş), sonra azalıyor (çıkış). Çağrıdan **önceki** satır (`çağrıldı`) büyükten küçüğe, çağrıdan **sonraki** satır (`döner`) küçükten büyüğe yazıldı. Bu gözlem, bölümün sonundaki çıktı sorularının anahtarı.

> **Sık Yapılan Hata: Problem küçülmüyorsa özyineleme bitmez**
>
> `return n * Faktoriyel(n);` yazarsan (yani `n - 1` yerine `n`), her çağrı **aynı** problemi tekrar sorar ve temel duruma hiç ulaşılmaz. Bir diğer sık hata, özyinelemeli çağrının sonucunu kullanmayı unutmaktır: `Faktoriyel(n - 1);` satırı tek başına yazılırsa hesaplanan değer çöpe gider. Değer döndüren özyinelemede çağrı mutlaka bir `return` ifadesinin içinde kullanılmalıdır.

15.4 Sayılarla özyineleme

Basamak toplamı, EBOB ve ikilik sayıya çevirme

Bölüm 2'de gördüğün `n % 10` (son basamak) ve `n / 10` (son basamağı at) ikilisi, sayılarla özyinelemenin temelidir:

- **Üs:** $a^n = a \times a^{n-1}$, $a^0 = 1$. `UsAl(2, 10)` toplam 11 kez çağrılır.
- **Basamak toplamı:** `4728`'in basamak toplamı = `8` + `472`'nin basamak toplamı. Tek basamaklı sayıda cevap sayının kendisidir.
- **EBOB (Öklid algoritması):** `Ebob(a, b) = Ebob(b, a % b)`, `b` sıfır olunca cevap `a`'dır.
- **İkilik sayıya çevirme:** `n`'in ikilik gösterimi = `n / 2`'nin ikilik gösterimi + son bit (`n % 2`).

sayiOzyineleme.cs

```
Console.WriteLine($"basamakTopla(4728) = {BasamakTopla(4728)}");
Console.WriteLine($"ebob(48, 18) = {Ebob(48, 18)}");
Console.WriteLine($"ekok(48, 18) = {48 * 18 / Ebob(48, 18)}");
Console.WriteLine($"ikilik(13) = {Ikilik(13)}");
Console.WriteLine($"ikilik(64) = {Ikilik(64)}");
Console.WriteLine($"2^10 = {UsAl(2, 10)}");
Console.WriteLine($"3^4  = {UsAl(3, 4)}");
Console.WriteLine($"5^0  = {UsAl(5, 0)}");

static int BasamakTopla(int n)
{
    if (n < 10)
    {
        return n;
    }
    return n % 10 + BasamakTopla(n / 10);
}

static int Ebob(int a, int b)
{
    if (b == 0)
```

```
    {
        return a;
    }
    return Ebob(b, a % b);
}

static string Ikilik(int n)
{
    if (n < 2)
    {
        return n.ToString();            // 0 veya 1: tek bit
    }
    return Ikilik(n / 2) + n % 2;       // önce baştaki bitler, sonra son bit
}

static long UsAl(long a, int n)
{
    if (n == 0)
    {
        return 1;                       // a^0 = 1
    }
    return a * UsAl(a, n - 1);          // a^n = a * a^(n-1)
}
```

Çıktı

```
basamakTopla(4728) = 21
ebob(48, 18) = 6
ekok(48, 18) = 144
ikilik(13) = 1101
ikilik(64) = 1000000
2^10 = 1024
3^4  = 81
5^0  = 1
```

`Ebob(48, 18)` çağrısının izi:

Çağrı	a	b	a % b
`Ebob(48, 18)`	48	18	12
`Ebob(18, 12)`	18	12	6
`Ebob(12, 6)`	12	6	0
`Ebob(6, 0)`	6	0	temel durum → **6** döner

`Ikilik(13)` ise şöyle açılır: `Ikilik(6) + 1` → `(Ikilik(3) + 0) + 1` → `((Ikilik(1) + 1) + 0) + 1` → `"1" + "1" + "0" + "1"` = `"1101"`. Özyinelemeli çağrı **önce**, son bit **sonra** eklendiği için bitler doğru sırada çıkar. Döngüyle yapsaydın bitler ters sırada elde edilirdi.

15.5 Fibonacci ve tekrar eden hesaplar

Fibonacci dizisinde her terim, kendinden önceki iki terimin toplamıdır: `0, 1, 1, 2, 3, 5, 8, 13, 21, ...` Tanım zaten özyinelemelidir:

- `Fib(0) = 0`, `Fib(1) = 1` (iki temel durum)
- `Fib(n) = Fib(n-1) + Fib(n-2)`

Koda çevirmek kolay. Ama metodun kaç kez çağrıldığını da sayalım. Sayacı `ref` ile gönderiyoruz: Metot içindeki artış, çağıran yerdeki `cagri` değişkenine yansır (Bölüm 6).

fibonacciSayac.cs

```
foreach (int n in new[] { 5, 10, 20, 30 })
{
    int cagri = 0;
    long sonuc = Fib(n, ref cagri);
    Console.WriteLine($"fib({n}) = {sonuc}, çağrı sayısı: {cagri:N0}");
}

static long Fib(int n, ref int cagri)
{
    cagri++;
    if (n <= 1)
    {
        return n;
    }
    return Fib(n - 1, ref cagri) + Fib(n - 2, ref cagri);
}
```

Çıktı

```
fib(5) = 5, çağrı sayısı: 15
fib(10) = 55, çağrı sayısı: 177
fib(20) = 6765, çağrı sayısı: 21.891
fib(30) = 832040, çağrı sayısı: 2.692.537
```

`fib(30)`'u bulmak için metot iki milyondan fazla kez çağrıldı! Neden? `Fib(5)`'in çağrı ağacına bakalım:

```
fib(5)
├── fib(4)
│   ├── fib(3)
│   │   ├── fib(2)
│   │   │   ├── fib(1)
│   │   │   └── fib(0)
│   │   └── fib(1)
│   └── fib(2)
│       ├── fib(1)
│       └── fib(0)
└── fib(3)
    ├── fib(2)
    │   ├── fib(1)
    │   └── fib(0)
    └── fib(1)
```

Her çağrı **iki** yeni çağrı yapıyor ve aynı değerler tekrar tekrar hesaplanıyor: `fib(3)` 2 kez, `fib(2)` 3 kez hesaplandı. `n` bir artınca çağrı sayısı yaklaşık 1,6 katına çıkar. Bu tür büyümeye **üstel** (exponential) büyüme denir.

Hafızalama (memoization)

Çözüm basit: Bir değeri **ilk kez** hesapladığında bir sözlüğe yaz; tekrar istendiğinde hesaplamak yerine sözlükten oku. Buna **hafızalama** (memoization) denir:

fibonacciMemo.cs

```
int cagri = 0;
var hafiza = new Dictionary<int, long>();
long sonuc = FibMemo(30, hafiza, ref cagri);
Console.WriteLine($"fib(30) = {sonuc}, çağrı sayısı: {cagri}");
long fib90 = FibMemo(90, hafiza, ref cagri);
Console.WriteLine($"fib(90) = {fib90}");

static long FibMemo(int n, Dictionary<int, long> hafiza, ref int cagri)
```

```
{
    cagri++;
    if (n <= 1)
    {
        return n;
    }
    if (hafiza.ContainsKey(n))
    {
        return hafiza[n];              // daha önce hesaplandı
    }
    long sonuc = FibMemo(n - 1, hafiza, ref cagri)
               + FibMemo(n - 2, hafiza, ref cagri);
    hafiza[n] = sonuc;
    return sonuc;
}
```

Çıktı

```
fib(30) = 832040, çağrı sayısı: 59
fib(90) = 2880067194370816120
```

Çağrı sayısı 2,7 milyondan **59**'a düştü; `fib(90)` bile anında hesaplandı. Dizi kullanmak da mümkündür (`long[] hafiza = new long[91]`), ama `Dictionary` "bu değeri hesapladım mı?" sorusuna doğrudan `ContainsKey` ile cevap verdiği için burada onu seçtik.

> **Sınavda Çıkar**
>
> "Saf özyinelemeli Fibonacci neden yavaştır?" sorusunun cevabı: **Aynı alt problemler defalarca hesaplanır** (tekrar eden hesaplar); çağrı sayısı üstel artar. Çözüm: hafızalama (sonuçları sözlükte saklamak) veya döngü ile hesaplamak. "`Fib(5)` kaç çağrı yapar?" (15) ve "`Fib(5)` hesaplanırken `Fib(2)` kaç kez çağrılır?" (3) da sık sorulur; ağacı çizerek çöz.

15.6 String ve dizilerle özyineleme

String ters çevirme ve palindrom

String'i küçültmenin yolu `Substring`'dir (Bölüm 8):

- **Ters çevirme:** `"KASA"`'nın tersi = `"ASA"`'nın tersi + `'K'`. Boş veya tek harfli metnin tersi kendisidir.
- **Palindrom:** Baştan ve sondan okunuşu aynı olan kelime (`kayak`). İlk ve son harf aynıysa, **ortadaki kısım** palindrom mu diye bak. Farklıysa hemen `false`.

stringOzyineleme.cs

```
Console.WriteLine(Ters("KASA"));
Console.WriteLine(Ters("özyineleme"));
string[] kelimeler = { "kayak", "ey edip adanada pide ye", "kasa", "abba" };
foreach (string k in kelimeler)
{
    Console.WriteLine($"{k} -> {PalindromMu(k)}");
}

static string Ters(string s)
{
    if (s.Length <= 1)
    {
        return s;
    }
    return Ters(s.Substring(1)) + s[0];
```

```
}

static bool PalindromMu(string s)
{
    if (s.Length <= 1)
    {
        return true;                            // boş veya tek harf
    }
    if (s[0] != s[s.Length - 1])
    {
        return false;                           // uçlar farklı
    }
    return PalindromMu(s.Substring(1, s.Length - 2));    // ortası
}
```

Çıktı

```
ASAK
emeleniyzö
kayak -> True
ey edip adanada pide ye -> True
kasa -> False
abba -> True
```

`Ters("KASA")` çağrısı: `Ters("ASA") + 'K'` → `(Ters("SA") + 'A') + 'K'` → `((Ters("A") + 'S') + 'A') + 'K'` → `"A" + 'S' + 'A' + 'K'` = `"ASAK"`. İlk harf her seferinde **sona** eklendiği için metin ters döner.

Dizide toplam ve en büyük eleman

Dizinin kendisini küçültmek yerine **indeks** ile ilerleriz: `Toplam(d, i)` = "`i` indeksinden sona kadar olan elemanların toplamı". Temel durum dizinin sonu (`i == d.Length`):

diziOzyineleme.cs

```
int[] sayilar = { 4, 9, 2, 7, 5 };
Console.WriteLine($"Toplam: {Toplam(sayilar)}");
Console.WriteLine($"En büyük: {EnBuyuk(sayilar, 0)}");
Console.WriteLine($"3. indeksten itibaren toplam: {Toplam(sayilar, 3)}");

// i. indeksten sona kadar olan elemanların toplamı (i verilmezse 0)
static int Toplam(int[] d, int i = 0)
{
    if (i == d.Length)
    {
        return 0;                       // toplanacak eleman kalmadı
    }
    return d[i] + Toplam(d, i + 1);
}

// i. indeksten sona kadar olan elemanların en büyüğü
static int EnBuyuk(int[] d, int i)
{
    if (i == d.Length - 1)
    {
        return d[i];                    // tek eleman kaldı
    }
    int kalaninEnBuyugu = EnBuyuk(d, i + 1);
    return Math.Max(d[i], kalaninEnBuyugu);
}
```

Çıktı
```
Toplam: 27
En büyük: 9
3. indeksten itibaren toplam: 12
```

Bu kalıp çok yaygındır: Asıl iş **indeks parametresi** alan metotta yapılır; `i` isteğe bağlıdır (Bölüm 6), bu yüzden dışarıdan `Toplam(sayilar)` diye sade çağrılır.

15.7 Hanoi kuleleri

Özyinelemenin gücünü en iyi gösteren problem budur. Üç çubuk var: A, B, C. A çubuğunda büyükten küçüğe dizilmiş `n` disk duruyor. Amaç bütün diskleri C'ye taşımak. Kurallar:

1. Bir hamlede **sadece bir** disk taşınır (en üstteki).
2. Büyük disk asla küçük diskin **üstüne** konamaz.

Döngüyle çözmeye çalışırsan kafan karışır. Özyinelemeyle düşün: En alttaki (en büyük) diski C'ye taşıyabilmek için üstündeki `n - 1` diskin B'de olması gerekir. Yani:

1. Üstteki `n - 1` diski A'dan **B**'ye taşı (C'yi yardımcı kullanarak) → aynı problem, bir disk eksik!
2. En büyük diski A'dan **C**'ye taşı.
3. B'deki `n - 1` diski **C**'ye taşı (A'yı yardımcı kullanarak) → yine aynı problem.

Temel durum: Taşınacak disk yoksa (`n == 0`) hiçbir şey yapma. Hamle sırasını `ref int sira` ile sayıyoruz.

hanoi.cs
```
int sira = 0;
Hanoi(3, 'A', 'C', 'B', ref sira);
Console.WriteLine($"Toplam hamle: {sira}");

static void Hanoi(int n, char kaynak, char hedef, char yardimci, ref int sira)
{
    if (n == 0)
    {
        return;
    }
    Hanoi(n - 1, kaynak, yardimci, hedef, ref sira);   // 1) n-1 diski yardımcıya
    sira++;                                             // 2) en büyüğü hedefe
    Console.WriteLine($"{sira}. hamle: disk {n}  {kaynak} -> {hedef}");
    Hanoi(n - 1, yardimci, hedef, kaynak, ref sira);   // 3) n-1 diski hedefe
}
```

Çıktı
```
1. hamle: disk 1  A -> C
2. hamle: disk 2  A -> B
3. hamle: disk 1  C -> B
4. hamle: disk 3  A -> C
5. hamle: disk 1  B -> A
6. hamle: disk 2  B -> C
7. hamle: disk 1  A -> C
Toplam hamle: 7
```

Hamle sayısını `H(n)` ile gösterelim: `n - 1` diski iki kez taşıyoruz, arada 1 hamle var: `H(n) = 2·H(n-1) + 1`, `H(1) = 1`. Buradan `H(n) = 2ⁿ - 1` çıkar.

Disk sayısı 1, 2, 3, 4, 10, 20 için hamle 1, 3, 7, 15, 1 023, 1 048 575'tir; 64 disk için $\approx 1{,}8 \times 10^{19}$.

15.8 Özyinelemeli ikili arama

Sıralı bir dizide aranan değeri bulmak için **ikili arama** (binary search), diziyi her adımda ikiye böler: Ortadaki elemana bakar; aranan daha küçükse sol yarıda, daha büyükse sağ yarıda aramaya devam eder. "Yarıda aramaya devam et" cümlesi zaten özyinelemelidir:

ikiliAramaOzyinelemeli.cs

```
int[] d = { 3, 8, 12, 17, 23, 31, 38, 45, 52, 60, 71 };
Console.WriteLine("52 aranıyor:");
Console.WriteLine($"Sonuç: {Ara(d, 52, 0, d.Length - 1)}");
Console.WriteLine("5 aranıyor:");
Console.WriteLine($"Sonuç: {Ara(d, 5, 0, d.Length - 1)}");

static int Ara(int[] d, int aranan, int alt, int ust)
{
    if (alt > ust)
    {
        return -1;                      // temel durum 1: aralık boş
    }
    int orta = (alt + ust) / 2;
    Console.WriteLine($"  alt={alt} ust={ust} orta={orta} d[orta]={d[orta]}");
    if (d[orta] == aranan)
    {
        return orta;                    // temel durum 2: bulundu
    }
    if (aranan < d[orta])
    {
        return Ara(d, aranan, alt, orta - 1);       // sol yarı
    }
    return Ara(d, aranan, orta + 1, ust);           // sağ yarı
}
```

Çıktı

```
52 aranıyor:
  alt=0 ust=10 orta=5 d[orta]=31
  alt=6 ust=10 orta=8 d[orta]=52
Sonuç: 8
5 aranıyor:
  alt=0 ust=10 orta=5 d[orta]=31
  alt=0 ust=4 orta=2 d[orta]=12
  alt=0 ust=1 orta=0 d[orta]=3
  alt=1 ust=1 orta=1 d[orta]=8
Sonuç: -1
```

Bu metodun **iki** temel durumu var: aranan bulunduysa ya da aranacak aralık boşaldıysa (`alt > ust`). 11 elemanlı dizide 52 yalnızca 2 adımda bulundu; 5 ise dizide olmadığı için aralık boşalana kadar 4 adım sürdü.

15.9 Temel durum unutulursa: Stack overflow

Temel durumu olmayan (ya da temel duruma hiç ulaşamayan) bir metot kendini durmadan çağırır. Her çağrı yığına yeni bir çerçeve ekler. Yığın belleği sınırlı olduğu için birkaç bin (ayarlara göre on binlerce) çağrıdan sonra yer biter. Bu durumda çalışma zamanı programı **kapatır**:

```
Console.WriteLine("Başlıyor...");
Say(1);
Console.WriteLine("Bu satıra ulaşılmaz.");

static int Say(int n) => Say(n + 1);    // temel durum YOK!
```

Bu programı çalıştırmak gerekmez. Ekrana önce `Stack overflow.` yazısı ve bir yığın izi basılır; iz içinde **aynı satırın** tekrar tekrar görünmesi sonsuz özyinelemenin işaretidir. `Bu satıra ulaşılmaz.` hiç yazdırılmaz.

Bu hata bir `StackOverflowException`'dır. Bu tür bir istisna `try-catch` ile **yakalanamaz**: `catch` bloğu hiç çalışmaz ve süreç çöker. Çözüm, kodu düzeltmektir.

> **Sık Yapılan Hata: Temel durum var ama ulaşılamıyor**
>
> `if (n == 0) return;` temel durumuyla `GeriSay(n - 2)` çağrısı yapan bir metot düşün. `GeriSay(6)` sorunsuz çalışır: 6, 4, 2, 0. Ama `GeriSay(5)` çağrısında `n` sırasıyla 5, 3, 1, −1, −3... olur ve **hiçbir zaman 0'a eşit olmaz**: `Stack overflow.` . Aynı tuzak `Faktoriyel(-3)` gibi beklenmeyen girdilerde de vardır. Güvenli yazım: Eşitlik yerine aralık kontrolü yap: `if (n <= 0)` .

15.10 Özyineleme mi, döngü mü?

Hangisini seçmeli? Kısaca: **Problem doğası gereği özyinelemeliyse** (ağaçlar, Hanoi, böl ve yönet algoritmaları, iç içe klasörleri gezmek) özyineleme; **basit tekrar** (sayma, toplama, dizi gezme) varsa döngü.

Özellik	Özyineleme	Döngü (iteration)
Durma koşulu	Temel durum	Döngü koşulu
Bellek	Her çağrı yığında bir çerçeve kaplar	Birkaç değişken, sabit bellek
Hız	Her çağrının bir maliyeti var	Genellikle daha hızlı
Sınırı aşarsa	`Stack overflow.` (program kapanır)	Sonsuz döngü (program donar)
Tipik kullanım	Ağaç/klasör gezme, Hanoi, merge sort, quicksort	Sayma, toplama, dizi gezme

> **Not: Kuyruk özyinelemesi**
>
> Özyinelemeli çağrı metodun **en son** işiyse (`return Ebob(b, a % b);` gibi, çağrıdan sonra yapılacak başka iş yok) buna **kuyruk özyinelemesi** (tail recursion) denir. Bunu döngüye çevirmek derleyicinin garantisi değildir; bu yüzden derin özyinelemede yığın taşması riskini unutma.

15.11 Çağrıdan önce mi, sonra mı?

Özyineleme çıktı sorularının neredeyse hepsi tek bir fikre dayanır: **Özyinelemeli çağrıdan önceki** satırlar inişte çalışır, **sonraki** satırlar çıkışta, yani **ters sırada** çalışır.

onceSonra.cs

```
Once(3);
Console.WriteLine("<- once(3)");
Sonra(3);
Console.WriteLine("<- sonra(3)");
IkiCagri(3);
Console.WriteLine("<- ikiCagri(3)");

static void Once(int n)
{
    if (n == 0)
    {
        return;
    }
    Console.Write($"{n} ");             // çağrıdan ÖNCE
    Once(n - 1);
```

```
}

static void Sonra(int n)
{
    if (n == 0)
    {
        return;
    }
    Sonra(n - 1);
    Console.Write($"{n} ");             // çağrıdan SONRA
}

static void IkiCagri(int n)
{
    if (n == 0)
    {
        return;
    }
    IkiCagri(n - 1);
    Console.Write($"{n} ");             // iki çağrının ARASINDA
    IkiCagri(n - 1);
}
```

Çıktı

```
3 2 1 <- once(3)
1 2 3 <- sonra(3)
1 2 1 3 1 2 1 <- ikiCagri(3)
```

- `Once(3)`: Yazdırma inişte yapılır → `3 2 1`.
- `Sonra(3)`: `Sonra(2)`, `Sonra(1)`'i bekler... İlk yazdırma en dipteki `Sonra(1)`'de olur → `1 2 3`.
- `IkiCagri(3)`: Hanoi ile aynı yapı. `IkiCagri(2)` → `1 2 1`, sonra `3`, sonra yine `1 2 1`. Toplam $2^3 - 1 = 7$ sayı.

Sınavda Çıkar: Çıktı sorusu çözme yöntemi

1. Temel durumu bul ve en dipteki çağrının ne yaptığını yaz.
2. Çağrıdan **önceki** `Write`'ları inerken, **sonraki** `Write`'ları çıkarken yaz.
3. Birden fazla çağrı varsa küçük `n` için sonucu bul (`f(1)`, `f(2)`) ve büyük `n`'i onlar cinsinden kur: `IkiCagri(3)` = `IkiCagri(2)` + `"3 "` + `IkiCagri(2)`.

Bölüm Özeti – Bilmen Gerekenler

- **Özyineleme:** Metodun kendini daha küçük bir problemle çağırması.
- İki parça şart: **temel durum** (kendini çağırmadan cevap verir) ve **özyinelemeli durum** (problemi küçültüp kendini çağırır). Her çağrı temel duruma yaklaşmalı.
- Her çağrının **çağrı yığınında** kendi çerçevesi (kendi parametreleri) vardır. İnişte çağrılar birikir, çıkışta sonuçlar ters sırada hesaplanır (LIFO).
- Faktöriyel: `n * f(n-1)`; üs: `a * UsAl(a, n-1)`; basamak toplamı: `n % 10 + f(n / 10)`; EBOB: `Ebob(b, a % b)`; ikilik: `Ikilik(n / 2) + n % 2`.
- Saf Fibonacci aynı değerleri tekrar tekrar hesapladığı için **üstel** sayıda çağrı yapar; `Dictionary` ile hafızalama veya döngü bunu çözer.
- Hanoi: `n` disk için $2^n - 1$ hamle; iki özyinelemeli çağrı, arada bir hamle.
- Temel durum yoksa veya ulaşılamıyorsa → `Stack overflow.` ve program kapanır. `StackOverflowException` yakalanamaz.
- Çağrıdan **önceki** satırlar inişte (büyükten küçüğe), **sonraki** satırlar çıkışta (küçükten büyüğe) çalışır.

Sorular

A) Çoktan Seçmeli

1 Özyinelemeli bir metodun **temel durumu** (base case) için aşağıdakilerden hangisi doğrudur?

(A) Metodun kendini en az iki kez çağırdığı kısımdır.

(B) Sadece `void` metotlarda bulunur.

(C) Metodun kendini çağırmadan doğrudan sonuç verdiği durumdur; özyinelemeyi durdurur.

(D) İsteğe bağlıdır; derleyici sonsuz çağrıyı zaten engeller.

(E) Her zaman metodun son satırında yazılmalıdır.

2 Aşağıdaki metot için `F(5)` çağrısı hangi değeri döndürür?

```
static int F(int n)
{
    if (n <= 0)
    {
        return 0;
    }
    return n + F(n - 2);
}
```

(A) `15`

(B) `8`

(C) `Stack overflow.` oluşur

(D) `6`

(E) `9`

3 5 diskli Hanoi kuleleri problemi en az kaç hamlede çözülür?

(A) `31` (B) `32` (C) `25` (D) `10` (E) `63`

4 Temel durumu olmayan bir özyinelemeli metot çağrıldığında genellikle ne olur?

(A) Program derlenmez.
(B) Program hiç durmadan sonsuza kadar çalışır.
(C) `NullReferenceException` fırlatılır.
(D) Yığın dolar ve program `Stack overflow.` ile kapanır.
(E) Metot otomatik olarak `0` döndürür.

5 Aşağıdaki metot neyi hesaplar ve `Gizem(4, 3)` çağrısının sonucu nedir?

```
static int Gizem(int a, int b)
{
    if (b == 0)
    {
        return 0;
    }
    return a + Gizem(a, b - 1);
}
```

(A) `a` üzeri `b`; sonuç `64`
(B) `a × b`; sonuç `12`
(C) `a + b`; sonuç `7`
(D) `b` üzeri `a`; sonuç `81`
(E) `a × b`; sonuç `7`

B) Çıktı Tahmini

6 Aşağıdaki program ekrana ne yazdırır?

```
Yaz(5);
Console.WriteLine();
Yaz(4);

static void Yaz(int n)
{
    if (n > 0)
    {
        Console.Write($"{n} ");
        Yaz(n - 2);
        Console.Write($"{n * 10} ");
    }
}
```

7 Aşağıdaki program ekrana ne yazdırır?

```
Console.WriteLine(M("KASACI"));
Console.WriteLine(M("ABCDE"));
Console.WriteLine(M("XY").Length);

static string M(string s)
{
    if (s.Length <= 1)
    {
        return s;
    }
    return M(s.Substring(2)) + s[0];
}
```

8 Aşağıdaki program ekrana ne yazdırır?

```
int sonuc = F(3);
Console.WriteLine($"| sonuç = {sonuc}");

static int F(int n)
{
    if (n <= 0)
    {
        return 1;
    }
    Console.Write($"{n} ");
    return F(n - 1) + F(n - 2);
}
```

C) Kod Yazma

9 Bir string'de belirli bir karakterin kaç kez geçtiğini **döngü kullanmadan**, özyinelemeyle bulan `static int Say(string s, char c)` metodunu yaz ve programda dene. Örnek:

```
muhendislik içinde 'i': 2
banana içinde 'a': 3
kasa içinde 'x': 0
```

10 **Hiç döngü kullanmadan** aşağıdaki iki deseni çizen `YildizUcgen` programını yaz. İpucu: Aynı satıra `k` tane yıldız basan özyinelemeli bir `Yildizlar(int k)` metodu yaz; üçgenleri çizen metotlar da özyinelemeli olsun. Önce `Ucgen(4)`, sonra `TersUcgen(3)` çağrılmıştır:

```
*
**
***
****
***
**
*
```

Çözümler

Cevap anahtarı

1 C	2 E	3 A	4 D	5 B

1 Cevap: C

Temel durum, cevabı doğrudan bilinen ve metodun kendini **çağırmadığı** durumdur; özyinelemeyi durduran budur. A özyinelemeli durumu tarif eder (üstelik iki çağrı şart değildir). Değer döndüren metotlarda da temel durum vardır (B yanlış). Derleyici sonsuz özyinelemeyi engellemez; program çalışırken `Stack overflow.` oluşur (D yanlış). Temel durumun yeri serbesttir; genellikle metodun **başına** yazılır (E yanlış).

2 Cevap: E

`F(5) = 5 + F(3)`, `F(3) = 3 + F(1)`, `F(1) = 1 + F(-1)`, `F(-1) = 0` (çünkü `-1 <= 0`). Çıkışta toplanır: `1 + 0 = 1`, `3 + 1 = 4`, `5 + 4 = 9`.

```
Çıktı
9
```

C tuzaktır: Temel durum `n == 0` olsaydı tek sayılarda 0'a hiç ulaşılamaz ve `Stack overflow.` oluşurdu. Ama burada `n <= 0` kullanıldığı için `-1` de temel durumdur. A, `n - 1` ile azaltılsaydı çıkacak sonuçtur (`5 + 4 + 3 + 2 + 1`).

3 Cevap: A

`n` disk için en az hamle sayısı $2^n - 1$'dir: $2^5 - 1 = 32 - 1 = 31$. B, `-1`'i unutanların; E ise 6 diskin cevabıdır ($2^6 - 1 = 63$).

4 Cevap: D

Her çağrı yığına yeni bir çerçeve ekler; yığın belleği sınırlı olduğu için bir noktada dolar ve program `Stack overflow.` ile kapanır. Sonsuz döngüden farkı budur: Sonsuz döngü bellek tüketmeden sonsuza kadar dönebilir (B), ama sonsuz özyineleme kısa sürede çöker. Program sorunsuz derlenir (A yanlış). C yanlıştır: Bu durumda `NullReferenceException` değil, yığın taşması oluşur.

5 Cevap: B

Metot `b` kez `a` toplar: `Gizem(4, 3) = 4 + Gizem(4, 2) = 4 + 4 + Gizem(4, 1) = 4 + 4 + 4 + Gizem(4, 0) = 4 + 4 + 4 + 0 = 12`. Yani tekrarlı toplama ile **çarpma** yapar: `a × b`.

```
Çıktı
12
```

`a` üzeri `b` için `a * Gizem(a, b - 1)` ve temel durumda `return 1` gerekirdi (A). E'de işlem doğru ama sonuç yanlış.

6

Çağrıdan önceki `Write` inişte, sonraki `Write` çıkışta çalışır.

- `Yaz(5)`: İnişte `5`, `3`, `1` yazılır; `Yaz(-1)` hiçbir şey yapmaz. Çıkışta ters sırayla `1*10`, `3*10`, `5*10` → `10 30 50`.
- `Yaz(4)`: İnişte `4 2`; `Yaz(0)` hiçbir şey yapmaz (`0 > 0` yanlış). Çıkışta `20 40`.

```
Çıktı
5 3 1 10 30 50
4 2 20 40
```

7

Metot her adımda **ilk iki** karakteri atıp (`Substring(2)`) kalanı işliyor ve ilk karakteri **sona** ekliyor. Sonuçta 0., 2., 4. ... indeksteki karakterler **ters sırada** yazılır.

- `M("KASACI")` = `M("SACI") + 'K'` = `(M("CI") + 'S') + 'K'`. `M("CI")` = `M("") + 'C'` = `"C"`. Sonuç: `"CSK"`.
- `M("ABCDE")` = `M("CDE") + 'A'` = `(M("E") + 'C') + 'A'`. `M("E")` tek harf: `"E"`. Sonuç: `"ECA"`.
- `M("XY")` = `M("") + 'X'` = `"X"`, uzunluğu 1.

```
Çıktı
CSK
ECA
1
```

8

`Write` her çağrının başında (çağrılardan önce) çalışır; `n <= 0` olan çağrılar bir şey yazmadan `1` döndürür.

`F(3)` `3` yazar, `F(2)` `2` yazar, `F(1)` `1` yazar; `F(0)` ile `F(-1)` bir şey yazmaz ve 1 döner. Dönüşler: `F(1) = 2`, `F(2) = 3`. Sonra `F(3)` ikinci kez `F(1)`'i çağırır: `1` yazar, `F(3) = 3 + 2 = 5`.

Yazdırma sırası: `3 2 1 1`. Sonra `Main` sonucu yazar. `F(1)`'in **iki kez** çağrıldığına dikkat: Fibonacci'deki tekrar eden hesaplar.

```
Çıktı
3 2 1 1 | sonuç = 5
```

9

Temel durum: Boş string'de karakter yoktur → `0`. Özyinelemeli durum: İlk karakter `c` ise 1, değilse 0; buna geri kalan string'deki (`Substring(1)`) sayıyı ekle.

karakterSay.cs

```
Console.WriteLine("muhendislik içinde 'i': " + Say("muhendislik", 'i'));
Console.WriteLine("banana içinde 'a': " + Say("banana", 'a'));
Console.WriteLine("kasa içinde 'x': " + Say("kasa", 'x'));

static int Say(string s, char c)
{
    if (s.Length == 0)
    {
        return 0;
    }
    int ilk = s[0] == c ? 1 : 0;
    return ilk + Say(s.Substring(1), c);
}
```

Çıktı

```
muhendislik içinde 'i': 2
banana içinde 'a': 3
kasa içinde 'x': 0
```

10

Üç metot gerekiyor:

- `Yildizlar(k)`: Bir yıldız basıp `Yildizlar(k – 1)` çağırır, `k == 0` ise durur. `Ucgen(n)`: Önce `Ucgen(n – 1)`, **sonra** `n` yıldızlık satır. `TersUcgen(n)`: Önce `n` yıldızlık satır, **sonra** `TersUcgen(n – 1)`.

yildizUcgen.cs

```
Ucgen(4);
TersUcgen(3);

static void Yildizlar(int k)
{
    if (k == 0)
    {
        return;
    }
    Console.Write("*");
    Yildizlar(k - 1);
}

static void Ucgen(int n)
{
    if (n == 0)
    {
        return;
    }
    Ucgen(n - 1);                       // önce küçük üçgen
    Yildizlar(n);                       // sonra en uzun satır
    Console.WriteLine();
}

static void TersUcgen(int n)
{
    if (n == 0)
    {
        return;
    }
```

```
    Yildizlar(n);                       // önce en uzun satır
    Console.WriteLine();
    TersUcgen(n - 1);                   // sonra küçük üçgen
}
```

Çıktı

```
*
**
***
****
***
**
*
```

DENEME SINAVI

Deneme Final Sınavı

Bu deneme sınavı kitabın **tamamını** kapsar; ama ağırlık dönemin ikinci yarısındadır: sınıflar ve nesneler, kalıtım, polimorfizm, soyut sınıf ve arayüzler, istisnalar, koleksiyonlar ve LINQ, dosyalar ve özyineleme. İlk sekiz bölümün konuları (metotlar, diziler, `string`) soruların içine gömülü olarak karşına çıkacak. Sorular, 1. sınıf final sınavlarının zorluğunda ve tuzaklarıyla hazırlandı.

Bölüm	Soru sayısı	Soru başına puan	Toplam
A) Çoktan seçmeli	12	2	24
B) Çıktı tahmini	5	8	40
C) Kod yazma	4	9	36
Toplam	**21**		**100**

Süre: 120 dakika. Bilgisayar, kitap ve not kullanmak yok; sadece kâğıt ve kalem. Çıktılar Türkçe Windows'ta göründüğü gibidir (`3.5` → `3,5`).

> **İpucu: Önce süre tutarak çöz**
>
> 120 dakikalık bir sayaç kur ve süre bitene kadar çözümlere bakma. Kabaca **20 dakikayı** testlere, **40 dakikayı** çıktı sorularına, **60 dakikayı** kod yazmaya ayır. Çıktı sorularında sınıf hiyerarşisi varsa önce kâğıda küçük bir şema çiz: kim kimden türüyor, hangi metodu kim `override` ediyor.

> **Not: Kendini nasıl puanlarsın?**
>
> - **Çıktı sorularında** her satır eşit puan değerindedir. Boşluk, satır sonu ve `True` / `true` gibi farklar önemlidir.
> - **Kod sorularında** çözümdeki program tek doğru cevap değildir. Senin kodun istenen yapıyı (sınıf, arayüz, özyineleme...) kullanıyor ve aynı sonucu veriyorsa tam puan al. İstenen yapıyı kullanmadıysan (ör. özyineleme yerine döngü) yarım puan al; her derleme hatası için 1 puan kır.
> - Test cevaplarında konunun anlatıldığı bölüm parantez içinde yazıyor. Yanlış yaptığın konuların **Sınavda Çıkar** ve **Sık Yapılan Hata** kutularını tekrar oku.

Sorular

A) Çoktan Seçmeli

1 Aşağıdaki program ne yazdırır?

```
var s1 = new NoktaS { X = 1 };
var s2 = s1;
s2.X = 9;
var c1 = new NoktaC { X = 1 };
var c2 = c1;
c2.X = 9;
Console.WriteLine(s1.X + " " + c1.X);

struct NoktaS { public int X; }
class NoktaC { public int X; }
```

(A) `1 1`
(B) `9 9`
(C) `1 9`
(D) `9 1`
(E) Derleme hatası verir.

2 Aşağıdaki programın çıktısı nedir?

```
new B();
Console.WriteLine();

class A
{
    public A() { Console.Write("A"); }
    public A(int x) { Console.Write("a" + x); }
}

class B : A
{
    public B() : this(5) { Console.Write("B"); }
    public B(int x) { Console.Write("b"); }
}
```

(A) `AB` (B) `AbB` (C) `a5bB` (D) `Ba5b` (E) `bB`

3 Aşağıdaki programın çıktısı nedir?

```
Hayvan h = new Kedi();
Console.WriteLine($"{h.Ses()} {h.Ad()} {((Kedi)h).Ad()}");

class Hayvan
{
    public virtual string Ses() => "...";
    public string Ad() => "Hayvan";
}

class Kedi : Hayvan
{
    public override string Ses() => "Miyav";
    public new string Ad() => "Kedi";
}
```

(A) `Miyav Kedi Kedi`
(B) `... Hayvan Kedi`
(C) `Miyav Hayvan Hayvan`
(D) `Miyav Hayvan Kedi`
(E) `... Kedi Kedi`

4 Soyut sınıflar ve arayüzler için aşağıdakilerden hangisi **doğrudur**?

(A) `abstract` bir sınıftan `new` ile nesne oluşturulabilir.
(B) Bir sınıf `class C : A, B` biçiminde iki ayrı sınıftan türeyebilir.
(C) Bir sınıf bir sınıftan türeyip aynı anda birden fazla arayüzü uygulayabilir.
(D) `abstract` bir sınıfın kurucusu (constructor) olamaz.
(E) Arayüz adı `I` ile başlamazsa derleme hatası alınır.

5 Aşağıdaki program ne yazdırır?

```
Ogr[] dizi = { new("Ali", 80), new("Ece", 90), new("Can", 70) };
Array.Sort(dizi);
Console.WriteLine(string.Join(" ", dizi.Select(o => o.Ad)));

record Ogr(string Ad, int Not) : IComparable<Ogr>
{
    public int CompareTo(Ogr? diger) =>
        diger is null ? 1 : diger.Not.CompareTo(Not);
}
```

(A) `Ece Ali Can` (B) `Can Ali Ece` (C) `Ali Ece Can` (D) `Ali Can Ece` (E) `Ece Can Ali`

6 Aşağıdaki kod parçası için hangisi doğrudur?

```
int b = 0;
try
{
    Console.WriteLine(10 / b);
}
catch (Exception)
{
    Console.WriteLine("Genel");
}
catch (DivideByZeroException)
{
    Console.WriteLine("Sıfıra bölme");
}
```

(A) `Genel` yazdırır.
(B) `Sıfıra bölme` yazdırır.
(C) Önce `Genel`, sonra `Sıfıra bölme` yazdırır.
(D) Program `DivideByZeroException` ile çöker.
(E) Derleme hatası verir.

7 Aşağıdaki program ne yazdırır?

```
Console.WriteLine(Dene(0) + " " + Dene(1) + " " + Dene(2));

static string Dene(int kod)
{
    try
    {
        if (kod > 0) throw new InvalidOperationException("k" + kod);
        return "ok";
    }
    catch (InvalidOperationException e) when (e.Message == "k1")
    {
        return "bir";
    }
    catch (InvalidOperationException)
    {
        return "diger";
    }
}
```

(A) `ok diger diger`
(B) `ok bir diger`
(C) `ok bir bir`
(D) `ok bir` yazar, sonra `InvalidOperationException` ile çöker.
(E) Derleme hatası verir.

8 Aşağıdaki program için hangisi doğrudur?

```
var liste = new List<int> { 1, 2, 3, 4 };
foreach (int x in liste)
{
    if (x == 2) liste.Remove(x);
}
Console.WriteLine(liste.Count);
```

A) `4` yazdırır.
B) `3` yazdırır.
C) `2` yazdırır.
D) Çalışırken `InvalidOperationException` fırlatır.
E) Derleme hatası verir.

9 Aşağıdaki program ne yazdırır?

```
var d = new Dictionary<string, int>();
d["a"] = 1;
d["a"] = 2;
d.Add("b", 3);
d["b"] += 10;
bool bulundu = d.TryGetValue("c", out int v);
Console.WriteLine($"{d.Count} {d["a"]} {d["b"]} {bulundu} {v}");
```

A) `3 2 13 False 0`
B) `2 1 13 False 0`
C) `2 2 13 False 0`
D) `2 2 13 True 0`
E) `d["a"] = 2` satırında `ArgumentException` ile çöker.

10 Aşağıdaki program ne yazdırır?

```
var sayilar = new List<int> { 1, 2, 3 };
var cift = sayilar.Where(x => x % 2 == 0);
sayilar.Add(4);
Console.WriteLine(cift.Count());
```

A) `2`
B) `1`
C) `3`
D) `4`
E) `InvalidOperationException` ile çöker.

11 Aşağıdaki program ne yazdırır?

```
string s = "Programlama";
s.ToUpper();
string t = s.Substring(3, 4);
Console.WriteLine(s + " " + t + " " + s.Length + " " + (t == "gram"));
```

A) `PROGRAMLAMA gram 11 True`
B) `Programlama gram 11 False`
C) `Programlama g 11 True`
D) `Programlama gram 10 True`
E) `Programlama gram 11 True`

12 Aşağıdaki program ne yazdırır?

```
int x = 3, y = 4;
int r = Isle(x, ref y, out int z);
Console.WriteLine($"{x} {y} {z} {r}");

static int Isle(int a, ref int b, out int c)
{
    a++;
    b *= 2;
    c = a + b;
    return c - 1;
}
```

A) `4 8 12 11`
B) `3 8 12 11`
C) `3 4 12 11`
D) `3 8 12 12`
E) Derleme hatası verir.

B) Çıktı Tahmini

13 Aşağıdaki program ekrana ne yazdırır?

```
Calisan[] dizi = { new Memur("Ali"), new Mudur("Ece") };
foreach (Calisan c in dizi)
{
    Yazici.Yaz(c);
}
Memur m = new Mudur("Can");
Yazici.Yaz(m);
Calisan k = m;
Console.WriteLine(k.Unvan() + " " + (k is Memur) + " " + Calisan.Sayi);

abstract class Calisan
{
    public static int Sayi = 0;
    protected string _ad;

    protected Calisan(string ad)
    {
        _ad = ad;
        Sayi++;
    }

    public abstract double Maas();
```

```
    public virtual string Unvan() => "Çalışan";
    public override string ToString() => Unvan() + " " + _ad + " " + Maas();
}

class Memur : Calisan
{
    public Memur(string ad) : base(ad) { }
    public override double Maas() => 1000;
}

class Mudur : Memur
{
    public Mudur(string ad) : base(ad) { }
    public override double Maas() => base.Maas() * 1.5 + 0.5;
    public override string Unvan() => "Müdür";
}

class Yazici
{
    public static void Yaz(Calisan c) => Console.WriteLine("C: " + c);
    public static void Yaz(Memur m) => Console.WriteLine("M: " + m);
}
```

14 Aşağıdaki program ekrana ne yazdırır?

```
Console.WriteLine(F("20"));
Console.WriteLine(F("abc"));
try
{
    Console.WriteLine(F("0"));
}
catch (DivideByZeroException)
{
    Console.WriteLine("DZ");
}
Console.WriteLine("son");

static int F(string s)
{
    try
    {
        using var k = new Kaynak(s);
        int n = int.Parse(s);
        return 100 / n;
    }
    catch (FormatException)
    {
        Console.Write("FE ");
        return -1;
    }
    finally
    {
        Console.Write("F ");
    }
}

class Kaynak : IDisposable
{
    private readonly string _ad;

    public Kaynak(string ad)
    {
        _ad = ad;
        Console.Write("A" + ad + " ");
```

```
    }

    public void Dispose() => Console.Write("K" + _ad + " ");
}
```

15 Aşağıdaki program ekrana ne yazdırır?

```
var liste = new List<Ogrenci>
{
    new("Ece", 85), new("Ali", 70), new("Can", 85), new("Ayşe", 92)
};
Console.WriteLine(string.Join(" ", liste.OrderBy(o => o.Not).Select(o => o.Ad)));
Console.WriteLine(liste[3]);
Console.WriteLine(liste.Contains(new Ogrenci("Ali", 70)) + " "
    + liste.IndexOf(new Ogrenci("Can", 85)));
Console.WriteLine(liste.Count(o => o.Not >= 85) + " " + liste.Average(o => o.Not));

var harf = new Dictionary<char, int>();
foreach (var o in liste)
{
    char c = o.Ad[0];
    if (harf.TryGetValue(c, out int adet)) harf[c] = adet + 1;
    else harf[c] = 1;
}
foreach (var kv in harf.OrderBy(k => k.Key))
{
    Console.Write($"{kv.Key}={kv.Value} ");
}
Console.WriteLine();

record Ogrenci(string Ad, int Not);
```

16 Aşağıdaki program ekrana ne yazdırır?

```
Console.WriteLine(Oz.F(5) + " " + Oz.Sayac);
Console.WriteLine(Oz.Ters("final"));
Oz.Yaz(3);
Console.WriteLine();

static class Oz
{
    public static int Sayac = 0;

    public static int F(int n)
    {
        Sayac++;
        if (n <= 1) return n;
        return F(n - 1) + F(n - 2);
    }

    public static string Ters(string s)
    {
        if (s.Length <= 1) return s;
        return Ters(s.Substring(1)) + s[0];
    }

    public static void Yaz(int n)
    {
        if (n == 0) return;
        Console.Write(n + " ");
        Yaz(n - 1);
        Console.Write(n + " ");
    }
}
```

17 Aşağıdaki program ekrana ne yazdırır?

```
int puan = 82;
string harf = (puan / 10) switch { 9 => "AA", 8 => "BA", 7 => "BB", _ => "FF" };
double ort = (70 + 85 + 92) / 3;
double ort2 = (70 + 85 + 92) / 3.0;
Console.WriteLine($"{harf} {ort} {ort2:F2} {Math.Round(2.5)} {Math.Round(3.5)}");

string s = "bilgi";
string t = s.ToUpper();
s.Replace("i", "e");
Console.WriteLine(s + " " + t + " " + s.Substring(1, 3) + " " + (s == "bilgi"));

int[] a = { 4, 8, 15 };
int[] b = a;
b[0] = 99;
Console.WriteLine(string.Join(",", a) + " " + a.Length);
```

C) Kod Yazma

18 `Kesir` sınıfını yaz:

- `Pay` ve `Payda`: kurucuda atanan, dışarıdan yalnız okunabilen (`set`'siz) iki özellik.
- `PaydaSifirException`: `Exception`'dan türeyen, mesajı `"Payda sıfır olamaz"` olan kendi istisna sınıfın.
- Kurucu: Payda 0 ise bu istisnayı fırlatsın. Payda negatifse işareti paya taşısın. Kesri **sadeleştirerek** saklasın; EBOB'u **özyinelemeli** bir `private static int Ebob(int a, int b)` metoduyla bulsun.
- `Kesir Topla(Kesir k)` ve `Kesir Carp(Kesir k)`: Sonucu **yeni** bir `Kesir` olarak döndürsün; mevcut nesneler değişmesin.
- `ToString()`: `pay/payda` biçiminde.

Ana kodda test et. Beklenen çıktı:

```
2/4 -> 1/2
1/2 + 1/3 = 5/6
1/2 * 1/3 = 1/6
3/-6 -> -1/2
0/5 -> 0/1
Hata: Payda sıfır olamaz
```

19 Aşağıdaki sınıf yapısını kur:

- `IOlceklenebilir` arayüzü: `void Olcekle(double k)` metodu.
- Soyut `Sekil` sınıfı: `Ad` özelliği; soyut `Alan()` ve `Cevre()` metotları; `IComparable<Sekil>` (alana göre küçükten büyüğe); adı, alanı ve çevreyi tek satırda yazan `ToString()`. Biçim: ad 10 genişlikte sola yaslı, alan 7 ve çevre 6 genişlikte, ikisi de 2 ondalık basamaklı.
- `Daire` (yarıçap): `Sekil`'den türer ve `IOlceklenebilir`'i uygular (yarıçapı `k` katına çıkarır).
- `Dikdortgen` (en, boy): `Sekil`'den türer. `Kare` (kenar): `Dikdortgen`'den türer.

Ana kodda bir `List<Sekil>` içine 1.5 yarıçaplı bir daire, 4×2.5'lik bir dikdörtgen ve 3 kenarlı bir kare koy. **Ölçeklenebilir** şekilleri 2 kat büyüt, listeyi sırala; sonra her şekli, toplam alanı ve en büyük şekli yazdır. Beklenen çıktı:

```
Kare       alan=   9,00 çevre= 12,00
Dikdörtgen alan=  10,00 çevre= 13,00
Daire      alan=  28,27 çevre= 18,85
Toplam alan: 47,27
En büyük: Daire      alan=  28,27 çevre= 18,85
```

20 `metin.txt` dosyasını okuyan bir program yaz. Dosyada noktalama işareti yok; kelimeler bir veya daha fazla boşlukla ayrılmış. Program:

1. her kelimenin kaç kez geçtiğini **büyük/küçük harf farkı gözetmeden** saysın (`Dictionary` kullan) ve kelimeleri alfabetik sırayla yazdırsın,
2. toplam kelime sayısını, farklı kelime sayısını ve en sık geçen kelimeyi yazdırsın,
3. kelime–sayı listesini `rapor.txt` dosyasına yazsın.

Dosya yoksa program çökmesin, `Dosya bulunamadı: metin.txt` yazıp bitsin. Kullanılacak dosya:

metin.txt

```
Nesne yönelimli bir dildir
sınıf nesne için bir şablondur
Her nesne bir sınıfın örneğidir
nesne  oluşturmak kolaydır
```

21 Kullanıcıdan disk sayısını ve bir metin okuyan programı yaz:

- `static void Hanoi(int n, char kaynak, char hedef, char ara, ref int sayac)`: Hanoi Kuleleri'nin hamlelerini **özyinelemeli** olarak `1. hamle: A -> C` biçiminde yazdırsın; `sayac` hamle numarasını tutsun.
- `static int HamleSayisi(int n)`: Hamle sayısını `2·H(n−1)+1` formülüyle **özyinelemeli** hesaplasın.
- `static bool PalindromMu(string s)`: Metnin tersi kendisine eşit mi, **özyinelemeli** denetlesin (döngü yok).

Hamleler `A` kulesinden `C` kulesine olsun. Program hamleleri yazdırsın, sayacı formülle karşılaştırsın, sonra metnin palindrom olup olmadığını yazsın. Örnek çalışma:

```
Disk sayısı: 3
1. hamle: A -> C
2. hamle: A -> B
3. hamle: C -> B
4. hamle: A -> C
5. hamle: B -> A
6. hamle: B -> C
7. hamle: A -> C
Toplam hamle: 7 (formül: 7)
Metin: kayak
kayak palindromdur
```

Çözümler

Cevap anahtarı

1 C	2 B	3 D	4 C	5 A	6 E	7 B	8 D
9 C	10 A	11 E	12 B				

1 Cevap: C

`struct` bir **değer tipidir**: `var s2 = s1;` bir **kopya** üretir, `s2.X = 9` yalnız kopyayı değiştirir; `s1.X` 1 kalır. `class` bir **başvuru tipidir**: `c2 = c1` aynı nesneyi gösterir, `c2.X = 9` değişikliği `c1` üzerinden de görünür. (Bölüm 9)

- A) ikisini de kopya sanıyor; B) ikisini de başvuru sanıyor; D) tersini sanıyor.
- E) Derleme hatası yok: `struct` alanları atanabilir.

Çıktı
```
1 9
```

2 Cevap: B

`new B()` → `B()` çalışır. İlk satırı `: this(5)` olduğu için **aynı sınıftaki** `B(int)` çağrılır. `B(int)` bir `: base(...)` yazmadığından derleyici gizli bir `: base()` ekler: `A()` çalışır ve `A` yazar. Sonra `B(int)`'in gövdesi `b`'yi, en son `B()`'nin gövdesi `B`'yi yazar.

`A(int)` hiç çalışmaz (C yanlış): `base(x)` yazan yoktu. Kural: Üst sınıfın kurucusu her zaman **önce** biter; açıkça seçilmediyse parametresiz olan çalışır. (Bölüm 10)

Çıktı
```
AbB
```

3 Cevap: D

- `Ses()` `virtual`/`override` olduğu için hangi metodun çalışacağı **nesnenin gerçek tipine** (`Kedi`) göre belirlenir: `Miyav`.
- `Ad()` `virtual` değildir; `Kedi`'deki `new` sadece **gizler**. Hangi `Ad()` çalışacak, **referansın tipine** bakılarak seçilir: `h` bir `Hayvan` referansı → `Hayvan`; `(Kedi)h` bir `Kedi` referansı → `Kedi`. (Bölüm 10)

Çıktı
```
Miyav Hayvan Kedi
```

4 Cevap: C

C# sınıflarda **tekli kalıtım** vardır (B yanlış), ama bir sınıf istediği kadar arayüz uygulayabilir: `class Ordek : Hayvan, IUcabilir, IYuzebilir`. (Bölüm 11)

- A) Soyut sınıftan `new` ile nesne oluşturmak CS0144 hatasıdır.
- D) Soyut sınıfın kurucusu **olabilir**; alt sınıf `: base(...)` ile onu çağırır.
- E) `I` öneki bir adlandırma kuralıdır, derleyici zorlamaz.

5 Cevap: A

`Array.Sort` karşılaştırmayı `CompareTo`'dan alır: negatif dönerse `this` önce gelir. Burada `diger.Not.CompareTo(Not)` yazıldığı için karşılaştırma **ters**: notu büyük olan önce gelir → büyükten küçüğe `Ece (90) Ali (80) Can (70)`. Küçükten büyüğe sıralama için `Not.CompareTo(diger.Not)` yazılırdı (B). (Bölüm 11)

Çıktı
```
Ece Ali Can
```

6 Cevap: E

`catch` blokları yukarıdan aşağıya denenir. `Exception` her istisnanın üst sınıfı olduğundan ilk blok her şeyi yakalar; ikinci blok **hiçbir zaman** çalışamaz. C# bunu derleme hatası sayar:

Derleyici çıktısı – program derlenmez
```
test6.cs(10,8): error CS0160: A previous catch clause already catches all exceptions of this or of a
super type ('Exception')
```

Kural: Özel (alt sınıf) istisnalar **önce**, genel olanlar **sonra** yazılır. (Bölüm 12)

7 Cevap: B

`when` süzgeci, `catch` bloğunun yalnız koşul doğruysa çalışmasını sağlar.

- `Dene(0)`: istisna yok → `ok`.
- `Dene(1)`: mesaj `k1` → ilk `catch` süzgeçten geçer → `bir`.
- `Dene(2)`: mesaj `k2` → süzgeç `false`, sıradaki `catch` (süzgeçsiz) yakalar → `diger`.

Aynı türde iki `catch` yazmak, ilkinde `when` olduğu sürece hata değildir (E yanlış). (Bölüm 12)

Çıktı
```
ok bir diger
```

8 Cevap: D

`foreach` arka planda bir sayaç (enumerator) kullanır. Döngü sürerken listeye `Add`/`Remove` yaparsan sayaç bozulur ve sıradaki elemana geçerken `InvalidOperationException` ("Collection was modified...") fırlar. `2` silindiği an hata oluşmaz; döngü bir sonraki adıma geçmek isteyince oluşur (A, B ve C bu yüzden yanlış).

Çıktı – program yakalanmamış istisna ile sonlanır
```
Unhandled exception. System.InvalidOperationException: Collection was modified; enumeration
operation may not execute.
   at Program.<Main>$(String[] args) in test8.cs:line 2
```

Çözüm: Listeyi sondan başa `for` ile gez ya da silinecekleri başka listeye topla. (Bölüm 13)

9 Cevap: C

- İndeksleyiciyle atama (`d["a"] = 2`) anahtar yoksa **ekler**, varsa **değiştirir**; hata vermez (E yanlış). `d.Add("a", 2)` yazsaydın `ArgumentException` alırdın.
- Sözlükte `a` ve `b` var → `Count` 2, `d["a"]` 2, `d["b"]` 3 + 10 = 13.
- `TryGetValue` olmayan anahtarda `False` döndürür ve `out` değişkenine varsayılan değeri (0) koyar; `d["c"]` olsaydı `KeyNotFoundException` fırlardı. (Bölüm 13)

Çıktı
```
2 2 13 False 0
```

10 Cevap: A

LINQ sorguları **ertelenmiş çalışır** (deferred execution): `Where(...)` satırı hiçbir şey hesaplamaz, sadece sorguyu tanımlar. Süzme, ancak `Count()` çağrıldığında ve **o anki** listeyle yapılır: `1, 2, 3, 4` → çiftler `2` ve `4` → `2`. B şıkkı süzmenin tanım anında yapıldığını sanıyor. Liste sayılırken değişmediği için hata yoktur (E). (Bölüm 13)

Çıktı
```
2
```

11 Cevap: E

- `string` değişmezdir: `ToUpper()` yeni bir metin döndürür, `s` aynı kalır; sonuç kullanılmadığı için bir etkisi olmadı (A yanlış).
- `==` `string`'lerde **içeriği** karşılaştırır → `True` (B yanlış).
- `Substring(3, 4)`: **başlangıç 3, uzunluk 4** → `gram` (C bitiş indeksi sanıyor). `Length` 11'dir (D yanlış). (Bölüm 8)

Çıktı
```
Programlama gram 11 True
```

12 Cevap: B

- `a` değer parametresidir: metot kopyayı artırır, `x` 3 kalır (A yanlış).
- `ref int b` çağıranın `y`'sinin takma adıdır: `b *= 2` → `y` 8 olur (C yanlış).
- `out int c` metot içinde atanır: `c = 4 + 8` → `z` 12. Dönüş değeri `c - 1` = 11 (D yanlış). (Bölüm 6)

Çıktı
```
3 8 12 11
```

13

İki kuralı birlikte uygula:

- **Aşırı yükleme (overload)** derleme zamanında, referansın **tipine** göre seçilir. Döngüdeki `c` bir `Calisan` olduğu için iki eleman da `Yaz(Calisan)` ile yazılır (`C:`). `m` bir `Memur` olduğu için `Yaz(Memur)` seçilir (`M:`), nesne `Mudur` olsa bile.
- **`override`** çalışma zamanında, nesnenin **gerçek** tipine göre işler. `ToString` içindeki `Unvan()` ve `Maas()` nesnenin kendi sınıfındaki metotlara gider.

`Memur`: `Unvan()` `Calisan`'dan miras (`Çalışan`), `Maas()` 1000. `Mudur`: `Müdür`, `1000 * 1.5 + 0.5` = 1500,5.

Ondalıklı sayı Türkçe kültürde virgülle yazılır; `1000` ise tam sayı değerli `double` olduğu için ondalıksız görünür. Son satır: `k.Unvan()` → `Müdür`; `Mudur` bir `Memur` olduğundan `k is Memur` → `True`; `Sayi` static'tir ve 3 nesne oluşturuldu.

Çıktı
```
C: Çalışan Ali 1000
C: Müdür Ece 1500,5
M: Müdür Can 1500,5
Müdür True 3
```

14

`finally`, `try/catch` içinde `return` olsa bile metottan çıkmadan hemen önce çalışır. `using var k` ise `try` bloğunun sonunda (ya da blok hata ile terk edilirken) `Dispose` çağırır; yani `catch` ve `finally`'den **önce**.

- `F("20")`: `A20`; `return 5` hazırlanır, blok biterken `K20`, sonra `finally` → `F`. `Console.WriteLine` → `5`.
- `F("abc")`: `Aabc`; `int.Parse` `FormatException` fırlatır, `using` `Kabc` yazar, sonra `catch` → `FE`, `finally` → `F`, dönüş −1.
- `F("0")`: `A0`; `100 / 0` → `DivideByZeroException`, `K0` yazılır, bu `F`'te yakalanmaz ama `finally` → `F` çalışır. `Main` tarafı yakalar: `DZ`. İçteki `WriteLine` hiç çalışmaz.

Çıktı
```
A20 K20 F 5
Aabc Kabc FE F -1
A0 K0 F DZ
son
```

15

- `OrderBy` notu **küçükten büyüğe** sıralar ve **kararlıdır**: eşit notlu `Ece` ve `Can` eski sırasını korur. `liste`'nin kendisi değişmez; `liste[3]` hâlâ `Ayşe`'dir. `record` için `ToString` otomatik üretilir: `Ogrenci { Ad = Ayşe, Not = 92 }`.
- `record` eşitliği **değere** göre çalışır: yeni `Ogrenci("Ali", 70)` listede bulunur (`True`); `Can` özgün listede 2. indekstedir.
- `Count(o => o.Not >= 85)`: Ece, Can, Ayşe → 3. `Average` = 332 / 4 = 83 (`double` ama ondalık kısmı yok → `83`).
- Sözlükte sıra garanti değildir; bu yüzden `OrderBy(k => k.Key)` ile anahtara göre sıralandı: `A` (Ali, Ayşe) 2, `C` 1, `E` 1. Her çiftten sonra bir boşluk bırakıldığı için satır boşlukla biter.

Çıktı
```
Ali Ece Can Ayşe
Ogrenci { Ad = Ayşe, Not = 92 }
True 2
3 83
A=2 C=1 E=1
```

16

`F` (Fibonacci) ve çağrı sayısı: `F(5)` = 5. Çağrı sayısı C(n) = 1 + C(n−1) + C(n−2), C(0) = C(1) = 1: C(2) = 3, C(3) = 5, C(4) = 9, C(5) = **15**. İfade soldan sağa değerlendirilir: önce `Oz.F(5)` çalışır, `Oz.Sayac` ondan **sonra** okunur; bu yüzden 0 değil 15 yazılır.

`Ters`: `Ters("final")` = `Ters("inal") + 'f'` = … = `lanif`.

`Yaz`: Özyinelemeli çağrıdan **önceki** `Write` inerken (`3 2 1`), **sonraki** `Write` geri dönerken (`1 2 3`) çalışır.

```
Çıktı
5 15
lanif
3 2 1 1 2 3
```

17

- `82 / 10` tamsayı bölmesidir → 8 → `BA`.
- `(70 + 85 + 92) / 3`: iki taraf da `int` → 247 / 3 = 82; sonuç `double`'a atanınca `82`. `3.0` ile bölününce `82,3333...` → `F2` ile `82,33` (Türkçe kültürde virgül).
- `Math.Round` varsayılan olarak **banker yuvarlaması** yapar: `2.5` → `2`, `3.5` → `4`.
- `s.Replace(...)` yeni bir metin döndürür ve burada atılmıştır; `s` hâlâ `bilgi`'dir. `ToUpper()` Türkçe kültürde `i` → `İ` yapar: `BİLGİ`. `Substring(1, 3)`: indeks 1'den 3 karakter → `ilg`. `==` içeriği karşılaştırır → `True`.
- `b = a` kopya değil, aynı diziye ikinci referanstır: `b[0] = 99` → `a` da değişir.

```
Çıktı
BA 82 82,33 2 4
bilgi BİLGİ ilg True
99,8,15 3
```

18

Sadeleştirme kurucuda bir kez yapılır; `Pay` ve `Payda` yalnız `get`'li olduğu için nesne bir daha değişmez. `Topla` ve `Carp` yeni `Kesir` oluşturduğundan sonuç da otomatik sadeleşir. EBOB için Öklid algoritması: `Ebob(a, b) = Ebob(b, a % b)`, `Ebob(a, 0) = a`.

kesir.cs

```
var a = new Kesir(2, 4);
var b = new Kesir(1, 3);
Console.WriteLine("2/4 -> " + a);
Console.WriteLine($"{a} + {b} = {a.Topla(b)}");
Console.WriteLine($"{a} * {b} = {a.Carp(b)}");
Console.WriteLine("3/-6 -> " + new Kesir(3, -6));
Console.WriteLine("0/5 -> " + new Kesir(0, 5));
try
{
    new Kesir(5, 0);
}
catch (PaydaSifirException e)
{
    Console.WriteLine("Hata: " + e.Message);
}

class PaydaSifirException : Exception
{
    public PaydaSifirException() : base("Payda sıfır olamaz") { }
}

class Kesir
{
    public int Pay { get; }
    public int Payda { get; }

    public Kesir(int pay, int payda)
    {
        if (payda == 0) throw new PaydaSifirException();
        if (payda < 0)
```

```
        {
            pay = -pay;
            payda = -payda;
        }
        int e = Ebob(Math.Abs(pay), payda);
        Pay = pay / e;
        Payda = payda / e;
    }

    private static int Ebob(int a, int b) => b == 0 ? a : Ebob(b, a % b);
    public Kesir Topla(Kesir k) =>
        new(Pay * k.Payda + k.Pay * Payda, Payda * k.Payda);
    public Kesir Carp(Kesir k) => new(Pay * k.Pay, Payda * k.Payda);
    public override string ToString() => Pay + "/" + Payda;
}
```

Çıktı

```
2/4 -> 1/2
1/2 + 1/3 = 5/6
1/2 * 1/3 = 1/6
3/-6 -> -1/2
0/5 -> 0/1
Hata: Payda sıfır olamaz
```

`0/5` durumuna dikkat: `Ebob(0, 5)` = 5 olduğu için kesir `0/1`'e sadeleşir. Kurucudaki `pay` ve `payda` parametreleri, aynı adlı özelliklerden büyük/küçük harfle ayrılır (`pay` ≠ `Pay`).

19

`Sekil` soyut olduğu için ortak davranışı (`ToString`, `CompareTo`) bir kez yazar; `Alan()` çağrıları sanal bağlama ile her nesnenin kendi metoduna gider. `Kare`, `Dikdortgen`'in formüllerini kullanır; adını üst sınıfa iletebilmek için `Dikdortgen`'e `protected` bir kurucu ekledik. Yalnız `Daire` ölçeklenebilir olduğundan `is IOlceklenebilir o` ile hem tip denetimi hem dönüştürme yapılır.

sekiller.cs

```
var sekiller = new List<Sekil>
{
    new Daire(1.5), new Dikdortgen(4, 2.5), new Kare(3)
};
foreach (var s in sekiller)
{
    if (s is IOlceklenebilir o) o.Olcekle(2);
}
sekiller.Sort();
foreach (var s in sekiller) Console.WriteLine(s);
Console.WriteLine($"Toplam alan: {sekiller.Sum(s => s.Alan()):F2}");
Console.WriteLine("En büyük: " + sekiller[sekiller.Count - 1]);

interface IOlceklenebilir
{
    void Olcekle(double k);
}

abstract class Sekil : IComparable<Sekil>
{
    public string Ad { get; }

    protected Sekil(string ad) => Ad = ad;

    public abstract double Alan();
```

```
    public abstract double Cevre();

    public int CompareTo(Sekil? diger) =>
        diger is null ? 1 : Alan().CompareTo(diger.Alan());

    public override string ToString() =>
        $"{Ad,-10} alan={Alan(),7:F2} çevre={Cevre(),6:F2}";
}

class Daire : Sekil, IOlceklenebilir
{
    private double _r;

    public Daire(double r) : base("Daire") => _r = r;

    public override double Alan() => Math.PI * _r * _r;
    public override double Cevre() => 2 * Math.PI * _r;
    public void Olcekle(double k) => _r *= k;
}

class Dikdortgen : Sekil
{
    private readonly double _en, _boy;

    public Dikdortgen(double en, double boy) : this("Dikdörtgen", en, boy) { }

    protected Dikdortgen(string ad, double en, double boy) : base(ad)
    {
        _en = en;
        _boy = boy;
    }

    public override double Alan() => _en * _boy;
    public override double Cevre() => 2 * (_en + _boy);
}

class Kare : Dikdortgen
{
    public Kare(double kenar) : base("Kare", kenar, kenar) { }
}
```

Çıktı

```
Kare       alan=   9,00 çevre= 12,00
Dikdörtgen alan=  10,00 çevre= 13,00
Daire      alan=  28,27 çevre= 18,85
Toplam alan: 47,27
En büyük: Daire      alan=  28,27 çevre= 18,85
```

Sıralama `Sort()` ile alana göre yapıldığı için en büyük şekil listenin sonundadır. `Daire` ölçeklendikten sonra yarıçap 3 olur; çıktıdaki 28,27 bu yüzdendir (ölçeklemeden önce 7,07 olurdu).

20

`File.ReadAllLines` dosya yoksa `FileNotFoundException` fırlatır; `try/catch` ile yakalayıp programı bitiriyoruz. Birden fazla boşluk yüzünden `Split(' ')` boş parçalar üretirdi; `StringSplitOptions.RemoveEmptyEntries` bunları atar. Sayım için `TryGetValue`: anahtar varsa sayıyı artır, yoksa 1 yap. Sözlük sırasızdır, bu yüzden yazdırmadan önce `OrderBy` ile sıralıyoruz.

kelimeSayac.cs

```
var sayac = new Dictionary<string, int>();
int toplam = 0;
string[] satirlar;
try
{
    satirlar = File.ReadAllLines("metin.txt");
}
catch (FileNotFoundException)
{
    Console.WriteLine("Dosya bulunamadı: metin.txt");
    return;
}
foreach (string satir in satirlar)
{
    foreach (string p in satir.Split(' ', StringSplitOptions.RemoveEmptyEntries))
    {
        string kelime = p.ToLower();
        sayac[kelime] = sayac.TryGetValue(kelime, out int n) ? n + 1 : 1;
        toplam++;
    }
}

var sirali = sayac.OrderBy(kv => kv.Key).ToList();
string enSik = "";
foreach (var kv in sirali)
{
    Console.WriteLine($"{kv.Key,-12}{kv.Value}");
    if (enSik == "" || kv.Value > sayac[enSik]) enSik = kv.Key;
}
Console.WriteLine($"Toplam: {toplam}, farklı: {sayac.Count}");
Console.WriteLine($"En sık: {enSik} ({sayac[enSik]})");

File.WriteAllLines("rapor.txt", sirali.Select(kv => kv.Key + " " + kv.Value));
Console.WriteLine("rapor.txt yazıldı.");
```

Çıktı

```
bir         3
dildir      1
her         1
için        1
kolaydır    1
nesne       4
oluşturmak  1
örneğidir   1
sınıf       1
sınıfın     1
şablondur   1
yönelimli   1
Toplam: 17, farklı: 12
En sık: nesne (4)
rapor.txt yazıldı.
```

`Nesne` ile `nesne` aynı kelime sayıldı çünkü her kelime `ToLower()` ile küçültüldü. Türkçe kültürde `ToLower()` büyük `I`'yı noktasız `ı` yapar; metinde büyük `I` olsaydı sonuç buna göre olurdu. Sıralama Türkçe alfabeye göre yapıldı: `örneğidir` `o`'dan sonra, `şablondur` `s`'den sonra gelir.

21

`Hanoi`: `n` diski `kaynak`'tan `hedef`'e taşımak için önce üstteki `n-1` diski `ara`'ya, sonra en büyük diski `hedef`'e, en son `n-1` diski `hedef`'e taşırız; `n == 0` temel durumdur. `ref int sayac` bütün çağrıların **aynı** sayacı artırmasını sağlar. `HamleSayisi`: `H(0) = 0`, `H(n) = 2·H(n-1) + 1` → $2^n - 1$. `PalindromMu`: uçlar farklıysa `false`; aynıysa uçları atıp kalanı denetle; boş ya da tek harfli metin palindromdur.

ozyineleme.cs

```
Console.Write("Disk sayısı: ");
int diskSayisi = Convert.ToInt32(Console.ReadLine());
int hamle = 0;
Hanoi(diskSayisi, 'A', 'C', 'B', ref hamle);
Console.WriteLine($"Toplam hamle: {hamle} (formül: {HamleSayisi(diskSayisi)})");

Console.Write("Metin: ");
string metin = Console.ReadLine() ?? "";
string sonuc = PalindromMu(metin) ? "palindromdur" : "palindrom değildir";
Console.WriteLine(metin + " " + sonuc);

static void Hanoi(int n, char kaynak, char hedef, char ara, ref int sayac)
{
    if (n == 0) return;
    Hanoi(n - 1, kaynak, ara, hedef, ref sayac);
    sayac++;
    Console.WriteLine($"{sayac}. hamle: {kaynak} -> {hedef}");
    Hanoi(n - 1, ara, hedef, kaynak, ref sayac);
}

static int HamleSayisi(int n) => n == 0 ? 0 : 2 * HamleSayisi(n - 1) + 1;

static bool PalindromMu(string s)
{
    if (s.Length <= 1) return true;
    if (s[0] != s[s.Length - 1]) return false;
    return PalindromMu(s.Substring(1, s.Length - 2));
}
```

Çıktı

```
Disk sayısı: 3
1. hamle: A -> C
2. hamle: A -> B
3. hamle: C -> B
4. hamle: A -> C
5. hamle: B -> A
6. hamle: B -> C
7. hamle: A -> C
Toplam hamle: 7 (formül: 7)
Metin: kayak
kayak palindromdur
```

Çıktıda mor görünen `3` ve `kayak` klavyeden yazılan girdidir. Özyinelemeli çağrılarda `ref` anahtar sözcüğünü hem tanımda hem çağrıda yazmayı unutma. Her çağrı yeni parametre kopyaları alır, `ref` ise tek bir değişkeni paylaşır.

EK A

C# Cheat Sheet

Sınavdan önceki son tekrar için kitabın sıkıştırılmış hali; anlamadığın satırda ilgili bölüme dön. Çıktılar Türkçe ayarlı Windows'taki gibi yazıldı (`3,5`); koddaki sabitler noktalıdır (`3.5`).

Üst düzey deyimler (dosyanın tamamı ana kod olur):

```
using System.Text;                // StringBuilder için gerekir
Console.WriteLine("Merhaba");     // her komut ; ile biter
```

Klasik biçim (Main metodu açık yazılır):

```
class Program
{
    static void Main(string[] args)
    {
        Console.WriteLine("Merhaba");
    }
}
```

Temeller

İlkel tipler

```
sbyte    1 B  −128 … 127
byte     1 B  0 … 255
short    2 B  −32 768 … 32 767
int      4 B  ±2,1 milyar
(varsayılan tamsayı)
uint     4 B  0 … 4,29 milyar
long     8 B  8_000_000_000L       (L soneki)
float    4 B  3.14f                (f soneki,
~7 hane)
double   8 B  3.14
(varsayılan ondalık)
decimal 16 B  3.14m                (m soneki,
para için)
char     2 B  'A' = 65             (UTF-16
birimi)
bool          true / false
string        "metin"              (başvuru
tipi, değiştirilemez)
```

- Otomatik: `int → long → float → double`; ters yön cast ister: `(int) 3.99` → `3` (keser).
- `int x = 3.5;` ve `decimal d = 3.14;` derlenmez; soneki unutma.
- Alan ve dizi elemanının varsayılanı `0`, `false`, `null`; **yerel değişkenin varsayılanı yok**: atamadan kullanmak derleme hatasıdır.

var, const

```
var sayi = 10;              // int (tip
çıkarımı); ilk değer zorunlu
const double Pi = 3.14159;   // derleme
zamanı sabiti; değiştirilemez
```

Operatörler (yüksekten düşüğe)

```
1   x.y  f(x)  a[i]  x++  x--  new
2   ++x  --x  -x  !  ~  (tip)
3   *  /  %
4   +  -
5   <<  >>
6   <  <=  >  >=  is  as
7   ==  !=
8   &
9   ^
10  |
11  &&
12  ||
13  ??
14  ?:
15  =  +=  -=  *=  /=  %=  ??=
```

- `7 / 2` → `3` · `7 / 2.0` → `3,5` · `-7 / 2` → `-3` · `-7 % 3` → `-1`
- `x++` önce kullanır sonra artırır; `++x` önce artırır.
- `"a" + 1 + 2` → `"a12"` · `1 + 2 + "a"` → `"3a"`
- `byte b = 1; b += 5;` derlenir; `b = b + 5;` derlenmez.
- `&&` ve `||` kısa devre yapar: sonuç belliyse sağ taraf çalışmaz.
- `==` ile `string` karşılaştırması içeriğe bakar (büyük/küçük harf duyarlı).

Bit işlemleri

```
5 & 3    1      5 | 3    7      5 ^ 3    6
~5      -6      1 << 3   8      16 >> 2  4
```

Taşma ve null

```
int x = int.MaxValue;
x+
+;                                    // hata
vermez: x = -2147483648
int? yas = null;
int deger = yas ??
0;                  // ?? : soldaki null ise
sağdaki
bool gecti = puan >= 50;
string sonuc = gecti ? "geçti" :
"kaldı";   // ?: üçlü operatör
```

- Sabit ifadede taşma derleme hatasıdır; değişkenle yapılan taşma sessizdir. `checked(...)` `OverflowException` fırlatır.

Girdi ve dönüşüm

```
string ad = Console.ReadLine() ??
"";            // boş gelirse ""
int n = Convert.ToInt32(Console.ReadLine());
double x =
Convert.ToDouble(Console.ReadLine()); // tr-
TR: 1,68 yaz
int m =
int.Parse("42");                        //
bozuksa FormatException
if (int.TryParse(Console.ReadLine(), out int
k)) { /* k kullanılabilir */ }
```

- `string s = Console.ReadLine();` nullable uyarısı (CS8600) verir; build bu yüzden kırılır, `?? ""` ile yaz.
- tr-TR'de ondalık ayraç virgüldür: `1.68` yanlış okunur, `1,68` yaz.
- `"i".ToUpper()` tr-TR'de `"İ"` verir.

Biçimlendirme

```
{x:F2}     3,50        iki ondalık, sabit
{x:N0}     1.234.567   binlik ayraç (tr-TR'de
nokta)
{x:D3}     007         en az 3 hane
{oran:P0}  yüzde biçimi (0,25 değeri yüzde 25
olarak)
{ad,8}     sağa yaslı, 8 karakter
{ad,-8}    sola yaslı, 8 karakter
```

Koşul ve switch

```
if (puan >= 85) harf = "A";
else if (puan >= 70) harf = "B";
else harf = "F";

switch (gun)                  // klasik: her
case break/return ile biter
{
    case 1:
    case
7:                  // boş case'ler
birleşebilir
        tur = "Tatil";
        break;
    default:
        tur = "İş";
        break;
}

tur = gun switch              // switch
ifadesi
{
    1 or 7 => "Tatil",
    _ => "İş"
};
```

```
string r = n switch { < 0 => "negatif", 0 =>
"sıfır", _ => "pozitif" };
if (sekil is Daire d) { /* d kullanılabilir
*/ }
```

- case'ten case'e düşmek derleme hatasıdır (CS0163).
- `if (x > 0);` boş komuttur; blok her zaman çalışır.

Döngüler

```
for (int i = 0; i < n; i++) { }
while (kosul) { }           // 0+ kez
do { } while (kosul);       // en az 1 kez
foreach (int x in dizi) { } // for-each
break;                      // döngüden çık
continue;                   // sonraki tura
geç
```

Metotlar, diziler, String

Metot

```
static int Topla(int a, int b) => a +
b;        // ifade gövdeli
static void Yaz(string s) {
Console.WriteLine(s); }
static void Artir(ref int x) { x+
+; }           // çağıranın değişkenini
değiştirir
static bool Dene(string s, out int
sonuc)       // out: metot değer atar
    => int.TryParse(s, out sonuc);
static void Selamla(string ad, string unvan
= "Öğr.") { }
static int Toplam(params int[] sayilar) =>
sayilar.Sum();
static double Topla(double a, double b) => a
+ b;   // aşırı yükleme
```

```
int sayi = 5;
Artir(ref
sayi);                      // sayi = 6
Selamla("Ali");                         //
isteğe bağlı parametre
Selamla(unvan: "Dr.", ad: "Ali");     //
adlandırılmış argüman
```

- `int` değer ile geçer; dizi başvuru ile geçer: `d[0] = 5` çağıranın dizisini değiştirir.
- Yalnız dönüş tipi farklı olan iki metot aşırı yüklenemez.
- `ref` ve `out` argümanında `ref` / `out` yazılır; `out` parametresi metot bitmeden atanmalıdır.

Diziler

```
int[] d = new int[5];          // {0,0,0,0,0}
int[] e = [3, 1, 2];           // koleksiyon ifadesi (C# 12)
e.Length                       // 3, parantez yok
e[^1]                          // son eleman
int[,] m = new int[3, 4];      // dikdörtgen dizi
m[1, 2]                        // m[1][2] değil
m.GetLength(0)                 // 3 (satır); GetLength(1) → 4
int[][] j = new int[3][];      // düzensiz (jagged): her satır ayrı dizi
j[0] = new int[] { 1, 2 };
int[] b = e;                   // AYNI dizi (başvuru)
Array.Sort(e);                 // e: 1, 2, 3 (yerinde)
Array.Reverse(e);
Array.IndexOf(e, 2);           // indeks; yoksa -1
```

- Geçersiz indeks: `IndexOutOfRangeException`. `a == b` dizilerde referansı karşılaştırır.

String (s = "Merhaba")

```
s.Length              7
s[1]                  'e'
s.Substring(1, 3)     "erh"     (başlangıç, uzunluk)
s.Substring(3)        "haba"
s.IndexOf('a')        4         (yoksa -1)
s.LastIndexOf('a')    6
s.Contains("hab")     true
s.StartsWith("Mer")   true
s.ToUpper()           "MERHABA"
s.Replace('a', 'o')   "Merhobo"
"  x ".Trim()         "x"
"a,b".Split(',')      ["a", "b"]
(string[])
string.Join("-", d)   "a-b"
string.Equals(a, b,
StringComparison.OrdinalIgnoreCase)
```

```
using System.Text;
var sb = new StringBuilder("ab");
sb.Append(3).Append('c');     // "ab3c"
sb.Insert(0, "x");            // "xab3c"
string t = sb.ToString();
```

- `string` değiştirilemez: `s.ToUpper();` tek başına `s`'yi değiştirmez; `s = s.ToUpper();` yaz.

Sınıf

```
public class Ogrenci
{
    private static int _sayac =
0;         // static: tüm nesnelerin ortak alanı
    private int
_notDegeri;                // alan (field)
    public string Ad { get;
init; }        // yalnız oluşturulurken atanır
    public int
Not                            // özellik (property)
    {
        get => _notDegeri;
        set { if (value >= 0 && value <=
100) _notDegeri = value; }
    }
    public Ogrenci(string ad, int
puan)    // kurucu: dönüş tipi yok
    {
        Ad =
ad;                            // this.Ad = ad; ile aynı
        Not = puan;
        _sayac++;
    }
    public Ogrenci(string ad) : this(ad, 0)
{ }
    public override string ToString() => Ad;
}
```

- Hiç kurucu yazmazsan parametresiz kurucu otomatik eklenir; bir tane yazarsan eklenmez.
- `static` üye `this`'e ve nesne alanlarına erişemez.
- Erişim belirleyicileri: `public`, `private` (varsayılan), `protected`, `internal`.
- `struct` değer tipidir, `class` başvuru tipidir. `record Nokta(int X, int Y);` değiştirilemez veri sınıfıdır.

Kalıtım ve polimorfizm

```
class Kisi
{
    public string Ad { get; }
    public Kisi(string ad) { Ad = ad; }
    public virtual string Tanit() => Ad;
}
class Ogretmen : Kisi
{
    public Ogretmen(string ad) : base(ad) { }
    public override string Tanit() =>
base.Tanit() + " (öğr.)";
}
Kisi h = new Ogretmen("Ali");      // yukarı
dönüşüm
h.Tanit();                         // "Ali
(öğr.)"
if (h is Ogretmen o) { }           // aşağı
dönüşüm: önce is ile kontrol et
Ogretmen? b = h as Ogretmen;       //
başarısızsa null
```

- Tek sınıftan türetilir: `class Kedi : Hayvan`; taban kurucuya `: base(...)` ile gidilir. `virtual` / `override` olmadan gizleme olur: `new` gerekir, CS0114 uyarısı çıkar.
- `abstract class` nesnesi oluşturulamaz; `abstract` üye alt sınıfta yazılmak zorundadır.
- `interface IUcabilir { void Uc(); }` — arayüz adları `I` ile başlar. `class Kus : Hayvan, IUcabilir`.
- `sealed class` başka sınıf tarafından türetilemez.

İstisnalar, koleksiyonlar, dosya

try / catch / finally

```
try
{
    int n = int.Parse(s);
    Console.WriteLine(100 / n);
}
catch (FormatException e) {
Console.WriteLine(e.Message); }
catch (DivideByZeroException) {
Console.WriteLine("Sıfıra bölünemez"); }
finally { Console.WriteLine("her zaman"); }

throw new ArgumentException("geçersiz
değer");
```

- Hata yakalamak zorunlu değildir; yakalanmayan istisna programı durdurur.
- `catch` içinde `throw;` istisnayı aynen iletir; `throw ex;` yığın izini sıfırlar.
- Özel istisna: `class YasHatasi : Exception { public YasHatasi(string m) : base(m) { } }`

```
FormatException            int.Parse("abc")
DivideByZeroException      x / 0 (tamsayı)
IndexOutOfRangeException   dizi sınırı dışı
NullReferenceException     null nesnenin üyesi
KeyNotFoundException       Dictionary'de
olmayan anahtar
FileNotFoundException      olmayan dosyayı
okumak
```

using

```
using var okuyucu = new
StreamReader("veri.txt");   // blok bitince
kapanır
```

Koleksiyonlar

```
List<int> liste =
new();       // hedef türlü new
liste.Add(5);  liste.Remove(5);
liste.RemoveAt(0);
liste.Count                    // özellik
(parantez yok)
liste.Contains(5);  liste[0];  //
indeksleyici

Dictionary<string, int> notlar = new();
notlar["ali"] = 85;            // ekle /
güncelle
notlar.TryGetValue("ali", out int n);   //
yoksa false, n = 0
notlar.ContainsKey("ali");
notlar.GetValueOrDefault("x"); // yoksa 0
foreach (var kv in notlar)
Console.WriteLine($"{kv.Key} {kv.Value}");

HashSet<int> kume = new() { 1, 2, 2 };   //
Count = 2 (tekrar yok)
Queue<int> kuyruk = new();     // FIFO:
Enqueue, Dequeue
Stack<int> yigin =
new();      // LIFO: Push, Pop
```

- `Dictionary` sıralı tutmaz; sıralı anahtar için `SortedDictionary`.
- `foreach` içinde koleksiyonu değiştirmek `InvalidOperationException` verir.

Generics

```
static T IlkEleman<T>(List<T> l) => l[0];
```

LINQ

```
int[] sayilar = { 1, 2, 3, 4, 5 };
var cift = sayilar.Where(x => x % 2 ==
0);    // 2, 4
var kareler = sayilar.Select(x => x *
x);     // 1, 4, 9, 16, 25
var sirali = sayilar.OrderByDescending(x =>
x);
int adet = sayilar.Count(x => x >
2);         // 3
int toplam =
sayilar.Sum();                  // 15
double ort =
sayilar.Average();              // 3
int ilk =
sayilar.First();                   // boşsa
istisna
int ilkVeyaYok = sayilar.FirstOrDefault(x =>
x > 100);   // yoksa 0
```

- `liste.Count` (özellik) ile `dizi.Count()` (LINQ metodu) farklıdır.

Dosyalar

```
File.WriteAllText("veri.txt", "ilk
satır\n");     // üzerine yazar
File.AppendAllText("veri.txt", "yeni
satır\n");   // sonuna ekler
string tum = File.ReadAllText("veri.txt");
string[] satirlar =
File.ReadAllLines("veri.txt");
bool varMi = File.Exists("veri.txt");

using var yazici = new StreamWriter("c.txt",
append: true);
yazici.WriteLine("satır");

using var okuyucu2 = new
StreamReader("veri.txt");
while (okuyucu2.ReadLine() is string
satir)       // null olunca biter
{
    string[] p =
satir.Split(';');               // CSV
}
```

- `new StreamWriter("c.txt")` ve `WriteAllText` eski içeriği **siler**; eklemek için `append: true` veya `AppendAllText`.
- Göreli yol, programın çalıştığı klasöre göre çözülür (Visual Studio'da genelde `bin\Debug\net10.0\`).

EK B

Terimler Sözlüğü

Kitapta geçen terimlerin Türkçe karşılıkları, İngilizceleri ve kısa açıklamaları. Terimler Türkçe adlarına göre alfabetik sıralandı. **Bölüm** sütunu terimin ilk geçtiği bölümü gösterir; "Proje 1" Öğrenci Not Sistemi'ni, "Final" dönem sonu sınavını ifade eder. İngilizce kaynak okurken, hata mesajlarını anlamaya çalışırken veya hocanın İngilizce terim kullandığı sorularda bu tabloya bak.

A	İngilizce	Açıklama	Bölüm
ad alanı	*namespace*	Tipleri gruplayan ad; ör. `System.Collections.Generic`.	1
akış	*stream*	Verinin sırayla okunup yazıldığı kanal (ör. `FileStream`).	14
alan	*field*	Sınıf veya yapı içinde tanımlanan, nesnenin verisini tutan değişken.	9
anahtar	*key*	Dictionary'de bir değere ulaşmak için kullanılan benzersiz değer.	13
anahtar kelime	*keyword*	Dilde özel anlamı olan, ad olarak kullanılamayan sözcük (`int`, `class`).	2
ara dil (IL)	*Intermediate Language*	Derleyicinin ürettiği, çalışma zamanında JIT ile makine koduna çevrilen ara kod.	1
aralık	*range*	Dizi ya da `string` için `başlangıç..bitiş` yazımı; bitiş indeksi dahil değildir (`a[1..4]`).	7
arayüz	*interface*	Bir sınıfın uygulaması gereken üyeleri tanımlayan sözleşme (`IUcabilir`).	11
asenkron	*asynchronous*	Beklenen işi (dosya okuma gibi) programı kilitlemeden yürüten çalışma biçimi; `async`, `await` ve `Task` ile yazılır.	14
aşırı yükleme	*overloading*	Aynı adlı, farklı parametreli metotlar tanımlama.	6
atama	*assignment*	`=` ile bir değişkene değer koyma.	2

B	İngilizce	Açıklama	Bölüm
banker yuvarlaması	*banker's rounding*	`Math.Round`'un varsayılanı: tam ortadaki sayıyı en yakın çift sayıya yuvarlar (`Math.Round(2.5)` → 2).	2
başvuru tipi	*reference type*	Değişkenin nesnenin adresini tuttuğu tip (`class`, `string`, dizi).	8
bayrak	*flag*	Doğru/yanlış durumunu tutan `bool` değişken.	5
belge yorumu	*documentation comment*	`///` ile yazılan, XML biçiminde belgelenen yorum.	1
biriktirme	*accumulator*	Döngüde toplam, çarpım gibi değeri her turda güncelleyen değişken.	5
birincil kurucu	*primary constructor*	Parametreleri sınıf adının yanında alan kurucu (`class Ogrenci(string ad)`).	9
bir kayma hatası	*off-by-one error*	Döngü sınırını bir eksik ya da fazla yazma hatası (`i <= dizi.Length`).	5
bool ifade	*boolean expression*	Sonucu `true` veya `false` olan ifade.	4

C-Ç	İngilizce	Açıklama	Bölüm
CSV	*Comma-Separated Values*	Değerleri `;` veya `,` ile ayrılmış düz metin tablo dosyası.	14
çağrı	*call*	Bir metodu adı ve argümanlarıyla çalıştırma.	6
çağrı yığını	*call stack*	Çalışmakta olan metot çağrılarının sırasını tutan bellek bölgesi.	6
çalışma zamanı	*runtime*	Programın derlenip çalıştığı süre; derleme zamanının karşıtı.	1
çerçeve	*stack frame*	Her metot çağrısı için yığında ayrılan, yerel değişkenleri tutan bellek parçası.	6
çöp toplayıcı	*garbage collector*	Erişilemeyen nesnelerin belleğini otomatik geri veren .NET bileşeni.	9

D	İngilizce	Açıklama	Bölüm
değişken	*variable*	Adı ve tipi olan, değer saklayan bellek alanı.	2
değiştirilebilir	*mutable*	Oluşturulduktan sonra içeriği değiştirilebilen nesne (`StringBuilder`, `List<T>`).	8
değiştirilemez	*immutable*	Oluşturulduktan sonra içeriği değiştirilemeyen tip; `string` bunun örneğidir.	2
demet	*tuple*	Birkaç değeri tek pakette toplayan yapı; `(int bolum, int kalan)` ile bir metottan çoklu değer döndürmeyi sağlar.	6
derleme zamanı	*compile time*	Kodun derleyici tarafından denetlendiği ve IL'e çevrildiği aşama.	11
derleyici	*compiler*	Kaynak kodu IL'e çeviren araç (C# derleyicisi: Roslyn).	1
destek alanı	*backing field*	Otomatik olmayan özelliğin değerini sakladığı gizli alan.	9
deyim	*statement*	Tek başına çalıştırılabilen komut; genelde `;` ile biter.	1
dinamik bağlama	*dynamic binding*	Çağrılacak metodun çalışma zamanında nesnenin gerçek tipine göre seçilmesi (`virtual`).	10
dizi	*array*	Aynı tipte, sabit sayıda elemanı indeksle tutan yapı (`int[]`).	7
doğal sıralama	*natural ordering*	Nesnenin `IComparable` ile tanımlanan varsayılan sıralaması.	13
doğruluk tablosu	*truth table*	Mantıksal ifadenin tüm doğru/yanlış kombinasyonlarındaki sonucunu gösteren tablo.	4
doğrusal arama	*linear search*	Elemanları baştan sona tek tek tarayan O(n) arama.	7
döngü	*loop*	Bir kod bloğunu koşul sağlandıkça tekrar eden yapı (`for`, `while`).	5

E	İngilizce	Açıklama	Bölüm
elmas problemi	*diamond problem*	Çoklu kalıtımdaki belirsizlik; C#'ta bir sınıfın tek taban sınıfı olduğu için oluşmaz.	10
enterpolasyonlu metin	*string interpolation*	`$"Ad: {ad}"` biçiminde metin içine değer yerleştiren yazım.	2
erişim belirleyici	*access modifier*	`public`, `private`, `protected`, `internal` gibi üyenin görünürlüğünü belirleyen sözcük.	9
ertelenmiş çalıştırma	*deferred execution*	LINQ sorgusunun, sonucu kullanılana kadar çalıştırılmaması.	13
ezme	*override*	Alt sınıfta üst sınıfın `virtual` metodunu yeniden yazma (`override`).	10

F-G-Ğ-H	İngilizce	Açıklama	Bölüm
field anahtar sözcüğü	*field keyword*	C# 14: özelliğin `get`/`set` bloklarında derleyicinin ürettiği gizli destek alanını gösteren sözcük.	9
genel tip	*generic type*	Tip parametresi alan sınıf veya metot; `List<T>` bir örnektir.	13
gizleme	*hiding*	Türetilmiş sınıfın `new` ile üst sınıfın üyesini gizlemesi (`virtual`/`override` yoksa).	10
gölgeleme	*shadowing*	Yerel değişken veya parametrenin, aynı adlı alanı gizlemesi.	9
göreli yol	*relative path*	Programın çalıştığı klasöre göre belirlenen dosya yolu (`veri.txt`).	14
hafızalama	*memoization*	Aynı girdi için daha önce hesaplanan sonucu saklayıp tekrar kullanma.	15
ham metin sabiti	*raw string literal*	`"""..."""` ile yazılan, `\` ve `"` karakterlerinin kaçış gerektirmediği çok satırlı metin (C# 11).	8
hata listesi	*Error List*	Visual Studio'da derleme hatalarını ve uyarılarını listeleyen pencere.	1

I-İ-J	İngilizce	Açıklama	Bölüm
iç içe if	*nested if*	Bir `if` bloğunun içinde başka bir `if`.	4
ifade	*expression*	Bir değer üreten kod parçası (`x + 1`).	4
ikili arama	*binary search*	Sıralı dizide aralığı her adımda yarıya indiren O(log n) arama.	15
ilişkisel operatörler	*relational operators*	`<`, `<=`, `>`, `>=`, `==`, `!=` gibi iki değeri karşılaştıran operatörler.	4
ilk giren ilk çıkar	*FIFO*	Kuyrukta (`Queue<T>`) ilk eklenen elemanın ilk çıkarıldığı düzen.	13
indeks	*index*	Elemanın 0'dan başlayan sıra numarası (`dizi[0]`).	7
isimlendirme geleneği	*naming conventions*	Değişken, sınıf ve metot adlarının yazım kuralları (ör. PascalCase, camelCase).	2
istisna	*exception*	Çalışma zamanında oluşan, normal akışı bozan olay; `try-catch` ile yakalanır.	1
işaretsiz tip	*unsigned type*	Eksi değer tutmayan tam sayı tipi: `byte`, `ushort`, `uint`, `ulong`.	2
iterasyon	*iteration*	Döngünün her bir turu.	5
izleme tablosu	*trace table*	Kodu elle çalıştırırken değişken değerlerini adım adım yazdığın tablo.	5
JSON serileştirme	*JSON serialization*	Nesneyi JSON metnine çevirme (`JsonSerializer.Serialize`) ve geri okuma (`Deserialize<T>`).	14

K	İngilizce	Açıklama	Bölüm
kaçış dizisi	*escape sequence*	`\n`, `\t`, `\"` gibi `\` ile başlayan özel karakter dizisi.	1
kalıtım	*inheritance*	Bir sınıfın başka bir sınıfın üye ve davranışlarını devralması (`: TabanSinif`).	10
kapsam	*scope*	Bir değişkenin erişilebildiği kod bölgesi.	6
kapsülleme	*encapsulation*	Alanları `private` yapıp erişimi özellik ve metotlarla denetleme.	9
kaynak kod	*source code*	İnsanın yazdığı program metni (`.cs` dosyası).	1
kısa devre	*short-circuit evaluation*	Ve (`&&`) ile veya'da sonuç belliyse sağ tarafın hiç çalıştırılmaması.	4
koleksiyon	*collection*	Birden çok elemanı tutan yapı (`List<T>`, `Dictionary<TKey,TValue>`).	13
koleksiyon ifadesi	*collection expression*	`int[] a = [1, 2, 3];` gibi koleksiyonu kısa yazan ifade (C# 12).	7
kompozisyon	*composition*	Bir sınıfın başka bir sınıfı alan olarak tutarak onun işlevini kullanması.	10
koşul	*condition*	Sonucu `true` veya `false` olan ifade.	4
kurucu	*constructor*	Nesne oluşturulurken çalışan, sınıf adıyla aynı, dönüş tipsiz metot.	9
kuyruk özyinelemesi	*tail recursion*	Özyinelemeli çağrının metodun son işlemi olduğu biçim.	15

L	İngilizce	Açıklama	Bölüm
lambda ifadesi	*lambda expression*	Kısa, isimsiz metot: `x => x * 2`.	13
LINQ	*Language Integrated Query*	Dile gömülü sorgulama yapısı; `Where`, `Select`, `OrderBy` ile koleksiyonları süzer.	13

M	İngilizce	Açıklama	Bölüm
mantıksal operatörler	*logical operators*	Doğru/yanlış değerleri birleştiren ve, veya, değil işleçleri.	4
metot	*method*	Bir işi yapan, adı olan ve çağrılabilen kod bloğu.	1
metot imzası	*method signature*	Metodun adı ve parametre tipleri; dönüş tipi imzaya dahil değildir.	6
mutlak yol	*absolute path*	Sürücüden başlayan tam dosya yolu (`C:\veri\a.txt`).	14

N	İngilizce	Açıklama	Bölüm
nesne	*object*	Bir sınıftan `new` ile oluşturulan, durumu ve davranışı olan varlık.	9
nesne başlatıcı	*object initializer*	`new Ogrenci { Ad = "Ali" }` biçiminde özellikleri oluşturma anında atama.	9
nöbetçi değer	*sentinel value*	Girdinin bittiğini bildirmek için seçilen özel değer (ör. `-1`).	5
nullable değer tipi	*nullable value type*	`null` da tutabilen değer tipi: `int?`, `double?`, `bool?` (`HasValue`, `Value`).	9

O–Ö	İngilizce	Açıklama	Bölüm
olay	*event*	Bir nesnenin başkalarına haber verme yolu; `event` ile tanımlanır, `+=` ile abone olunur.	13
öbek	*heap*	`new` ile oluşturulan sınıf nesnelerinin (ve içlerindeki alanların) tutulduğu bellek bölgesi.	9
ön ek artırma	*prefix increment*	`++x`: önce artırır, sonra yeni değeri kullanır.	2
örnek	*instance*	Bir sınıftan oluşturulmuş belirli bir nesne.	9
örnek oluşturma	*instantiation*	`new` ile sınıftan nesne üretme işlemi.	9
özellik	*property*	`get` ve `set` ile alana denetimli erişim sağlayan üye (`public int Not { get; set; }`).	9
özyineleme	*recursion*	Bir metodun kendini daha küçük bir problemle çağırması.	15
özyinelemeli durum	*recursive case*	Metodun kendini daha küçük girdiyle çağırdığı dal.	15

P–R	İngilizce	Açıklama	Bölüm
paralel diziler	*parallel arrays*	Aynı indeksin farklı dizilerde aynı kaydı gösterdiği, ayrı ayrı tutulan diziler.	Proje 1
referans	*reference*	Nesnenin bellekteki adresine işaret eden değer.	7

S–Ş	İngilizce	Açıklama	Bölüm
sabit değer	*literal*	Koda doğrudan yazılan değer: `42`, `3.14`, `'A'`, `"Ali"`.	2
salt okunur	*read-only*	Değeri okunabilen ama atanamayan üye (ör. dizinin `Length` özelliği).	7
sınıf	*class*	Nesnelerin alanlarını ve metotlarını tanımlayan şablon.	1
son ek artırma	*postfix increment*	`x++`: önce mevcut değeri kullanır, sonra artırır.	2
son giren ilk çıkar	*LIFO*	Yığında (`Stack<T>`) son eklenen elemanın ilk çıkarıldığı düzen.	13
sondan indeks	*index from end*	`^` ile yazılan, sondan sayan indeks: `a[^1]` son elemandır.	7
sonsuz döngü	*infinite loop*	Koşulu hiç yanlış olmadığı için bitmeyen döngü.	5
sorgu sözdizimi	*query syntax*	LINQ'te `from x in ... select x` biçiminde yazılan sorgu.	13
soyut sınıf	*abstract class*	`new` ile nesnesi oluşturulamayan, soyut üye içerebilen sınıf.	11
soyut üye	*abstract member*	Gövdesi olmayan, türetilmiş sınıfın yazmak zorunda olduğu üye.	11

T	İngilizce	Açıklama	Bölüm
taban sınıf	*base class*	Bir sınıfın türediği üst sınıf.	10
tanımlama	*declaration*	Değişken veya metodu tipi ve adıyla bildirme.	2
tanımlayıcı	*identifier*	Değişken, metot veya sınıfa verilen ad.	2
taşma	*overflow*	Değerin tipin sınırını aşıp ters uçtan dönmesi; değişkenle yapıldığında hata vermez.	2
temel durum	*base case*	Özyinelemenin kendini çağırmadan döndüğü durum.	15
temsilci	*delegate*	Bir metodu değer gibi taşıyan tip (`Func`, `Action`, `Predicate<T>`); lambda bir temsilci değeridir.	13
tip çıkarımı	*type inference*	Derleyicinin tipi sağdaki değerden bulması (`var`).	2
tohum	*seed*	Rastgele sayı üretecinin başlangıç değeri; `new Random(42)` her çalıştırmada aynı diziyi verir.	6

U-Ü	İngilizce	Açıklama	Bölüm
üçlü operatör	*ternary operator*	`koşul ? a : b` biçimindeki tek satırlık if-else.	4
üst düzey deyimler	*top-level statements*	Dosyanın doğrudan `Console.WriteLine` ile başlamasına izin veren özellik; `Main` yazmaya gerek kalmaz.	1
üstel büyüme	*exponential growth*	Her adımda değerin katlanarak arttığı büyüme (ör. 2^n).	15

V-Y-Z	İngilizce	Açıklama	Bölüm
varsayılan değer	*default value*	Alan ve dizi elemanına otomatik verilen değer: `0`, `false`, `null`.	7
varsayılan kurucu	*default constructor*	Hiç kurucu yazılmazsa derleyicinin eklediği parametresiz kurucu.	9
yakalama	*catch*	İstisnayı `catch` bloğunda yakalayıp işleme.	12
yerel değişken	*local variable*	Metot veya blok içinde tanımlanan, sadece orada geçerli değişken.	2
yerinde	*in-place*	Yeni dizi oluşturmadan, aynı dizi üzerinde yapılan işlem (`Array.Sort`).	7
yığın	*stack*	Metotların yerel değişkenlerinin tutulduğu, son giren ilk çıkar mantığıyla çalışan bellek bölgesi; yerel değer tipi değişkenler burada durur.	9
yol	*path*	Dosyanın konumunu gösteren metin (`veri.txt`, `C:\veri\a.txt`).	14
yorum	*comment*	Derleyicinin yok saydığı açıklama: `//`, `/* */`, `///`.	1
yorumlayıcı	*interpreter*	Kodu satır satır okuyup o anda çalıştıran program; derleyicinin karşıtı.	1
yukarı dönüşüm	*upcasting*	Türetilmiş sınıf nesnesini taban sınıf tipindeki bir değişkenle göstermek.	10
yüksek seviyeli programlama dili	*high-level language*	İnsanın okuyabildiği, makine dilinden soyutlanmış programlama dili (ör. C#).	1
zincirleme	*chaining*	Metot çağrılarını art arda bağlayarak tek satırda yazma (`sb.Append(1).Append(2)`).	8

EK C

Sık Karşılaşılan Hata Mesajları

Bu ekte, C# yazarken en sık göreceğin derleme hatalarını, uyarıları ve çalışma zamanı istisnalarını (exception) topladık. Her mesaj, onu **gerçekten** üreten küçük bir programla gösteriliyor. Program bloğunun altındaki metin, derleyicinin (compiler) ya da çalışma zamanının verdiği çıktının kendisidir. Mesajlar İngilizcedir; Türkçe açıklamaları yanlarında bulacaksın.

Derleyici mesajını okumak

Derleyici bir sorun bulursa şöyle bir satır yazar:

```
ad.cs(3,9): error CS0103: The name 'Toplam' does not exist in the current context
```

- `ad.cs`: sorunlu dosyanın adı. Kendi projende bu genellikle `Program.cs` olur.
- `(3,9)`: 3. satır, 9. sütun. Derleyici sorunu bu konumda fark etti; asıl neden daha önce olabilir.
- `error` ya da `warning`: `error` derlemeyi durdurur, program hiç çalışmaz. `warning` (uyarı) derlemeyi durdurmaz, ama genellikle bir sorunun işaretidir.
- `CS0103`: hata kodu. Türkçe Windows'ta Visual Studio kullanırsan mesajın metni Türkçe çıkabilir; **kod her zaman aynıdır**. Bu yüzden mesajın metnini değil, kodu ara: `CS0103 C#`.

Visual Studio'da aynı bilgiler **Hata Listesi** penceresinde durur; bir satıra çift tıklarsan kod o satırda açılır.

> **İpucu: Hata mesajı okuma alışkanlığı**
>
> - Derleme hatalarında **her zaman ilk hatadan başla**. Tek bir eksik `;` ya da `}` arkasından birçok sahte hata doğurabilir; ilkini düzeltip yeniden derle.
> - Eksik noktalı virgül hatasında konum, `;` in olması gereken yerdir: satırın sonu. Satır numarası, `;` yi eklemen gereken satırı gösterir.
> - Çalışma zamanı hatasında yığın izinde (stack trace) `in ... ad.cs:line N` içeren ilk satır, hatanın olduğu yerdir.

Derleme hataları

Aşağıdaki programların her biri bilerek hatalıdır. Derleyici, altındaki mesajı verir ve program çalışmaz.

`CS1002: ; expected`

```
Console.WriteLine("Merhaba")
Console.WriteLine("Dunya");
```

Derleyici çıktısı – program derlenmez

```
noktaliVirgul.cs(1,29): error CS1002: ; expected
```

Ne demek? Derleyici, 1. satırdaki komutun bittiğini gösteren `;` işaretini bekledi ama bulamadı. Hata, `;` in olması gereken yerde, yani 1. satırın sonunda gösterilir.

Nasıl düzeltilir? Satırın sonuna `;` ekle. Her deyim (komut, değişken tanımı) `;` ile biter; yeni satıra geçmek onu bitirmez.

`CS0103: does not exist in the current context`

```
int toplam = 10;
Console.WriteLine(Toplam);
```

Derleyici çıktısı – program derlenmez

```
isimYok.cs(2,19): error CS0103: The name 'Toplam' does not exist in the current context
```

Ne demek? "Bu adı tanımıyorum." C# büyük/küçük harfe duyarlıdır: `Toplam` ile `toplam` iki ayrı addır. Sen `toplam` tanımladın.

Nasıl düzeltilir? Üç olası neden var: (1) ad yanlış yazılmış ya da harf farklı; (2) değişken kapsam dışında (ör. `for` bloğunun içinde tanımlanıp dışarıda kullanılmış); (3) sınıf için `using` satırı eksik (ör. `Regex` için `using System.Text.RegularExpressions;`).

`CS0029: Cannot implicitly convert type`

```
int yas = "19";
Console.WriteLine(yas);
```

Derleyici çıktısı – program derlenmez

```
metinSayi.cs(1,11): error CS0029: Cannot implicitly convert type 'string' to 'int'
```

Ne demek? `"19"` bir metindir (`string`), `int` değildir. C# metni sayıya kendiliğinden çevirmez; tip uyumsuzluğu bu yüzden derleme anında yakalanır.

Nasıl düzeltilir? Dönüşümü sen yap: `int yas = int.Parse("19");` (Bölüm 3). Kullanıcıdan gelen metin için `int.TryParse` daha güvenlidir (Bölüm 12).

`CS0266: An explicit conversion exists`

```
double ortalama = 85.5;
int tam = ortalama;
Console.WriteLine(tam);
```

Derleyici çıktısı – program derlenmez

```
ondalikTam.cs(2,11): error CS0266: Cannot implicitly convert type 'double' to 'int'. An explicit
conversion exists (are you missing a cast?)
```

Ne demek? `double` → `int` dönüşümü veri kaybedebileceği için (`85.5` → `85`) otomatik yapılmaz. Mesajın sonundaki "are you missing a cast?" kısmı, bilinçli bir dönüşüm (cast) yazmanı önerir.

Nasıl düzeltilir? Kesirli kısmın atılacağını bilerek yaz: `int tam = (int)ortalama;`. Sonuç `85` olur; yuvarlama yapılmaz. Değer ondalıklı kalmalıysa değişkenin tipini `double` bırak.

> **Sınavda Çıkar: Sınavda çıkar**
>
> `int x = "5";` (CS0029) ile `int x = 2.5;` (CS0266) farkını sorabilirler. Birincisinde mesaj yalnızca tipin uymadığını söyler; ikincisinde ise bir cast önerisi vardır. Mesajın sonuna bakarak ayırt et.

`CS0165: Use of unassigned local variable`

```
int puan = 75;
string harf;
if (puan >= 50) harf = "Geçti";
Console.WriteLine(harf);
```

Derleyici çıktısı – program derlenmez

```
atamaYok.cs(4,19): error CS0165: Use of unassigned local variable 'harf'
```

Ne demek? `harf` yalnızca `puan >= 50` doğruysa atanıyor. Derleyici her yolda `harf`'in bir değer aldığını kanıtlayamadığı için yerel değişkeni kullanmana izin vermez. Yerel değişkenlerin (alanların aksine) varsayılan değeri yoktur.

Nasıl düzeltilir? Tanımlarken ilk değer ver (`string harf = "";`) ya da her yolda atama yap: `else harf = "Kaldı";` ekle.

`CS0128: already defined in this scope`

```
int sayi = 10;
int sayi = 20;
Console.WriteLine(sayi);
```

Derleyici çıktısı – program derlenmez

```
ikiKez.cs(2,5): error CS0128: A local variable or function named 'sayi' is already defined in this
scope
```

Ne demek? Aynı kapsamda (aynı blokta) aynı adla ikinci bir yerel değişken tanımlanamaz. İkinci tanım birinciyi değiştirmez; derleyici çakışmayı reddeder.

Nasıl düzeltilir? İkinciyi atamaya çevir (`sayi = 20;`) ya da adını değiştir (`sayi2`). Ayrı bloklardaki aynı ad serbesttir: iki ayrı `for` döngüsünde `int i` yazabilirsin.

`CS0161: not all code paths return a value`

```
Console.WriteLine(Isaret(5));
static int Isaret(int n)
{
    if (n > 0) return 1;
    else if (n < 0) return -1;
}
```

Derleyici çıktısı – program derlenmez

```
donusYok.cs(2,12): error CS0161: 'Isaret(int)': not all code paths return a value
```

Ne demek? `Isaret` `int` döndüreceğini söylüyor, ama `n == 0` durumunda `return` olmadan bitiyor. Derleyici, `if` koşullarının hepsinin yanlış olabileceğini düşünür ve bunu kabul etmez.

Nasıl düzeltilir? Metodun sonuna her durumu karşılayan bir `return` ekle (`return 0;`) ya da son `else if`'i düz `else` yap.

`CS0163: Control cannot fall through from one case label`

```
int secim = 1;
switch (secim)
{
    case 1: Console.WriteLine("Bir");
    case 2: Console.WriteLine("İki"); break;
}
```

Derleyici çıktısı – program derlenmez

```
dusme.cs(4,5): error CS0163: Control cannot fall through from one case label ('case 1:') to another
```

Ne demek? `case 1` bloğu `break` ile bitmiyor; program kendiliğinden `case 2`'ye düşmek istiyor. C#'ta bu bir derleme hatasıdır. Böylece unutulmuş bir `break` daha baştan yakalanır.

Nasıl düzeltilir? Her `case`'i `break`, `return` ya da `throw` ile bitir. Aynı işi yapan etiketleri birleştirmek serbesttir: `case 1:` ve `case 2:` bir şey yazmadan art arda gelebilir.

`CS1501: No overload for method`

```
Console.WriteLine(Ortalama(70, 80, 90));
static double Ortalama(int a, int b) => (a + b) / 2.0;
```

Derleyici çıktısı – program derlenmez

```
argumanSayisi.cs(1,19): error CS1501: No overload for method 'Ortalama' takes 3 arguments
```

Ne demek? `Ortalama` iki parametre alıyor; sen üç argüman verdin. Derleyici, bu sayıda argüman alabilecek bir sürüm (aşırı yükleme, overload) bulamadığını söyler.

Nasıl düzeltilir? Çağrıyı tanıma uydur (iki sayı ver) ya da üç parametreli yeni bir `Ortalama` yaz. C#, aynı adla farklı parametre listesine sahip metotlara izin verir. Argümanların tipi de tanıma uymalıdır.

`CS0120: An object reference is required`

```
Console.WriteLine(Hesap.Topla(2, 3));
class Hesap
{
    public int Topla(int a, int b) => a + b;
}
```

Derleyici çıktısı – program derlenmez

```
statikDegil.cs(1,19): error CS0120: An object reference is required for the non-static field,
method, or property 'Hesap.Topla(int, int)'
```

Ne demek? `Topla` nesneye ait (static olmayan) bir metot. Sınıf adıyla (`Hesap.Topla`) çağrılamaz; hangi `Hesap` nesnesinin `Topla`'sının çalışacağı belli değildir.

Nasıl düzeltilir? İki yol var: nesne oluşturup onun üzerinden çağır (`new Hesap().Topla(2, 3)`), ya da metot gerçekten nesneye bağlı değilse `static` yap. Bölüm 6'da yazdığın yardımcı metotlar `static` olduğu için ana koddan doğrudan çağrılabiliyordu.

`CS0122: is inaccessible due to its protection level`

```
Console.WriteLine(Ogrenci.Sayac);
class Ogrenci
{
    static int Sayac = 3;
}
```

Derleyici çıktısı – program derlenmez

```
erisim.cs(1,27): error CS0122: 'Ogrenci.Sayac' is inaccessible due to its protection level
```

Ne demek? `Sayac`'a erişim yetkin yok. Erişim belirleyici (access modifier) yazılmadığında sınıf üyesi varsayılan olarak `private` olur: yalnızca kendi sınıfının içinde kullanılabilir.

Nasıl düzeltilir? Dışarıdan kullanılacaksa `public static int Sayac = 3;` yaz. Ama alanı dışarı açmak genelde iyi değildir; özellik (property) kullanmak daha doğrudur (Bölüm 9).

`CS0200: cannot be assigned to -- it is read only`

```
string ad = "Ali";
ad[0] = 'a';
Console.WriteLine(ad);
```

Derleyici çıktısı – program derlenmez

```
saltOkunur.cs(2,1): error CS0200: Property or indexer 'string.this[int]' cannot be assigned to -- it
is read only
```

Ne demek? `string` değişmezdir (immutable): bir metnin karakteri değiştirilemez. `ad[0]` yalnızca okunabilir bir indeksleyicidir (indexer). Aynı mesaj, yalnızca `get` olan kendi özelliğine (property) değer atamaya çalıştığında da çıkar.

Nasıl düzeltilir? Yeni bir metin üret: `ad = "a" + ad.Substring(1);`. Karakterleri tek tek değiştirmen gerekiyorsa `ToCharArray()` ile karakter dizisine çevir, değiştir ve `new string(dizi)` ile yeniden oluştur (Bölüm 8).

`CS0019: Operator cannot be applied to operands`

```
bool gecti = true;
int puan = 5;
Console.WriteLine(gecti + puan);
```

Derleyici çıktısı – program derlenmez

```
boolTopla.cs(3,19): error CS0019: Operator '+' cannot be applied to operands of type 'bool' and
'int'
```

Ne demek? `+` operatörü `bool` ile `int` arasında tanımlı değildir. C# farklı tipleri kendiliğinden birleştirmez; metin birleştirme bunun dışındadır.

Nasıl düzeltilir? Ne yapmak istediğine karar ver. Sonucu yazdırmak için `$"{gecti} {puan}"` gibi metin enterpolasyonu kullan; koşula göre hesap yapacaksan `if (gecti)` yaz.

`CS1061`: `does not contain a definition for`

```
string ad = "Ali";
Console.WriteLine(ad.Lenght);
```

Derleyici çıktısı – program derlenmez

```
yazimHatasi.cs(2,22): error CS1061: 'string' does not contain a definition for 'Lenght' and no
accessible extension method 'Lenght' accepting a first argument of type 'string' could be found (are
you missing a using directive or an assembly reference?)
```

Ne demek? `string` türünde `Lenght` diye bir üye yok. Doğrusu `Length`; bu bir yazım hatasıdır. Mesajın sonundaki "using directive" uyarısı bu örnekte yanıltıcıdır: sorun yazımdadır, `using` ile ilgisi yoktur.

Nasıl düzeltilir? Nokta koyduğunda açılan öneri listesi (IntelliSense) bu hatayı baştan önler. Doğru ad: `ad.Length` (Bölüm 8).

Uyarılar

Uyarı (warning) programı durdurmaz, ama çoğu zaman bir sorunun işaretidir. Aşağıdaki programlar bilerek uyarı üretir; kitabın diğer programlarında uyarı yoktur.

`CS8600`: `Converting null literal or possible null value`

```
string ad = Console.ReadLine();
Console.WriteLine($"Merhaba {ad}");
```

Derleyici uyarısı (warning)

```
uyariNull.cs(1,13): warning CS8600: Converting null literal or possible null value to non-nullable
type.
```

Ne demek? `Console.ReadLine()` girdi akışı kapanmışsa `null` dönebilir. C#, boş olabilecek bir değeri (`string?`) boş olamayan bir `string` değişkene atadığını söylüyor.

Nasıl düzeltilir? Bölüm 3'teki kalıbı kullan: `string ad = Console.ReadLine() ?? "";`. `null` gelirse boş metin atanır ve uyarı kaybolur.

`CS0168` ve `CS0219`: `never used`

```
int sayac;
int puan = 5;
Console.WriteLine("Merhaba");
```

Derleyici uyarısı (warning)

```
kullanilmayan.cs(1,5): warning CS0168: The variable 'sayac' is declared but never used
kullanilmayan.cs(2,5): warning CS0219: The variable 'puan' is assigned but its value is never used
```

Ne demek? `sayac` hiç kullanılmadı (CS0168). `puan`'a değer verildi ama hiç okunmadı (CS0219). Program çalışır, ama bu satırlar gereksizdir.

Nasıl düzeltilir? Gereksiz değişkeni sil. Değişkeni gerçekten kullanacaksan okuyan bir satır yaz (ör. `Console.WriteLine(puan);`).

`CS0642: Possible mistaken empty statement`

```
int x = 7;
if (x > 0);
    Console.WriteLine("Pozitif");
```

```
Derleyici uyarısı (warning)
bosIfNoktali.cs(2,11): warning CS0642: Possible mistaken empty statement
```

Ne demek? `if (x > 0);` satırının sonundaki `;` `if`'in gövdesini boş bırakır. Bu yüzden `Console.WriteLine` koşuldan bağımsız her zaman çalışır. Alttaki girintiye bakıp kandırılma; derleyici girintiye değil noktalı virgüle bakar.

Nasıl düzeltilir? `if (x > 0)` satırının sonundaki `;` yi sil. Koşul sağlanınca birden fazla komut çalışacaksa süslü parantezle grupla.

Çalışma zamanı hataları

Derlenen program da çalışırken hata verebilir. Yakalanmayan bir istisna (exception) programı durdurur ve konsola şu biçimde yazar:

```
Unhandled exception. System.IstisnaAdi: aciklama
   at ... in Program.cs:line N
```

İlk satır istisnanın adını ve açıklamasını, `in ... line N` kısmı hatanın dosyada nerede olduğunu söyler. Windows'ta bu yol tam klasör yolu olarak görünür.

Sık karşılaşılan istisnalar:

İstisna (exception)	Ne zaman olur?	Sık çözüm
`FormatException`	Metin sayıya çevrilemez (`int.Parse("12a")`)	`int.TryParse` ile kontrol et
`IndexOutOfRangeException`	Dizinin sınırı dışına erişilir	Döngüyü `i < dizi.Length` yap ya da `foreach` kullan
`NullReferenceException`	`null` bir referans üzerinden üye kullanılır	Kullanmadan önce `null` kontrolü yap ya da `?.` kullan
`DivideByZeroException`	Tam sayı sıfıra bölünür	Böleni kontrol et
`KeyNotFoundException`	Sözlükte olmayan anahtar `[]` ile okunur	`TryGetValue` kullan (Bölüm 13)
`InvalidCastException`	Nesne istenen tipe dönüştürülemez (ör. `(int)(object)"5"`)	`is` ile tipi kontrol et
`FileNotFoundException`	Açılmak istenen dosya yok	`File.Exists` ile kontrol et, yolu doğrula (Bölüm 14)
`InvalidOperationException`	Nesnenin durumu işleme uygun değil (ör. boş dizide `First()`)	`Any()` ile kontrol et ya da `FirstOrDefault()`
`OverflowException`	`checked` bağlamında tamsayı taşar	Değer aralığını kontrol et
`StackOverflowException`	Sonsuz özyineleme; **yakalanamaz**, program çöker	Temel durum ekle (Bölüm 15)

> **● Sık Yapılan Hata: Sık yapılan hata**
>
> İstisnayı `catch { }` ile boş bırakıp geçmek hatayı gizler; program yanlış sonuçla devam eder ya da hata başka bir yerde ortaya çıkar. Önce nedenini düzelt, yakalamayı son çare olarak kullan (Bölüm 12).

`FormatException: input string was not in a correct format`

```
Console.WriteLine(int.Parse("42") + 1);
Console.WriteLine(int.Parse("12a") + 1);
```

Çıktı – program yakalanmamış istisna ile sonlanır

```
43
Unhandled exception. System.FormatException: The input string '12a' was not in a correct format.
   at Program.<Main>$(String[] args) in formatHatasi.cs:line 2
```

Ne demek? `int.Parse` metni sayıya çeviremedi. Mesaj sorunlu metni tek tırnak içinde gösterir. İlk satır çalıştı (43 yazdı); ikincisinde program durdu. `"3.5"` ve `""` da aynı hatayı verir.

Nasıl düzeltilir? Kullanıcıdan gelen metni `int.TryParse` ile çevir: `if (int.TryParse(metin, out int sayi)) ...`.

`IndexOutOfRangeException: Index was outside the bounds of the array`

```
int[] notlar = { 70, 85, 90, 60, 75 };
int toplam = 0;
for (int i = 0; i <= notlar.Length; i++) toplam += notlar[i];
Console.WriteLine(toplam);
```

Çıktı – program yakalanmamış istisna ile sonlanır

```
Unhandled exception. System.IndexOutOfRangeException: Index was outside the bounds of the array.
   at Program.<Main>$(String[] args) in indeksHatasi.cs:line 3
```

Ne demek? 5 elemanlı bir dizinin geçerli indeksleri 0–4'tür. `i <= notlar.Length` koşulu 5. indekse de bakar; `notlar[5]` yoktur. C# her dizi erişiminde sınırı denetler, bu yüzden hata çalışma anında görünür.

Nasıl düzeltilir? Koşulu `i < notlar.Length` yap. Ya da indeks hiç kullanma: `foreach (int n in notlar) toplam += n;`.

`NullReferenceException: Object reference not set to an instance of an object`

```
string[] adlar = new string[3];
adlar[0] = "Ali";
Console.WriteLine(adlar[1].ToUpper());
```

Çıktı – program yakalanmamış istisna ile sonlanır

```
Unhandled exception. System.NullReferenceException: Object reference not set to an instance of an
object.
   at Program.<Main>$(String[] args) in referansHatasi.cs:line 3
```

Ne demek? `new string[3]` üç elemanlı bir dizi oluşturur, ama elemanlar `null`'dır; henüz metin nesnesi yoktur. `adlar[1]` `null` iken `ToUpper()` çağrılamaz. Bu mesaj hangi değişkenin `null` olduğunu söylemez; satırı kendin bulman gerekir.

Nasıl düzeltilir? Dizi elemanlarına kullanmadan önce değer ver. Değer `null` olabilecekse `adlar[1]?.ToUpper()` yaz: `null` ise sonuç `null` olur, hata vermez.

`DivideByZeroException: Attempted to divide by zero`

```
int toplam = 240;
int adet = 0;
Console.WriteLine("Ortalama: " + toplam / adet);
```

Çıktı – program yakalanmamış istisna ile sonlanır

```
Unhandled exception. System.DivideByZeroException: Attempted to divide by zero.
   at Program.<Main>$(String[] args) in sifiraBolme.cs:line 3
```

Ne demek? Tam sayı sıfıra bölündü. `double` ile bölmede hata çıkmaz; sonuç sonsuz değeri olur. Bu fark sınavda sık sorulur.

Nasıl düzeltilir? Bölmeden önce böleni kontrol et: `if (adet != 0) Console.WriteLine(toplam / adet);`. "Hiç eleman yok" durumu bu hatanın klasik nedenidir.

`KeyNotFoundException: The given key was not present in the dictionary`

```
var notlar = new Dictionary<string, int> { ["Ali"] = 80 };
Console.WriteLine(notlar["Veli"]);
```

Çıktı – program yakalanmamış istisna ile sonlanır

```
Unhandled exception. System.Collections.Generic.KeyNotFoundException: The given key 'Veli' was not
present in the dictionary.
   at Program.<Main>$(String[] args) in anahtarYok.cs:line 2
```

Ne demek? Sözlükte (`Dictionary`, Bölüm 13) `"Veli"` anahtarı yok. `[]` ile okurken anahtar yoksa C# varsayılan bir değer döndürmez; istisna fırlatır.

Nasıl düzeltilir? `TryGetValue` kullan: `if (notlar.TryGetValue("Veli", out int puan)) ...`. Anahtarın var olup olmadığını önceden bilmek istersen `ContainsKey` de kullanılabilir.

Hata ayıklama

Hata verdiğinde şu sırayla ilerle:

1. **İlk hatayı oku.** Hata listesinin en üstündeki hatayla başla; sonrakiler çoğu zaman bunun sonucudur.
2. **Hata kodunu ara.** `CS0103` gibi kodu aratırsan, Visual Studio'nun dili ne olursa olsun açıklama bulursun.
3. **Hatanın yerine değil nedenine bak.** Derleyici sorunu fark ettiği satırda gösterir; asıl neden birkaç satır yukarıda olabilir. Hatalı satırın üstünü de oku.
4. **Kesme noktası (breakpoint) koy.** Satırın sol kenarına tıkla ya da imleç o satırdayken **F9**'a bas. **F5** ile hata ayıklamayı başlat; program kesme noktasında durur.
5. **Adım adım ilerle. F10** (Step Over) satırı çalıştırıp sonrakine geçer; **F11** (Step Into) çağrılan metodun içine girer. Değişkenin üzerine gelerek değerini görebilirsin.
6. **Düzelt ve yeniden derle.** Düzeltmeden sonra `dotnet build` ile derlemeyi tekrar çalıştır; sahte hataların çoğu bu sırada kaybolur.